Computer Graphics

CSE 4101: Computer Graphics & Image Processing

Course Teacher: Md. Mubtashim Abrar Nihal (Batch 2K21)

Comprehensive Course Slide Deck with Embedded Navigation & Table of Contents

Table of Contents

1 1. Introduction to Computer Graphics p. 3

2 2. Graphics Hardware p. 40

3 3. 2D Transformations p. 68

4 4. 3D Transformations p. 106

5 5. View Transformations p. 180

6 6. Projective Transformations p. 212

7 7. Clipping p. 255

8 8. Introduction to Scan Conversion p. 332

9 9. Line Drawing Algorithms p. 349

10 10. Region Filling Algorithms p. 381

11 11. Local Illumination Models p. 405

12 12. Global Illumination Models p. 487

13 13. Shading Models p. 565

14 14. Hidden Surface Removal p. 618

CSE 4207: Computer Graphics

Introduction

Dr. Sk. Md. Masudul Ahsan

Professor

Dept. of CSE, KUET

Computer Graphics

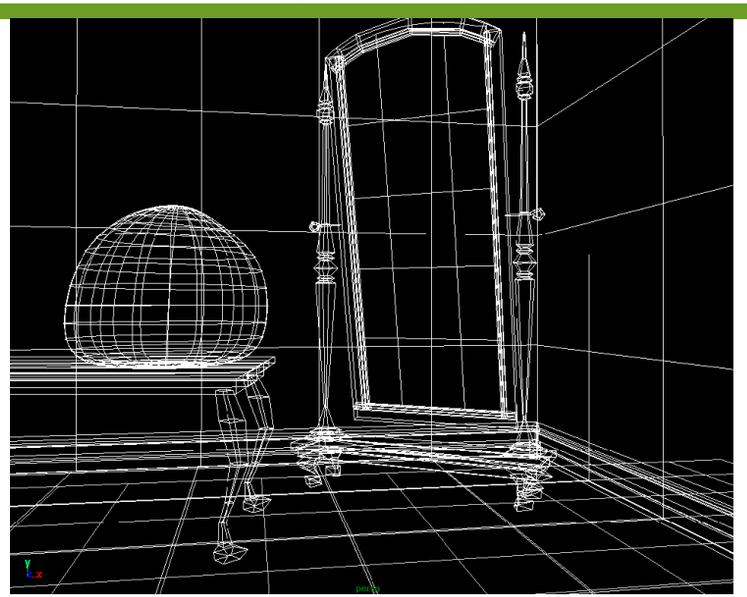
- Computer Graphics commonly mean the creation, storage and manipulation of geometric objects (modeling) and their images (rendering)

- ~ is concerned with the creation and manipulation of **synthetic** images and animations using the aid of computer.

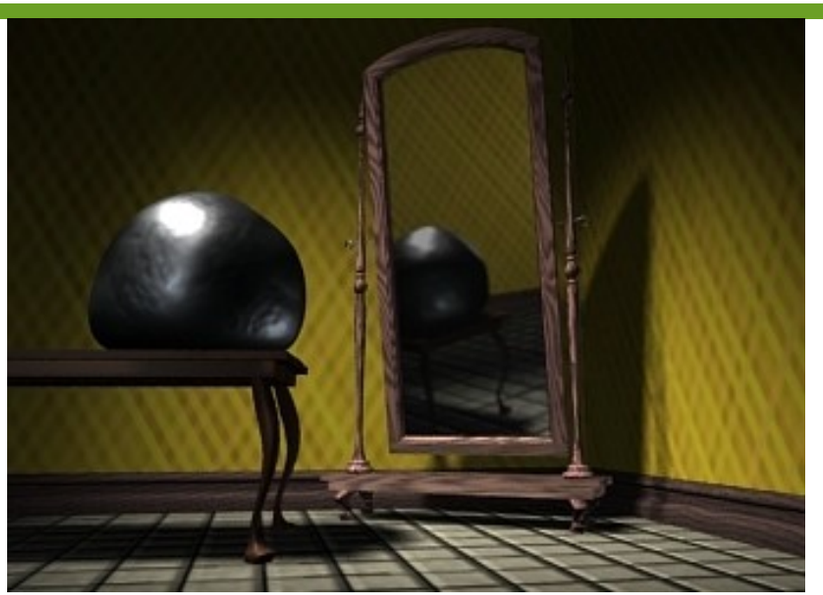
Computer graphics are pictures and movies created using computers – usually referring to image data created by a computer specifically with help from specialized graphical hardware and software.

- wikipedia

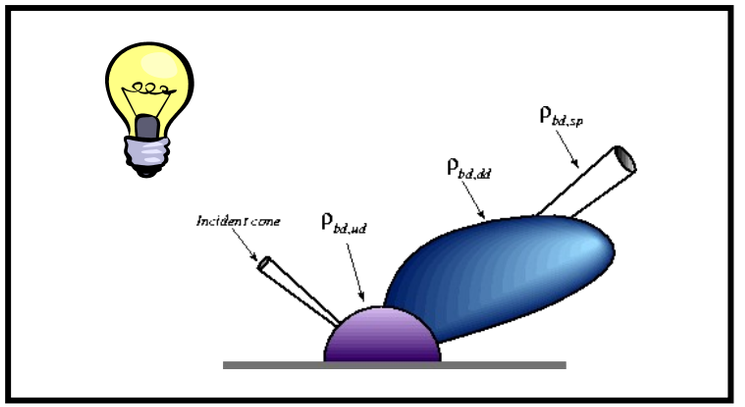
Computer graphics?



3D geometry



projection



physics

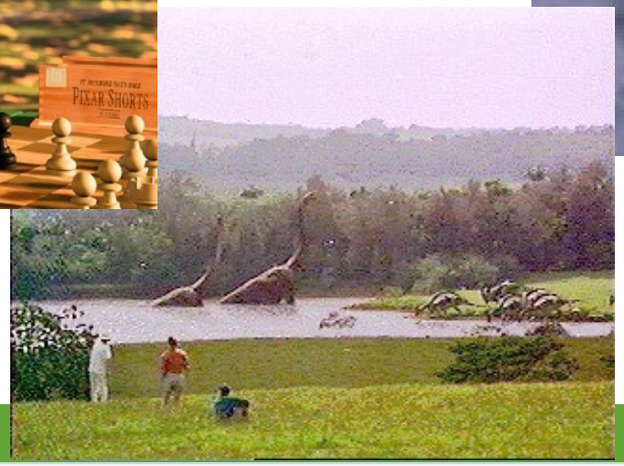
Simulation

Why Study Computer Graphics?

- Graphics is cool
  - I like to see what I'm doing
  - I like to show people what I'm doing
- Graphics is interesting
  - Involves *simulation*, AI, *algorithms*, *architecture*…
- I'll never get an Oscar for my acting
  - But maybe I'll get one for my CG special effects
- Graphics is fun
- Very interdisciplinary
  - Within CS: systems, compilers, languages, computer architecture, algorithms, numerical techniques
  - Math, physics, art, perception, architecture, manufacturing
- Helps you understand why the world looks the way it does
  - You can "see" the result

What drives computer graphics?

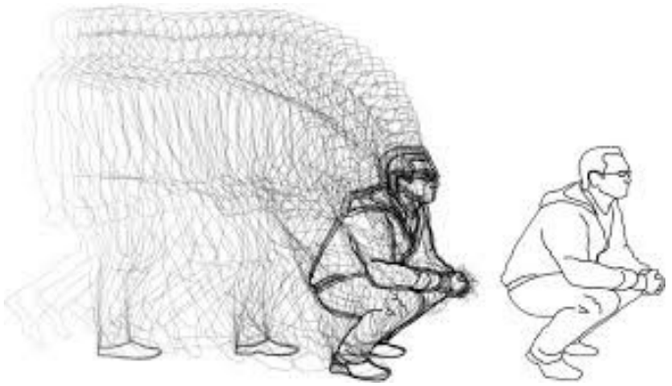
- Movie Industry
  - Movie production, Animations,
    Special Effect (super reality)
  - Big budgets and tight schedules
  - Defines our expectations



What drives computer graphics?

- Movie production (special effects & full-length feature films)
  - Matte (traveling Matte – blue/green screen)
  - Rotoscoping to computer animation

https://en.wikipedia.org/wiki/Chroma_key

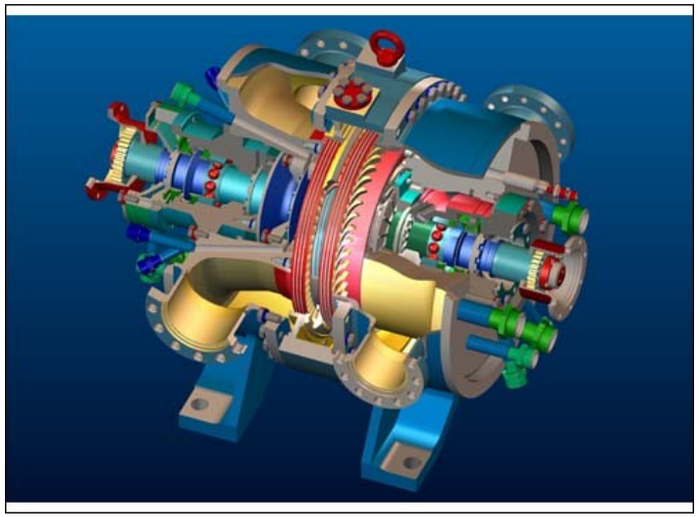


What drives computer graphics?

- Game Industry
  - The newest driving force in CG
    - Why? Volume and Profit
    - This is why we have commodity GPUs
  - Focus on interactivity
  - Cost effective solutions

What drives computer graphics?

- Computer Aided Design
  - Structural design – design of buildings, bridges, car, aircraft, many more
  - Electrical circuit, mechanical design
  - Integration of computing and display resources
  - Reduced design cycles == faster systems, sooner



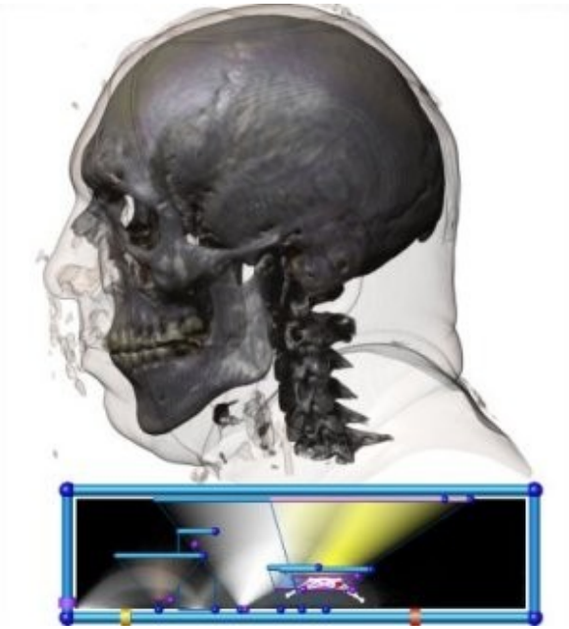
Important Application Areas

- Human-computer Interaction (HCI)
  - Augmented Reality

http://augmentedrealityoverview.blogspot.com/2011/11/edibear.html

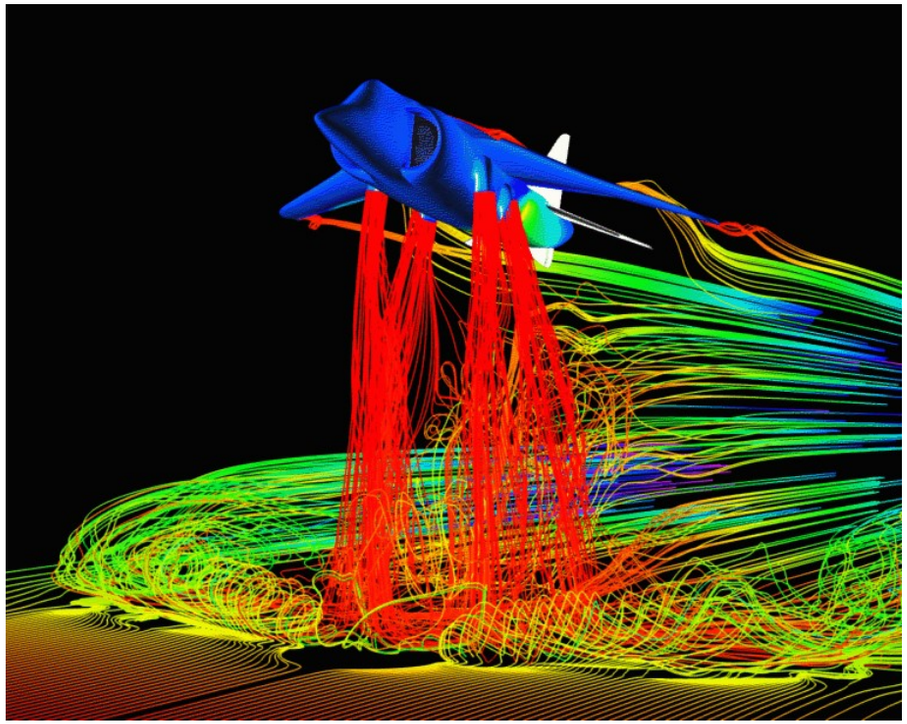
Graphics Applications

- Medical Visualization
  - Tools for visualizing diagnosis
    - No cheating or tricks allowed
  - Drive issues of precision and correctness
  - Focus on presentation and interpretation of data
  - Construction of models from acquired data

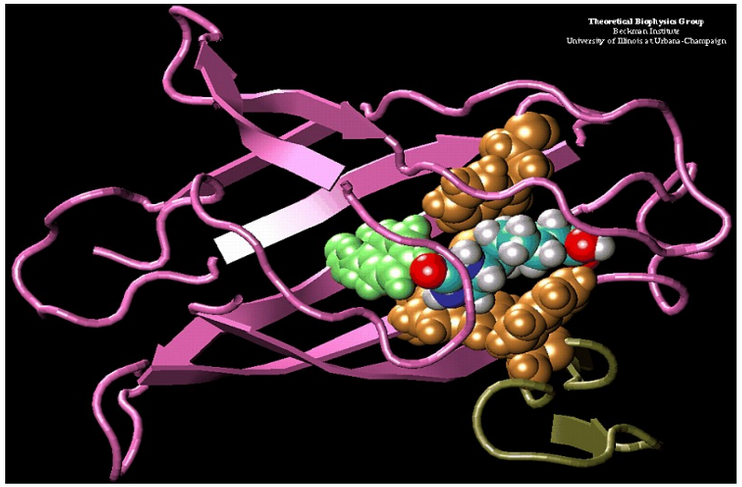


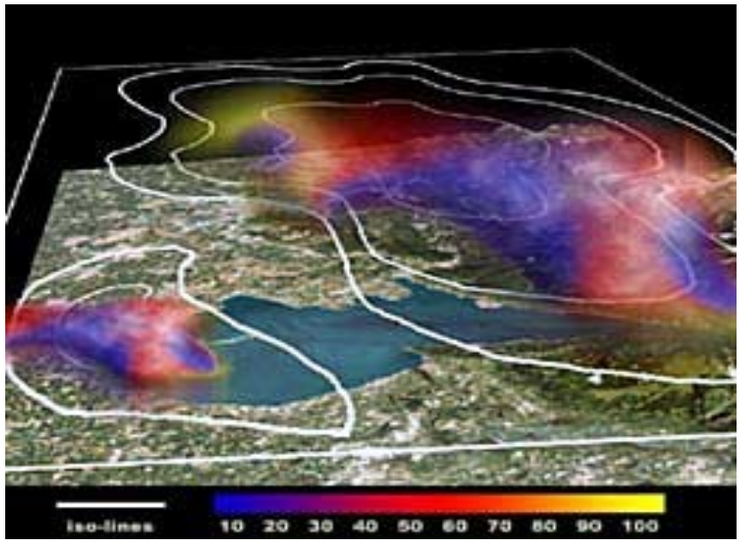
Graphics Applications

- Scientific Data Visualization
  - CG is superb tool for presenting complex scientific data in a way that can be easily grasped



Airflow around a Harrier Jet *(NASA*





Computer Graphics - Applications

- Education and Training
  - Computer generated model can aid easy understanding (education)
  - Various simulator can be used for practice / training session for aircraft pilot, heavy equipment operators.

- Computer Art, publishing
  - Fine and commercial art
  - Slide, Book, Magazine design

What's Our Scope?

- Not a Tutorial on Commercial Software
  - 3DMax, Maya, Photoshop, etc.
- Not about Graphics Business
  - 3D online-game, E-commerce, etc.
- Graphics = Algorithm for Visual Simulation
  - Modeling, Rendering, Animation

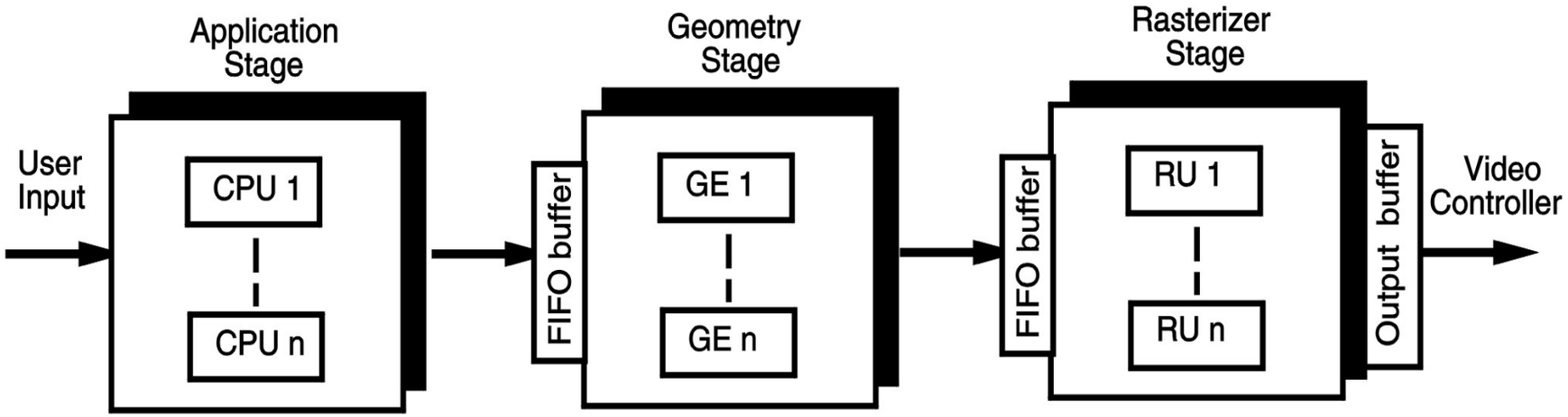
Computer Graphics - Main Themes

- Imaging
  - Representing 2D images
- Modeling
  - Representing 3D objects
    - Surface Modeling
    - Solid Modeling
    - Procedure Modeling
    - Simplification
- Rendering
  - Constructing 2D images from 3D models
    - Polygonal Rendering
    - Global Rendering
    - Texture Mapping
- Animation
  - Simulating changes over time
    - Keyframe Animation
    - Inverse Kinematics
    - Dynamics
    - Motion Capture

Graphics / Rendering Pipeline

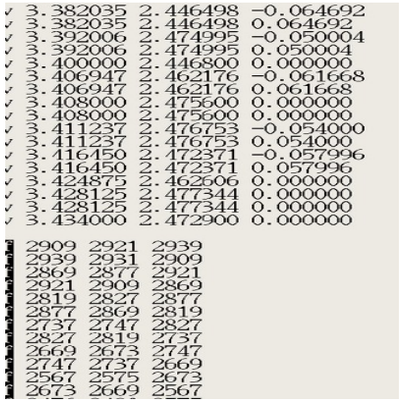
There are three stages

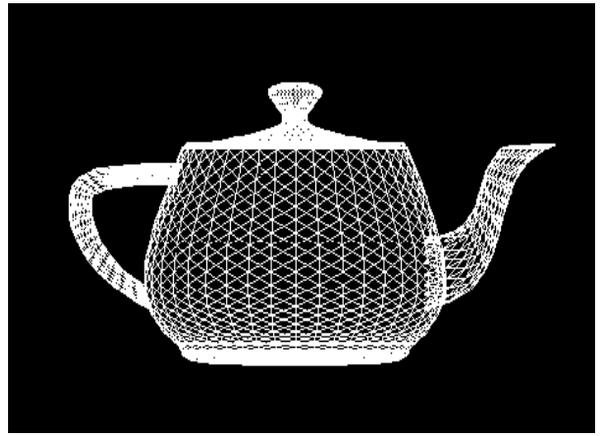
- Application Stage
- Geometry Stage
- Rasterization Stage



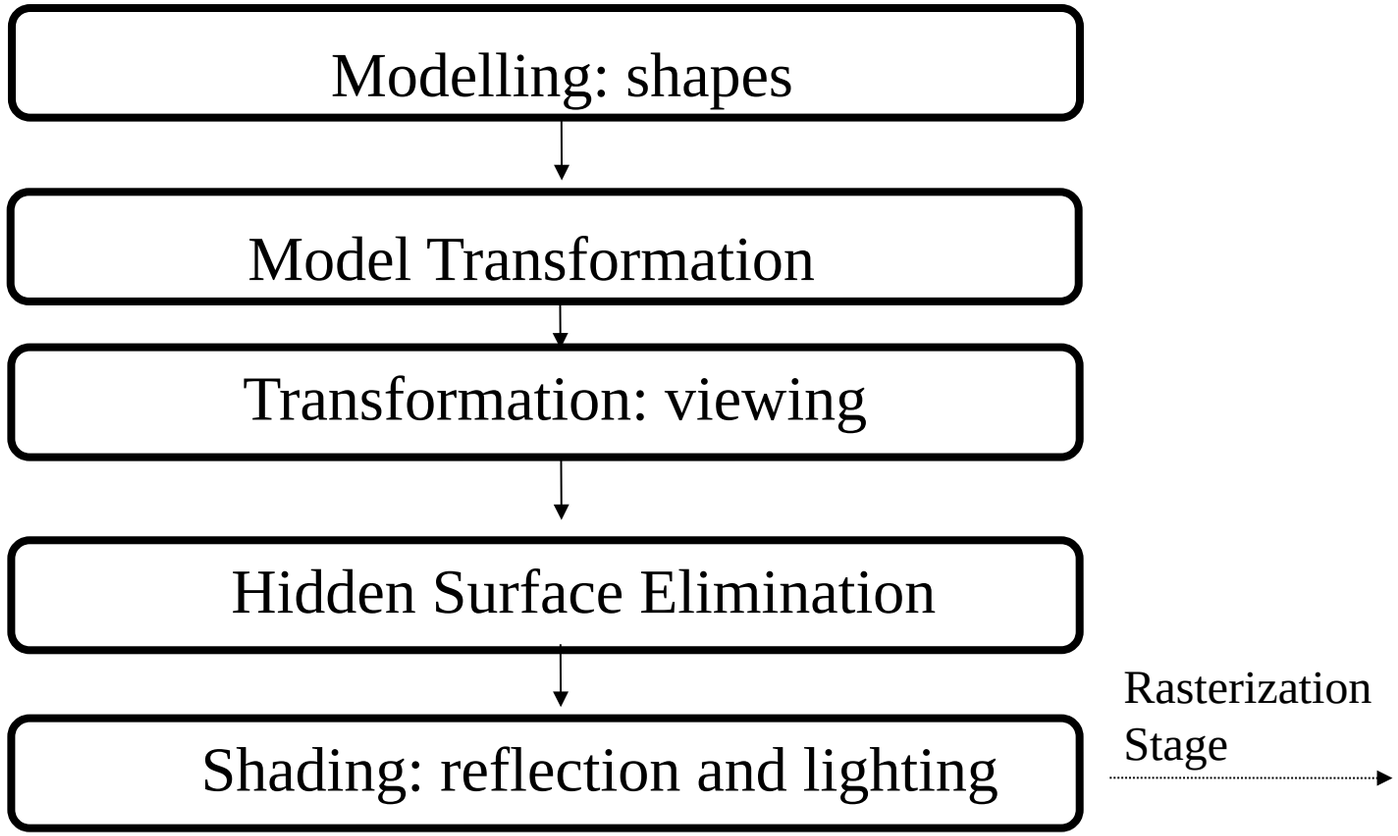
Application stage

- Entirely done in software by the CPU
- Read Data
  - the world geometry database,
  - User's input by mice, trackballs, trackers, or sensing gloves
- In response to the user's input, the application stage change the view or scene

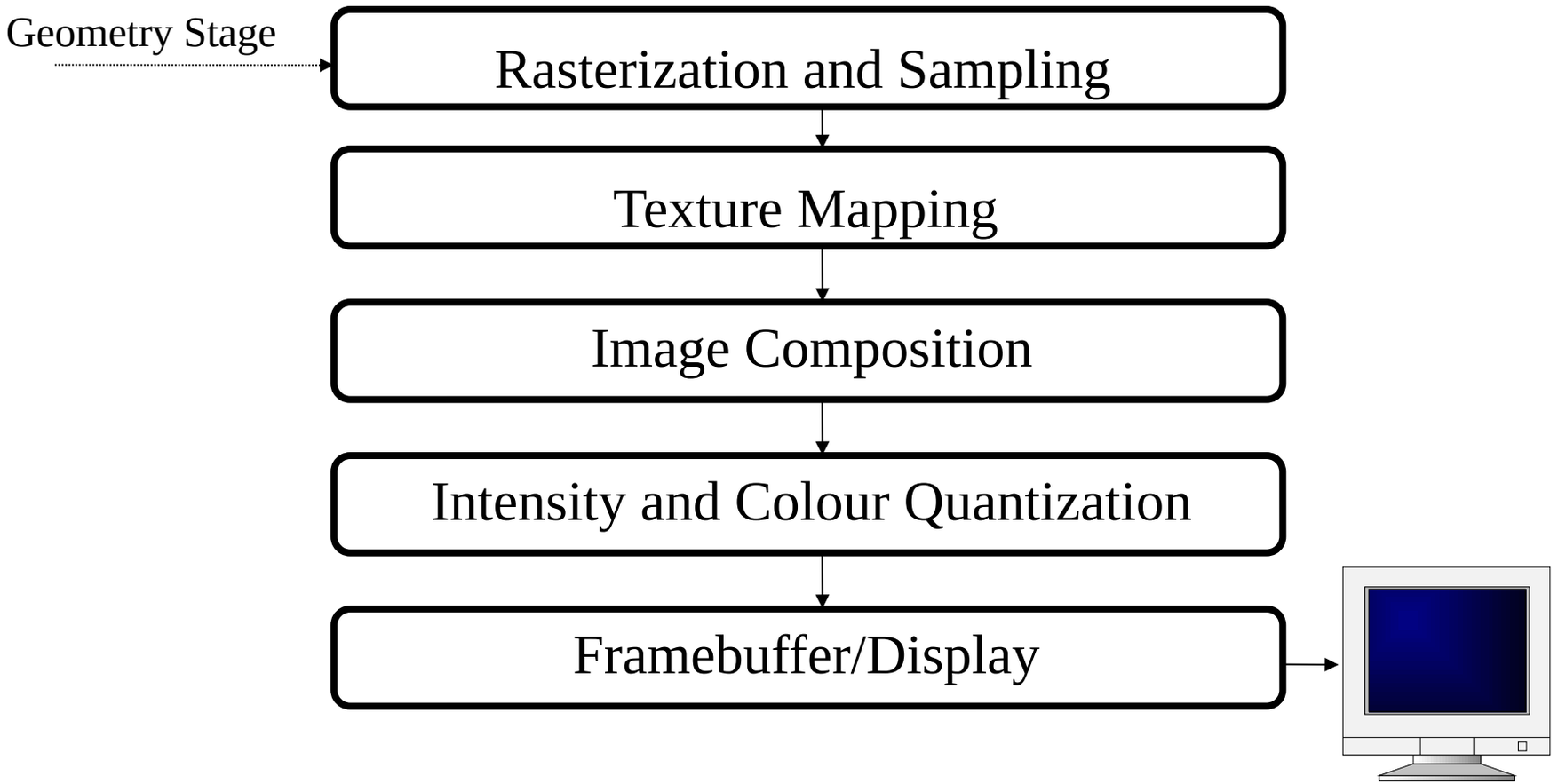




Geometry Stage

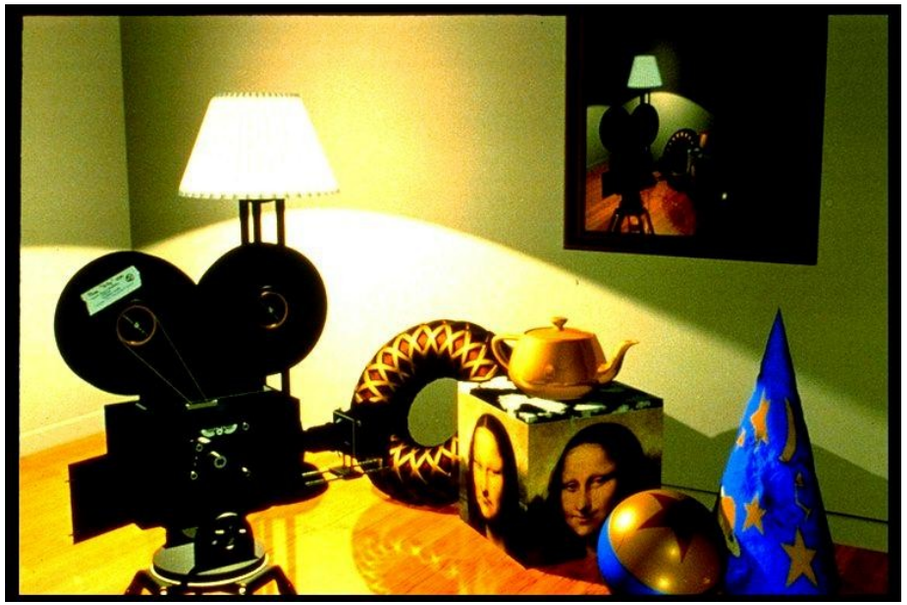


Rasterization Stage



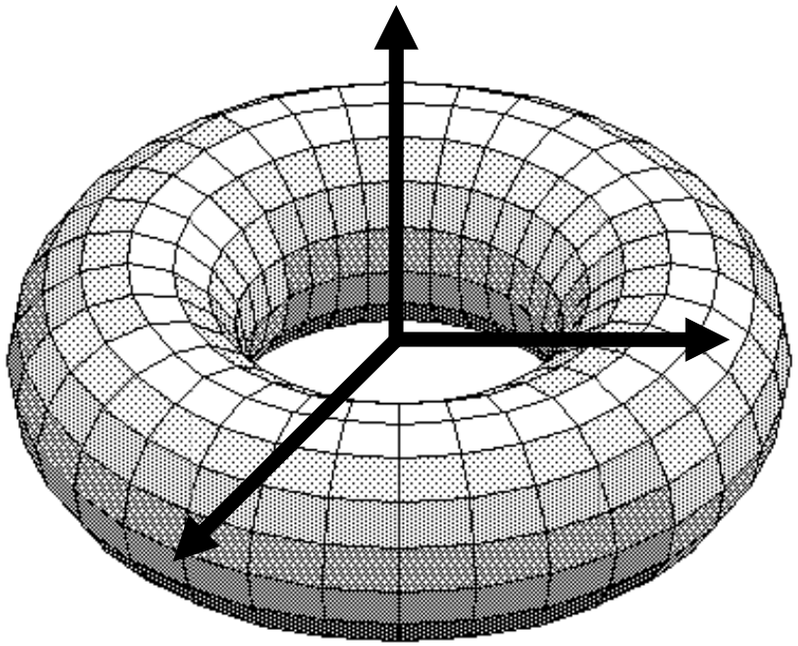
An example thro’ the pipeline…

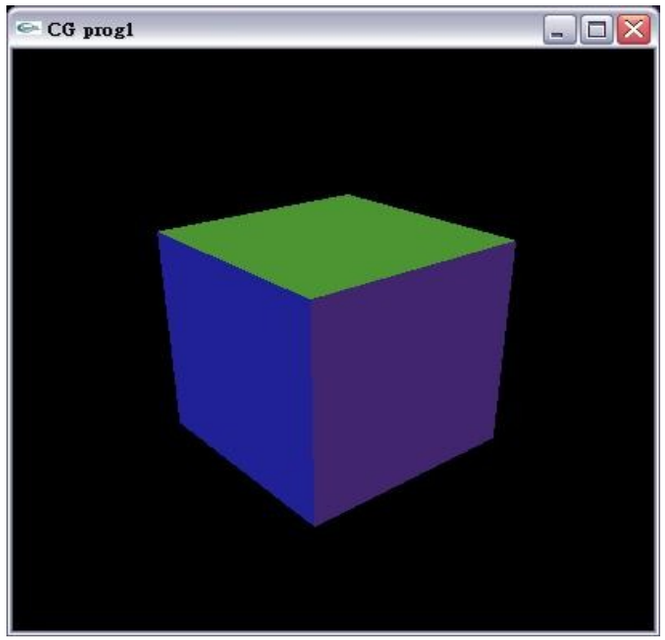
The scene we are trying to represent:



Preparing Shape Models

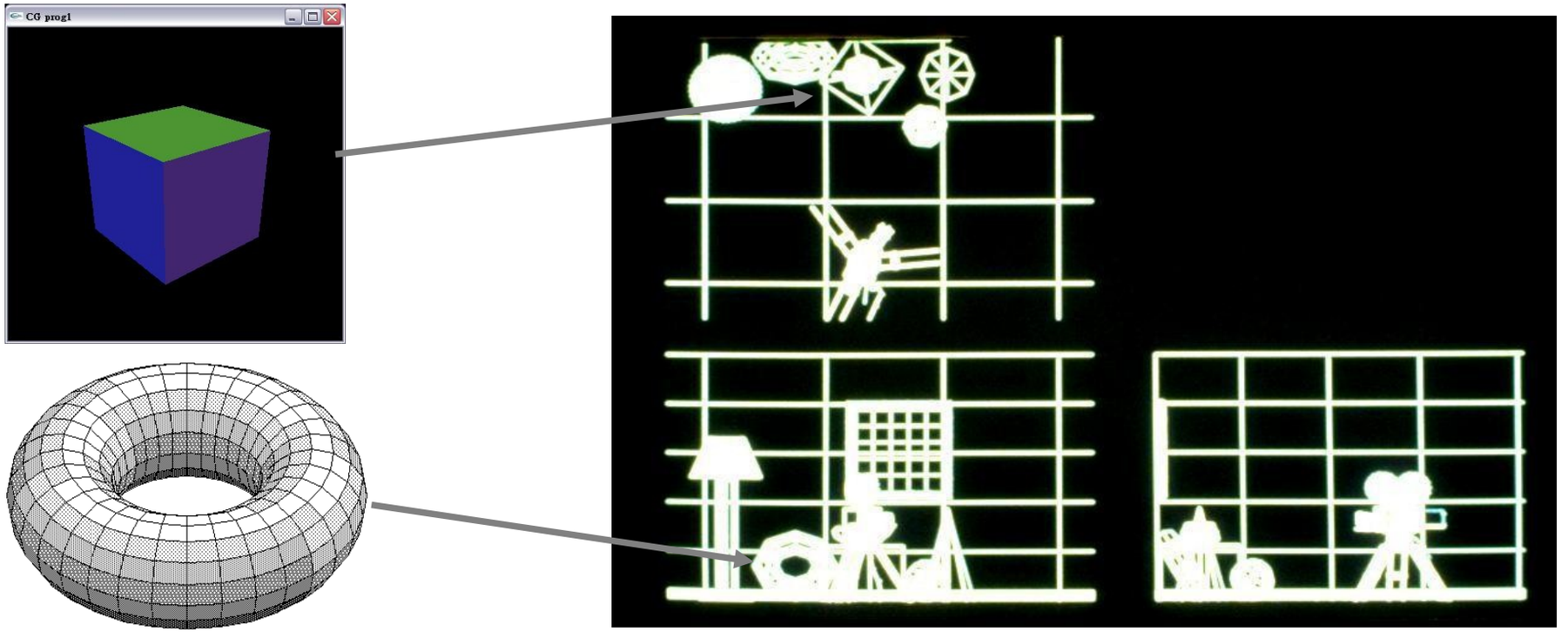
- Designed by polygons, parametric curves/surfaces, implicit surfaces and etc.
- Defined in its own coordinate system





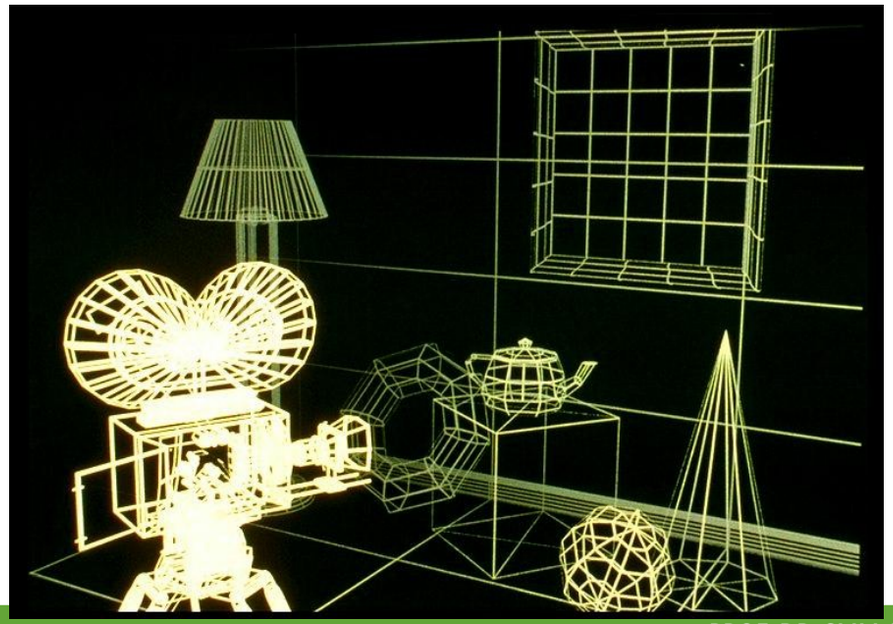
Model Transformation

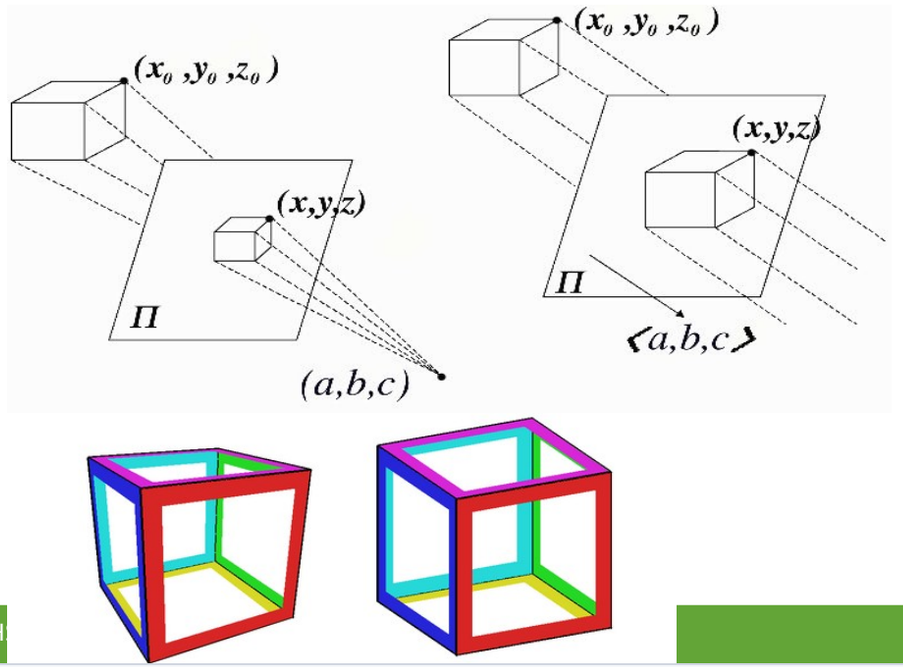
- ✓ Objects put into the scene by applying translation, scaling and rotation
- ✓ The location of all the vertices are updated by this transformation



Perspective Projection

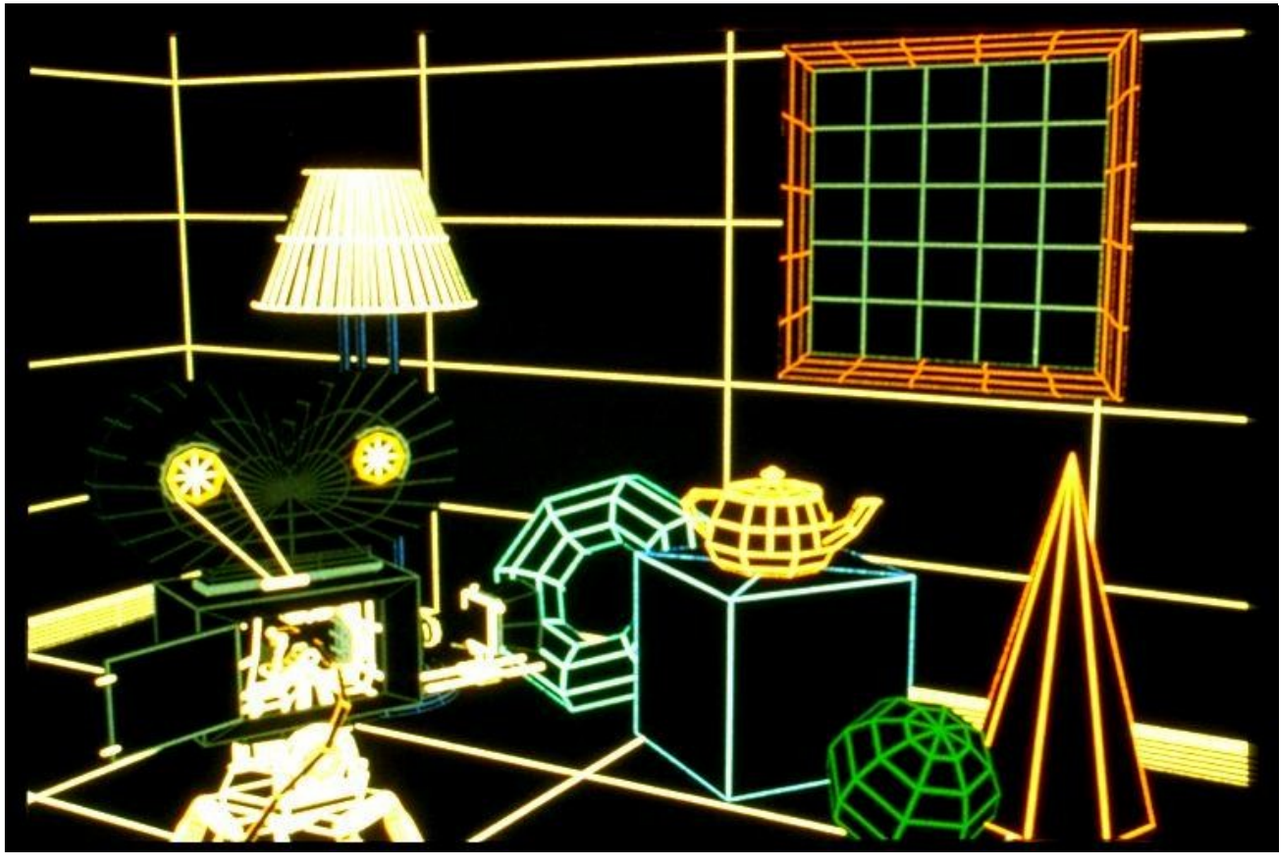
- We want to create a picture of the scene viewed from the camera
- We apply a perspective transformation to convert the 3D coordinates to 2D coordinates of the screen
- Objects far away appear smaller, closer objects appear bigger





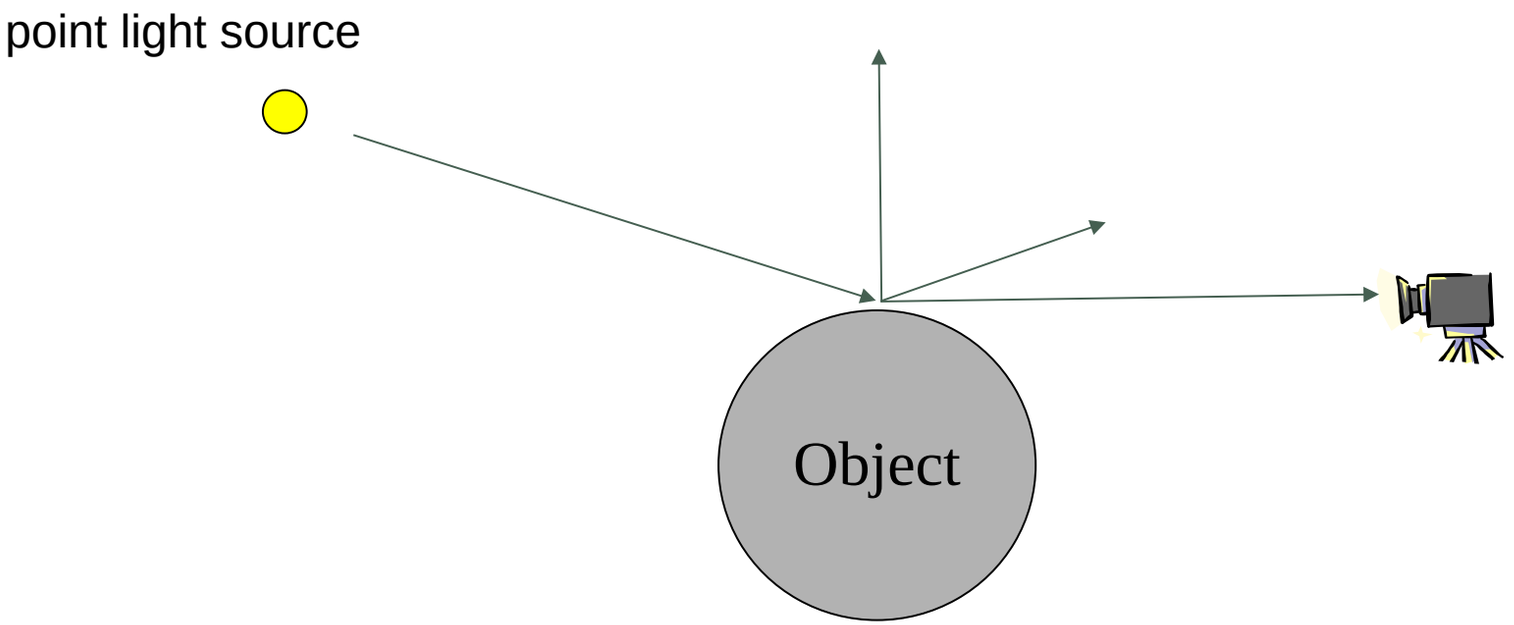
Hidden Surface Removal

- Objects occluded by other objects must not be drawn



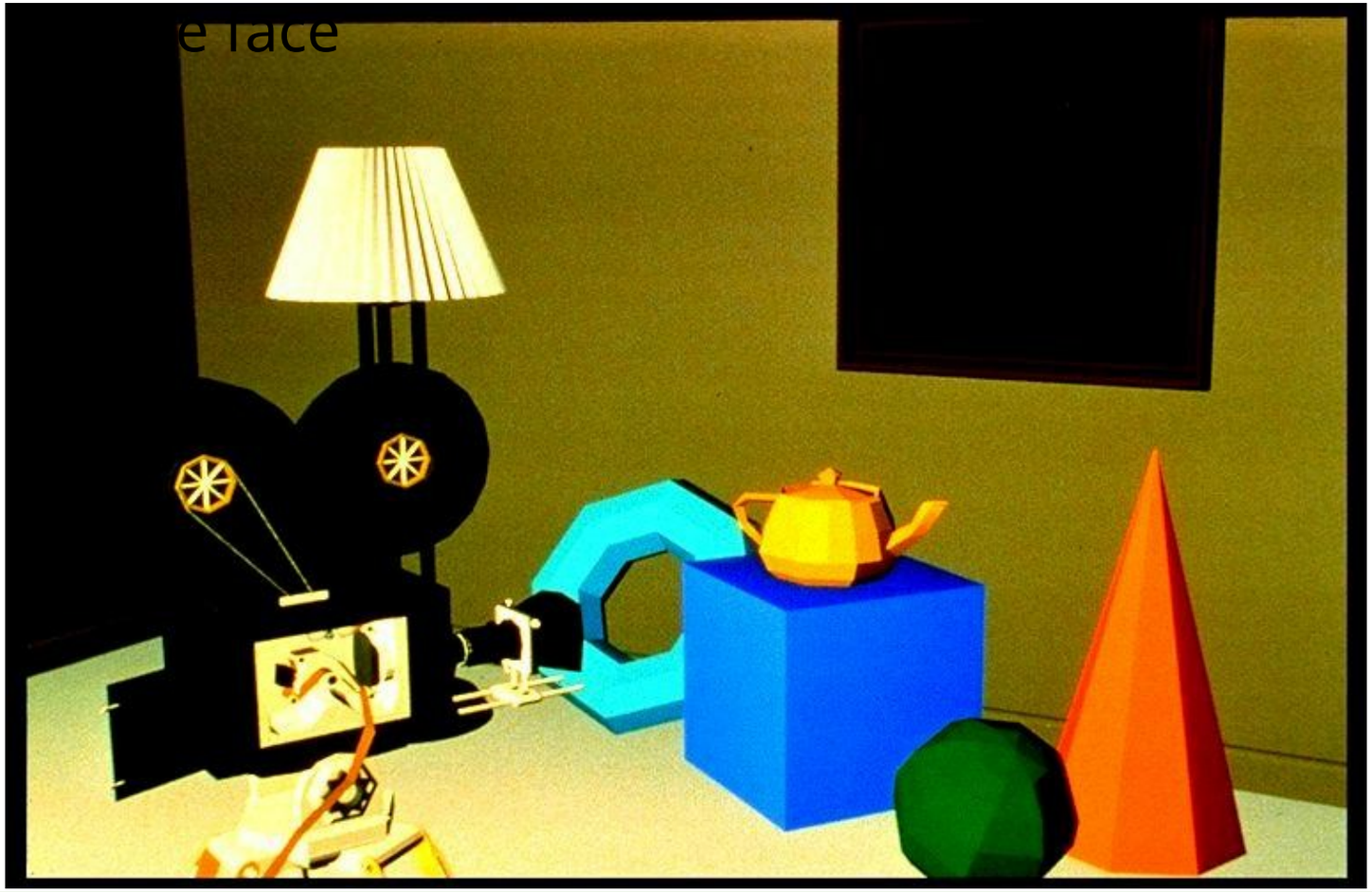
Shading

- Now we need to decide the colour of each pixels taking into account
  - the object's colour, lighting condition and the camera position



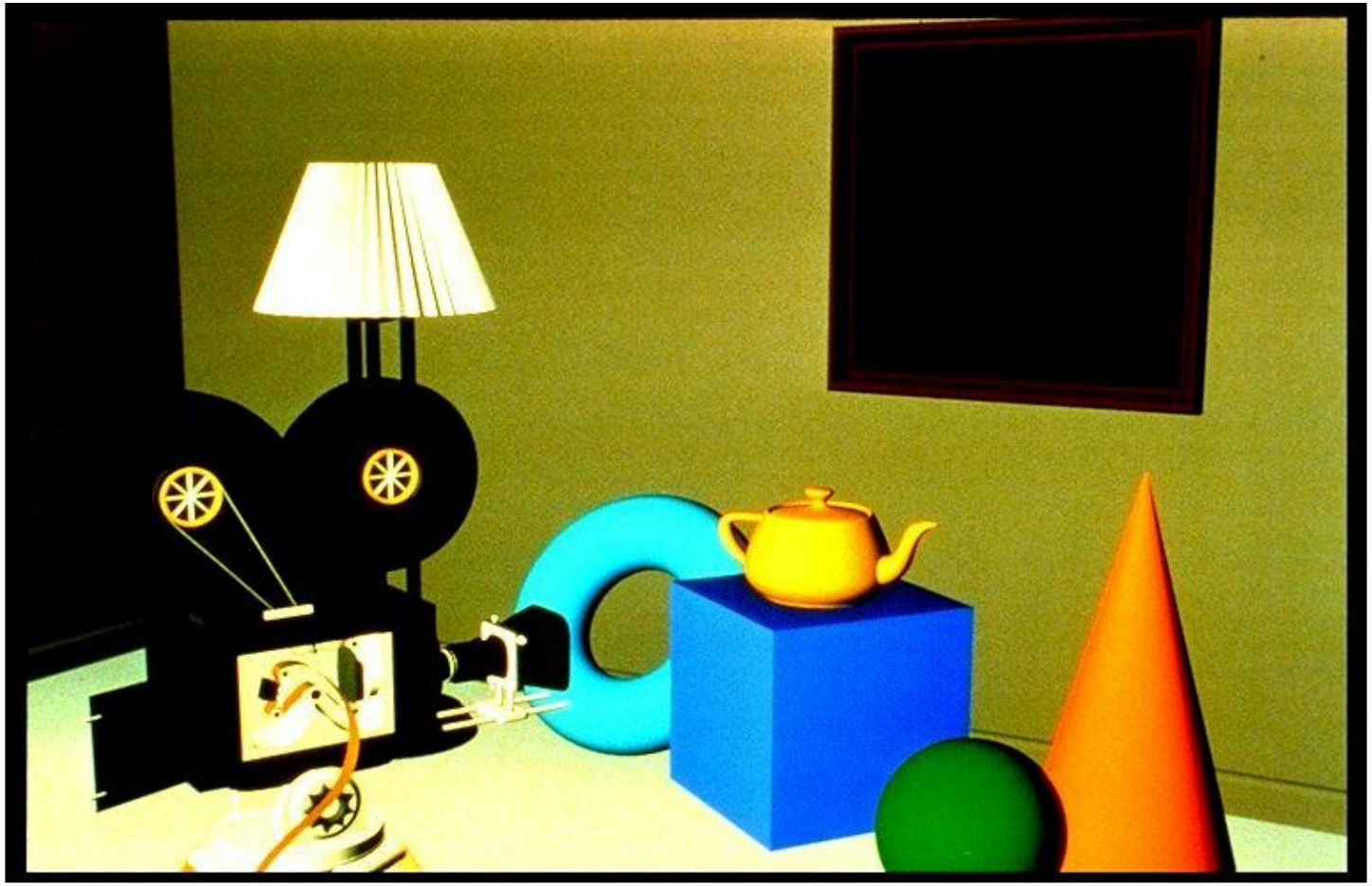
Shading – Flat Shading

- Objects coloured based on its own colour and the lighting condition
- One colour e face



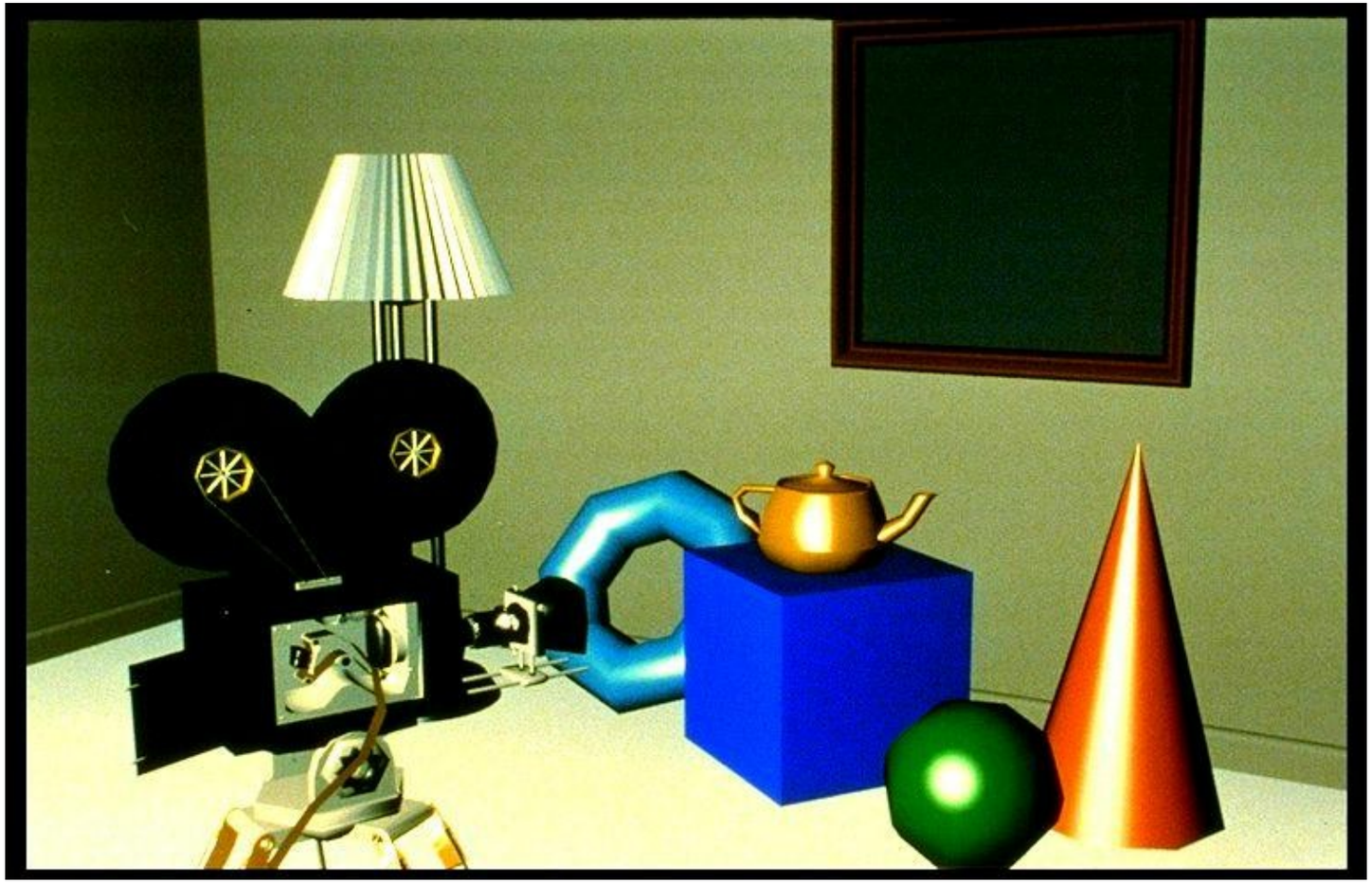
Gouraud shading, no specular highlights

- Lighting calculation per vertex

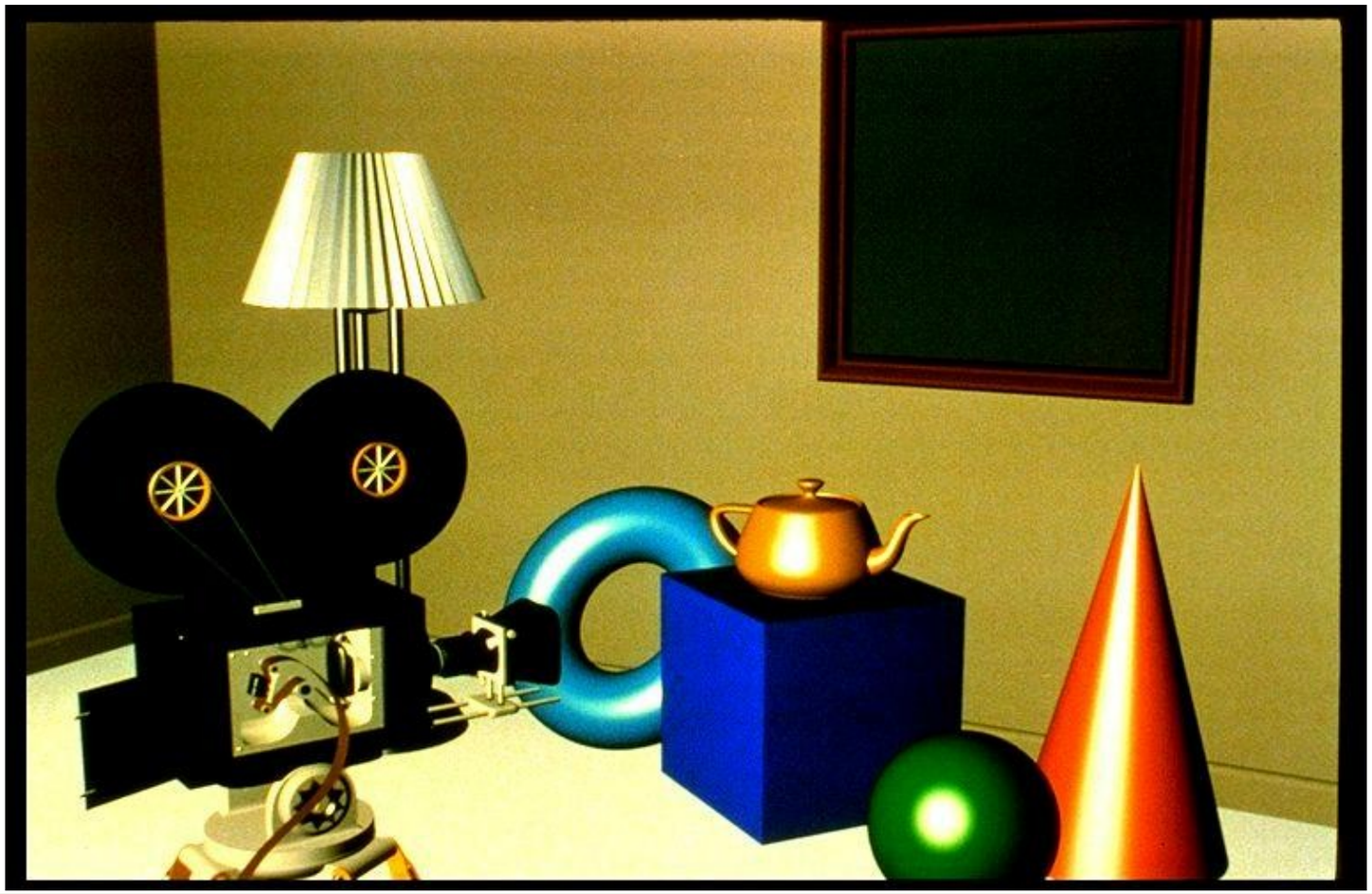


Specular highlights added

- Light perfectly reflected in a mirror-like way



Phong shading

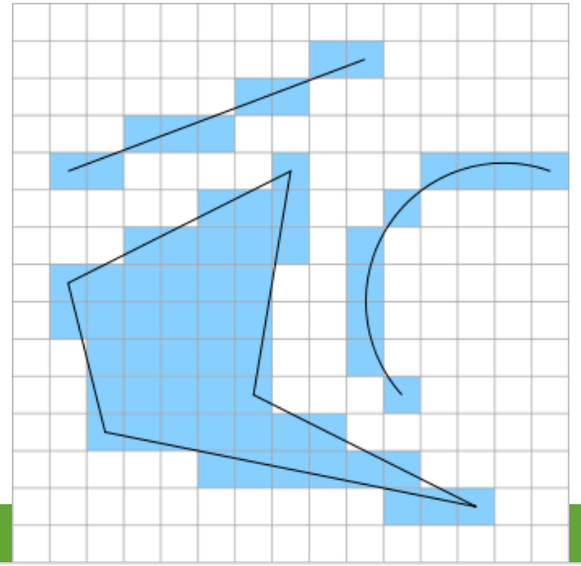


Rasterization

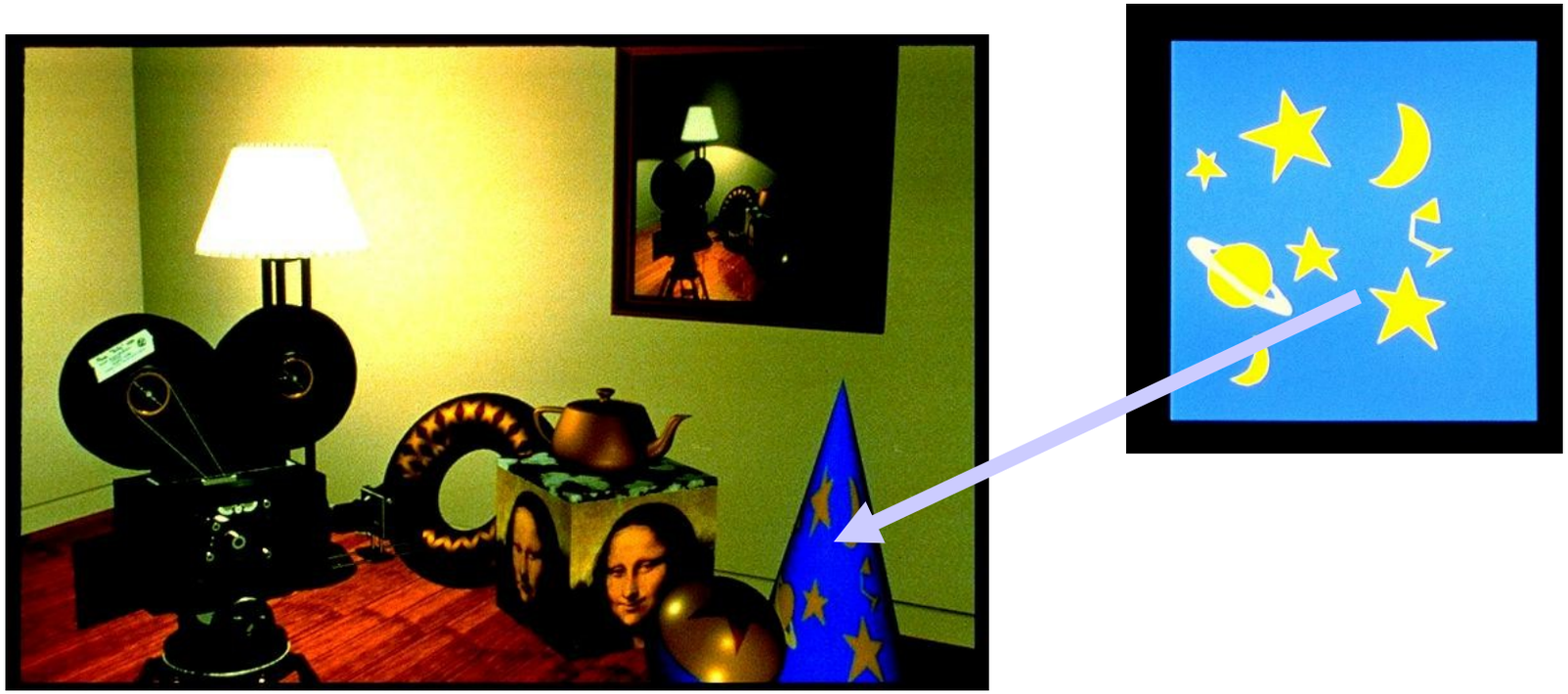
Converts the vertex information output by the geometry pipeline into pixel information needed by the video display

Aliasing: distortion artifacts produced when representing a high-resolution signal at a lower resolution.

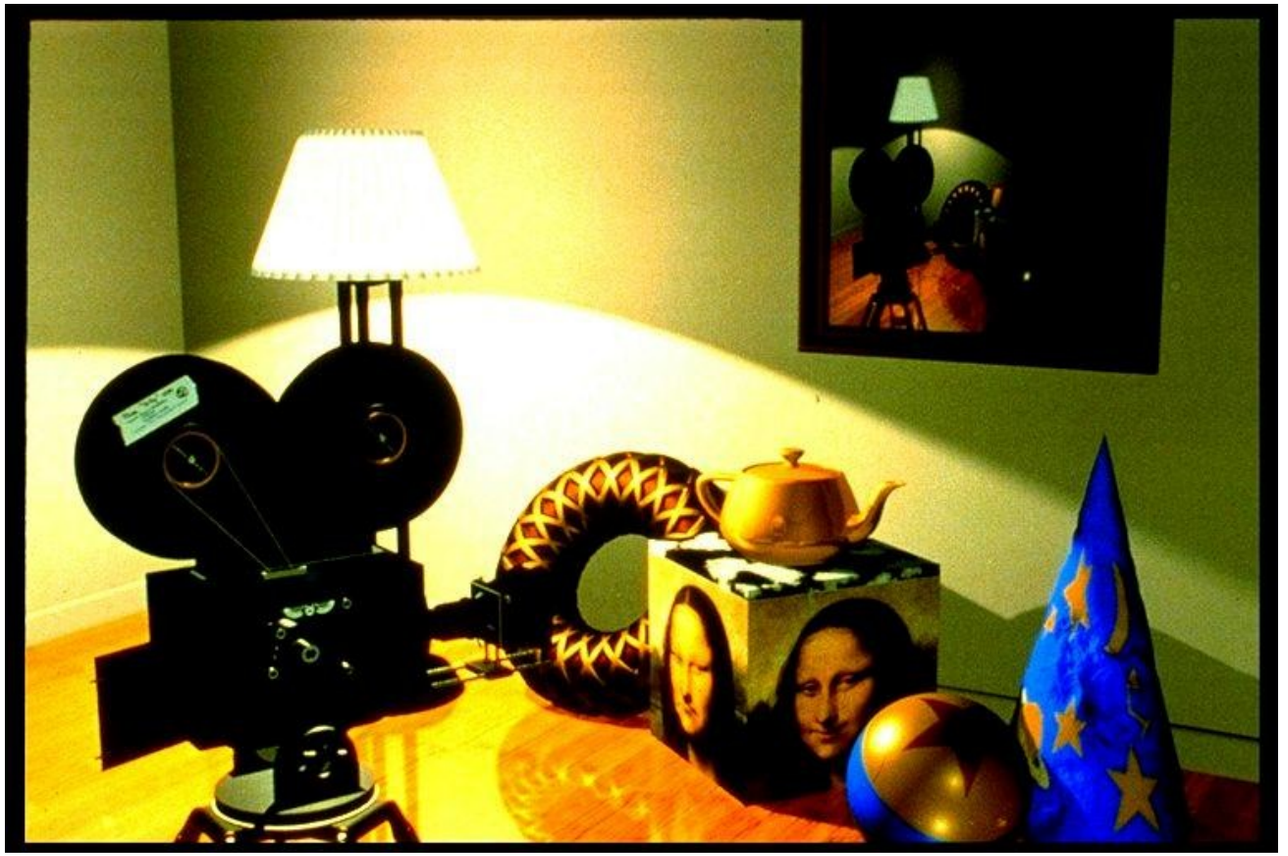
Anti-aliasing : technique to remove aliasing



Texture mapping

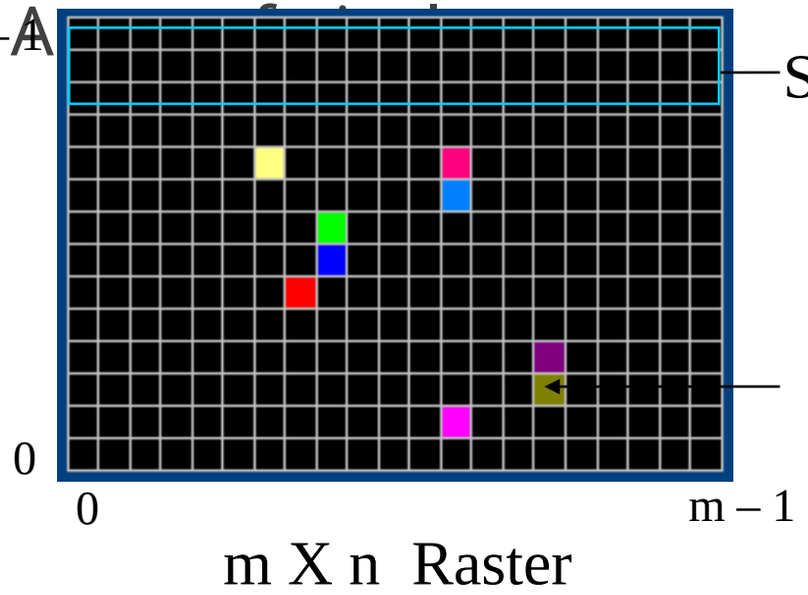


Reflections, shadows & Bump mapping



Graphics Definitions

- Most common display used for CG are Raster displays
  - CRT, Flat panel
- Display surface : On which image is presented
- RASTER : A rectangular array of points or dots on the surface
- PIXEL : One dot or picture element of the raster
- SCAN LINE : A



We don't have any pixel like (1.2, 5.8). In raster device co-ordinates can take integer values only.

Graphics Definitions

Vertex
- point in 3D

Edge
- line in 3D connecting two vertices

Polygon/Face/Facet
- arbitrary shape formed by connected vertices
- fundamental unit of 3D computer graphics

Mesh
- set of connected polygons forming a surface (or object)
- :

Graphics Definitions

- The definition of image is stored in a 2 dimensional array in memory called Frame Buffer
- The display hardware produces the image line by line (Raster Lines)
- The high level process of taking a graphics model and producing an image is called Rendering
- The lower level process of drawing individual pixels or filling polygonal regions of the image by modifying the frame buffer is known as Scan Conversion or Rasterization

Frame Buffer

- a video output device that drives a video display from a memory containing the color for every pixel
- A frame buffer may be thought of as computer memory organized as a two-dimensional array with each (x,y) addressable location corresponding to one pixel.
- Bit Planes or Bit Depth is the number of bits corresponding to each pixel.
- A typical frame buffer resolution might be
  - 640 x 480 x 8 (x-resolution x y-resolution x color bit depth)
  - 800 x 600 x 16
  - 1024 x 768 x 24

Conclusion

- Computer graphics: generating 2D synthetic images of a 3D world represented in a computer.
- Main tasks:
  - *modeling*: (shape) creating and representing the geometry of objects in the 3D world
  - *rendering*: (light, perspective) generating 2D images of the objects

Recommended Texts

1. **Computer Graphics**
   - Donald Hearn, M Pauline Baker
2. **Computer Graphics**
   - R. Plastock, Zhigang Xiang
   - (Schaum's Outline Series) McGraw Hill,
3. **Computer Graphics: Principles & Practice**
   - J. D. Foley, A. van Dam, S. K. Feiner, J. F. Hughes
4. **Fundamentals of Computer Graphics**
   - Steve Marschner, Peter Shirley
5. **Computer Graphics using OpenGL**
   - F S Hill

- Mathematics for Computer Graphics
  - John Vince
- Mathematics for 3D Game Programming and Computer Graphics
  - Eric Lengyel
- OpenGL Programming Guide
  - M. Woo, J. Neider, T. Davis, D. Shreiner

Graphics Hardware

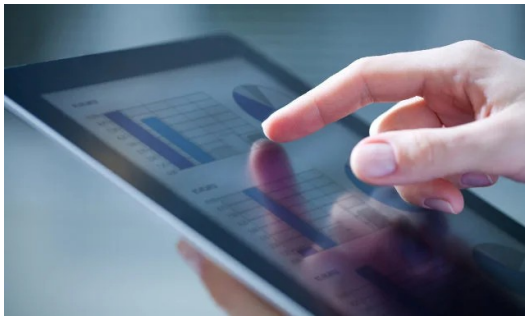
Graphics System

- Typical graphical system consists of host computer with support of fast processor, large memory, frame buffer and
  - Hardwares
    - Input Devices
    - Output Devices
  - Softwares

Input Devices

- Most systems have a keyboard and one or more additional devices specially designed for interactive input. This includes:
  - Touch Panels
  - Light Pens
  - Graphics Tablets
  - Micro-Phone
  - Film Recorder
  - Mouse
  - Joysticks
  - Data Gloves

Input Devices



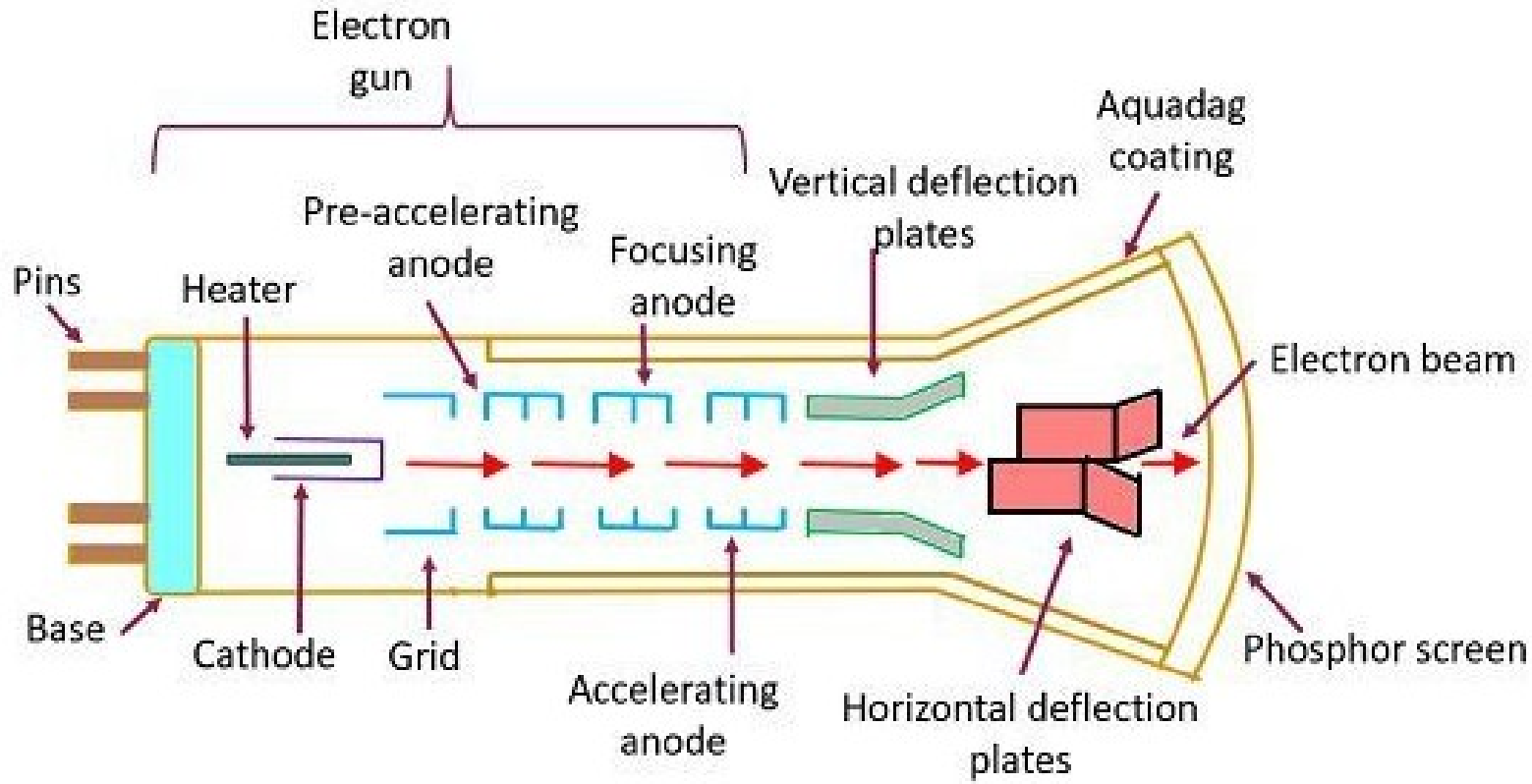
Output Devices

- Computer Graphics can use many different output devices or display devices:
  - Cathode Ray Tube (CRT)
  - Vector/Random Scan Display
  - Raster Scan Display
  - Colored Monitors
  - LED, LCD, TFT Screens

The Cathode Ray Tubes (CRTs)

- Most common display device today
- Characterized by:
  - Evacuated glass bottle
  - Extremely high voltage
  - Heating element (filament)
  - Electrons pulled towards anode focusing cylinder
  - Vertical and horizontal deflection plates
  - Beam that strikes phosphor coating on front of tube

The Cathode Ray Tubes (CRTs)



How CRT works

- It converts electrical signal (voltage, current) to visual signal.
- The electron gun contains a filament that, when heated, emits a stream of electrons.
- Electrons are focused with an electromagnet into a sharp beam and directed to a specific point of the face of the picture tube
- The front surface of the picture tube is coated with small phospher dots
- When the beam hits a phospher dot it glows with a brightness proportional to the strength of the beam and how long it is hit

How CRT works(cont.)

- In a CRT the focusing system acts like a light lens with a focal length such that the center of focus is the screen.
- The horizontal and vertical deflectors allow the electron beam to be focused on any spot on the screen.
- The screen is coated with a special organic compound called a phosphor.
- For color systems there are groups of three different phosphors, one to produce red shades, one for green shades, and one for blue shades.

How CRT works(cont.)

- Electrons hit the screen phosphor molecules and cause a ground state to singlet excited state transition.
- Most of the phosphors relax back to the ground state by emitting a photon of light which is called fluorescence.
  - This happens very rapidly so that all of the molecules which fluoresce do so in under a millisecond.
- These phosphors then emit light, called phosphorescence, that decays slower but still rapidly (in about 15-20 milliseconds)
  - So, there is the need to refresh the screen by

Phosphor

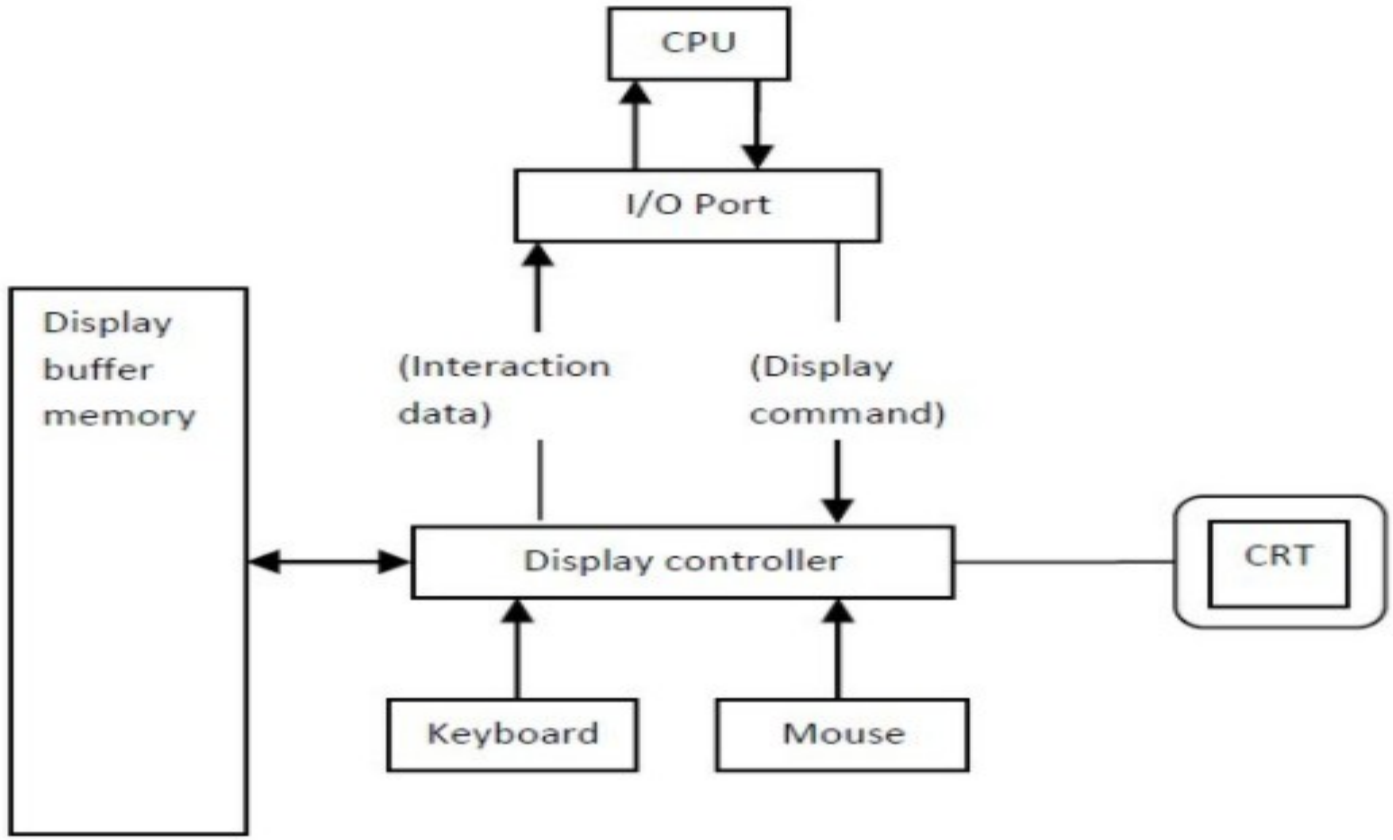
Phosphors – terms…

- Flourescence: Light emitted while the phosphor is being struck by electrons
- Phosphorescence: Light emitted once the electron beam is removed
- Persistence: The time from the removal of the excitation to the moment when phospherescence has decayed to 10% of the initial light output
  - High persistence phosphors allow for a lower refresh rate, avoids flickers
  - Low persistence phosphors require a high refresh rate to prevent flicker.

CRTs - Display Technologies

- Two main types:
  - Vector/Random Display
  - Raster Display

Vector Displays



Vector Displays

- The architecture of vector display consists of display controller, CPU, display buffer memory and CRT.
- Display controller is connected as an I/O peripheral to the CPU.
- Display buffer stores computer produced display list or display program.
- The Program contains point & line plotting commands with end point co-ordinates as well as character plotting commands.
- Display controller interprets command and sends digital and point co-ordinates to a vector generator.

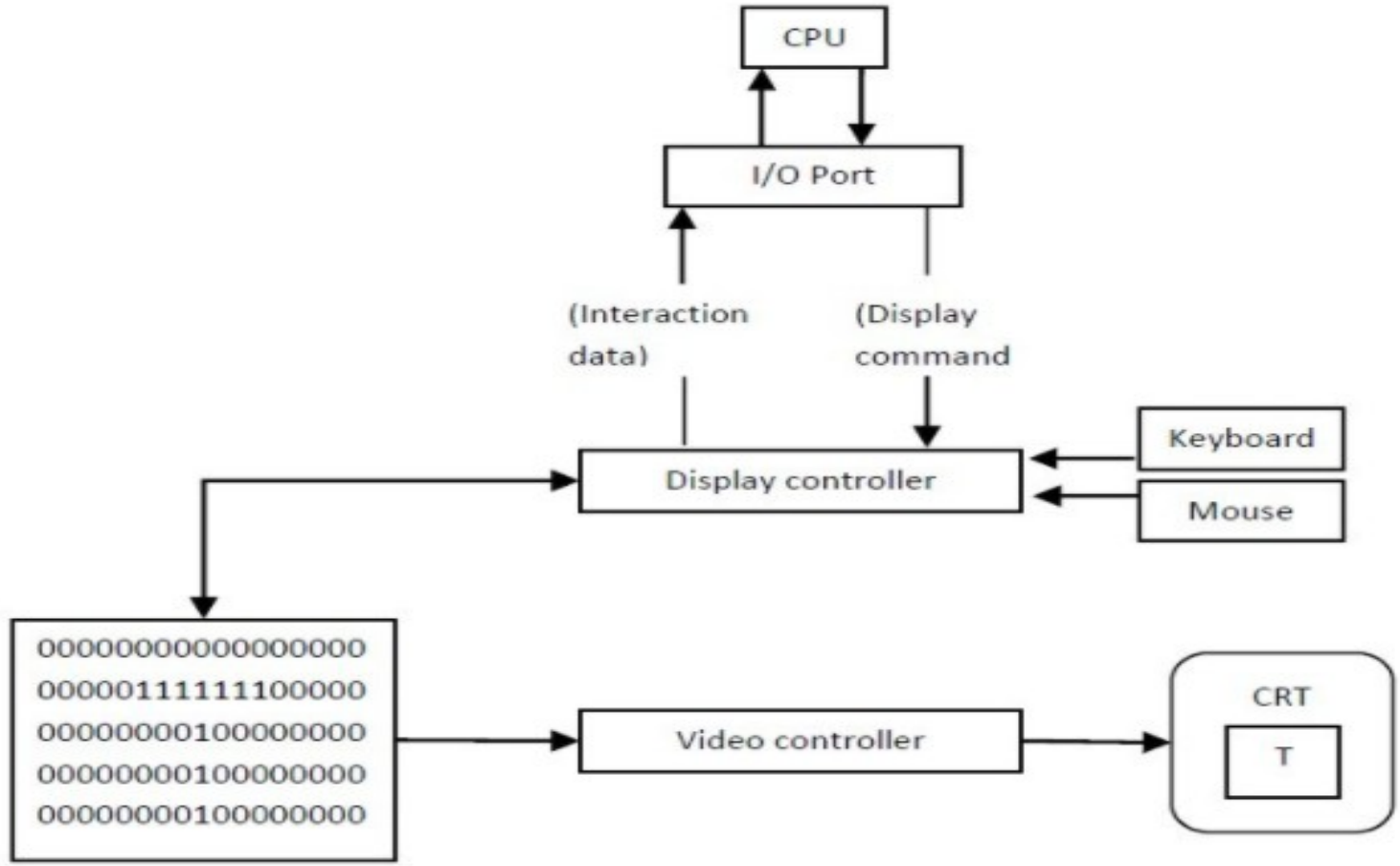
Vector Displays

- Vector generator then converts the digital co-ordinate value to analog voltages for beam deflection circuits that displace an electron beam which points on the CRT's screen.
- As beam is deflected from end point to end point hence this techniques is also called random scan.
- We know as beam strikes phosphors coated screen it emits light but that light decays after few milliseconds and therefore it is necessary to repeat through the display list to refresh the screen at least 30 times per second to avoid flicker.
- As display buffer is used to store display list and used to refreshing, it is also called refresh buffer.

Vector Displays

- If we want line between point p1 & p2 then we directly drive the beam deflection circuitry which focus beam directly from point p1 to p2
- If we do not want to display line from p1 to p2 and just move then we can blank the beam as we move it.
- To move the beam across the CRT, the information about both magnitude and direction is required. This information is generated with the help of vector graphics generator.

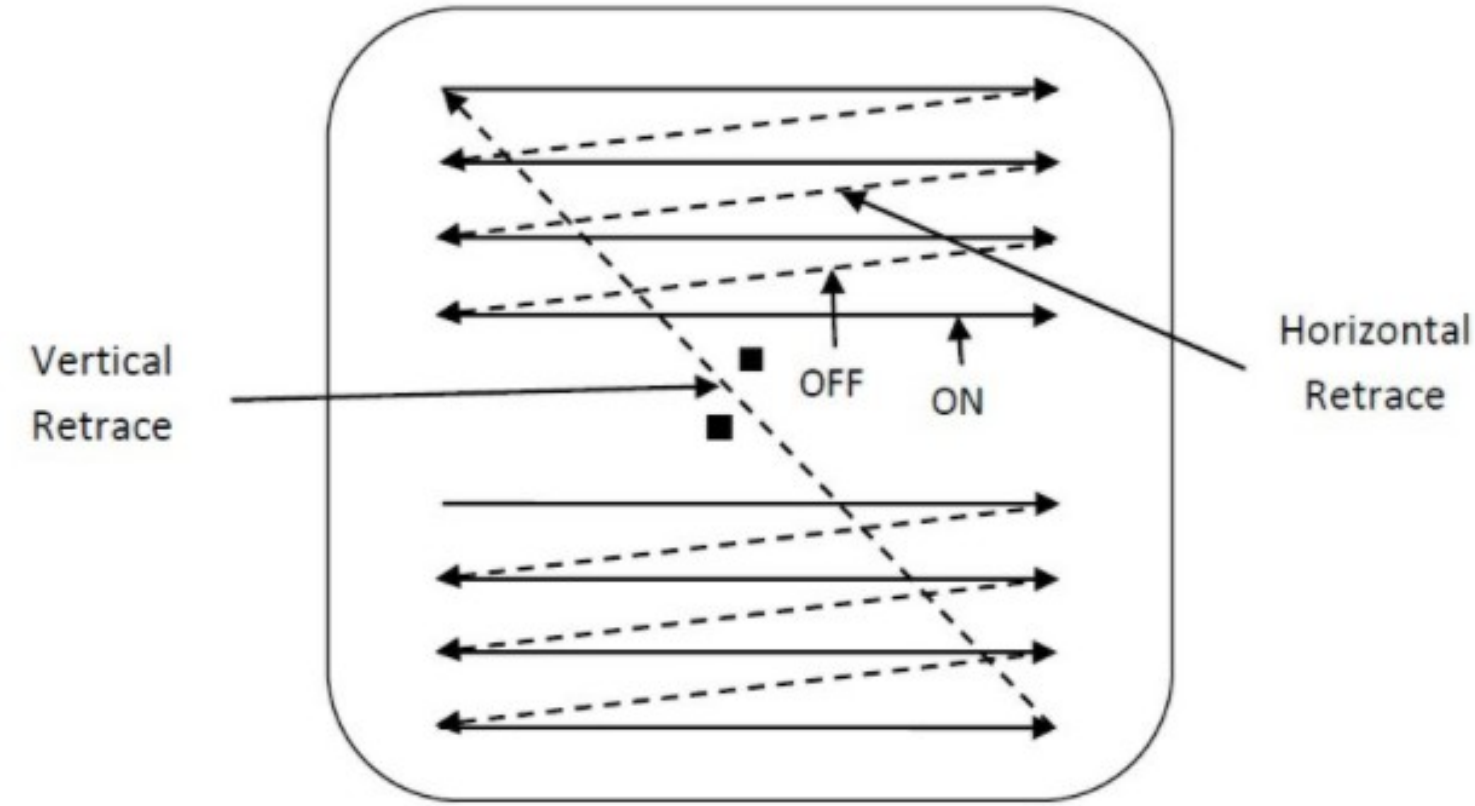
Raster Displays



Raster Displays

- The architecture of Raster display. It consists of a display controller, CPU, video controller, refreshes buffer, keyboard, mouse, and CRT.
- The display image is stored in the form of 1’s and 0’s in the refresh buffer.
- The video controller reads this refresh buffer and produces the actual image on screen.
- It will scan one line at a time from top to bottom & then back to the top.

Raster Displays



Raster Displays

- The horizontal and vertical deflection signals are generated to move the beam all over the screen in a pattern shown in Image.
  - Here beam is swept back & forth from left to the right.
  - When beam is moved from left to right it is ON.
  - When beam is moved from right to left it is OFF
  - The process of moving beam from right to left after completion of row is known as Horizontal Retrace.
  - When beam is reached at the bottom of the screen. It is made OFF and rapidly retraced back to the top left to start again and process of moving back to top is known as Vertical Retrace.

Raster Displays

- The screen image is maintained by repeatedly scanning the same image by refreshing of Screen.
- Frame Buffer is a special area of memory is dedicated to graphics. It holds set of intensity values for all the screen points
- That intensity is retrieved from frame buffer and display on screen one row at a time.
- Each screen point referred as pixel or Pel (Picture Element).
- Each pixel can be specified by its row and column Numbers.

Vector vs Raster Displays

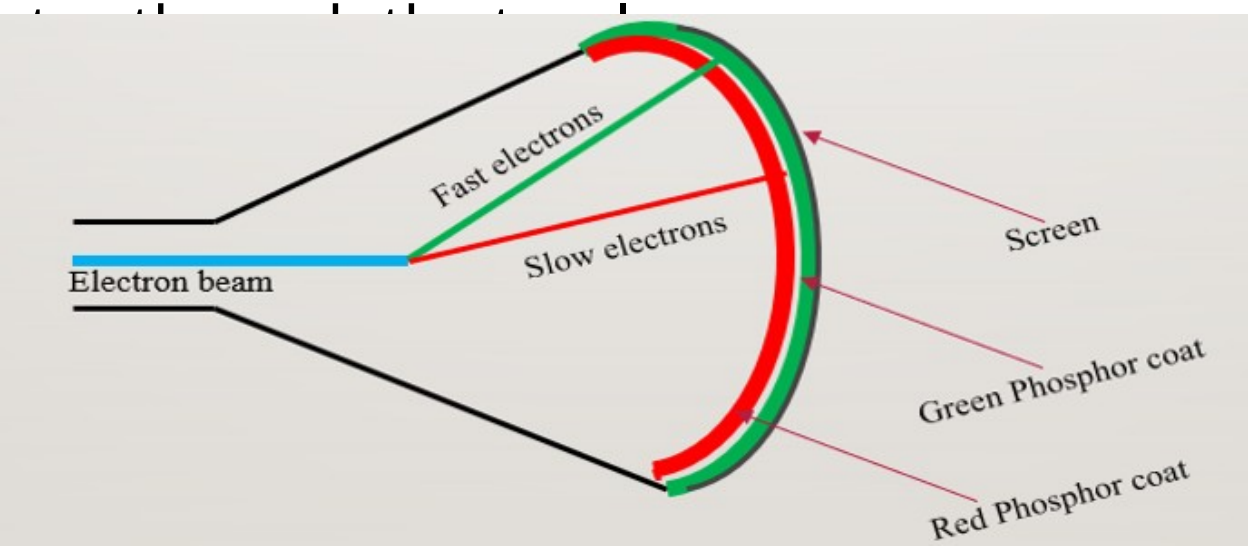
Vector Scan Display	Raster Scan Display
1. In vector scan display the beam is moved between the end points of the graphics primitives.	1. In raster scan display the beam is moved all over the screen one scan line at a time, from top bottom and then back to top,
2. Vector display flickers when the number of primitives in the buffer becomes too large.	2. In raster display, the refresh process is independent of the complexity of the image.
3. Scan conversion is not required.	3. Graphics primitives are specified in terms of their endpoints and must be scan converted into their corresponding pixels in the frame buffer.
4. Scan conversion hardware is not required.	4. Because each primitive must be scan-converted,real time dynamics is for more computational and requires separate scan conversion hardware.
5. Vector display draws a continuous and smooth lines.	5. Raster display can display mathematically smooth lines, polygons, and boundaries of curved primitives only by approximating them with pixels on the raster grid.
6. Cost is more.	6. Cost is low.
7. Vector display only draws lines and characters.	7. Raster display has ability to display areas filled with solid colours or patterns.

Color CRT Monitors

- Displays color pictures by using combination of phosphors that emit different colored light.
- Two techniques for producing color displays on CRT:
  - Beam Penetration Method
  - Shadow Mask Method

Beam Penetration Method

- Used with *random scan monitors*
- The screen has two layers of phosphor*: usually red and green*
- Color depends on how far the electron beam penetr

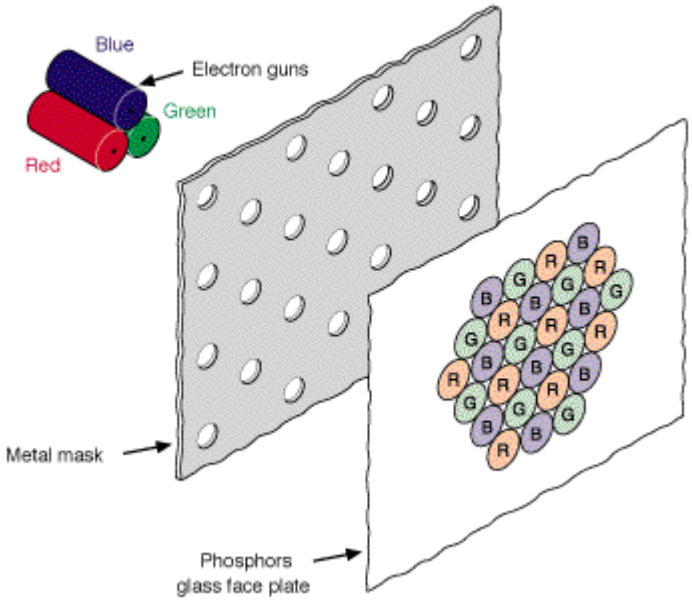


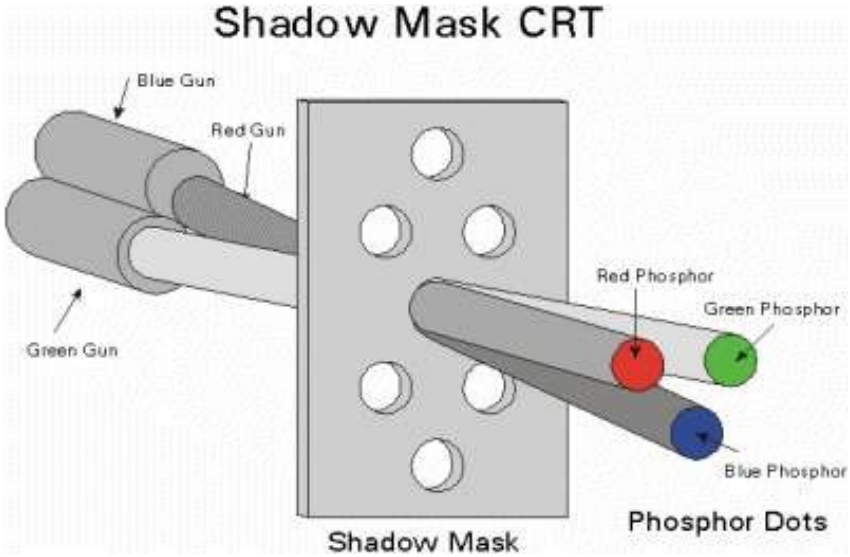
Beam Penetration Method(cont.)

- The speed of electrons – beam acceleration voltage. ➡
- Slow Electrons ➡ *Red* color
- Fast Electrons *Green* color ➡
- Intermediate Speed Electrons *Yellow* and *orange* colors
- Quality of Picture is *not so good* and is *inexpensive*.
- Can display only *four colors*.

Shadow-Mask Method

- Color CRTs have
  - Three electron guns
  - A metal shadow mask to differentiate the beams





Shadow-Mask Method

- Uses Red, green and Blue-in suitable proportions to get a combination of colors
- Three electron gun placed one by the side of the other to form a triangle or a “Delta”
- Each pixel point on the screen is also made up of 3 types of phosphors to produce red, blue and green colors.
- Before the phosphor screen is a metal screen, called a “shadow mask”.
- The 3 electron beams are deflected and focused as a group onto the shadow mask, which contains a series of holes aligned with the phosphor-dot patterns.
- The 3 beams pass through a hole in the shadow mask and activate a dot triangle, which appears as a small color spot on the screen.

Reference

- **Computer Graphics**
  - Donald Hearn, M Pauline Baker
  - Pearson Education

Modeling Transformations

2D Transformations

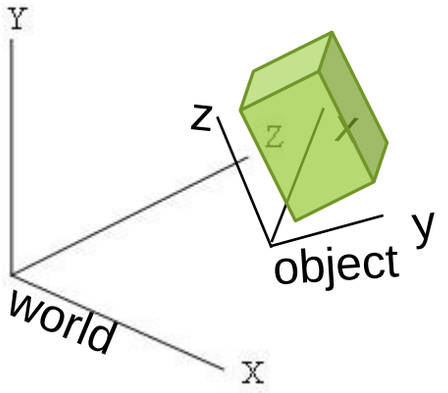
Dr. Sk. Md. Masudul Ahsan
Professor

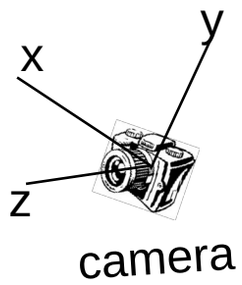
Modeling Transformations

- ~ places the objects defined by their own coordinate system into a common scene of world coordinate system.
- Moving or transforming an object can be achieved by
  - **Geometric transformation** – changes in orientation, size and shape of objects are accomplished by altering coordinate description of objects.
  - **Coordinate Transformation** – same effect is attained by holding the object stationary while coordinate system is transformed relative to object.

Use of Transformations

- Modeling
  - Build complex models by positioning simple components
  - Transform from object coordinates to world coordinates
- Viewing
  - Placing the virtual camera in the world
  - Specifying transformation from world to camera coordinates
- Animation
  - Vary transformations over time to create moti





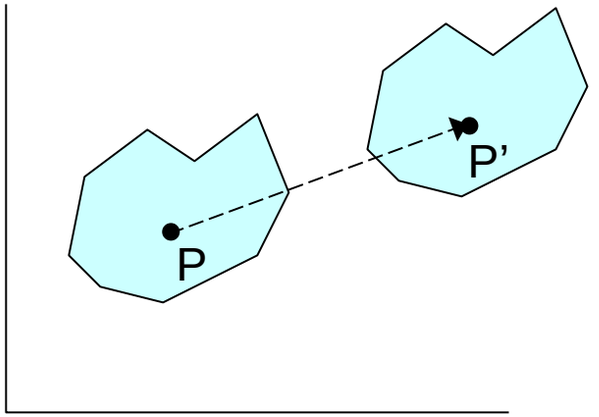
Translation – 2D

- If displacement is the vector $V = t_x I + t_y J$, new point P′(x′,y′) can be found by adding translation distance (t_x,t_y) to the old point P(x,y)

$$i.e. \quad x' = x + t_x \;,\; y' = y + t_y \quad \text{--- (i)}$$

Representing the points and displacement as a column vector

$$P = \begin{bmatrix} x \\ y \end{bmatrix} \quad P' = \begin{bmatrix} x' \\ y' \end{bmatrix} \quad T = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

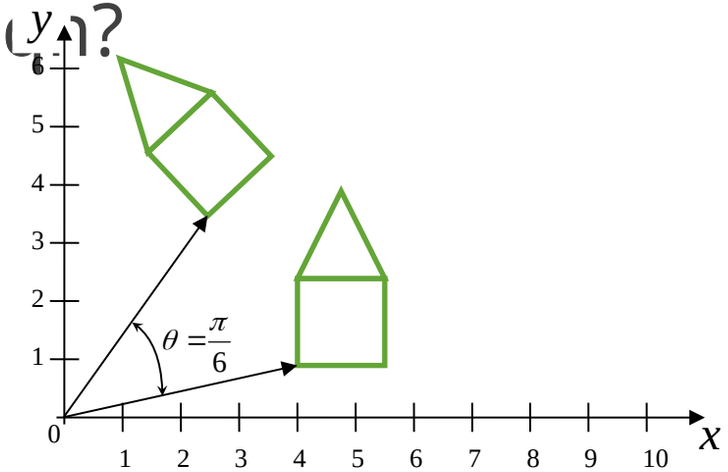


Eq. (i) becomes

$$P' = P + T$$

Rotation – 2D

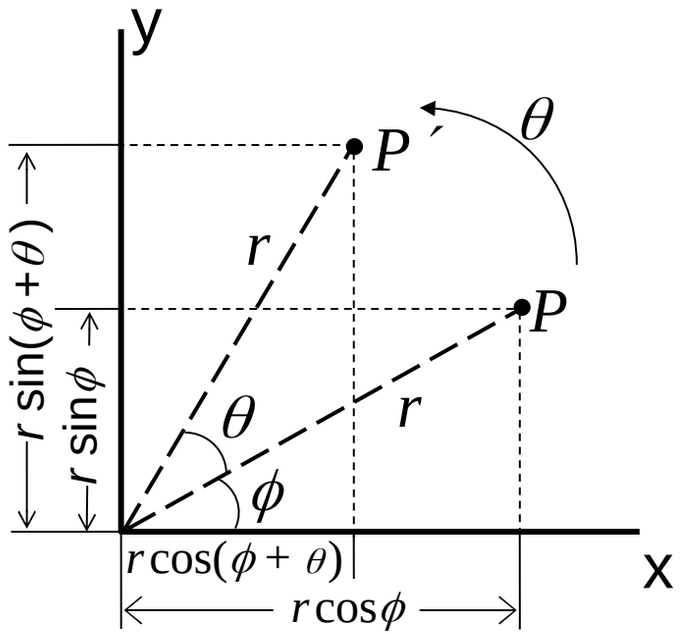
- In 2d rotation, object is repositioned along a circular path in xy plane.
  - Rotates all coordinates by a specified angle
- For rotation we specify
  - Rotation angle (θ)
    - Counter clockwise – positive angle
    - Clockwise – negative angle
  - Pivot point (x_r,y_r) – center of rotation
- What is the transformation eqn?



Rotation – 2D, origin as pivot point

$$P = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} r\cos\phi \\ r\sin\phi \end{bmatrix}$$

$$P' = \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} r\cos(\phi+\theta) \\ r\sin(\phi+\theta) \end{bmatrix}$$

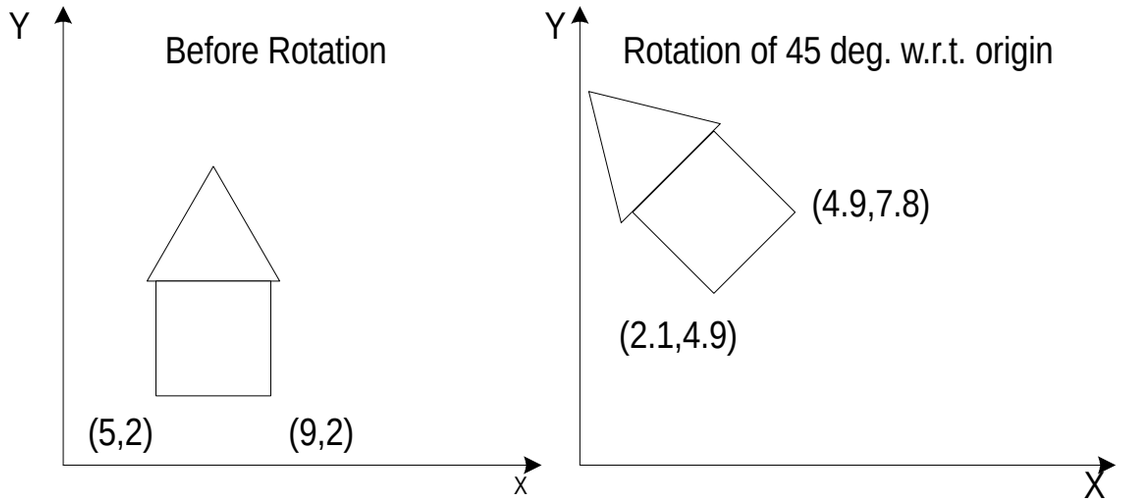


$$\text{expand } (\phi+\theta) \Rightarrow \begin{cases} x' = r\cos\phi\cos\theta - r\sin\phi\sin\theta \\ y' = r\cos\phi\sin\theta + r\sin\phi\cos\theta \end{cases}$$

$$\text{but } \begin{matrix} x = r\cos\phi \\ y = r\sin\phi \end{matrix} \Rightarrow \begin{matrix} x' = x\cos\theta - y\sin\theta \\ y' = x\sin\theta + y\cos\theta \end{matrix}$$

Rotation – 2D

$$x' = x \cdot \cos\theta - y \cdot \sin\theta$$
$$y' = x \cdot \sin\theta + y \cdot \cos\theta$$



This eq can be represented in matrix form as -

$$P' = R \cdot P$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \cdot \cos\theta - y \cdot \sin\theta \\ x \cdot \sin\theta + y \cdot \cos\theta \end{bmatrix} = \begin{bmatrix} \quad \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Scaling – 2D

Scaling alters the size of an object

Transformed coordinates P′(x′, y′) are found by multiplying the coordinate values P(x, y) of each vertex of object by **scaling factor** s_x and s_y.

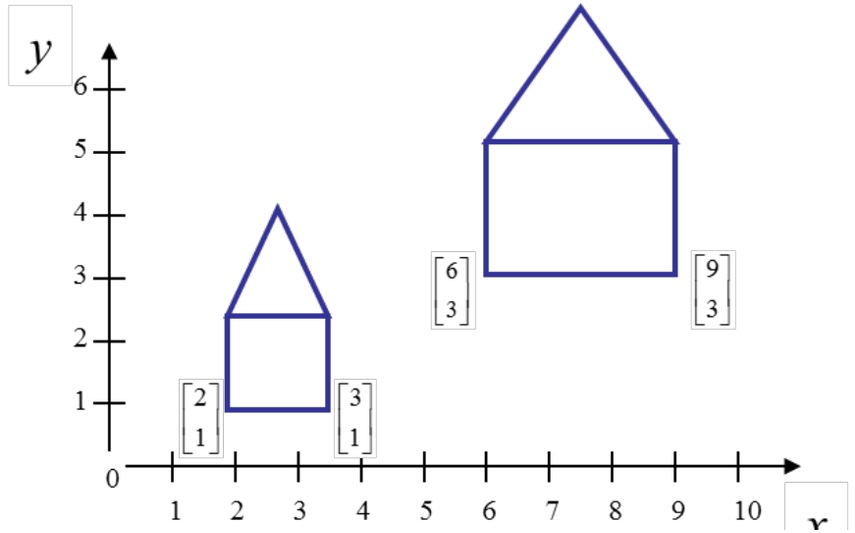
ie.

$$x' = s_x \cdot x \quad , \quad y' = s_y \cdot y$$

In matrix form

$$P' = S \cdot P$$

$$\begin{bmatrix} x \cdot s_x \\ y \cdot s_y \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

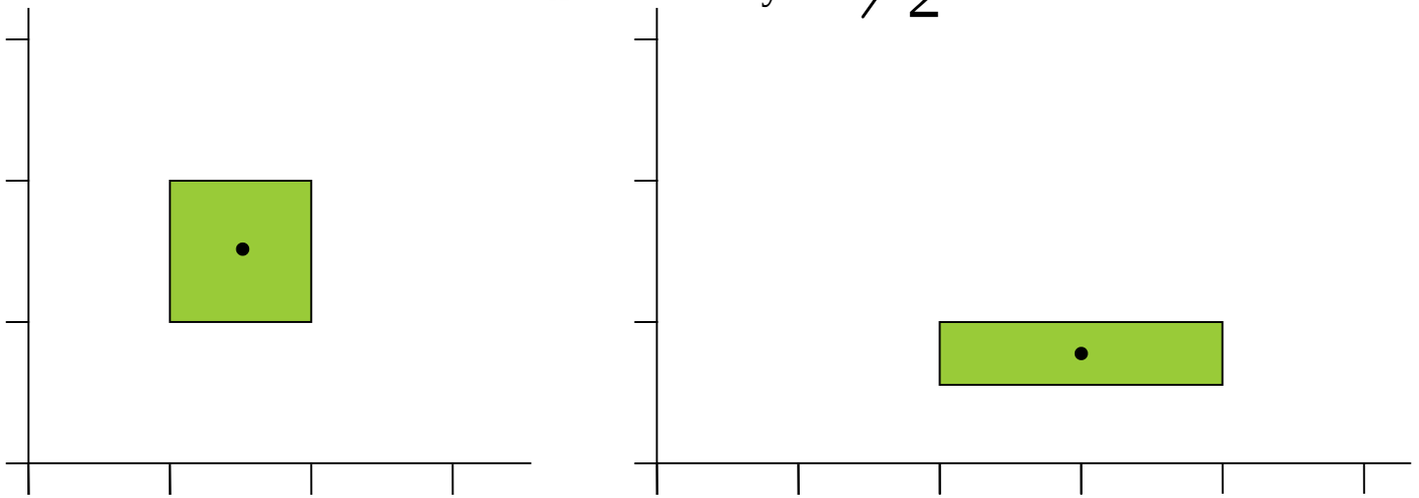


WATCH OUT: Objects grow and move!

Scaling – 2D

- Types of Scaling:
  - Differential (s_x != s_y)
  - Uniform (s_x = s_y)
- After scaling, new object is located at a different position relative to origin

$$scale\ factors \quad s_x = 2,\ s_y = \frac{1}{2}$$



Homogeneous Coordinates

A point (x, y) can be re-written in **homogeneous coordinates** as (x_h, y_h, h)

The **homogeneous parameter** h is a non-zero value such that: $x = \frac{x_h}{h}$ $y = \frac{y_h}{h}$

We can then write any point (x, y) as (h_x, h_y, h)

We can conveniently choose $h = 1$ so that (x, y) becomes $(x, y, 1)$

Why Homogeneous Coordinates?

- Mathematicians commonly use homogeneous coordinates as they allow scaling factors to be removed from equations
- Translation, scaling and rotation are expressed (non-homogeneously) as:
  - translation: $P' = P + T$
  - Scale: $P' = S \cdot P$
  - Rotate: $P' = R \cdot P$
- Composition is difficult to express, since translation not expressed as a matrix multiplication
- Using homogeneous coordinates allows us use matrix multiplication to calculate transformations – extremely efficient!

Homogeneous Translation

$$x' = x + t_x \quad , \quad y' = y + t_y$$

- Representing the point as a homogeneous column vector
  - The translation of a point by (t_x, t_y) can be written in matrix form as:

$$\mathbf{x}' = \mathbf{T}_{(t_x, t_y)} \quad \mathbf{x}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Homogeneous Co-ordinates

- For rotation

$$x' = x \cdot \cos\theta - y \cdot \sin\theta \quad , \quad y' = x \cdot \sin\theta + y \cdot \cos\theta$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$R_{(\theta)}$$

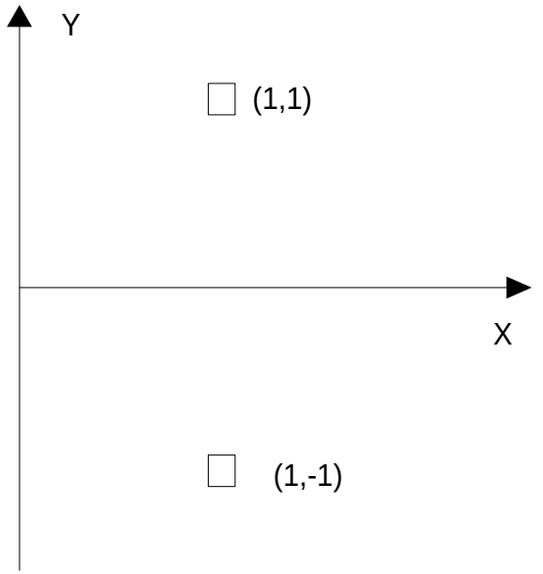
- For Scaling

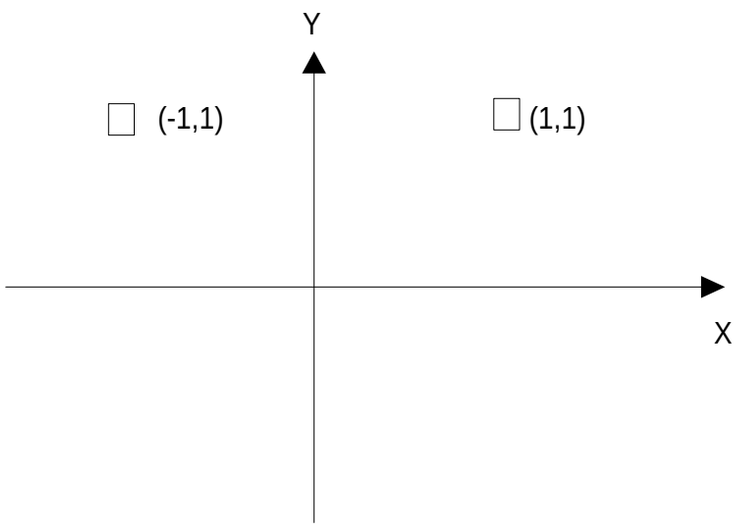
$$x' = s_x \cdot x \quad , \quad y' = s_y \cdot y$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_x & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$S_{(s_x, s_y)}$$

Mirror Reflection





Reflection about X - axis

$$x' = x \quad y' = -y$$

$$M_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Reflection about Y - axis

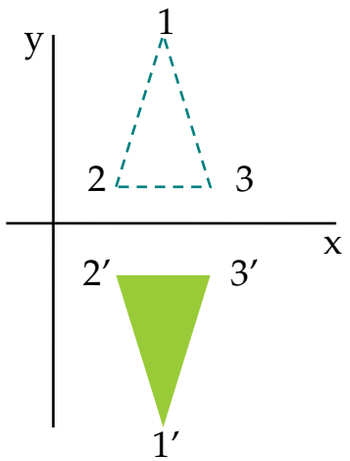
$$x' = -x \quad y' = y$$

$$M_y = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Reflection

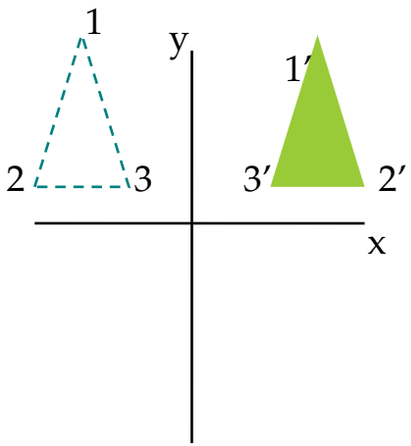
Reflection with respect to the axis

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



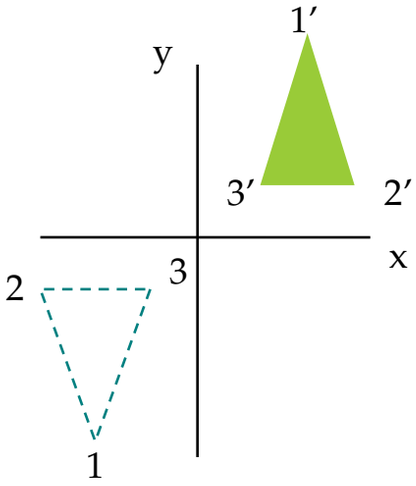
About x axis

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



About y axis

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



About origin

Shearing Transformation

$$x' = x + sh_x$$
$$y' = y$$

$$x' = x$$
$$y' = sh_y + y$$

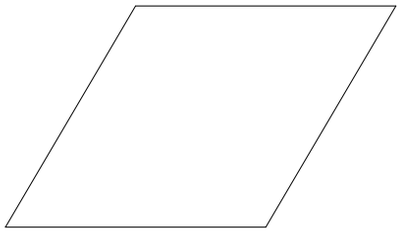
$$x' = x + sh_x$$
$$y' = sh_y + y$$

$$SH_{(sh_x)} = \begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

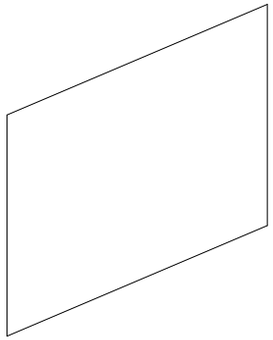
$$SH_{(sh_y)} = \begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$SH_{(sh_x,sh_y)} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

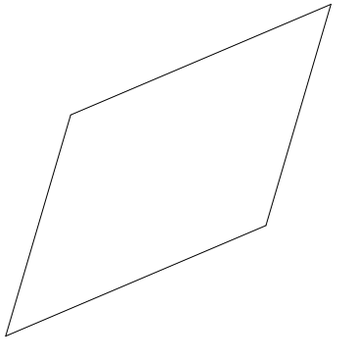
unit cube



Sheared in X direction



Sheared in Y direction



Sheared in both X and Y direction

Inverse 2D - Transformations

- Transformations can easily be reversed using inverse transformations

Translaiton: $T^{-1}_{(tx,ty)} = T_{(-tx,-ty)} = \begin{bmatrix} 1 & 0 & -tx \\ 0 & 1 & -ty \\ 0 & 0 & 1 \end{bmatrix}$

Rotation : $R^{-1}_{(\theta)} = R_{(-\theta)} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Sclaing : $S^{-1}_{(sx,sy)} = S_{(1/sx,1/sy)} = \begin{bmatrix} 1/sx & 0 & 0 \\ 0 & 1/sy & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Mirror Ref: $M^{-1}_{x} = M_{x}$

$M^{-1}_{y} = M_{y}$

$$\mathbf{SH}^{-1}_{(shx,sh_y)} = \frac{-1}{1 - sh_x sh_y} \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Composite Transformation

- **Transformations can be combined by matrix multiplication**

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left(\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$$p' = \mathrm{T}(tx, ty)\ \mathrm{R}(\theta)\ \mathrm{S}(sx, sy)\ p$$

- **Efficiency with premultiplication**
  - Matrix multiplication is associative

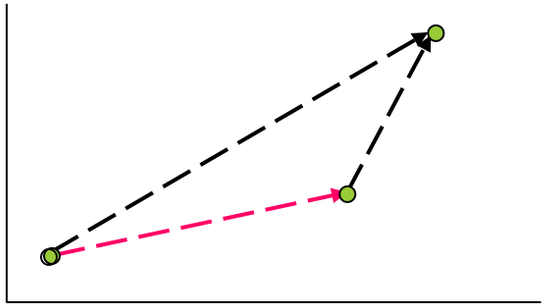
$$p' = (T \times (R \times (S \times p))) \rightarrow p' = (T \times R \times S) \times p$$

Composite Transformation

- Translation: 2 successive ~ are applied to point P

$$P' = T(t_{x2}, t_{y2}) \cdot [T(t_{x1}, t_{y1}) \cdot P] = [T(t_{x2}, t_{y2}) \cdot T(t_{x1}, t_{y1})] \cdot P$$

$$Now,\ T(t_{x2}, t_{y2}) \cdot T(t_{x1}, t_{y1}) = \begin{bmatrix} 1 & 0 & t_{x2} \\ 0 & 1 & t_{y2} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & t_{x1} \\ 0 & 1 & t_{y1} \\ 0 & 0 & 1 \end{bmatrix}$$



$$= \begin{bmatrix} 1 & 0 & t_{x1} + t_{x2} \\ 0 & 1 & t_{y1} + t_{y2} \\ 0 & 0 & 1 \end{bmatrix} = T(t_{x1} + t_{x2}, t_{y1} + t_{y2})$$

- ie. Successive translations are additive

Composite Transformation - DIY

- Successive Rotation are additive, and
- Successive Scaling are multiplicative

Properties of rigid-body transformation

The following Matrix is Orthogonal if the upper left 2X2 matrix has the
following properties

$$\begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

1.A) Each row are **unit vector**.

$\text{sqrt}(r_{11} * r_{11} + r_{12} * r_{12}) = 1$

B) Each column are unit vector.

$\text{sqrt}(c_{11} * c_{11} + c_{12} * c_{12}) = 1$

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

2.A) Rows will be **perpendicular** to each other

$(r_{11}, r_{12}) . (r_{21}, r_{22}) = 0$

B) Columns will be perpendicular to each other

$(c_{11}, c_{12}) . (c_{21}, c_{22}) = 0$

e.g. Rotation matrix is orthogonal

- **Orthogonal Transformation ⇒ Rigid-Body Transformation**
- **For any orthogonal matrix B ⇒ $B^{-1} = B^T$**

Linear Transformations

- **Linear transformations are combinations of …**
  - Scale
  - Rotation
  - Shear, and
  - Mirror

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

- **Properties of linear transformations**
  - Satisfies:
  - Origin maps to origin
  - Lines map to lines
  - Parallel lines remain parallel
  - Ratios are preserved
  - Closed under composition

$$T(s_1 p_1 + s_2 p_2) = s_1 T(p_1) + s_2 T(p_2)$$

Affine Transformations

- **Affine transformations are combinations of**
  - Linear transformations, and
  - Translations

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

- **Properties of affine transformations**
  - Origin does not map to origin
  - Lines map to lines
  - Parallel lines remain parallel **(Find how to prove it for exam)**
  - Ratios are preserved
  - Closed under composition

Projective Transformations

- **Projective transformations…**
  - Affine transformations, and
  - Projective warps

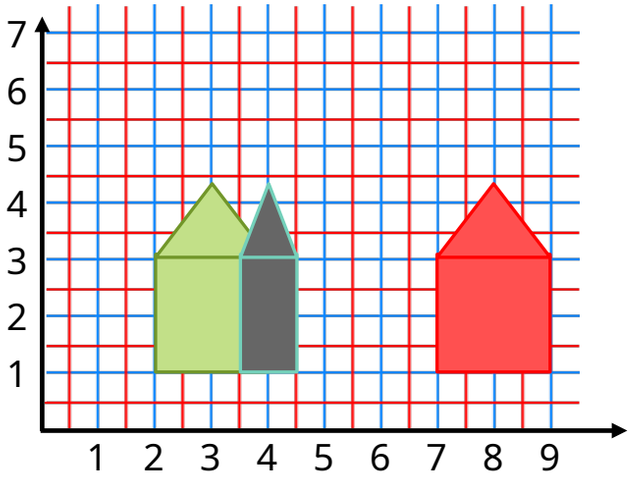
$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

- **Properties of projective transformations**
  - Origin does not map to origin
  - Lines map to lines
  - Parallel lines do not necessarily remain parallel
  - Ratios are not preserved
  - Closed under composition

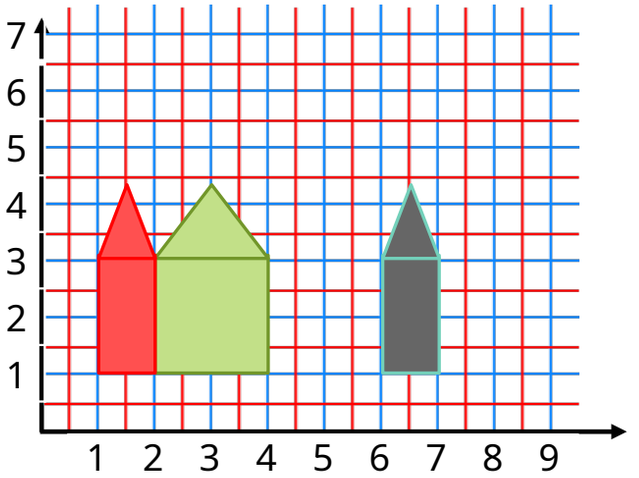
Commutativity of Transformation Matrices

- Matrix multiplication is not commutative

- Some non-commutative Compositions:
  - Non-uniform scale, Rotate
  - Translate, Scale
  - Rotate, Translate



Translate(5,0)-> Scale(0.5,1)



Scale(0.5,1) -> Translate(5,0)

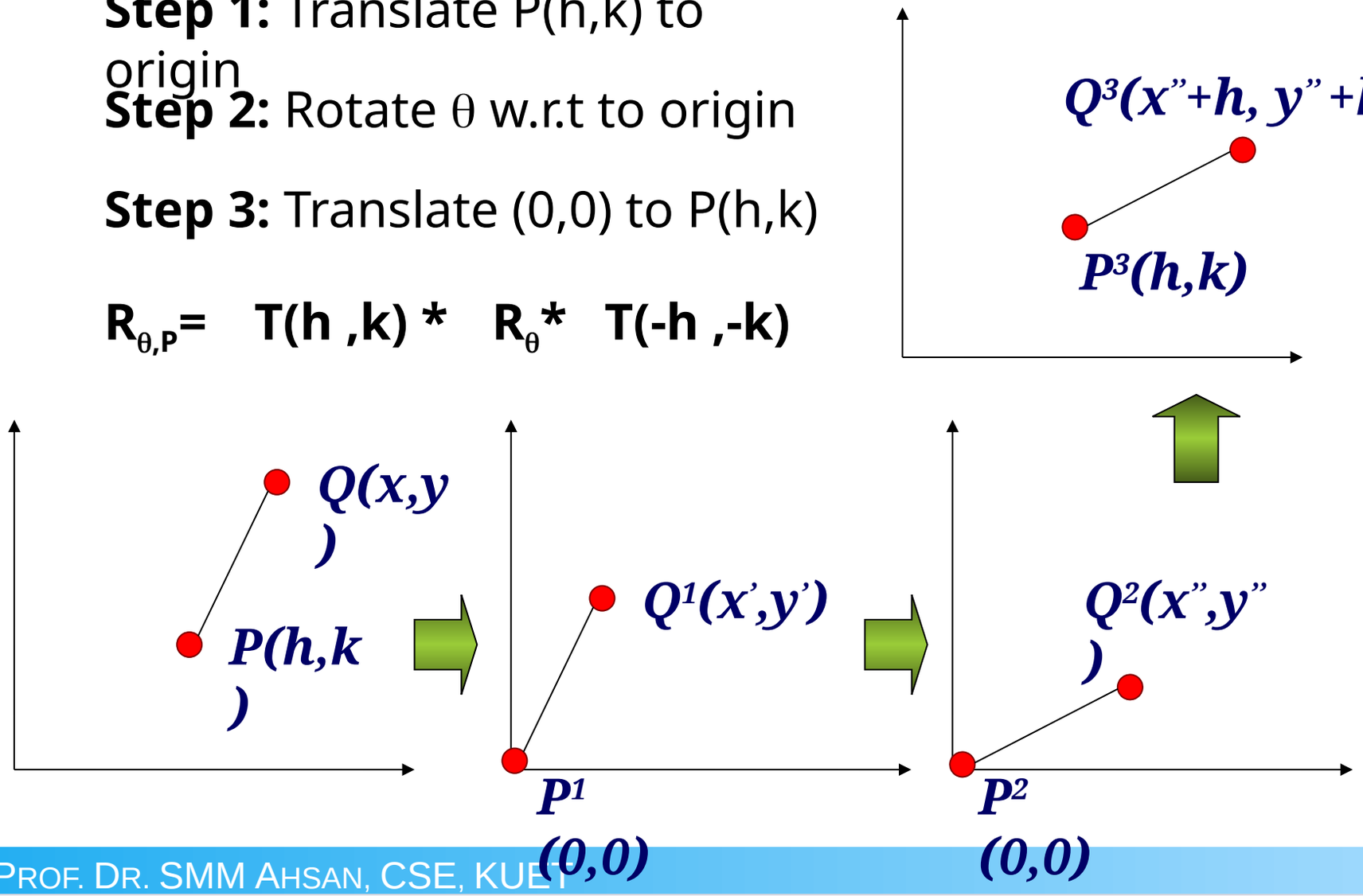
General Pivot point, P(h,k), Rotation of θ : $R_{\theta,P}$

Step 1: Translate P(h,k) to origin

Step 2: Rotate θ w.r.t to origin

Step 3: Translate (0,0) to P(h,k)

$R_{\theta,P}$= T(h ,k) * R_{θ}* T(-h ,-k)



General Pivot point, P(h,k), Rotation of θ : $R_{\theta,P}$

$$R_{\theta,p} = T_{(h,k)} \bullet R_{\theta} \bullet T_{(-h,-k)}$$

$$R_{\theta,p} = \begin{bmatrix} 1 & 0 & h \\ 0 & 1 & k \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -h \\ 0 & 1 & -k \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos\theta & -\sin\theta & h \\ \sin\theta & \cos\theta & k \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -h \\ 0 & 1 & -k \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos\theta & -\sin\theta & h(1-\cos\theta)+k\sin\theta \\ \sin\theta & \cos\theta & k(1-\cos\theta)-h\sin\theta \\ 0 & 0 & 1 \end{bmatrix}$$

Example

- Rotate the point P(4,5) 45° about the point A(1,1)

$$P' = R_{45^\circ,A} \bullet P = T_{(1,1)} \bullet R_{45^\circ} \bullet T_{(-1,-1)} \bullet P$$

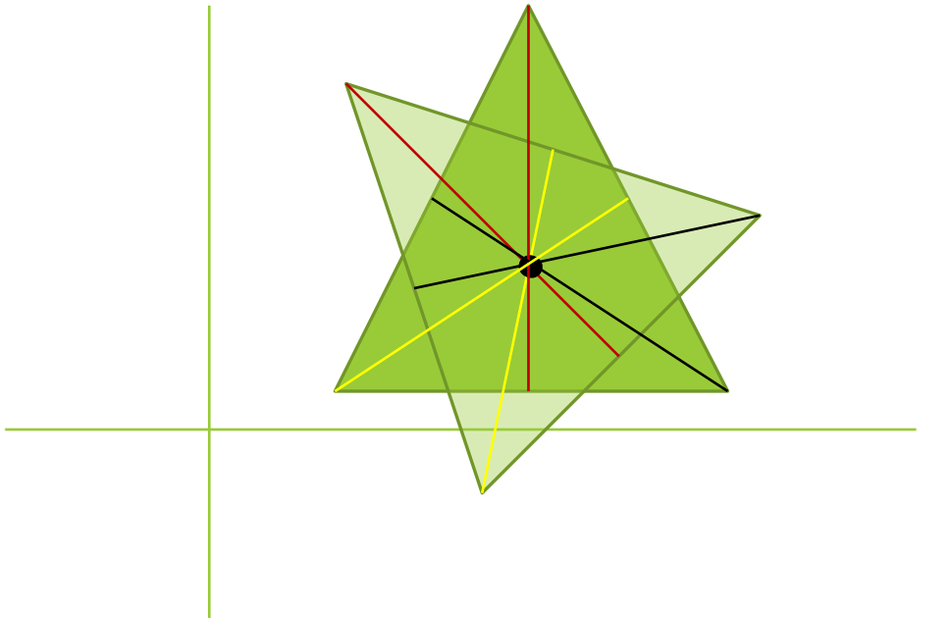
$$= \begin{bmatrix} 1 & 0 & h \\ 0 & 1 & k \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -h \\ 0 & 1 & -k \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & h(1-\cos\theta)+k\sin\theta \\ \sin\theta & \cos\theta & k(1-\cos\theta)-h\sin\theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{2} & 1(1-1/\sqrt{2})+1.1/\sqrt{2} \\ 1/\sqrt{2} & 1/\sqrt{2} & 1(1-1/\sqrt{2})-1.1/\sqrt{2} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix} = \begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{2} & 1 \\ 1/\sqrt{2} & 1/\sqrt{2} & 1-\sqrt{2} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1-1/\sqrt{2} \\ 1+7/\sqrt{2} \\ 1 \end{bmatrix}$$

DIY

- Rotate the triangle keeping the center fixed



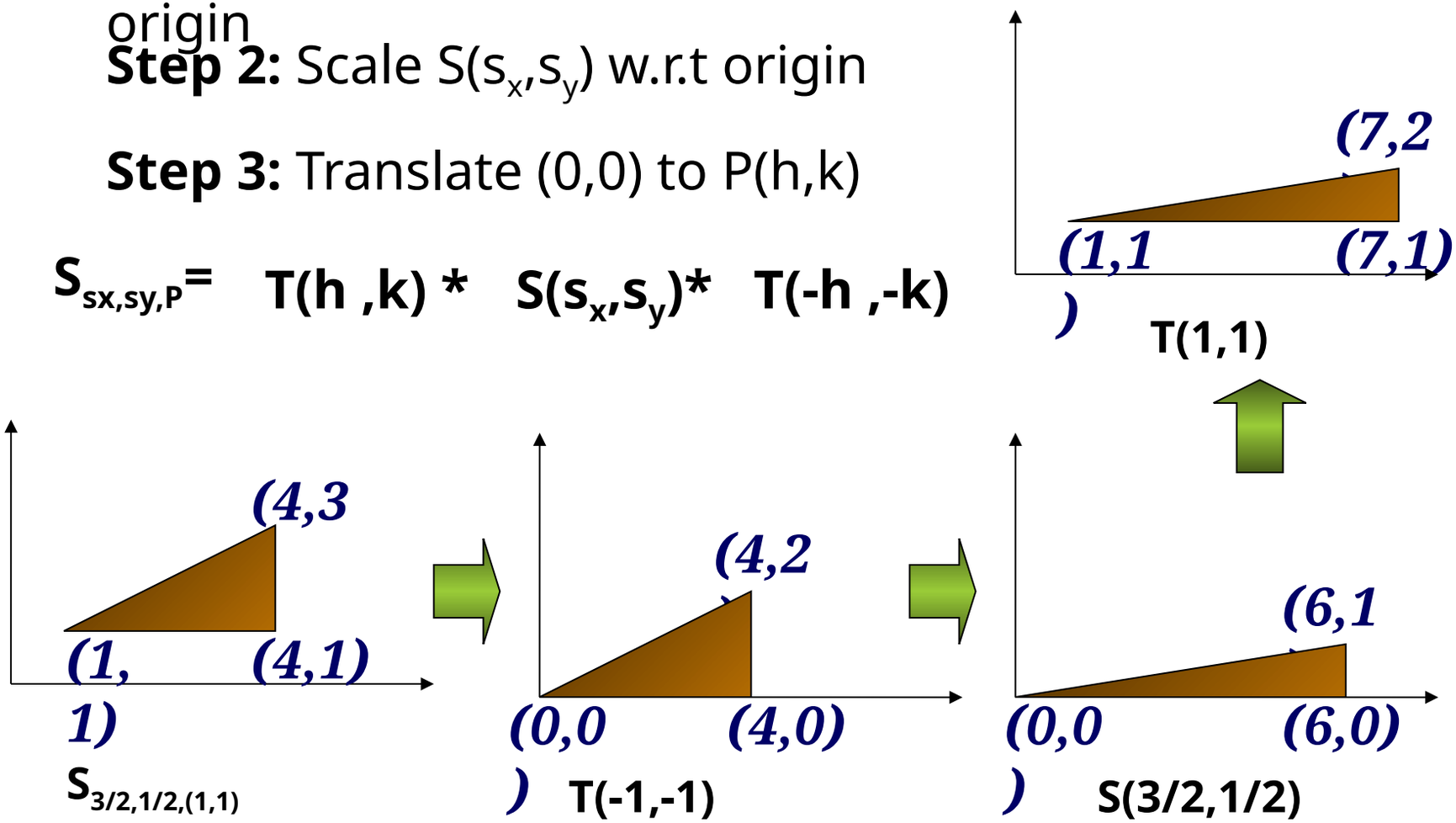
Fixed point P(h,k) Scaling : $S_{sx,sy,p}$

Step 1: Translate P(h,k) to origin

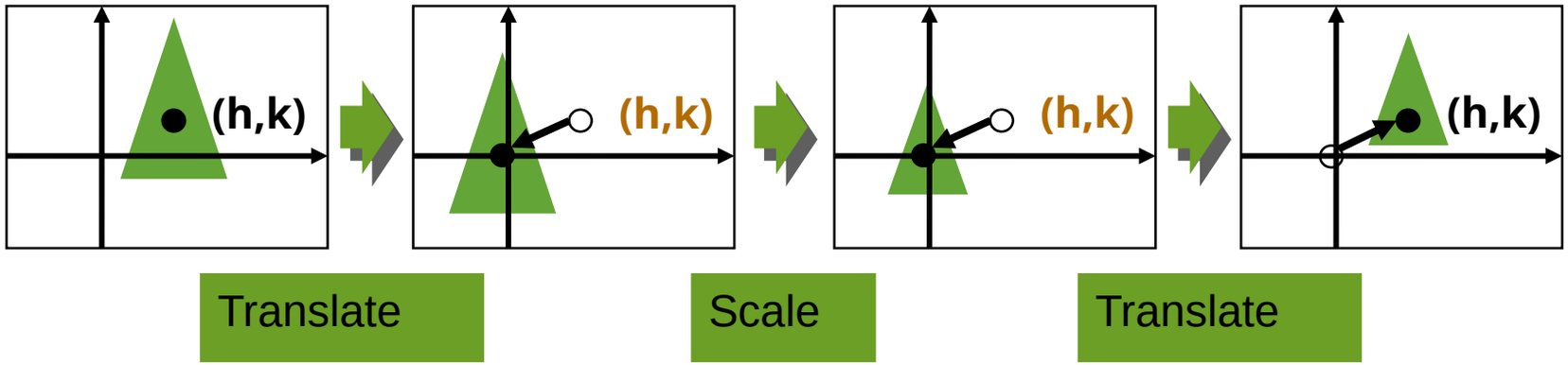
Step 2: Scale S(s_x,s_y) w.r.t origin

Step 3: Translate (0,0) to P(h,k)

$S_{sx,sy,P}$= T(h ,k) * S(s_x,s_y)* T(-h ,-k)



Fixed point P(h,k) Scaling : $S_{sx,sy,p}$

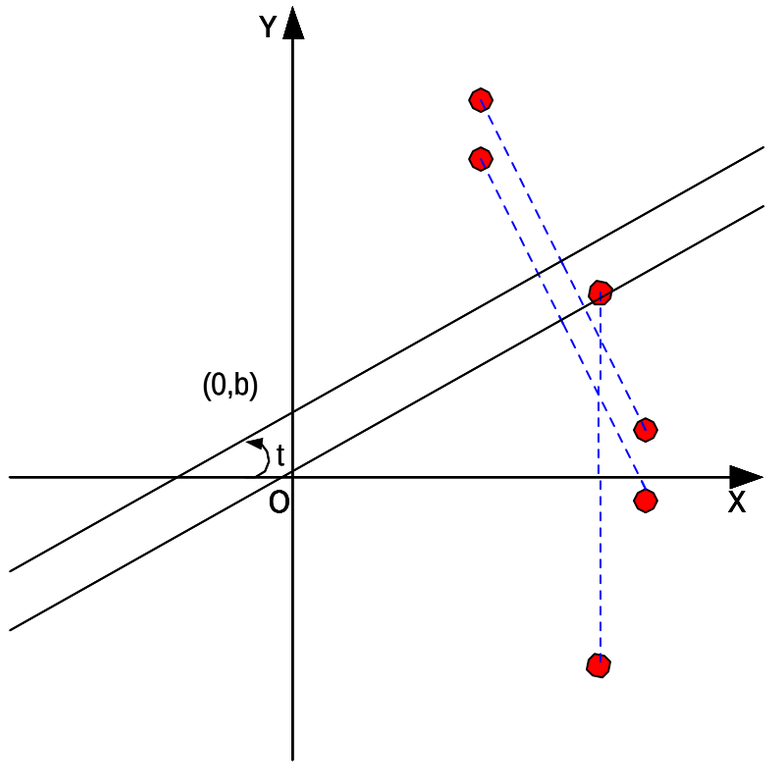


$$T(h,k)\cdot S(s_x,s_y)\cdot T(-h,-k)=S(s_x,s_y,h,k)$$

$$\begin{bmatrix}1 & 0 & h\\ 0 & 1 & k\\ 0 & 0 & 1\end{bmatrix}\cdot\begin{bmatrix}s_x & 0 & 0\\ 0 & s_y & 0\\ 0 & 0 & 1\end{bmatrix}\cdot\begin{bmatrix}1 & 0 & -h\\ 0 & 1 & -k\\ 0 & 0 & 1\end{bmatrix}=\begin{bmatrix}s_x & 0 & h(1-s_x)\\ 0 & s_y & k(1-s_y)\\ 0 & 0 & 1\end{bmatrix}$$

Reflection about line L, M_L

Step 1: Translate (0,b) to origin
Step 2: Rotate -θ degrees
Step 3: Mirror reflect about X-axis
Step 4: Rotate θ degrees
Step 5: Translate origin to (0,b)



$$M_L = T(0,b) * R(\theta) * M_x * R(-\theta) * T(0,-b)$$

Reflection about line L, M_L

$$M_L = T(0, b) * R(\theta) * M_x * R(-\theta) * T(0, -b)$$

$$M_L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & b \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -b \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & b \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ \sin\theta & -\cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -b \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} cos^2\theta - sin^2\theta & 2\sin\theta cos\theta & 0 \\ 2\sin\theta cos\theta & -(cos^2\theta - sin^2\theta) & b \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -b \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \frac{1}{1+tan^2\theta} \begin{bmatrix} 1-tan^2\theta & 2tan\theta & 0 \\ 2tan\theta & -(1-tan^2\theta) & b(1+tan^2\theta) \\ 0 & 0 & 1+tan^2\theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -b \\ 0 & 0 & 1 \end{bmatrix}$$

$$M_L = \frac{1}{m^2+1} \begin{bmatrix} 1-m^2 & 2m & -2bm \\ 2m & m^2-1 & 2b \\ 0 & 0 & m^2+1 \end{bmatrix}$$

$$\sin(2\theta) = 2\sin\theta\cos\theta = (\sin\theta + \cos\theta)^2 - 1 = \frac{2\tan\theta}{1+\tan^2\theta}$$

$$\cos(2\theta) = \cos^2\theta - \sin^2\theta = 2\cos^2\theta - 1 = 1 - 2\sin^2\theta = \frac{1-\tan^2\theta}{1+\tan^2\theta}$$

Problem

Show that two successive reflections about any line (two lines) in the xy plane that intersects the coordinate origin is equivalent to a rotation in the xy plane about the origin

$$M_{L1} = \begin{bmatrix} cos^2\theta_1 - sin^2\theta_1 & 2\sin\theta_1 cos\theta_1 & 0 \\ 2\sin\theta_1 cos\theta_1 & -(cos^2\theta_1 - sin^2\theta_1) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos 2\theta_1 & \sin 2\theta \\ \sin 2\theta_1 & -\cos 2\theta_1 \end{bmatrix} = \begin{bmatrix} \cos A & \sin A \\ \sin A & -\cos A \end{bmatrix}$$

$$M_{L2} = \begin{bmatrix} \cos B & \sin B \\ \sin B & -\cos B \end{bmatrix}$$

$$M_{L2}M_{L1} = \begin{bmatrix} \cos A\cos B + \sin A\sin B & \sin A\cos B - \cos A\sin B \\ \cos A\sin B - \sin A\cos B & \cos A\cos B + \sin A\sin B \end{bmatrix} = \begin{bmatrix} \cos(A-B) & \sin(A-B) \\ -\sin(A-B) & \cos(A-B) \end{bmatrix}$$

Problem

Reflecting a point $\boldsymbol{P} = \langle x, y \rangle$ about an arbitrary axis passing through points $\boldsymbol{P}_a = \langle x_a, y_a \rangle$ and $\boldsymbol{P}_b = \langle x_b, y_b \rangle$, show that

$$x_5 = (x - x_a) \cdot \cos(2\theta) + (y - y_a) \cdot \sin(2\theta) + x_a$$

$$y_5 = (x - x_a) \cdot \sin(2\theta) - (y - y_a) \cdot \cos(2\theta) + y_a$$

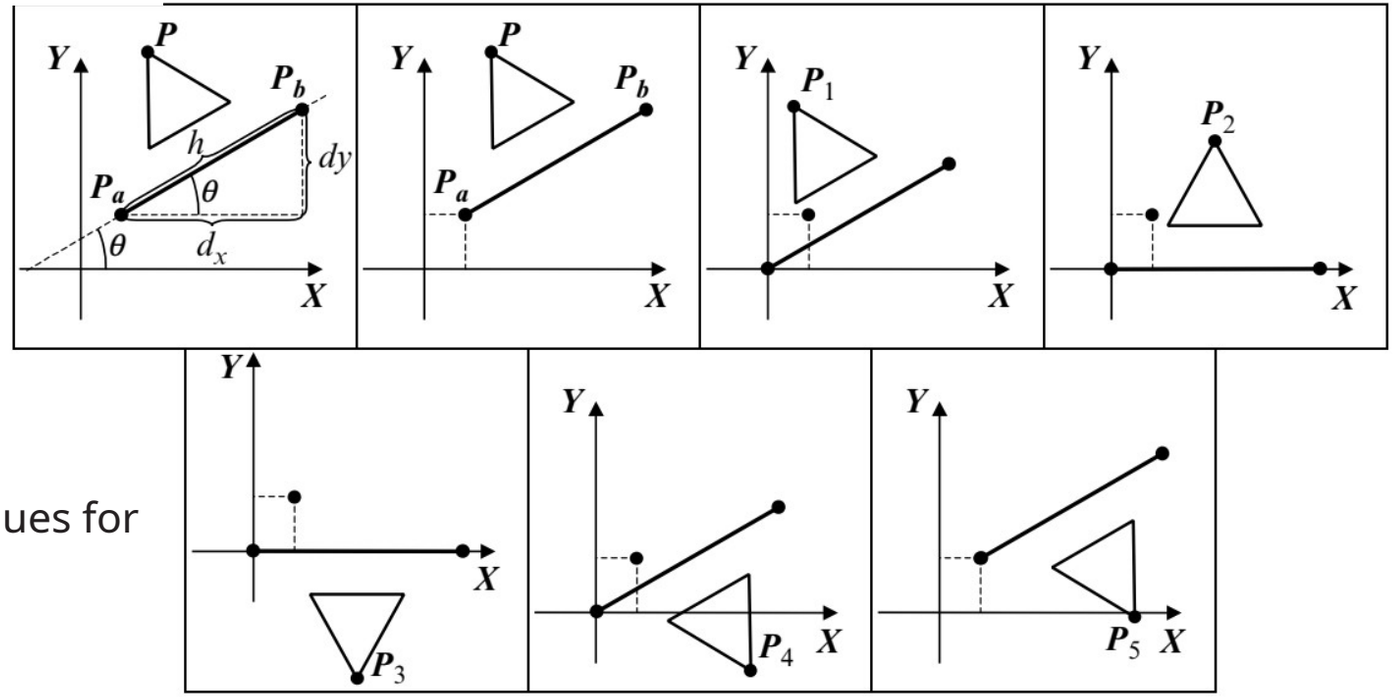


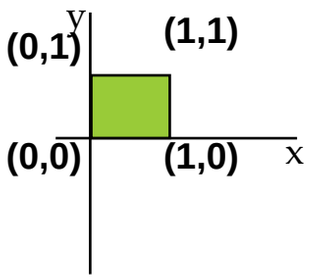
FIGURE 5.12. A reflection transformation about an arbitrary axis.

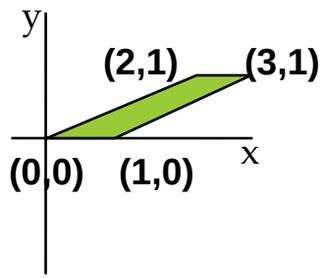
Mathematical and Computer Programming Techniques for Computer Graphics – by Peter Comninos

Shear – with ref line

- **Converted to a parallelogram**

$$x' = x + sh_x \cdot y, \quad y' = y \qquad \begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



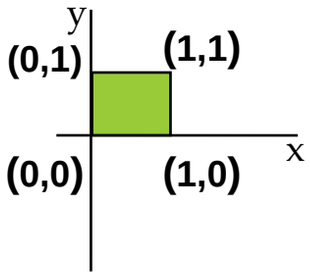


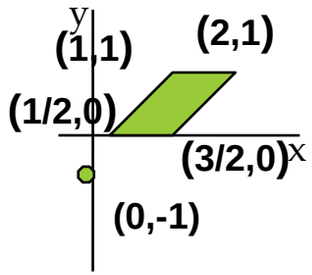
($Sh_x=2$)

- **Transformed to a shifted parallelogram**

(Y = Yref)

$$x' = x + sh_x \cdot (y - y_{ref}), \quad y' = y \qquad \begin{bmatrix} 1 & sh_x & -sh_x \cdot y_{ref} \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$





($Sh_x=1/2$, $y_{ref}=-1$)

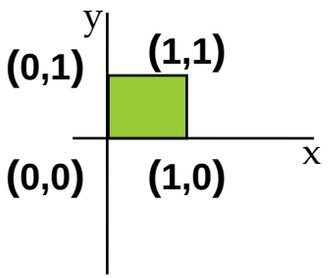
Shear – with ref line

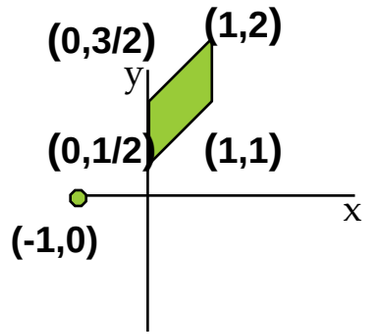
- **Transformed to a shifted parallelogram**

(X = Xref)

$$\begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & -sh_y \cdot x_{ref} \\ 0 & 0 & 1 \end{bmatrix}$$

$x' = x, \quad y' = sh_y \cdot (x - x_{ref}) + y$





($Sh_y=1/2$, $x_{ref}=-1$)

Problems to be solved:

Schaum's outline series:

Problems:

- 4.2
- 4.3, 4.4, 4.5 => $R_{\theta,P}$
- 4.6, 4.7, 4.8 => $S_{sx,sy,,P}$
- 4.9, 4.10, 4.11, 4.21 => M_L
- 4.12 => Shearing
- 4.16, 4.18, 4.29, 4.34

Hierarchical modeling
https://www.youtube.com/watch?v=4iNJdWXsFQ4

3D Modelling Transform

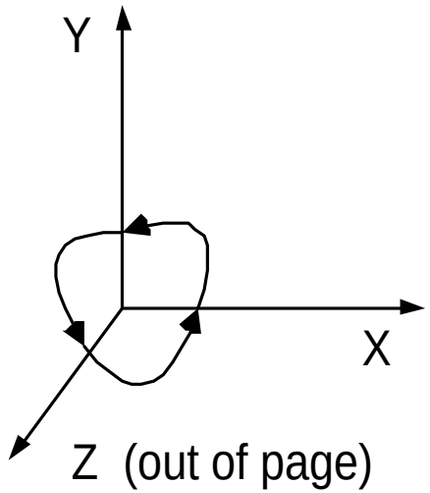
Dr. Sk. Md. Masudul Ahsan
Professor

3D Transformations

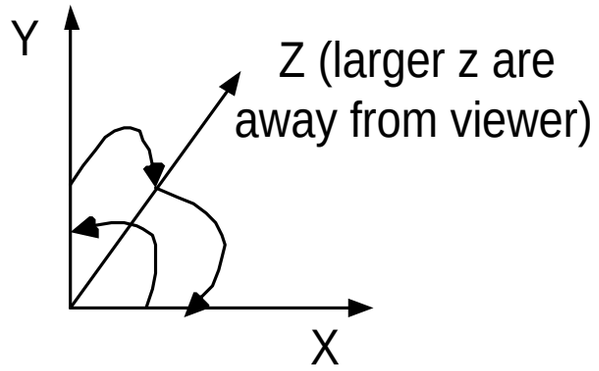
- Basics of 3D geometry
- Basic 3D Transformations
- Composite Transformations

Orientation

- Right-hand orientation
- Left-hand orientation



Right-handed orentation



Left-handed orentation

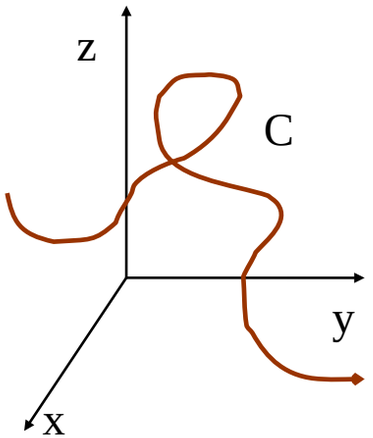
3D Equation of Curve & Line

Curve

Non-Parametric form $C: y = f(x), z = g(x)$

Parametric form

$$C: \quad \begin{aligned} x &= f(t) \\ y &= g(t) \quad a \le t \le b \\ z &= h(t) \end{aligned}$$

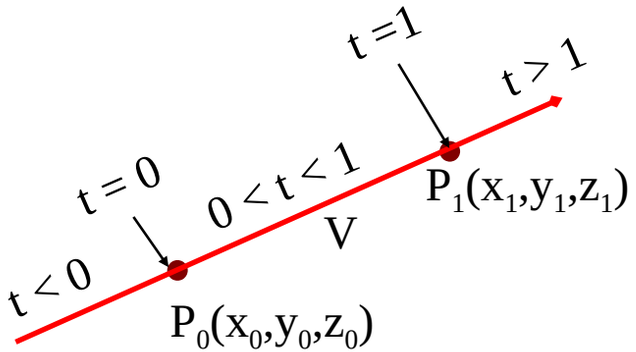


Line

$$\bar{V} = \overline{P_0 P_1} = P_1 - P_0$$

$$L: \quad \begin{aligned} x &= x_0 + (x_1 - x_0) \times t \\ y &= y_0 + (y_1 - y_0) \times t \quad 0 \le t \le 1 \\ z &= z_0 + (z_1 - z_0) \times t \end{aligned}$$

$$L = P_0 + t(P_1 - P_0) = P_0 + t\bar{V}$$

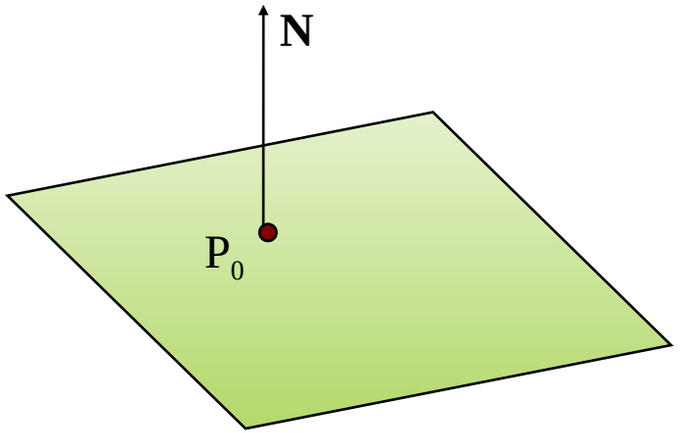


3D Equation of Surface & Plane

- Parametric equations of Surface & Plane
- Surface

$$S: \quad \begin{aligned} x &= f(s,t) \\ y &= g(s,t) \\ z &= h(s,t) \end{aligned} \qquad \begin{aligned} a \le s \le b \\ c \le t \le d \end{aligned}$$

- Plane : with Normal, **N**



$$Ax + By + Cz + D = 0$$

$$\overline{N} = A\hat{i} + B\hat{j} + C\hat{k}$$

3D Plane

- Ways of defining a plane
  1. 3 points P_0, P_1, P_2 on the plane
  2. Plane Normal **N** & P_0 on plane
  3. Plane Normal **N** & a vector **V** on the plane

Plane Passing through P_0, P_1, P_2

$$\overline{N} = \overrightarrow{P_0P_1} \times \overrightarrow{P_0P_2} = A\hat{i} + B\hat{j} + C\hat{k}$$

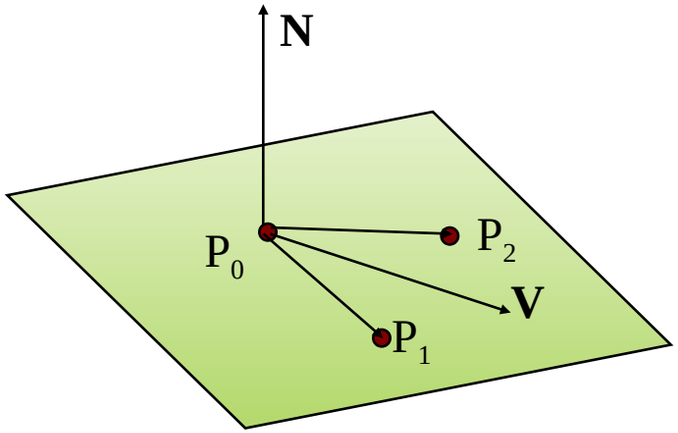
if P(x, y, z) is on the plane

$$\overline{N} \bullet \overrightarrow{P_0P} = 0$$

$$\Rightarrow (A\hat{i} + B\hat{j} + C\hat{k}) \bullet \left[(x - x_0)\hat{i} + (y - y_0)\hat{j} + (z - z_0)\hat{k}\right] = 0$$

$$\Rightarrow Ax + By + Cz + D = 0$$

where $D = -(Ax_0 + By_0 + Cz_0)$



3D Plane

Equation of a plane in Cartesian coordinates

Example

Find a point P_0 and the perpendicular vector $\mathbf{n}$ to the plane $2x + 4y - z = 3$.

Solution: The equation of a plane is $n_x x + n_y y + n_z z = d$.

The components of the normal vector $\mathbf{n}$ are the coefficients that multiply the variables x, y and z. Hence,

$$\mathbf{n} = \langle 2, 4, -1 \rangle.$$

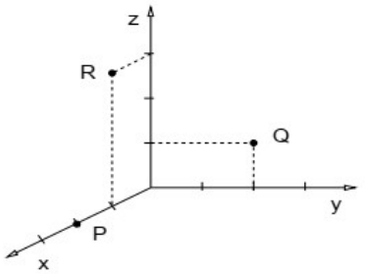
A point P_0 on the plane is simple to find. Just look for the intersection of the plane with one of the coordinate axis.

For example: set $y = 0$, $z = 0$ and find x from the equation of the plane: $2x = 3$, that is $x = 3/2$. Therefore, $P_0 = (3/2, 0, 0)$. ◁

Equation of a plane in Cartesian coordinates

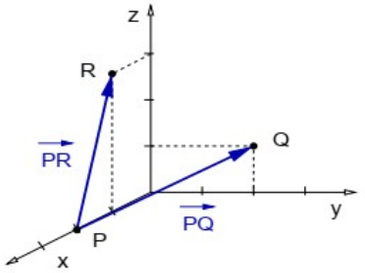
Example

Find the equation of the plane containing the points $P = (2, 0, 0)$, $Q = (0, 2, 1)$, $R = (1, 0, 3)$.



Solution:

Find two tangent vectors to the plane, for example, $\overrightarrow{PQ} = \langle -2, 2, 1 \rangle$ and $\overrightarrow{PR} = \langle -1, 0, 3 \rangle$.

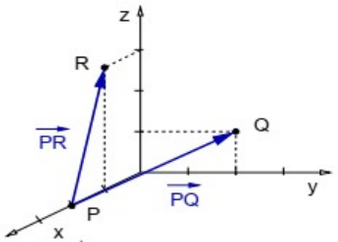


3D Plane

Equation of a plane in Cartesian coordinates

Solution:

Find two tangent vectors to the plane, for example, $\overrightarrow{PQ} = \langle -2, 2, 1 \rangle$ and $\overrightarrow{PR} = \langle -1, 0, 3 \rangle$.



Find a vector $\mathbf{n}$ perpendicular to both $\overrightarrow{PQ}$ and $\overrightarrow{PR}$.
One way is using the cross product: $\mathbf{n} = \overrightarrow{PQ} \times \overrightarrow{PR}$. That is,

$$\mathbf{n} = \overrightarrow{PQ} \times \overrightarrow{PR} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ -2 & 2 & 1 \\ -1 & 0 & 3 \end{vmatrix} = (6-0)\,\mathbf{i} - (-6+1)\,\mathbf{j} + (0+2)\,\mathbf{k}.$$

The result is: $\mathbf{n} = \langle 6, 5, 2 \rangle$. Choose any point on the plane, say $P = (2, 0, 0)$. Then, the equation of the plane is:

$$6(x-2) + 5y + 2z = 0.$$

◁

Vectors in 3D

Have length and direction

$V = [a, b, c]$

Length is given by the Euclidean Norm

$|V| = \sqrt{(a^2 + b^2 + c^2)}$

Dot Product

$V \bullet U = [x_v, y_v, z_v] \bullet [x_u, y_u, z_u]$

$= x_v * x_u + y_v * y_u + z_v * z_u$

$= |V| \, |U| \cos ß$

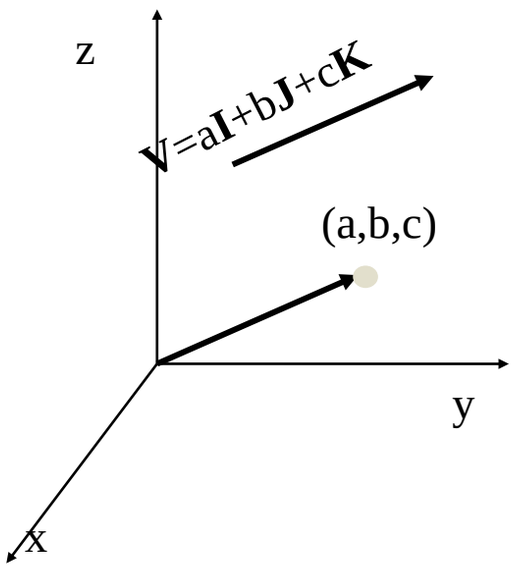
Cross Product

$V \times U$

$= [y_v * z_u - z_v y_u , -x_v * z_u + z_v * x_u , x_v * y_u - y_v * x_u]$

$= \eta \, |V| \, |U| \sin ß$

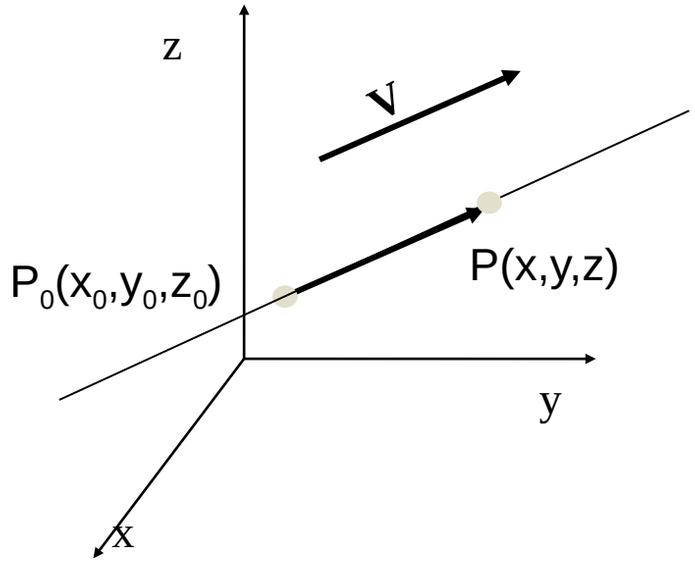
$V \times U = - (U \times V)$



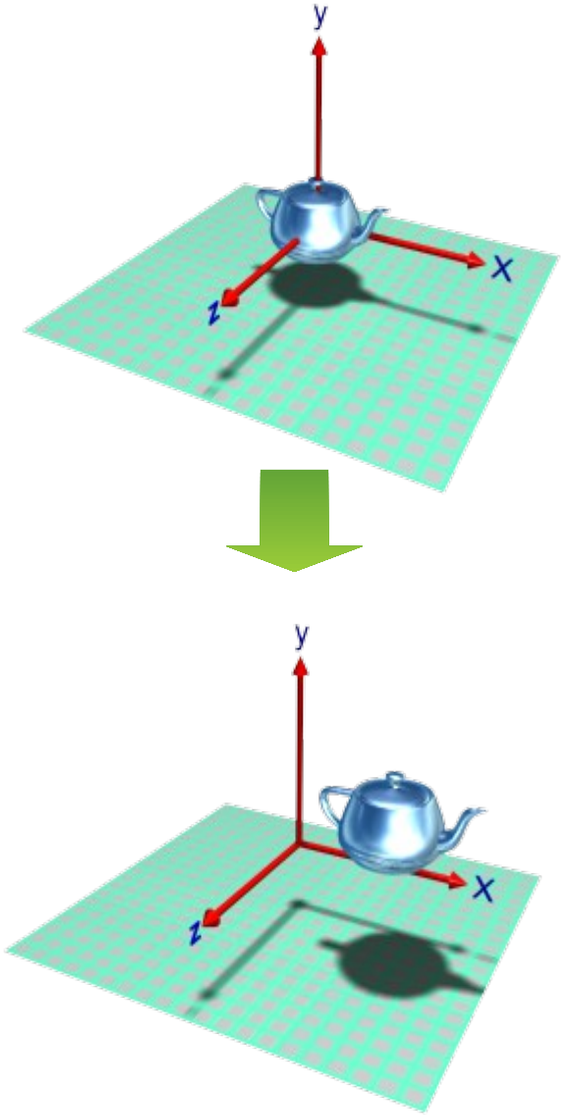
Vector Equation of Line

- A line is defined by
  - A direction vector V
  - A point $P_0(x_0,y_0,z_0)$

$$L = P_0 + t(P_1 - P_0) = P_0 + t\overline{V}$$
$$= (1 - t)P_0 + tP_1$$



Translation – 3D



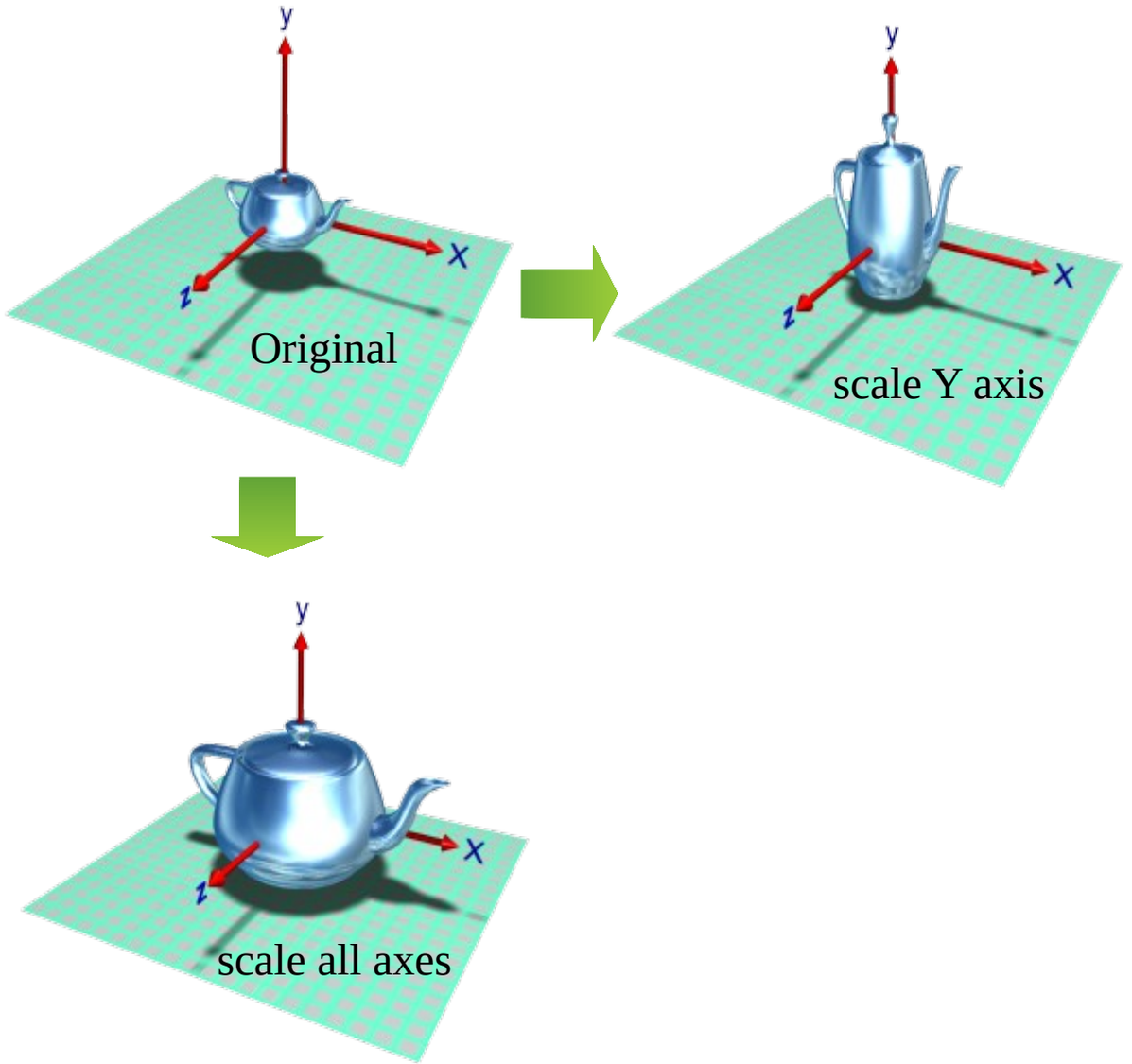
$$x' = x + d_x$$
$$y' = y + d_y$$
$$z' = z + d_z$$

$$\begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x + d_x \\ y + d_y \\ z + d_z \\ 1 \end{bmatrix}$$

$$\Downarrow \qquad \Downarrow \qquad \Downarrow$$

$$T(d_x, d_y, d_z) \quad * \quad P \quad = \quad P'$$

Scaling – 3D



$$x' = s_x * x$$

$$y' = s_y * y$$

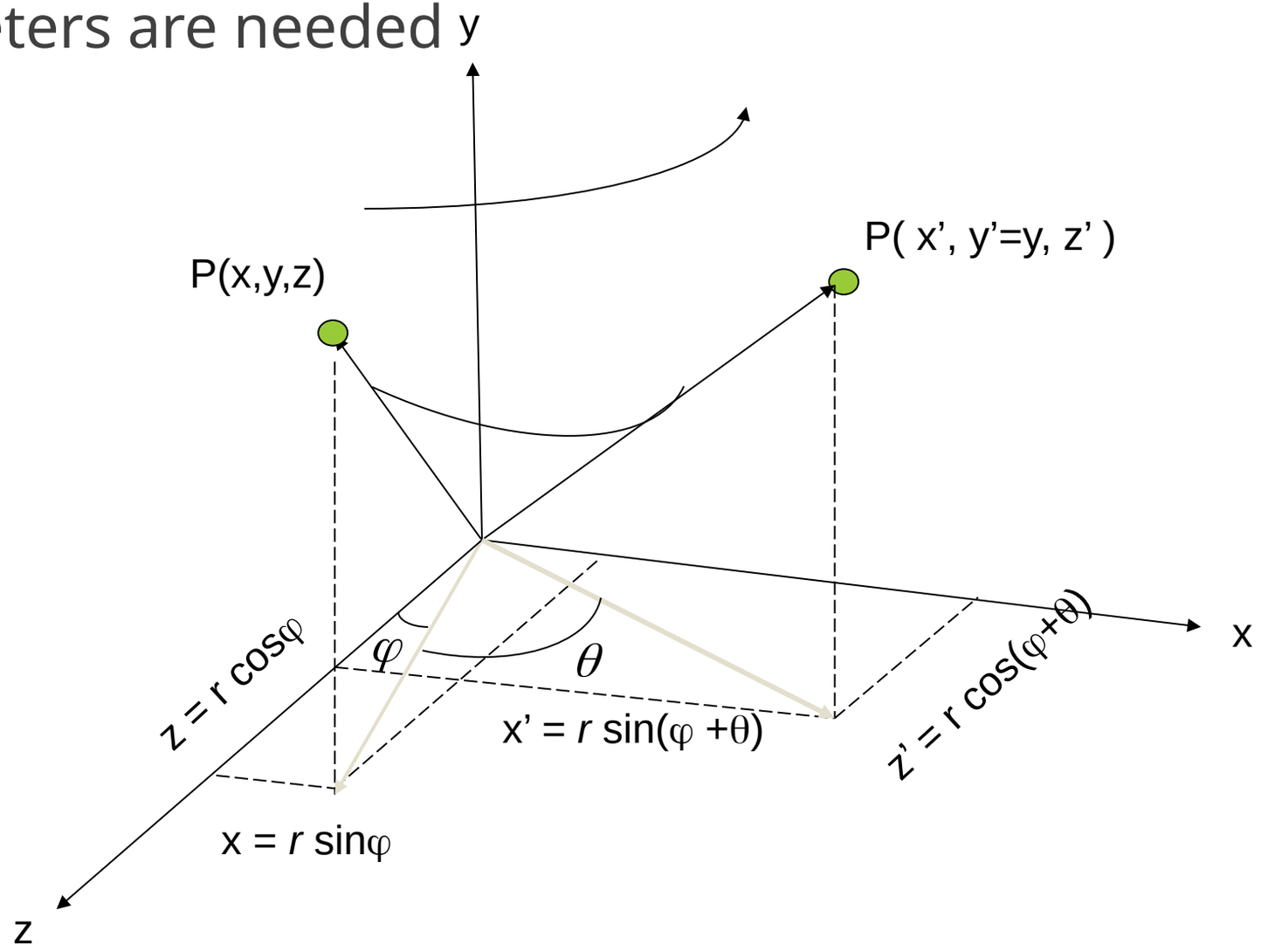
$$z' = s_z * z$$

$$S(s_x, s_y, s_z) \quad * \quad P \quad = \quad P'$$

$$\begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x * s_x \\ y * s_y \\ z * s_z \\ 1 \end{bmatrix}$$

Rotation in 3D

- For 3D-Rotation 2 parameters are needed
  - Angle of rotation
  - Axis of rotation
- Rotation about y axis



Rotation in 3D

$$x = r\sin\varphi$$

$$z = r\cos\varphi$$

$$x' = r\sin(\varphi+\theta) = r\sin\varphi\cos\theta + r\cos\varphi\sin\theta$$

$$= x\cos\theta + z\sin\theta$$

$$y' = y$$

$$z' = r\cos(\varphi+\theta) = r\cos\varphi\cos\theta - r\sin\varphi\sin\theta$$

$$= z\cos\theta - x\sin\theta$$

$$= -x\sin\theta + z\cos\theta$$

Rotation in 3D

About y-axis

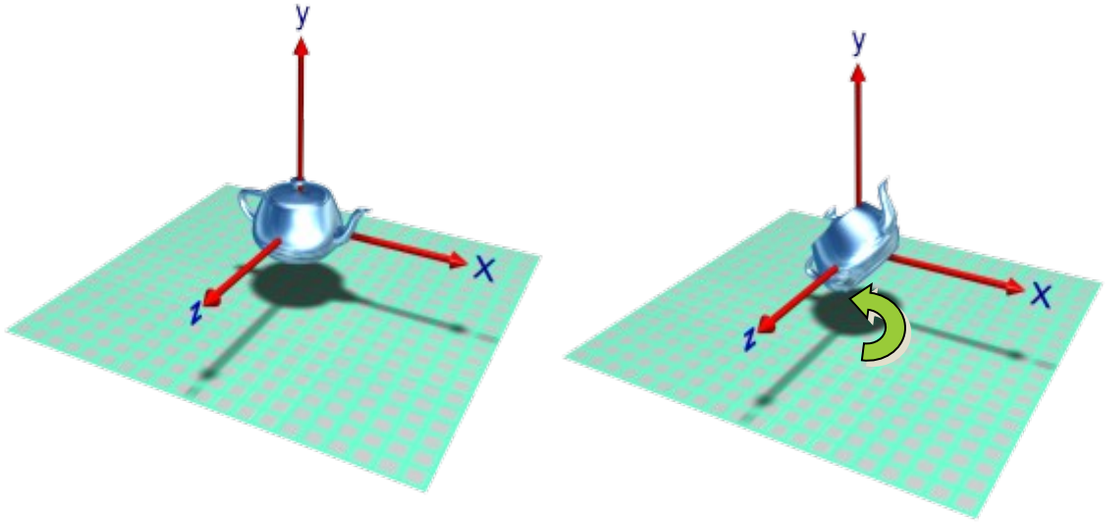
$$R_{\theta,j} \quad * \quad P \quad = \quad P'$$

$$\Downarrow \qquad\qquad \Downarrow \qquad\qquad \Downarrow$$

$$\begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x*\cos\theta + z*\sin\theta \\ y \\ -x*\sin\theta + z*\cos\theta \\ 1 \end{bmatrix}$$

Rotation – 3D

- Rotation about z-axis:



$$R_{\theta,k} \quad * \quad P \quad = \quad P'$$

$$\Downarrow \qquad\qquad \Downarrow \qquad\qquad \Downarrow$$

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x*\cos\theta - y*\sin\theta \\ x*\sin\theta + y*\cos\theta \\ z \\ 1 \end{bmatrix}$$

Rotation about X-axis

About x-axis

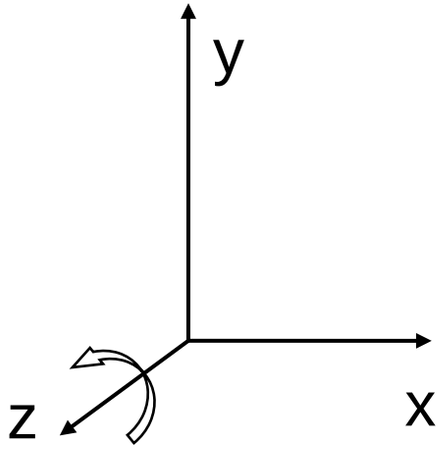
$$R_{\theta,i} \quad * \quad P \quad = \quad P'$$

$$\Downarrow \qquad\qquad \Downarrow \qquad\qquad \Downarrow$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y*\cos\theta - z*\sin\theta \\ y*\sin\theta + z*\cos\theta \\ 1 \end{bmatrix}$$

Coordinate-Axes Rotations

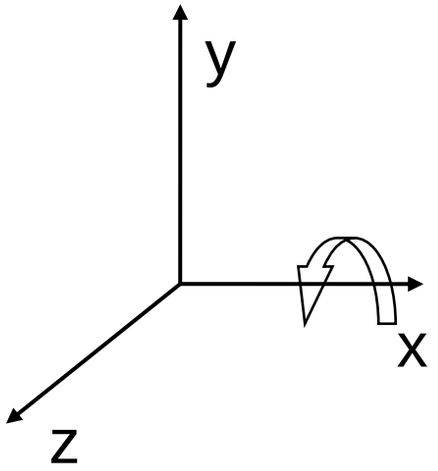
- Z-Axis Rotation



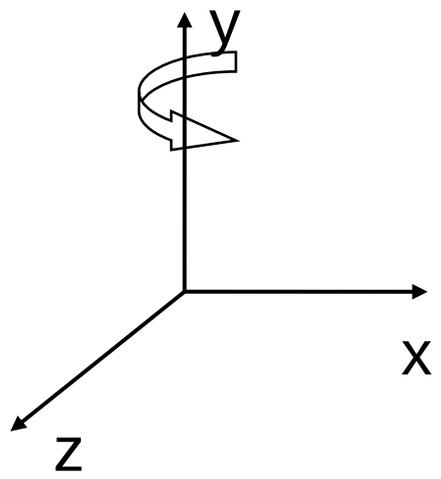
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- X-Axis Rotation

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



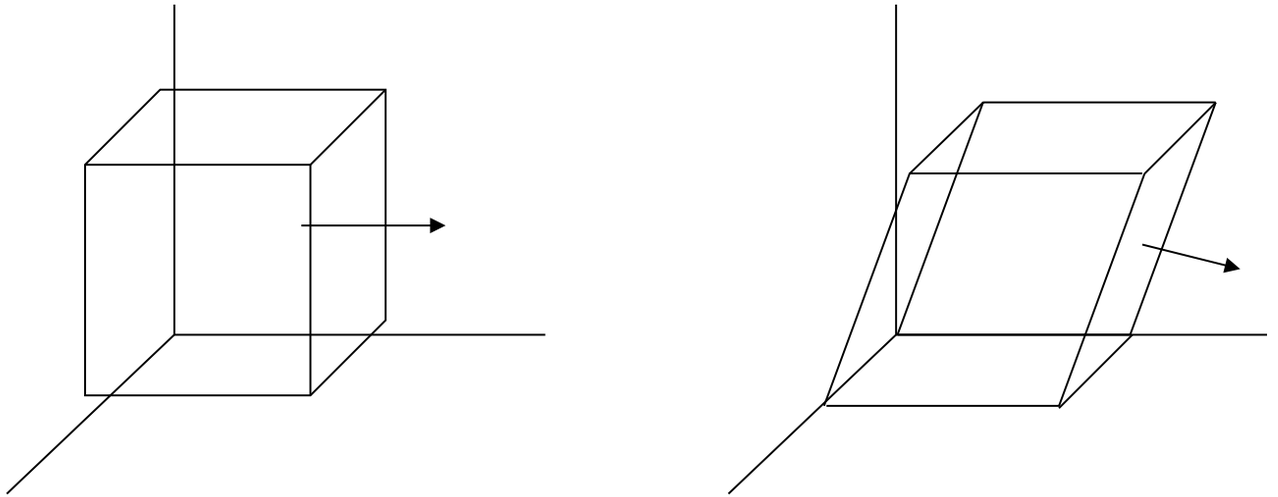
- Y-Axis Rotation



$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

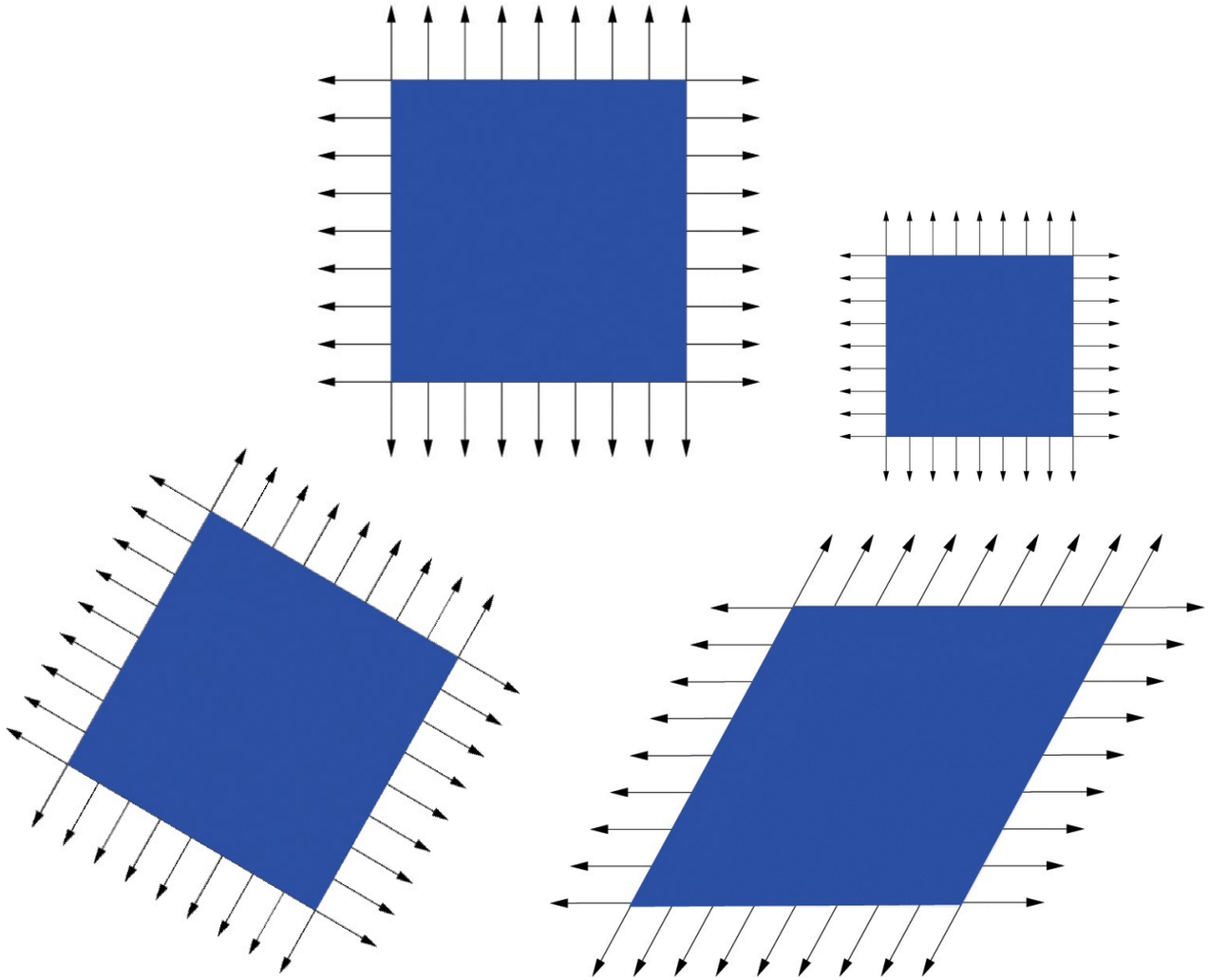
Object Transformation

- **Line**: Can be transformed by transforming the end points
- **Plane**:(**described by 3-points**) Can be transformed by transforming the 3-points
- **Plane**:(**described by a point and Normal**) Point is transformed as usual. Special treatment is needed for transforming Normal



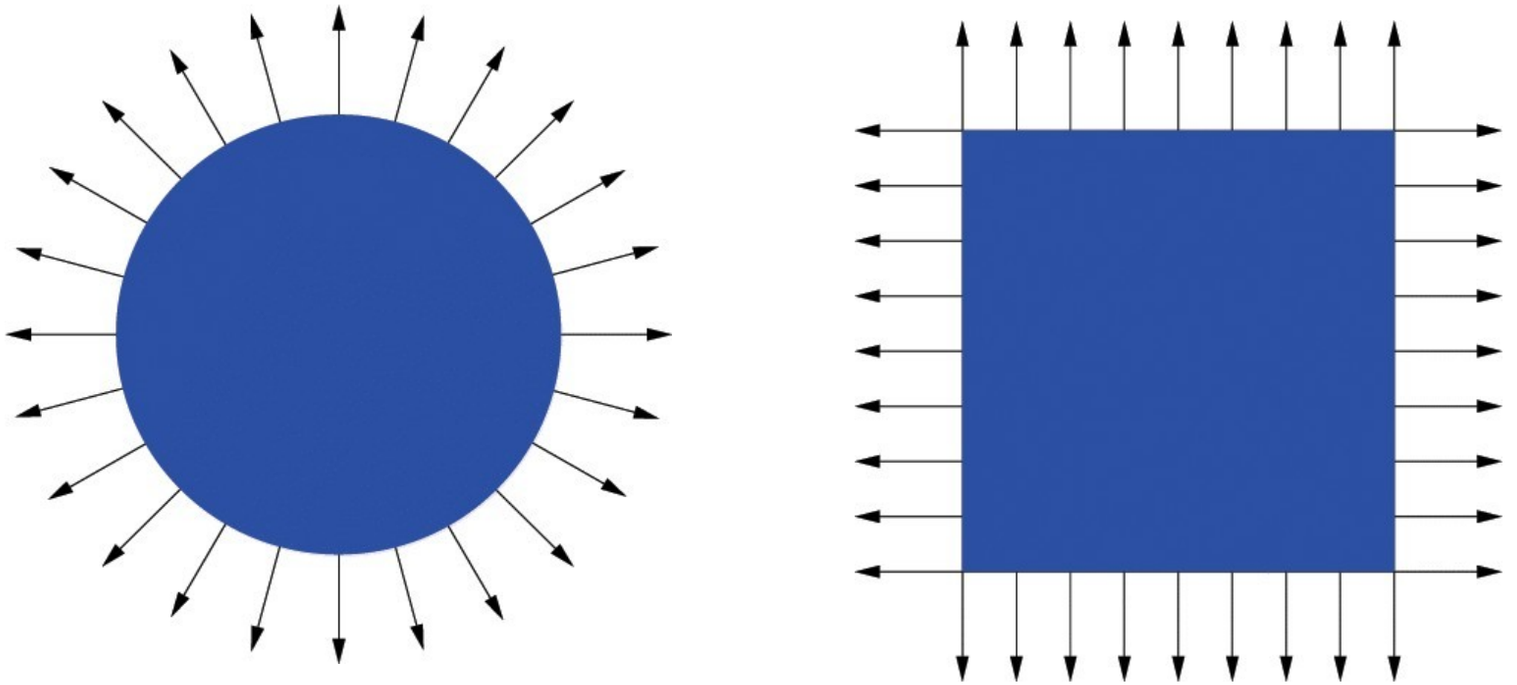
Transform Normal like Object?

- translation?
- rotation?
- isotropic scale?
- scale?
- reflection?
- shear?
- perspective?



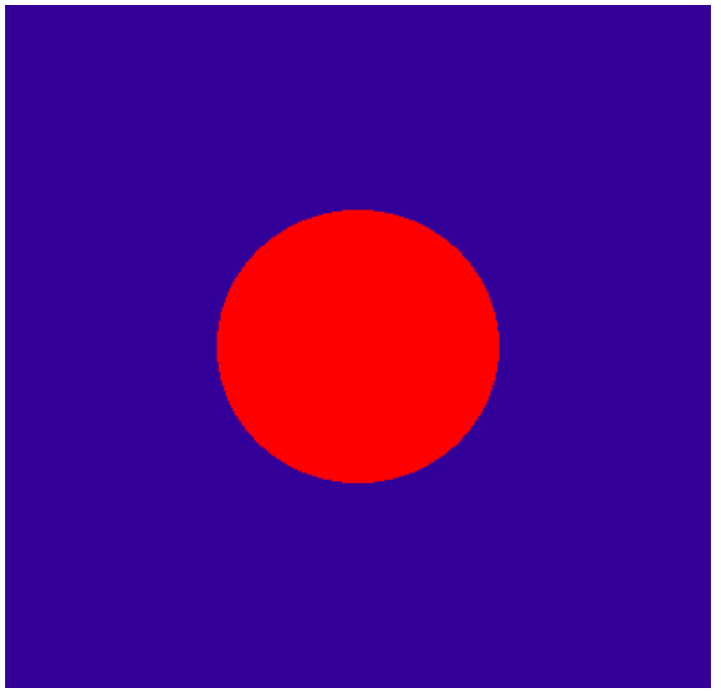
Normal

- Surface Normal: unit vector that is locally perpendicular to the surface

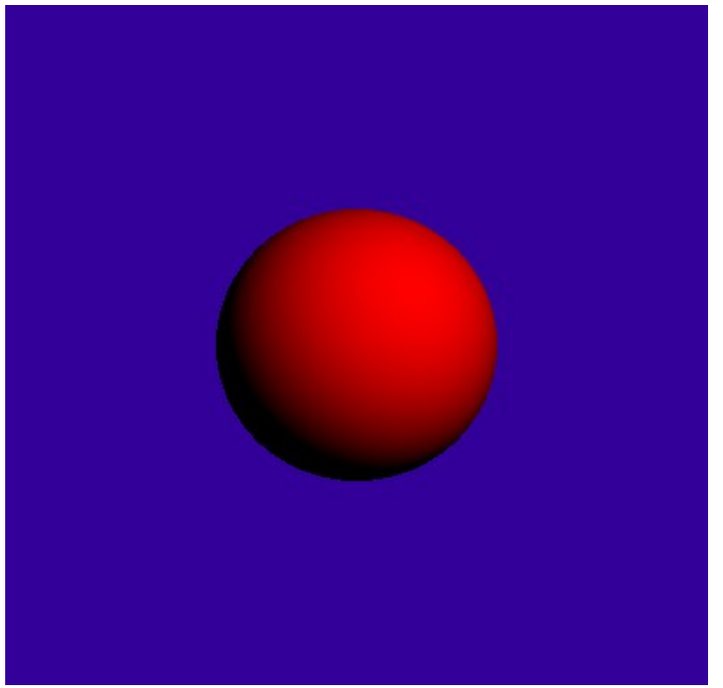


Why is the Normal important?

- It's used for shading — makes things look 3D!

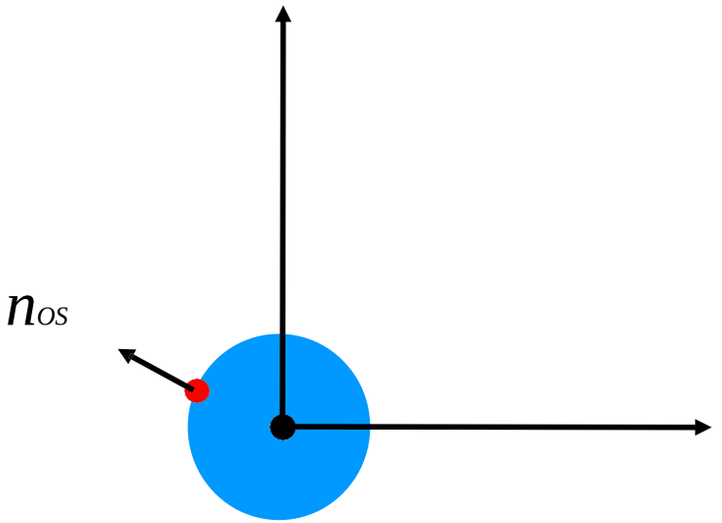


object color only

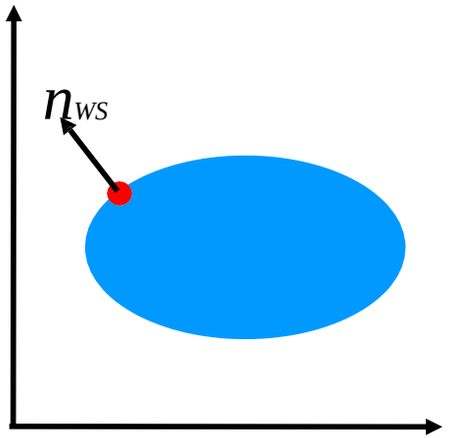


Diffuse Shading

How do we transform normals?



Object Space



World Space

Transforming Normal Vector

Let the plan be represented by point P and normal N

$$\mathrm{N} = \begin{bmatrix} A & B & C & D \end{bmatrix}^T \quad and \quad P = \begin{bmatrix} x & y & z & 1 \end{bmatrix}^T$$

such that $N^T * P = Ax + By + Cz + D = 0$

Let M be the transformation to be applyed on P and

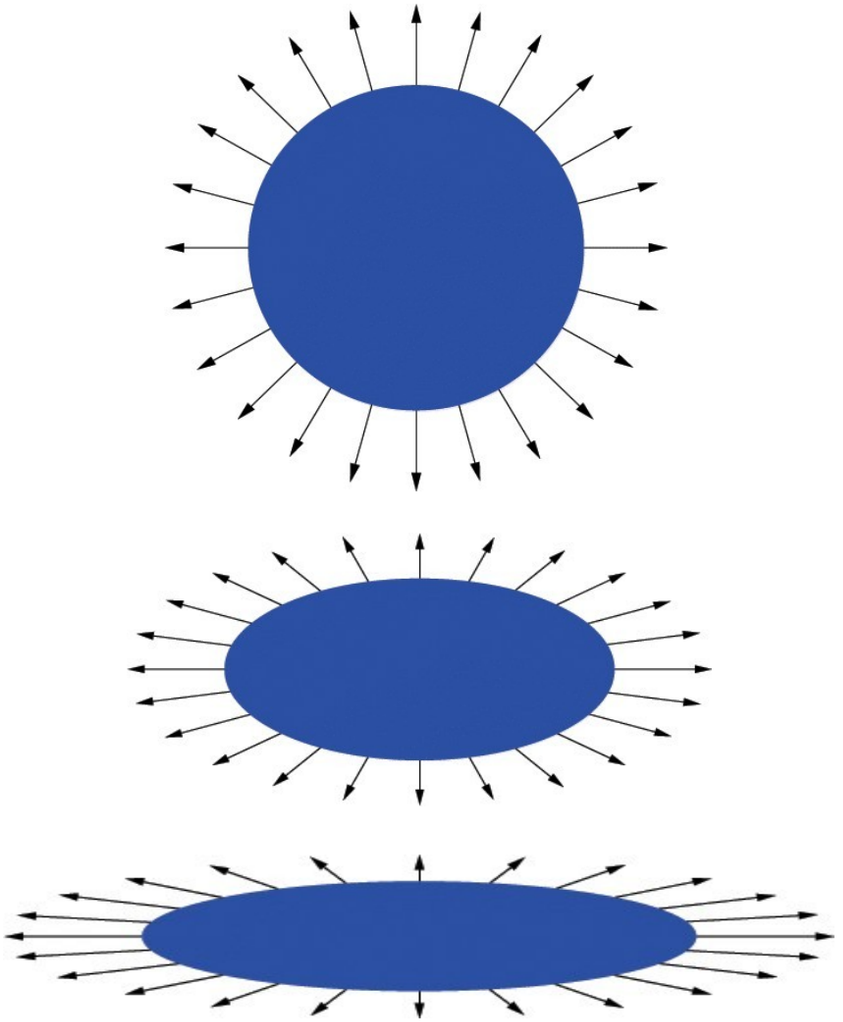
Q be a transformation such that

$$\left.\begin{aligned} (Q * N)^T * M * P = 0 \\ N^T * Q^T * M * P = 0 \end{aligned}\right\rangle \text{ which leads to } \left\| \begin{aligned} Q^{\mathrm{T}} * M = I \\ \therefore Q^{\mathrm{T}} = M^{-1} \end{aligned}\right.$$

$$\therefore N' = (M^{-1})^T * N$$

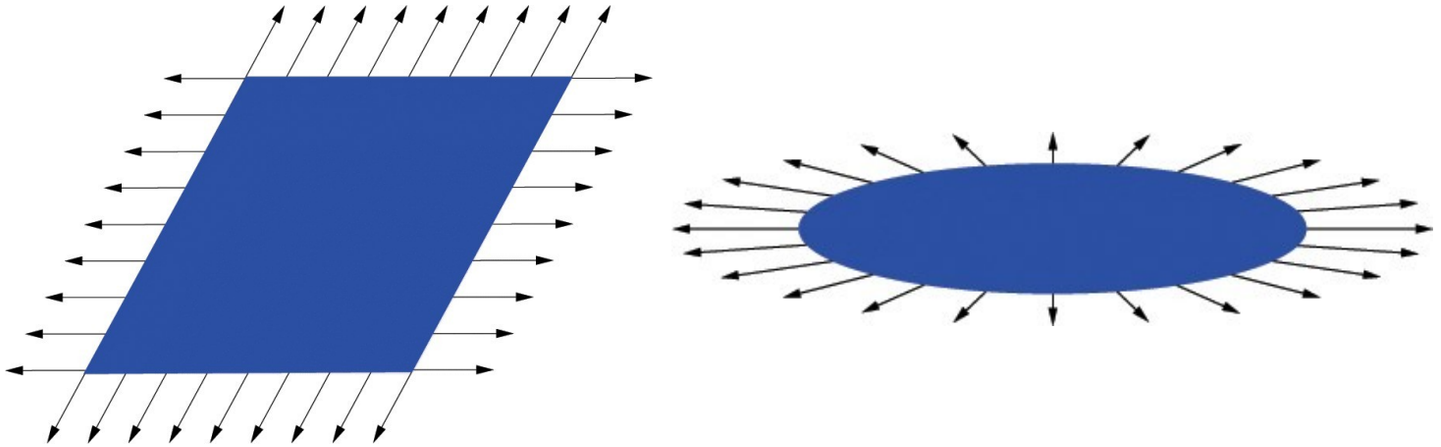
Transform Normal like Object?

- translation?
- rotation?
- isotropic scale?
- scale?
- reflection?
- shear?
- perspective?

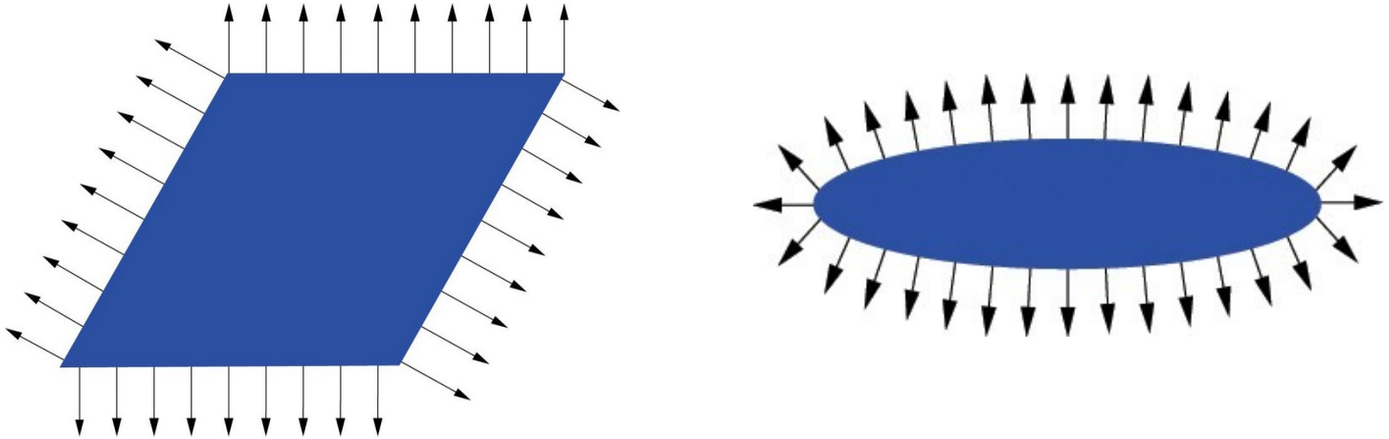


Transformation for shear and scale

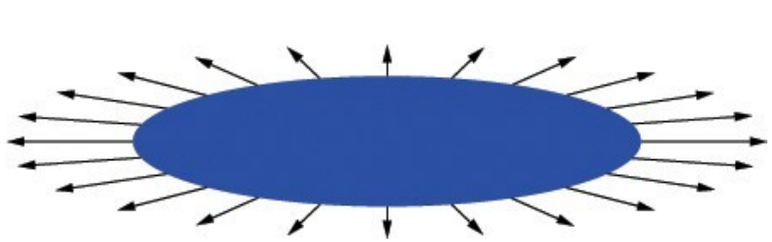
Incorrect Normal Transformation

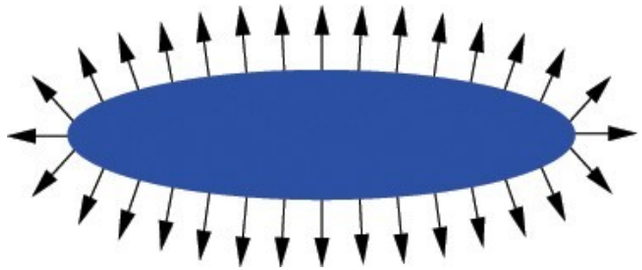


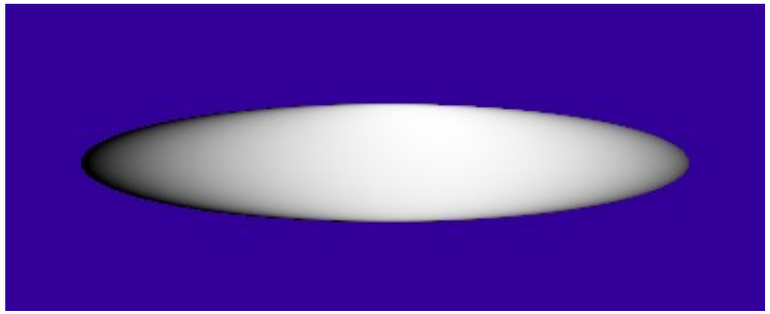
Correct Normal Transformation

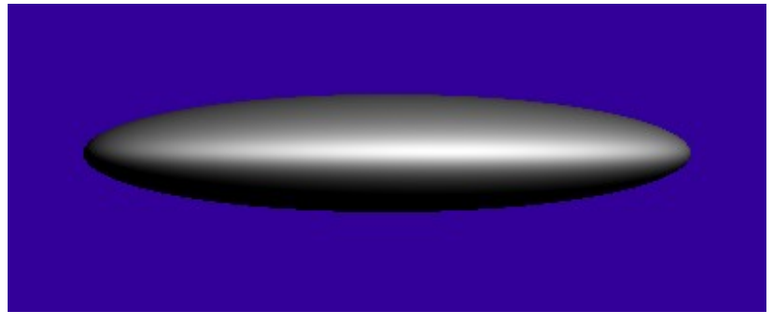


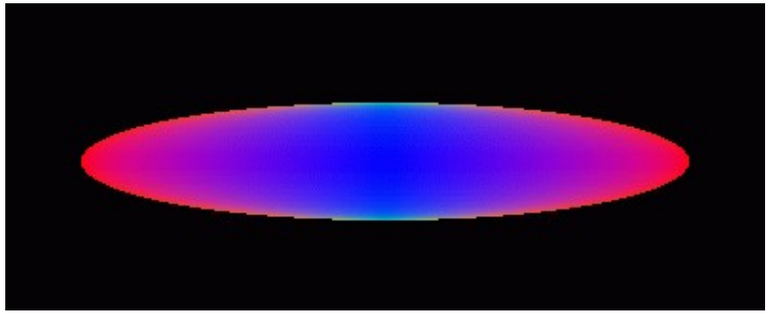
More Normal Visualizations

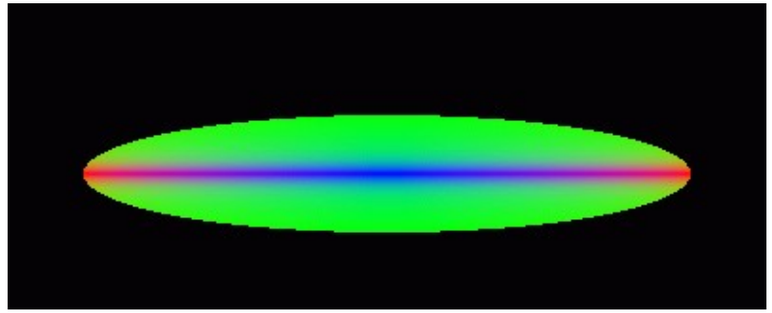












Incorrect Normal Transformation

Correct Normal Transformation

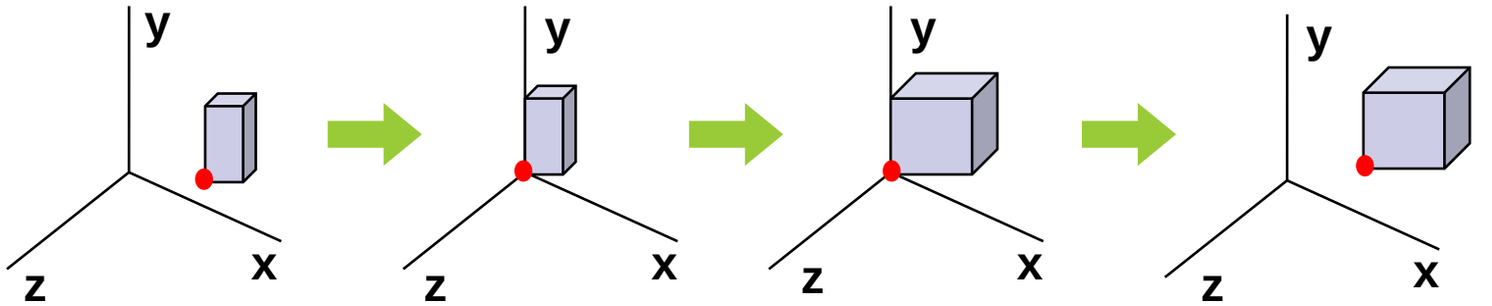
Composite Transformations – 3D

- **Some of the composite transformations to be studied are:**

  - Relative Scaling
  - $A_{\mathbf{V},\mathbf{N}}$ = aligning a vector **V** with a vector **N**
  - $R_{\theta,L}$ = rotation about an axis L(**V**, P)
  - $S_{sx,sy,P}$= scaling w.r.t. point P

Relative Scaling

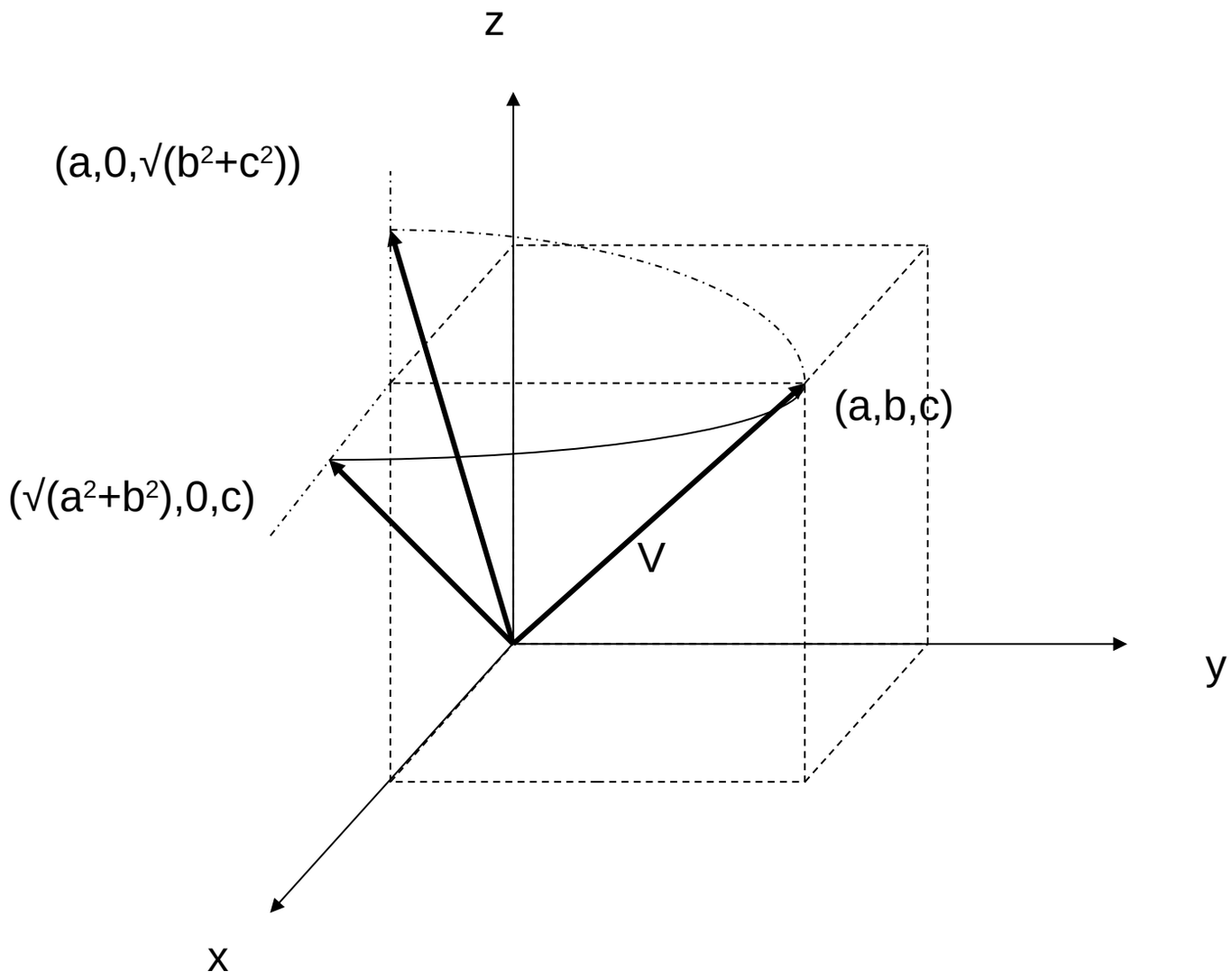
- Scaling with a Selected Fixed Position



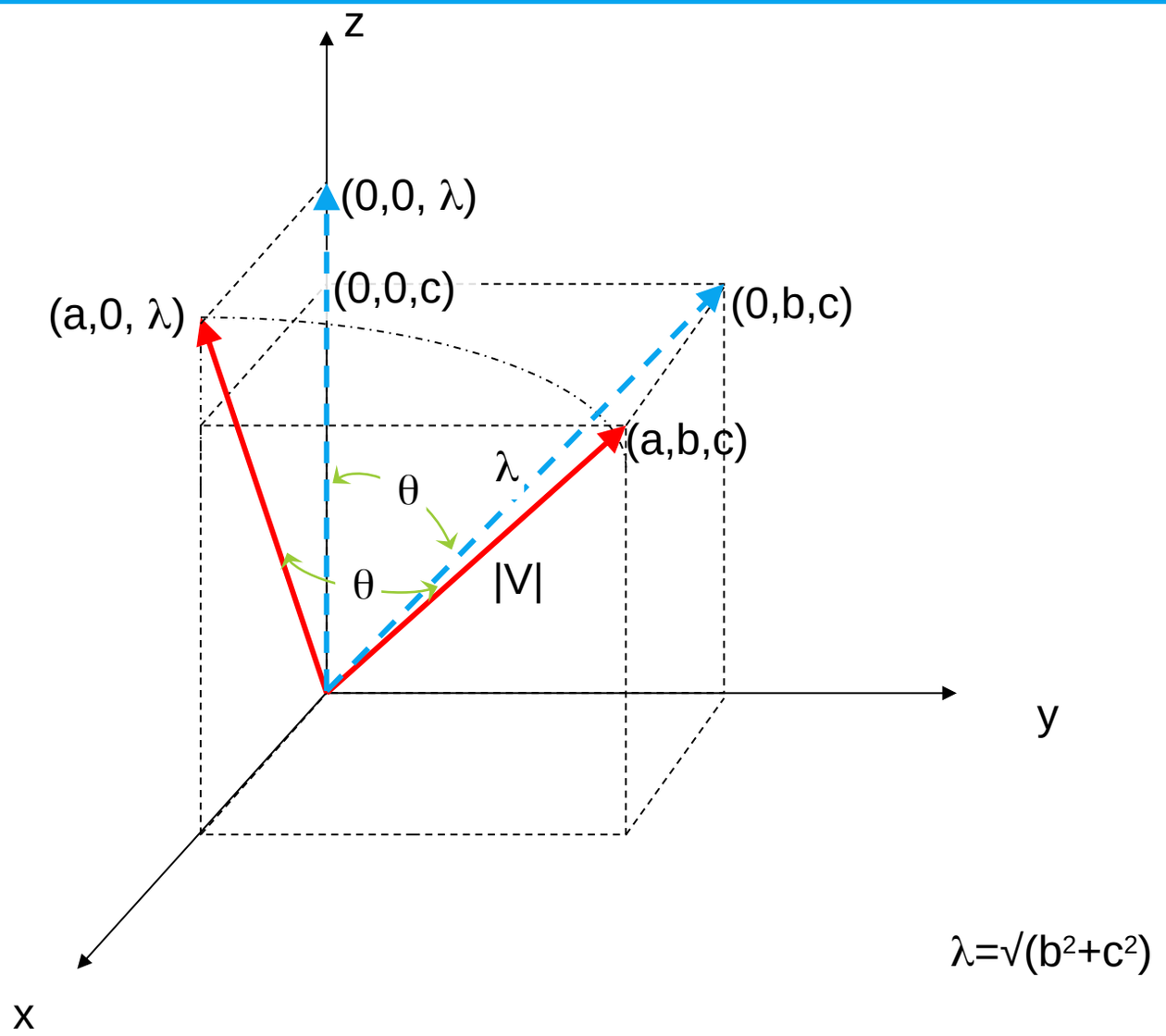
Original position | Translate | Scaling | Inverse Translate

$$T(x_f, y_f, z_f) \cdot S(s_x, s_y, s_z) \cdot T(-x_f, -y_f, -z_f) = \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & x_f \\ 0 & 1 & 0 & y_f \\ 0 & 0 & 1 & z_f \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_f \\ 0 & 1 & 0 & -y_f \\ 0 & 0 & 1 & -z_f \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Av: Aligning vector V with k



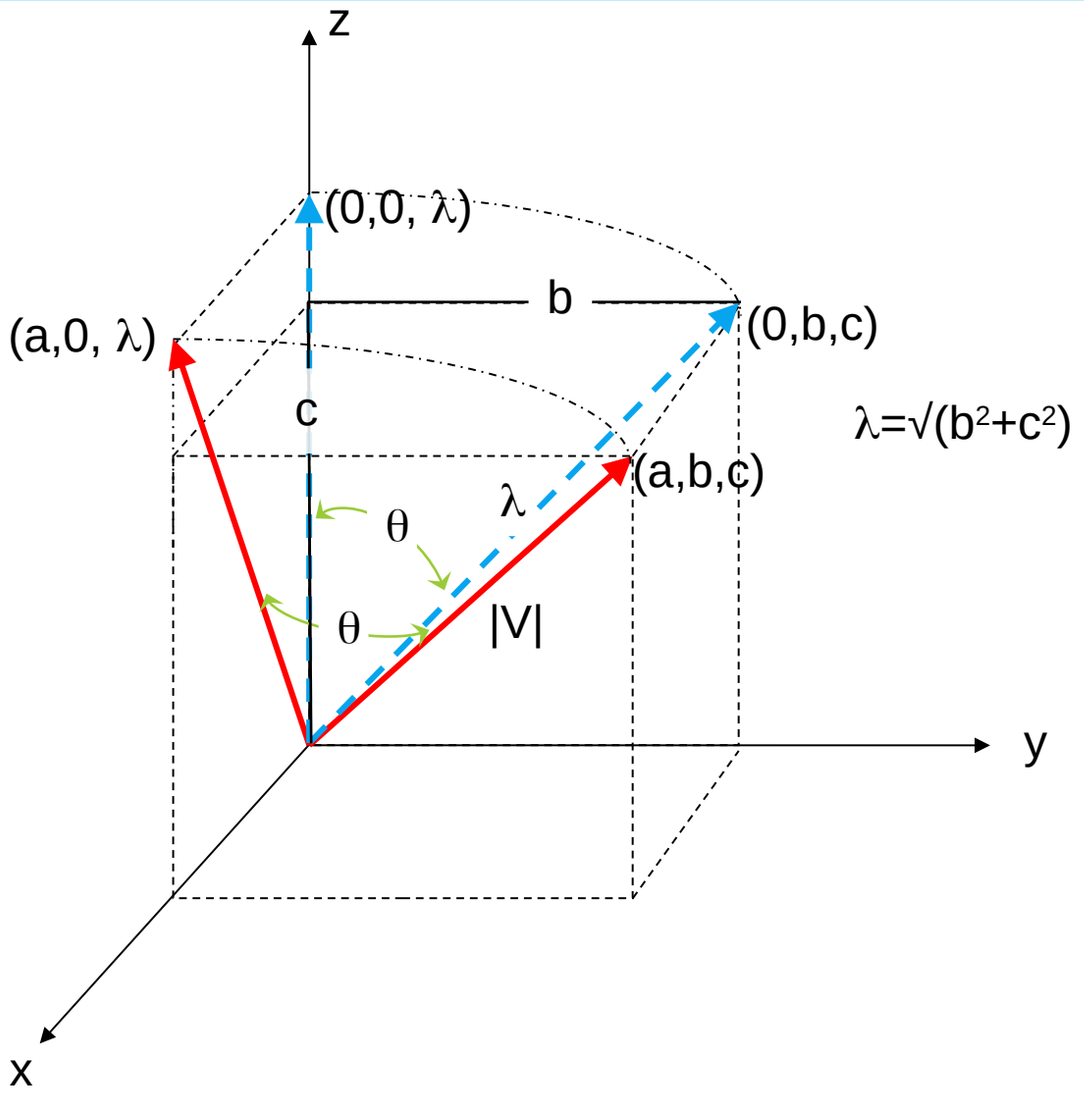
Av: Aligning vector V with k



A_V : aligning vector V with k

Step 1: Rotate about x-axis by θ

$$\left.\begin{aligned}\sin\theta &= \frac{b}{\lambda}\\ \cos\theta &= \frac{c}{\lambda}\end{aligned}\right\} \quad \lambda = \sqrt{b^2 + c^2}$$



$$A_v \qquad R_{\theta,i}$$

$$=$$

A_V : aligning vector V with k

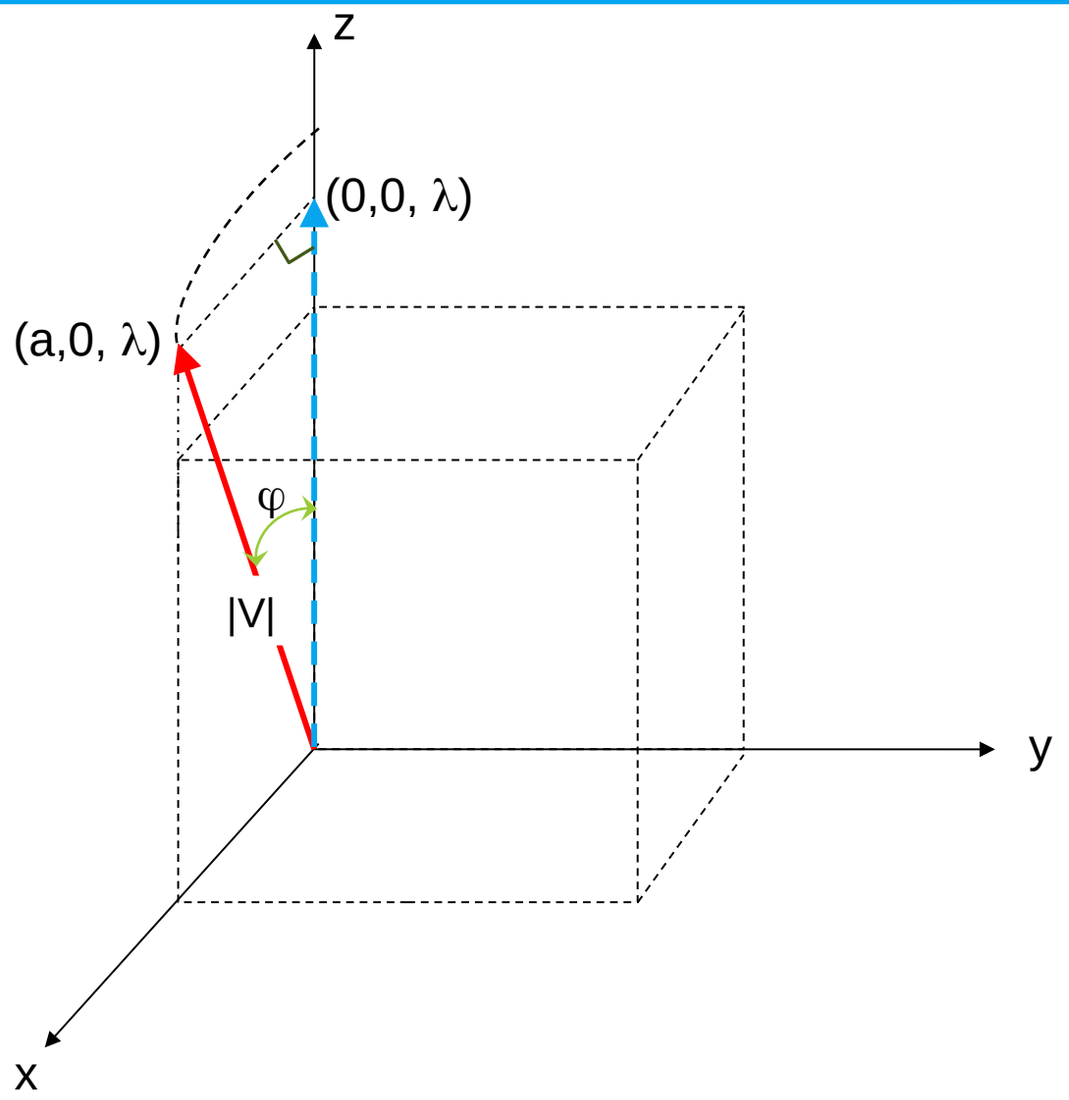
Step 1: Rotate V about x-axis by θ

$$\left.\begin{aligned}\sin\theta &= \frac{b}{\lambda}\\ \cos\theta &= \frac{c}{\lambda}\end{aligned}\right\} \quad \lambda = \sqrt{b^2 + c^2}$$

Step 2: Rotate V about y-axis by -φ

$$\left.\begin{aligned}\sin(-\varphi) &= \frac{-a}{|V|}\\ \cos(-\varphi) &= \frac{\lambda}{|V|}\end{aligned}\right\} \quad |V| = \sqrt{a^2 + b^2 + c^2}$$

$$A_v = R_{-\varphi,j} * R_{\theta,i}$$



Av: Alignment transformation

$$A_v = \begin{bmatrix} \cos(-\phi) & 0 & \sin(-\phi) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(-\phi) & 0 & \cos(-\phi) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_V = \begin{bmatrix} \frac{\lambda}{|V|} & 0 & \frac{-a}{|V|} & 0 \\ 0 & 1 & 0 & 0 \\ \frac{a}{|V|} & 0 & \frac{\lambda}{|V|} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{c}{\lambda} & \frac{-b}{\lambda} & 0 \\ 0 & \frac{b}{\lambda} & \frac{c}{\lambda} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_V = \begin{bmatrix} \frac{\lambda}{|V|} & \frac{-ab}{\lambda|V|} & \frac{-ac}{\lambda|V|} & 0 \\ 0 & \frac{c}{\lambda} & \frac{-b}{\lambda} & 0 \\ \frac{a}{|V|} & \frac{b}{|V|} & \frac{c}{|V|} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Av: Alignment transformation

- If we normalize **V = aI + bJ + cK** to a unit vector $v = aI + bJ + cK$ before rotation the matrix become simple enough
- Where a = **a / |V|** b = **b / |V|** c = **c / |V|**

$$A_v = \begin{bmatrix} \lambda & \frac{-ab}{\lambda} & \frac{-ac}{\lambda} & 0 \\ 0 & \frac{c}{\lambda} & \frac{-b}{\lambda} & 0 \\ a & b & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

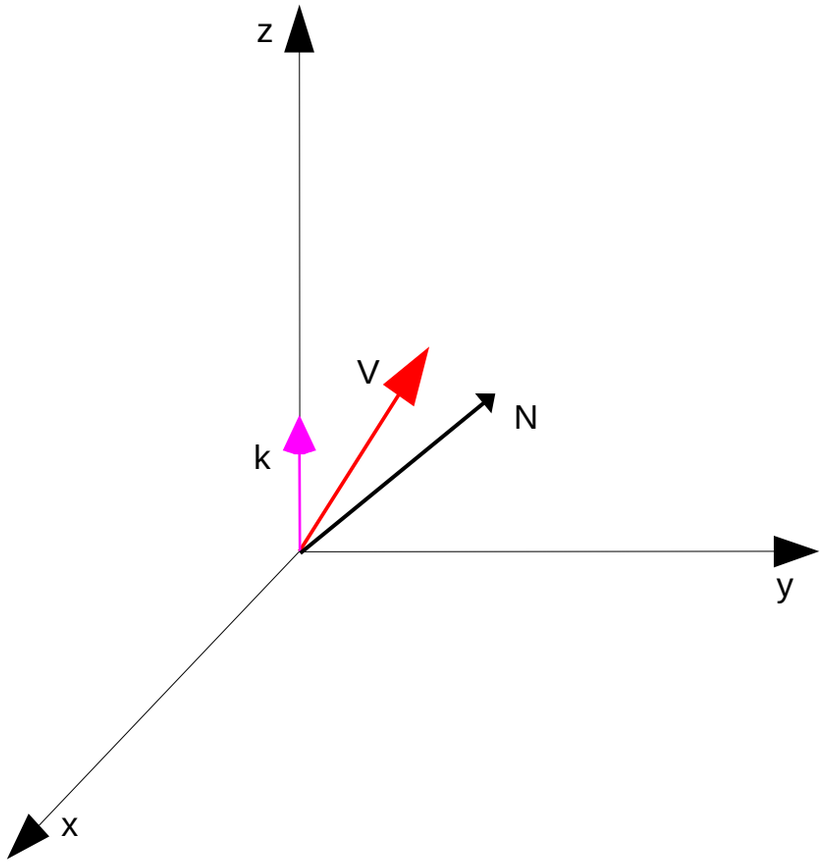
Inverse Alignment

- Aligns the vector k with vector v

$$A_v^{-1} = (R_{-\phi,j} . R_{\theta,i})^{-1} = R_{\theta,i}^{-1} . R_{-\phi,j}^{-1} = R_{-\theta,i} . R_{\phi,j} = A_v^T$$

A_V : aligning vector v (V) with n (N)

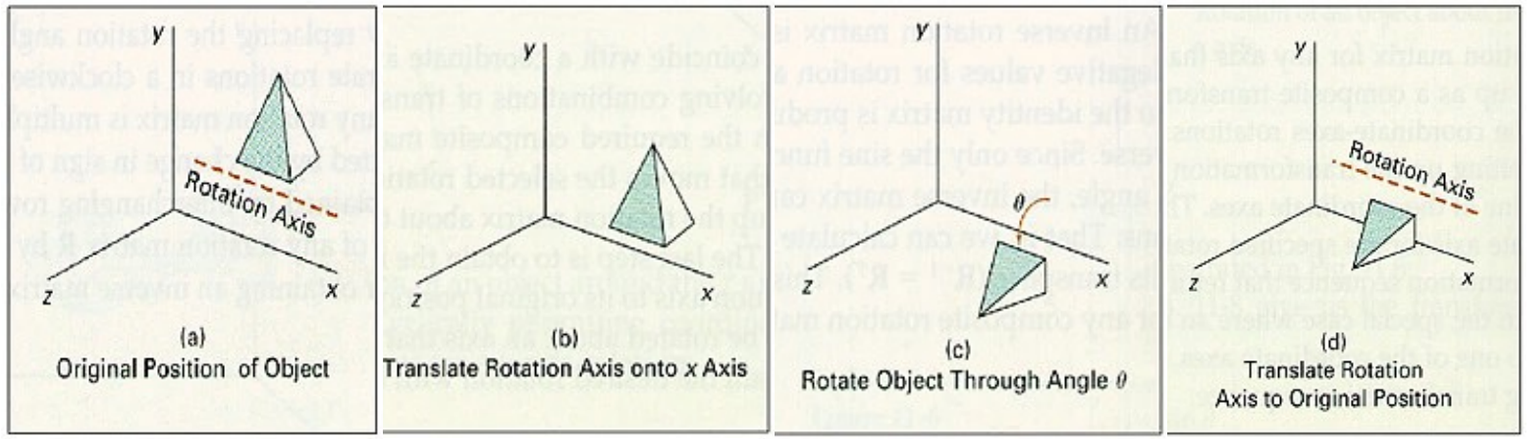
- $A_V^{-1} = A_V^T$
- $A_{V,N} = A_N^{-1} * A_V$



$$A_V = \begin{bmatrix} \frac{\lambda}{|V|} & \frac{-ab}{\lambda|V|} & \frac{-ac}{\lambda|V|} & 0 \\ 0 & \frac{c}{\lambda} & \frac{-b}{\lambda} & 0 \\ \frac{a}{|V|} & \frac{b}{|V|} & \frac{c}{|V|} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

General 3D Rotations

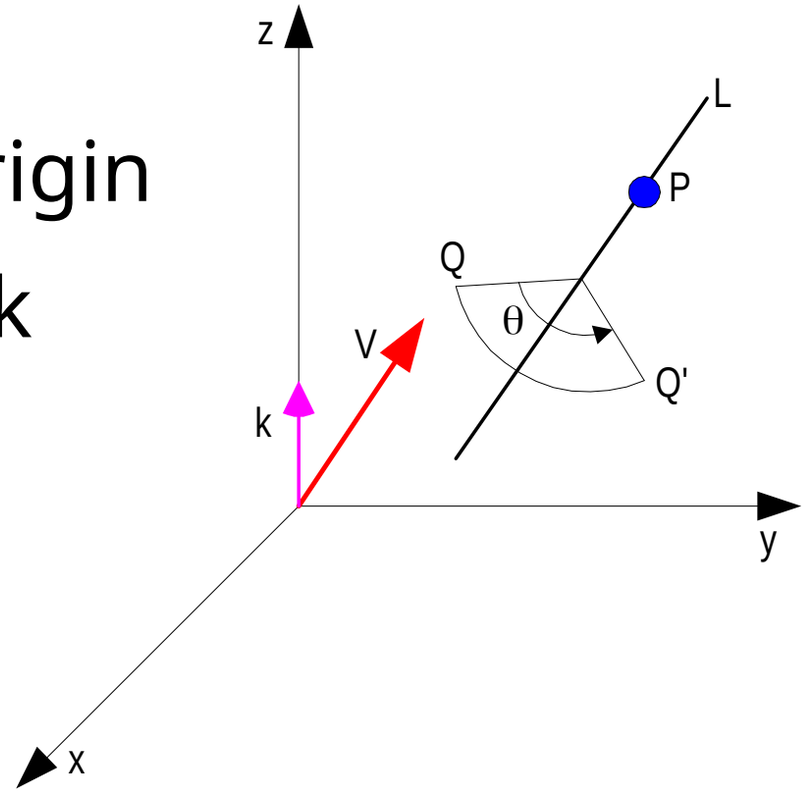
- Rotation about an Axis that is Parallel to One of the Coordinate Axes
  - **Translate** the object so that the rotation axis coincides with the parallel coordinate axis
  - Perform the specified **rotation** about that axis
  - **Translate** the object so that the rotation axis is moved **back** to its original position



$R_{\theta,L}$: Rotation about an axis defined by line L

- Let L be represented by vector V and passing through point P

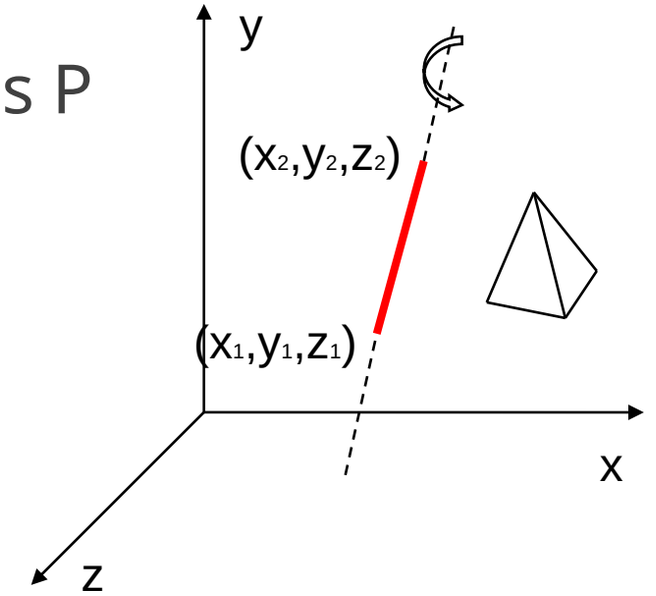
1. Translate P to the origin
2. Align V with vector k
3. Rotate θ° about k
4. Reverse step 2
5. Reverse step 1



$$R_{\theta,L} = T_{-P}^{-1} * A_V^{-1} * R_{\theta,k} * A_V * T_{-P}$$

$R_{\theta,L}$: Rotation about an axis defined by line L

- Let, L be represented by two points P1 and P2
- Then find V = P2 – P1, and use any one point as P

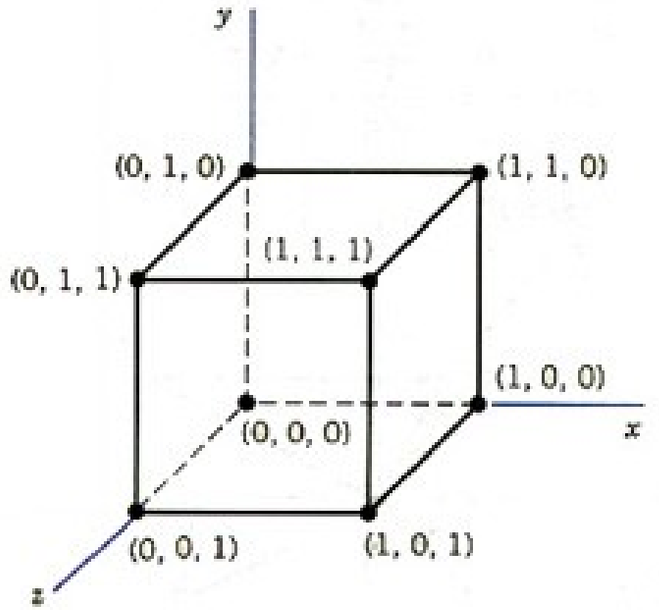


$$R_{\theta,L} = T_{-P}^{-1} * A_V^{-1} * R_{\theta,k} * A_V * T_{-P}$$

$$= T_P * R_{\alpha,i} * R_{\beta,j} * R_{\theta,k} * R_{-\beta,i} * R_{\alpha,i} * T_{-P}$$

Example

Find the new coordinates of a unit cube 90°-rotated about an axis defined by its endpoints A(2,1,0) and B(3,3,1).



A Unit Cube

Example

$$\mathbf{R}(\theta) = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{\sqrt{5}}{5} & \frac{2\sqrt{5}}{5} & 0 \\ 0 & -\frac{2\sqrt{5}}{5} & \frac{\sqrt{5}}{5} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{30}}{6} & 0 & \frac{\sqrt{6}}{6} & 0 \\ 0 & 1 & 0 & 0 \\ -\frac{\sqrt{6}}{6} & 0 & \frac{\sqrt{30}}{6} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} \frac{\sqrt{30}}{6} & 0 & -\frac{\sqrt{6}}{6} & 0 \\ 0 & 1 & 0 & 0 \\ \frac{\sqrt{6}}{6} & 0 & \frac{\sqrt{30}}{6} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{\sqrt{5}}{5} & -\frac{2\sqrt{5}}{5} & 0 \\ 0 & \frac{2\sqrt{5}}{5} & \frac{\sqrt{5}}{5} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -2 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0.166 & -0.075 & 0.983 & 1.742 \\ 0.742 & 0.667 & 0.075 & -1.151 \\ -0.650 & 0.741 & 0.167 & 0.560 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Example

- Multiplying $\mathbf{R}(\theta)$ by the point matrix of the original cube

$$\lfloor \mathbf{P}' \rfloor = \mathbf{R}(\theta) \cdot \lfloor \mathbf{P} \rfloor$$

$$\lfloor \mathbf{P}' \rfloor = \begin{bmatrix} 0.166 & -0.075 & 0.983 & 1.742 \\ 0.742 & 0.667 & 0.075 & -1.151 \\ -0.650 & 0.741 & 0.167 & 0.560 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 2.650 & 1.667 & 1.834 & 2.816 & 2.725 & 1.742 & 1.909 & 2.891 \\ -0.558 & -0.484 & 0.258 & 0.184 & -1.225 & -1.151 & -0.409 & -0.483 \\ 1.467 & 1.301 & 0.650 & 0.817 & 0.726 & 0.560 & -0.091 & 0.076 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Rotation using Quaternion

Quaternions

One Real Part + Three Imaginary Part

$$q = s + ia + jb + kc$$

Properties:

$$i^2 = j^2 = k^2 = -1$$

$$ij = -ji = k$$

$$jk = -kj = i$$

$$ki = -ik = j$$

Addition and Scalar Multiplication:

$$q_1 + q_2 = (s_1 + s_2) + i(a_1 + a_2) + j(b_1 + b_2) + k(c_1 + c_2)$$

$$dq_1 = s_1 d + ia_1 d + jb_1 d + kc_1 d$$

Quaternions

Scalar 's' + Vector "$\mathbf{v} = (x, y, z)$"

$$\mathbf{q} = (s, \mathbf{v}) \quad \text{or} \quad \mathbf{q} = [s + \mathbf{v}]$$

Addition:

$$\mathbf{q_1} \pm \mathbf{q_2} = (s_1 \pm s_2, \mathbf{v}_1 \pm \mathbf{v}_2)$$

$$q^{-1} = \frac{1}{|q|^2}(s, -\mathbf{v})$$

Multiplication

$$\mathbf{q}_1 = [s_1 + \mathbf{v}_1] = [s_1 + x_1\mathbf{i} + y_1\mathbf{j} + z_1\mathbf{k}]$$
$$\mathbf{q}_2 = [s_2 + \mathbf{v}_2] = [s_2 + x_2\mathbf{i} + y_2\mathbf{j} + z_2\mathbf{k}]$$

$$qq^{-1} = q^{-1}q = (1, 0)$$

$$\mathbf{q}_1\mathbf{q}_2 = [(s_1s_2 - x_1x_2 - y_1y_2 - z_1z_2) + (s_1x_2 + s_2x_1 + y_1z_2 - y_2z_1)\mathbf{i}$$
$$+ (s_1y_2 + s_2y_1 + z_1x_2 - z_2x_1)\mathbf{j} + (s_1z_2 + s_2z_1 + x_1y_2 - x_2y_1)\mathbf{k}$$

$$\mathbf{q_1}\mathbf{q_2} = (s_1s_2 - \mathbf{v}_1 \cdot \mathbf{v}_2, s_1\mathbf{v}_2 + s_2\mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2)$$

Quaternions

- Pure quaternion

$$\mathbf{q} = [0 + \mathbf{v}] = [\mathbf{v}]$$

$$\mathbf{q}_1 = [\mathbf{v}_1] = [x_1\mathbf{i} + y_1\mathbf{j} + z_1\mathbf{k}]$$
$$\mathbf{q}_2 = [\mathbf{v}_2] = [x_2\mathbf{i} + y_2\mathbf{j} + z_2\mathbf{k}]$$

$$\mathbf{q_1 q_2} = (\mathbf{v}_1 \times \mathbf{v}_2)$$

- Conjugate

$$\mathbf{q}^* = [s - \mathbf{v}]$$

$$\mathbf{qq}^* = \mathbf{q}^*\mathbf{q} = \mathbf{q}.\mathbf{q} = |\mathbf{q}|^2 = q^2$$

- Magnitude

$$|\mathbf{q}| = \sqrt{s^2 + x^2 + y^2 + z^2}$$

$$|\mathbf{q}|^2 = s^2 + \mathbf{v} \cdot \mathbf{v}$$

- Inverse

$$\mathbf{q}^{-1} = \frac{\mathbf{q}^*}{|\mathbf{q}|^2} = \frac{s - x\mathbf{i} - y\mathbf{j} - z\mathbf{k}}{|\mathbf{q}|^2}$$

$$\mathbf{qq}^{-1} = \mathbf{q}^{-1}\mathbf{q} = (1, \mathbf{0})$$

A unit quaternion has a magnitude equal to 1

3D Rotation with Quaternion

- point P(x, y, z) is represented in quaternion form by

$$\mathbf{P} = [0 + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}]$$

- It can be shown that a position vector P is rotated through an angle θ about an axis using the following operation:

$$\mathbf{P'} = \mathbf{qPq^{-1}}$$

  - where the axis and angle of rotation are encoded within the unit quaternion **q**, and **P'** is the rotated vector.

3D Rotation with Quaternion

- To rotate a point P(x, y, z) an angle θ about any axis **u**
- Use the following steps:

1. Convert the point P(x, y, z) to a pure quaternion $\mathbf{P} = [0 + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}]$ $\mathbf{P} = (0, \mathbf{p})$

2. Define the axis of rotation as a unit vector $\hat{\mathbf{u}} = u_x\mathbf{i} + u_y\mathbf{j} + u_z\mathbf{k}$ $\hat{\mathbf{u}} = \frac{\mathbf{u}}{|\mathbf{u}|}$

2. Define the transforming quaternions $\mathbf{q} = [s + \mathbf{v}]$ and $\mathbf{q}^{-1} = [s - \mathbf{v}]$

$$\mathbf{q} = \left[\cos\frac{\theta}{2} + \sin\frac{\theta}{2}\hat{\mathbf{u}}\right] \qquad \mathbf{q}^{-1} = \left[\cos\frac{\theta}{2} - \sin\frac{\theta}{2}\hat{\mathbf{u}}\right]$$

4. Compute $\mathbf{P}' = \mathbf{qPq}^{-1}$ and unpack (x', y', z')

$$P'(x', y', z') \Leftarrow \mathbf{P}' = [0 + x'\mathbf{i} + y'\mathbf{j} + z'\mathbf{k}]$$

Rotations with Quaternions

- Carry out the quaternion operation $\mathbf{p}' = q\mathbf{P}q^{-1}$

$$\mathbf{qPq}^{-1} = (\mathbf{s},\ \mathbf{v})(\mathbf{0},\ \mathbf{p})(\mathbf{s},\ -\mathbf{v})$$

$$= (s.0 - \mathbf{v}.\mathbf{p},\ s\mathbf{p} + 0\mathbf{v} + \mathbf{v}\times\mathbf{p})(s, -\mathbf{v})$$

$$= (-\mathbf{v}.\mathbf{p},\ s\mathbf{p} + \mathbf{v}\times\mathbf{p})(s, -\mathbf{v})$$

$$= [-s\mathbf{v}.\mathbf{p} + (s\mathbf{p} + \mathbf{v}\times\mathbf{p}).\mathbf{v},\ (\mathbf{v}.\mathbf{p})\mathbf{v} + s(s\mathbf{p} + \mathbf{v}\times\mathbf{p}) - (s\mathbf{p} + \mathbf{v}\times\mathbf{p})\times\mathbf{v}]$$

$$= [(\mathbf{v}\times\mathbf{p}).\mathbf{v},\ (\mathbf{v}.\mathbf{p})\mathbf{v} + s^2\mathbf{p} + s(\mathbf{v}\times\mathbf{p}) - s(\mathbf{p}\times\mathbf{v}) - \mathbf{v}\times\mathbf{p}\times\mathbf{v}]$$

$$= [0,\ (\mathbf{v}.\mathbf{p})\mathbf{v} + s^2\mathbf{p} + 2s(\mathbf{v}\times\mathbf{p}) + \mathbf{v}\times\mathbf{v}\times\mathbf{p}]$$

Since, A.(A×B)=0, AxB is perpendicular to A

$$= (\mathbf{v}.\mathbf{p})\mathbf{v} + s^2\mathbf{p} + 2s(\mathbf{v}\times\mathbf{p}) + (\mathbf{v}.\mathbf{p})\mathbf{v} - \mathbf{v}^2\mathbf{p}$$

since Since A×(B×C)=(A·C)B−(A·B)C

$$\mathbf{qPq}^{-1} = \left(s^2 - \mathbf{v}^2\right)\mathbf{P} + 2s\mathbf{v}\times\mathbf{P} + 2(\mathbf{v}\cdot\mathbf{P})\mathbf{v}$$

Rotations with Quaternions: Matrix form

- Carry out the quaternion operation $\mathbf{P}' = q\mathbf{P}q^{-1}$

$$\mathbf{qPq}^{-1} = (s+\mathbf{v})\mathbf{P}(s-\mathbf{v})$$

$$\mathbf{qPq}^{-1} = (s^2 - \mathbf{v}^2)\mathbf{P} + 2s\mathbf{v}\times\mathbf{P} + 2(\mathbf{v}\cdot\mathbf{P})\mathbf{v}$$

$$= (s + ai + bj + ck)(xi + yj + zk)(s - ai - bj - ck)$$

$$= [x(s^2 + a^2 - b^2 - c^2) + 2y(ab - sc) + 2z(ac + sb)]\mathbf{i} +$$

$$[2x(ab + sc) + y(s^2 - a^2 + b^2 - c^2) + 2z(bc - sa)]\mathbf{j} +$$

$$[2x(ac - sb) + 2y(bc + sa) + z(s^2 - a^2 - b^2 + c^2)]\mathbf{k}$$

$$= \begin{bmatrix} i & j & k \end{bmatrix} \begin{bmatrix} s^2 + a^2 - b^2 - c^2 & 2(ab - sc) & 2(ac + sb) \\ 2(ab + sc) & (s^2 - a^2 + b^2 - c^2) & 2(bc - sa) \\ 2(ac - sb) & 2(bc + sa) & (s^2 - a^2 - b^2 + c^2) \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$\mathbf{P}' = \mathbf{M}_{R(\theta)}.\mathbf{P}$$

Rotations with Quaternions : Matrix form

- Since $|q|^2 = s^2 + a^2 + b^2 + c^2 = 1$

$$\mathbf{M}_{R(\theta)} = \begin{bmatrix} 1 - 2b^2 - 2c^2 & 2ab - 2sc & 2ac + 2sb \\ 2ab + 2sc & 1 - 2a^2 - 2c^2 & 2bc - 2sa \\ 2ac - 2sb & 2bc + 2sa & 1 - 2a^2 - 2b^2 \end{bmatrix}$$

$$= \begin{bmatrix} u_x^2(1-\cos\theta) + \cos\theta & u_x u_y(1-\cos\theta) - u_z \sin\theta & u_x u_z(1-\cos\theta) + u_y \sin\theta \\ u_y u_x(1-\cos\theta) + u_z \sin\theta & u_y^2(1-\cos\theta) + \cos\theta & u_y u_z(1-\cos\theta) - u_x \sin\theta \\ u_z u_x(1-\cos\theta) - u_y \sin\theta & u_z u_y(1-\cos\theta) + u_x \sin\theta & u_z^2(1-\cos\theta) + \cos\theta \end{bmatrix}$$

Putting explicit values of q = (s, a, b, c), and apply following trigonometry

$$\cos^2\frac{\theta}{2} - \sin^2\frac{\theta}{2} = 1 - 2\sin^2\frac{\theta}{2} = \cos\theta, \qquad 2\cos\frac{\theta}{2}\sin\frac{\theta}{2} = \sin\theta$$

Example

- Rotation about z axis
  - Set the unit quaternion: $s = \cos\frac{\theta}{2},\ \mathbf{v} = (0,\ 0,\ 1)\sin\frac{\theta}{2}$
  - Substitute $a=b=0$, $c=\sin(\theta/2)$ into the matrix:

$$\mathbf{M}_R(\theta) = \begin{bmatrix} 1-2\sin^2\frac{\theta}{2} & -2\cos\frac{\theta}{2}\sin\frac{\theta}{2} & 0 \\ 2\cos\frac{\theta}{2}\sin\frac{\theta}{2} & 1-2\sin^2\frac{\theta}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$1-2\sin^2\frac{\theta}{2} = \cos\theta$$

$$2\cos\frac{\theta}{2}\sin\frac{\theta}{2} = \sin\theta$$

Rotation about a Line using Quaternion

- Let L be represented by vector V and passing through point P
  1. Translate P to the origin
  2. Use $M_{R(\theta)}$ for rotation
  3. Reverse step 1

$$R_{\theta,L} = T_P * M_{R(\theta)} * T_{-P}$$

i.e. $$M_{R(\theta)} = A_V^{-1} * R_{\theta,k} * A_V$$

$$= R_{\alpha,i} * R_{\beta,j} * R_{\theta,k} * R_{-\beta,i} * R_{\alpha,i}$$

Rodrigues' rotation formula

$$\mathbf{M}_{R(\theta)} = \begin{bmatrix} u_x^2(1-\cos\theta)+\cos\theta & u_x u_y(1-\cos\theta)-u_z\sin\theta & u_x u_z(1-\cos\theta)+u_y\sin\theta \\ u_y u_x(1-\cos\theta)+u_z\sin\theta & u_y^2(1-\cos\theta)+\cos\theta & u_y u_z(1-\cos\theta)-u_x\sin\theta \\ u_z u_x(1-\cos\theta)-u_y\sin\theta & u_z u_y(1-\cos\theta)+u_x\sin\theta & u_z^2(1-\cos\theta)+\cos\theta \end{bmatrix}$$

$$= \sin\theta \begin{bmatrix} 0 & -u_z & u_y \\ u_z & 0 & -u_x \\ -u_y & u_x & 0 \end{bmatrix} + (1-\cos\theta) \begin{bmatrix} -u_y^2-u_z^2 & u_x u_y & u_x u_z \\ u_x u_y & -u_x^2-u_z^2 & u_y u_z \\ u_x u_z & u_y u_z & -u_x^2-u_y^2 \end{bmatrix} + \mathbf{I}$$

$$= \sin\theta\mathbf{K} + (1-\cos\theta)\mathbf{K}^2 + \mathbf{I}$$

K is linear a transformation matrix that maps vector **v----->u×v**

K is formed from another vector **u**

Here, u is the axis of rotation

Finding an Axis and Angle from a Rotation Matrix

$$M_{R(\theta)} - M_{R(\theta)}^{T} = \sin\theta\mathbf{K} + (1 - \cos\theta)\mathbf{K}^2 + \mathbf{I} -$$

$$(\sin\theta\mathbf{K}^T + (1 - \cos\theta)(\mathbf{K}^2)^T + \mathbf{I}^T)$$

$$= 2\sin\theta\mathbf{K}$$

Since $\mathbf{K}^T = -\mathbf{K}$

$(\mathbf{K}^2)^T = \mathbf{K}^2$

Trace (rotation matrix) = $1+2\cos\theta$

when $\sin\theta \neq 0$, We can get axis of rotation

DIY

- Exhibit a unit quaternion that performs a rotation of 60 degrees about the axis 0,3,4 .
- Consider the following matrix R that represents a 3D rotation. Determine the axis and the angle of the rotation that the above matrix

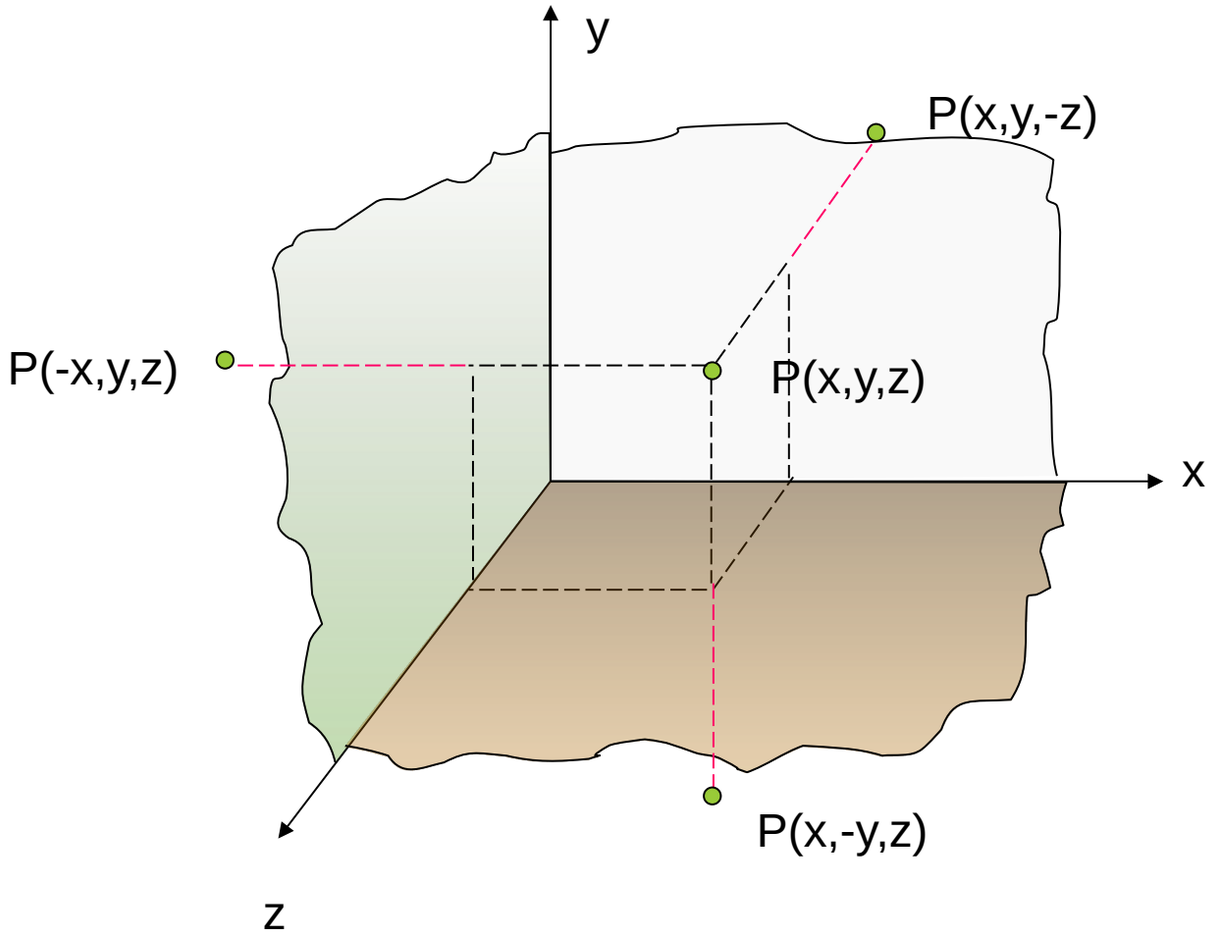
$$R = \begin{bmatrix} \frac{2}{3} & -\frac{1}{3} & \frac{2}{3} \\ \frac{2}{3} & \frac{2}{3} & -\frac{1}{3} \\ -\frac{1}{3} & \frac{2}{3} & \frac{2}{3} \end{bmatrix}$$

Use trace of a rotation matrix, $1+2\cos\theta$

Further Study

- Mathematics for Computer Graphics
  John Vince
- Mathematics for 3D Game Programming and Computer Graphics
  Eric Lengyel

Mirror reflection



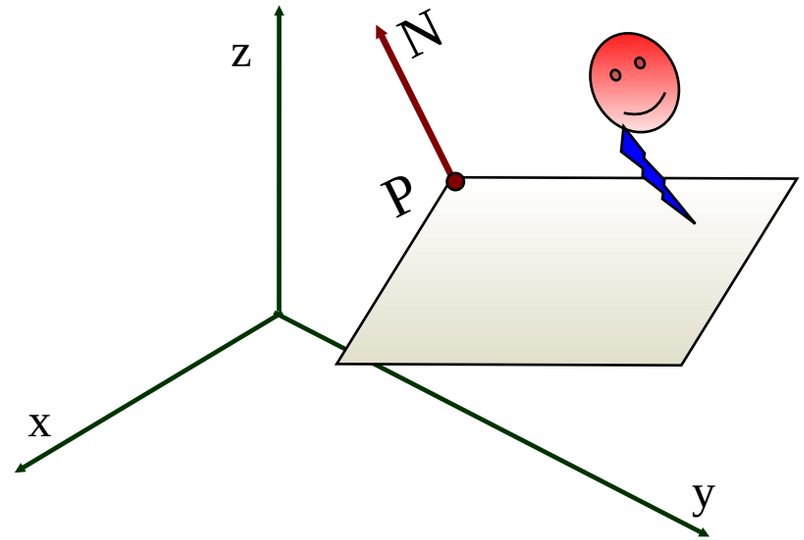
Mirror reflection

$$M_{xy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad M_{yz} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_{zx} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$M_{N,P}$: Mirror reflection

- Let the plane be represented by plane normal N and a point P in that plane

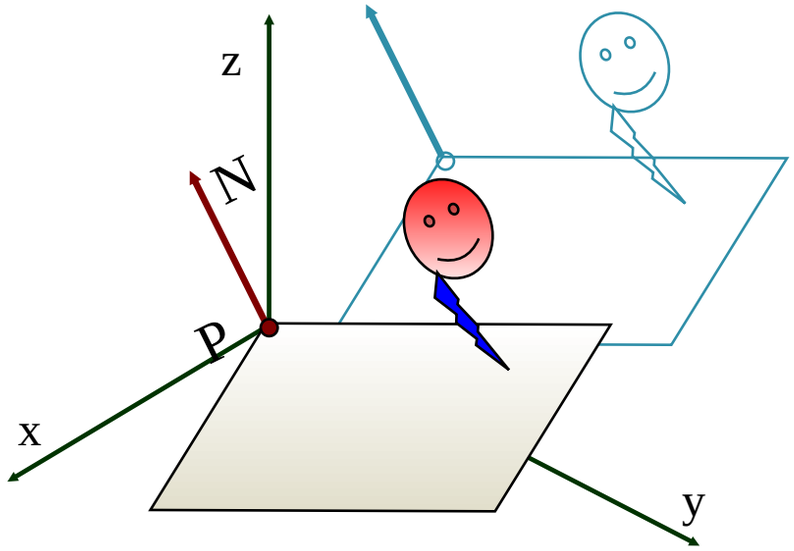


$$M_{N,P} =$$

$M_{N,P}$: Mirror reflection

- Let the plane be represented by plane normal N and a point P in that plane

1. Translate P to the origin

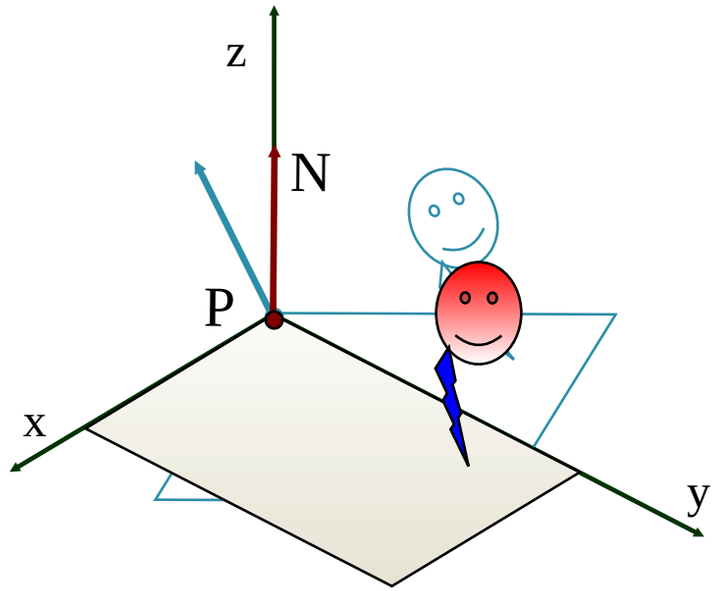


$M_{N,P}$ = T_{-P}

$M_{N,P}$: Mirror reflection

- Let the plane be represented by plane normal N and a point P in that plane

1. Translate P to the origin
2. Align N with vector k

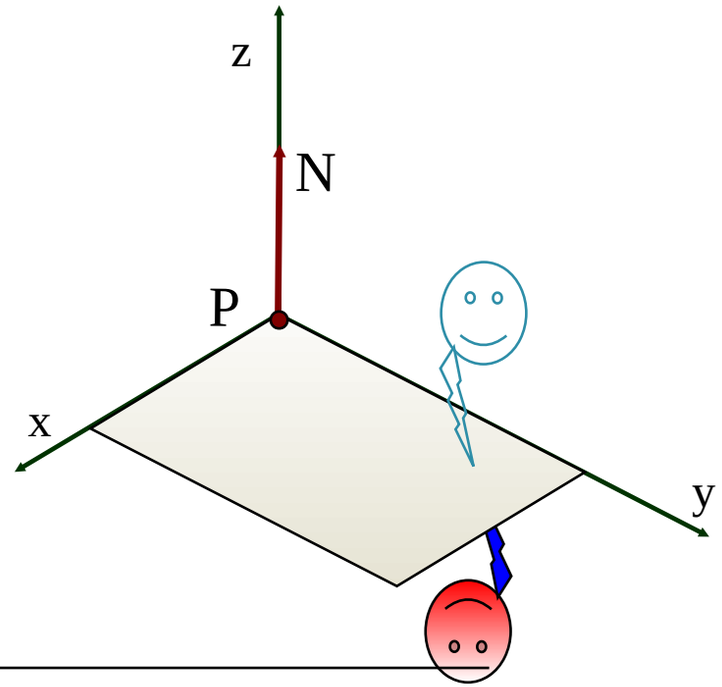


$M_{N,P} = A_N * T_{-P}$

$M_{N,P}$: Mirror reflection

- Let the plane be represented by plane normal N and a point P in that plane

1. Translate P to the origin
2. Align N with vector k
3. Reflect w.r.t xy-plane

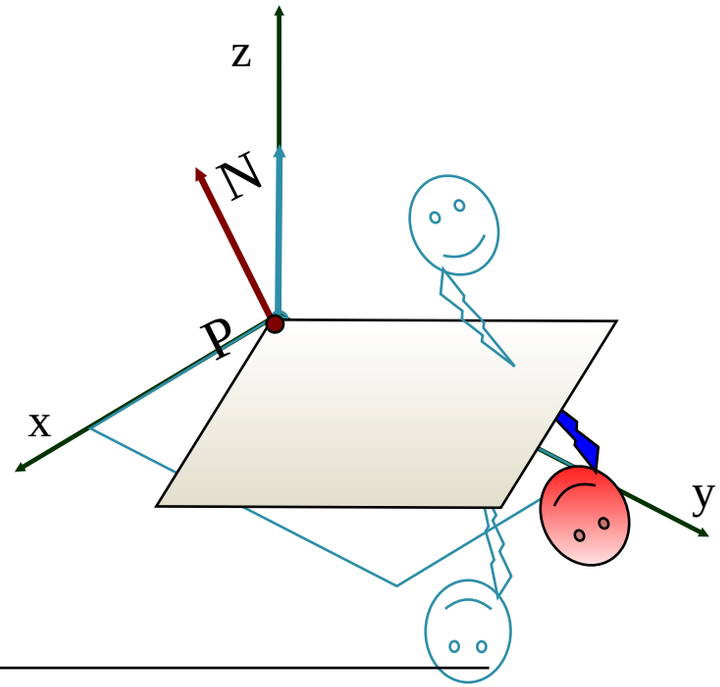


$$M_{N,P} = M_{xy} * A_N * T_{-P}$$

$M_{N,P}$: Mirror reflection

- Let the plane be represented by plane normal N and a point P in that plane

1. Translate P to the origin
2. Align N with vector k
3. Reflect w.r.t xy-plane
4. Reverse step 2

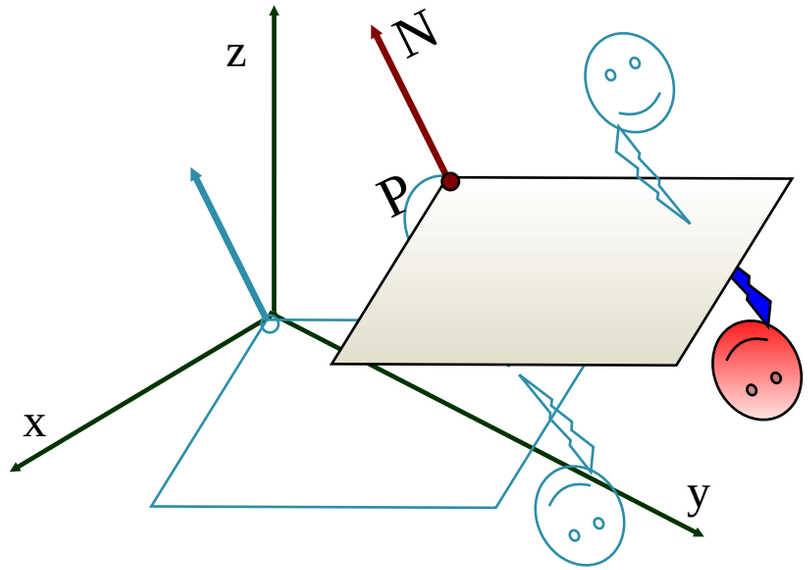


$$M_{N,P} = A_N^{-1} * M_{xy} * A_N * T_{-P}$$

$M_{N,P}$: Mirror reflection

- Let the plane be represented by plane normal N and a point P in that plane

1. Translate P to the origin
2. Align N with vector k
3. Reflect w.r.t xy-plane
4. Reverse step 2
5. Reverse step 1

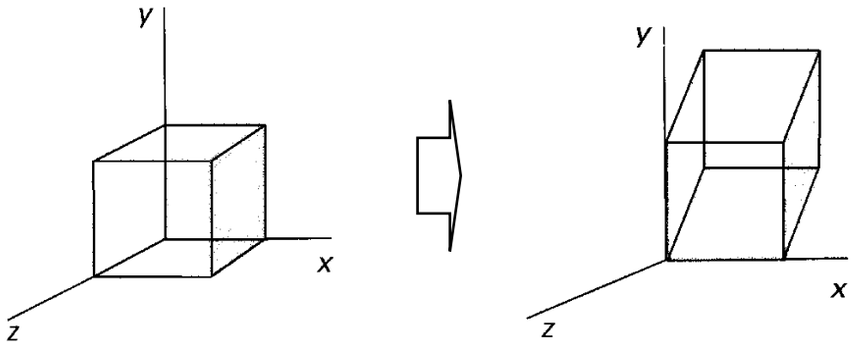


$$M_{N,P} = T_{-P}^{-1} * A_N^{-1} * M_{xy} * A_N * T_{-P}$$

Other Transformations

- Z-axis Shear

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



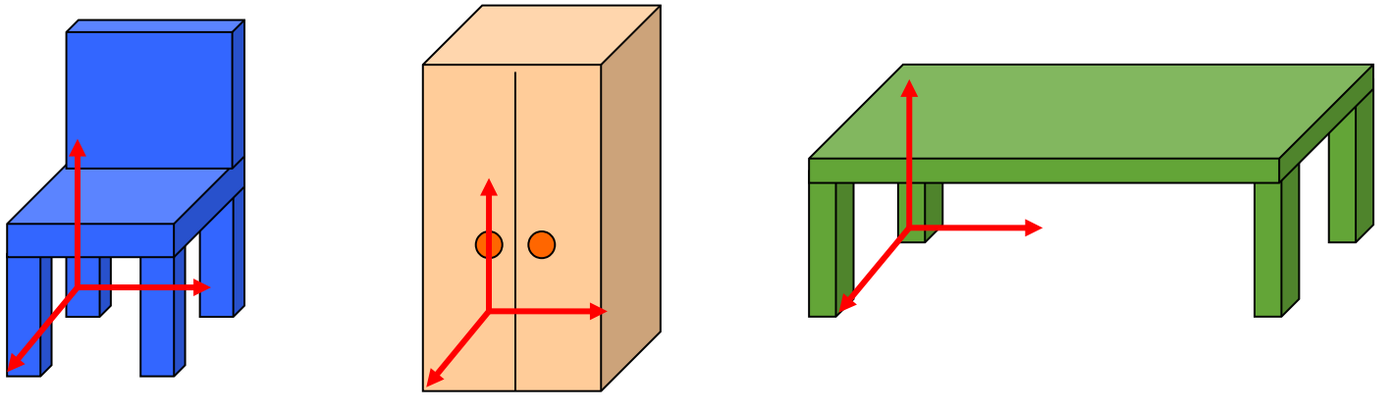
Solve it

Consider a polygon with points $P_1(2,1,1)$, $P_2(4,2,3)$ and $P_3(3,3,1)$. Find a composite transformation matrix to

1. Place the polygon in positive yz plane
2. Place P_1 at origin and P_1P_2 along positive z axis

Coordinate Transformations

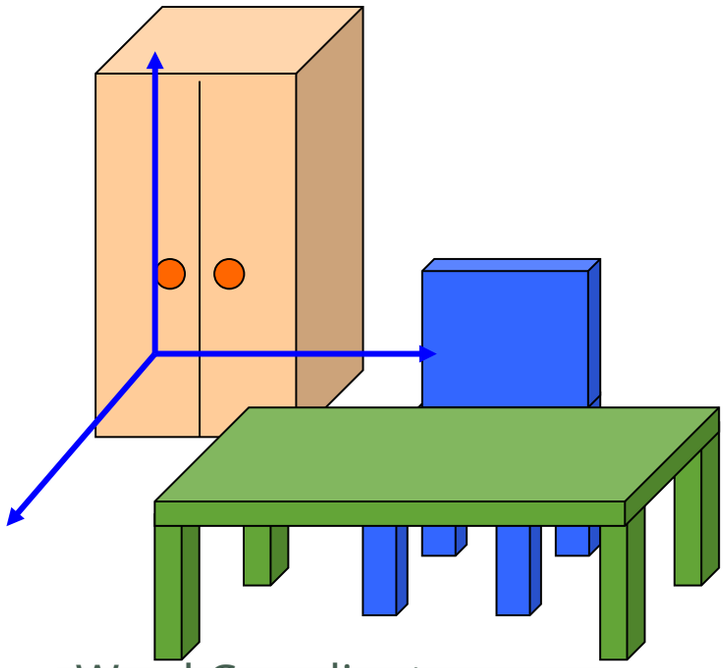
- Multiple Coordinate System
  - Local (modeling) coordinate system
  - World coordinate scene



Local Coordinate System

Coordinate Transformations

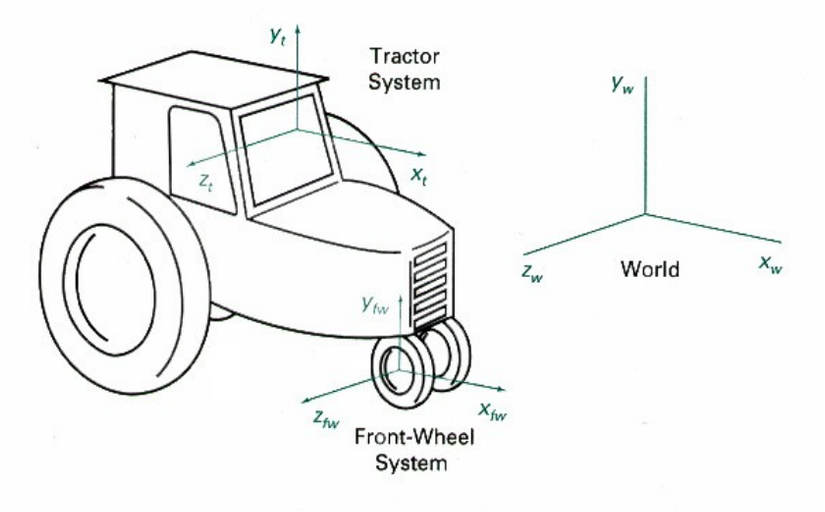
- Multiple Coordinate System
  - Local (modeling) coordinate system
  - World coordinate scene



Word Coordinate System

Coordinate Transformations

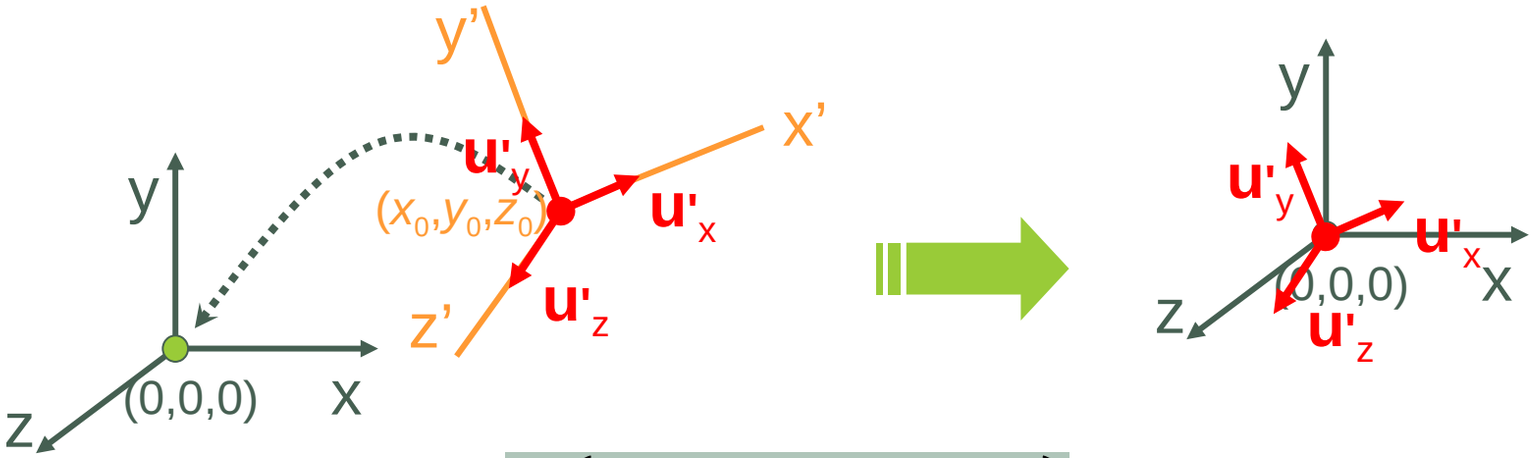
- Example – Simulation of Tractor movement
  - As tractor moves, **tractor coordinate system** and **front-wheel coordinate system** move in world coordinate system
  - **Front wheels** rotate in wheel coordinate system



Coordinate Transformations

- Transformation of an Object Description from One Coordinate System to Another
- Transformation Matrix
  - Bring the two coordinates systems into alignment

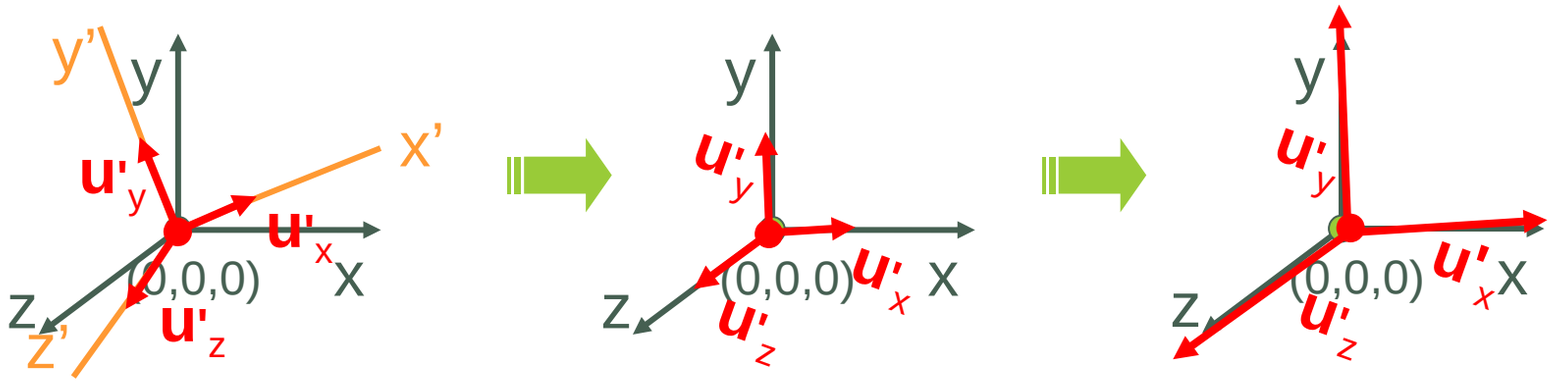
1. Translation



$$T(-x_0, -y_0, -z_0)$$

Coordinate Transformations

2. Rotation



$$\mathbf{R} = \begin{bmatrix} u'_{x_1} & u'_{x_2} & u'_{x_3} & 0 \\ u'_{y_1} & u'_{y_2} & u'_{y_3} & 0 \\ u'_{z_1} & u'_{z_2} & u'_{z_3} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Problems to be solved:

Schaum's outline series:

Problems:

- 6.1
- 6.2, 6.5, 6.9, 6.10, 6.11, 6.12 $\Rightarrow A_v$
- 6.3, 6.4 $\Rightarrow R_{\theta,L}$
- 6.6, 6.7, 6.8 $\Rightarrow M_{N,P}$

Acknowledgement
cgvr.korea.ac.kr

Problems to be solved:

3. Consider a 2D rectangle $ABCD$ where $A = (0, 0)$, $B = (2, 0)$, $C = (2, 1)$ and $D = (0, 1)$. We want to apply a 2D transformation to this rectangle which makes it a parallelopiped $ABEF$ where $E = (4, 1)$ and $F = (2, 1)$. a. What kind of transformation is this? What is the 3x3 matrix M achieving this transformation? c. What additional transformation N would we need to apply to $ABEF$ to get the parallelopiped $A'B'E'F'$ where $A' = (1, 2)$, $B' = (3, 2)$, $E' = (5, 3)$, and $F' = (3, 3)$? d. What is the final concatenated matrix in terms of M and N that will transform $ABCD$ to $A'B'E'F'$?

4. Derive the scaling matrix for scaling an object by a scale factor 3 along an arbitrary direction given by vector $u = (1, 2, 1)$ rooted at $(5, 5, 5)$.

5. Explain what transformation is produced by each of the following matrices when applied on a 4x1 homogeneous coordinate.

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad \begin{pmatrix} 1 & 0 & p & -p(1+r) \\ 0 & 1 & q & -q(1+r) \\ 0 & 0 & 1+r & -r(1+r) \\ 0 & 0 & 1 & -r \end{pmatrix} \tag{6.59}$$

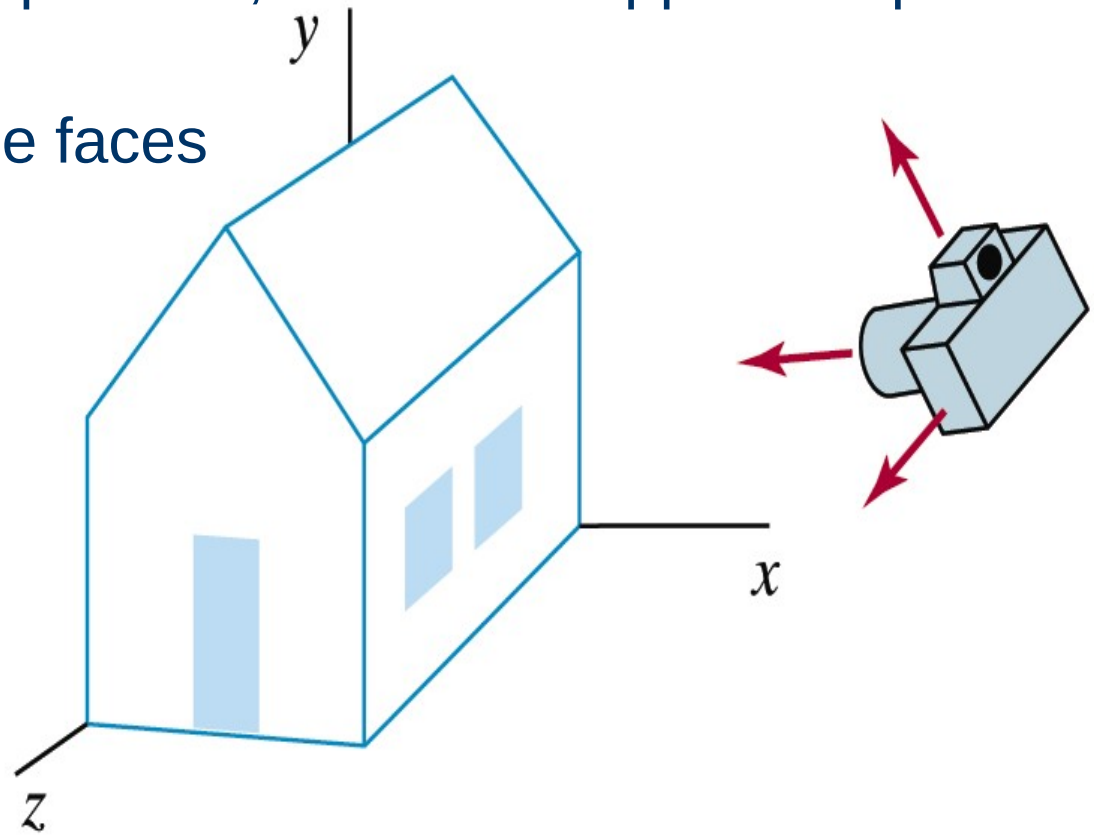
3D Viewing Transform

Dr. Sk. Md. Masudul Ahsan
Professor
Dept. of CSE, KUET

Remember the big thing – Camera Analogy

- 3D view of a scene is analogous to photographing
  - Need to position the camera
  - Need to decide camera orientation
  - When shutter is pressed, scene is cropped to aperture (window) size
  - Light from visible faces

projected into film

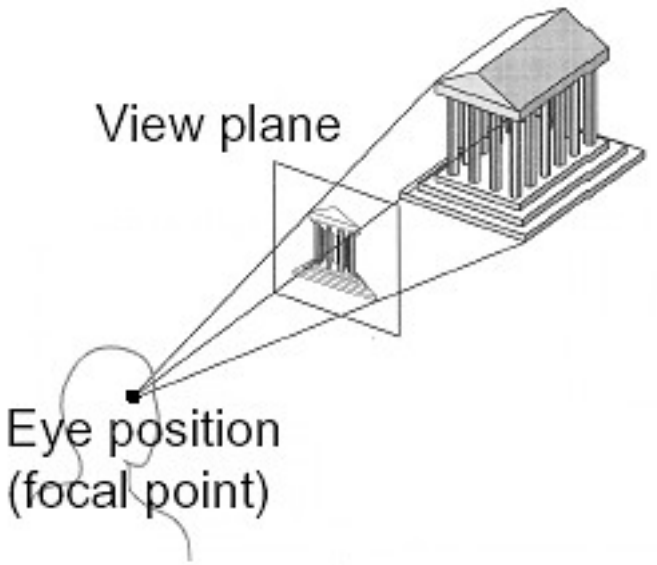


Camera Models

- The most common model is **pin-hole camera**
  - All captured light rays arrive along paths toward focal point without lens distortion (everything is in focus)
  - Sensor response proportional to radiance

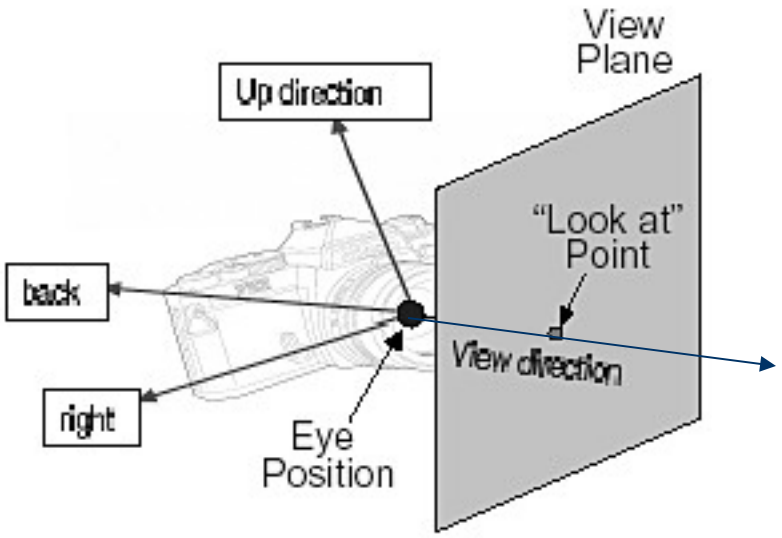
Other models consider…

- Depth of field
- Motion blur
- Lens distortion



Viewing Parameters

- Position
  - Eye position(px, py, pz)
- Orientation
  - View direction(dx, dy, dz)
  - Up direction(ux, uy, uz)
- Aperture
  - Field of view(xfov, yfov)
- Film plane
  - “look at” point
  - View plane normal



Transformations

P(x, y, z)

3D Object Coordinate

Model Transformation

3D World Coordinate

Viewing Transformation

3D Viewing Coordinate

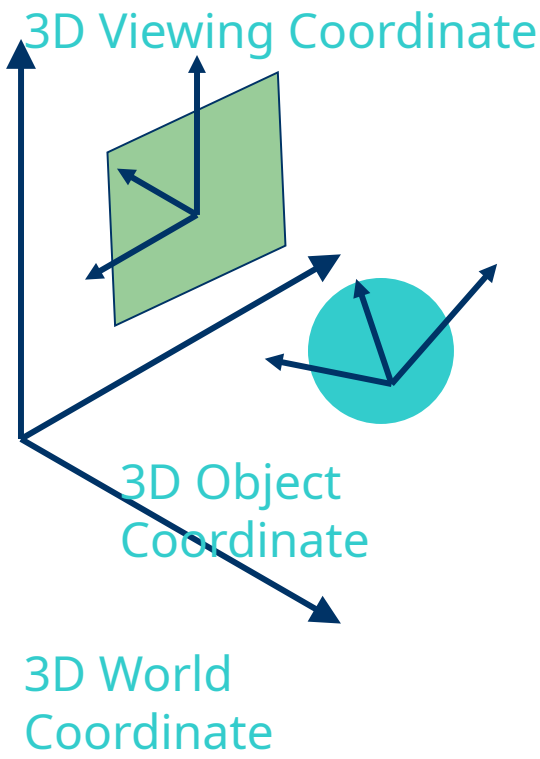
Projection Transformation

2D Projection Coordinate

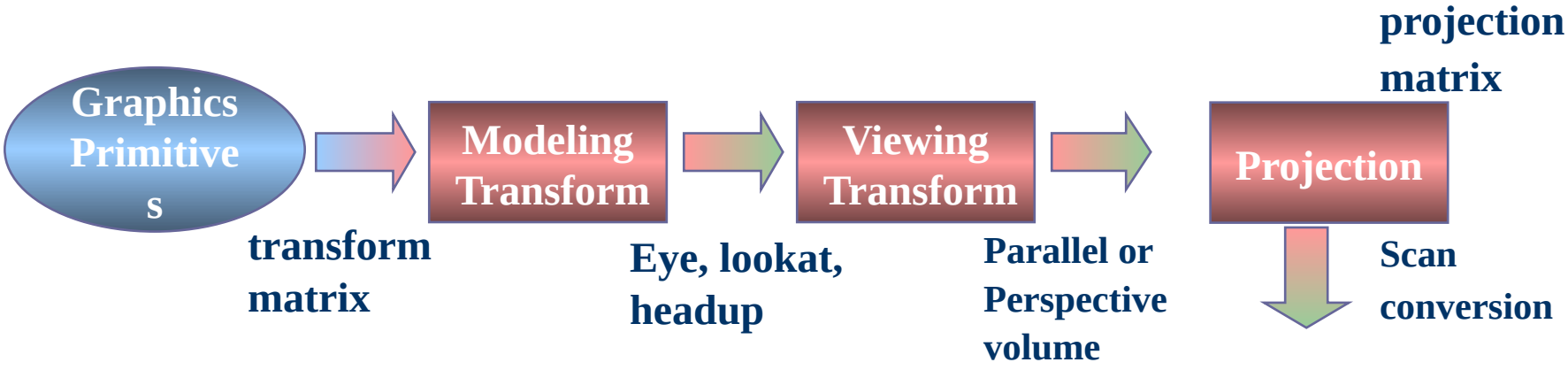
Viewport Transformation

2D Device Coordinate

p(x', y')



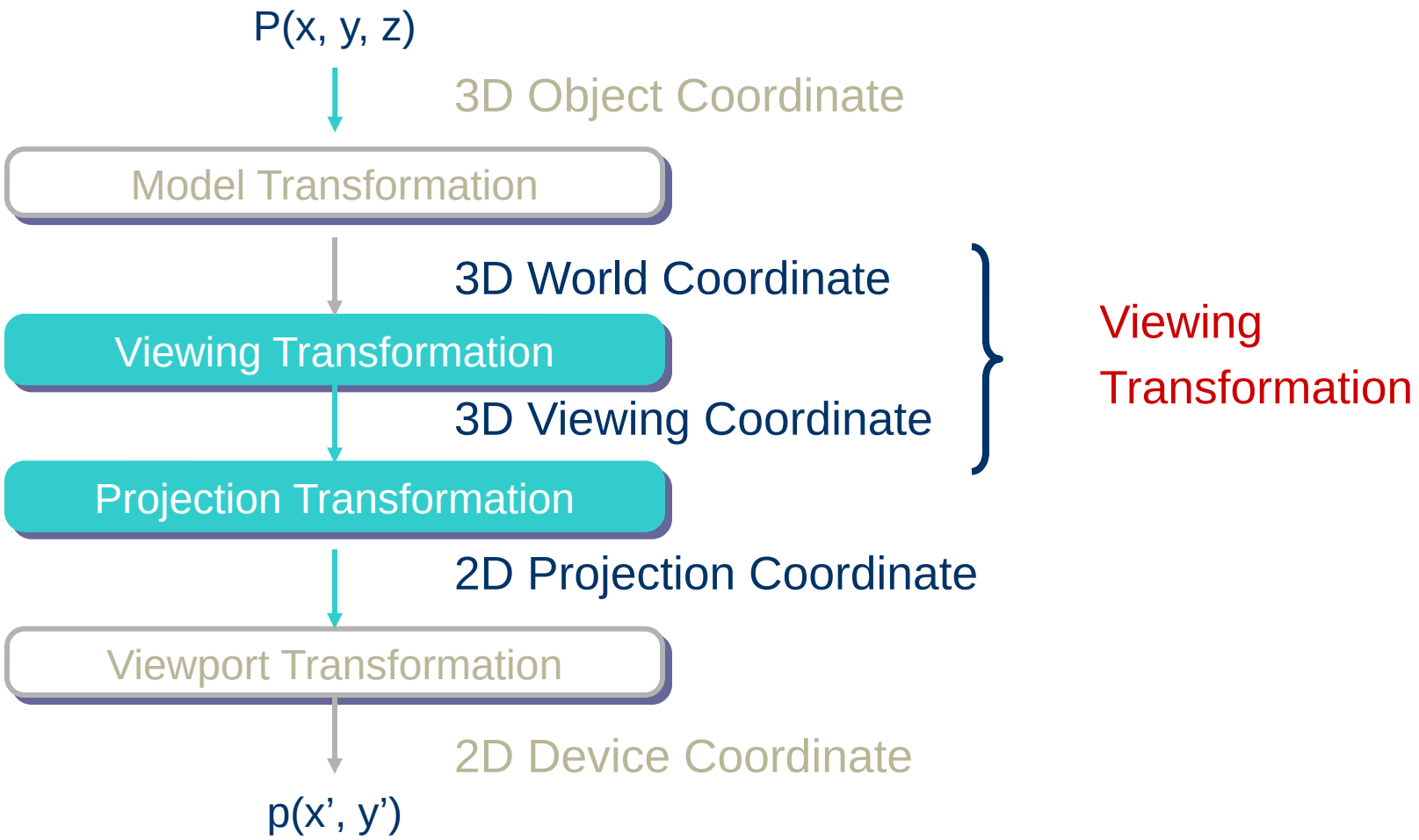
Graphics Pipeline



- Once scene has been modeled, world coordinates are converted to viewing coordinates
- Viewing coordinate system is used in graphics package for specifying viewer position and position of projection plane (film)

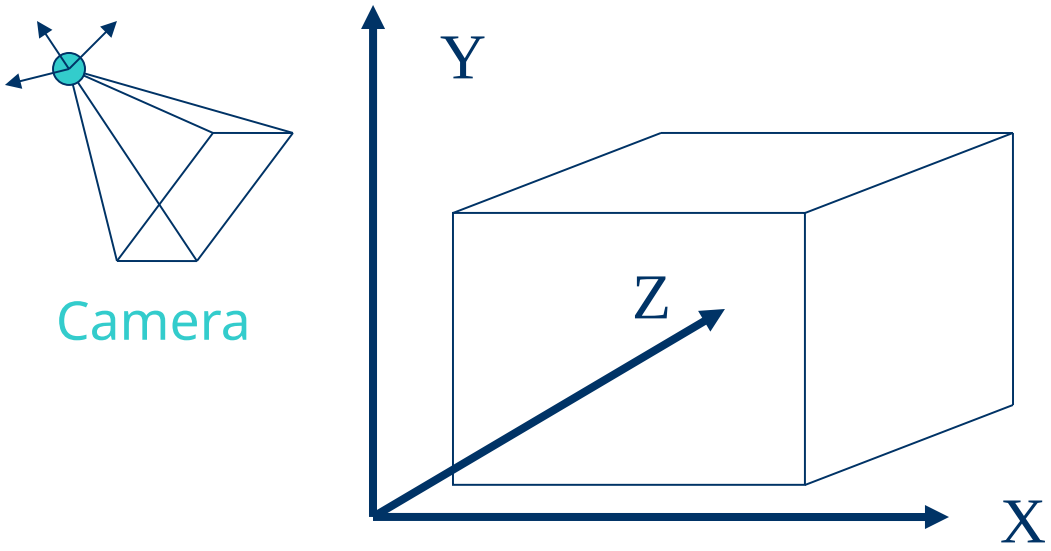
Prof. Dr. SMM Ahsan, CSE, KUET

Viewing Transformation



Viewing Transformation

- Mapping from world to Viewing coordinates
  - Origin moves to eye position
  - Up vector maps to Y axis
  - Right vector maps to X axis



Viewing Coordinate system

- Particular view of a scene is chosen by
  - Establishing Viewing Coordinate system
  - And then, a View plane (projection plane) is setup perpendicular to the viewing z_V axis.
- To establish Viewing Coordinate system (VCS)
  - First, a world coordinate position ($\mathbf{P_0}$) is picked as **view reference point,** it serves as **origin** of VCS.
  - Then, positive direction of z_V axis is specified by specifying **view plane normal vector (N)**
    - Another world position **(P)** is picked **–** *look at*

Prof. Dr. SMM Ahsan, CSE, KUET

Viewing Coordinate system

And finally, up direction of view is specified by **view up vector (V).**

- Generally specified by a world coordinate vector
- It gives an appx. direction of y_V axis
- Exact value of y_V axis then calculated by graphics package

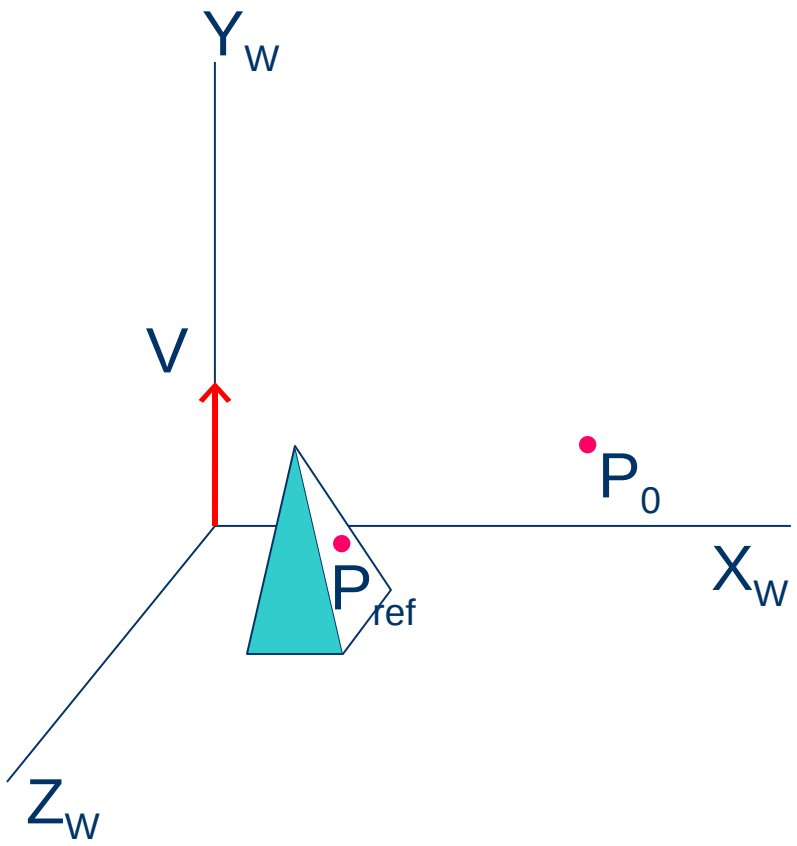
- Using N and V, graphics package compute third vector U perpendicular to both N and V, to denote x_V axis

  $U = V \times N$

- Vector denoting y_V axis can be computed by $N \times U$

Viewing Coordinate system

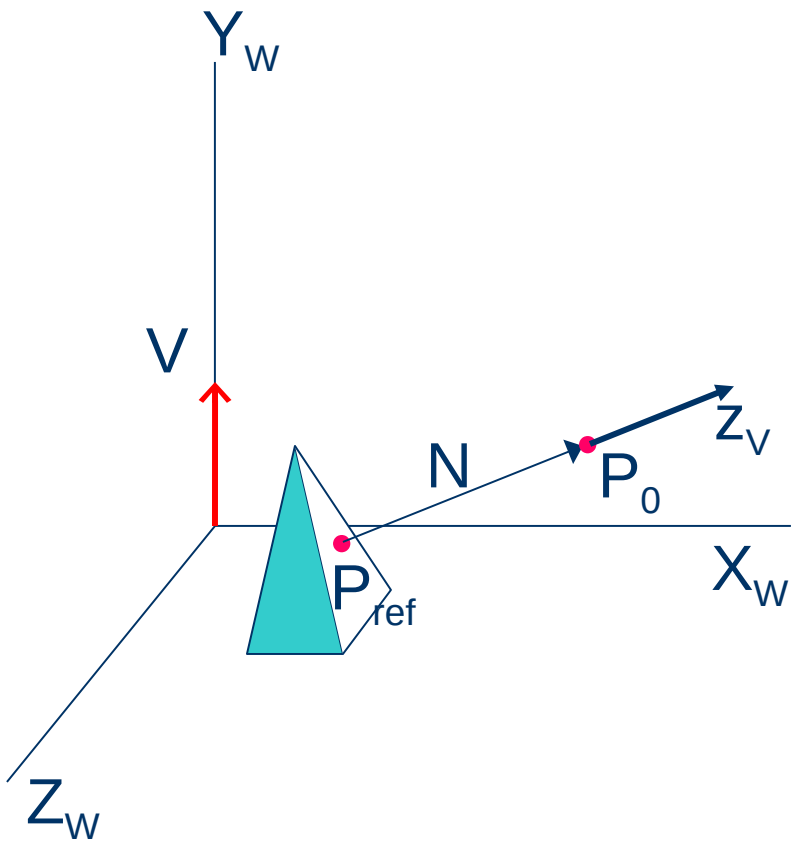
- Specification of VCS :
  - $\mathbf{P}_0$: *View* or *eye point*
  - $\mathbf{P}_{ref}$: *Center* or *look-at point*
  - **V**: *View-up vector*



Viewing Coordinate system

- computation of VCS :

$z_V = N = P_0 – Pref$

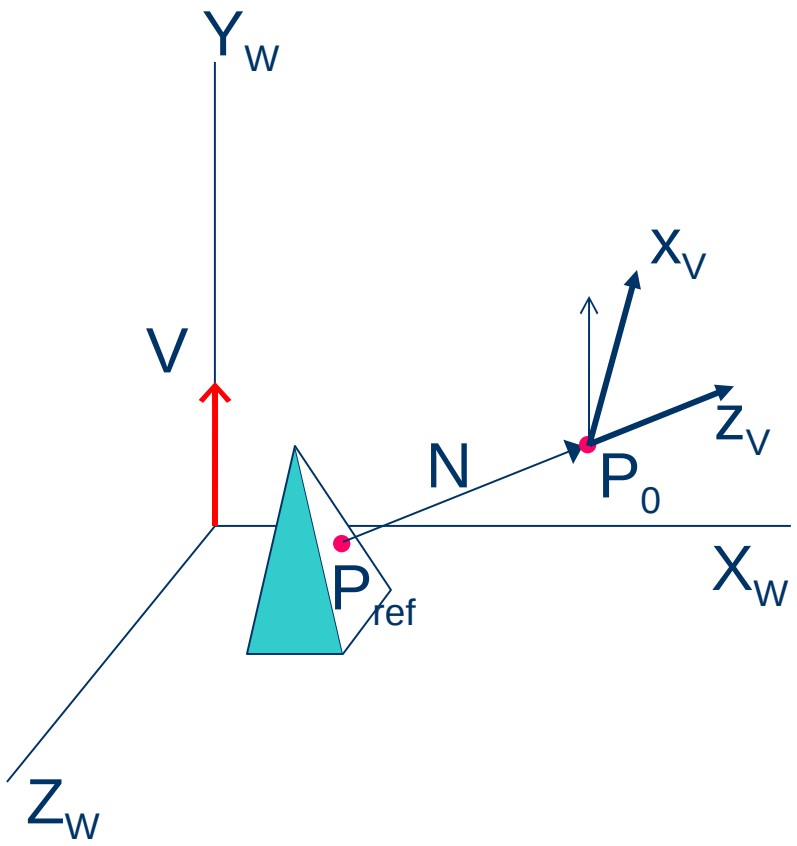


Viewing Coordinate system

- computation of VCS :

$$z_V = N = P_0 - P$$

$$x_V = V \times N = U$$



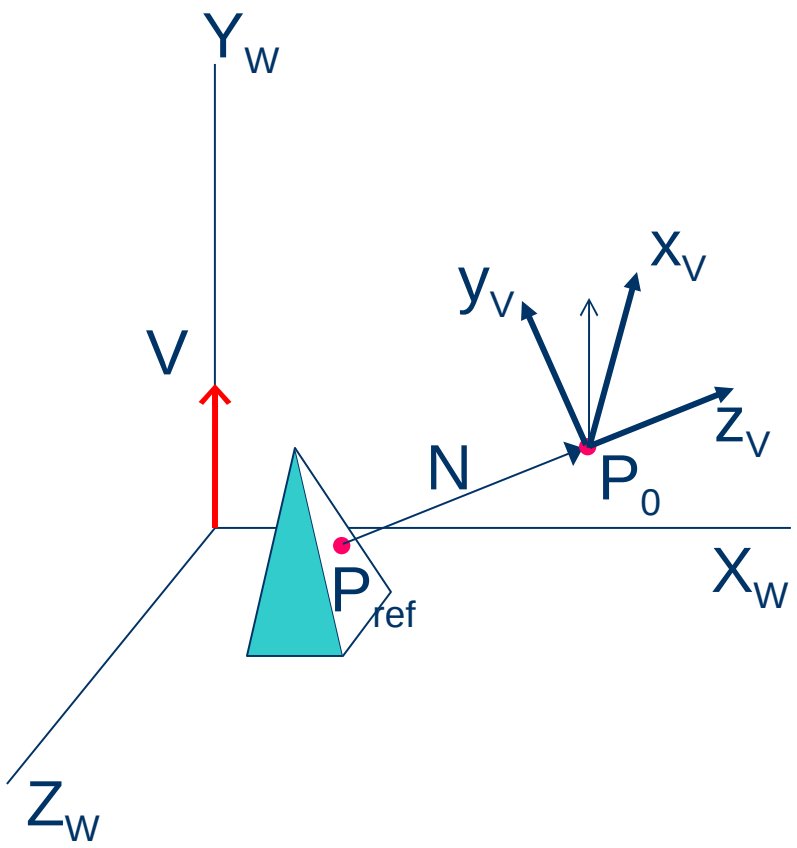
Viewing Coordinate system

- computation of VCS :

$$z_V = N = P_0 - P$$

$$x_V = V \times N = U$$

$$y_V = N \times U$$



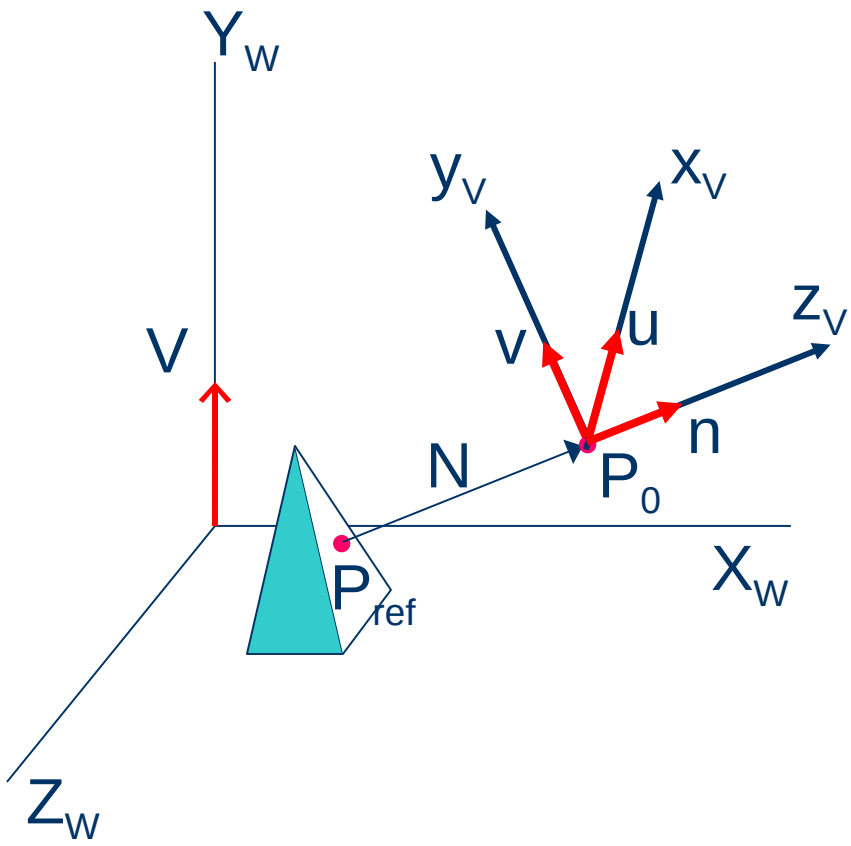
Prof. Dr. SMM Ahsan, CSE, KUET

Viewing Coordinate system

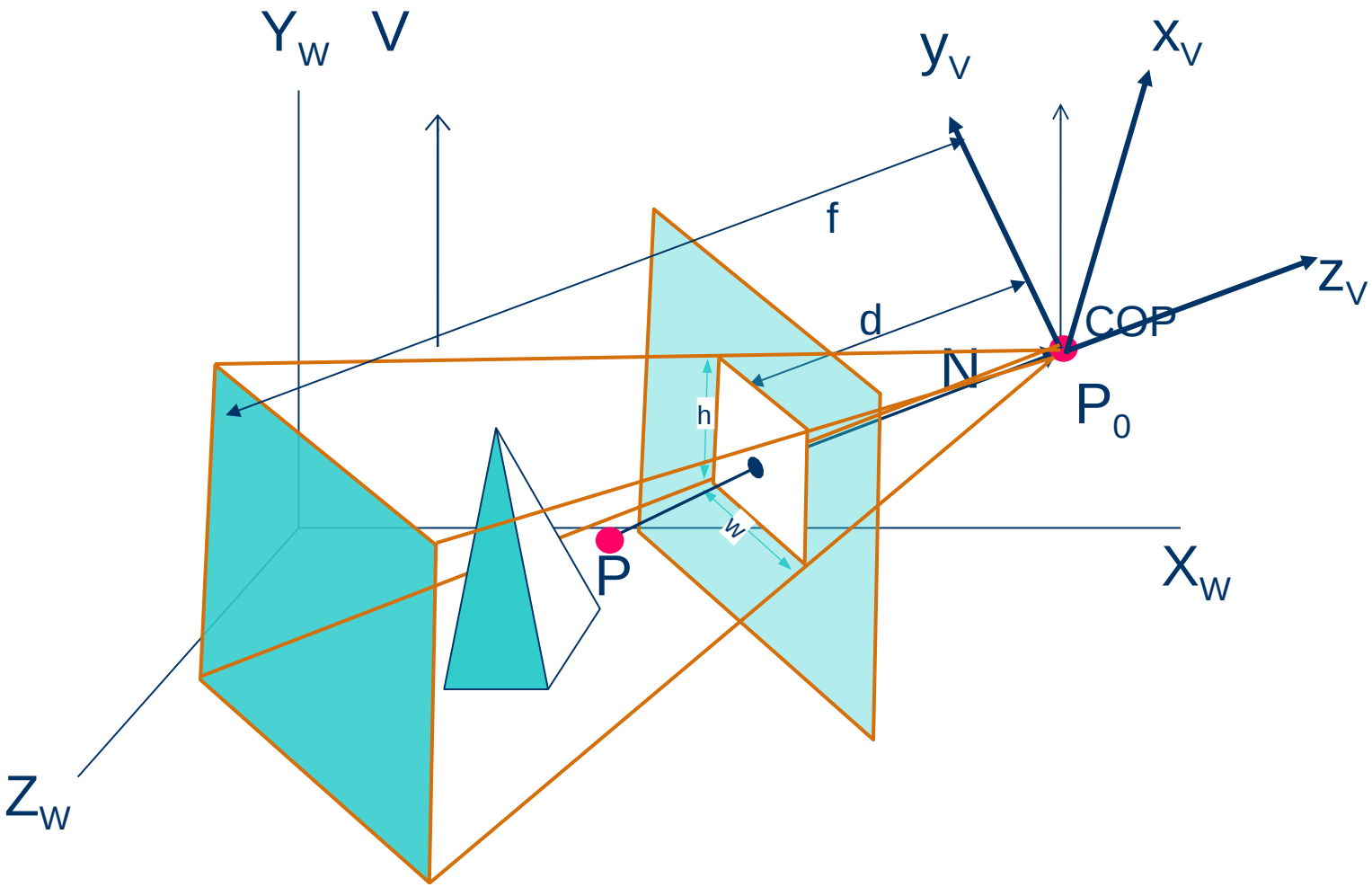
- $z_V = N = P_0 - P$
- $x_V = V \times N = U$
- $y_V = N \times U$

- All these computations are actually carried out with unit vectors **u**, **v**, **n**
- And the system is called **uvn** system
- Graphics package allow user to choose view plane position along **n** axis

  By specifying **view plane distance** from **viewing origin**



WCS, VCS and projection



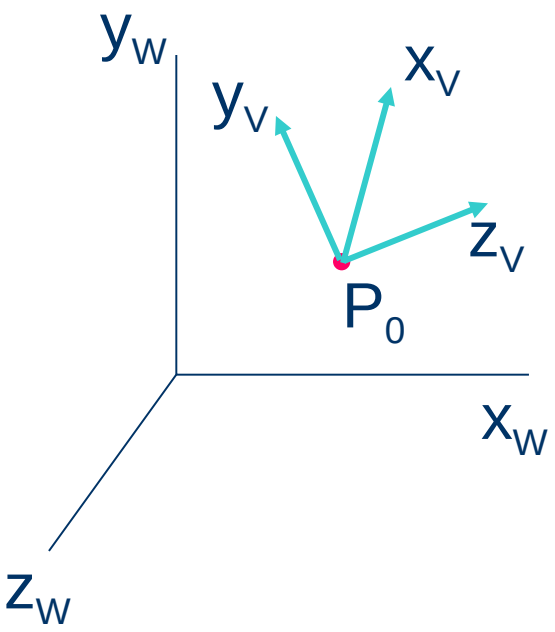
WC to VC transformation

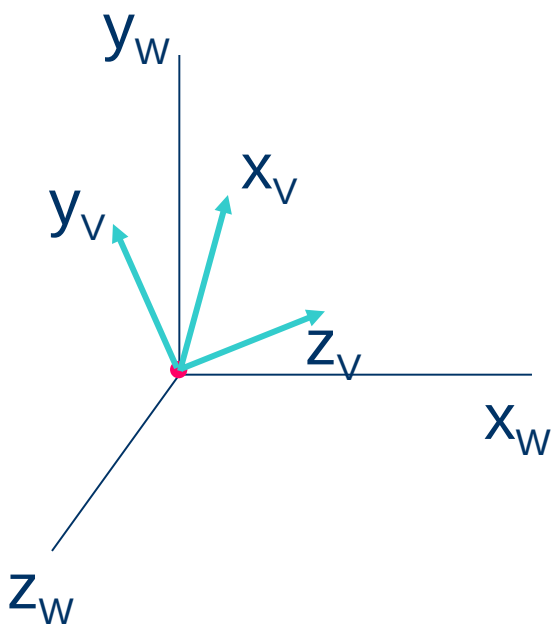
- Before projection, object description must be converted from world to viewing system
- Conversion is eqv. to a transformation that superimposes the viewing system to world system
- This transformation can be achieved by basic transformations
  1. Translate view reference point $P_0(x_0,y_0,z_0)$ to world origin
  2. Apply rotation to align x_V, y_V, z_V axes with x_W, y_W, z_W axes, respectively

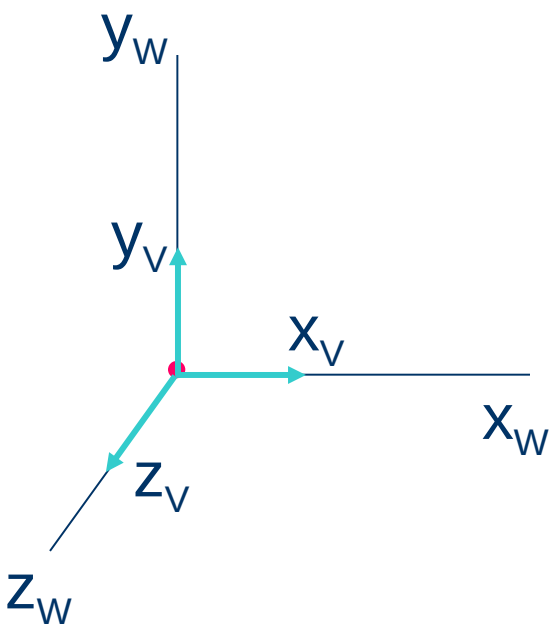
WC to VC transformation

- The translation matrix is

$$T = \begin{bmatrix} 1 & 0 & 0 & -x_0 \\ 0 & 1 & 0 & -y_0 \\ 0 & 0 & 1 & -z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$







Prof. Dr. SMM Ahsan, CSE, KUET

Translation from world to viewing coordinates

- Rotation matrix **R** can be found by up to three world coordinate axis rotations R_z. R_y. R_x = R
  - R_x places N or z_v to $x_w z_w$ plane
  - R_y aligns z_v to z_w axis
  - R_z aligns y_v to y_w axis

$$n = \frac{N}{|N|} = (n_x, n_y, n_z)$$

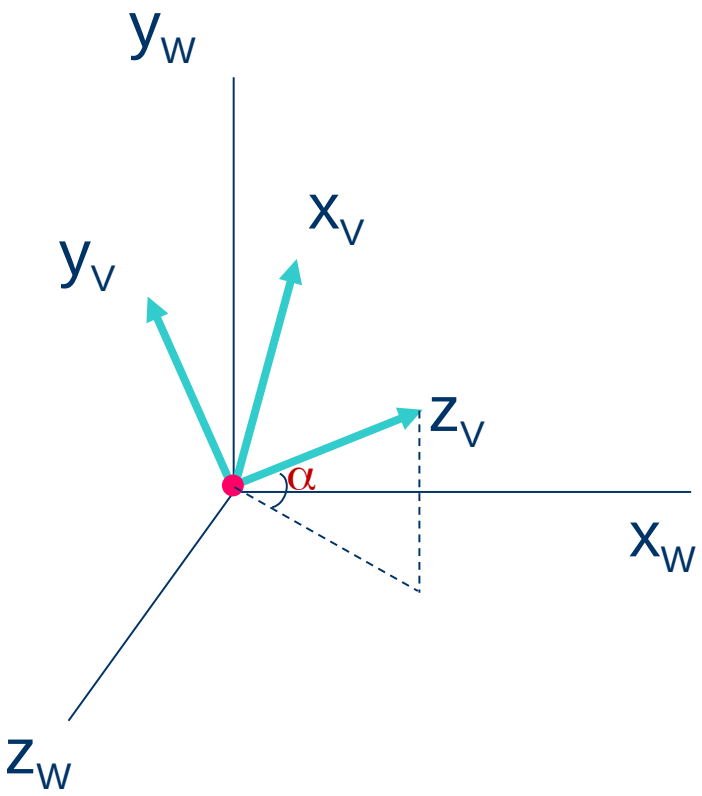
$$u = \frac{V \times N}{|V \times N|} = (u_x, u_y, u_z)$$

$$v = n \times u = (v_x, v_y, v_z)$$

- $M_{WC,VC}$ =R.T

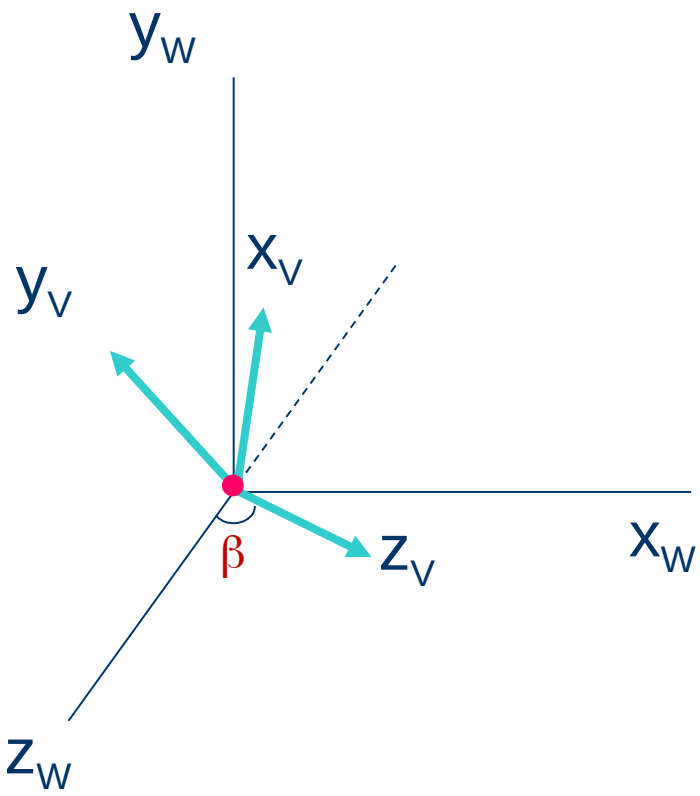
Prof. Dr. SMM Ahsan, CSE, KUET

Composite R



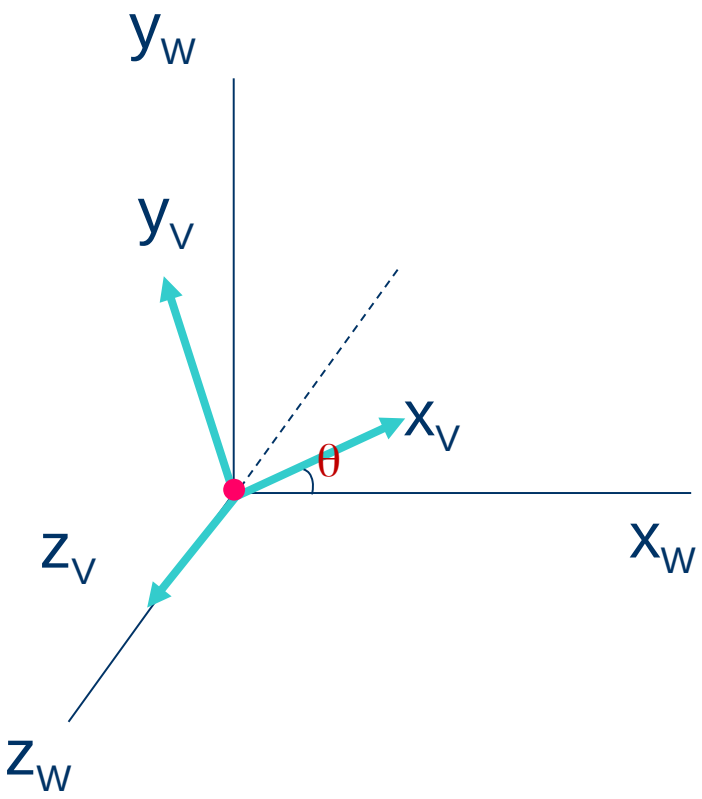
- Rotate α about X_W, R_i

Composite R



- Rotate α about X_w, R_i
  - Places Z_v in XZ plane
- Rotate β about Y_w, R_j

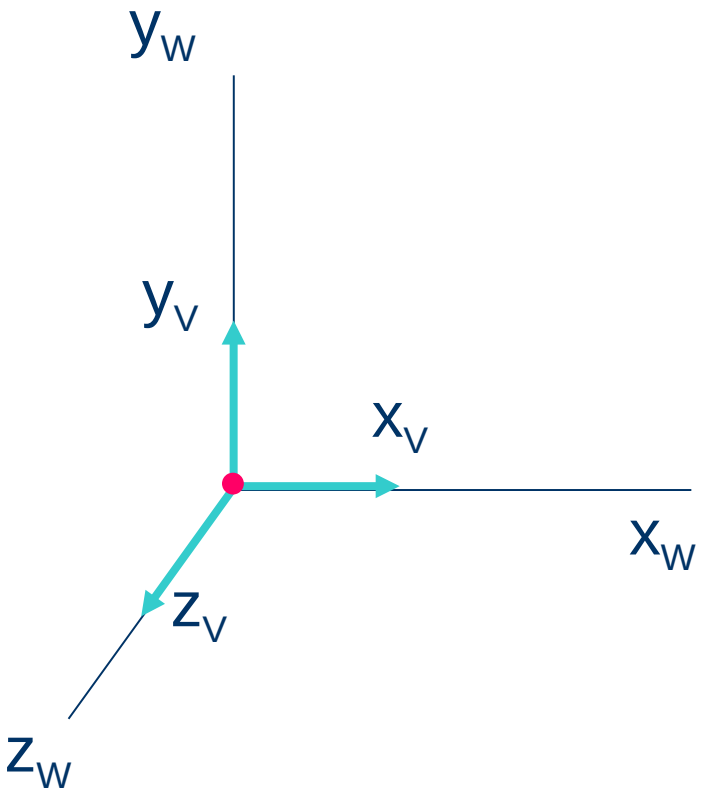
Composite R



- Rotate α about X_w, R_i
  - Places Z_v in XZ plane
- Rotate β about Y_w, R_j
  - Aligns Z_v with Z_w
  - X_v , Y_v are in XY plane
- Rotate θ about Z_w, R_k

Prof. Dr. SMM Ahsan, CSE, KUET

Composite R



- Rotate α about X_W, R_i
  - Places Z_v in XZ plane
- Rotate β about Y_W, R_j
  - Aligns Z_v with Z_w
  - X_v , Y_v are in XY plane
- Rotate θ about Z_W, R_k

Finding composite R

$$R = R_k R_j R_i = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\beta & 0 & \sin\beta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\beta & 0 & \cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha & 0 \\ 0 & \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos\theta\cos\beta & -\sin\theta & \cos\theta\sin\beta & 0 \\ \sin\theta\cos\beta & \cos\theta & \sin\theta\sin\beta & 0 \\ -\sin\beta & 0 & \cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha & 0 \\ 0 & \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos\theta\cos\beta & -\sin\theta\cos\alpha + \sin\alpha\cos\theta\sin\beta & \sin\alpha\sin\theta + \cos\alpha\cos\theta\sin\beta & 0 \\ \sin\theta\cos\beta & \cos\theta\cos\alpha + \sin\alpha\sin\theta\sin\beta & -\sin\alpha\cos\theta + \cos\alpha\sin\theta\sin\beta & 0 \\ -\sin\beta & \sin\alpha\cos\beta & \cos\alpha\cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

R is Rigid−body Transform – i.e. orthogonal matrix

i) *rows and columns* are unit vectors

ii) *vectors* are perpendicular *to each other*

Prof. Dr. SMM Ahsan, CSE, KUET

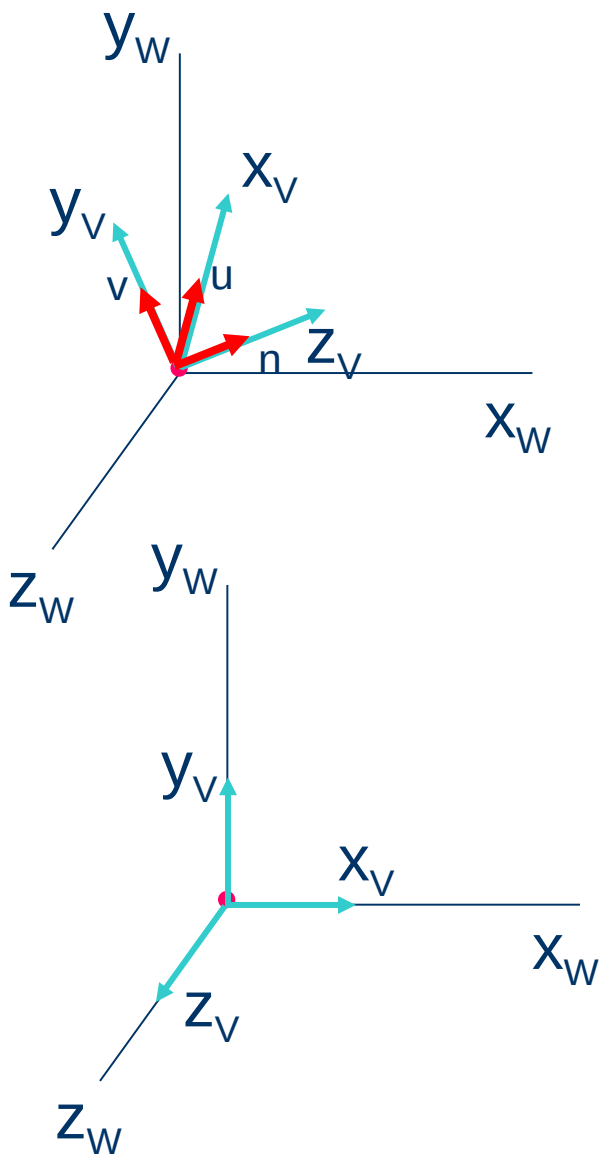
Finding composite R (another way)

Let R be

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} = \begin{bmatrix} R_{1x} & R_{1y} & R_{1z} \\ R_{2x} & R_{2y} & R_{2z} \\ R_{3x} & R_{3y} & R_{3z} \end{bmatrix}$$

Note: $R_{1x} \Rightarrow x$ component of vextor R_1

R aligns unit vectors **u, v, n** to **i, j, k** respectably



Finding R

we can write

$$\Rightarrow R \cdot \hat{u} = \hat{i} \qquad \Rightarrow R^T \cdot \hat{i} = \hat{u} \quad \left[\; R^{-1} = R^T \; \right]$$

$$\Rightarrow \begin{bmatrix} R_{1x} & R_{2x} & R_{3x} \\ R_{1y} & R_{2y} & R_{3y} \\ R_{1z} & R_{2z} & R_{3z} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \hat{u}$$

$$\Rightarrow \begin{bmatrix} R_{1x} \\ R_{1y} \\ R_{1z} \end{bmatrix} = \hat{u} = \vec{R_1}$$

similarly we can find that

$\vec{R_2} = \hat{v}$ and $\vec{R_3} = \hat{n}$

WC to VC transformation

- Rotation matrix **R** can be found by
  - Calculate unit vectors u,v,n
  - Form composite R directly

$$n = \frac{N}{|N|} = (n_x, n_y, n_z)$$

$$u = \frac{V \times N}{|V \times N|} = (u_x, u_y, u_z)$$

$$v = n \times u = (v_x, v_y, v_z)$$

$$R = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- $M_{WC,VC} = R.T$

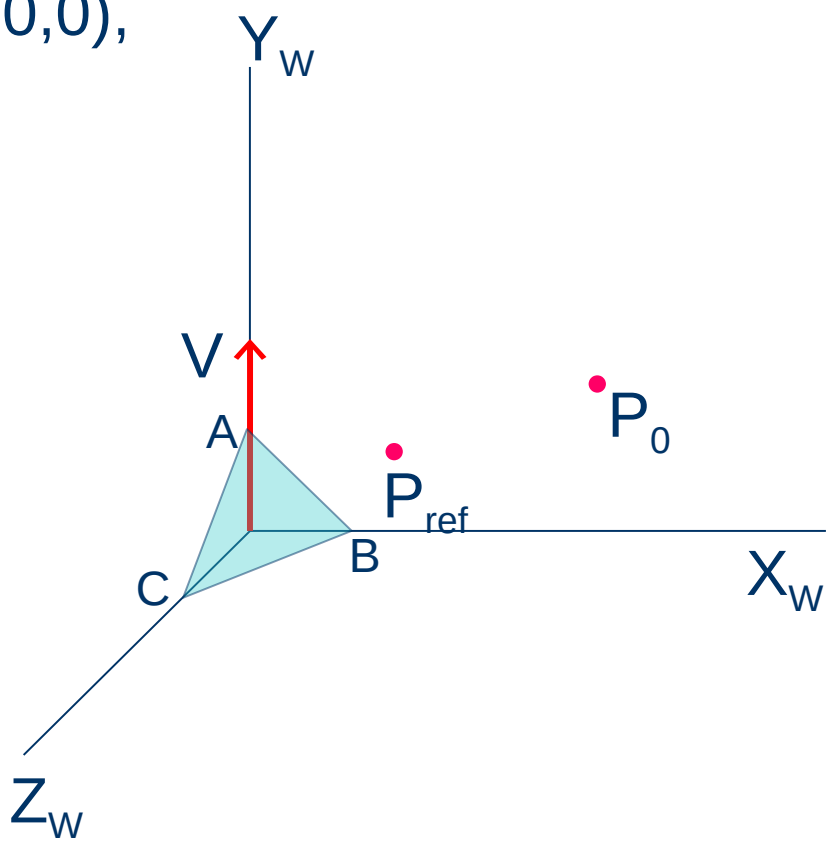
$$= \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_0 \\ 0 & 1 & 0 & -y_0 \\ 0 & 0 & 1 & -z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} u_x & u_y & u_z & dx \\ v_x & v_y & v_z & dy \\ n_x & n_y & n_z & dz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

***dx* = -P_0.u** **_dy_ = -P_0.v** **_dx_ = -P_0.n**

Prof. Dr. SMM Ahsan, CSE, KUET

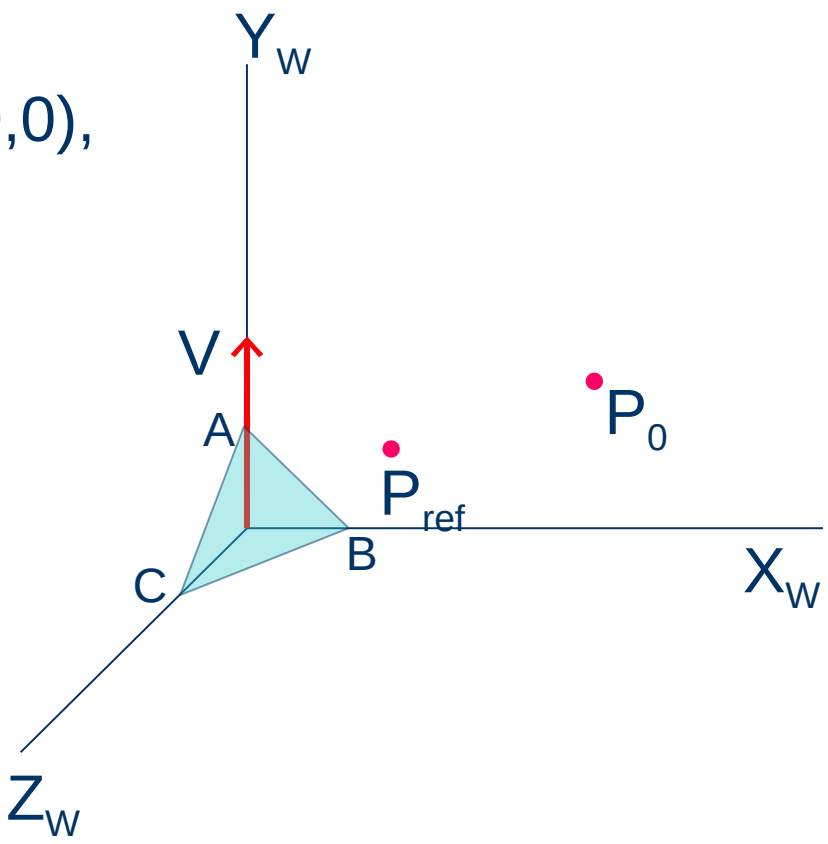
example

- Find the VCS values for the
  - Object: triangle A(0,1,0), B(1,0,0), C(0,0,1)
- when
  - Eye: (3,2,3)
  - Look at point: (2,1,2)
  - Up vector (0,1,0)



example

- Find the VCS values for the
  - Object: triangle A(0,1,0), B(1,0,0), C(0,0,1)
- when
  - Eye: (3,2,3)
  - Look at point: (2,1,2)
  - Up vector (0,1,0)



$N = P_0 - P_{ref} = (1,1,1)$

$$n = \left(\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}\right)$$

$$V \times N = (1, 0, -1)$$

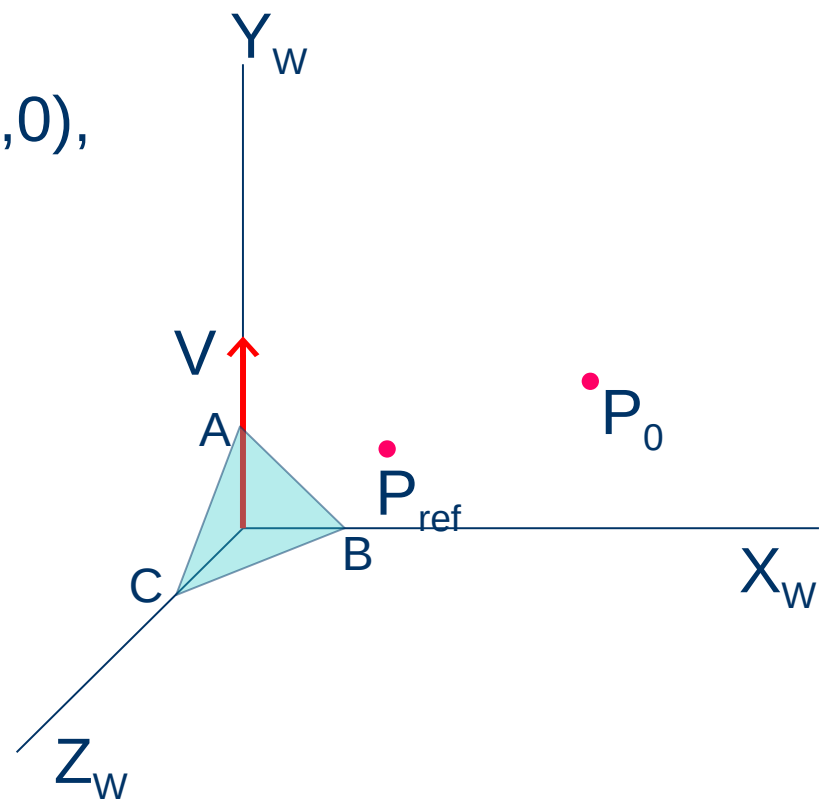
$$u = \left(\frac{1}{\sqrt{2}}, 0, -\frac{1}{\sqrt{2}}\right)$$

$$v = n \times u$$

$$= \left(-\frac{1}{\sqrt{6}}, \frac{2}{\sqrt{6}}, -\frac{1}{\sqrt{6}}\right)$$

example

- Find the VCS values for the
  - Object: triangle A(0,1,0), B(1,0,0), C(0,0,1)
- when
  - Eye: (3,2,3)
  - Look at point: (2,1,2)
  - Up vector (0,1,0)



$$u = \left(\frac{1}{\sqrt{2}}, 0, -\frac{1}{\sqrt{2}}\right)$$

$$v = \left(-\frac{1}{\sqrt{6}}, \frac{2}{\sqrt{6}}, -\frac{1}{\sqrt{6}}\right)$$

$$n = \left(\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}\right)$$

$dx = -P_0.u = 0$

$dy = -P_0.v = \frac{2}{\sqrt{6}}$

$dz = -P_0.n = \frac{-8}{\sqrt{3}}$

DIY

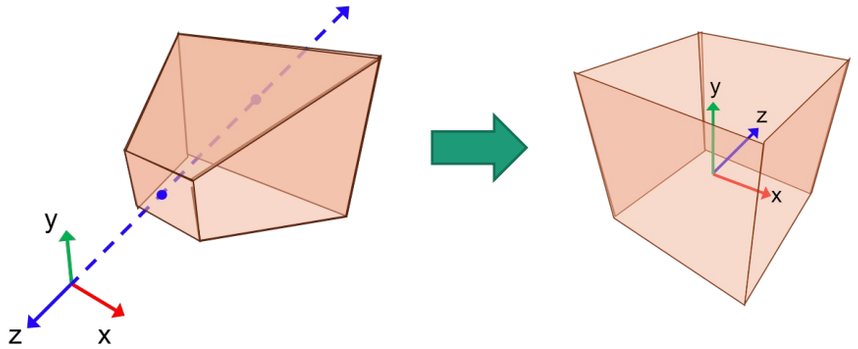
- Find the VCS values for the
  - Point: A(1,0.5,1)
- when
  - Eye: (2,2,3)+last 3 digit of your roll
  - Look at point: (2,1,1)
  - Up vector (0,1,0)

Acknowledgements
cgvr.korea.ac.kr

DIY

10. Write down the 4×4 rotation matrix that takes the orthonormal 3D vectors $\mathbf{u} = (x_u, y_u, z_u)$, $\mathbf{v} = (x_v, y_v, z_v)$, and $\mathbf{w} = (x_w, y_w, z_w)$, to orthonormal 3D vectors $\mathbf{a} = (x_a, y_a, z_a)$, $\mathbf{b} = (x_b, y_b, z_b)$, and $\mathbf{c} = (x_c, y_c, z_c)$. So $M\mathbf{u} = \mathbf{a}$, $M\mathbf{v} = \mathbf{b}$, and $M\mathbf{w} = \mathbf{c}$.

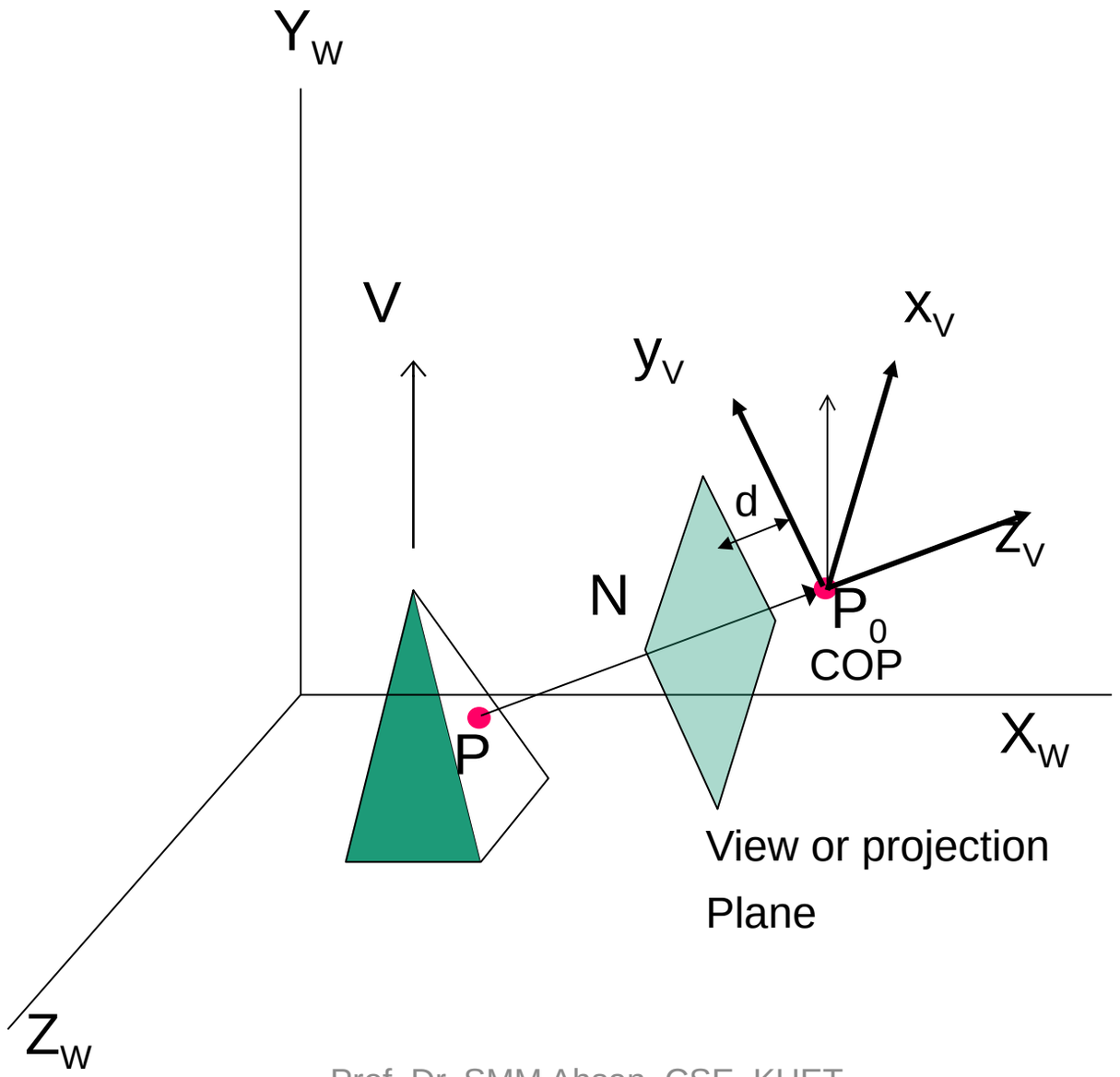
The camera's current position is at (14,25-16), the camera is looking at the point (37,42,20), and the up vector is along *(3i + 5j - 4k).* Find the transformation matrix that would align the direction that the camera is looking at along the negative Y axis, and align the direction that points to the right of the camera along the negative X axis.



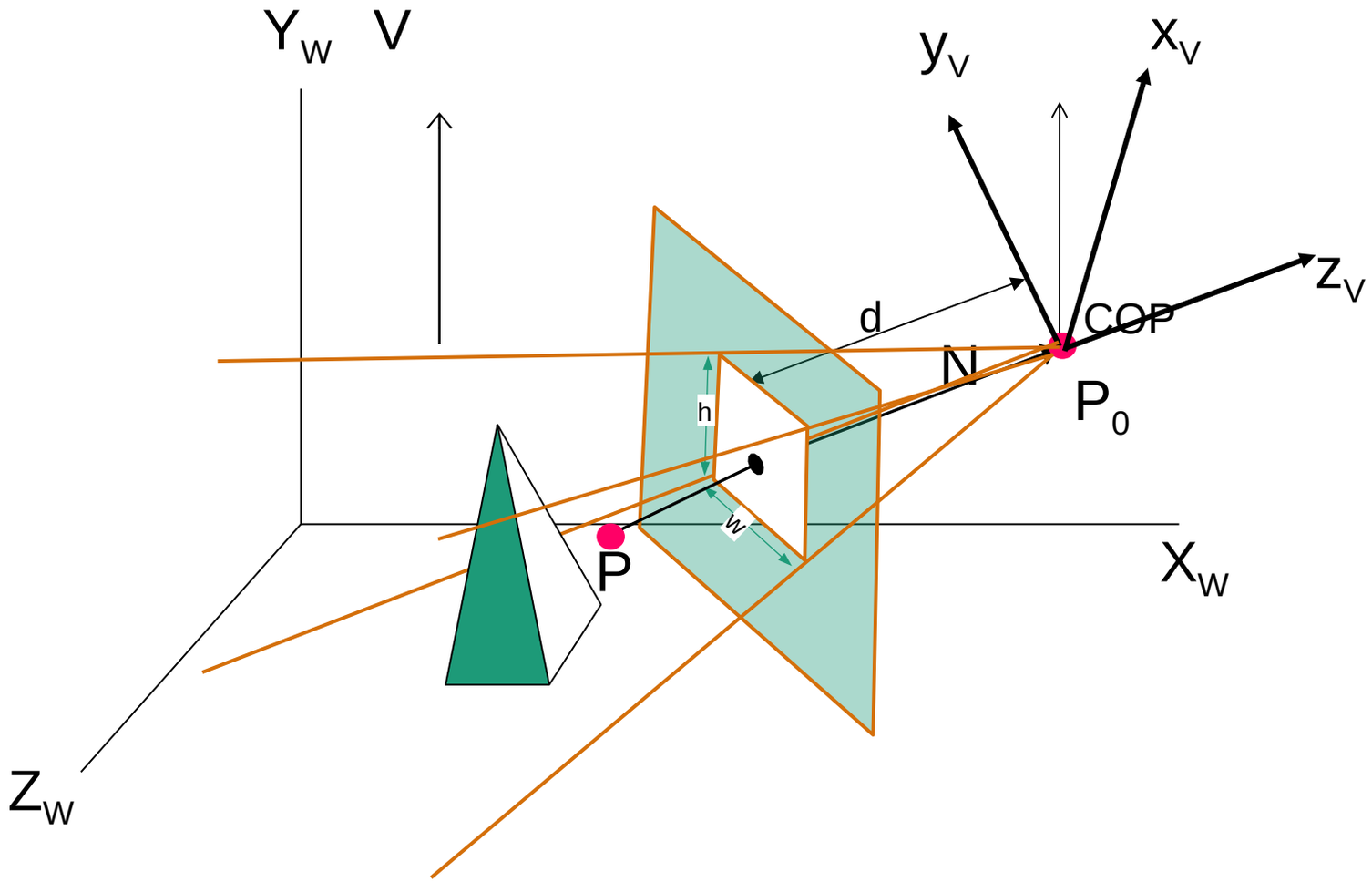
Projective Transformation

Dr. Sk. Md. Masudul Ahsan
Professor
Dept. of CSE, KUET

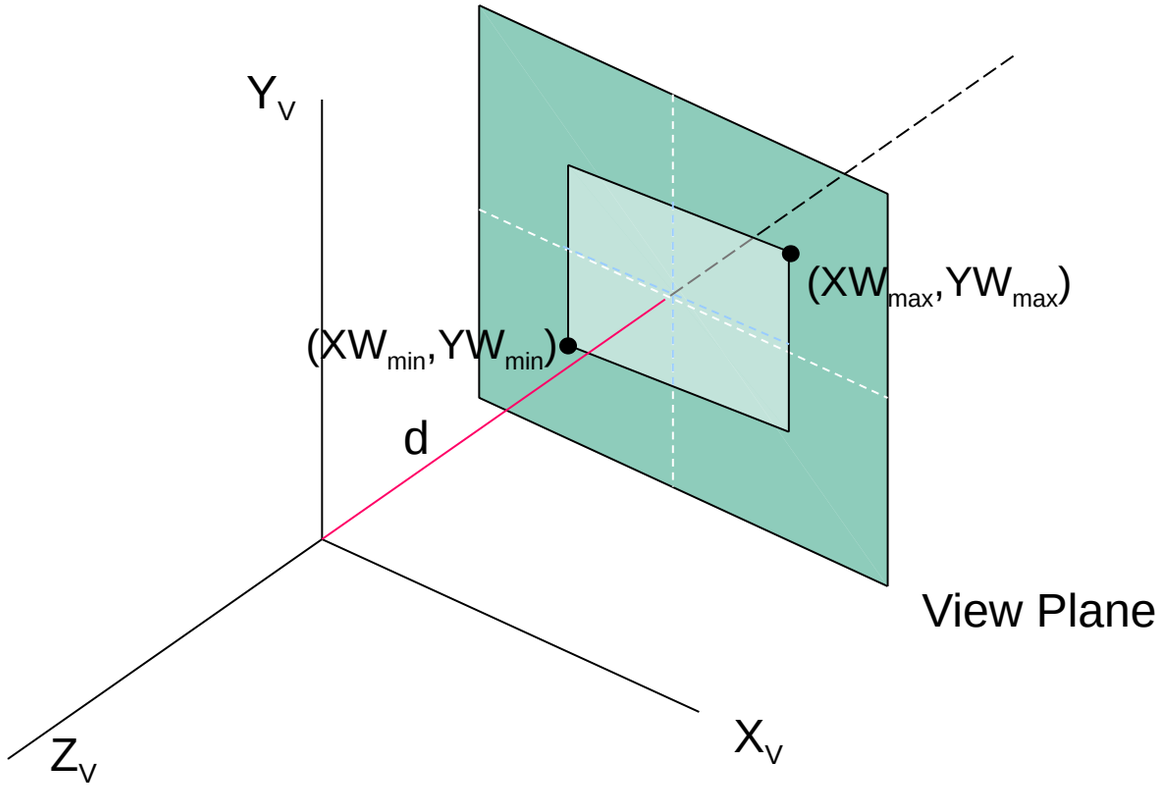
WCS, VCS and projection



View Window



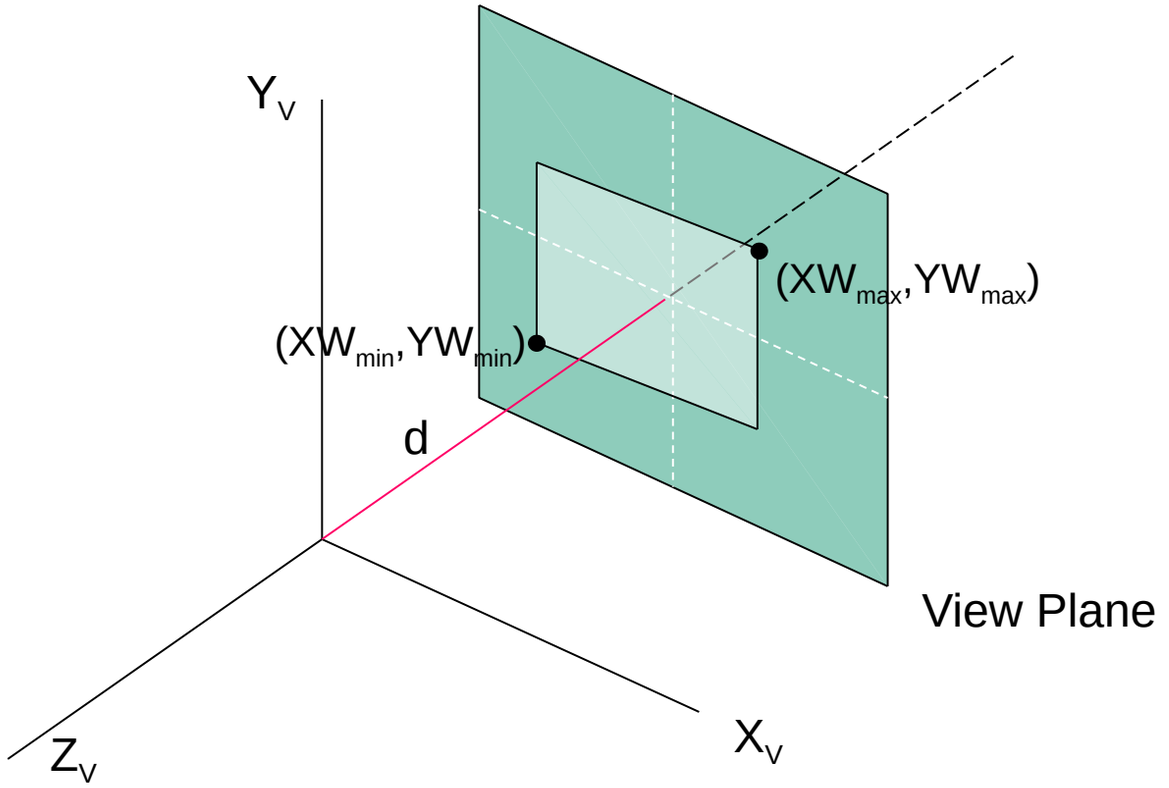
View Window - symmetric



$XW_{max} = -XW_{min}$,

$YW_{max} = -YW_{min}$

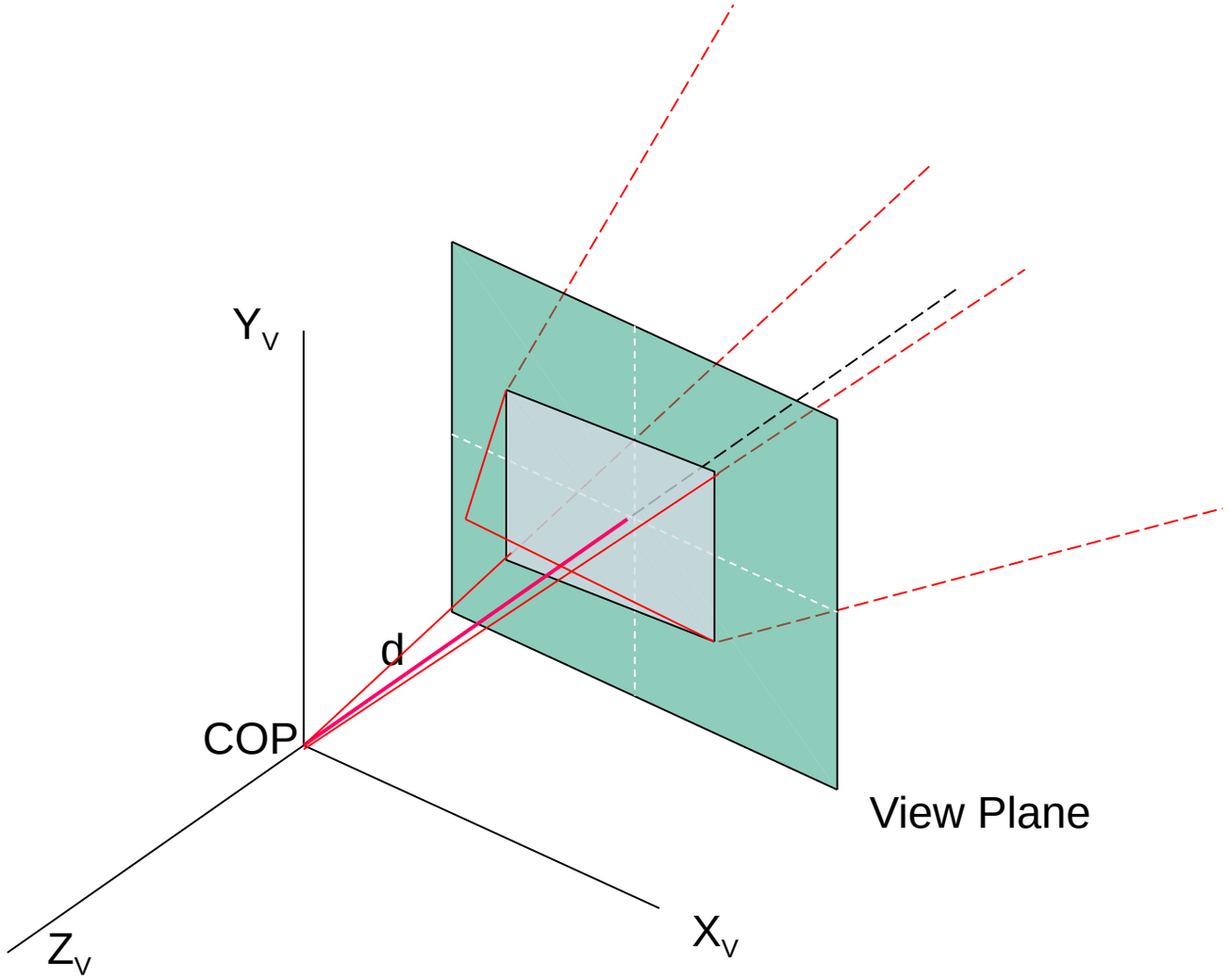
View Window - asymmetric



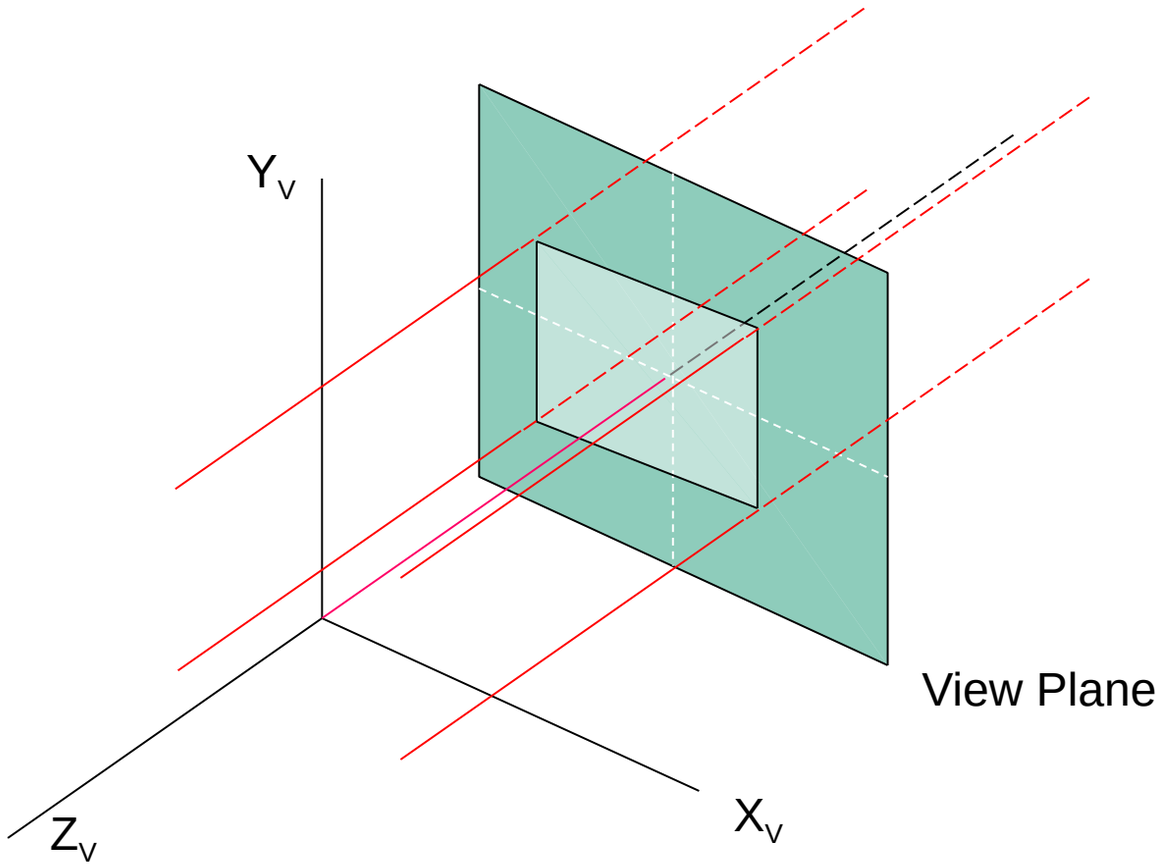
XW_{max} != $-XW_{min}$,

YW_{max} != $-YW_{min}$

View Volume - Perspective



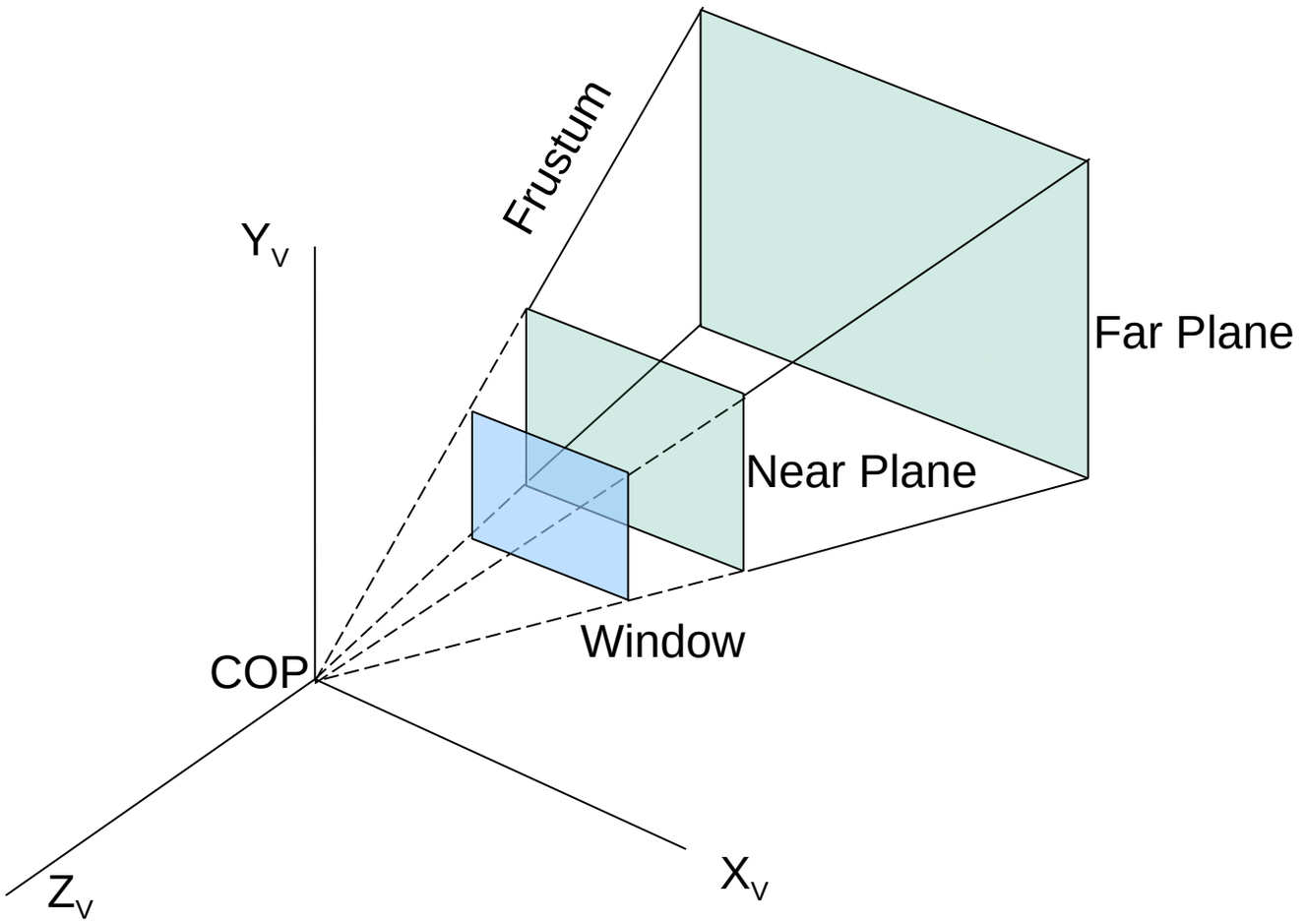
View Volume – Orthographic Projection



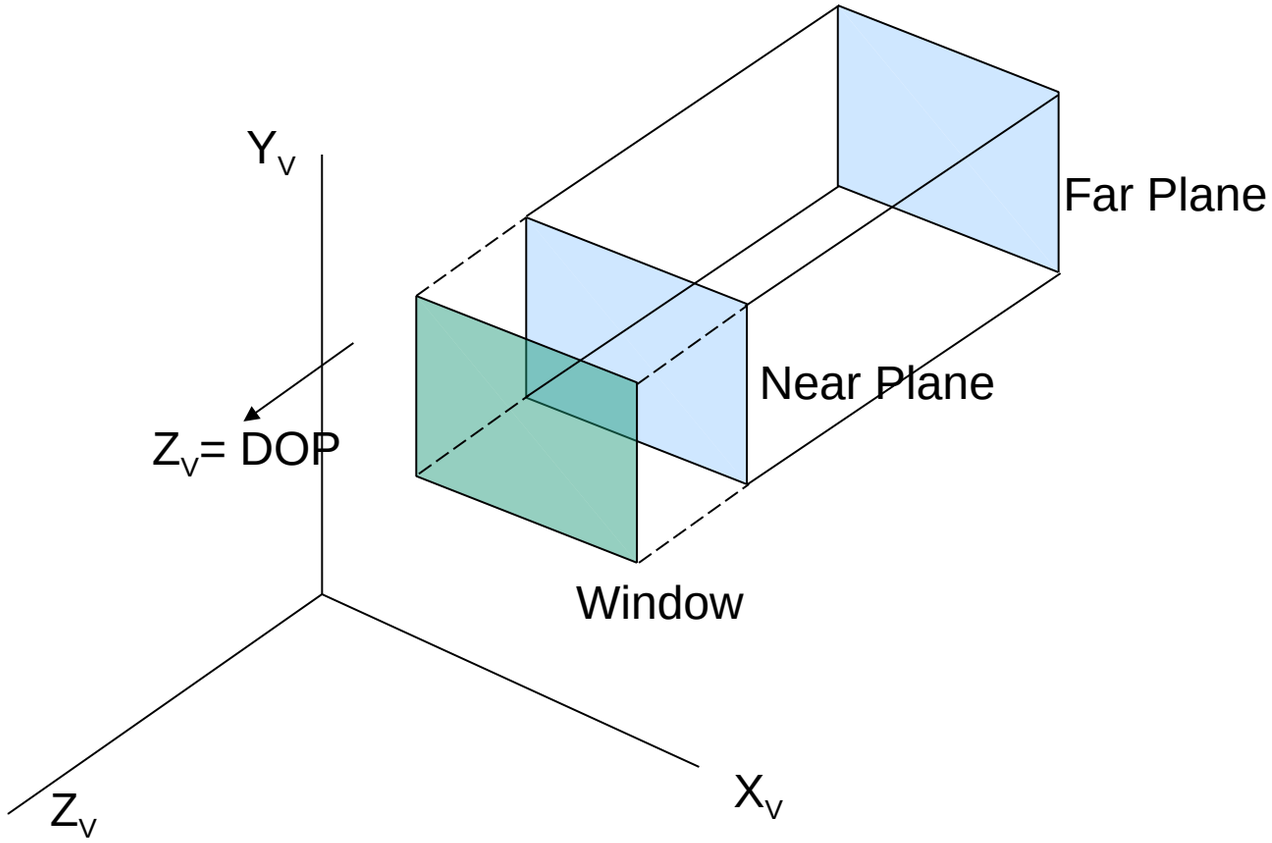
View Volume

- Finite ~ is obtained by limiting the extent of volume in Z_V direction
- Two plane parallel to view plane are used for this purpose, planes are –
  - **Near / Front plane** and **Far / Back plane**
  - Both must be same side of COP
- Adv. of using **Near, Far plane**
  - These planes eliminates parts of the scene from viewing operation based on depth.
  - Special adv. in case of perspective projection

Finite View Volume - Perspective

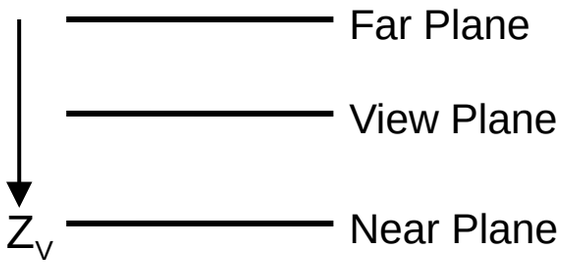


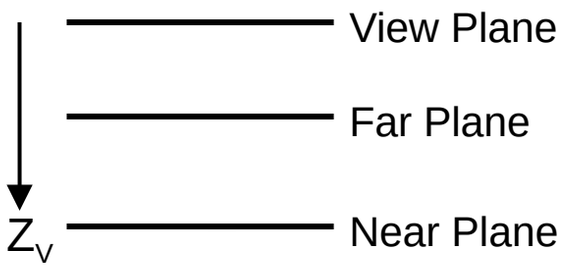
Finite View Volume – Orthographic Projection

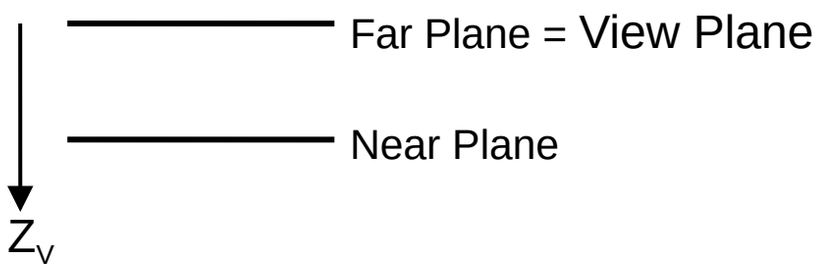


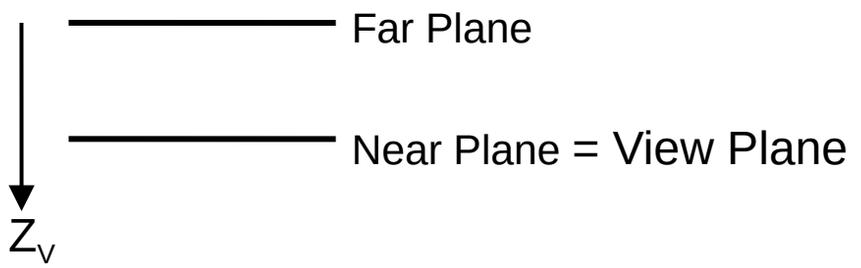
Relative placement

- ~ of **View, Near, Far planes** depend on
  - type of view we want &
  - limitation of Graphics package





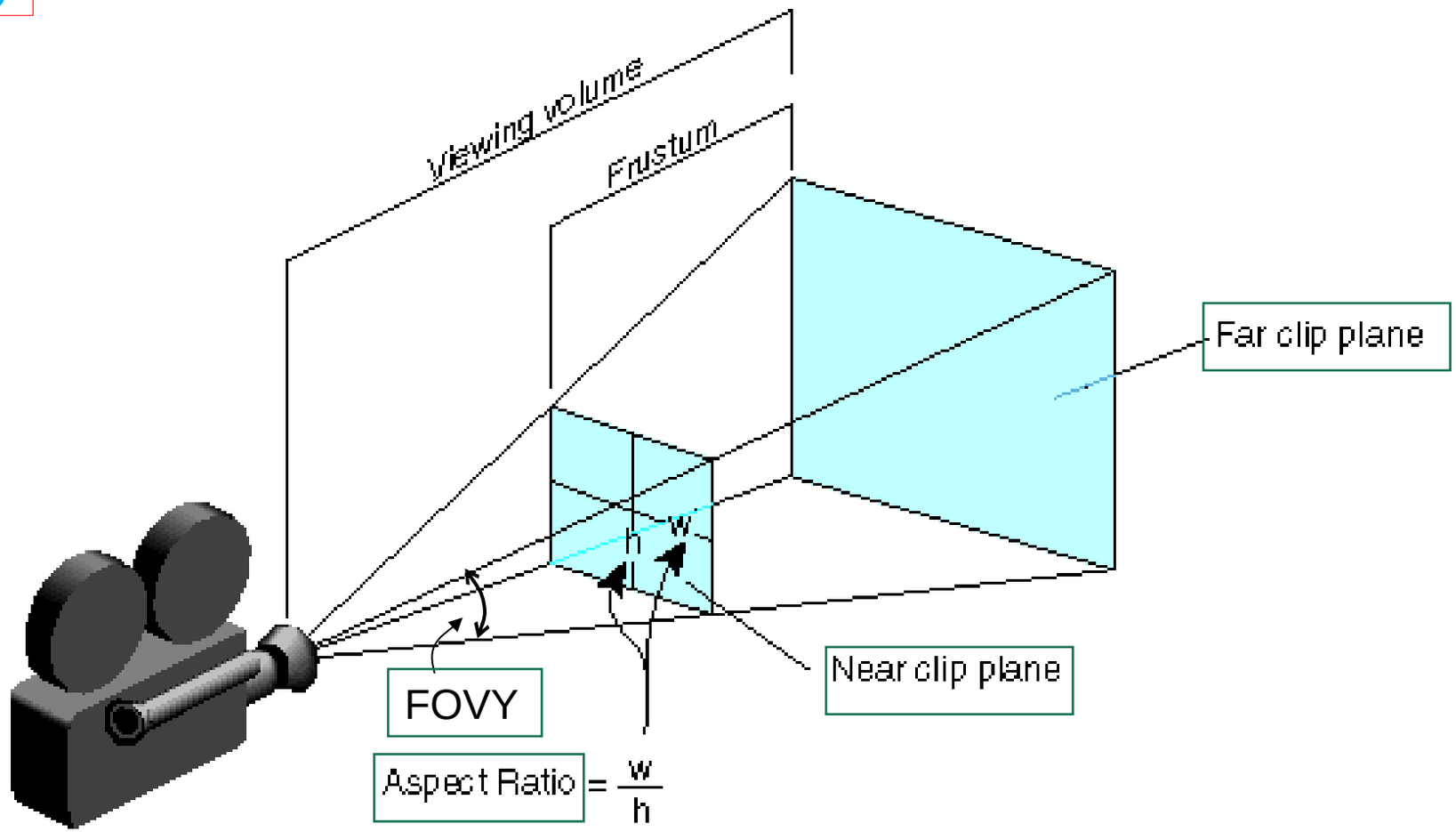




OpenGL Uses it

Symmetric view volume - Perspective

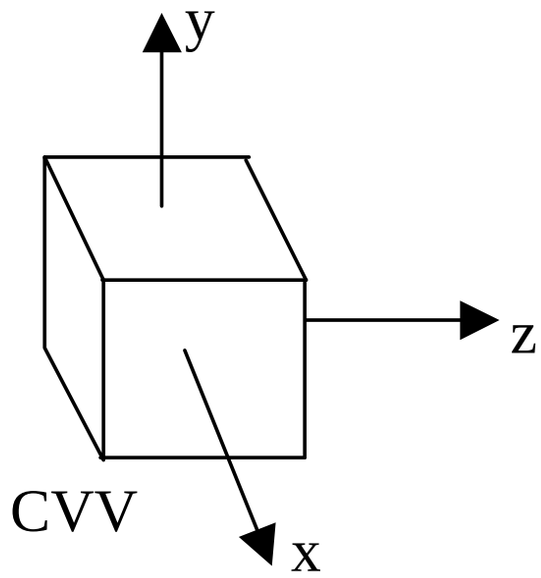
Another way



$h = 2d * \tan(fovy/2)$

Normalization Transformation

- Transforms VCS values to NDCS one
- Canonical View Volumes
  - The 3D object model is not actually clipped inside the projection view volume.
  - The view volume is instead mapped to a **canonical view volume** (CVV) which is a cube that extends from -1 to +1 in each dimension, having center at the origin.
  - The dimensions of the CVV facilitates a fast and efficient clipping.



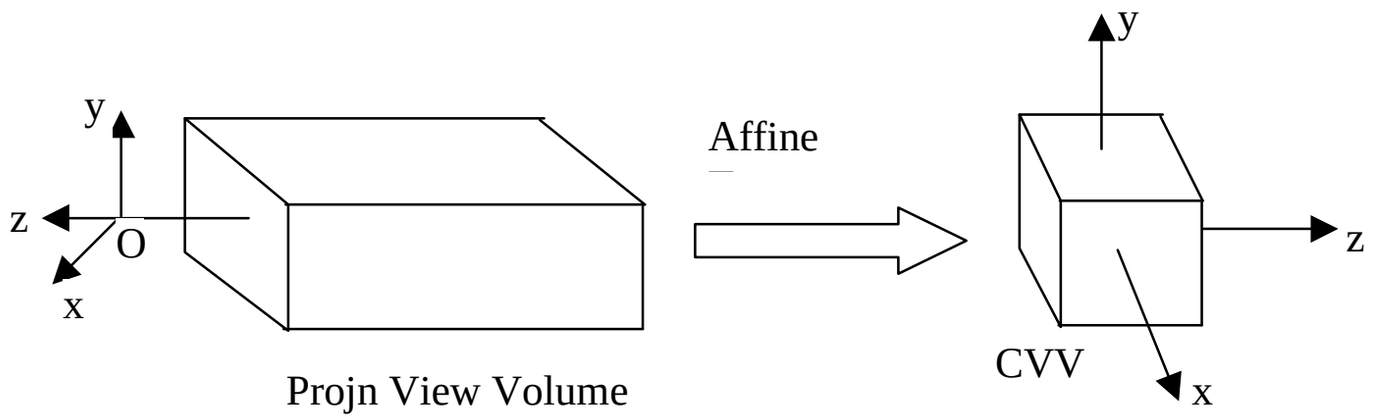
$$-1 \leq x \leq 1$$

$$-1 \leq y \leq 1$$

$$-1 \leq z \leq 1$$

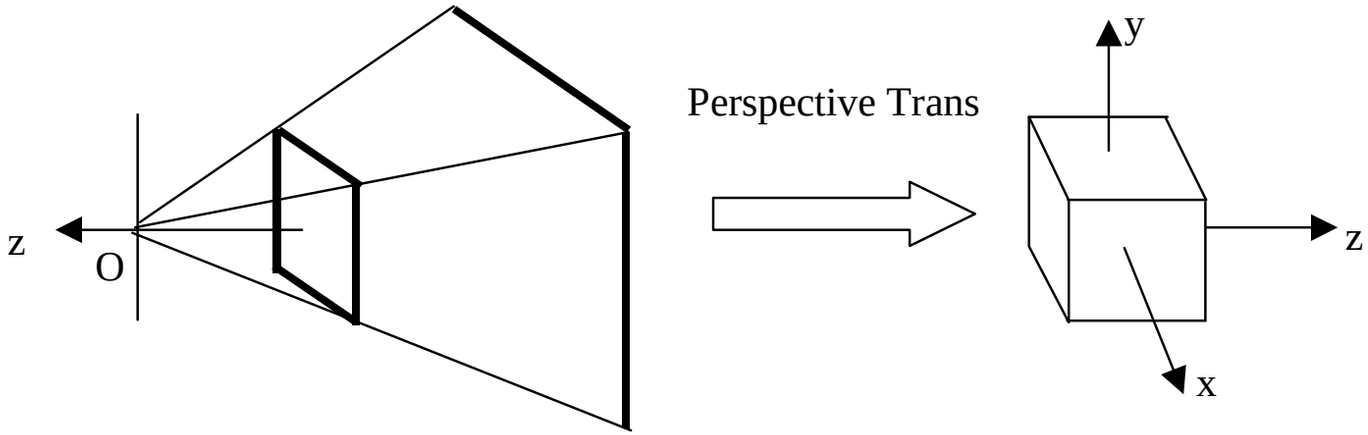
Mapping to CVV

Orthographic:



Projn View Volume

Perspective:



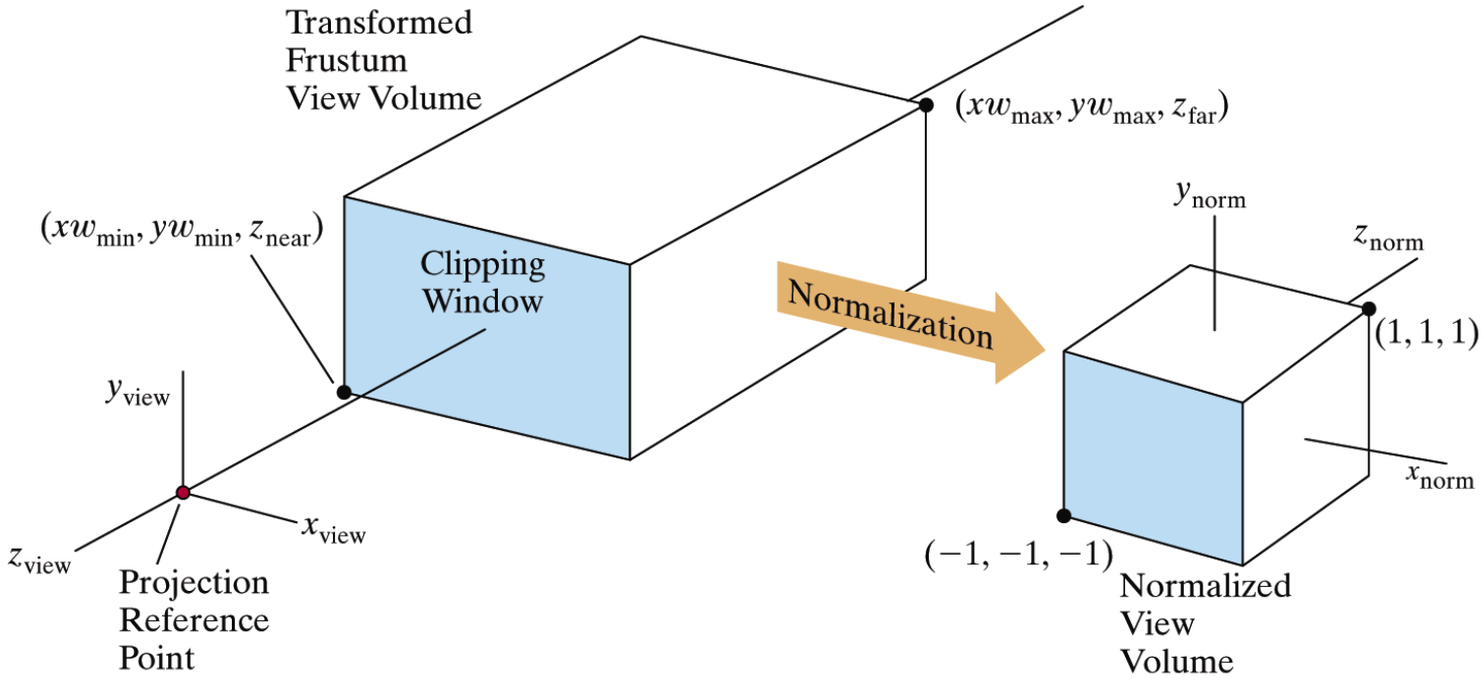
Projn View Volume

CVV

Normalization lets us clip against simple cube regardless of type of projection

Orthographic Transform

- The parallelepiped view volume is squeezed (scaled) around position (0, 0, 0) and the z axis is reversed

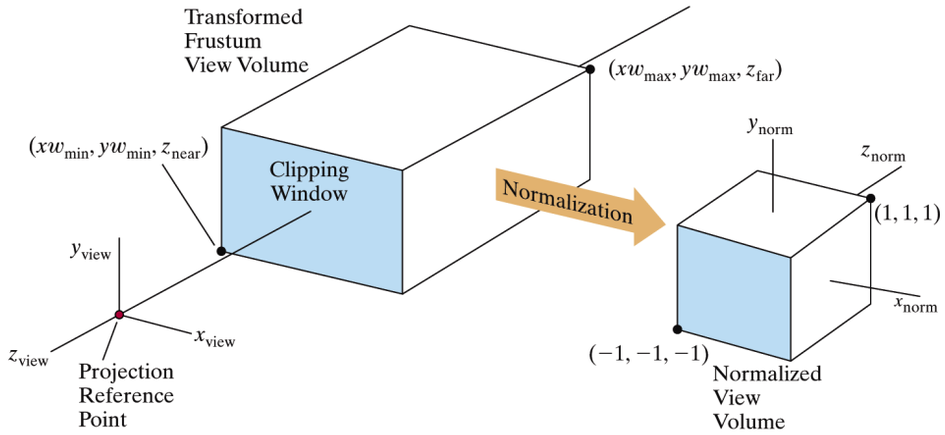


Orthographic Transform

$$T\left(-\frac{r+l}{2}, -\frac{t+b}{2}, \frac{|n|+|f|}{2}\right)$$

$$S\left(\frac{2}{r-l}, \frac{2}{t-b}, \frac{-2}{|f|-|n|}\right)$$

$$M_{orth,norm} = S.T$$



$$M_{orth,norm} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{2}{|n|-|f|} & \frac{|n|+|f|}{|n|-|f|} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Here,

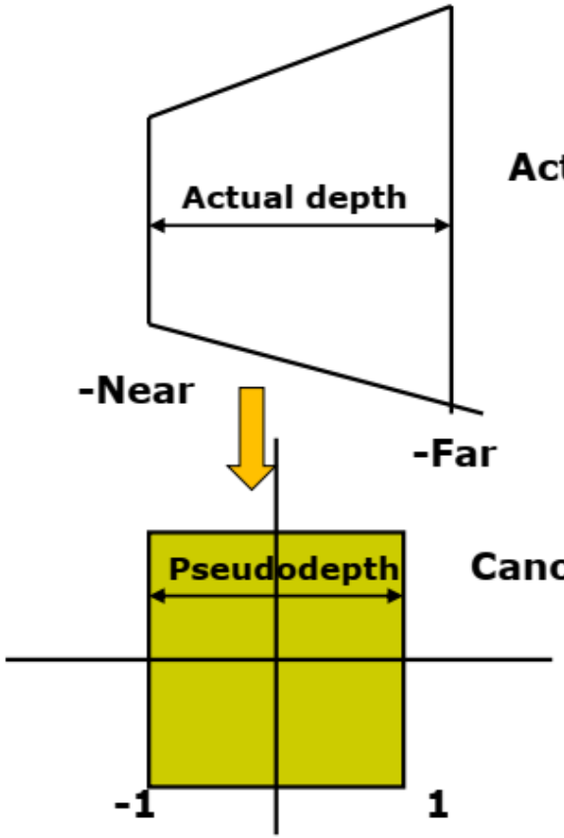
f<n<0

Or

0<|n|<|f|

Perspective Transform

- **Perspective transformation** maps actual z distance of perspective view volume to range [–1 to 1] (**Pseudodepth**) for canonical view volume



Actual view volume

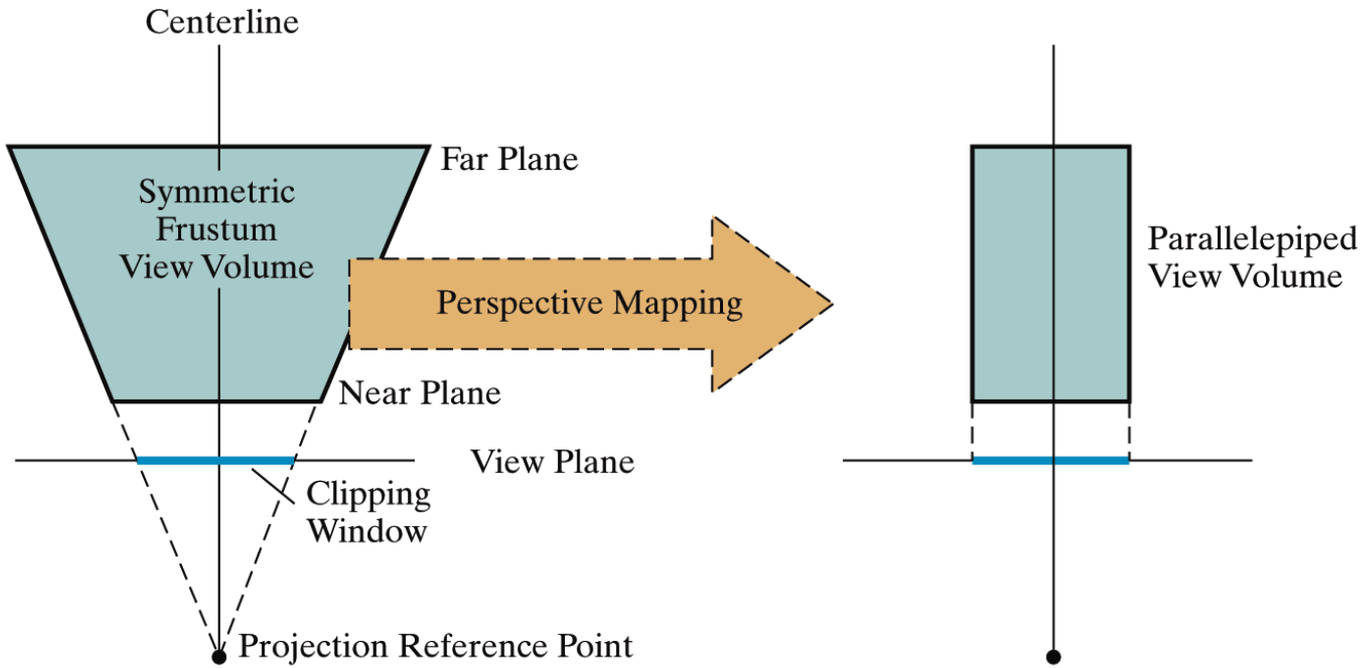
Canonical view volume

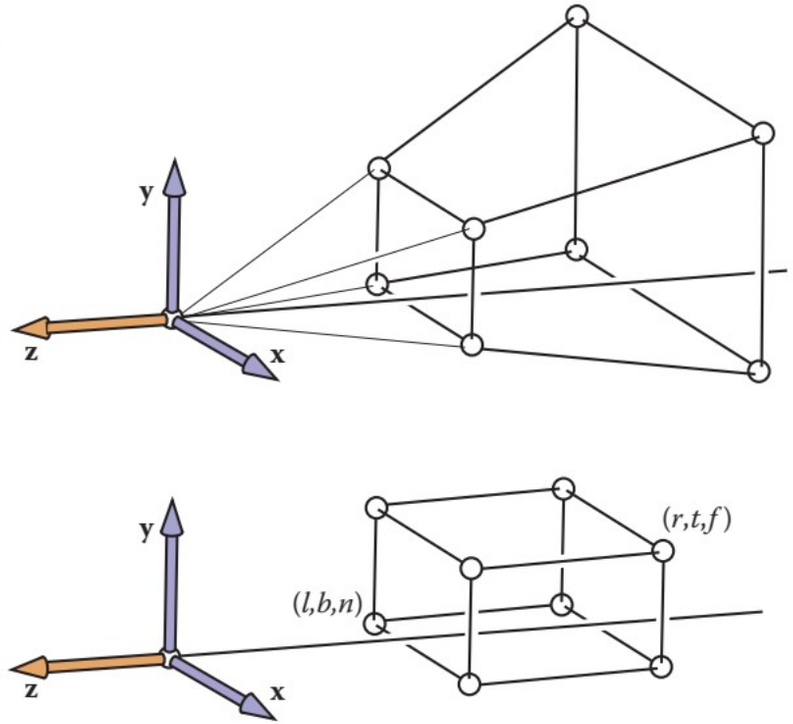
We want perspective Transformation and NOT classical projection!!

Set scaling z
Pseudodepth = az + b
Next solve for a and b

Perspective Transform

- The frustum shaped viewing volume has been converted to a parallelepiped
  - remember we preserved all z coordinate depth information
  - How?
- And then parallelepiped volume is mapped to CVV as





Perspective without Depth

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -\frac{1}{d} & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ -\frac{z}{d} \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \\ z \\ -\frac{z}{d} \end{pmatrix} \xrightarrow{\text{perspective division}} \begin{pmatrix} \frac{x}{-(z/d)} \\ \frac{y}{-(z/d)} \\ -d \\ 1 \end{pmatrix}$$

- The depth information is lost as the last two components are same
- But depth information of the projected points is essential for hidden surface removal and other purposes like blending, shading etc.

Perspective with Depth

- Sufficient to use **pseudodepth**
  - what is a good choice? Can we use z as pseudodepth? (farther point has more negative z value)
  - $z' = \alpha z + \beta$ which is a linear function before perspective division

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -\frac{1}{d} & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ \alpha z + \beta \\ -\frac{z}{d} \end{pmatrix} \xrightarrow{\text{perspective division}} \begin{pmatrix} \frac{x}{-(z/d)} \\ \frac{y}{-(z/d)} \\ -\frac{d(\alpha z + \beta)}{z} \\ 1 \end{pmatrix}$$

$$z' = -d \cdot \alpha - \frac{d \cdot \beta}{z}$$

For $\beta < 0$, z' is a monotonically increasing function of d*epth*.

Perspective Transform - Frustum to Parallelepiped

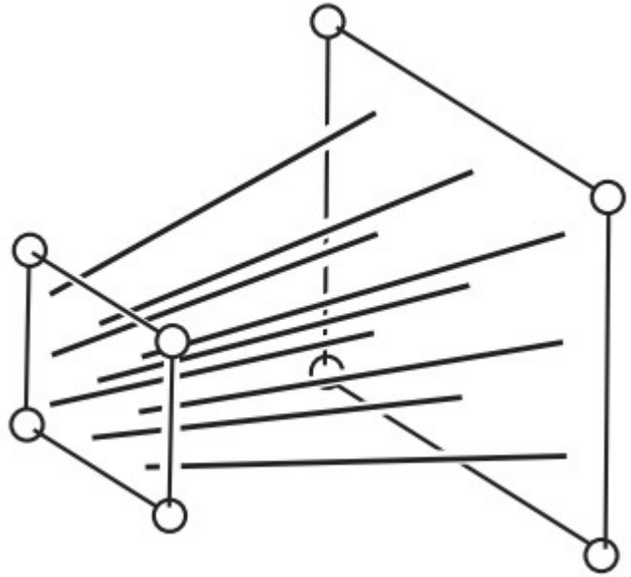
Rearrange the perspective transformation matrix

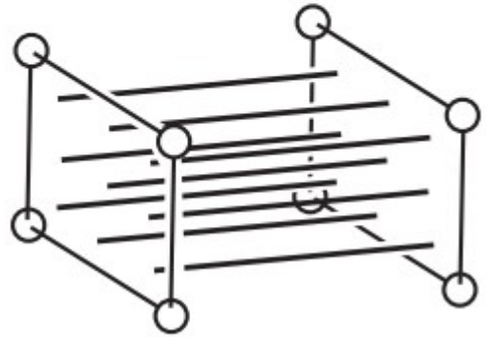
$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -\frac{1}{d} & 0 \end{pmatrix} \cong \begin{pmatrix} |n| & 0 & 0 & 0 \\ 0 & |n| & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Perspective Transform - Frustum to Parallelepiped

Chose α, β, such a way that a point in near plane / far plane perspectively projects on near / far plane

The perspective projection maps any line through the origin/eye to a line parallel to the z-axis and without moving the point on the line at z = n.





$$\begin{pmatrix} |n| & 0 & 0 & 0 \\ 0 & |n| & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} r & rf/n \\ t & tf/n \\ -|n| & -|f| \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} |n|r & rf \\ |n|t & tf \\ -\alpha|n| + \beta & -\alpha|f| + \beta \\ |n| & |f| \end{pmatrix} = \begin{pmatrix} r & r \\ t & t \\ \frac{-\alpha|n|+\beta}{|n|} & \frac{-\alpha|f|+\beta}{|f|} \\ 1 & 1 \end{pmatrix}$$

Perspective Transform - Frustum to Parallelepiped

Chose α, β, such a way that a point in near plane / far plane perspectively projects on near / far plane

$$\begin{pmatrix} |n| & 0 & 0 & 0 \\ 0 & |n| & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} r & rf/n \\ t & tf/n \\ -|n| & -|f| \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} |n|r & rf \\ |n|t & tf \\ -\alpha|n| + \beta & -\alpha|f| + \beta \\ |n| & |f| \end{pmatrix} = \begin{pmatrix} r & r \\ t & t \\ \frac{-\alpha|n|+\beta}{|n|} & \frac{-\alpha|f|+\beta}{|f|} \\ 1 & 1 \end{pmatrix}$$

Apply Boundary Condition

$$\frac{-\alpha|n| + \beta}{|n|} = -|n|$$

$$=> -\alpha|n| + \beta = -|n|^2$$

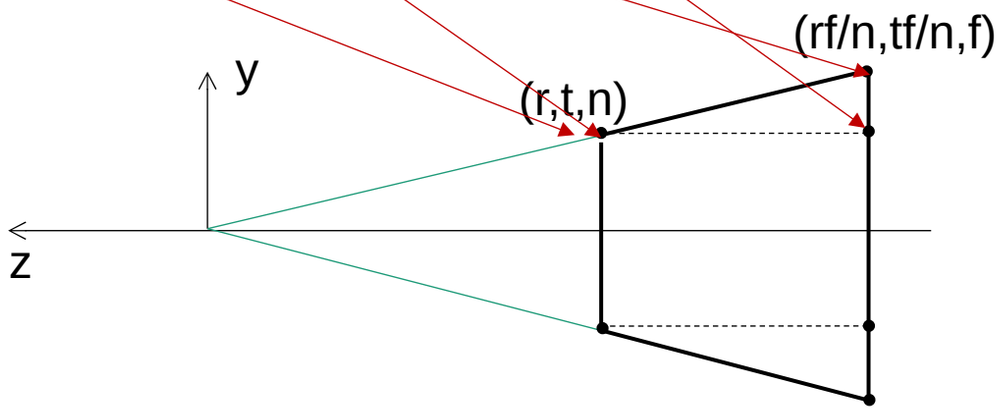
Similarly

$$-\alpha|f| + \beta = -|f|^2$$

Solving

$$\alpha = |n| + |f|$$

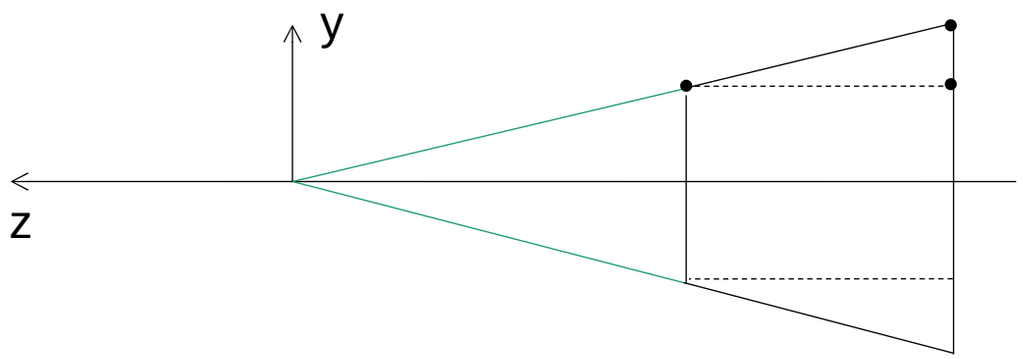
$$\beta = |n||f|$$



Perspective Transform - Parallelepiped to CVV

$$M_{per,norm} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{2}{|n|-|f|} & \frac{|n|+|f|}{|n|-|f|} \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} |n| & 0 & 0 & 0 \\ 0 & |n| & 0 & 0 \\ 0 & 0 & |n|+|f| & |n||f| \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

$$= \begin{bmatrix} \frac{2|n|}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2|n|}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{|n|+|f|}{|n|-|f|} & \frac{2|f||n|}{|n|-|f|} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$



Perspective transform

If window is symmetric, $r = -l$ and $t = -b$, then

$$M_{per,norm} = \begin{bmatrix} \frac{2|n|}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2|n|}{t-b} & 0 & 0 \\ 0 & 0 & \frac{|n|+|f|}{|n|-|f|} & \frac{2|f||n|}{|n|-|f|} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

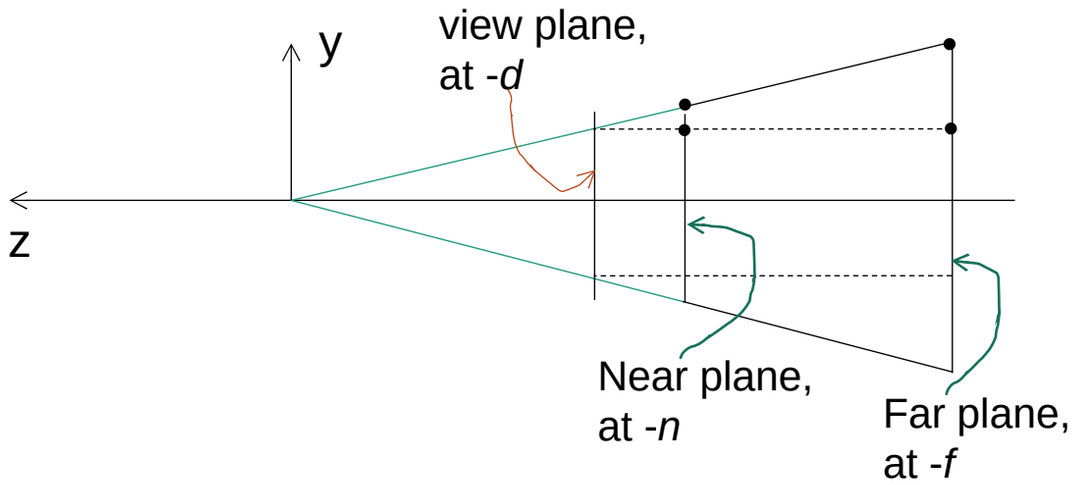
symmetric

$$M_{per,norm} = \begin{bmatrix} \frac{2|n|}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2|n|}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{|n|+|f|}{|n|-|f|} & \frac{2|f||n|}{|n|-|f|} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Asymmetric / general

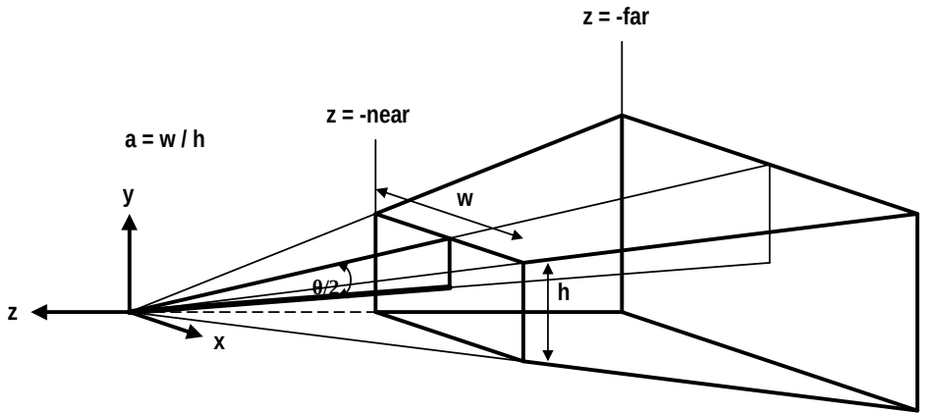
Perspective Transform - Parallelepiped to CVV

What if, view plane ≠ near plane



Perspective transform

$$M_{per,norm} = \begin{bmatrix} \frac{2|n|}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2|n|}{t-b} & 0 & 0 \\ 0 & 0 & \frac{|n|+|f|}{|n|-|f|} & \frac{2|f||n|}{|n|-|f|} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$



Window is symmetric, but defined by aspect ratio a, fovy θ, then

$$c = \cot\left(\frac{\theta}{2}\right) = \frac{|n|}{t} \Rightarrow t = \frac{|n|}{c}; \quad b = -\frac{|n|}{c}$$

$$a = \frac{w}{h} = \frac{r}{t} \Rightarrow r = at = \frac{a|n|}{c}; \quad l = -\frac{a|n|}{c}$$

$$M_{per,norm} = \begin{bmatrix} \frac{c}{a} & 0 & 0 & 0 \\ 0 & c & 0 & 0 \\ 0 & 0 & \frac{|n|+|f|}{|n|-|f|} & \frac{2|f||n|}{|n|-|f|} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Perspective Transformation Matrix

Another way

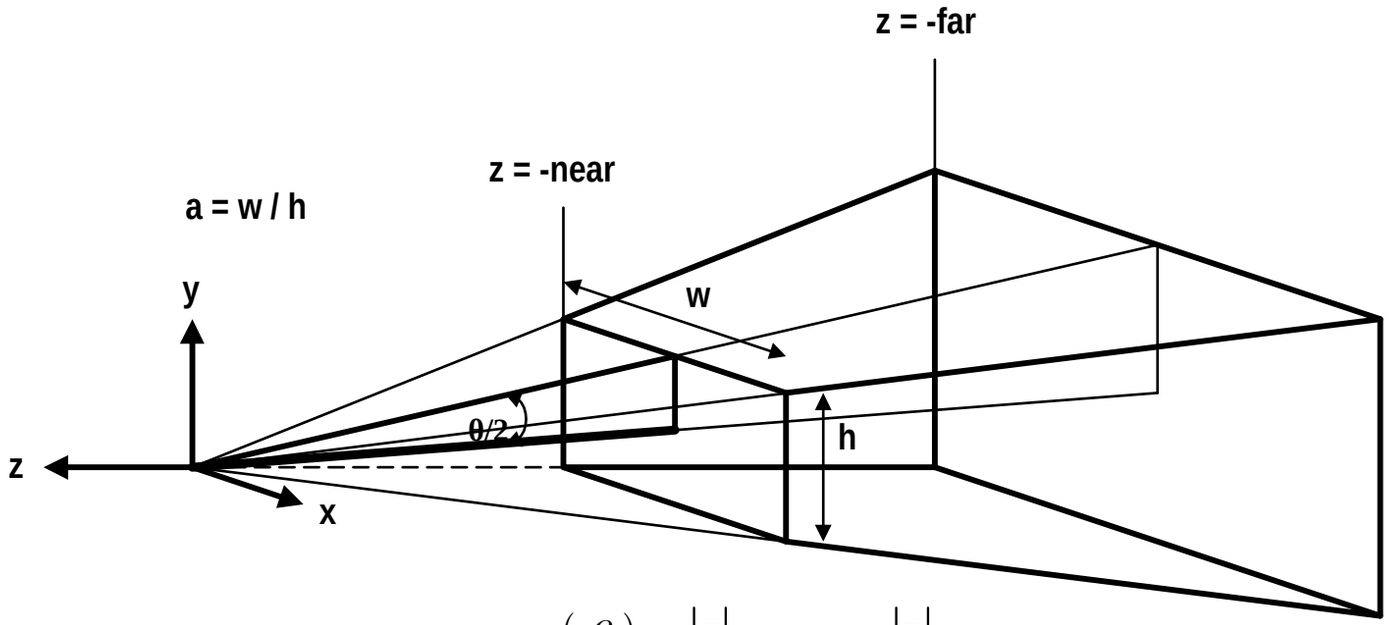
The matrix to perform ***perspective transformation***:

$$\begin{pmatrix} \gamma & 0 & 0 & 0 \\ 0 & \delta & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} \gamma \cdot x \\ \delta \cdot y \\ \alpha \cdot z + \beta \\ -z \end{pmatrix}$$

$$= \begin{pmatrix} \gamma \cdot x/(-z) \\ \delta \cdot y/(-z) \\ (\alpha \cdot z + \beta)/(-z) \\ 1 \end{pmatrix}$$

Prof. Dr. SMM Ahsan, CSE, KUET

Perspective Transformation Matrix - Symmetric

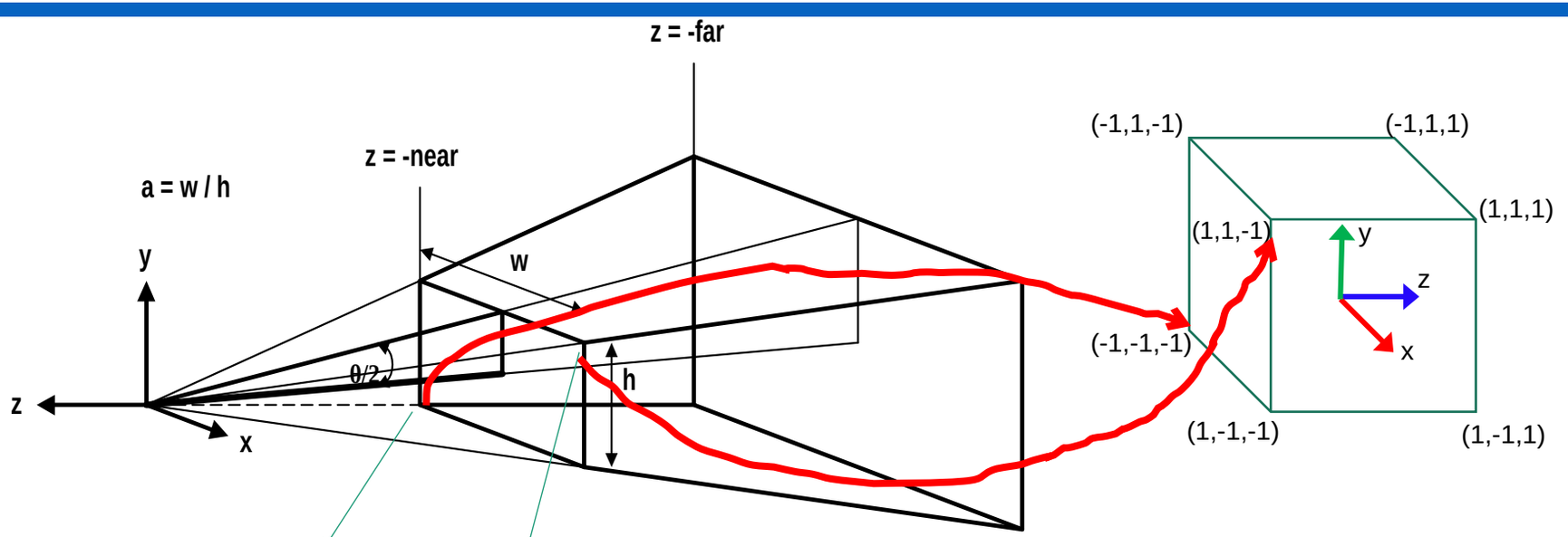


$$c = \cot\left(\frac{\theta}{2}\right) = \frac{|z|}{|y|} \Rightarrow |y| = \frac{|z|}{c}$$

$$a = \frac{w}{h} = \frac{|x|}{|y|} \Rightarrow |x| = a|y| = \frac{a|z|}{c}$$

$$\therefore (x, y, z) \Rightarrow \left(\pm\frac{a|z|}{c}, \pm\frac{|z|}{c}, z \right)$$

Perspective Transformation Matrix - Symmetric



$$\left(-\frac{a|n|}{c},-\frac{|n|}{c},-|n|\right)\rightarrow(-1,-1,-1)$$

$$\left(-\frac{a|n|}{c},\frac{|n|}{c},-|n|\right)\rightarrow(-1,1,-1)$$

$$\left(\frac{a|n|}{c},-\frac{|n|}{c},-|n|\right)\rightarrow(1,-1,-1)$$

$$\left(\frac{a|n|}{c},\frac{|n|}{c},-|n|\right)\rightarrow(1,1,-1)$$

$$\left(-\frac{af}{c},-\frac{f}{c},-f\right)\rightarrow(-1,-1,1)$$

$$\left(-\frac{af}{c},\frac{f}{c},-f\right)\rightarrow(-1,1,1)$$

$$\left(\frac{af}{c},-\frac{f}{c},-f\right)\rightarrow(1,-1,1)$$

$$\left(\frac{af}{c},\frac{f}{c},-f\right)\rightarrow(1,1,1)$$

all f are $|f|$

Prof. Dr. SMM Ahsan, CSE, KUET

Perspective Transformation Matrix - Symmetric

$$\left(-\frac{an}{c},-\frac{n}{c},-n\right)\rightarrow(-1,-1,-1)$$

$$\begin{pmatrix}\gamma & 0 & 0 & 0\\ 0 & \delta & 0 & 0\\ 0 & 0 & \alpha & \beta\\ 0 & 0 & -1 & 0\end{pmatrix}\cdot\begin{pmatrix}-\frac{an}{c}\\ -\frac{n}{c}\\ -n\\ 1\end{pmatrix}=\begin{pmatrix}-1\\ -1\\ -1\\ 1\end{pmatrix}\qquad\Leftrightarrow\begin{pmatrix}-\frac{\gamma an}{c}\\ -\frac{\delta n}{c}\\ -\alpha n+\beta\\ n\end{pmatrix}=\begin{pmatrix}-1\\ -1\\ -1\\ 1\end{pmatrix}$$

$$\Leftrightarrow\begin{pmatrix}-\frac{\gamma a}{c}\\ -\frac{\delta}{c}\\ \frac{-\alpha n+\beta}{n}\\ 1\end{pmatrix}=\begin{pmatrix}-1\\ -1\\ -1\\ 1\end{pmatrix}$$

$$\Rightarrow\ \gamma=\frac{c}{a}$$

$$\Rightarrow\ \delta=c$$

$$\Rightarrow\ -\alpha n+\beta=-n$$

Perspective Transformation Matrix - Symmetric

$$\left(-\frac{af}{c},-\frac{f}{c},-f\right)\rightarrow(-1,-1,1)$$

$$\begin{pmatrix}\gamma & 0 & 0 & 0\\ 0 & \delta & 0 & 0\\ 0 & 0 & \alpha & \beta\\ 0 & 0 & -1 & 0\end{pmatrix}\cdot\begin{pmatrix}-\frac{af}{c}\\ -\frac{f}{c}\\ -f\\ 1\end{pmatrix}=\begin{pmatrix}-1\\ -1\\ 1\\ 1\end{pmatrix}$$

$$\Leftrightarrow\begin{pmatrix}-\frac{\gamma af}{c}\\ -\frac{\delta f}{c}\\ -\alpha f+\beta\\ f\end{pmatrix}=\begin{pmatrix}-1\\ -1\\ -1\\ 1\end{pmatrix}$$

$$\Leftrightarrow\begin{pmatrix}-\frac{\gamma a}{c}\\ -\frac{\delta}{c}\\ \frac{-\alpha f+\beta}{f}\\ 1\end{pmatrix}=\begin{pmatrix}-1\\ -1\\ 1\\ 1\end{pmatrix}$$

$$\Rightarrow -\alpha f+\beta=f$$

$$-\alpha n+\beta=-n$$

$$\therefore\alpha=\frac{f+n}{n-f}$$

$$\therefore\beta=\frac{2fn}{n-f}$$

Perspective Transformation Matrix - Symmetric

The matrix to perform ***perspective transformation***:

$$\begin{pmatrix} \frac{c}{a} & 0 & 0 & 0 \\ 0 & c & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Perspective projection

- Using the Frustum parameters, and considering it a symmetric one
- Final matrix

$$\begin{bmatrix} \frac{2n}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2n}{t-b} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

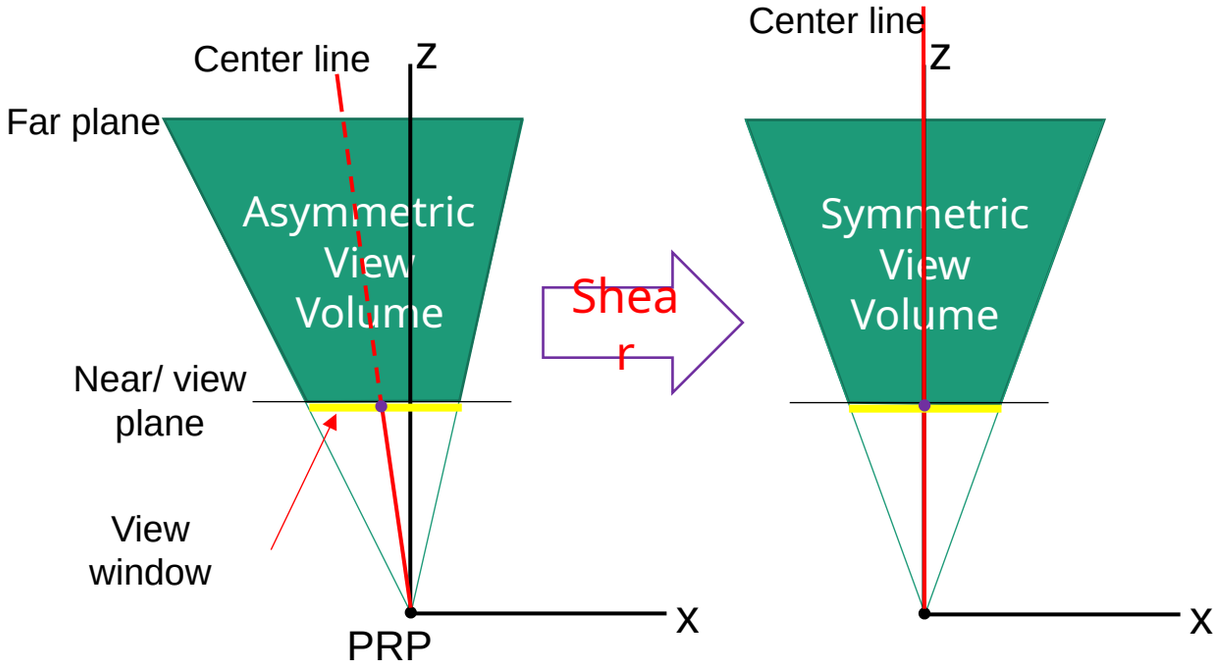
$$\begin{bmatrix} \frac{-2n}{r-l} & 0 & 0 & 0 \\ 0 & \frac{-2n}{t-b} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{-2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Baker Book

This is also true, here all n and f are absolute values

Perspective projection

- The view frustum may not be centered along the view vector, so it needs one more step, first shear the window.



$$\begin{pmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Shear matrix

Shear

$x' = x + az$

$y' = y + bz$

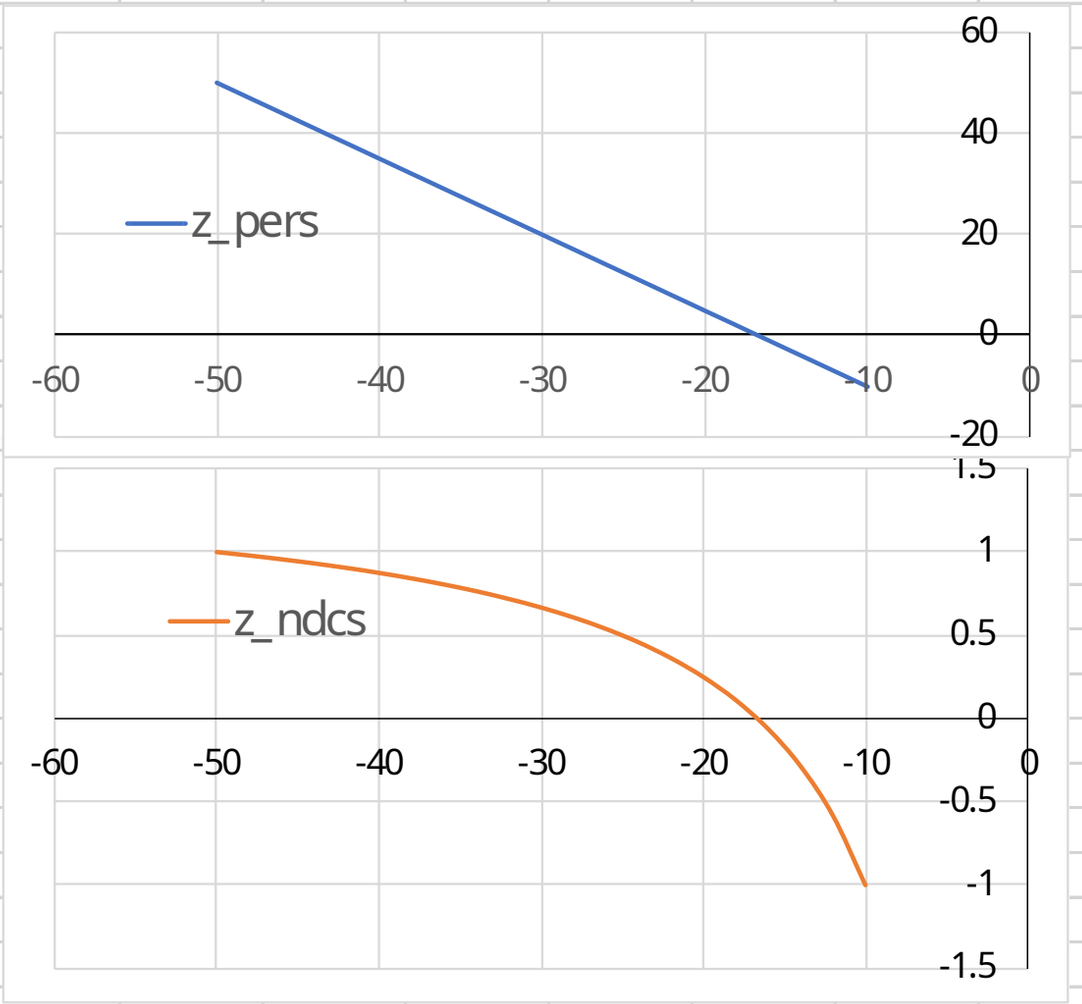
$((r+l)/2, (t+b)/2, -n) \rightarrow (0, 0, -n)$

$a = (r+l)/2n$

$b = (t+b)/2n$

Perspective projection

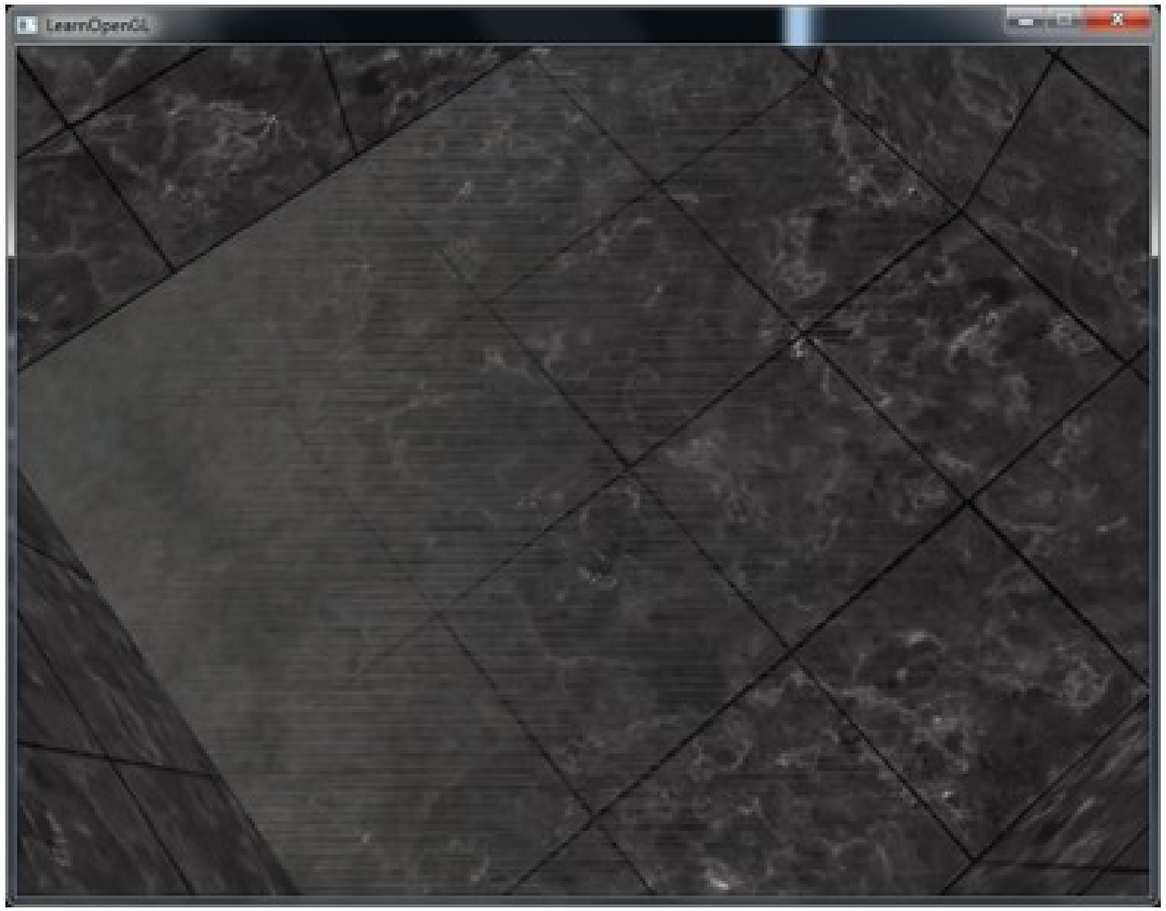
n	-10		alpha	-1.5
f	-50		beta	-25
z_eye	**z_pers**	**z_ndcs**	**p_divisor**	
-10	-10	-1	10	
-12	-7	-0.58333	12	
-14	-4	-0.28571	14	
-16	-1	-0.0625	16	
-18	2	0.111111	18	
-20	5	0.25	20	
-22	8	0.363636	22	
-24	11	0.458333	24	
-26	14	0.538462	26	
-28	17	0.607143	28	
-30	20	0.666667	30	
-32	23	0.71875	32	
-34	26	0.764706	34	
-36	29	0.805556	36	
-38	32	0.842105	38	
-40	35	0.875	40	
-42	38	0.904762	42	
-44	41	0.931818	44	
-46	44	0.956522	46	
-48	47	0.979167	48	
-50	50	1	50	



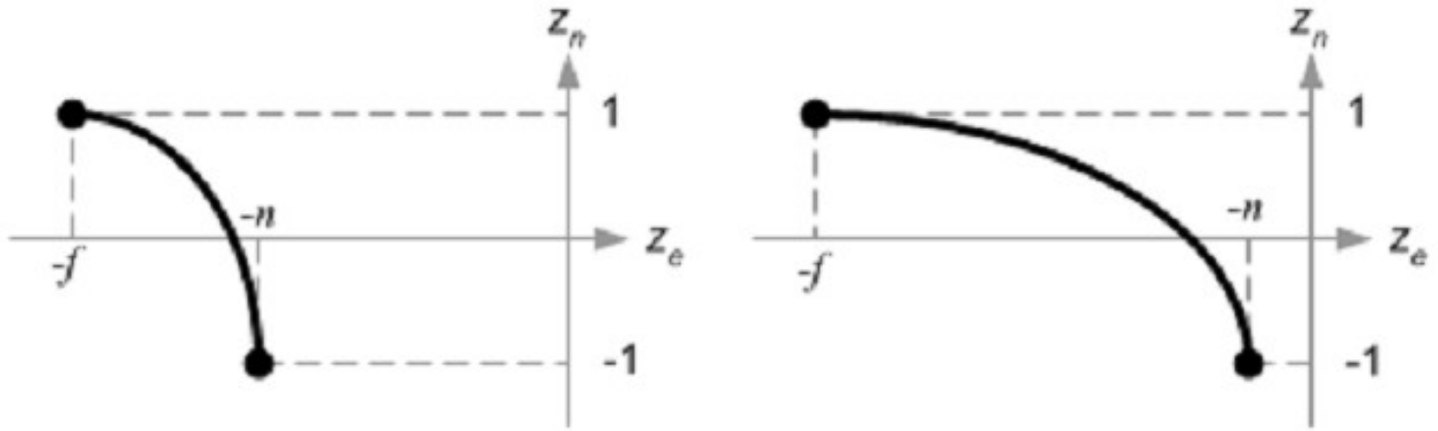
Z-Fighting Problem

- A common visual artifact might occur when two planes are closely aligned to each
  - The depth buffer does not have enough precision to figure out which one is in front of the other.

$$z' = -d \cdot \alpha - \frac{d \cdot \beta}{z}$$



Z-Fighting Problem



Comparison of Depth Buffer Precisions

$$z' = -d \cdot \alpha - \frac{d \cdot \beta}{z}$$

Z	Z′
97	0.9993752
98	0.9995877
99	0.9997959

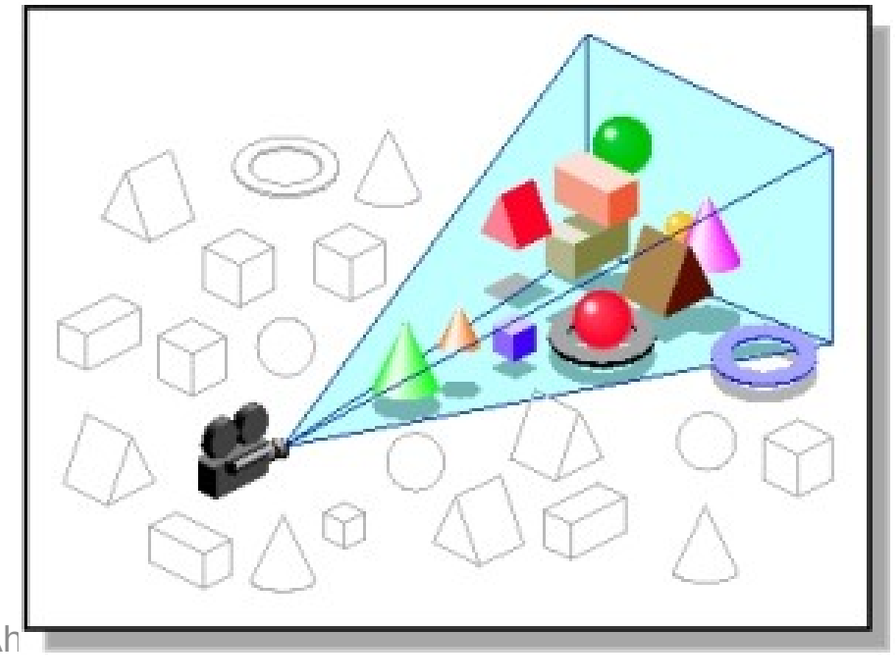
- N=1, F=100, z’ ranges from -1 to +1
- Prevention
  - *set the near plane as far as possible*
  - *never place objects too close to each other in a way that some of them closely overlap*
  - *use a higher precision depth buffer*

WCS, VCS, NDCS and projection

- You are given info about
  - P_0, P, V in WCS
  - d, f, a=h/w, fovy
- Let the apex of the pyramidal object in the scene $(1,2,1)_{WCS}$
- Find the NDCS value of the apex

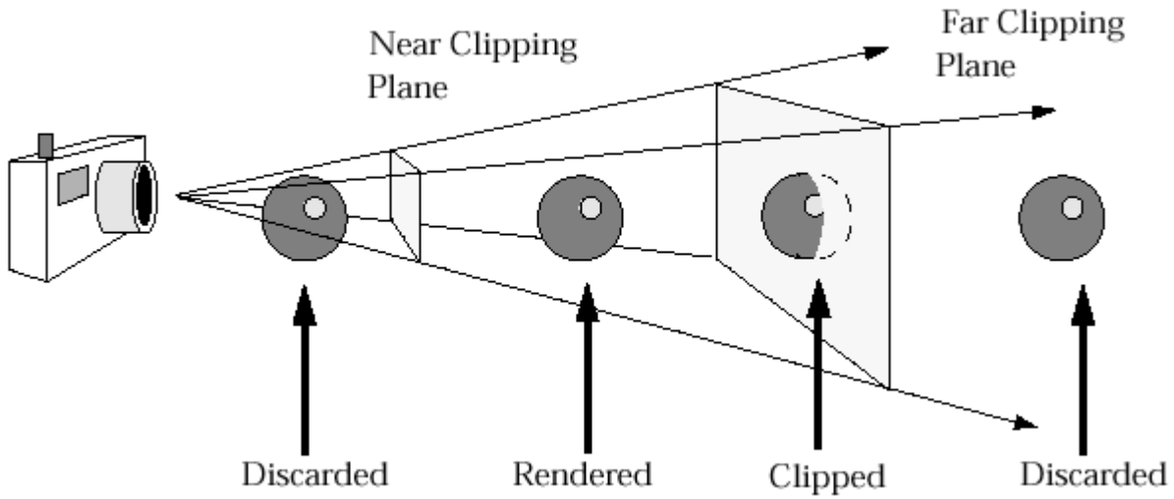
3-D Clipping

- Clipping removes objects that will not be visible
- The point of this is to remove computational effort
- Achieved in two basic steps
  - Discard objects that can’t be viewed
    - i.e. objects that are behind the camera, outside the field of view, or too far away
  - Clip objects that intersect with any clipping plane



3-D Clipping

- Discarding objects that cannot possibly be seen involves comparing an objects bounding box/sphere against the dimensions of the view volume
  - Can be done before or after projection



When Do We Clip?

- We perform clipping after normalisation are done
- So, we have the following:

$$\begin{bmatrix} x_h \\ y_h \\ z_h \\ h \end{bmatrix} = M_{Norm} \cdot \begin{bmatrix} x_e \\ y_e \\ z_e \\ w_e \end{bmatrix}$$

- We apply all clipping to these homogeneous coordinates

How to Clip

- Because we have a normalised clipping volume we can test for these regions as follows:

$$-1 \le \frac{x_h}{h} \le 1 \qquad -1 \le \frac{y_h}{h} \le 1 \qquad -1 \le \frac{z_h}{h} \le 1$$

- Rearranging these we get:

$$-h \le x_h \le h \quad -h \le y_h \le h \quad -h \le z_h \le h \quad \text{if } h > 0$$

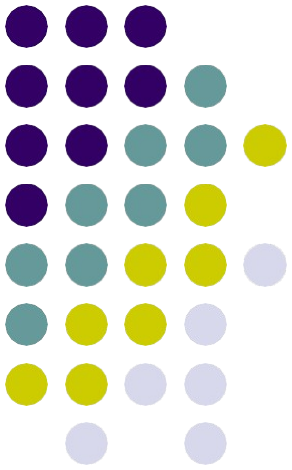
$$h \le x_h \le -h \quad h \le y_h \le -h \quad h \le z_h \le -h \quad \text{if } h < 0$$

- Objects that are partially within the viewing volume need to be clipped – just like the 2D case

Clipping

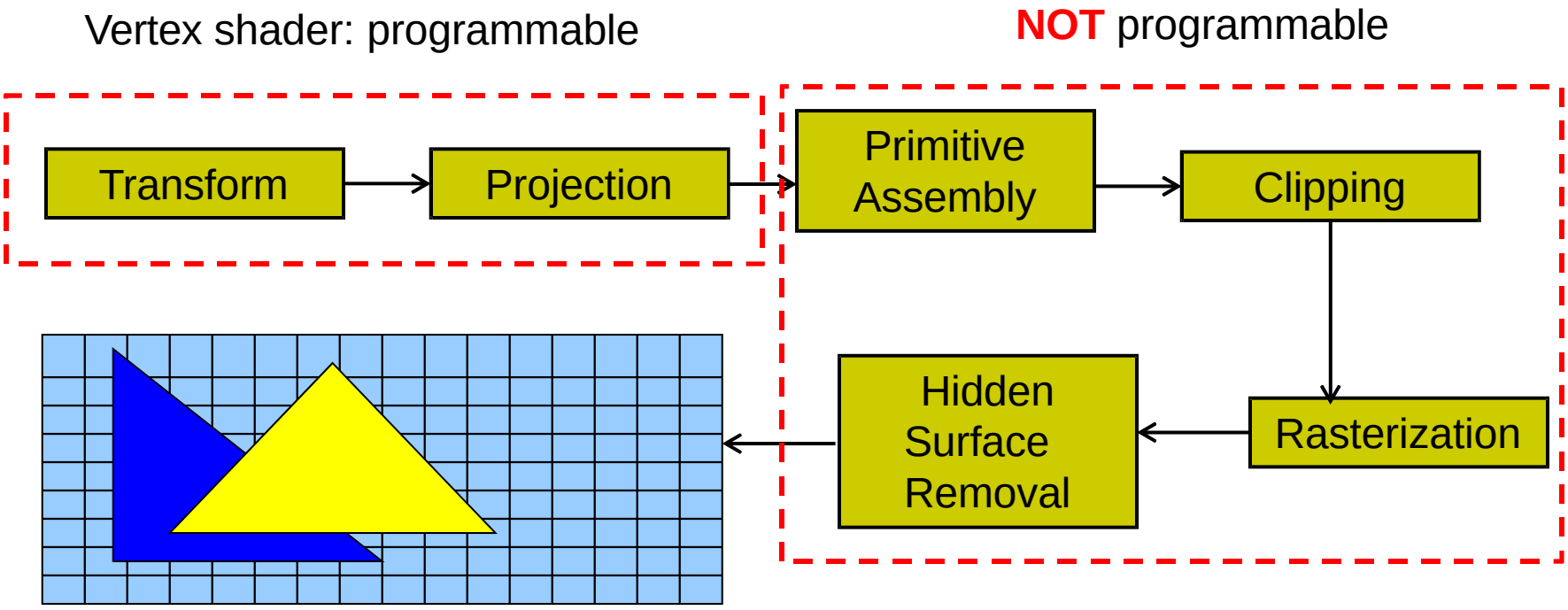
Nazirul Hasan

Lecturer
Dept. of Computer Science and Engineering
Khulna University of Engineering & Technology

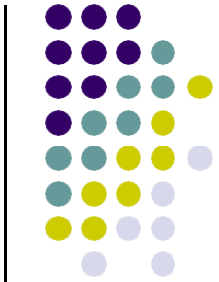


OpenGL Stages

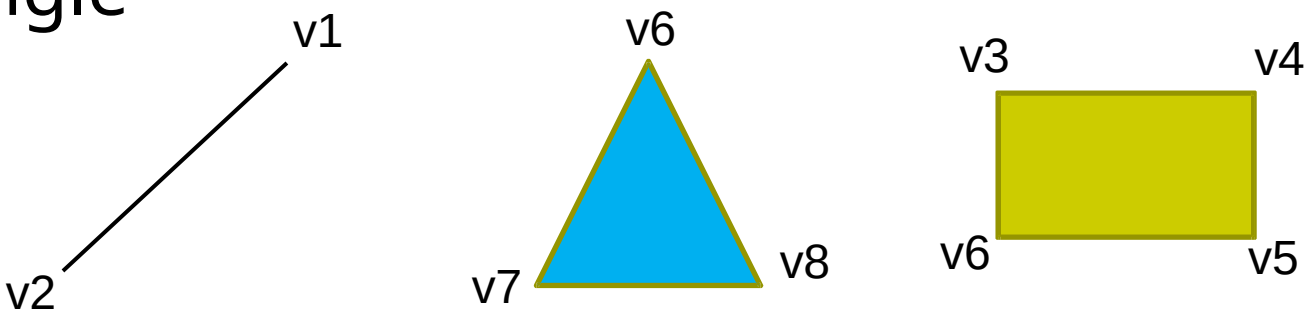
- After projection, several stages before objects drawn to screen
- These stages are non-programmable



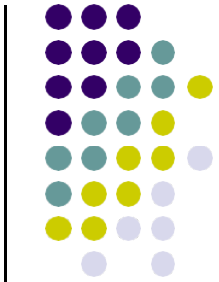
Primitive Assembly



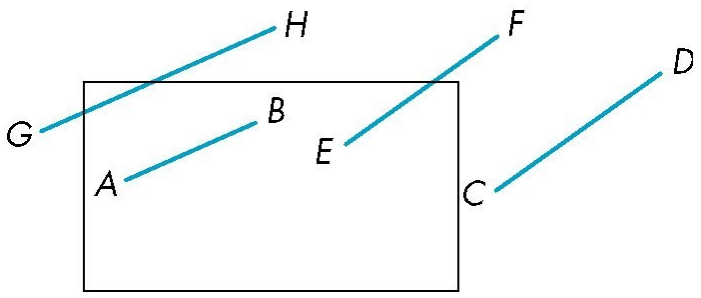
- Up till now: Transformations and projections applied to vertices individually
- **Primitive assembly:** After transforms, projections, individual vertices grouped back into primitives
- **E.g. v6, v7 and v8** grouped back into triangle



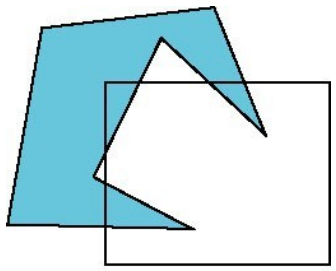
Clipping



- After primitive assembly, subsequent operations are per-primitive
- **Clipping:** Remove primitives (lines, polygons, text, curves) outside view frustum (canonical view volume)



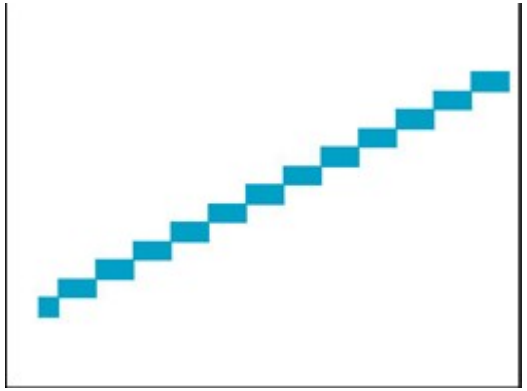
Clipping lines



Clipping polygons

Rasterization

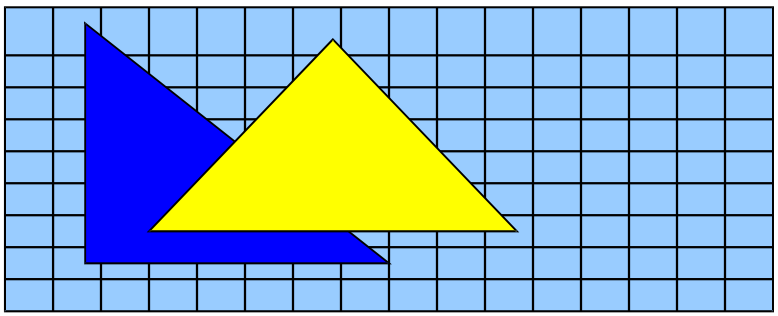
- Determine which pixels that primitives map to
  - Fragment generation
  - Rasterization or scan conversion



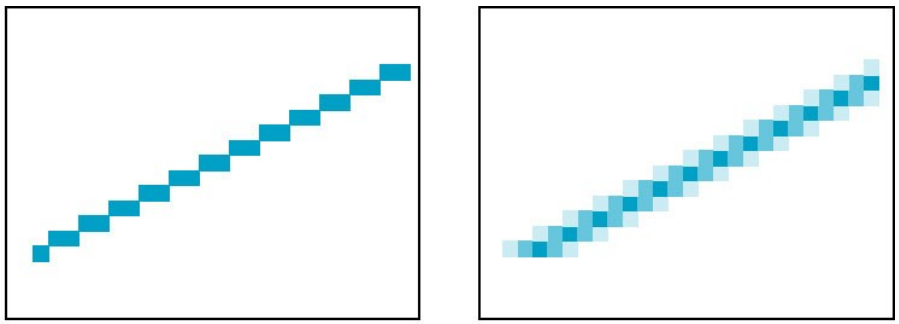
Fragment Processing

- Some tasks deferred until fragment processing

Hidden Surface Removal



Antialiasing

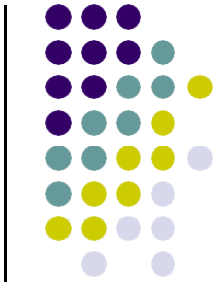


Transformation
Projection

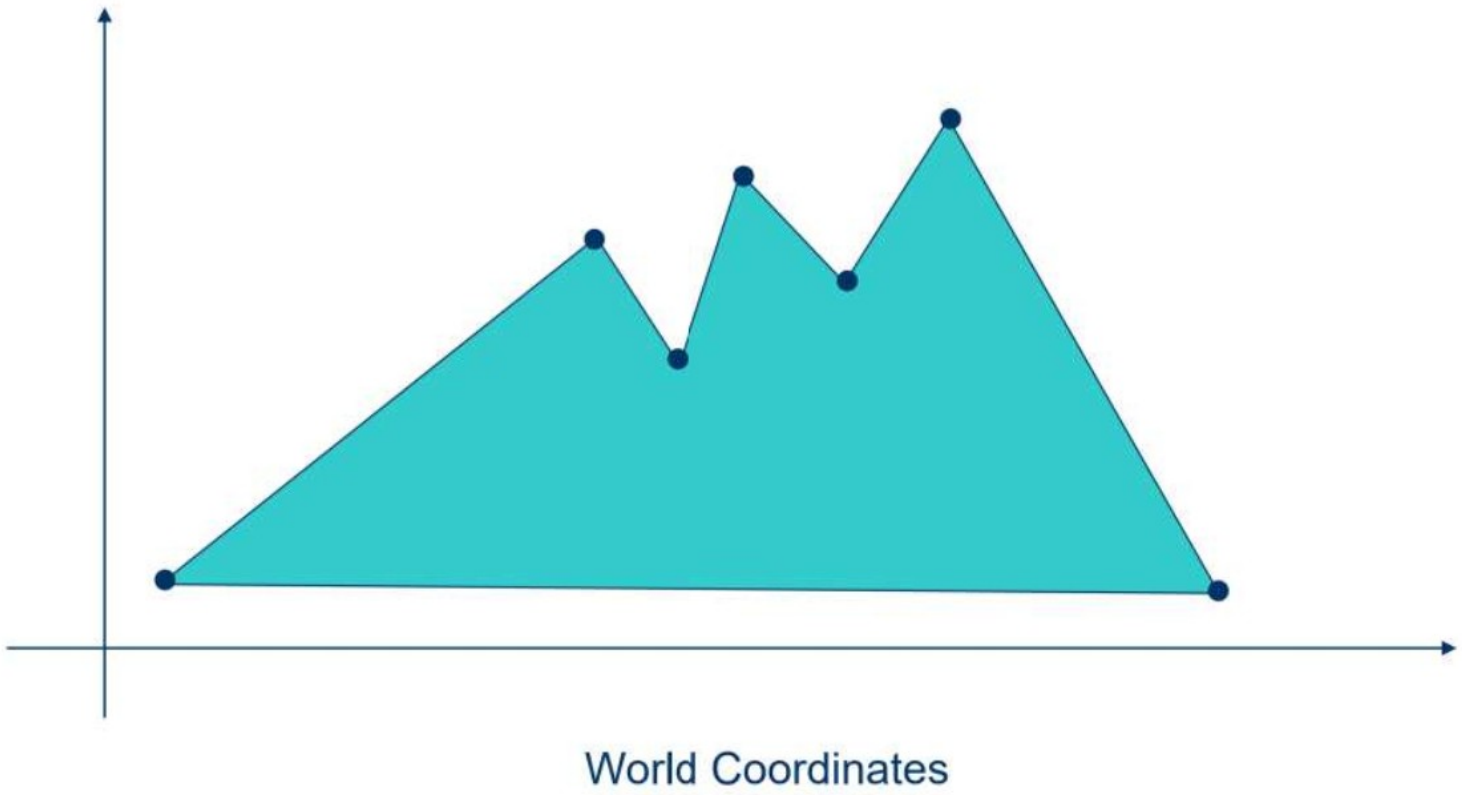
Hidden surface Removal
Antialiasing

Clipping

Clipping: Windowing

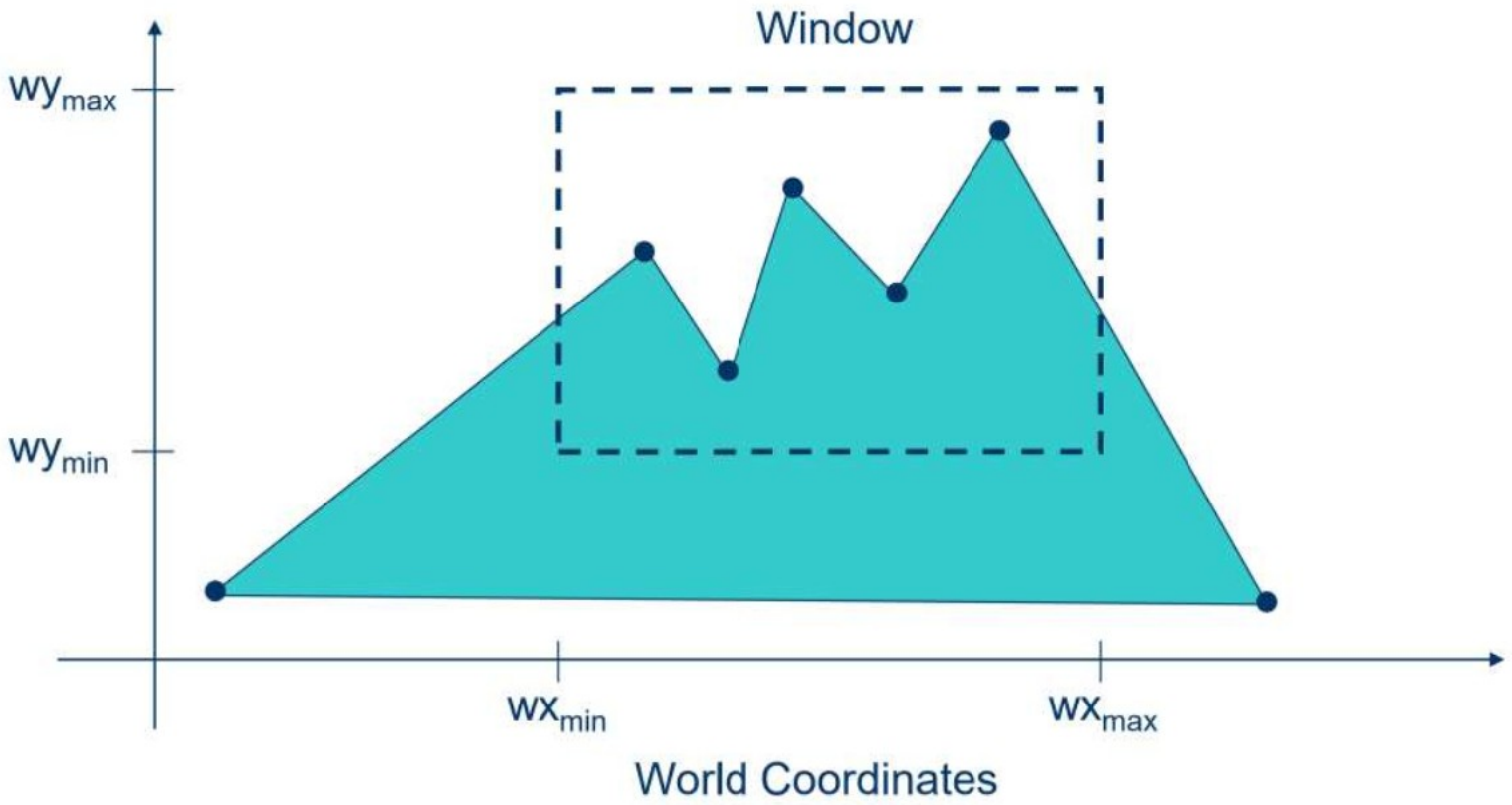


- A scene is made up of a collection of objects specified in world coordinates



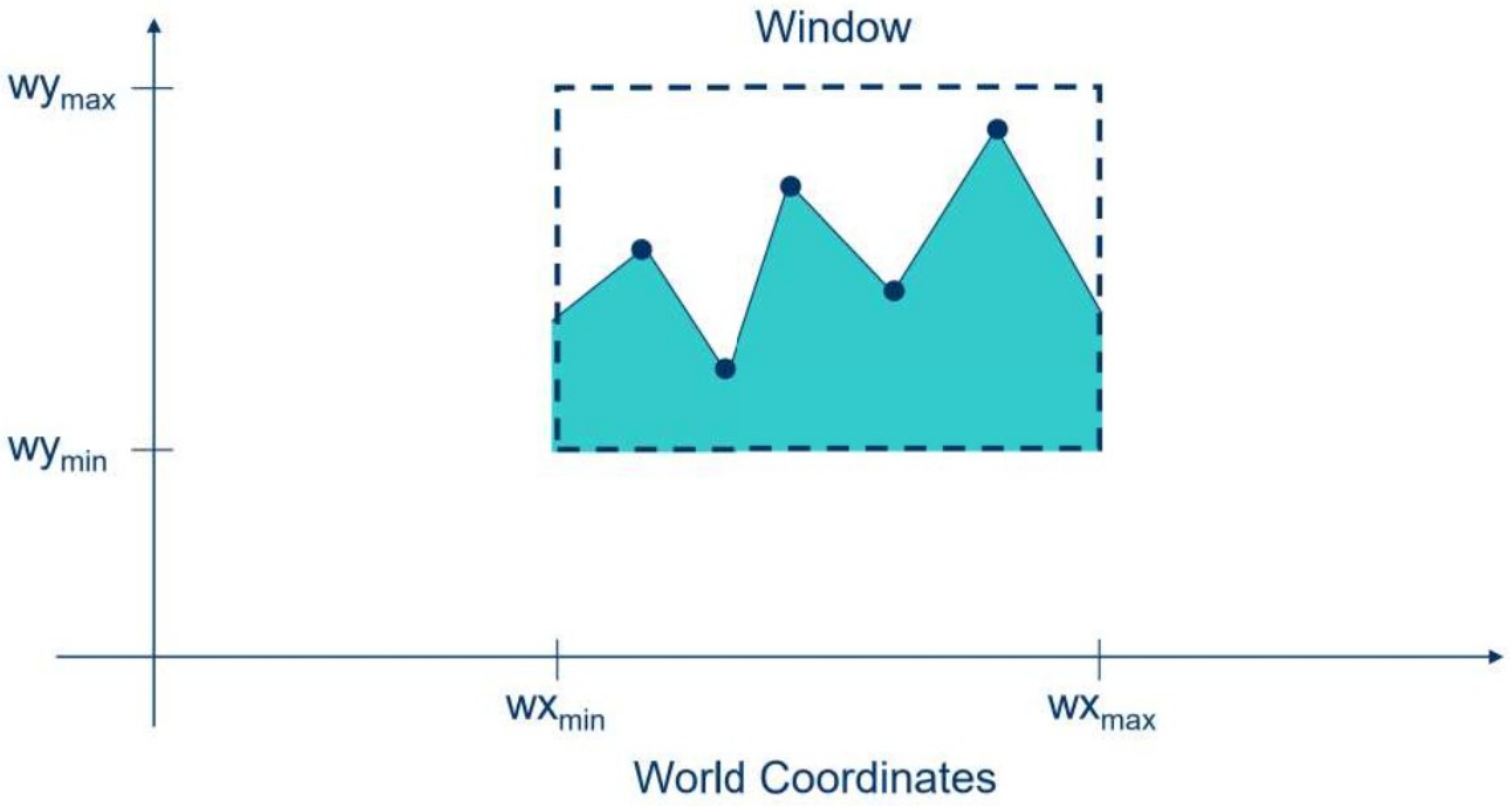
Clipping: Windowing

- When we display a scene only those objects within a particular window are displayed

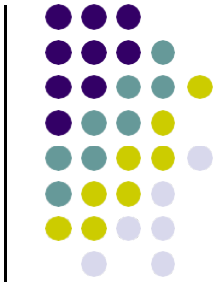


Clipping: Windowing

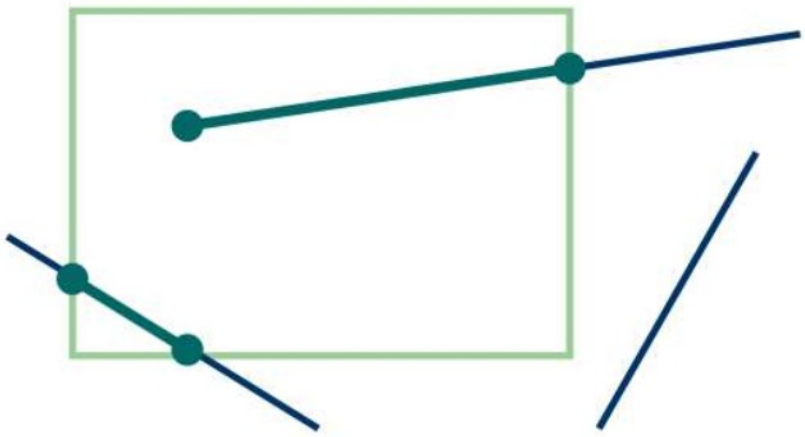
- Because drawing things to a display takes time we *clip* everything outside the window



Clipping



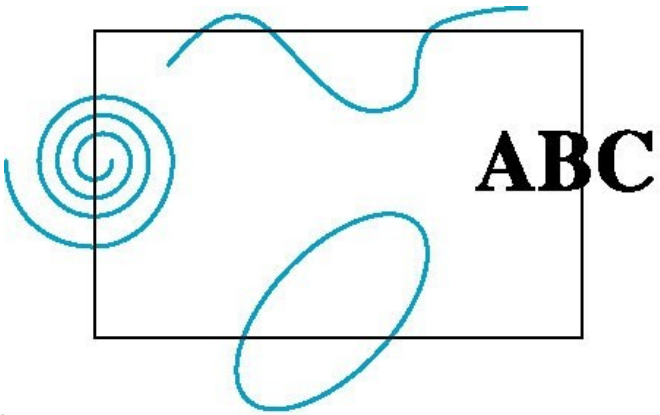
- Analytically calculating the portions of primitives (lines, triangles, polygons) within the clipping window

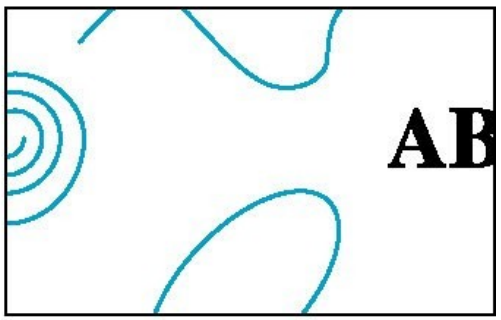


- Why
  - Bad idea to rasterize outside of framebuffer bounds
  - Also, don't waste time scan converting pixels outside window

Clipping

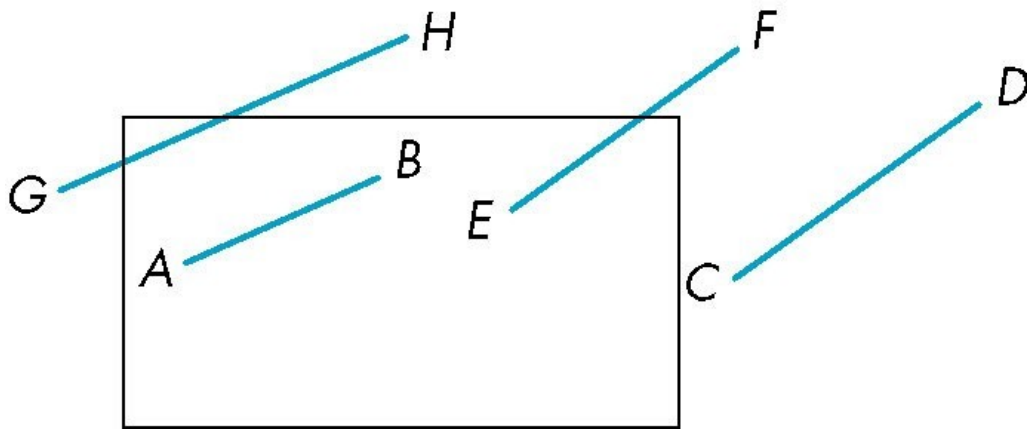
- 2D and 3D clipping algorithms
  - 2D against clipping window
  - 3D against clipping volume
- 2D clipping
  - Lines (e.g. dino.dat)
  - Polygons
  - Curves





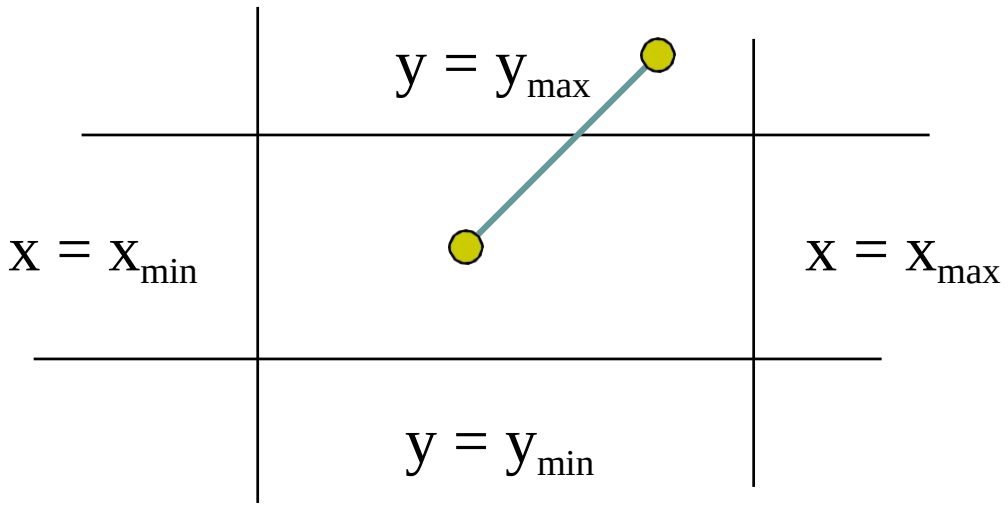
Clipping 2D Line Segments

- **Brute force approach:** compute intersections with all sides of the clipping window
  - Inefficient: one division per intersection

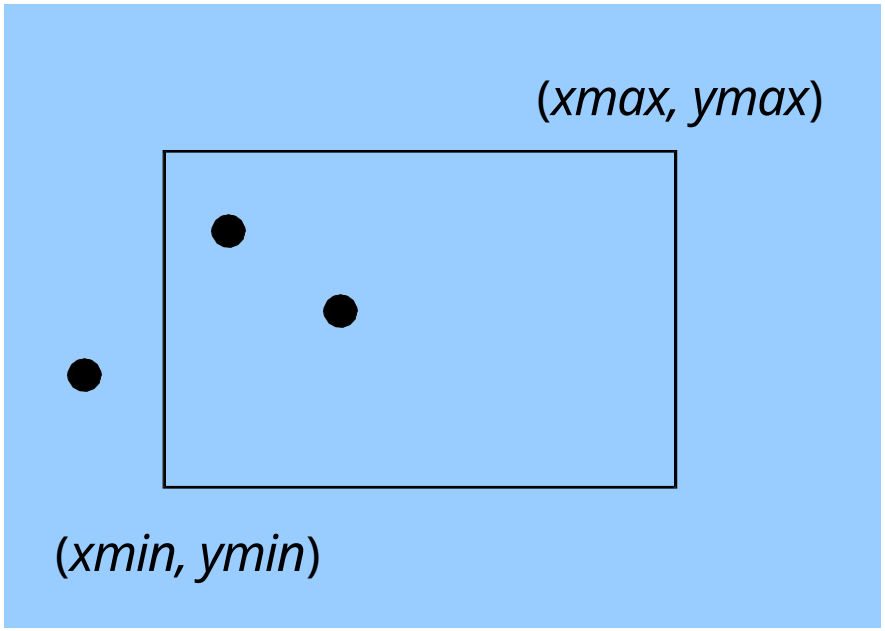


2D Clipping

- **Better Idea:** eliminate as many cases as possible without computing intersections
- **Cohen-Sutherland Clipping algorithm**



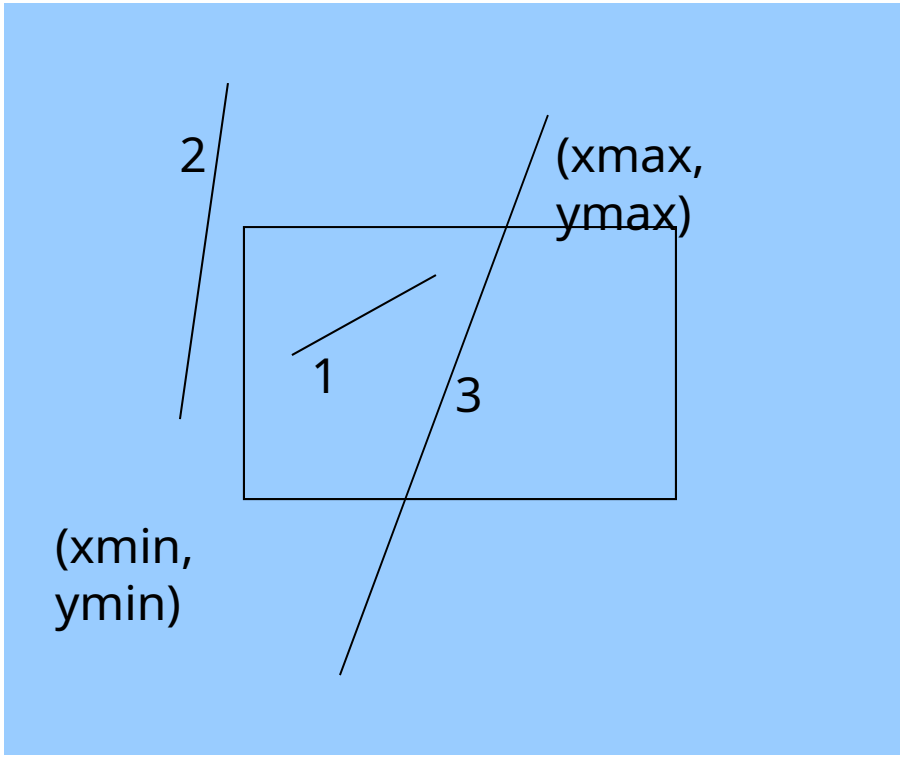
Clipping Points



Determine whether a point (x,y) is inside or outside of the world window.

If (xmin <= x <= xmax) **and** (ymin <= y <= ymax)
then the point (x,y) is inside else the point is outside

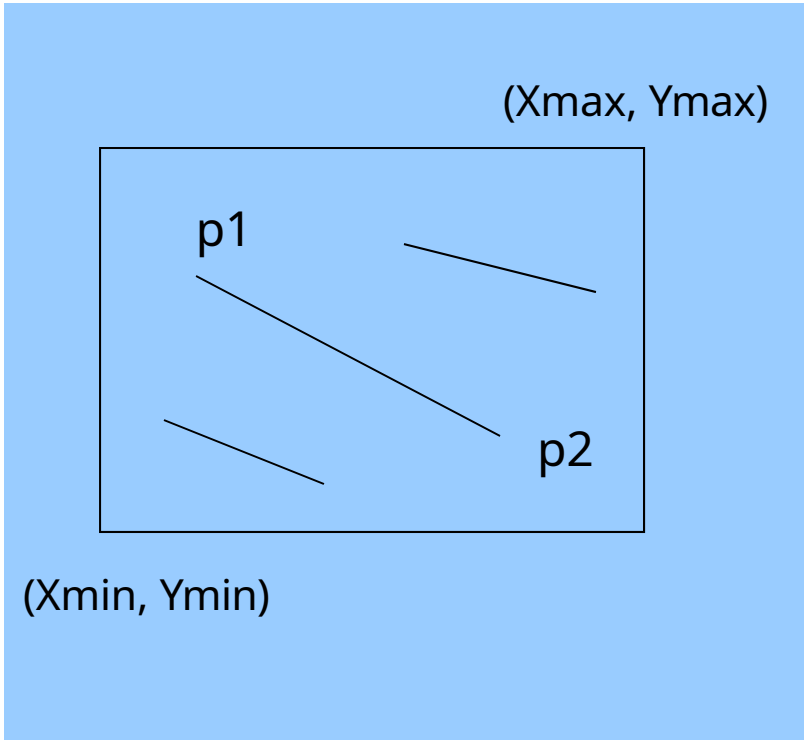
Clipping Lines



3 cases:

- **Case 1:** All of line in
- **Case 2:** All of line out
- **Case 3:** Part in, part out

Clipping Lines: Trivial Accept



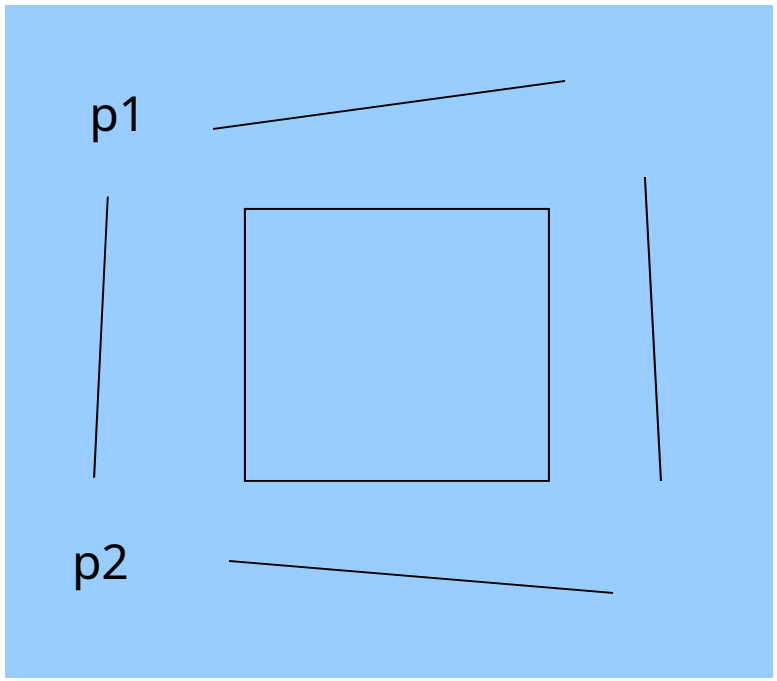
Case 1: All of line in Test line endpoints:

Xmin <= P1.x, P2.x <= Xmax and Ymin <= P1.y, P2.y <= Ymax

Note: simply comparing the x,y values of endpoints to the x,y values of the rectangle

Result: trivially accepted. Draw the line completely.

Clipping Lines: Trivial Reject



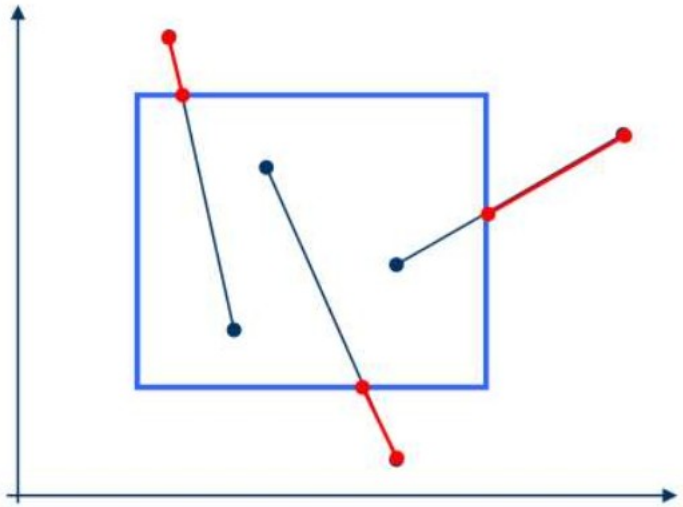
Case 2: All of line out Test line endpoints:

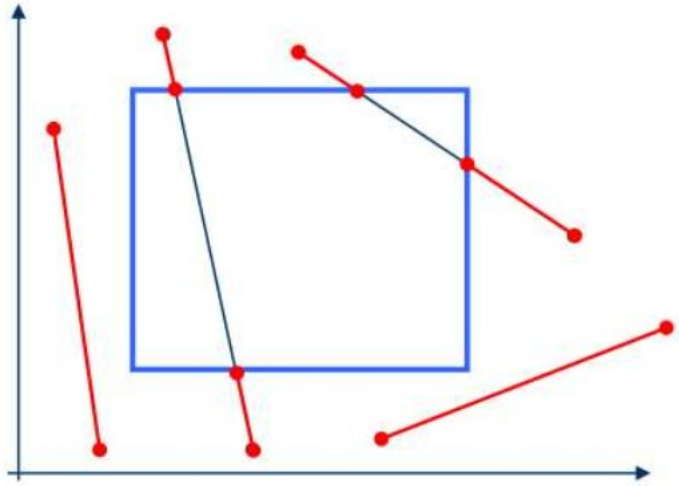
- ***p1.x, p2.x <= Xmin*** **OR**
- ***p1.x, p2.x >= Xmax*** **OR**
- ***p1.y, p2.y <= ymin*** **OR**
- ***p1.y, p2.y >= ymax***

Note: simply comparing the x,y values of endpoints to the x,y values of the rectangle

Result: trivially rejected. Don't draw the line.

Clipping Lines: Non-Trivial





Case 3: Part in, part out

Two variations:

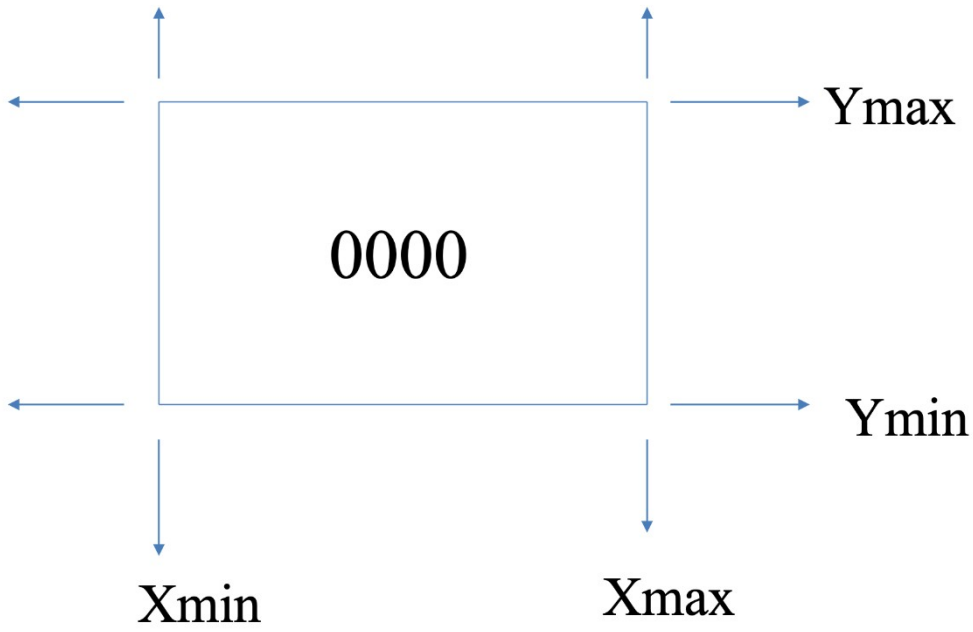
- One point in, other out
- Both points out, but part of the line cuts through the viewport

Cohen-Sutherland Algorithm

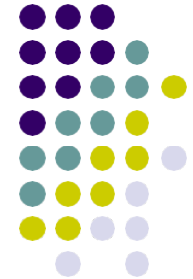
- The Cohen-Sutherland Line-Clipping Algorithm performs initial tests on a line to determine whether intersection calculations can be avoided.
  1. First, end-point pairs are checked for Trivial Acceptance.
  2. If the line cannot be trivially accepted, region checks are done for Trivial Rejection.
  3. **If the line segment can be neither trivially accepted nor rejected, it is divided into two segments at a clip edge, so that one segment can be trivially rejected.**
  - These three steps are performed iteratively until what remains can be trivially accepted or rejected.

Cohen-Sutherland: World Division

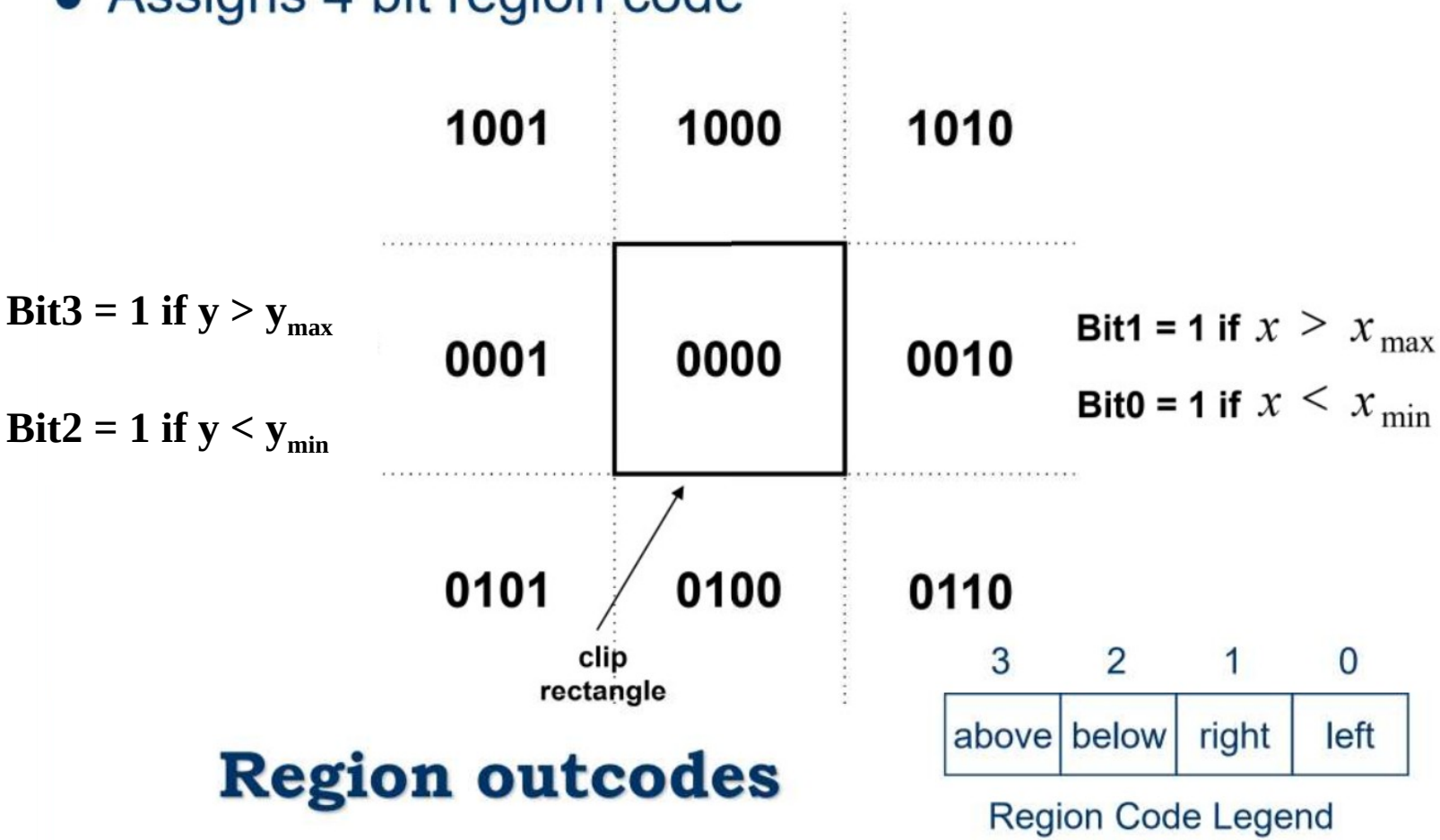
- World Space is divided into regions based on the window boundaries
  - Each region has a unique 4-bit region code
  - Region codes indicate the position of the regions with respect to the window



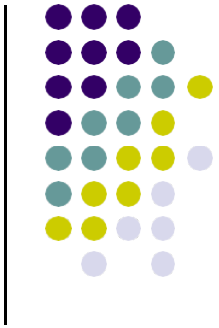
Cohen-Sutherland Algorithm



- Assigns 4 bit region code

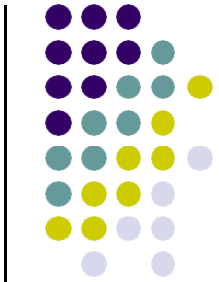


Cohen-Sutherland Algorithm



- A line segment can be trivially accepted (visible) if the outcodes of both the endpoints are zero.
- A line segment can be trivially rejected (not visible) if the logical AND of the outcodes of the endpoints is not zero.
- A line segment is clipping candidate if the logical AND of the outcodes of the endpoints is zero.

Pseudo Code



- ✓ Assign a region code for 2 cut points of a given line
- ✓ If both have region code 0000 then the line is accepted completely
- ✓ Else perform logical AND operation for both region codes
- ✓ If the result is not 0000, the line is outside
- ✓ Else line is partially inside

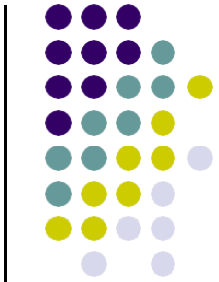
i. Choose an endpoint of the line that is outside the given rectangle

ii. find intersection point

iii. Replace the endpoint with the intersection point and update the region code

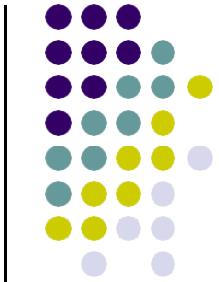
iv. Repeat step 2 until the line is trivially accepted or rejected.

Intersect point



- If bit 3 is 1, intersect with line $y = y_{max}$.
- If bit 2 is 1, intersect with line $y = y_{min}$
- If bit 1 is 1, intersect with line $x = x_{max}$
- If bit 0 is 1, intersect with line $x = x_{min}$

Intersect Point (X_i, Y_i)



$$x_i = x_{\min} \text{ or } x_{\max}$$
$$y_i = y_1 + m(x_i - x_1)$$

} If edge line is vertical

or

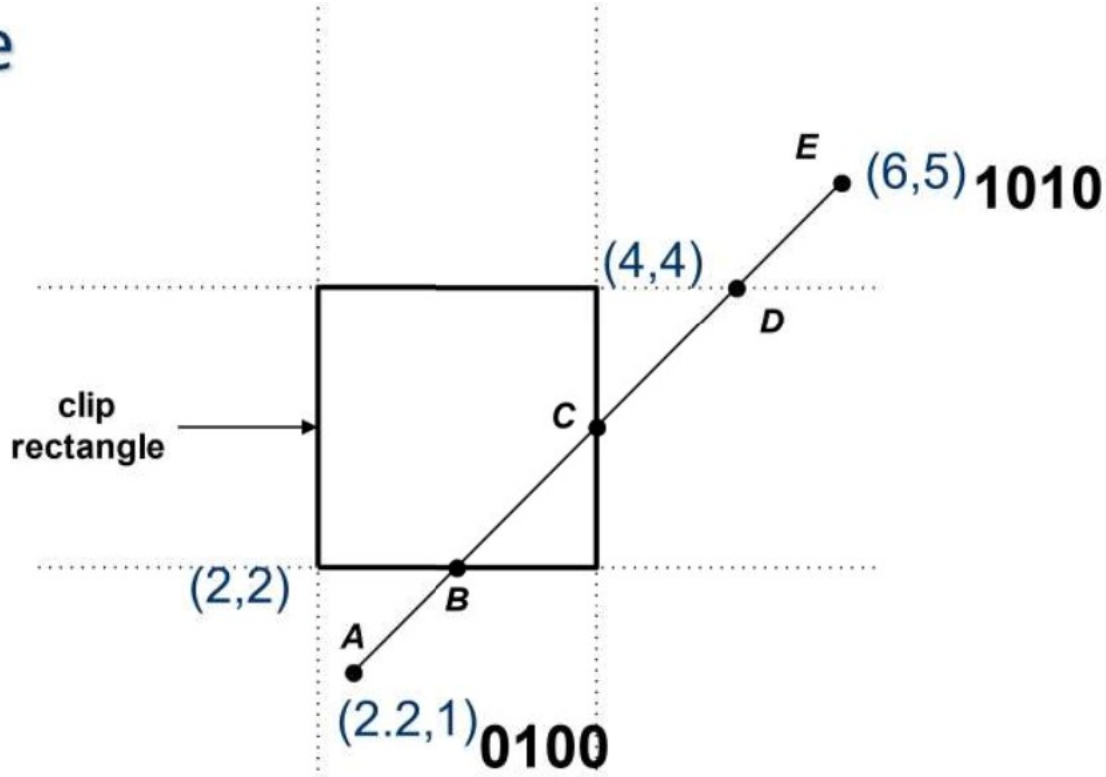
$$x_i = x_1 + (y_i - y_1)/m$$
$$y_i = y_{\min} \text{ or } y_{\max}$$

} If edge line is horizontal

$$where, \quad m = (y_2 - y_1)/(x_2 - x_2)$$

Cohen-Sutherland Algorithm

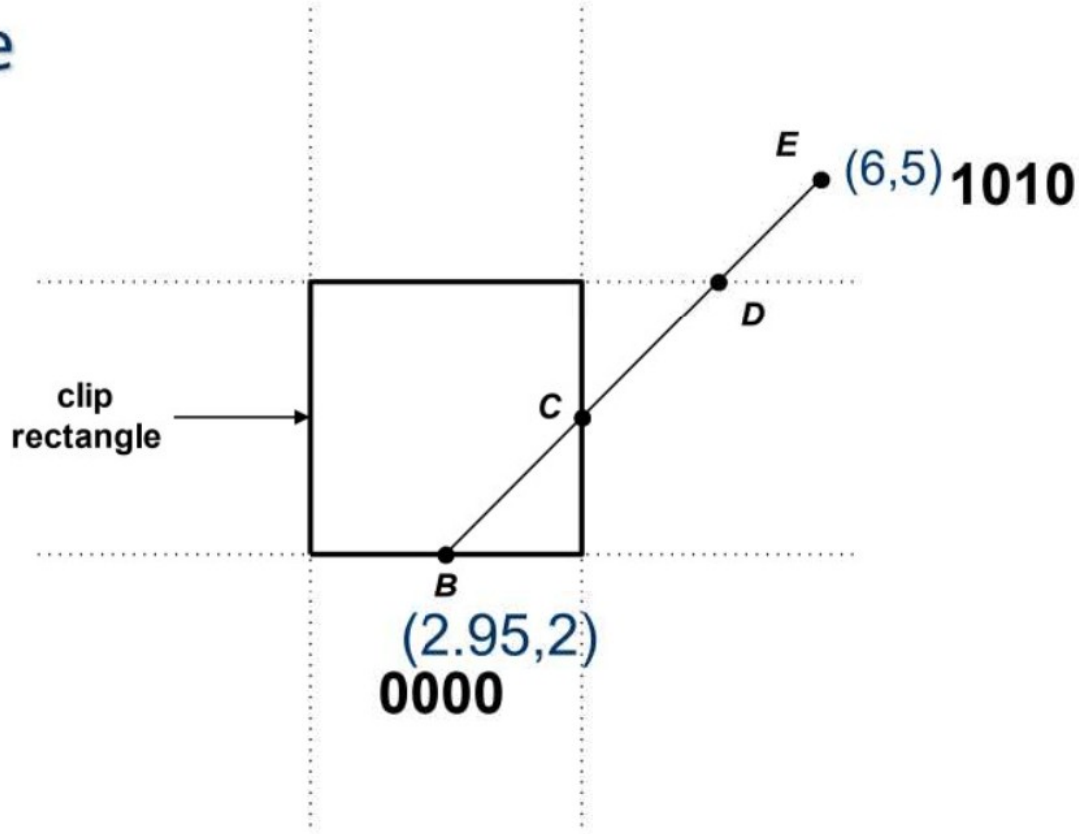
An Example



$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{5-1}{6-2.2} = \frac{4}{3.8} = 1.05$$

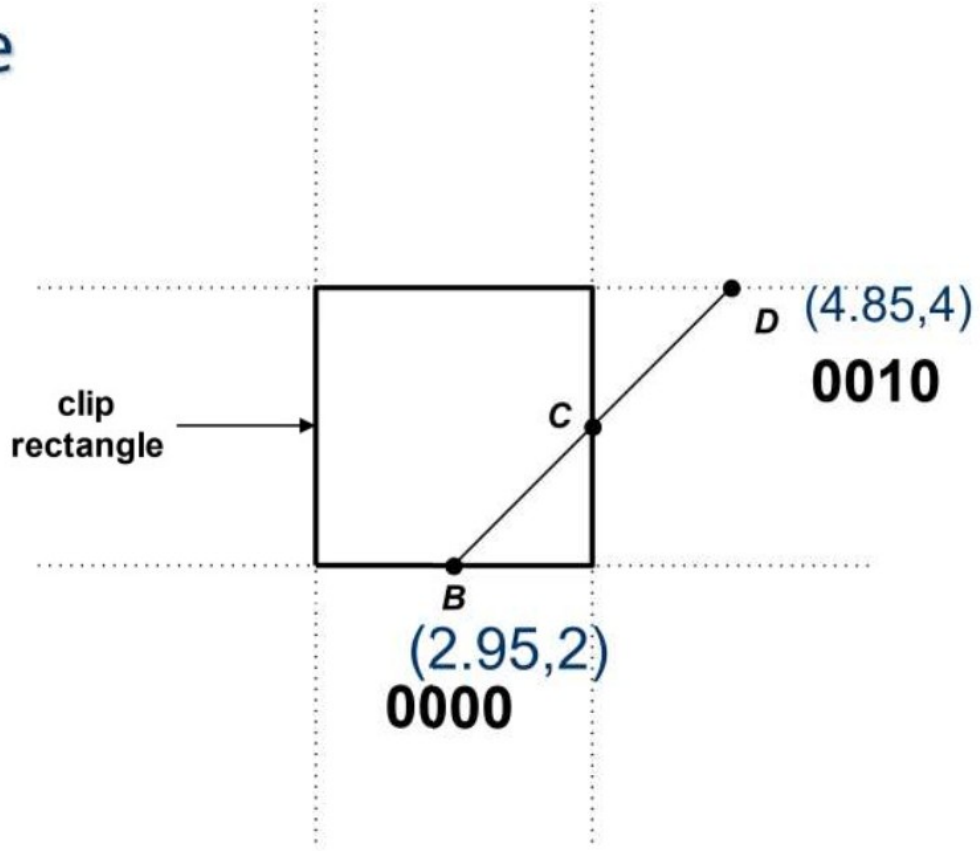
Cohen-Sutherland Algorithm

An Example



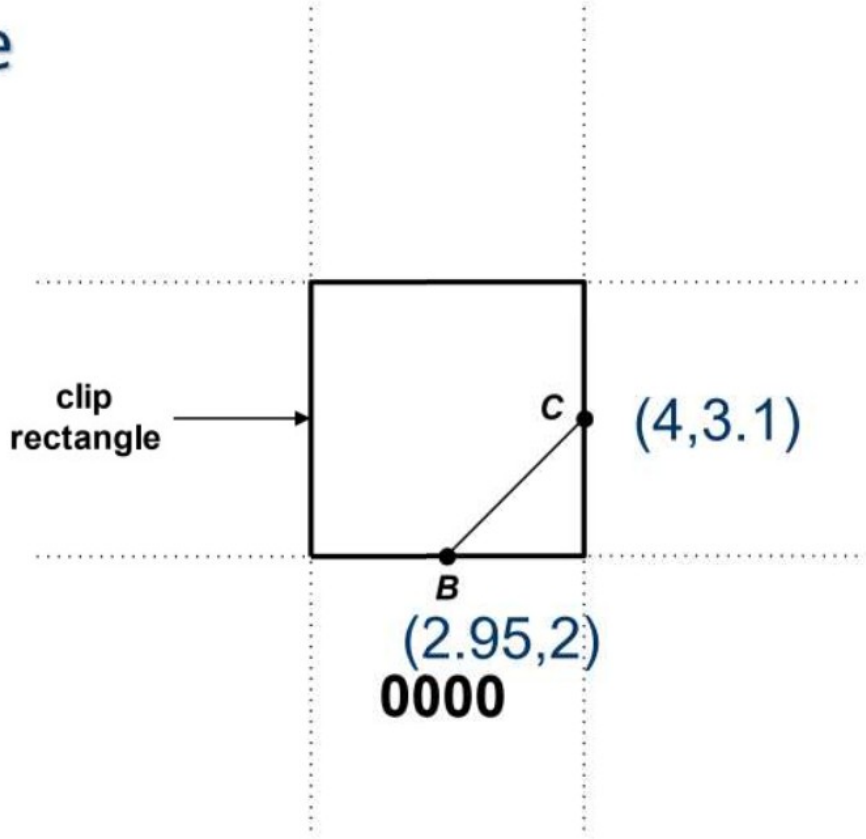
Cohen-Sutherland Algorithm

An Example

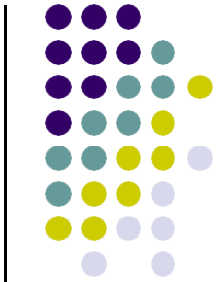


Cohen-Sutherland Algorithm

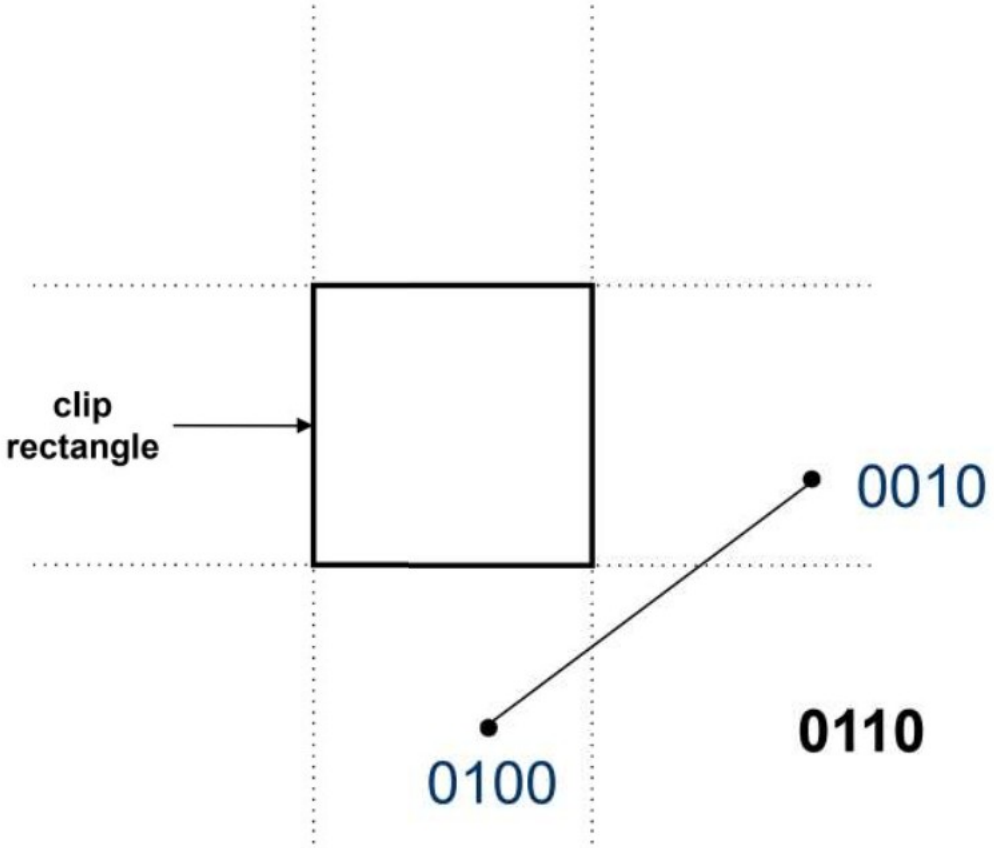
An Example



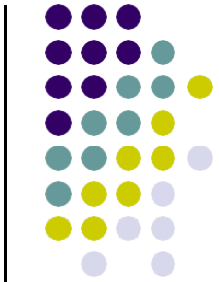
Cohen-Sutherland Algorithm



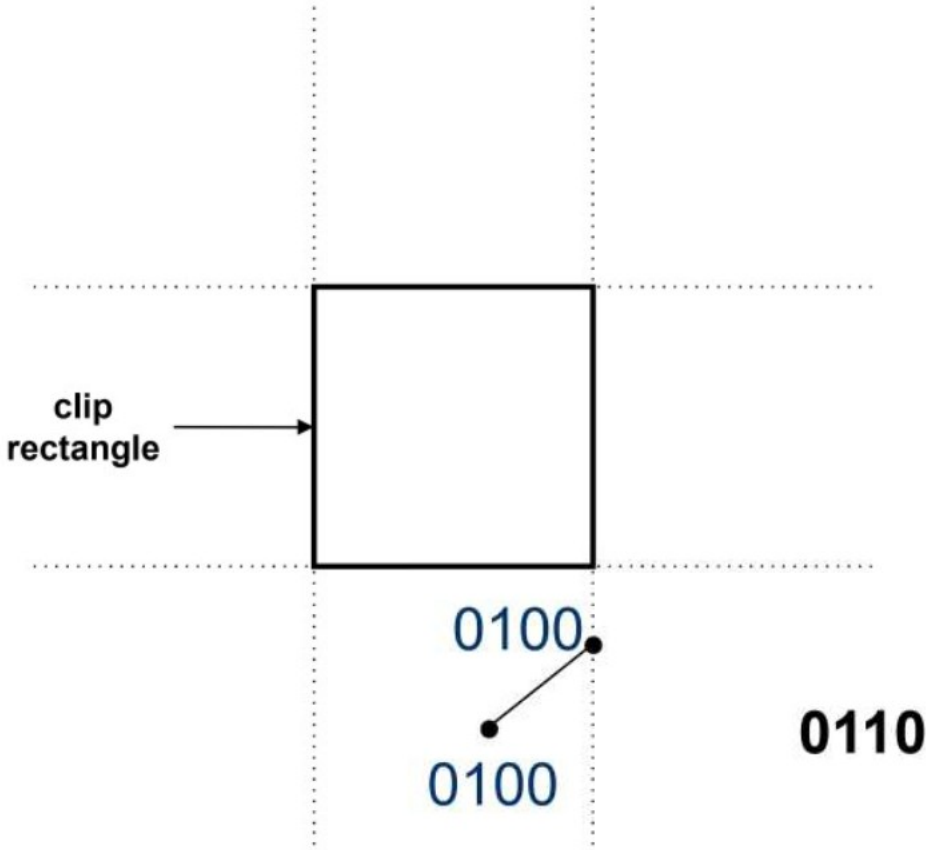
Another Example



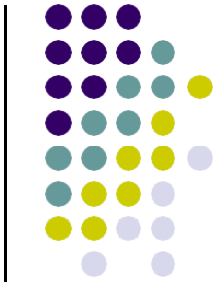
Cohen-Sutherland Algorithm



Another Example

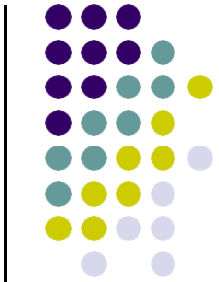


Cohen-Sutherland pseudocode (Hill)



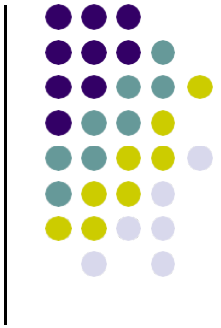
```
int clipSegment(Point2& p1, Point2& p2, RealRect W)
{
   do{
        if(trivial accept) return 1; // whole line survives
        if(trivial reject) return 0;  // no portion survives
        // now chop
        if(p1 is outside)
        //   find surviving segment
        {
            if(p1 is to the left) chop against left edge
            else if(p1 is to the right) chop against right edge
             else if(p1 is below) chop against the bottom edge
            else if(p1 is above) chop against the top edge
        }
```

Cohen-Sutherland pseudocode (Hill)



```
        else // p2 is outside
                // find surviving segment
        {
            if(p2 is to the left) chop against left edge
           else if(p2 is to right) chop against right edge
           else if(p2 is below) chop against the bottom edge
            else if(p2 is above) chop against the top edge
        }
   }while(1);
}
```

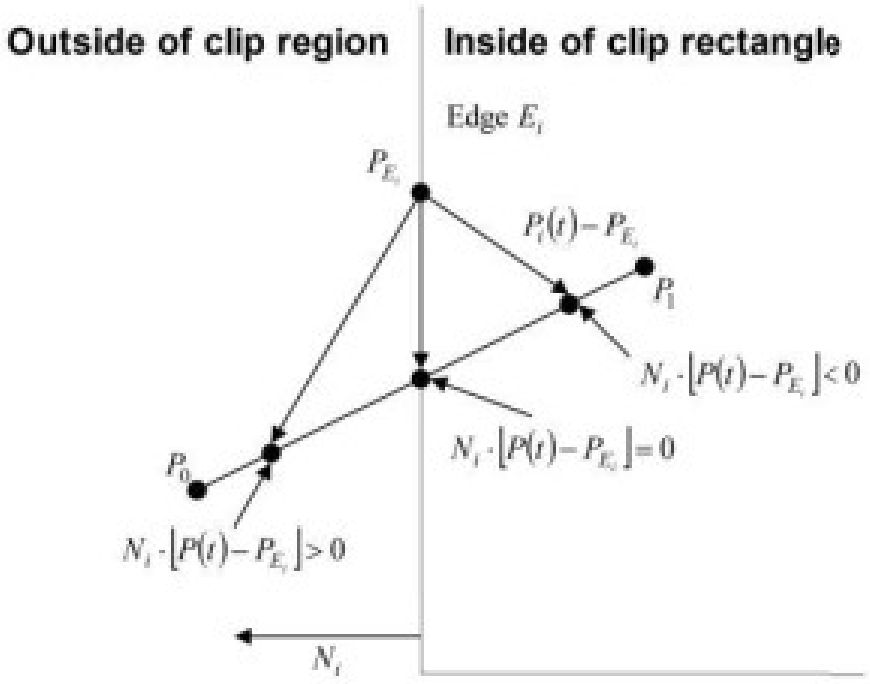
Parametric Line-Clipping



(1) This fundamentally different (from the Cohen-Sutherland algorithm) and generally more efficient algorithm was originally published by Cyrus and Beck.

(2) Liang and Barsky later independently developed a more efficient algorithm that is especially fast in the special cases of upright 2D and 3D clipping regions. They also introduced more efficient trivial rejection tests for general clip regions.

The Cyrus-Beck Algorithm



$$Line\ P_0P_1 : P(t) = P_0 + (P_1 - P_0)t$$

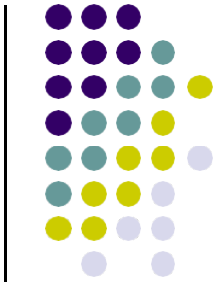
$$N_i \cdot \left[P(t) - P_{E_i} \right] = 0$$

$$\Rightarrow N_i \cdot \left[P_0 + (P_1 - P_0)t - P_{E_i} \right] = 0$$

$$\Rightarrow t = -\frac{N_i \cdot \left[P_0 - P_{E_i} \right]}{N_i \cdot (P_1 - P_0)}$$

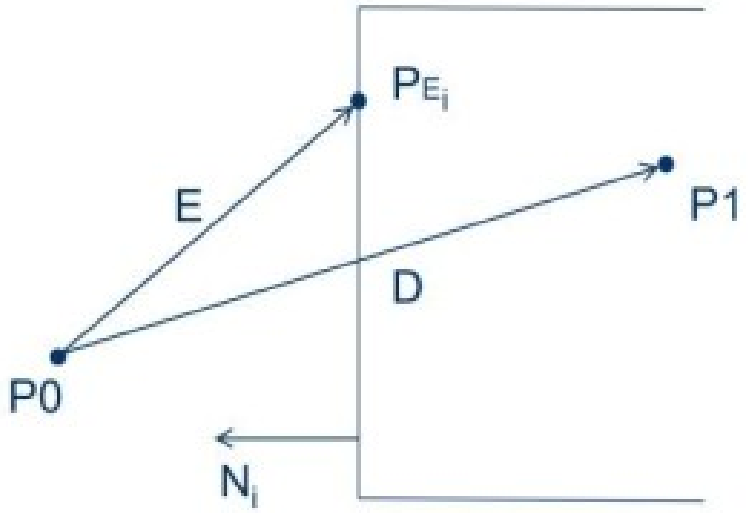
$$\Rightarrow t = \frac{N_i \cdot \left[P_{E_i} - P_0 \right]}{N_i \cdot D}, \quad D = P_1 - P_0$$

The Cyrus-Beck Algorithm



here

$$N_i \cdot D < 0$$



$$t = \frac{N_i \cdot \left[P_{E_i} - P_0 \right]}{N_i \cdot D} = \frac{N_i \cdot E}{N_i \cdot D}$$

t exists when

(1) $N_i \neq 0$

(2) $D \neq 0 \Rightarrow P_0 \neq P_1$

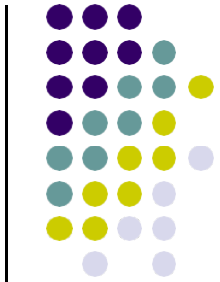
(3) $N_i \cdot D \neq 0$

PE = Potentially Entering

$$N_i \cdot D < 0 \Rightarrow PE$$

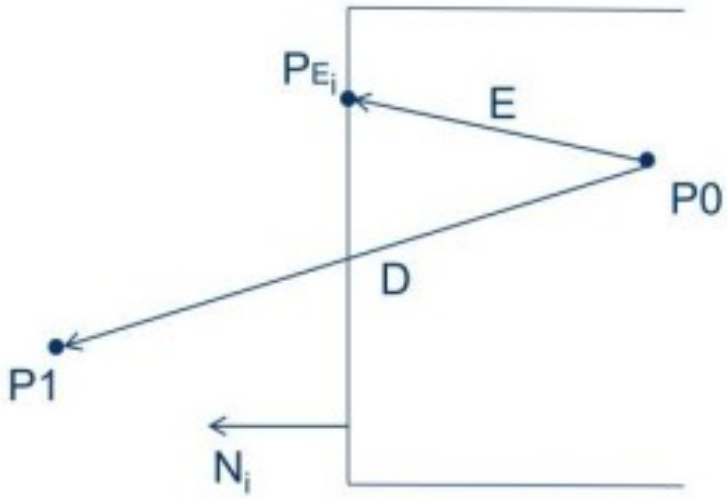
$$\Rightarrow Angle > 90°$$

The Cyrus-Beck Algorithm



here

$$N_i \cdot D > 0$$



$$t = \frac{N_i \cdot \left[P_{E_i} - P_0 \right]}{N_i \cdot D} = \frac{N_i \cdot E}{N_i \cdot D}$$

t exists when

(1) $N_i \neq 0$

(2) $D \neq 0 \Rightarrow P_0 \neq P_1$

(3) $N_i \cdot D \neq 0$

PL = Potentially Leaving

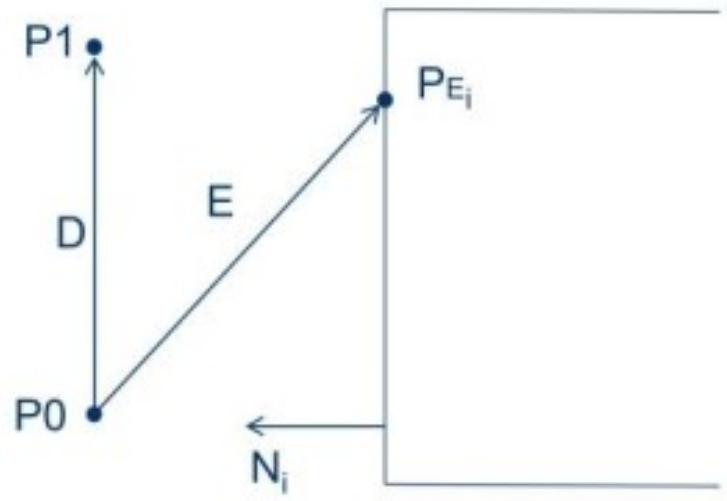
$$N_i \cdot D > 0 \Rightarrow PL$$

$$\Rightarrow Angle < 90°$$

The Cyrus-Beck Algorithm

here

$$N_i \cdot D = 0$$



$$t = \frac{N_i \cdot \left[P_{E_i} - P_0 \right]}{N_i \cdot D} = \frac{N_i \cdot E}{N_i \cdot D}$$

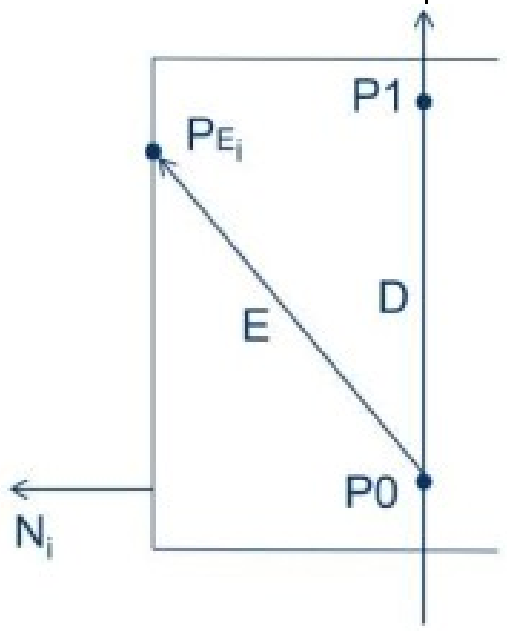
t exists when

(1) $N_i \neq 0$

(2) $D \neq 0 \Rightarrow P_0 \neq P_1$

(3) $N_i \cdot D \neq 0$

The Cyrus-Beck Algorithm



here

$$N_i \cdot D = 0$$

$$t = \frac{N_i \cdot \left[P_{E_i} - P_0 \right]}{N_i \cdot D} = \frac{N_i \cdot E}{N_i \cdot D}$$

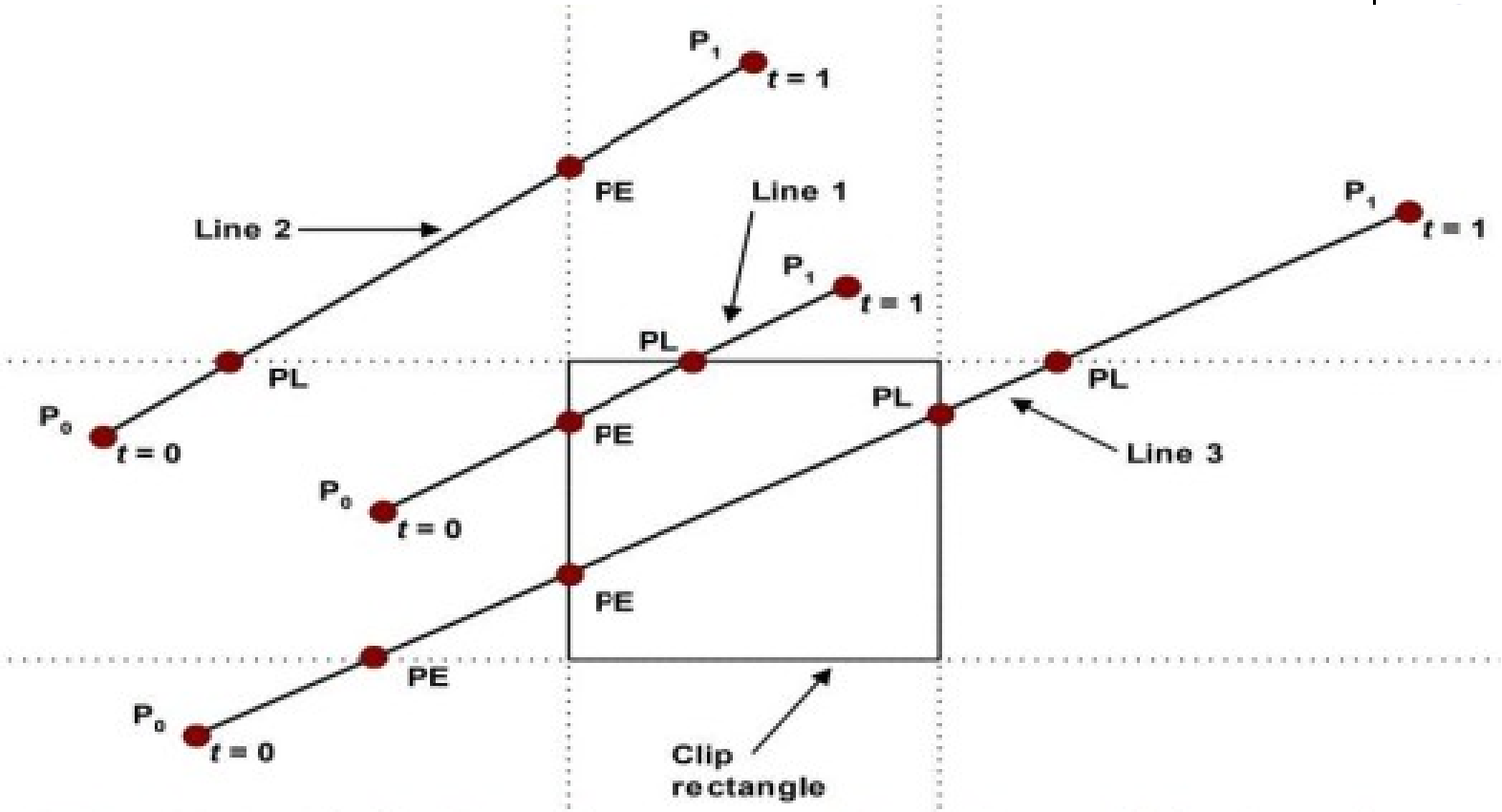
t exists when

(1) $N_i \neq 0$

(2) $D \neq 0 \Rightarrow P_0 \neq P_1$

(3) $N_i \cdot D \neq 0$

The Cyrus-Beck Algorithm



PE = Potentially Entering

$N_i \cdot D < 0 \Rightarrow PE$

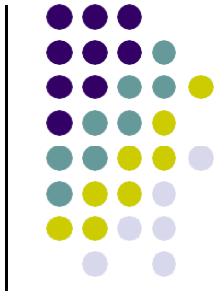
$\Rightarrow Angle > 90°$

PL = Potentially Leaving

$N_i \cdot D > 0 \Rightarrow PL$

$\Rightarrow Angle < 90°$

The Cyrus-Beck Algorithm



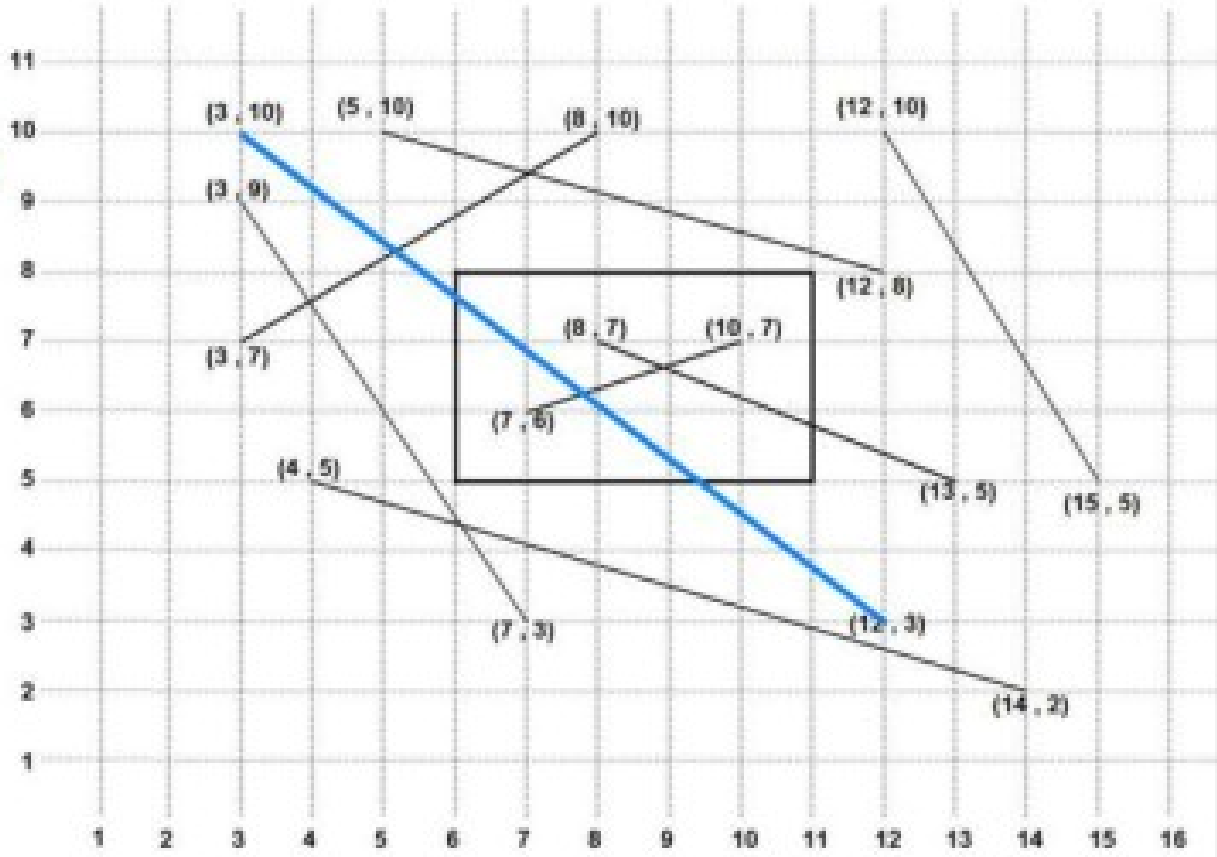
```
Precalculate Ni and PEi for each edge
for (each line segment to be clipped) {
    if (P1 == P0)
        line is degenerated, so clip as a point;
    else {
        tE = 0;  tL = 1;
        for (each candidate intersection with a clip edge) {
            if (Ni • D != 0) {   /* Ignore edges parallel to line */
                calculate t;
                use sign of  Ni • D to categorize as PE or PL;
                if (PE) tE = max(tE ,  t);
                if (PL) tL = min(tL ,  t);
            }
        }
        if (tE > tL) return NULL;
        else return P(tE) and P(tL) as true clip intersection;
    }
}
```

Liang-Barsky Improvement

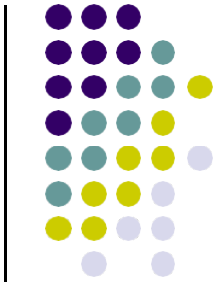
Clip edge $_i$	Normal N_i	P_{E_i}	$P_{E_i} - P_0$	$t = \frac{N_i \cdot (P_{E_i} - P_0)}{N_i \cdot D}$	$N_i \cdot D$
left: $x = x_{min}$	$(-1,0)$	(x_{min}, y)	$(x_{min} - x_0, y - y_0)$	$\frac{(x_{min} - x_0)}{(x_1 - x_0)}$	$-(x_1 - x_0)$
right: $x = x_{max}$	$(1,0)$	(x_{max}, y)	$(x_{max} - x_0, y - y_0)$	$\frac{(x_{max} - x_0)}{(x_1 - x_0)}$	$(x_1 - x_0)$
bottom: $y = y_{min}$	$(0,-1)$	(x, y_{min})	$(x - x_0, y_{min} - y_0)$	$\frac{(y_{min} - y_0)}{(y_1 - y_0)}$	$-(y_1 - y_0)$
top: $y = y_{max}$	$(0,1)$	(x, y_{max})	$(x - x_0, y_{max} - y_0)$	$\frac{(y_{max} - y_0)}{(y_1 - y_0)}$	$(y_1 - y_0)$

Example

Pmin	6,5		
pmax	11,8		
	L1		
P0	3,10		
P1	12,3		
x1-x0=dx	9		
y1-y0=dy	-7		
tl=(xmin-x0)/dx	3/9 =	0.333	PE
N.D=-dx for left	-9 (-ve)		
tr=(xmax-x0)/dx	8/9 =	0.889	PL
N.D=dx for right	9 (+ve)		
tb=(ymin-y0)/dy	-5/(-7) =	0.714	PL
N.D=-dy for bottom	7 (+ve)		
tt=(ymax-y0)/dy	-2/(-7) =	0.286	PE
N.D=dy for top	-7 (-ve)		
te = max(0, all PE)	0.333		
tl = min(1, all PL)	0.714		

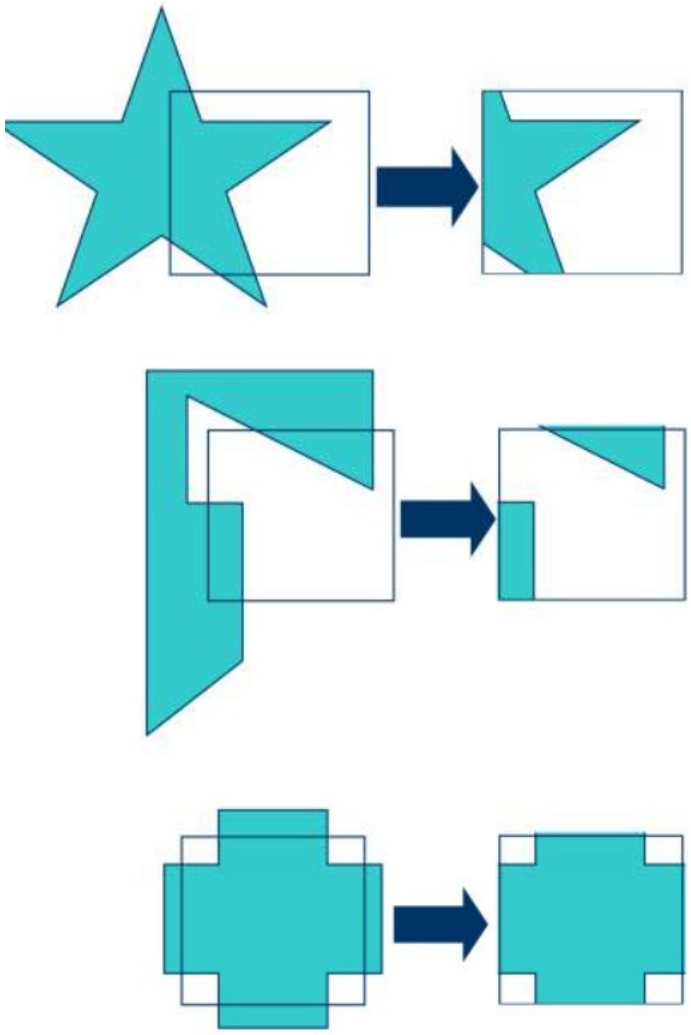


Comparison



- Cohen-Sutherland:
  - Repeated clipping is expensive
  - It is best used when trivial acceptance and rejection are possible for most lines
- Liang-Barsky:
  - Computation of t-intersections is cheap
  - Computation of (x,y) clip points is only done once
  - The algorithm doesn't consider trivial accepts/rejects
  - Best when many lines must be clipped

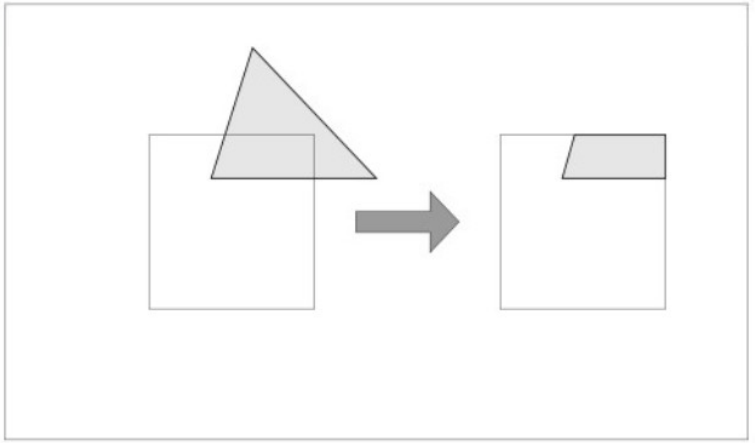
Polygon / Area Clipping



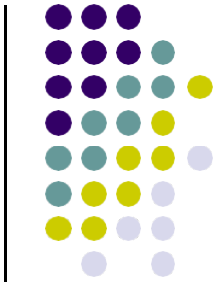
- Similarly to lines, areas must be clipped to a window boundary
- Consideration must be taken as to which portions of the area must be clipped

Polygon Clipping

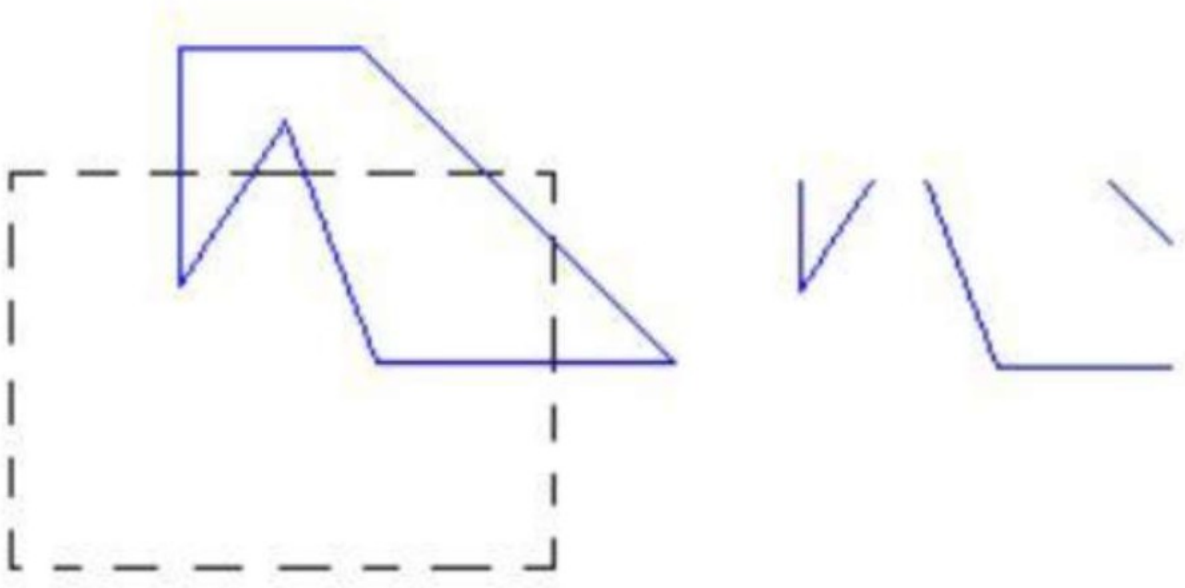
- We know how to clip a single line segment
  - How about a polygon in 2D?
  - How about in 3D?
- Clipping polygons is more complex than clipping the individual lines
  - Input: polygon
  - Output: polygon, or nothing



Polygon Clipping

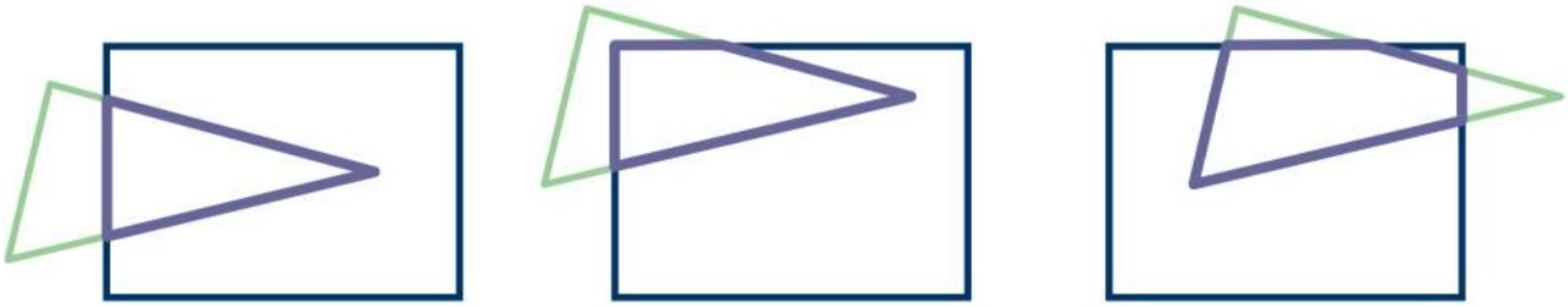


- To clip a polygon, we cannot directly apply a line-clipping method to the individual polygon edges because this approach would produce a series of unconnected line segments as shown in th



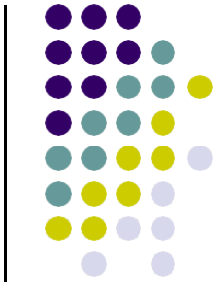
Why is Clipping Hard?

- What happens to a triangle during clipping?
- Possible outcomes:

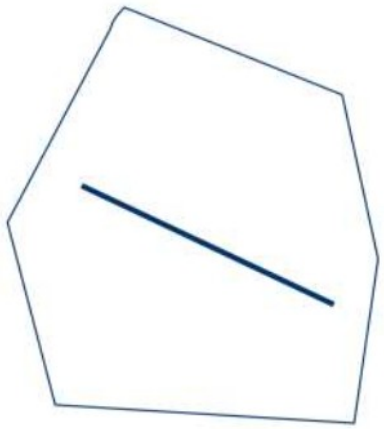


How many sides can a clipped triangle have?

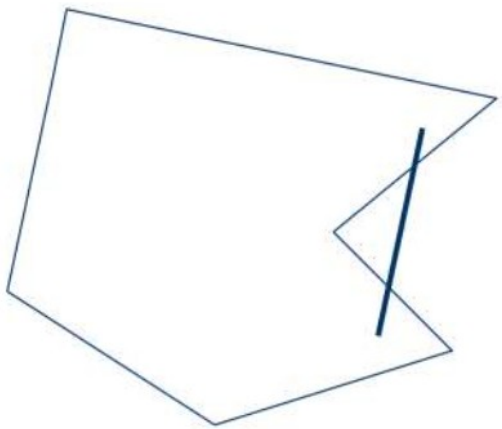
Polygon Clipping



- Convex polygonal clipping window.
  - **Convex polygon**: if the line joining two interior points lies completely inside the polygon.
  - O **on**

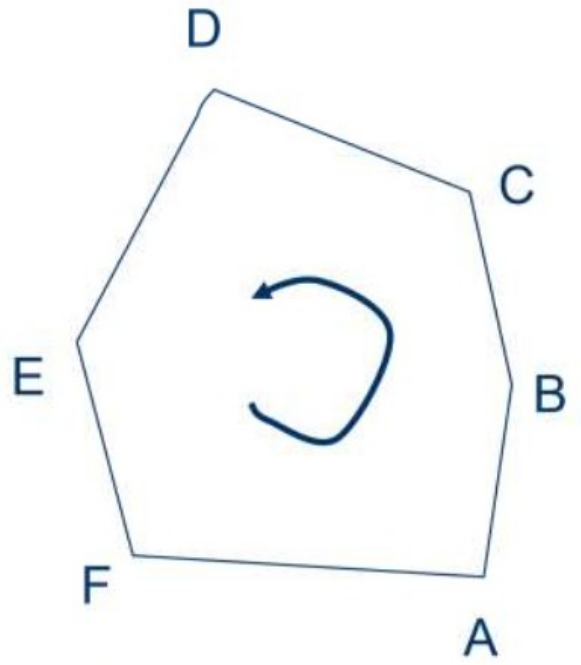


Convex

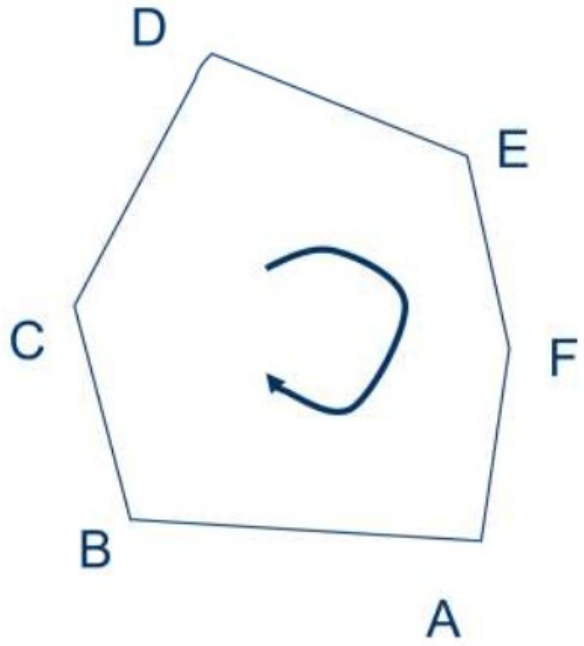


Concave

Polygon Clipping

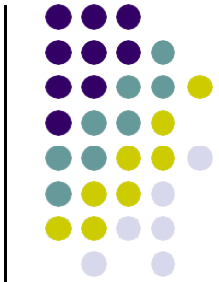


Positive Orientation



Negative Orientation

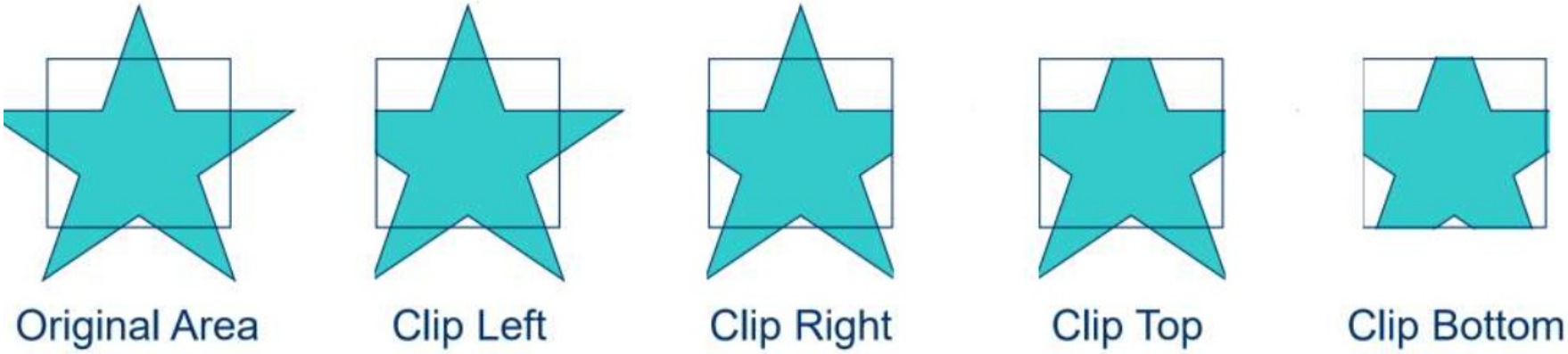
Polygon Clipping



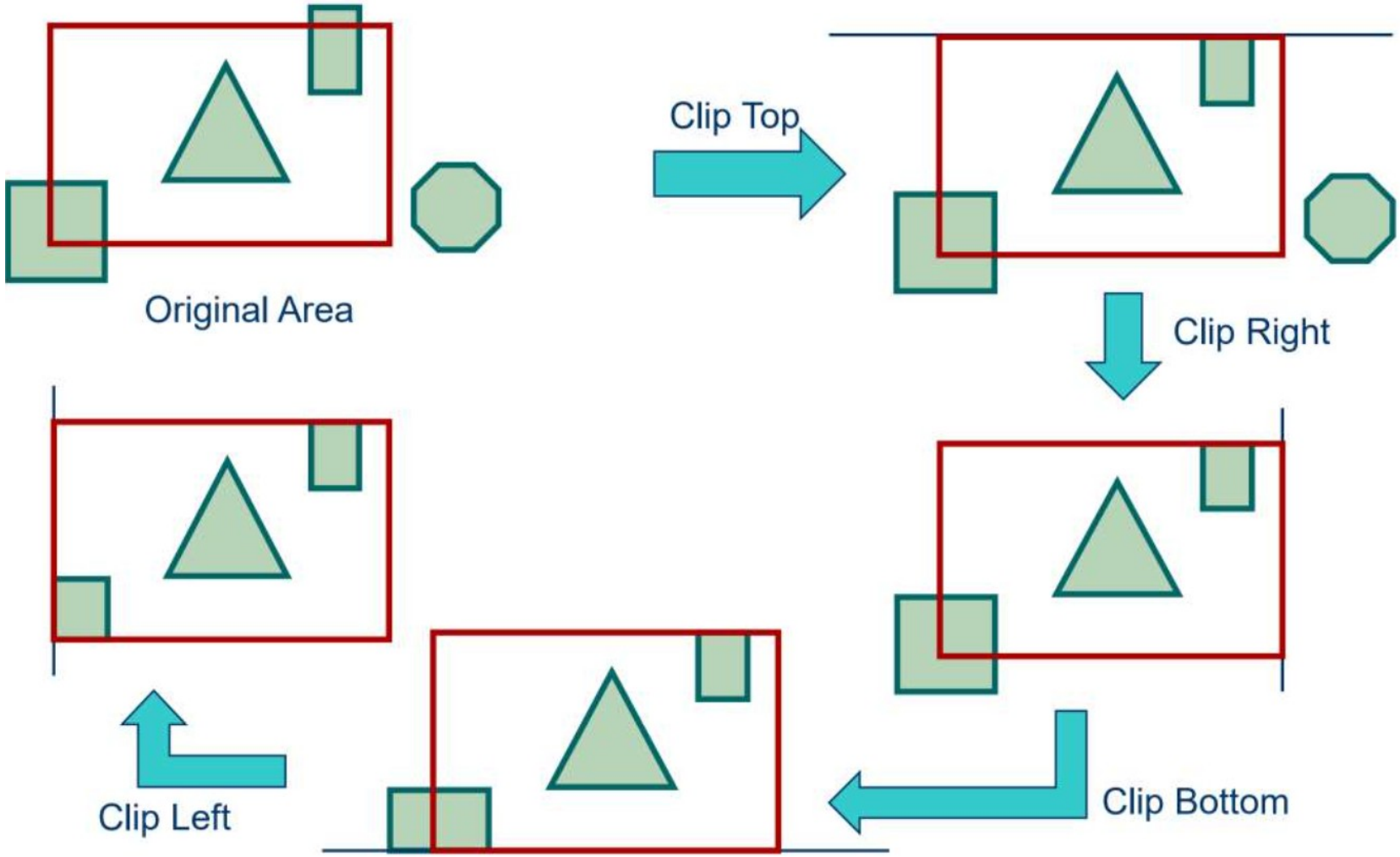
- The Left-hand side of any directed edge $\overline{P_{i_1}P_i}$ or $\overline{P_N P_1}$ points inside the polygon
- Let, a point P(x,y). If it is to the left of every edge of the polygon, it is inside the polygon

Sutherland-Hodgman Algorithm

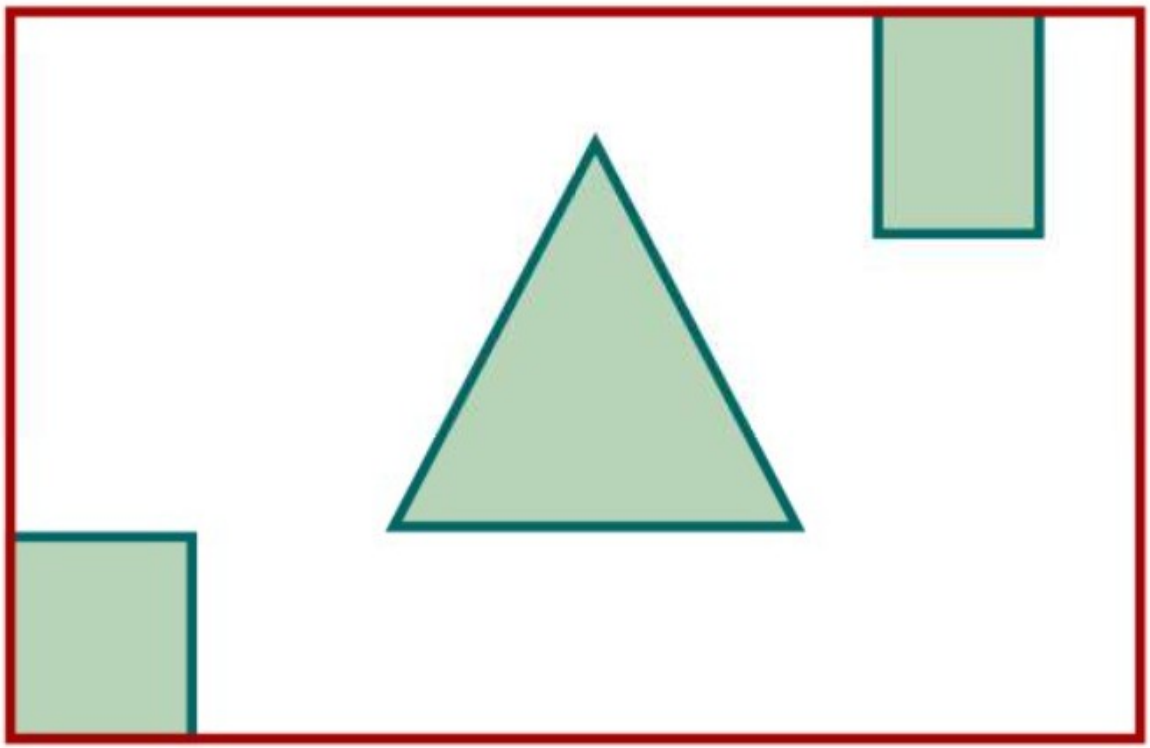
- A technique for clipping areas developed by Sutherland & Hodgman
- Put simply the polygon is clipped by comparing it against each boundary in turn



Sutherland-Hodgman Polygon Clipping

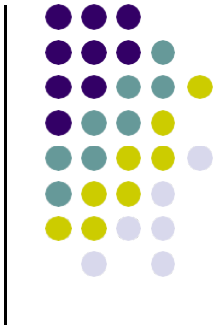


Polygon Clipping



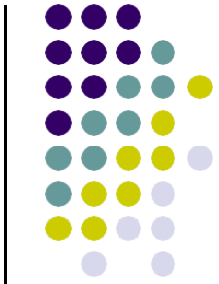
After Clipping

Sutherland-Hodgman Algorithm



- P_1, P_2, ..., P_N be vertex list of the polygon to be clipped (**subject polygon**).
- Edge E, defined by points A and B, be any edge of the positively oriented, convex **clipping polygon**
- Vertex output list - a list containing the vertices that are to be displayed after clipping

Sutherland-Hodgman Algorithm

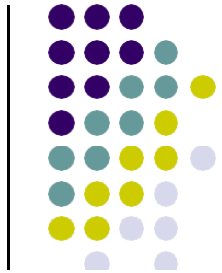


Consider edge $\overline{P_{i_1}P_i}$

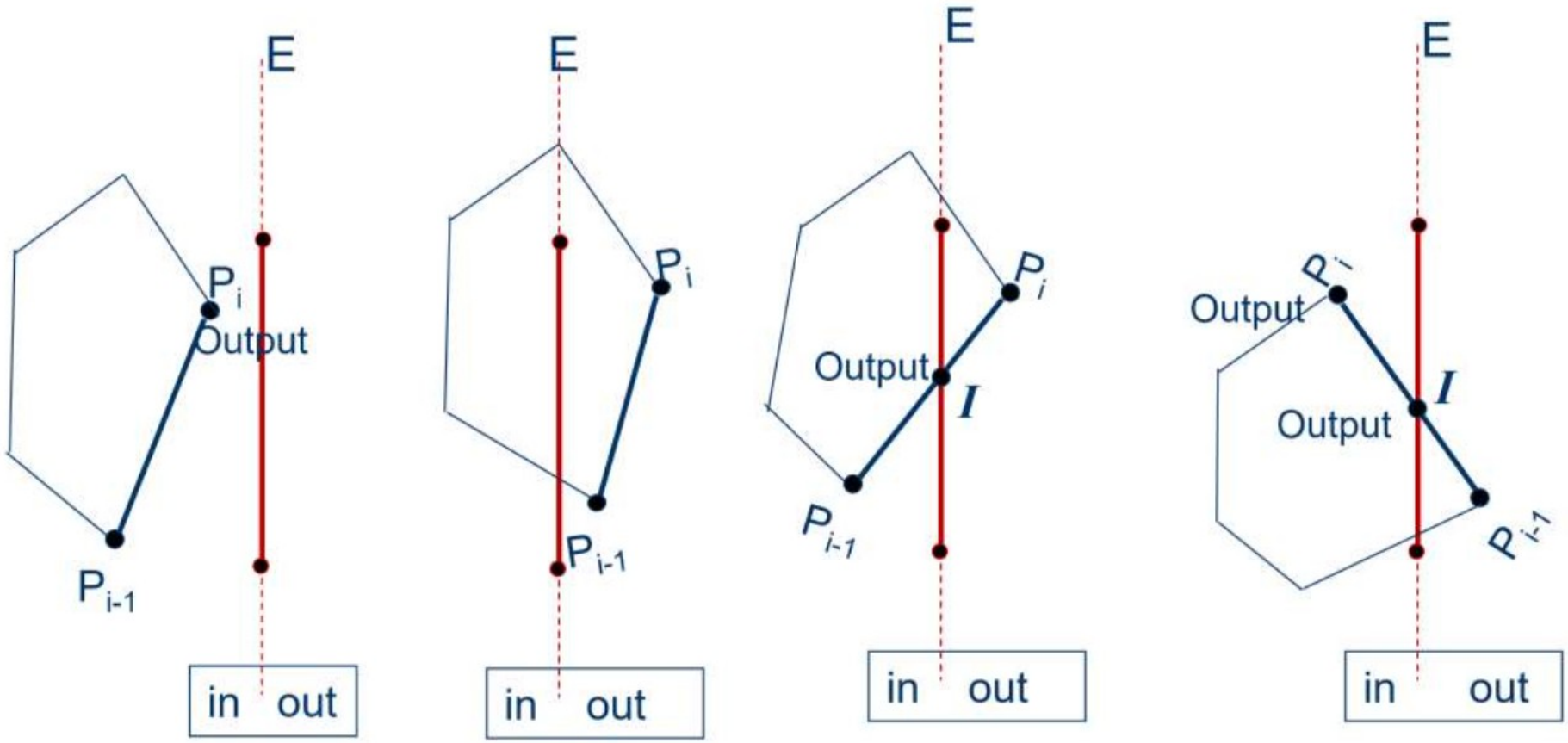
- If both P_{i-1} and P_i are left of the edge E, place P_i to the vertex output list
- If both P_{i-1} and P_i are right of the edge, place nothing to the vertex output list
- If P_{i-1} is left and P_i is right of the edge, find intersect point I and place I to the vertex output list
- If P_{i-1} is right and P_i is left of the edge, find intersect point I and place both I and P_i to the vertex output list

The Algorithm proceeds by passing each clipped polygon to the next edge of the window.

Sutherland-Hodgman Algorithm



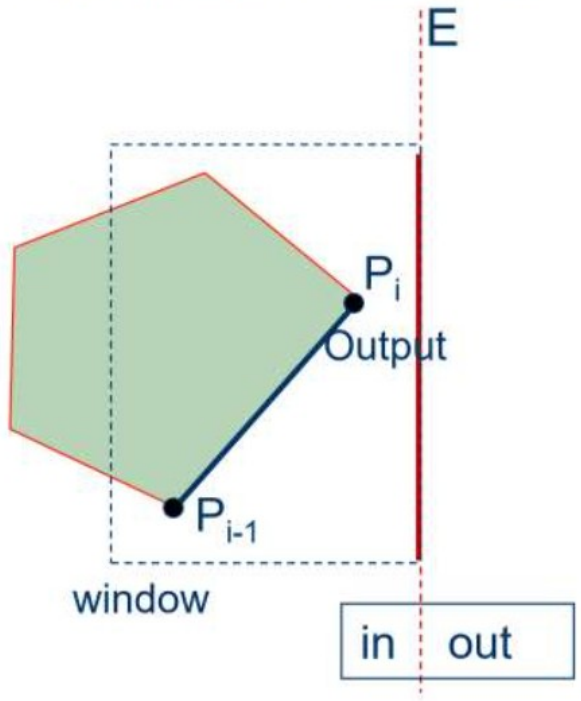
- 4 cases



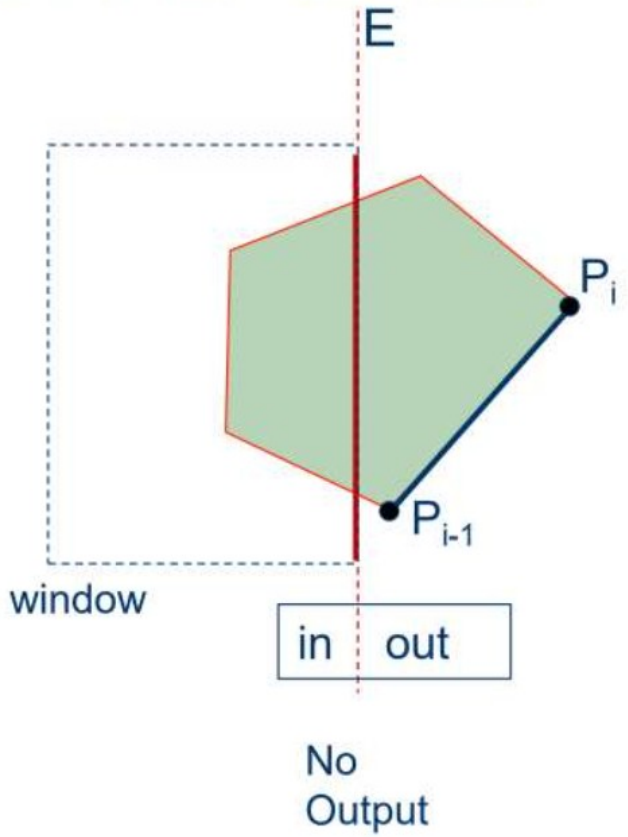
Sutherland-Hodgman Algorithm

- 4 cases

both P_{i-1} and P_i are left / inside



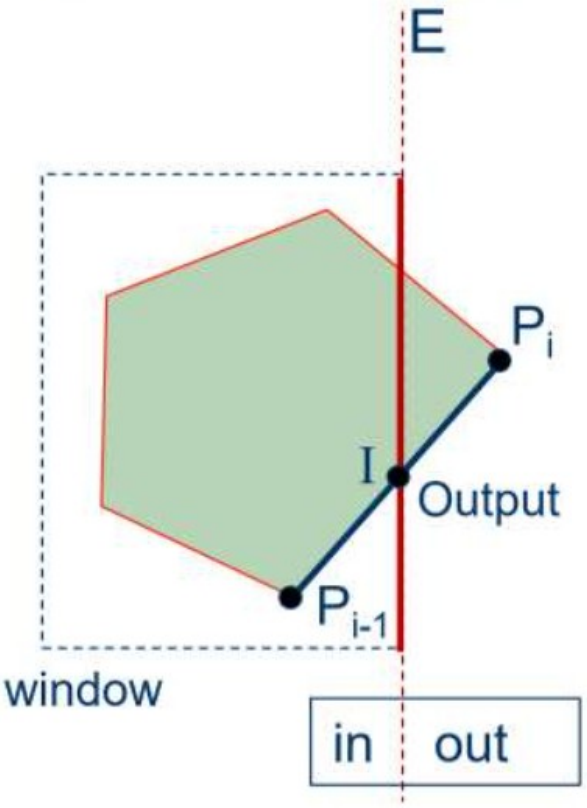
both P_{i-1} and P_i are right / outside



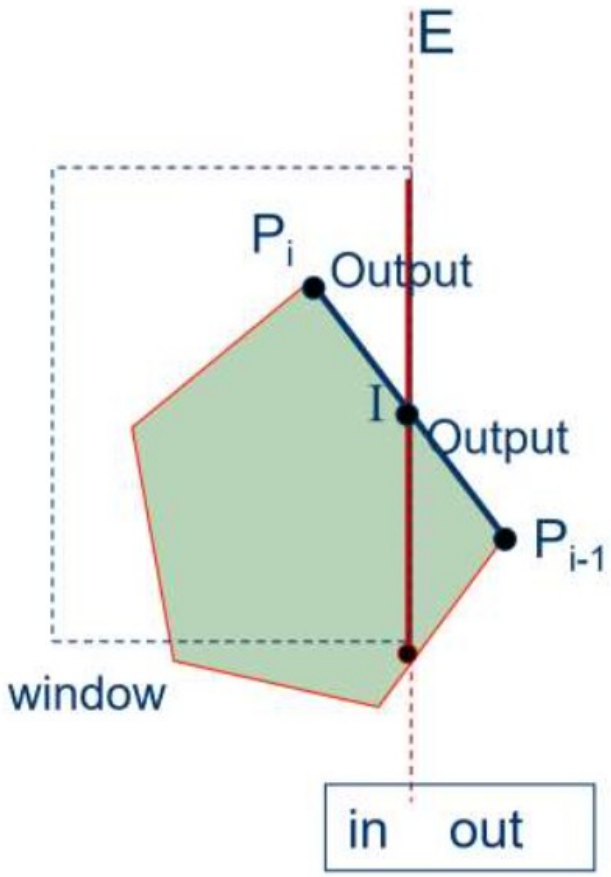
Sutherland-Hodgman Algorithm

- 4 cases

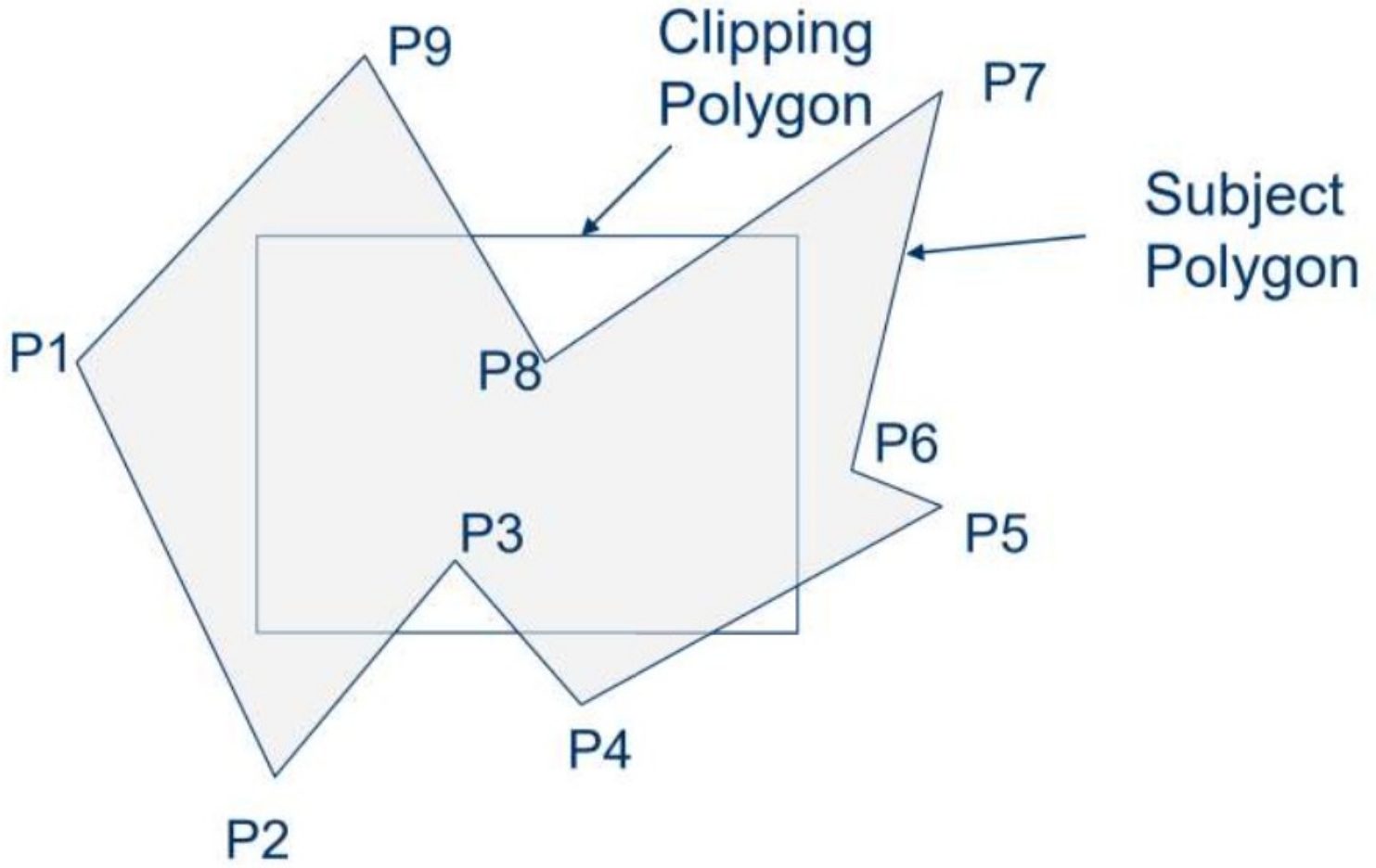
P_{i-1} is left and P_i is right



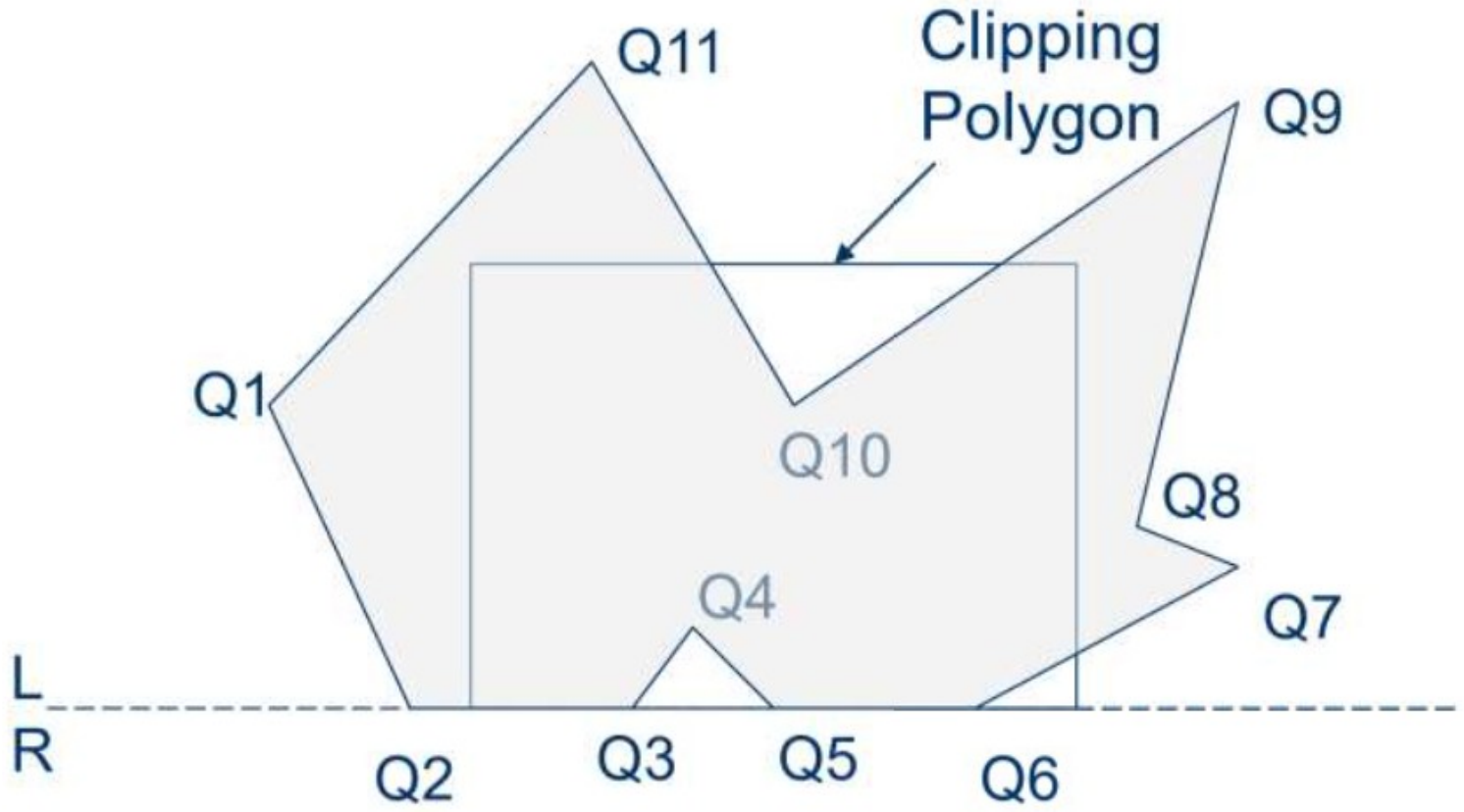
P_{i-1} is right and P_i is left



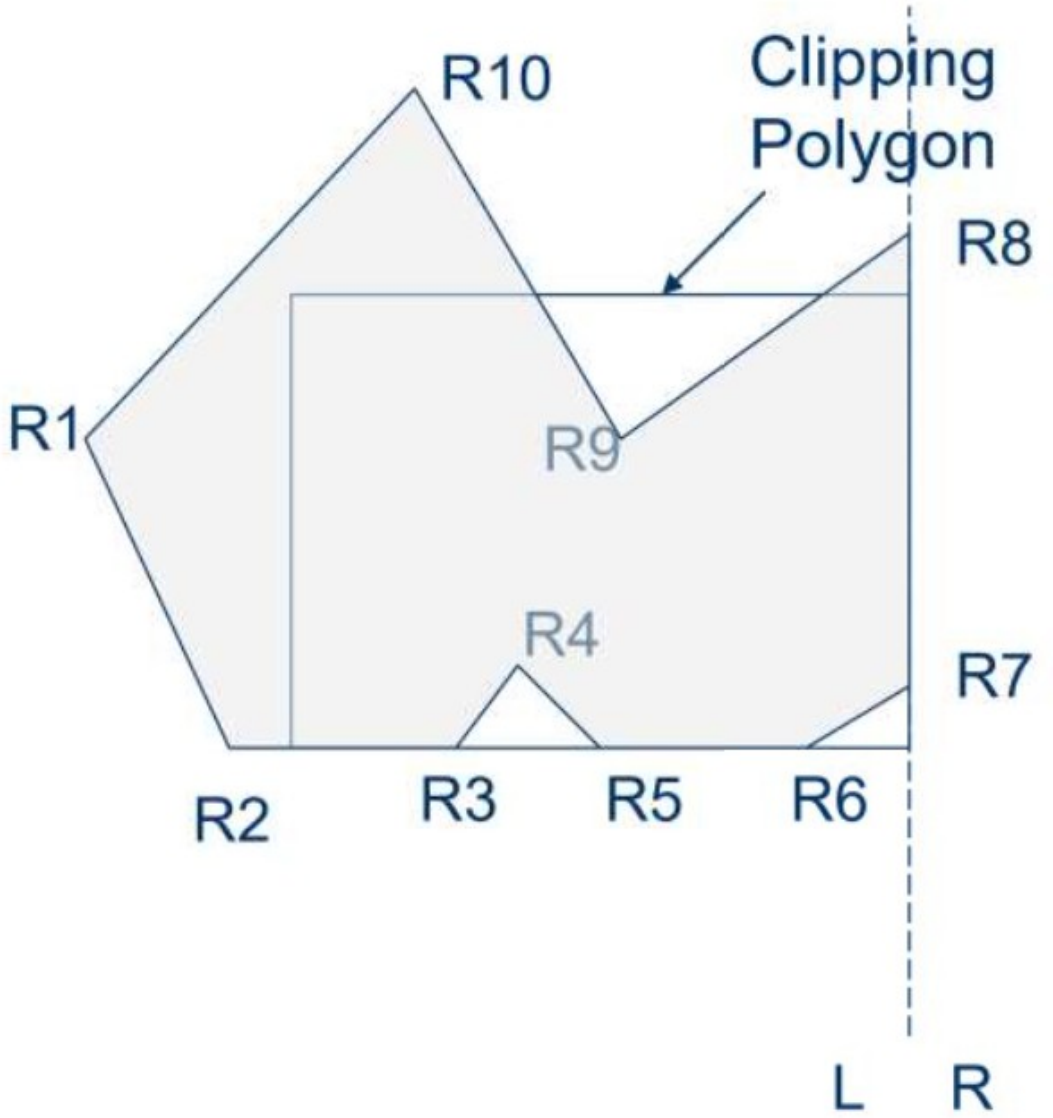
Example



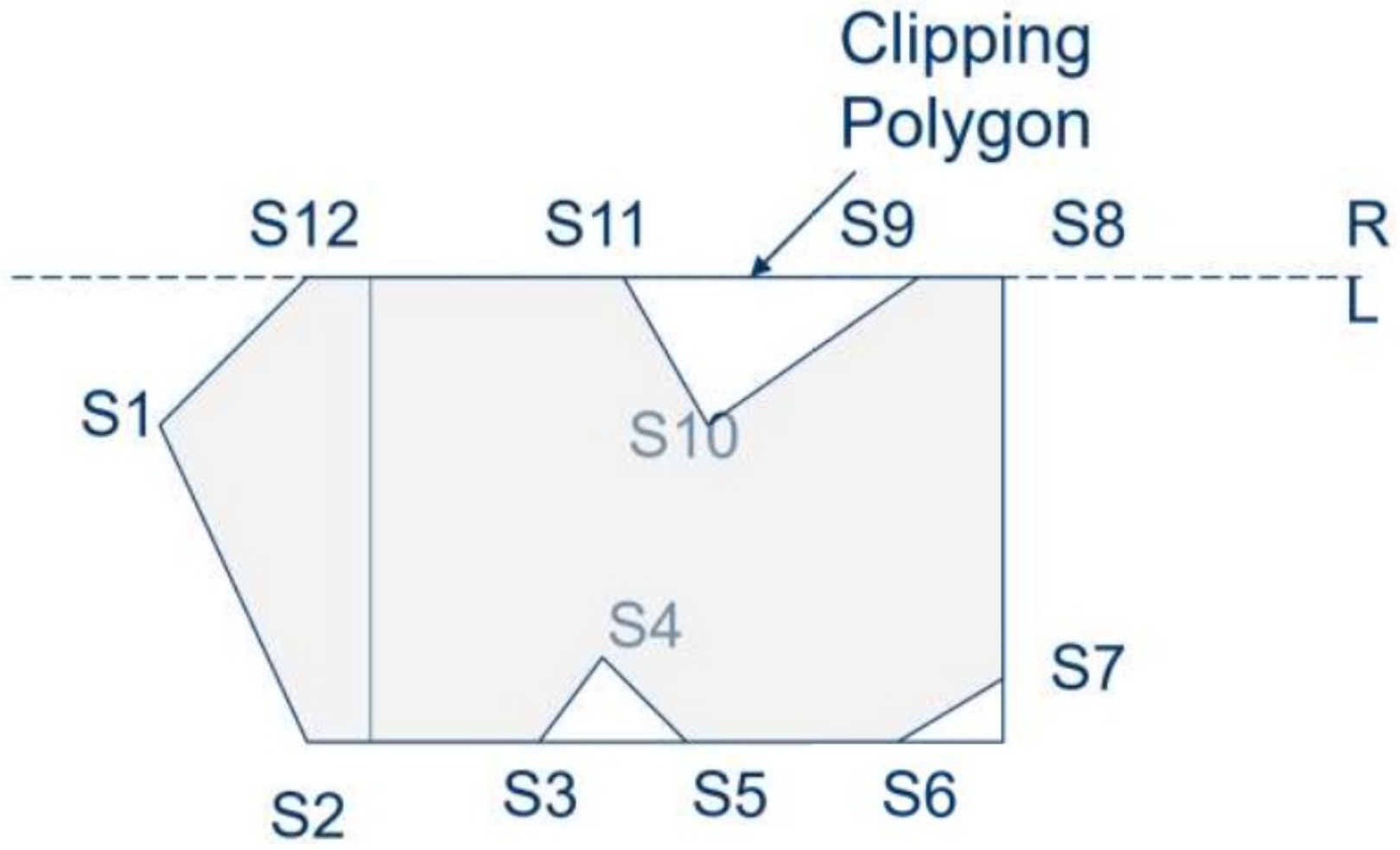
Example



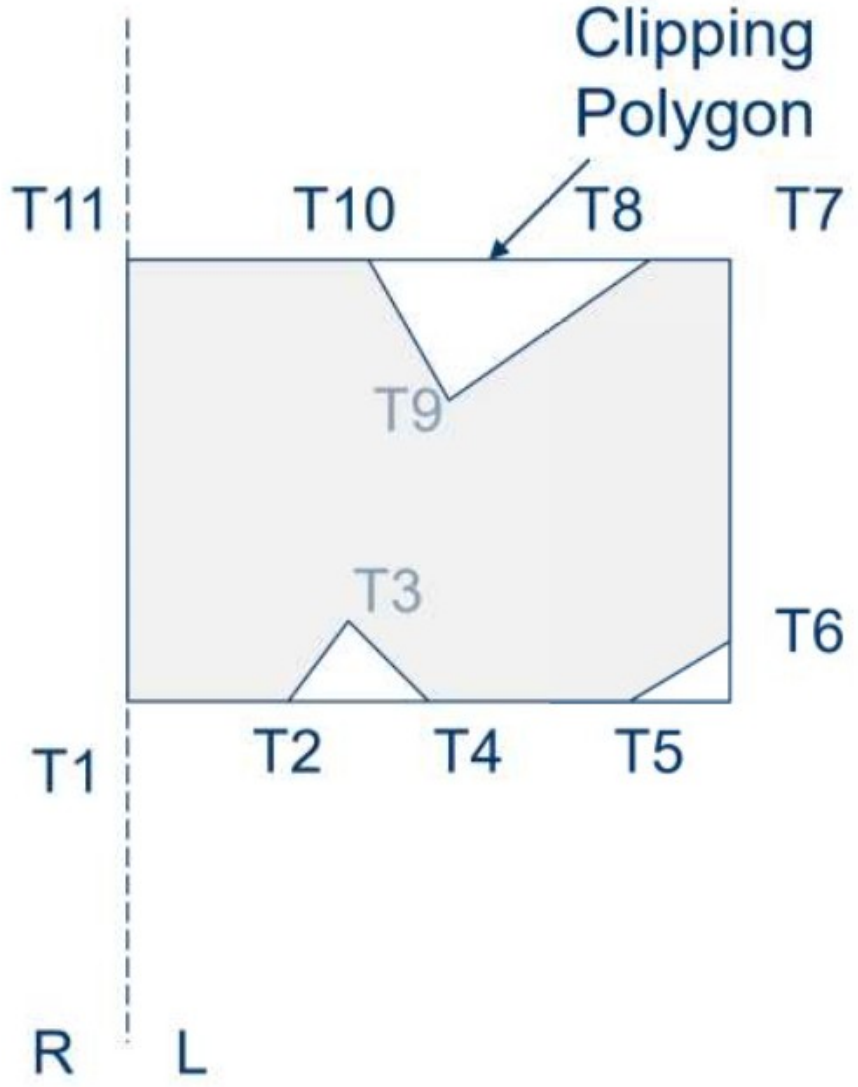
Example



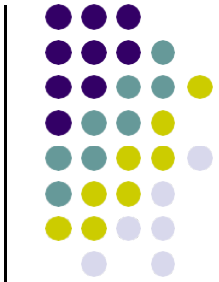
Example



Example



Point-to-line test



- A very general test to determine if a point p is “inside” a line L or plane L for 3D, defined by q and n:

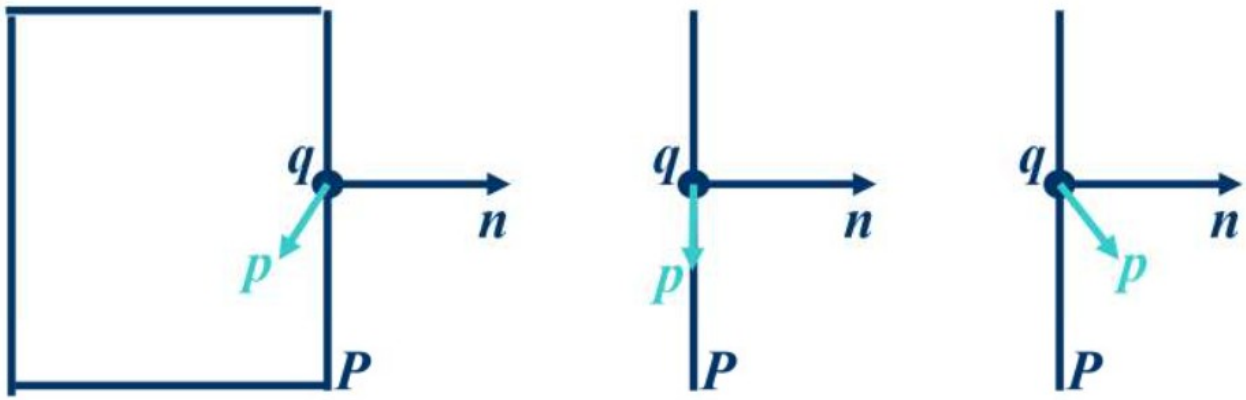
  $(p - q) \cdot n < 0$: p inside L

  $(p - q) \cdot n = 0$: p on L

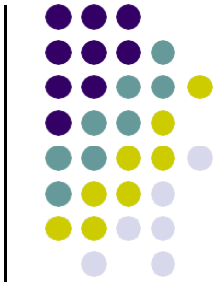
  $(p - q) \cdot n > 0$: p outside L

  Remember: $\boldsymbol{A} \cdot \boldsymbol{B} = |\boldsymbol{A}|\,|\boldsymbol{B}| \cos(\theta) = \boldsymbol{A}_x . \boldsymbol{B}_x + \boldsymbol{A}_y . \boldsymbol{B}_y$

  θ = angle between ***A*** and ***B***



Finding Line-edge interactions



- Use parametric definition of Line and edge:

$$L(t) = P_0 + t(P_1 - P_0)$$

- Line intersects an clipping edge where L(t) is on E
  - q is a point on E
  - n is normal to E

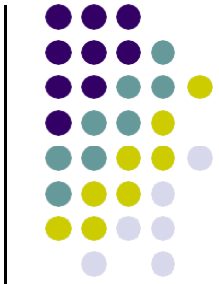
$$(L(t) - q) \cdot n = 0$$

$$(P_0 + t(P_1 - P_0) - q) \cdot n = 0$$

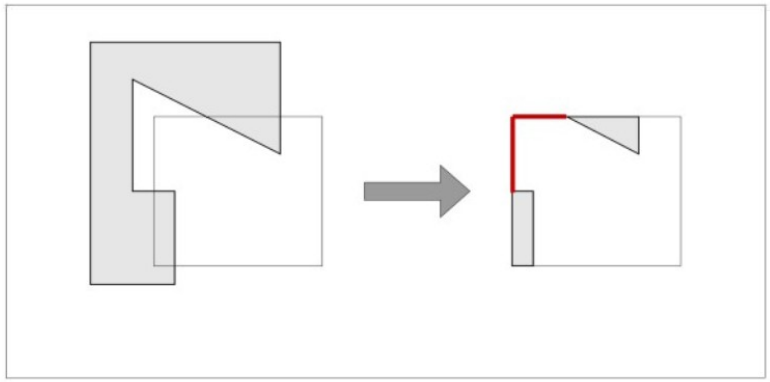
$$t = [(q - P_0) \cdot n] / [(P_1 - P_0) \cdot n]$$

  - The intersection point $I = L(t)$ for this value of t

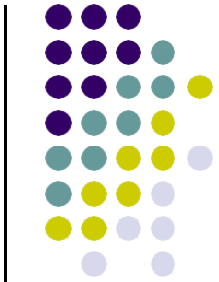
Unwanted effect



- The Sutherland-Hodgman algorithm correctly clips convex polygons, but concave polygons may be displayed with extraneous lines
- Since there is only one output vertex list, the last vertex in the list is always joined to the first vertex.

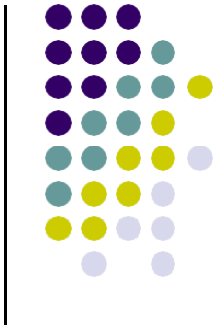


Weiler-Atherton Polygon Clipping



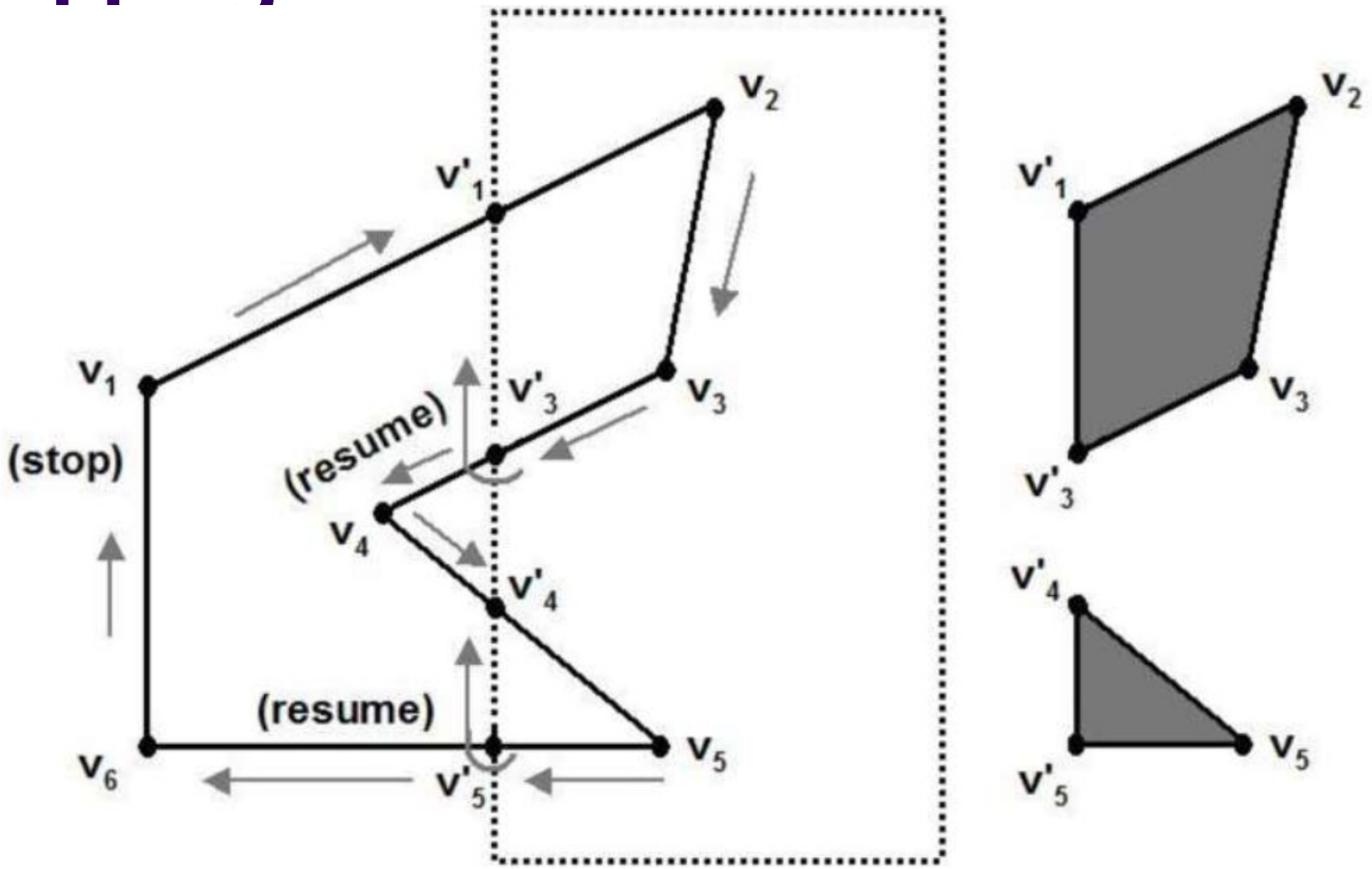
- can be used to clip either a convex or a concave polygon.
- The basic idea of this algorithm is that instead of proceeding around the polygon edges as vertices are processed, we will follow the window boundaries.
- The path we follow depends on:
  - polygon-processing direction (clockwise or counterclockwise)
  - The pair of polygon vertices
    - outside-to-inside or inside-to-outside.

Weiler-Atherton Polygon Clipping

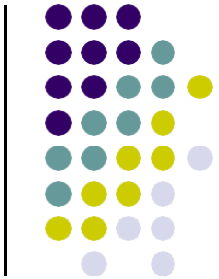


- For clockwise processing of polygon vertices, we use the following rules:
- For an outside-to-inside pair of vertices, follow polygon boundaries.
- For an inside-to-outside pair of vertices, follow window boundaries in a clockwise direction.

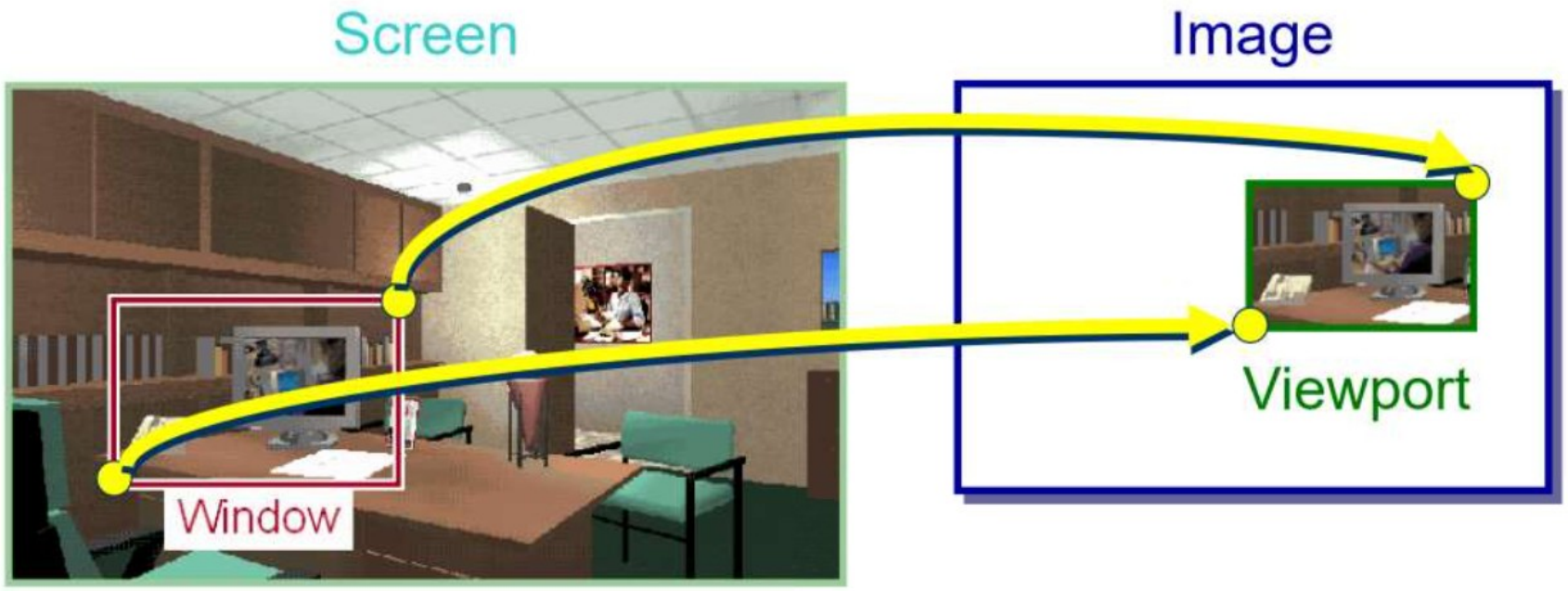
Weiler-Atherton Polygon Clipping



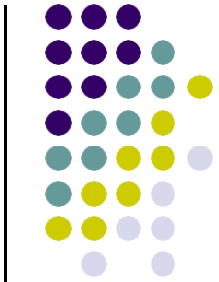
Viewport Transformation



- Transform 2D Geometric Primitives from Screen Coordinate System (Projection Coordinates) to Image Coordinate System (Device Coordinates)



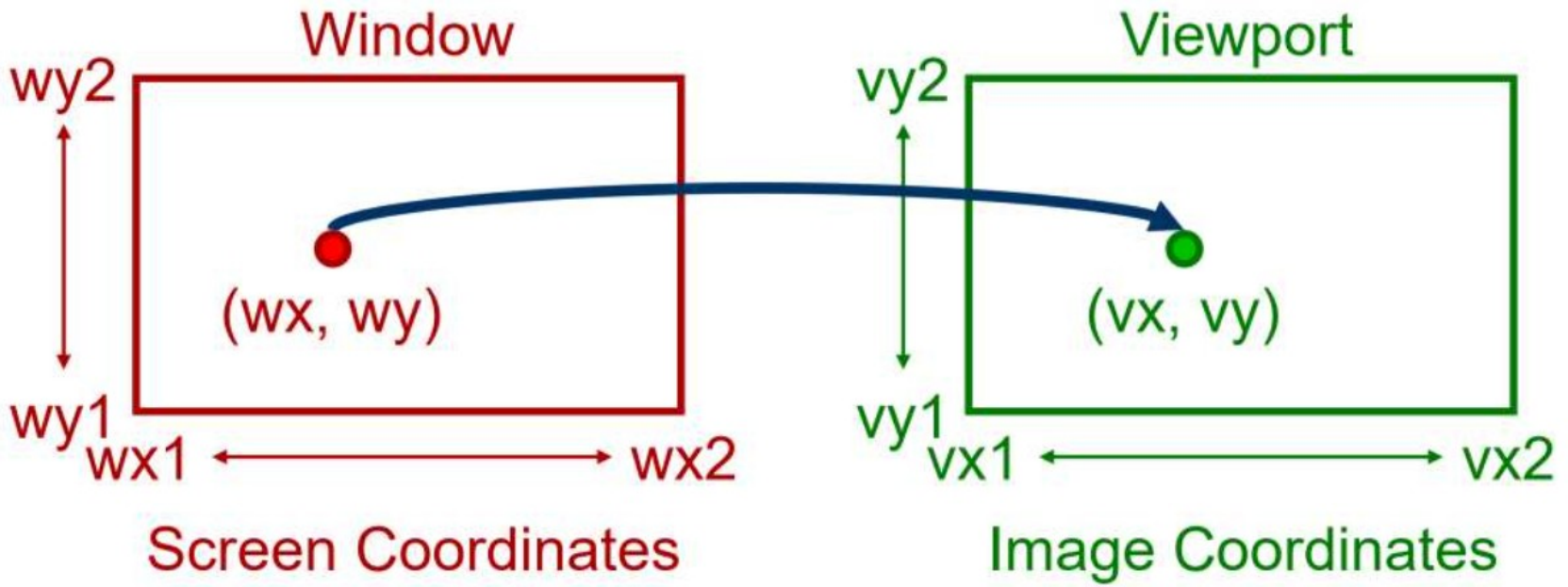
Window vs Viewport



- Window
  - World-coordinate area selected for display
  - What is to be viewed
- Viewport
  - Area on the display device to which a window is mapped
  - Where it is to be displayed

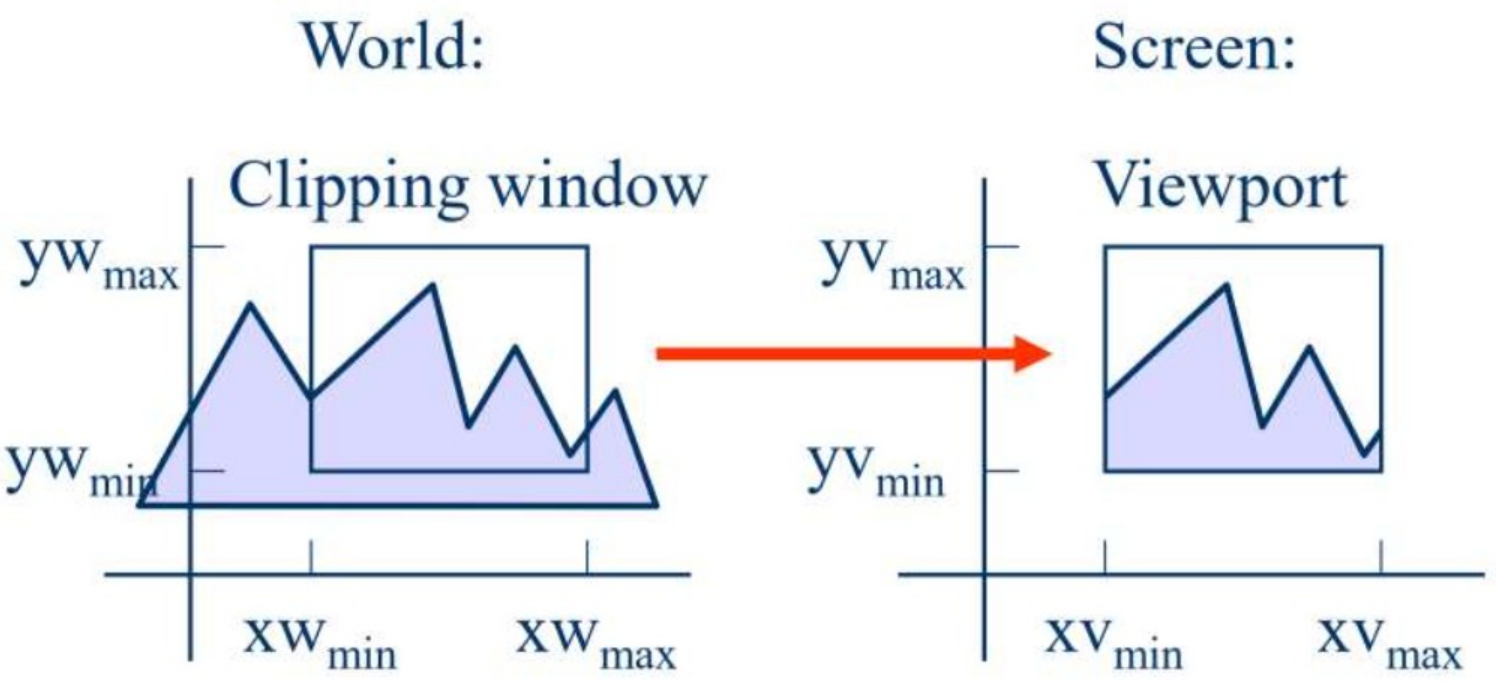
Viewport Transformation

- Window-to-Viewport Mapping




```
vx = vx1 + (wx - wx1) * (vx2 - vx1) / (wx2 - wx1);
vy = vy1 + (wy - wy1) * (vy2 - vy1) / (wy2 - wy1);
```

Viewport Transformation



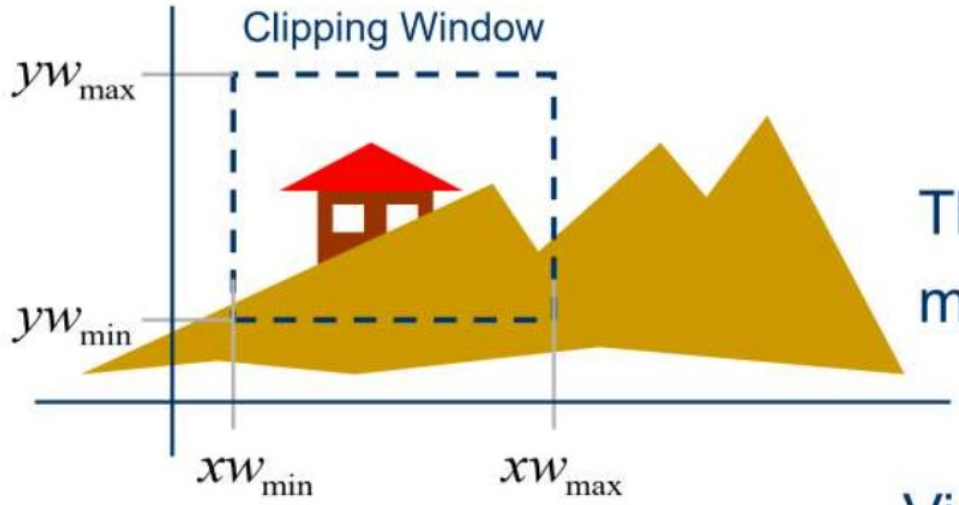
Clipping window:

What do we want to see?

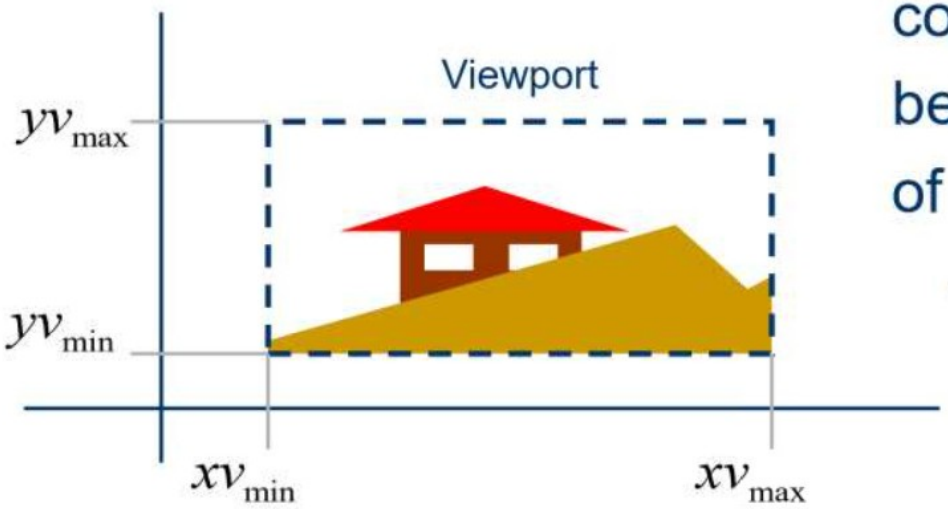
Viewport:

Where do we want to see it?

Viewport Transformation

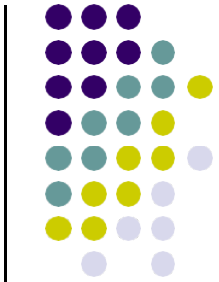


The clipping window is mapped into a viewport.



Viewing world has its own coordinates, which may be a non-uniform scaling of world coordinates.

Viewport Coordinates



References

- Angel and Shreiner, Interactive Computer Graphics, 6th edition
- Hill and Kelley, Computer Graphics using OpenGL, 3rd edition
- Dr. Sk. Md. Masudul Ahsan, (2022), L7 Clipping hov [PDF document], Khulna University of Engineering & Technology.

Scan Conversion

Output Primitives

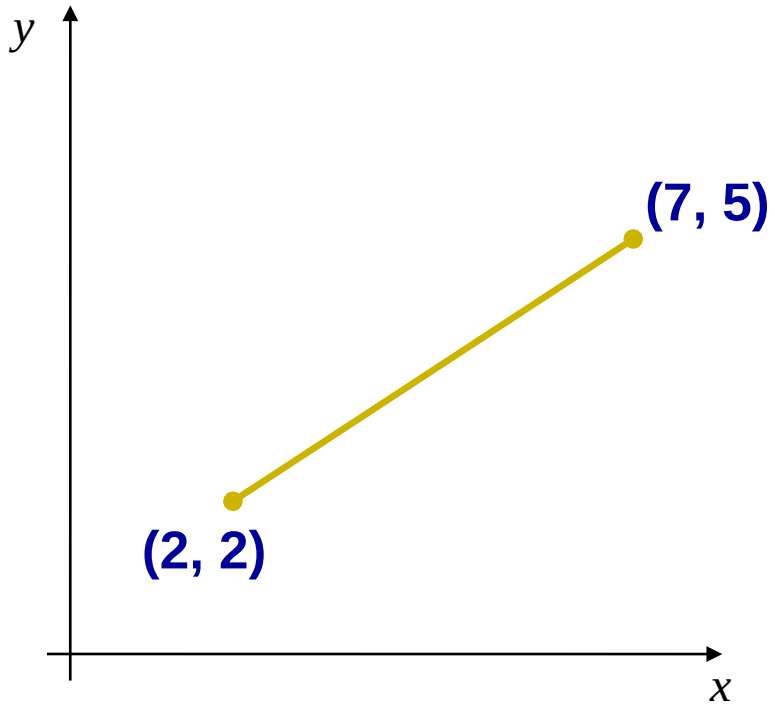
- Basic geometric structures used to describe scenes.
- Can be grouped into more complex structures.
- Each one is specified with input coordinate data and other information about the way that object is to be displayed.
- Examples: point, line and circle each one with specified coordinates.
- Construct the vector picture.

Digital Representation

- Display screen is divided into scan lines and columns.
- Pixels positions are referenced according to scan line number and column number (columns across scan lines).
  - Scan lines start from 0 at screen bottom, and columns start from 0 at the screen left side.
- Screen locations (or pixels) are referenced with integer values.
- The frame buffer stores the intensities temporarily.
- Video controller reads from the frame buffer and plots the screen pixels.

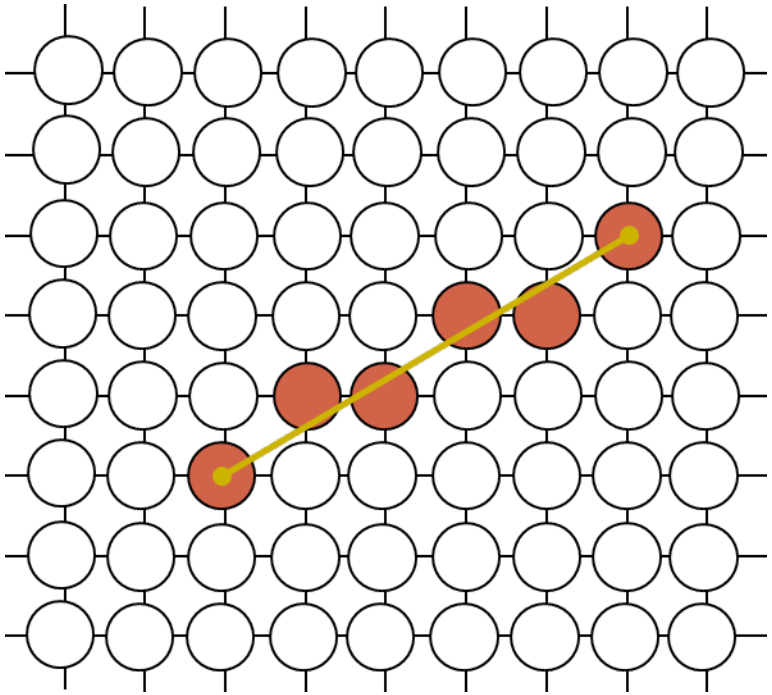
Digital Representation(cont.)

- A line segment in a scene is defined by the coordinate positions of the line end-points.



Digital Representation(cont.)

- But what happens when we try to draw this on a pixel based display?



- How do we choose which pixels to turn on?

Scan Conversion

- The process of representing continuous graphics object as a collection of discrete pixels is called Scan Conversion.
- For e.g a line is defined by its two end points & the line equation, where as a circle is defined by its radius, center position & circle equation.
- It is the responsibility of graphics system or the application program to convert each primitive from its geometric definition into a set of pixels that make up the primitive in image space. This conversion task is generally referred to as a scan conversion or rasterization.

Scan Conversion(cont.)

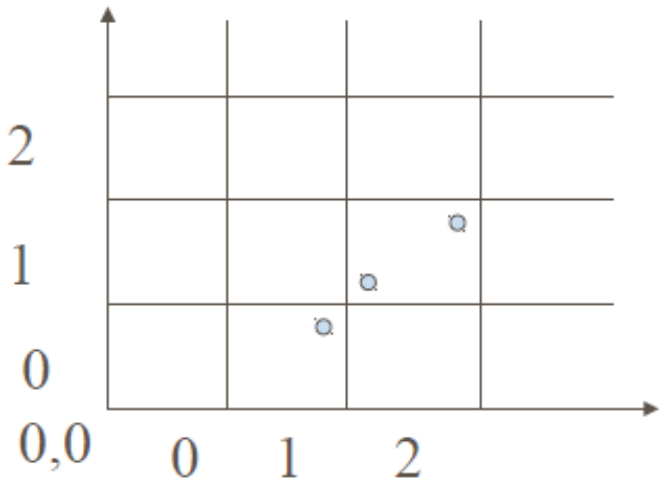
- Drawing lines, circles, and etc. on a grid implicitly involves approximation.
- Ideally, the following properties should be considered
  - smooth
  - continuous
  - pass through specified points
  - uniform brightness
  - efficient

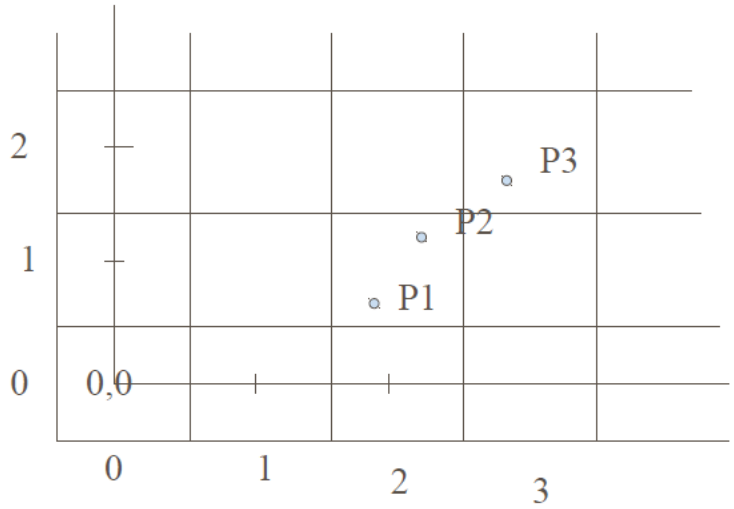
Scan Converting a Point

- A mathematical point (x,y) where x & y are real numbers within an image area, needs to be scan-converted to a pixel at location (x',y').
- Can be done by making x' & y' to be the integer part of x & y.
  - x'= Floor(x) and y'= Floor(y)
  - P1(1.7,0.8) is represented by pixel (1,0)
  - P2(2.2,1.3) and P3(2.8,1.9) are both represented by pixel (2,1).

Scan Converting a Point(cont.)

- Another approach is to scan convert (x,y) by making
  - x’ = Floor(x + 0.5) and y’ =Floor(y+0.5).
  - Points P1 and P2 are now both represented by pixel (2,1) whereas point P3 is represented by pixel (3,2).
- This essentially places the origin of the coordinate system for (x,y) at the center of pixel(0,0).





Scan Converting a Line

- A line is defined by its two end points & the slope intercept equation for a line:
  - y = mx + b, m = Slope of the line, b = the y intercept of a line
- Line drawing is done by:
  - Calculating intermediate positions between the endpoints.
  - Directing the output device to fill in the calculated positions as in the case of plotting single points.

Scan Converting a Line(cont.)

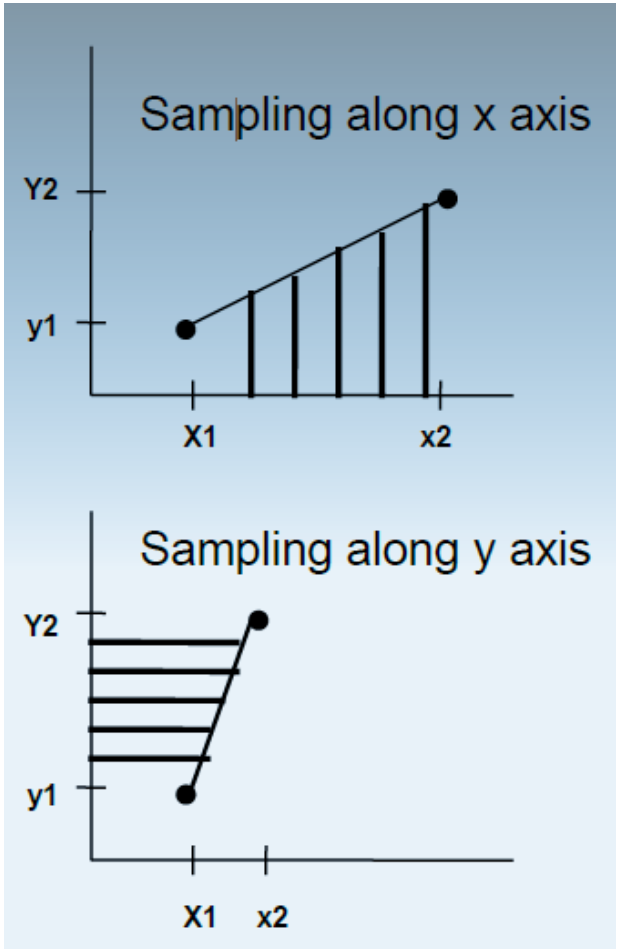
- Plotted positions may be only approximations to the actual line positions between endpoints.
  - A computed position (10.48, 20.51) is converted to pixel (10,21).
- This rounding causes the lines to be displayed with a stairstep appearance.
- Stairsteps are noticeable in low resolution systems, it can be improved by:
  - Displaying lines on high resolution systems.
  - Adjusting intensities along line path.

Scan Converting a Line(cont.)

- The Cartesian intercept equation for a straight line:
  - $y = m.x + b$
- For line segment starting in (x1,y1) and ending in (x2,y2), the slop is:
  - m= (y1-y2)/(x1-x2)
  - b= y1- m.x1
- For any given x interval Δx, we can compute the corresponding y interval Δy:
  - Δy= m . Δx
- Or x interval Δx from a given Δy:
  - Δx= Δy/m

Scan Converting a Line(cont.)

- On raster systems, lines are plotted with pixels, and step sizes in the horizontal and vertical directions are constrained by pixel separations.
- Scan conversion process samples a line at discrete positions and determine the nearest pixel to the line at each sampled position. (Incremental Fashion)



Line Drawing - Algorithm 1

- A Straightforward Implementation

```
DrawLine(int x1,int y1, int x2,int y2, int
color)
{
    float y;
    int x;

    for (x=x1; x<=x2; x++)
    {
        y = y1 + (x-x1)*(y2-y1)/(x2-x1)
        WritePixel(x, Round(y), color );
    }
}
```

Line Drawing - Algorithm 2

- A Better Implementation

```
DrawLine(int x1,int y1,int x2,int y2, int color)
{
    float m,y;
    int dx,dy,x;
    dx = x2 - x1;
    dy = y2 - y1;
    m = dy/dx;
    y = y1 + 0.5;
    for (x=x1; x<=x2; x++)
    {
        WritePixel(x, Floor(y), color );
        y = y + m;
    }
}
```

Line Drawing Algorithm Comparison

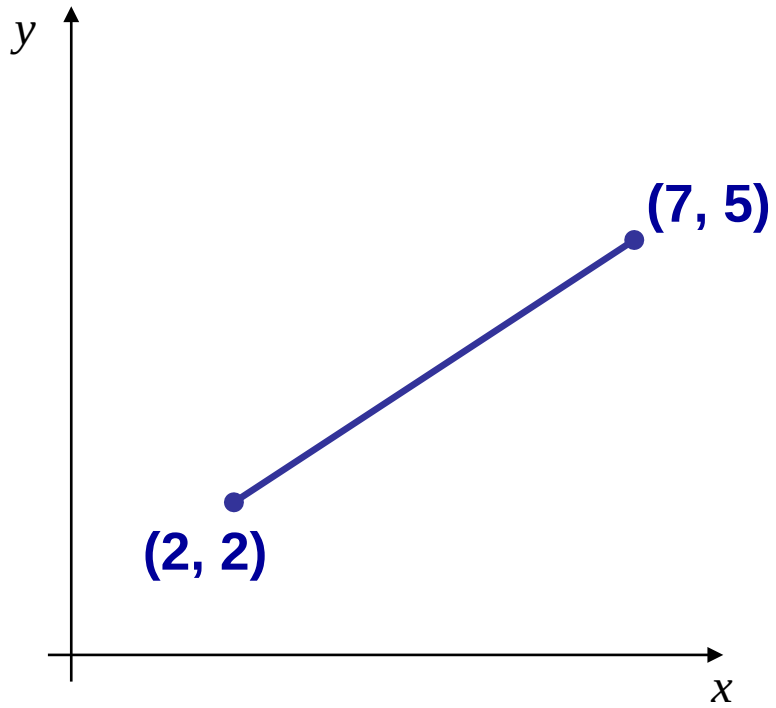
- Advantages over Algorithm 1
  - eliminates multiplication
  - improves speed
- Disadvantages
  - round-off error builds up
  - get pixel drift
  - rounding and floating point arithmetic still time consuming
  - works well only for $|m| < 1$
  - need to loop in y for $|m| > 1$
  - need to handle special cases

Reference

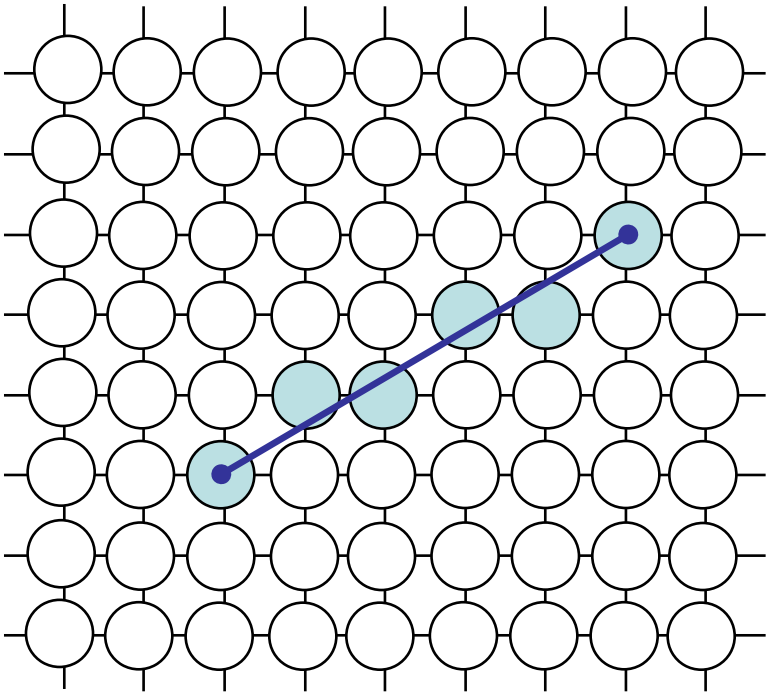
- **Computer Graphics**
  - R. Plastock, Zhigang Xiang
  - (Schaum's Outline Series) McGraw Hill

The Problem Of Scan Conversion

A line segment in a scene is defined by the coordinate positions of the line end-points



The Problem (cont…)



How do we choose which pixels to turn on?

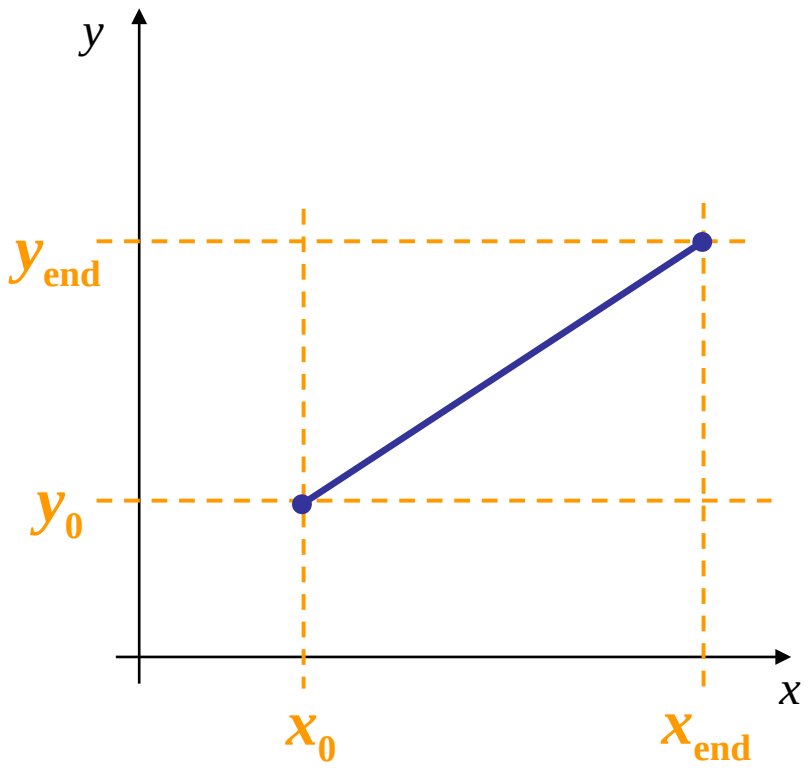
Considerations

Considerations to keep in mind:

- The line has to look good
  - Avoid *jaggies*
- It has to be lightening fast!
  - How many lines need to be drawn in a typical scene?
  - This is going to come back to bite us again and again

Line Equations

Let's quickly review the equations involved in drawing lines



Slope-intercept line equation:

$$y = m \cdot x + b$$

where:

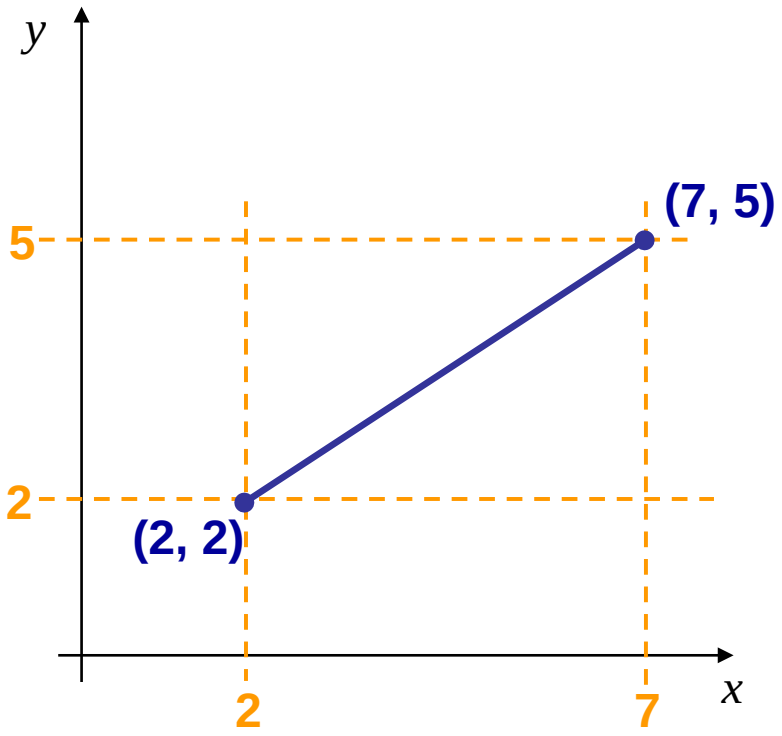
$$m = \frac{y_{end} - y_0}{x_{end} - x_0}$$

$$b = y_0 - m \cdot x_0$$

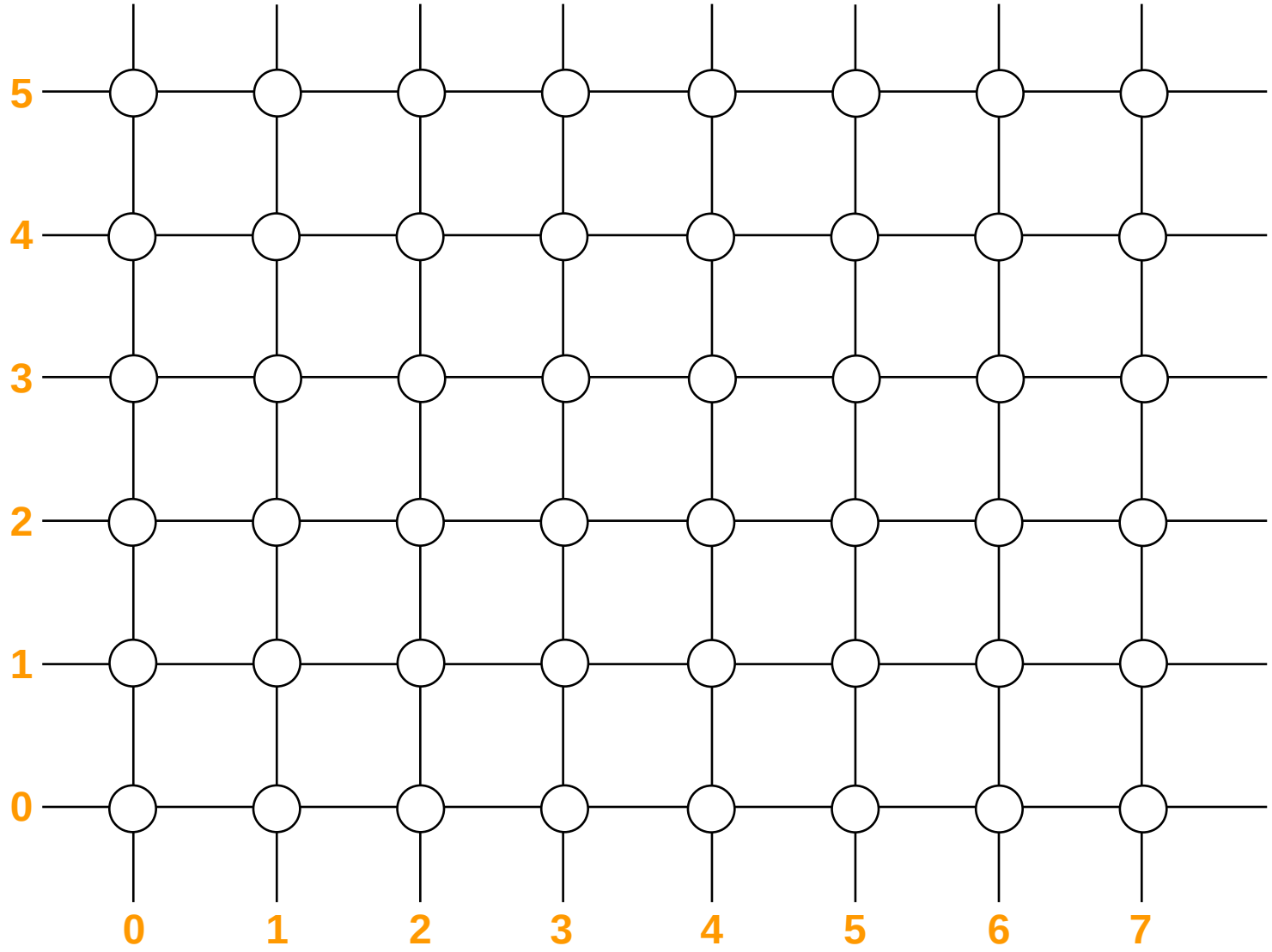
A Very Simple Solution

We could simply work out the corresponding y coordinate for each unit x coordinate

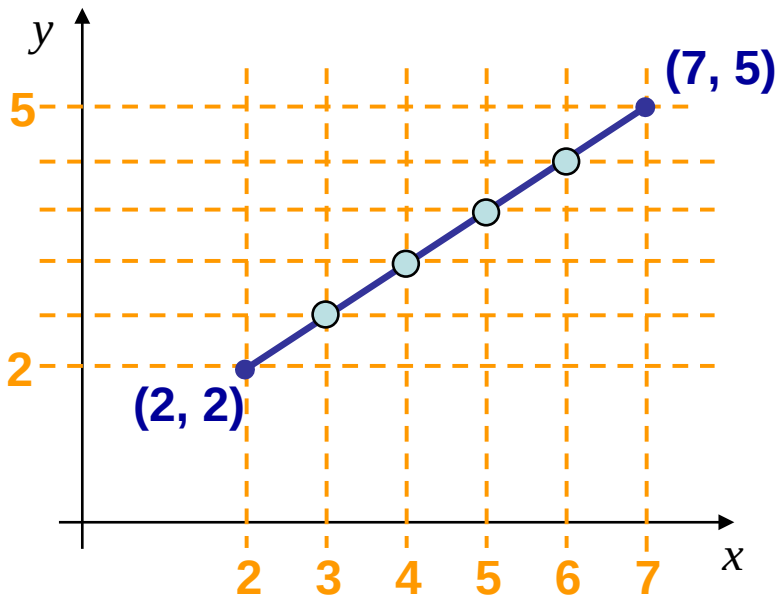
Let's consider the following example:



A Very Simple Solution (cont...)



A Very Simple Solution (cont...)



First work out m and b:

$$m = \frac{5 - 2}{7 - 2} = \frac{3}{5}$$

$$b = 2 - \frac{3}{5} \star 2 = \frac{4}{5}$$

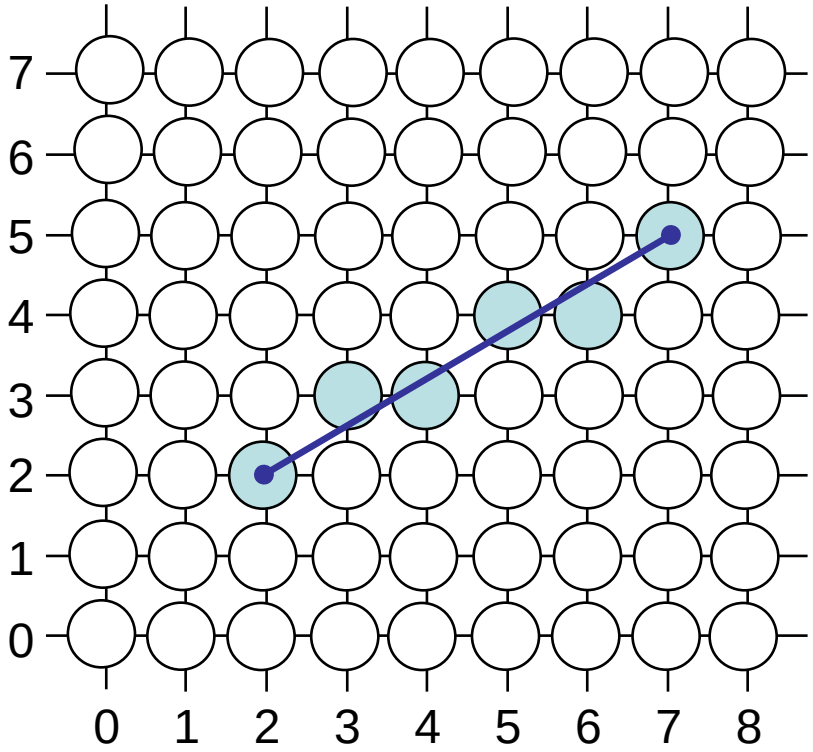
Now for each x value work out the y value:

$$y(3) = \frac{3}{5} \cdot 3 + \frac{4}{5} = 2\frac{3}{5} \qquad y(4) = \frac{3}{5} \cdot 4 + \frac{4}{5} = 3\frac{1}{5}$$

$$y(5) = \frac{3}{5} \cdot 5 + \frac{4}{5} = 3\frac{4}{5} \qquad y(6) = \frac{3}{5} \cdot 6 + \frac{4}{5} = 4\frac{2}{5}$$

A Very Simple Solution (cont...)

Now just round off the results and turn on these pixels to draw our line



$$y(3) = 2\frac{3}{5} \approx 3$$

$$y(4) = 3\frac{1}{5} \approx 3$$

$$y(5) = 3\frac{4}{5} \approx 4$$

$$y(6) = 4\frac{2}{5} \approx 4$$

A Very Simple Solution (cont...)

However, this approach is just way too slow

In particular look out for:

- The equation $y = mx + b$ requires the multiplication of m by x
- Rounding off the resulting y coordinates

We need a faster solution

A Quick Note About Slopes

In the previous example we chose to solve the parametric line equation to give us the y coordinate for each unit x coordinate

What if we had done it the other way around?

So this gives us: $x = \frac{y - b}{m}$

where: $m = \frac{y_{end} - y_0}{x_{end} - x_0}$ and $b = y_0 - m \cdot x_0$

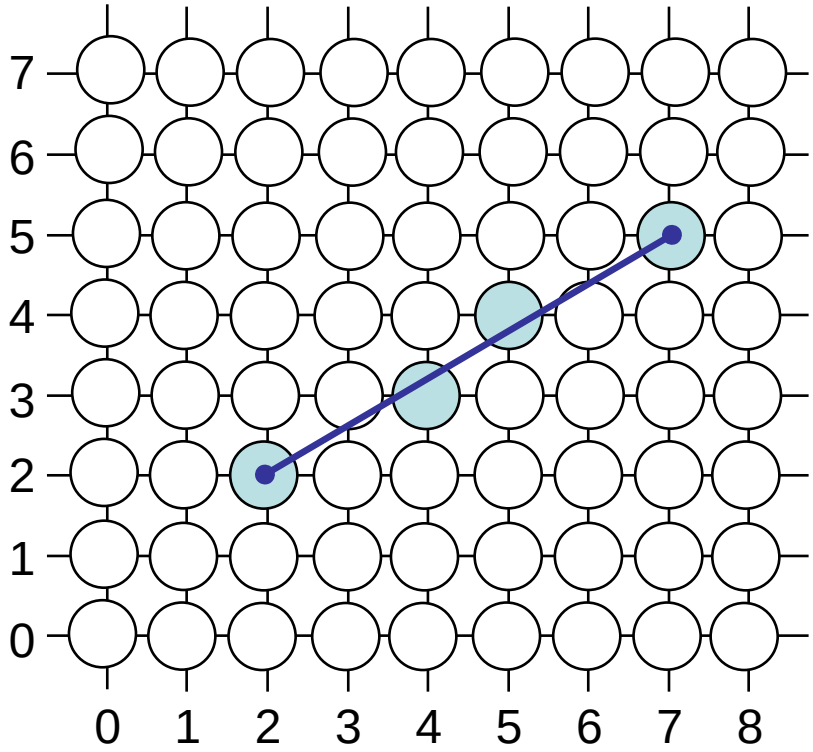
A Quick Note About Slopes (cont…)

Leaving out the details this gives us:

$$x(3) = 3\frac{2}{3} \approx 4 \qquad x(4) = 5\frac{1}{3} \approx 5$$

We can see easily that this line doesn't look very good!

We choose which way to work out the line pixels based on the slope of the line



A Quick Note About Slopes

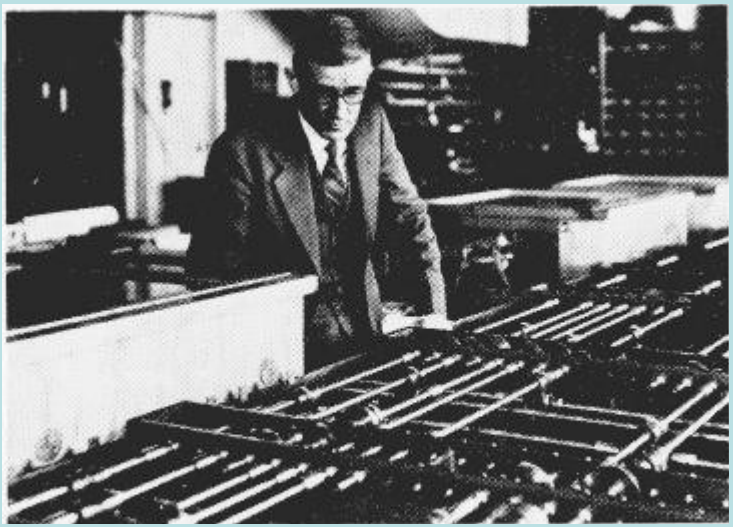
- If **|m|<1**, then for every integer value of x between and excluding x1 and x2, calculate the corresponding value of y using equation **Δy = m Δx** & scan convert (x,y).
- If **|m|>1**, then for every integer value of y between and excluding y1 and y2, calculate the corresponding value of x using equation **Δx = Δy/m** & scan convert (x,y).
- If **|m|=1**, **Δx = Δy.** In each case, a smooth line with slope m is generated between the specific endpoints.

The DDA Algorithm

The *digital differential analyzer* (DDA) algorithm takes an incremental approach in order to speed up scan conversion

Simply calculate y_{k+1}

based on y_k



The original differential analyzer was a physical machine developed by Vannevar Bush at MIT in the 1930's in order to solve ordinary differential equations.

More information here.

The DDA Algorithm (cont…)

Consider the list of points that we determined for the line in our previous example:

$$(2, 2), (3, 2^3/_5), (4, 3^1/_5), (5, 3^4/_5), (6, 4^2/_5), (7, 5)$$

Notice that as the x coordinates go up by one, the y coordinates simply go up by the slope of the line

This is the key insight in the DDA algorithm

The DDA Algorithm (cont…)

When the slope of the line is between -1 and 1 begin at the first point in the line and, by incrementing the x coordinate by 1, calculate the corresponding y coordinates as follows:

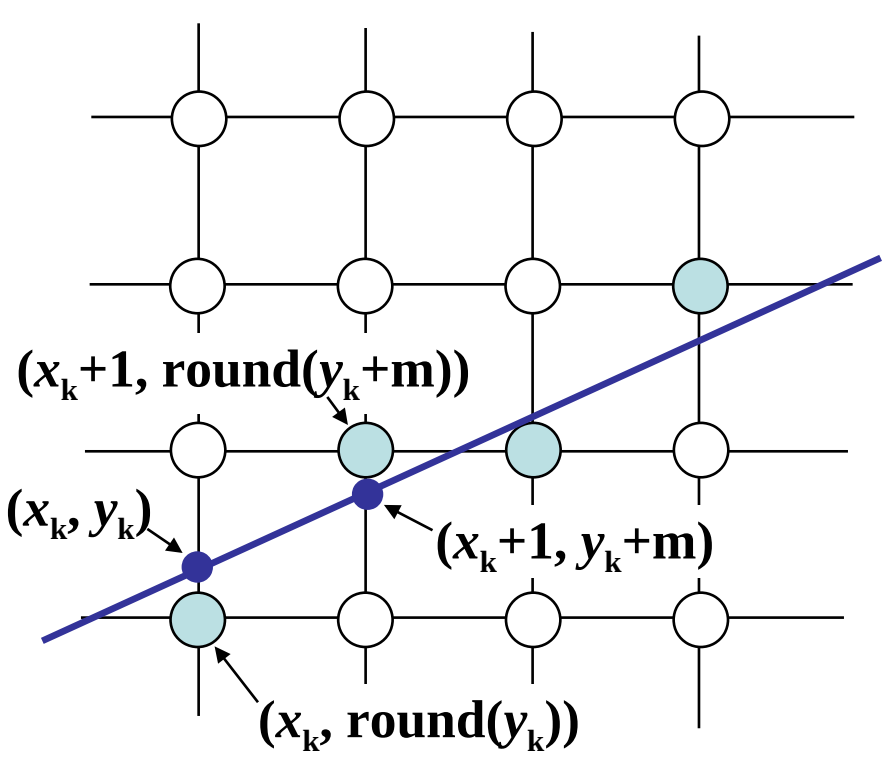
$$y_{k+1} = y_k + m$$

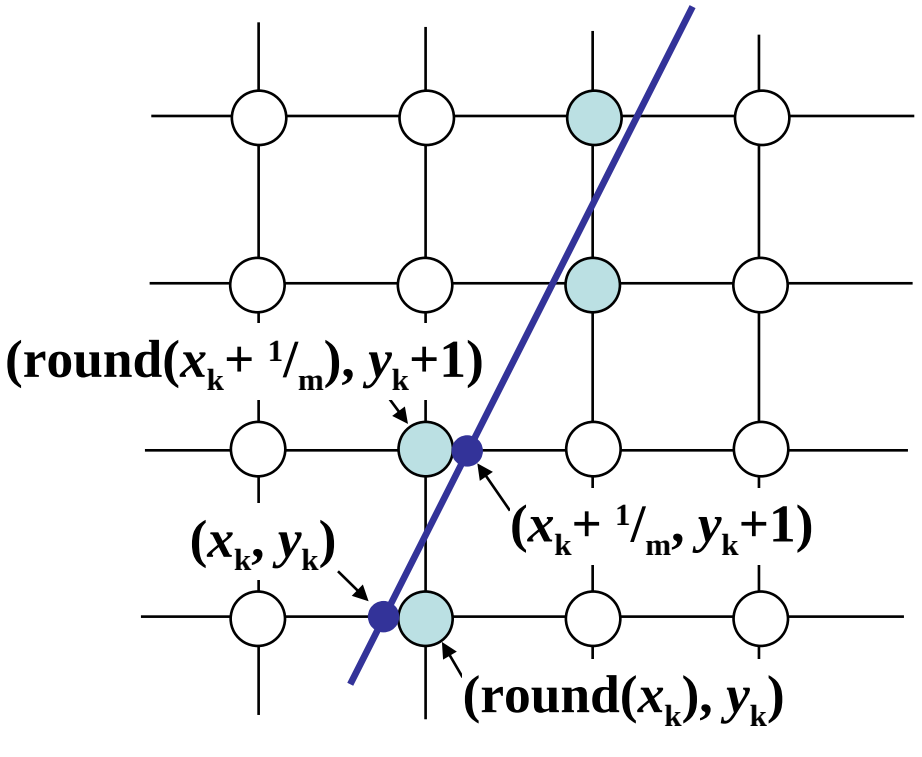
When the slope is outside these limits, increment the y coordinate by 1 and calculate the corresponding x coordinates as follows:

$$x_{k+1} = x_k + \frac{1}{m}$$

The DDA Algorithm (cont…)

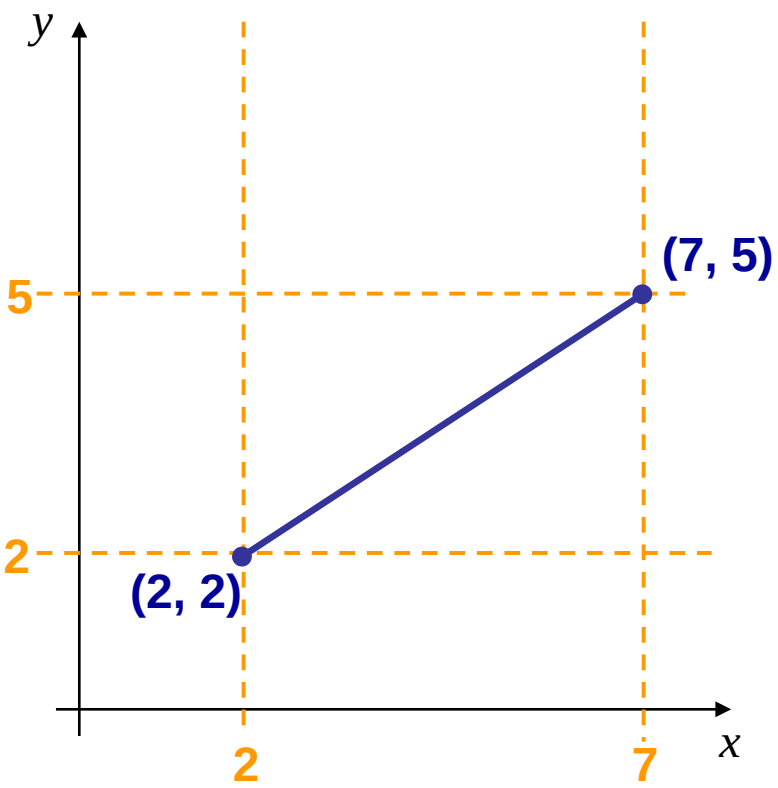
Again the values calculated by the equations used by the DDA algorithm must be rounded to match pixel values

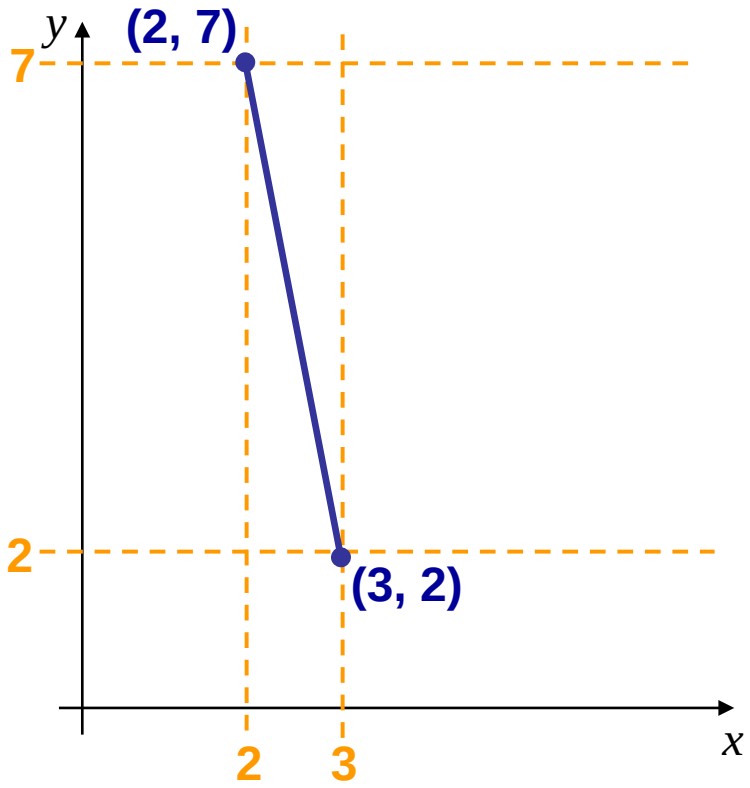




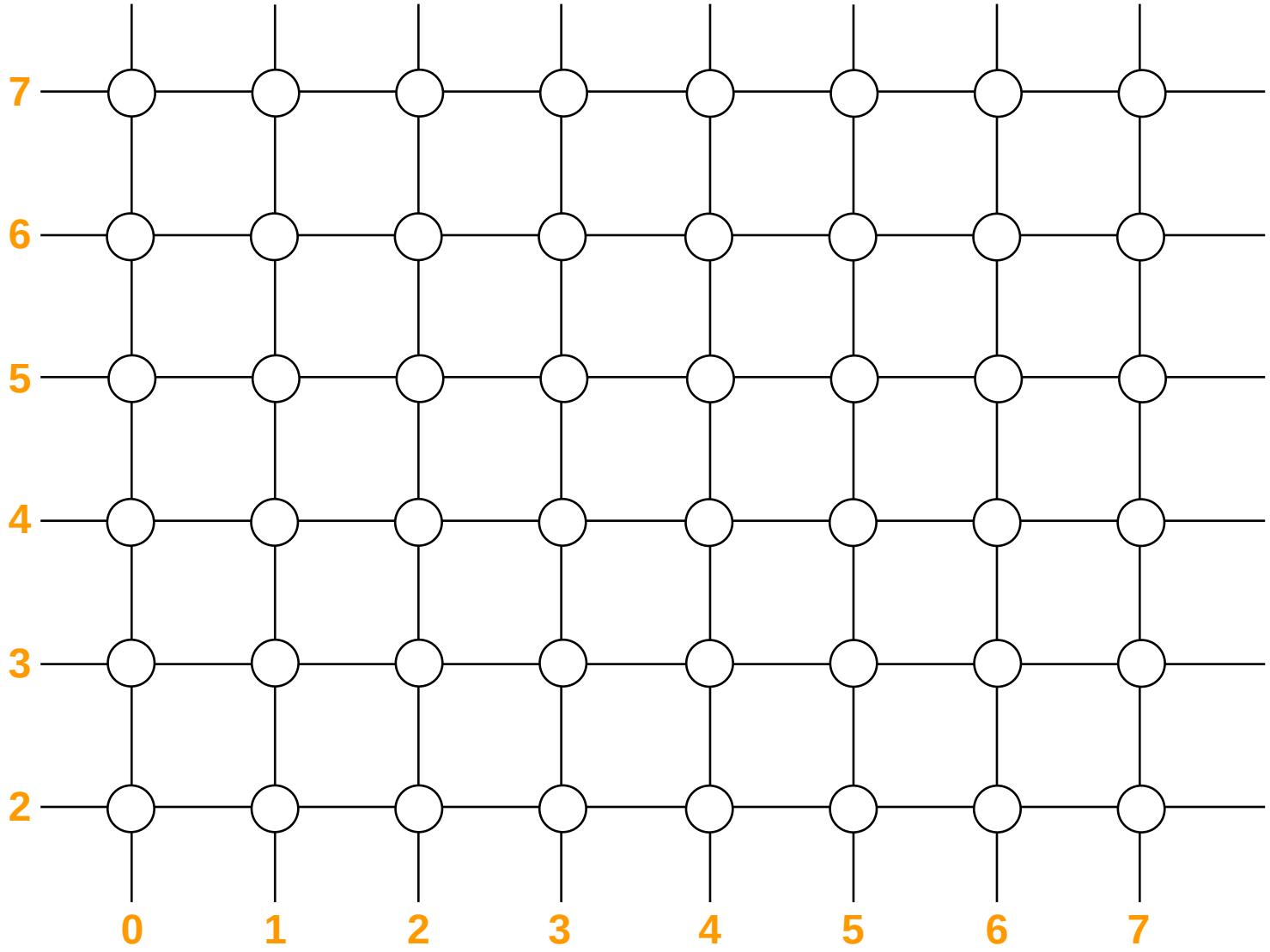
DDA Algorithm Example

Let’s try out the following examples:





DDA Algorithm Example (cont…)



The DDA Algorithm Summary

The DDA algorithm is much faster than our previous attempt

- In particular, there are no longer any multiplications involved

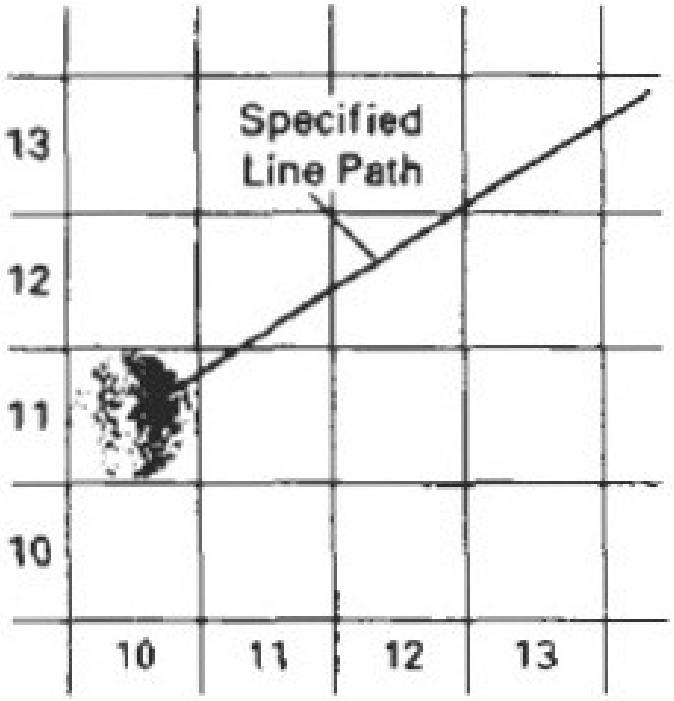
However, there are still two big issues:

- Accumulation of round-off errors can make the pixelated line drift away from what was intended
- The rounding operations and floating point arithmetic involved are time consuming

Line Drawing Algorithms
(Bresenham's Line Algorithm)

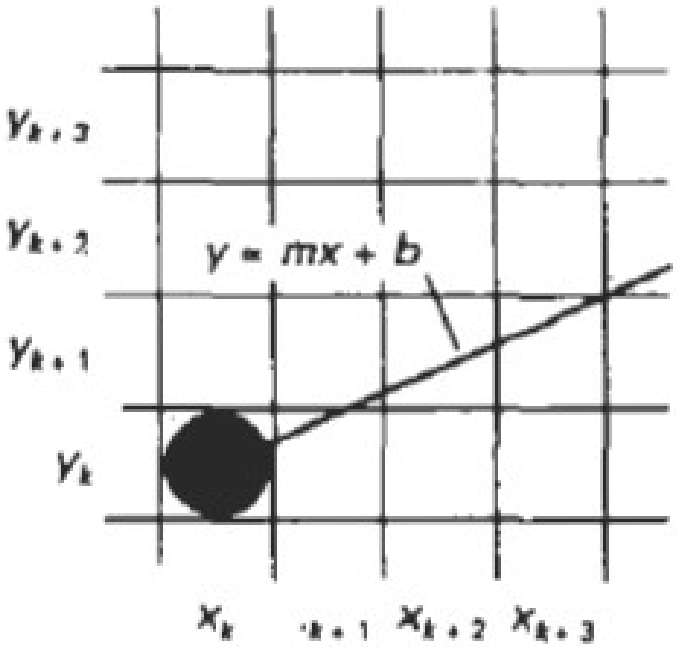
Bresenham's Line Algorithm

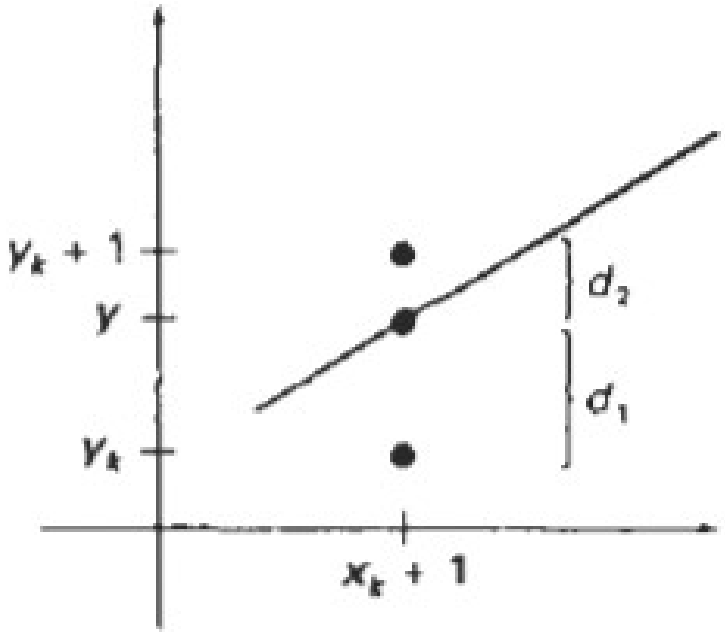
- DDA algorithm has a disadvantage, that is, the round off error. This error happens because the algorithm rounds off the actual floating point line values to integer values (for example, if actual value of a point of line is 12.34, the DDA algorithm rounds it to 12 and if the value is 12.62, it will round it to 13). This causes the calculated pixel positions to move away from the actual line path for long lines.
- For this reason we use another accurate and efficient line drawing algorithm called Bresenham's algorithm because it was developed by Bresenham.
- This algorithm draws line by using only integer calculations.



- Horizontal axis shows the pixel columns
- Vertical axis shows the line position.
- If we take unit x-intervals and we plot a point at (x_k,y_k), then in order to calculate the y values we need to check which of the two points is closer to the line path at each sample step. That is, we need to decide at the next sample position x_{k+1} whether to choose the pixel (x_{k+1},y_k) or (x_{k+1},y_{k+1}), that is, whether to choose point (11,11) or (11,12) for the next step

- These issues are solved by Bresenham's algorithm.
- It decides which value to take at next position whether (x_{k+1}, y_k) or (x_{k+1}, y_{k+1}). It does this by checking the sign of a decision parameter (p_k) which is equal to the difference between the separations of the two pixel positions (y_k and y_{k+1}) from the actual line path.
- If p_k is negative it takes the point as (x_{k+1}, y_k).
- If p_k is positive it takes point as (x_{k+1}, y_{k+1})


y_{k+3}
y_{k+2}
y_{k+1}
y_k
$y = mx + b$
x_k
x_{k+1}
x_{k+2}
x_{k+3}


$y_k + 1$
y
y_k
d_2
d_1
$x_k + 1$

- Suppose we have determined the first pixel position as (x_k,y_k).
- Now, we have to decide which pixel to plot at x_k+1?

  That is, whether (x_k+1,y_k) or (x_k+1, y_k+1).
- At position (x_k+1), we label separation between pixels y_k and y_k+1 from the line path (y) as d_1 and d_2. that is, $d_1 = y – y_k$; and $d_2 = y_k+1 - y$
- Then we calculate the decision parameter p_k as

  $p_k = (d_1-d_2)$. [or something similar!]

At pixel $x_k + 1$, we can calculate y as:

$$y = m.x + b$$

$$\Rightarrow y = m(x_k + 1) + b$$

Then $d_1 = y - y_k$

$$d_1 = m(x_k + 1) + b - y_k$$

$$d_2 = y_k + 1 - y$$

$$d_2 = y_k + 1 - (m(x_k + 1) + b)$$

$$d_2 = y_k + 1 - m(x_k + 1) - b$$

$$d_1 - d_2 = (m(x_k + 1) + b - y_k) - (y_k + 1 - m(x_k + 1) - b)$$

$$d_1 - d_2 = m(x_k + 1) + b - y_k - y_k - 1 + m(x_k + 1) + b$$

$$d_1 - d_2 = 2m(x_k + 1) - 2y_k + 2b - 1$$

The decision parameter p_k for k^{th} step is given as

$p_k = \delta x(d1 - d2)$

$\Rightarrow p_k = 2\delta y \,.\, x_k - 2\delta x.\, y_k + 2\delta y + \delta x\,(2b - 1)$

$\Rightarrow p_k = 2\delta y.\, x_k - 2\delta x.\, y_k + c$

c is a constant as it is not dependent on pixel positions (x_k, y_k)

The decision parameter p_k for step k + 1 is given as

$\rightarrow p_{k+1} = 2\delta y.\, x_{k+1} - 2\delta x.\, y_{k+1} + c$

$\rightarrow p_{k+1} - p_k = 2\delta y\,(x_{k+1} - x_k) - 2\delta x\,(y_{k+1} - y_k)$

Here, $x_{k+1} = x_k + 1$

$\Rightarrow p_{k+1} - p_k = 2\delta y\,(x_k + 1 - x_k) - 2\delta x\,(y_{k+1} - y_k)$

$\Rightarrow p_{k+1} - p_k = 2\delta y\,(1) - 2\delta x\,(y_{k+1} - y_k)$

$\Rightarrow p_{k+1} = p_k + 2\delta y - 2\delta x\,(y_{k+1} - y_k)$

- If the pixel at y_k is closer to line path than y_{k+1} (that is, $d_1 < d_2$) then p_k is negative and we take y-value as y_k that is we plot the point as (x_{k+1}, y_k).
- Then the term $(y_{k+1} – y_k)$ will become $(y_k – y_k)$, which is equal to 0.
- Therefore, $p_{k+1} = p_k + 2\delta y – 2\delta x$ (0)
- $p_{k+1} = p_k + 2\delta y$
- If y_{k+1} is closer to line path than y_k (that is $d_1 > d_2$) then p_k is positive and we take y-value as y_{k+1} that is we plot the point as (x_{k+1}, y_{k+1}).
- Then the term $(y_{k+1} – y_k)$ will become $(y_k + 1 – y_k)$, which is equal to 1.
- Therefore, $p_{k+1} = p_k + 2\delta y – 2\delta x$ (1)
- $p_{k+1} = p_k + 2\delta y – 2\delta x$

Algorithm

1. Input the line endpoints and store left endpoint in (x0, y0)
2. Load (x0, y0) into frame buffer and plot the first point.
3. Calculate δx, δy, 2δy and 2δy - 2δx

   calculate p0 = 2δy – δx

4. At each x_k, starting at k = 0, do the following

   if $p_k < 0$, next point to plot is (x_{k+1}, y_k) and

   $p_{k+1} = p_k + 2\delta y$

   if $p_k > 0$, next point to plot is (x_{k+1}, y_{k+1}) and

   $p_{k+1} = p_k + 2\delta y - 2\delta x$

Step 4 is repeated δx times.

That’s not all folks!

Unfortunately we just calculated a single case

There are another 7 cases which are similar

Assignment

1. Derive the algorithm for all the 8 octaves
2. Solve 1 problem for each octave with the corresponding algorithm.
3. Date of submission:

19/01/2026 Class Time

Thank You

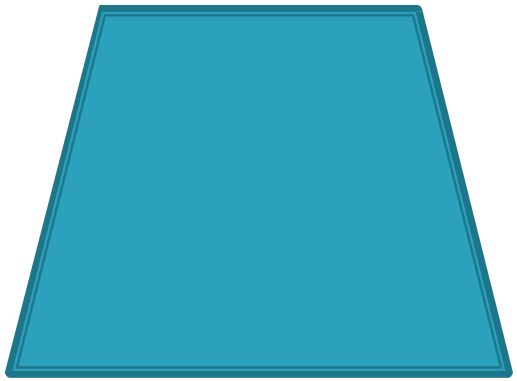
Scan Conversion: Region Filling

NAZIRUL HASAN

Lecturer, CSE, KUET

Region Filling Algorithm

- Region Filling algorithms are used to fill the interior of a polygonal shape.
- Many algorithms perform fill operations by first identifying the interior points, given the polygon boundary.

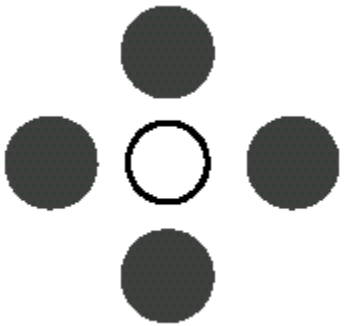


Region Filling

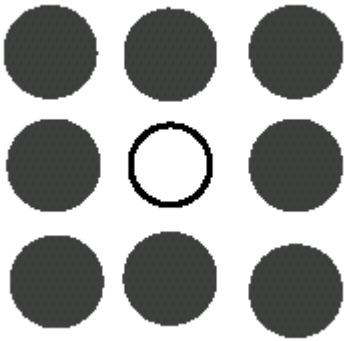
- Seed Fill Approaches
  - 2 algorithms: Boundary Fill and Flood Fill (Seed Fill)
  - works at the pixel level
  - suitable for interactive painting applications
- Scan-line Fill Approaches
  - works at the polygon level
  - better performance

Seed Fill Algorithms: Connectedness

- 4-connected region: From a given pixel, the region that you can get to by a series of 4 way moves (N, S, E and W)
- 8-connected region: From a given pixel, the region that you can get to by a series of 8 way moves (N, S, E, W, NE, NW, SE, and SW)



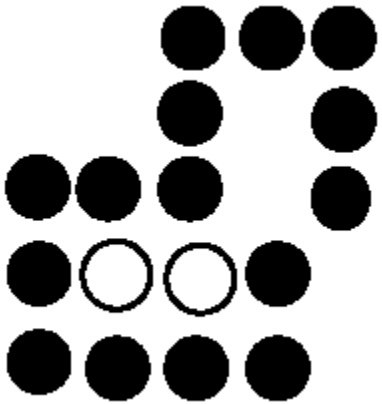
4-connected



8-connected

Boundary Fill Algorithm

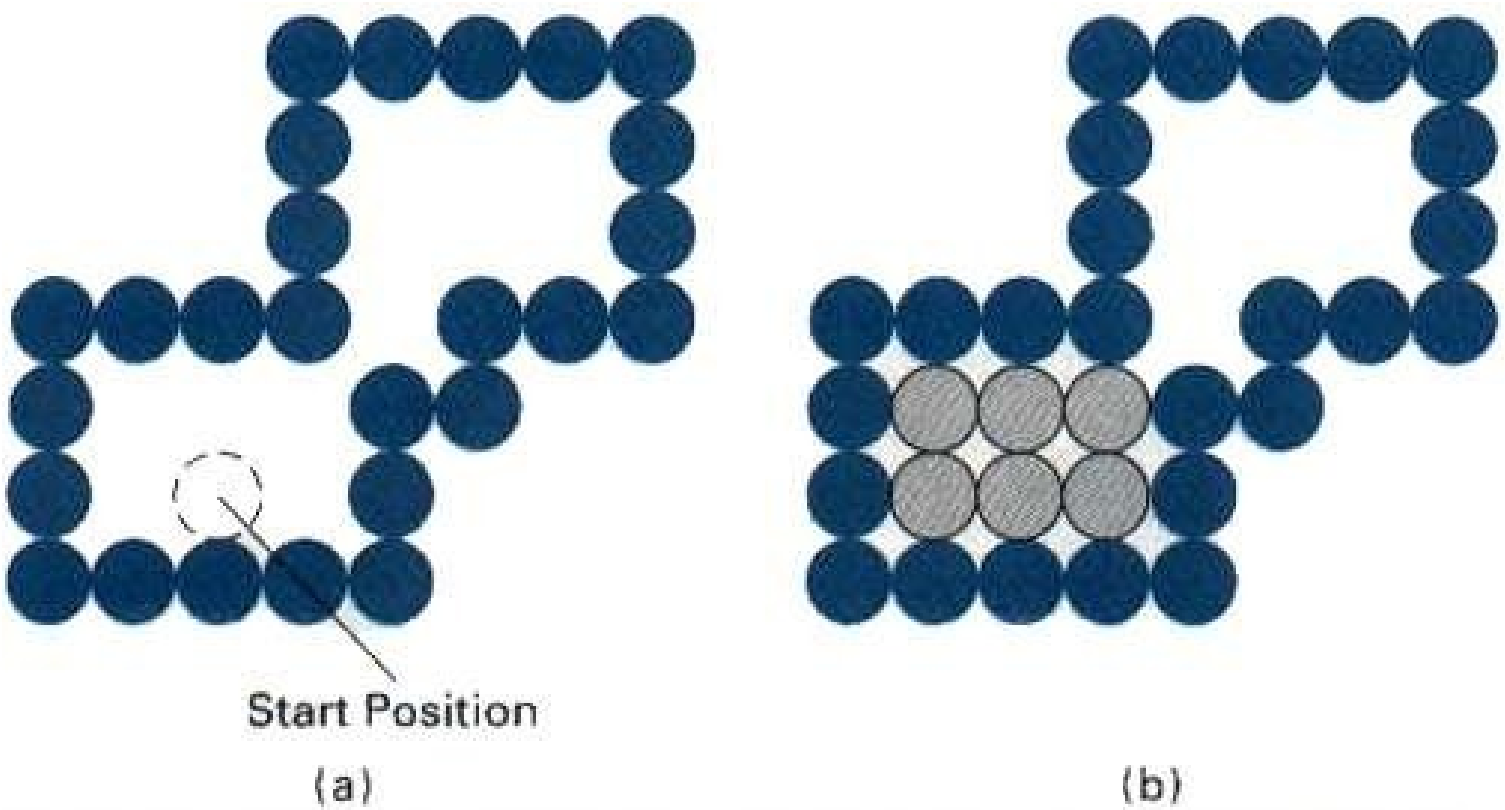
- Start at a point inside a region
- Paint the interior outward to the edge
- The edge must be specified in a single color
- Fill the 4-connected or 8-connected region
- 4-connected fill is faster, but can have problems:



Boundary Fill Algorithm (cont.)


```
void BoundaryFill4(int x, int y,
                              color newcolor, color edgecolor)
{
   int current;
   current = ReadPixel(x, y);
   if(current != edgecolor && current != newcolor)
   {
      BoundaryFill4(x+1, y, newcolor, edgecolor);
      BoundaryFill4(x-1, y, newcolor, edgecolor);
      BoundaryFill4(x, y+1, newcolor, edgecolor);
      BoundaryFill4(x, y-1, newcolor, edgecolor);
   }
}
```

Boundary Fill Algorithm (cont.)



The area defined within color boundary in (a) is only partially filled in (b) using a 4-connected boundary-fill algorithm

Flood Fill Algorithm

- Used when an area defined with multiple color boundaries
- Start at a point inside a region
- Replace a specified interior color (old color) with fill color
- Fill the 4-connected or 8-connected region until all interior points being replaced

Flood Fill Algorithm (cont.)

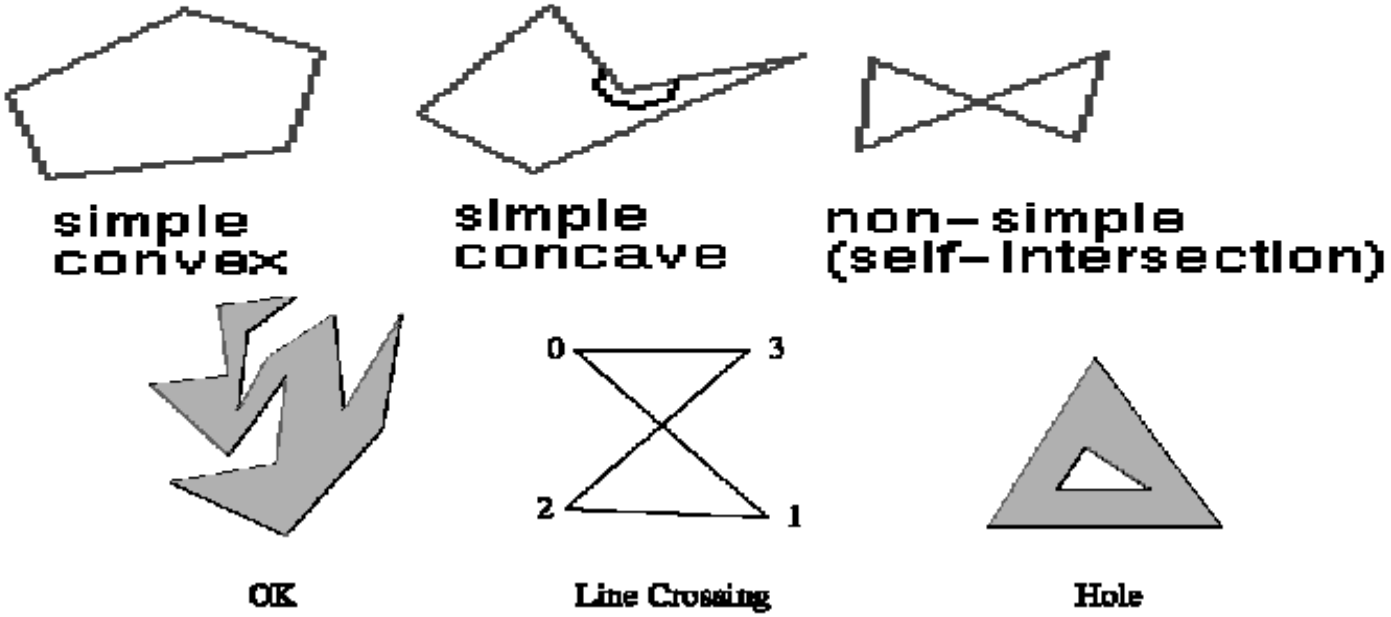

```
void FloodFill4(int x, int y, color newColor, color oldColor)
{
    if (ReadPixel(x, y) != oldColor) return;

    SetPixel(x, y, newColor);   // mark as filled

    FloodFill4(x+1, y, newColor, oldColor);
    FloodFill4(x-1, y, newColor, oldColor);
    FloodFill4(x, y+1, newColor, oldColor);
    FloodFill4(x, y-1, newColor, oldColor);
}
```

Other Versions

Can you convert the algorithms in a stack version?

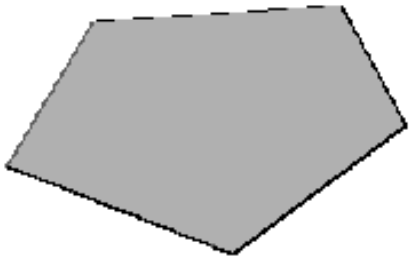
Polygon Types

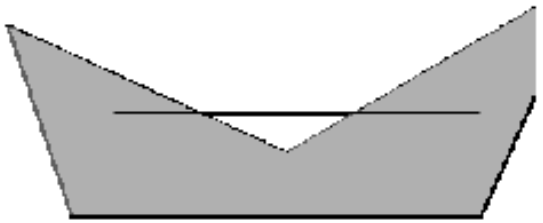
- simple convex, simple concave, non-simple (self-intersecting)
- want no holes, no intersections (line crossings)
- rectangles and triangles always simple convex



Convex, Concave, Degenerate

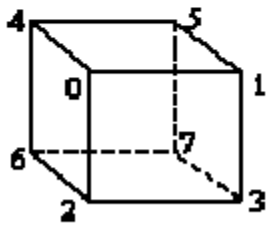
- Convex polygons are preferable to concave
  - Polygon is convex if for any two points inside polygon, the line segment joining these two points is also inside.





Polygon Representation

- Polygon Representation
  - ordered list of vertices
  - avoids redundant storage and computations
  - associate other information with vertices
    - colors, normals, textures



faces		vertex list	
#	vertex list	#	x,y,z
0	0,2,3,1	0	0,1,1
1	1,3,7,5	1	1,1,1
2	5,7,6,4	2	0,0,1
3	4,6,2,0	3	1,0,1
4	4,0,1,5	4	0,1,0
5	2,6,7,3	5	1,1,0
		6	0,0,0
		7	1,0,0

Scanline Fill Algorithm

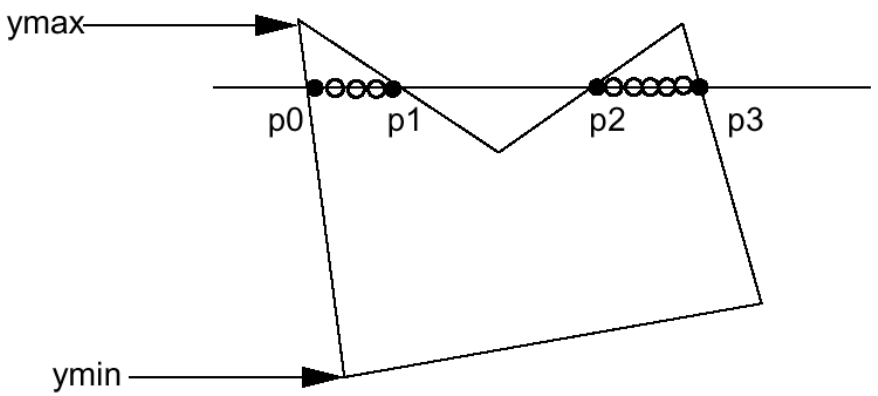
- Intersect scanline with polygon edges
- Fill between pairs of intersections
- Basic algorithm:

  For y = ymin to ymax

  1) intersect scanline y with each edge

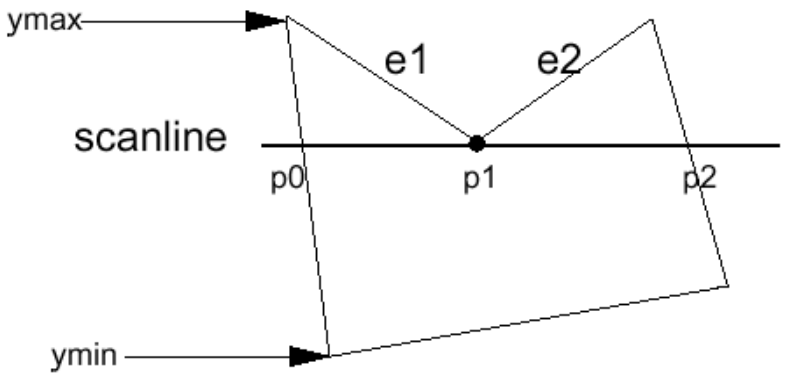
  2) sort intersections by increasing x [p0,p1,p2,p3]

  3) fill pairwise (p0->p1, p2->p3, …)



Special Handling

- Intersection is an edge end point

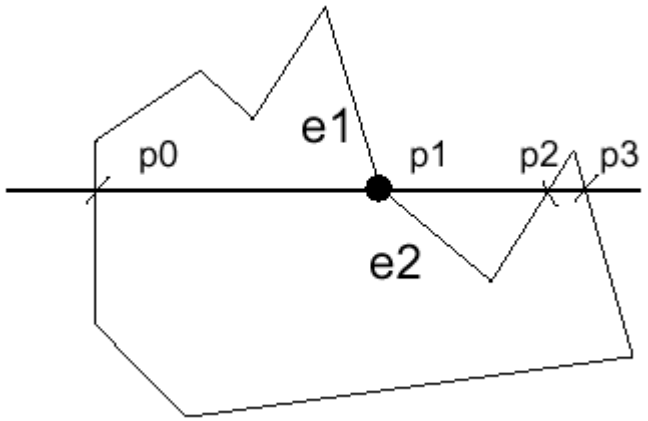


Case 1

Intersection points: (p0, p1, p2) ???
->(p0,p1,p1,p2) so we can still fill pairwise
->In fact, if we compute the intersection of the scanline with edge e1 and e2 separately, we will get the intersection point p1 twice. Keep both of the p1.

Special Handling (cont.)

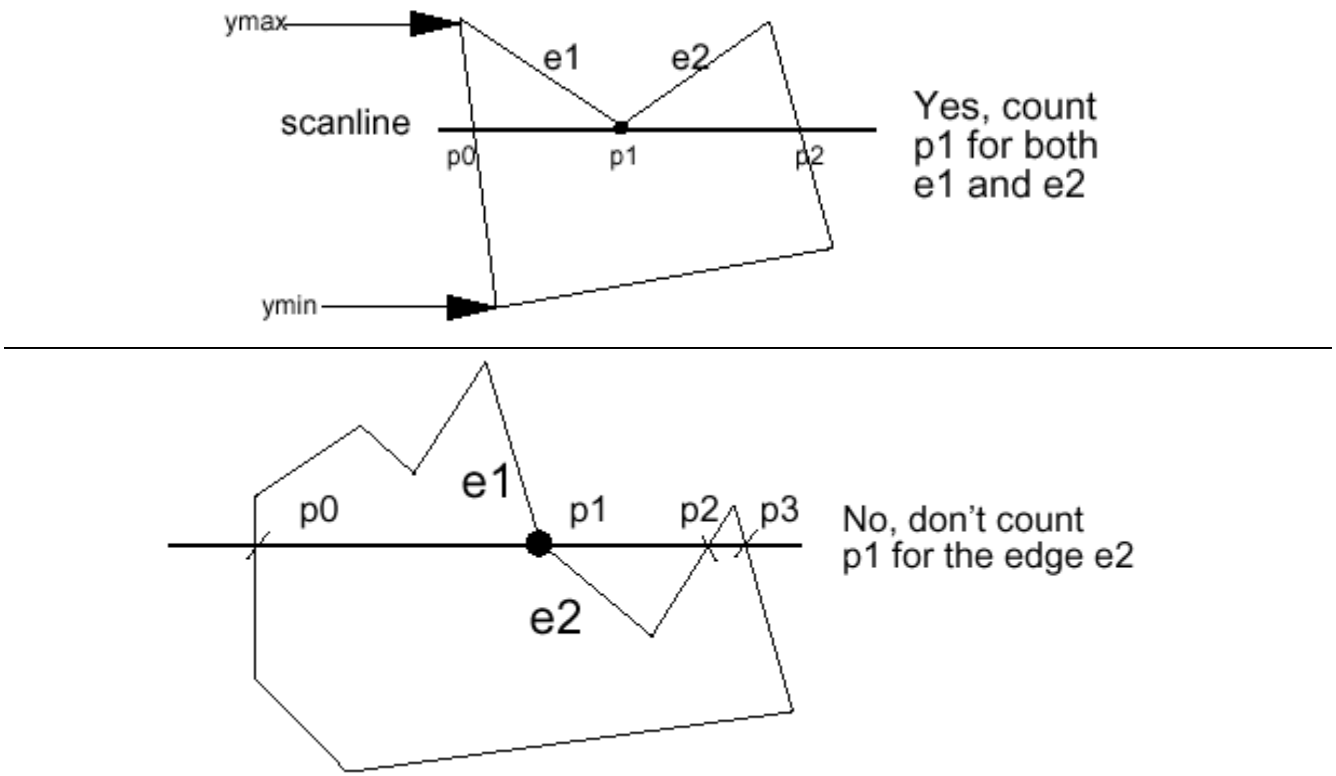
Case 2



However, in this case we don't want to count p1 twice (p0,p1,p1,p2,p3), otherwise we will fill pixels between p1 and p2, which is wrong.

Special Handling (cont.)

Summary: If the intersection is the ymin of the edge's endpoint, count it. Otherwise, don't.



Side Effects of Scan Conversion

The most common side effects when working with raster devices are:

- Aliasing
- Unequal intensity
- Overstrike
- Picket fence problem

Aliasing

- Refers to the plotting of a point in a location other than its true location in order to fit the point into the raster.

- Consider equation y = mx + b

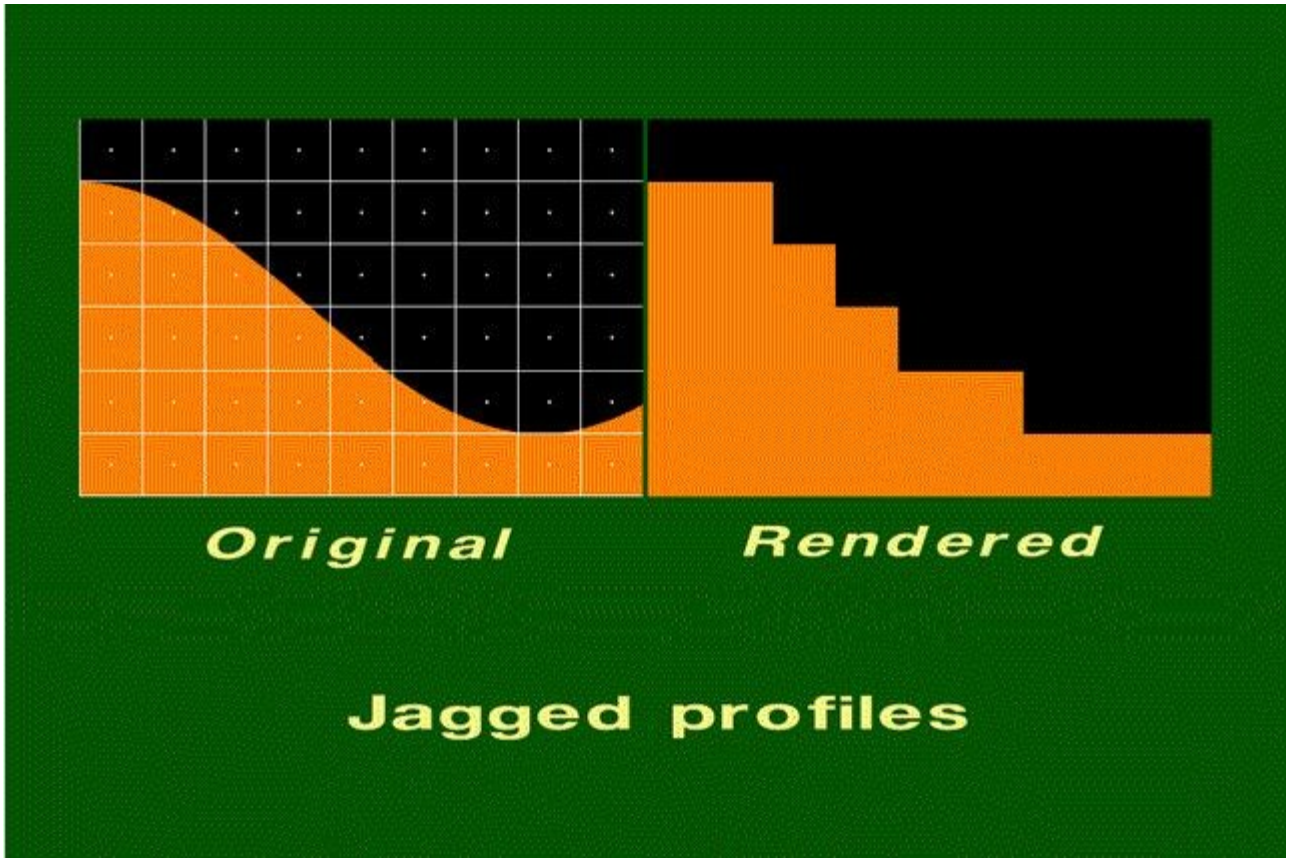
- For m = 0.5, b = 1 and x = 3 : y = 2.5

- So the point (3,2.5) is plotted at alias location (3,3)

Remedy

- Apply anti-aliasing algorithms. Most of these algorithms introduce extra pixels and pixels are intensified proportional to the area of that pixel covered by the object.

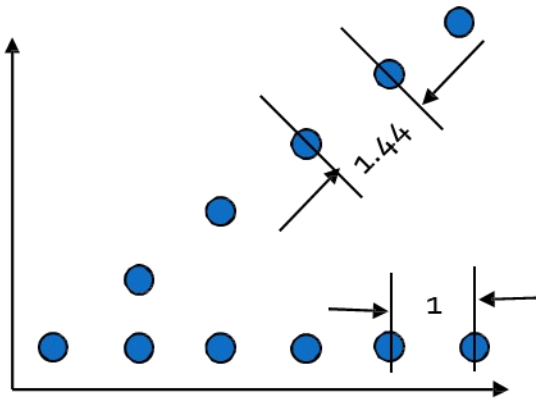
Aliasing



Unequal intensity

Human perception of light is dependent on

- Density and
- Intensity of light source.



Thus, on a raster display with perfect squareness, a diagonal line of pixels will appear dimmer that a horizontal or vertical line.

Remedy

1. If speed of scan conversion is main then ignore
2. By increasing the number of pixels in diagonal lines
3. By increasing the intensity of pixels on diagonal lines.

Overstrike

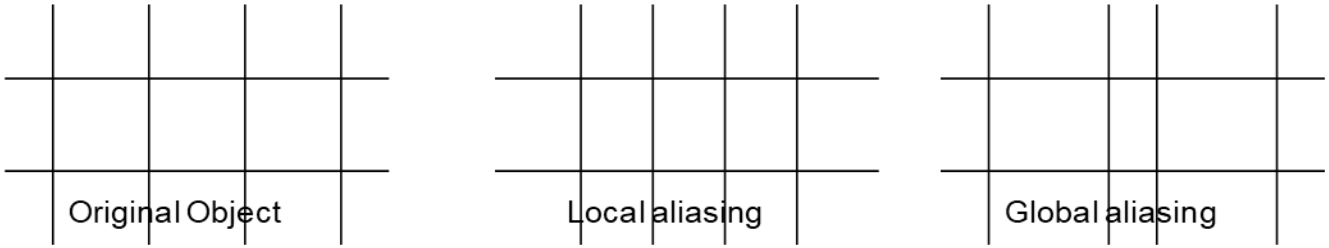
- The same pixel is written more than once.
- This results in intensified pixels in case of photographic media, such as slide or transparency

Remedy

- Check each pixel to see whether it has already been written to prior to writing a new point.

Picket fence problem

Occurs when a user attempts to scan-convert an object that will not fit exactly in the raster.



Remedy

LocalAliasing: the distances between pickets in the picket fence will be kept as close to their true relative distance as possible

Global Aliasing: the overall length of the picket fence will be approximately correct by the spacing between pickets will be distorted.

Reference

- **Computer Graphics**
  - R. Plastock, Zhigang Xiang
  - (Schaum's Outline Series) McGraw Hill

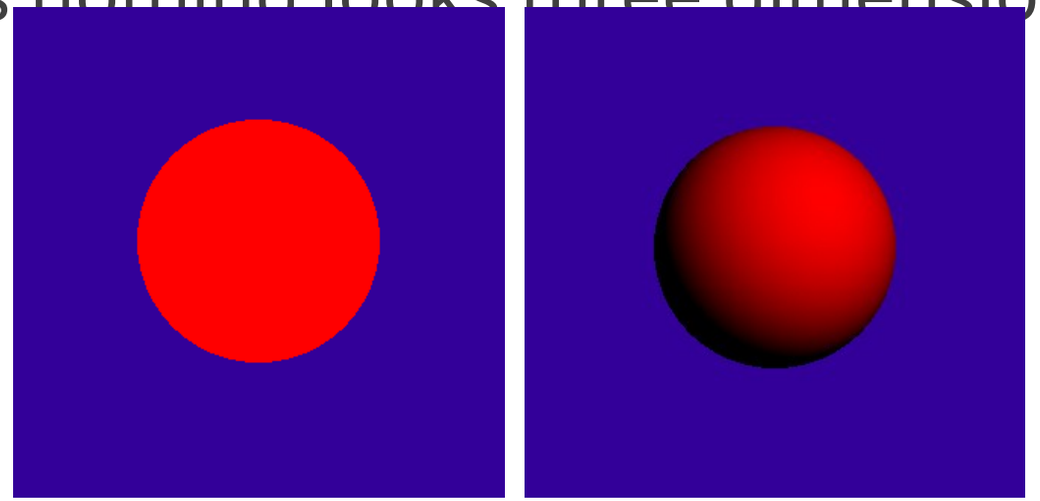
Illumination Model

PROF. DR. SK. MD. MASUDUL AHSAN

CSE, KUET

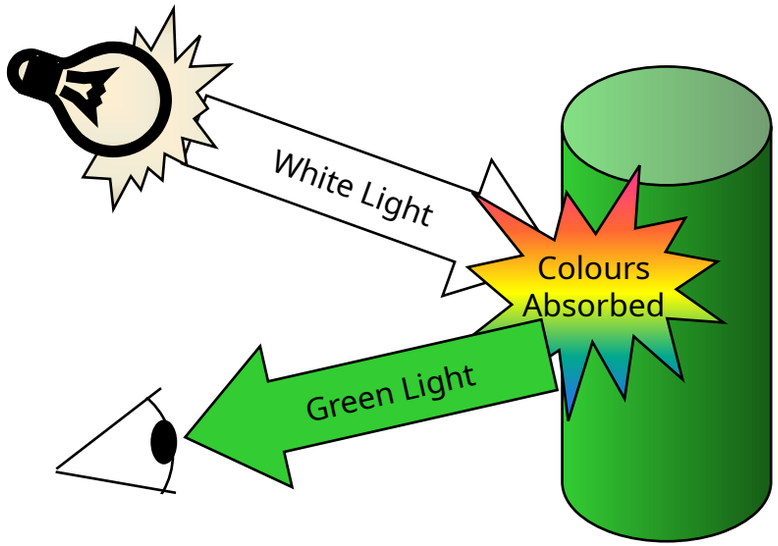
Why Lighting?

- Color is a fundamental attribute of our viewing experience. The perception of color arises from light energy entering our visual system.
- This complex process is relevant to computer graphics because a realistic image is one that seems indistinguishable from the light energy coming from a real scene.
- If we don’t have lighting effects nothing looks three dimensional!

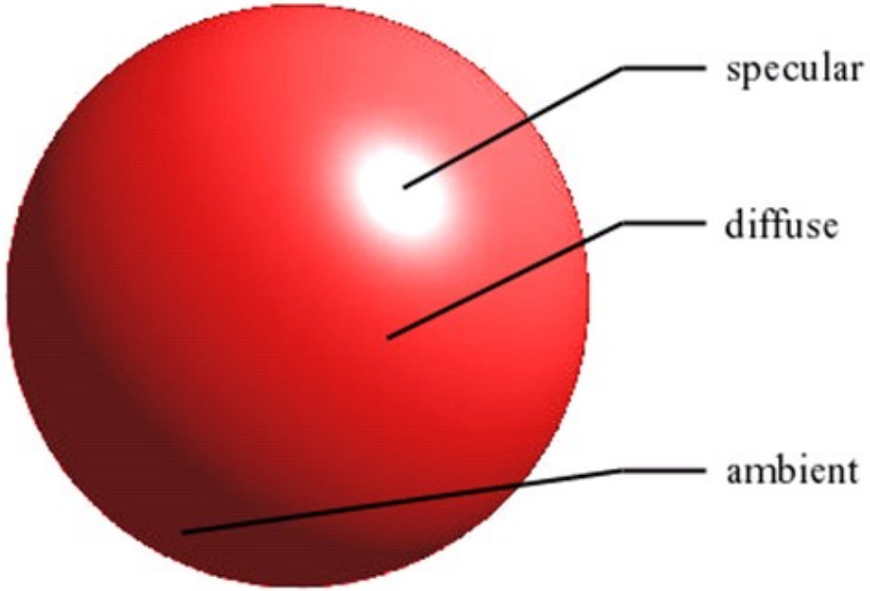


Basics of Reflected Light

- The colours that we perceive are determined by the nature of the light reflected from an object
- For example, if white light is shone onto a green object most wavelengths are absorbed, while green light is reflected from the object



Components of Reflections



Ambient – surface exposed to **indirect light** reflected from nearby objects.

Diffuse – reflection from **incident** light with equal intensity in all directions.

Depends on surface properties.

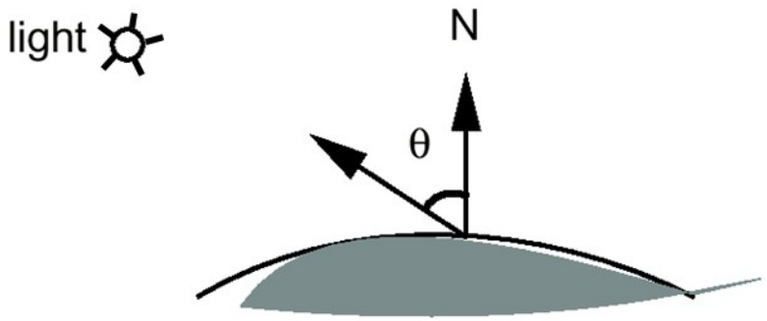
Specular – near **total** of the incident light around reflection angle.

Illumination Model

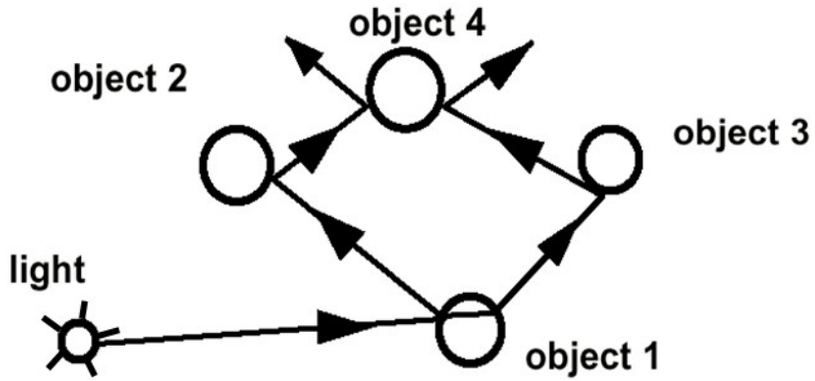
- An illumination model, also called a lighting model is used to calculate the **intensity of light** that we should see at a given point on the surface of an object.
  - Done by simulating some light attributes
- Can be
  - Local illumination
  - Global illumination

Illumination Model

- Local illumination
  - Models only the direct illumination from the source
    - Does not consider light reaching after bouncing off other obj
  - Only consider the light, the observer position, and the object material properties

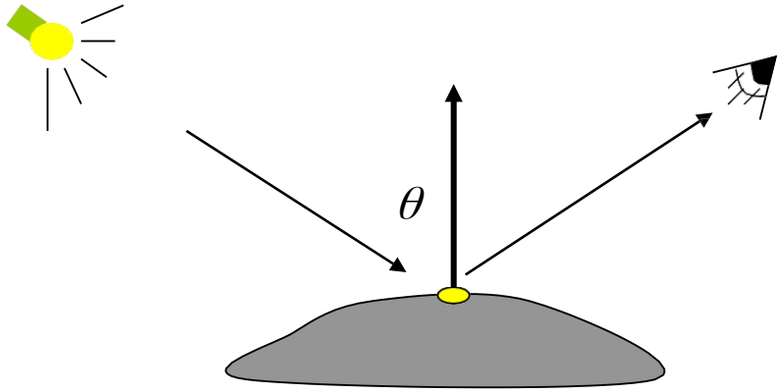


- Global illumination
  - take into account the interaction of light from all the surfaces in the scene



Local Illumination (Lighting) Model

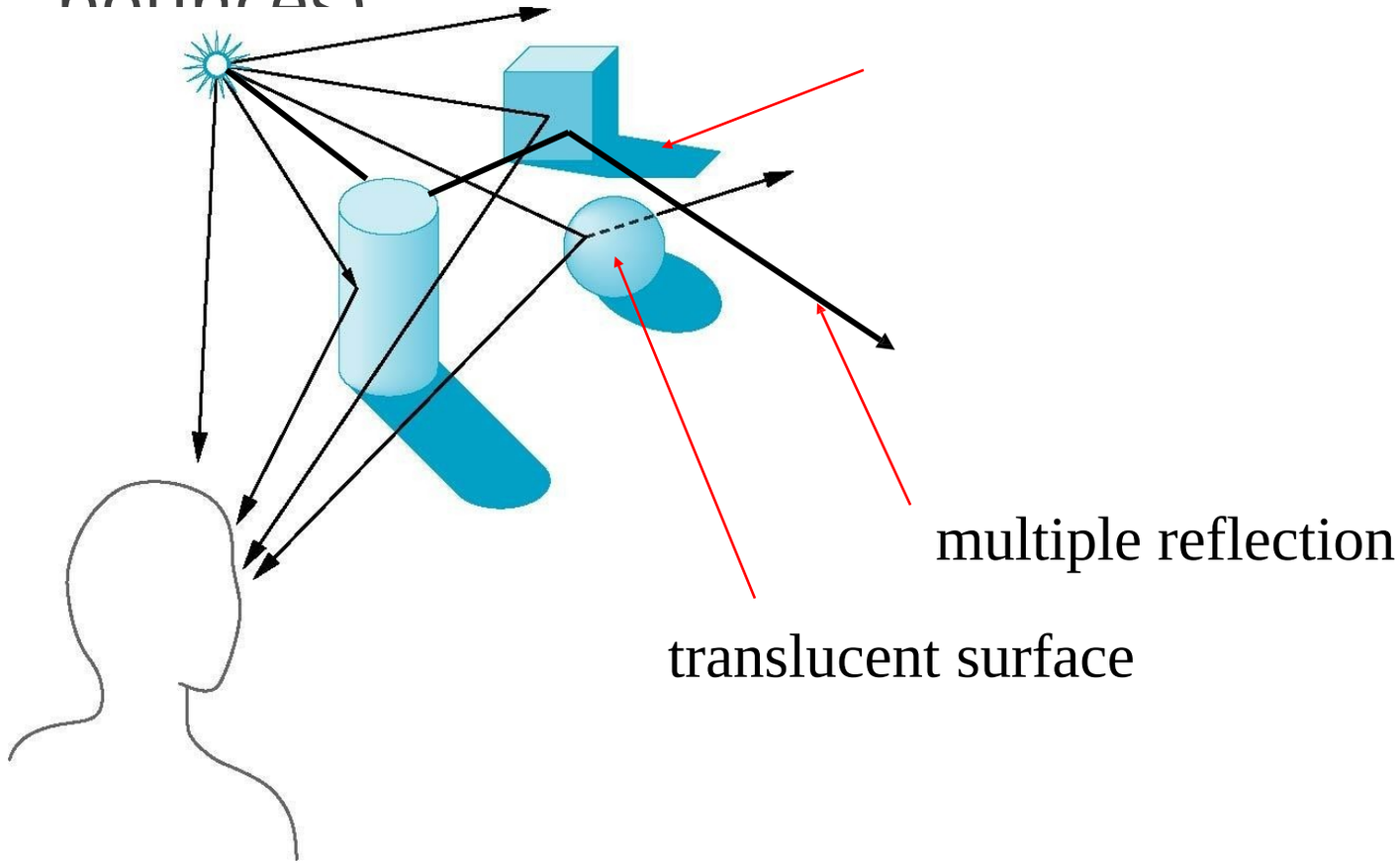
- One bounce!
  - Doesn't track inter-reflections, transmissions (e.g. OpenGl)

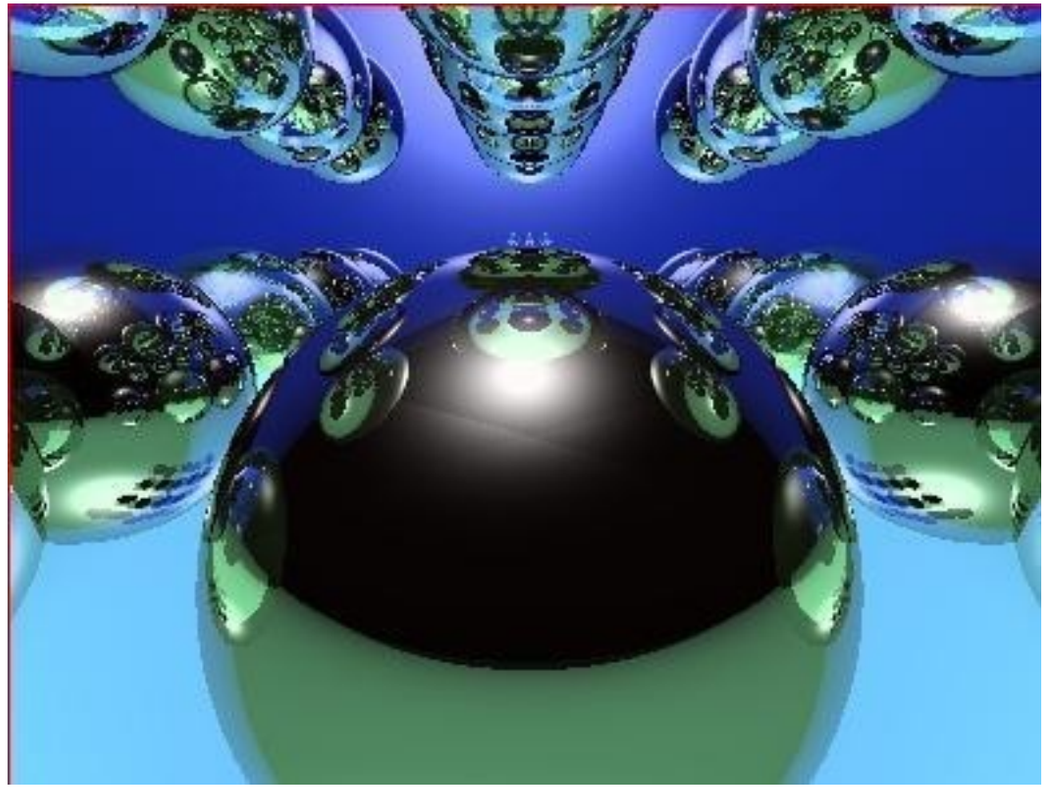


- Global Illumination (GI) is accurate, looks real
  - But raster graphics pipeline (e.g. OpenGL) renders each polygon independently (local rendering), no GI

Global Illumination (Lighting) Model

- interaction of light from all surfaces in the scene (track multiple bounces)





Things to Model

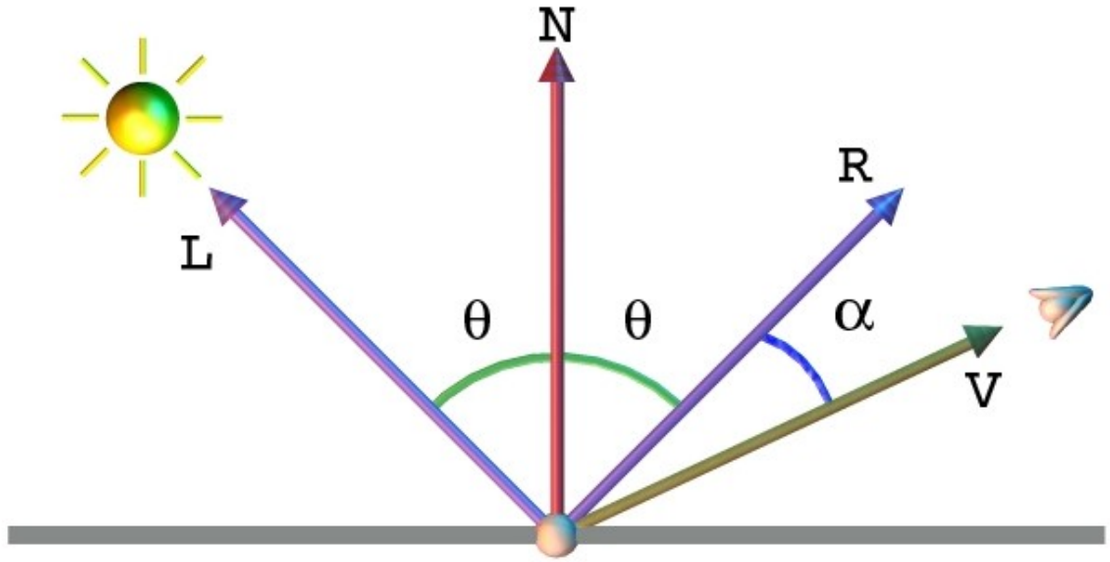
- Light sources
  - What color, intensity, lines through space
- Reflection of light off surfaces
  - How much light reflected in each direction
    - How are color and intensity changed
- Life on a surface

L: direction to light

N: normal vector

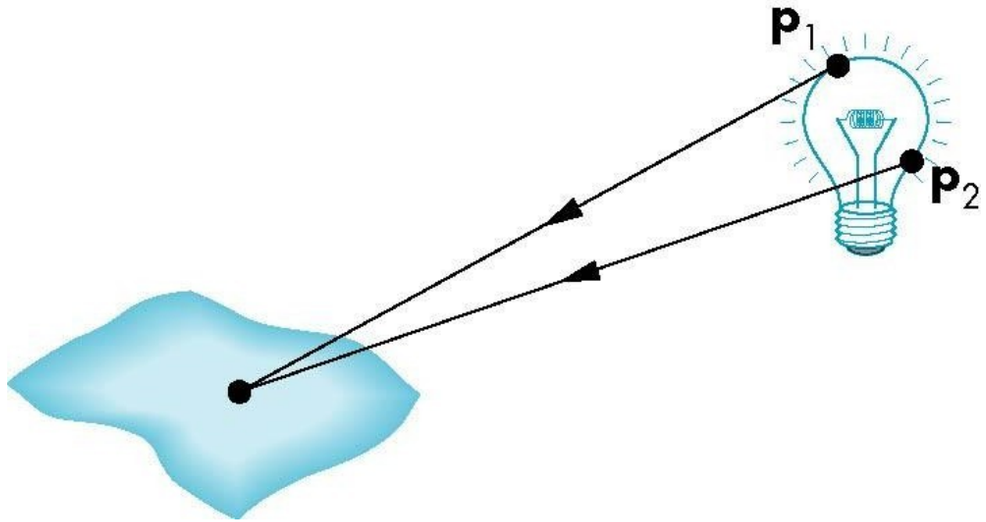
R: reflection of light about normal

V: direction to viewer (i.e. reflection direction of interest)



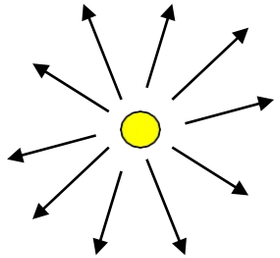
Light Sources

- General light sources are difficult to model (e.g. light bulb)
- Why? We must compute effect of light coming from all points on light source

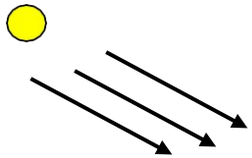


Light Sources Abstractions

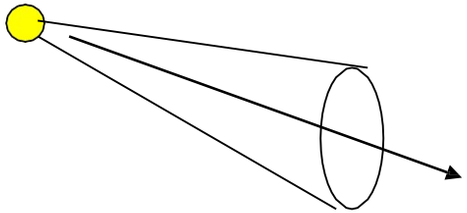
- We generally use simpler light sources
- Abstractions that are easier to model



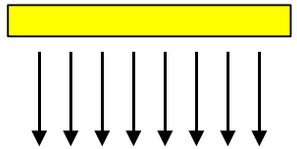
Point light



Directional light



Spot light



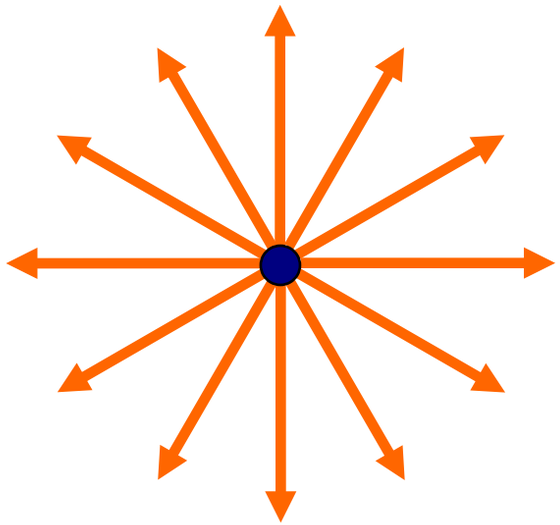
Area light

Light intensity can be **independent** or **dependent** of the distance between object and the light source

Point Light Sources

- Simplest model we can use for a light source
- Defined by:
  - The position (x,y,z) of the light
  - The RGB values for the colour of the light
- Radiates equal intensity in all direction

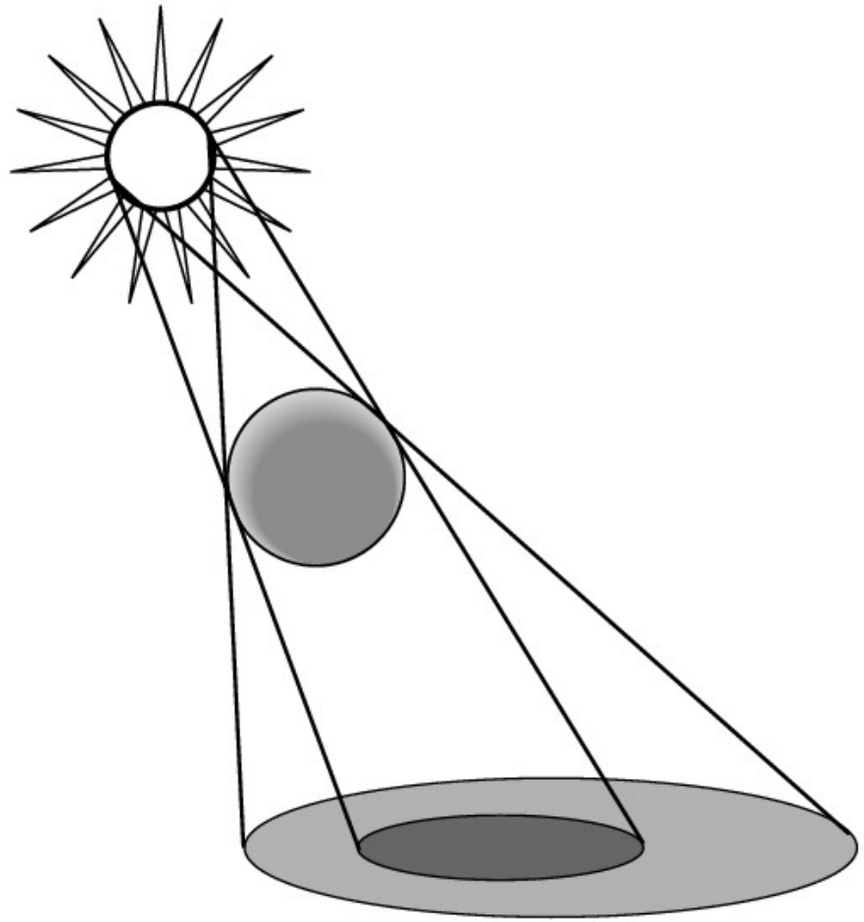
$$L = P_{light} - P_{surface}$$



Limitations of point lights

- Usually result in artificially high-contrast images
- Can generate umbra (full shadow) but not penumbra (partial shadow)
- *Area lights* generate softer shadows, but are usually used only in non-interactive graphics

  (e.g. raytracing or radiosity)



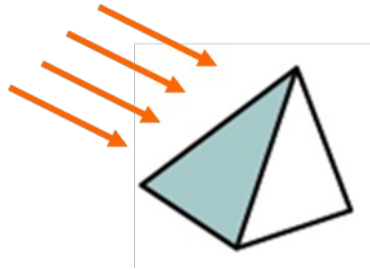
Directional Light Sources

- A large light source, like the sun, can be modelled as a point light source
- However, it will have very little directional effect
  - i.e. All light rays are are considered parallel

Specified by:

- direction (x,y,z)
- intensity (r,g,b)

L = -direction



Intensity Attenuation

- As light moves from a light source its intensity diminished
- At any distance d away from the light source the intensity diminishes by a factor of $1/d^2$
- This means that
  - a surface close to the light source (small ***d) receives a higher incident*** intensity from the source than a distant surface (large ***d).***

Intensity Attenuation

- However, using the factor $1/d^2$ does not produce very good results so we use something different
- We use instead in inverse quadratic function of the form:

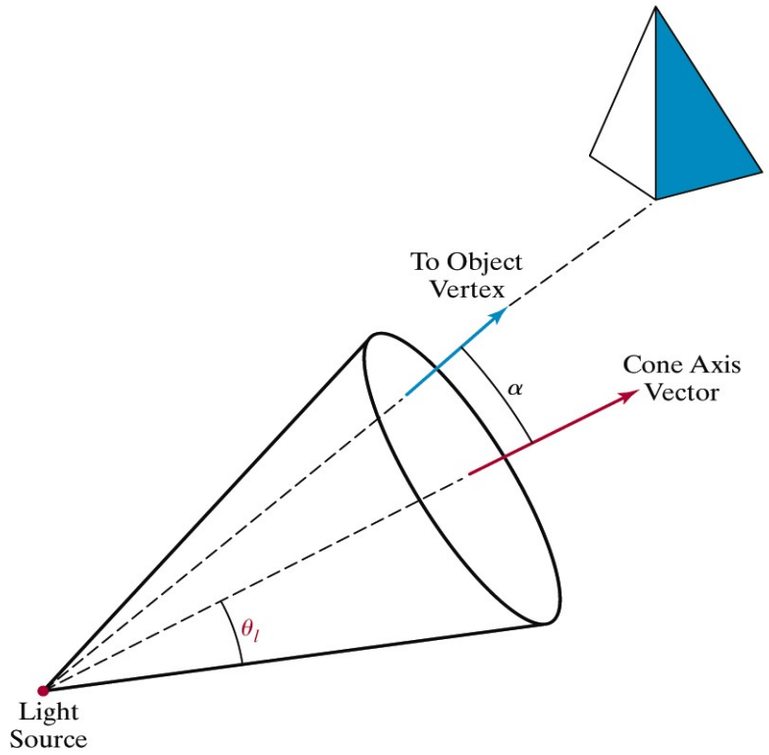
$$f_{\text{radial attenuation}}(d) = \begin{cases} 1.0, & \text{source at infinity} \\ \dfrac{1}{a_0 + a_1 d + a_2 d^2}, & \text{local source} \end{cases}.$$

- A user can then fiddle with the coefficients a_o, a_1, and a_2 , to obtain a variety of lighting effects
- The value of the constant term a_o can be adjusted to prevent **f(d) from becoming too large when d is very small.**

Spotlights

- To turn a point light source into a spotlight we simply add a vector direction and an angular limit θ_l
- If V_{light} and V_{obj} are the unit vector
- Then

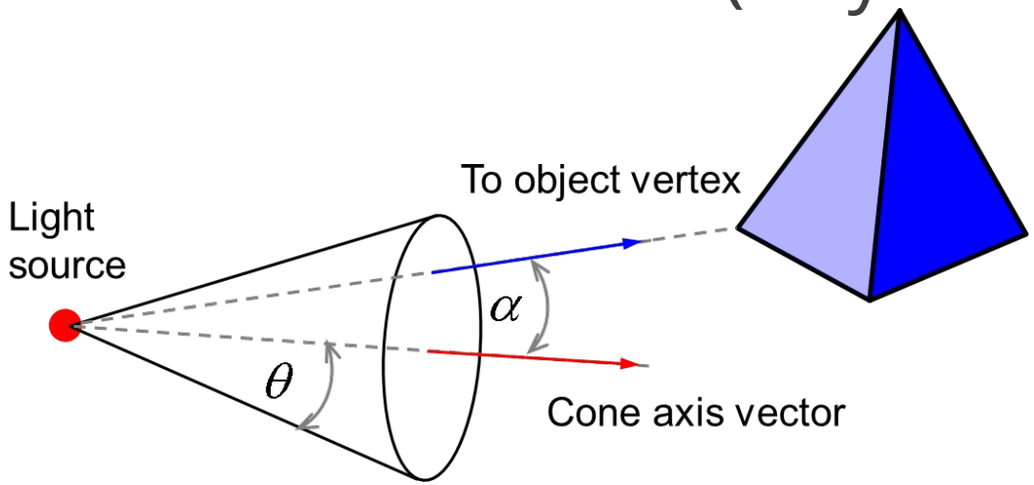
$$V_{obj} \cdot V_{light} = \cos\alpha$$



- If this angle is inside the light's angular limit then the object is within the spotlight

Spotlights - Angular Intensity Attenuation

- Points outside the cone are not illuminated (stay in dark).



- Angular attenuation is:

$$f_{\substack{\text{angular}\\\text{attenuation}}}(\alpha) = \begin{cases} 1.0, & \text{source not a spotlight} \\ 0.0, & \mathbf{V}_{\text{obj}} \cdot \mathbf{V}_{\text{light}} = \cos\alpha < \cos\theta \\ (\mathbf{V}_{\text{obj}} \cdot \mathbf{V}_{\text{light}})^{a}, & \text{otherwise} \end{cases}$$

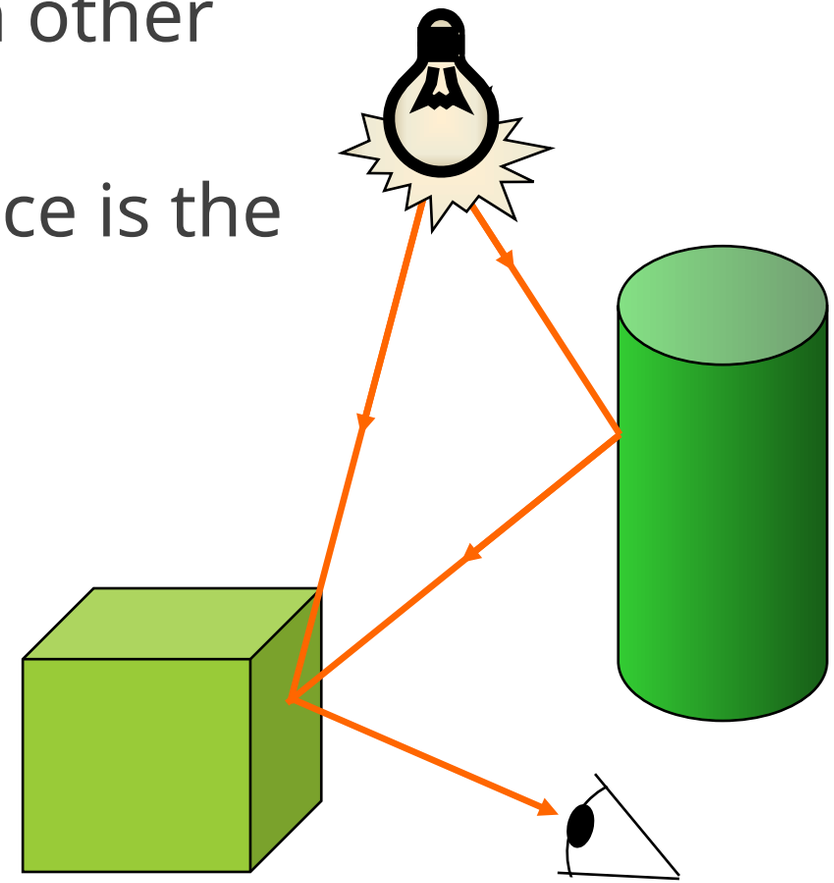
a is a constant

Phong Model

- **Phong model** is an empirical model developed in 1973 by Phong Bui Tuong
- Simple and fast method for calculating surface intensity at a given point
- Gives reasonably good results and is used in most graphics systems
- The important components are:
  - Ambient light
  - Diffuse reflection
  - Specular reflection
  - Emissive light
- For the most part we will consider only monochromatic light sources

Ambient Light

- A surface that is not exposed to direct light may still be lit up by reflections from other nearby objects – **ambient light**
- The total reflected light from a surface is the sum of the contributions from light sources and reflected light

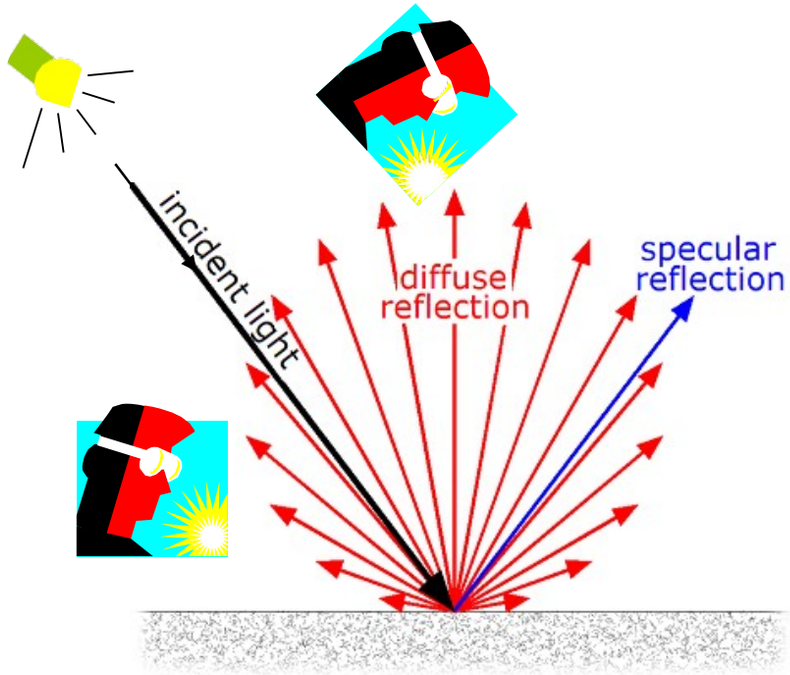


Ambient Light

- To incorporate background light Phong model simply set a general brightness level for a scene
- This approximates the global diffuse reflections (inter-reflections) from various surfaces within the scene
- This light intensity is simply denoted by I_a,
- It is
  - Independent of the light positions or surface directions
  - Independent of location of viewer
  - Has no direction

Diffuse Reflection

- Surfaces that are rough or grainy tend to reflect light in all directions
- This scattered light is called **diffuse reflection**
- This type of reflection is called Lambertian Reflection (thus, Lambertian surfaces)
- The brightness of the surface is independent of the observer position (since the light is reflected in all directions equally)



Eye position does not matter

Diffuse Reflection (cont…)

- Surface's material property:
  - A parameter k_d is set for each surface
  - k_d determines the fraction of light that is to be scattered as diffuse reflections from that surface
- This parameter is known as the **diffuse-reflection coefficient** or the **diffuse reflectivity**
- k_d is assigned a value between 0.0 and 1.0
  - 0.0: dull surface that absorbs almost all light
  - 1.0: shiny surface that reflects almost all light

Diffuse Reflection – Ambient Light

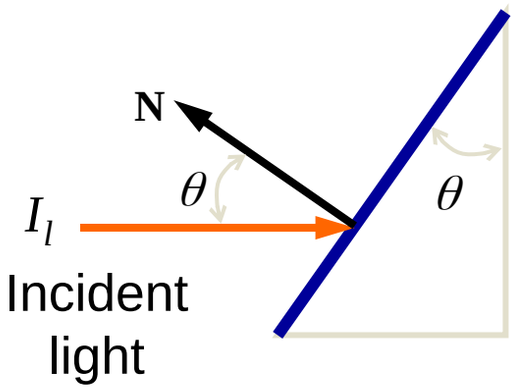
- For background lighting effects we can assume that every surface is fully illuminated by the scene's ambient light I_a
- Therefore the ambient contribution to the diffuse reflection is given as:

$$I_{ambdiff} = k_d I_a$$

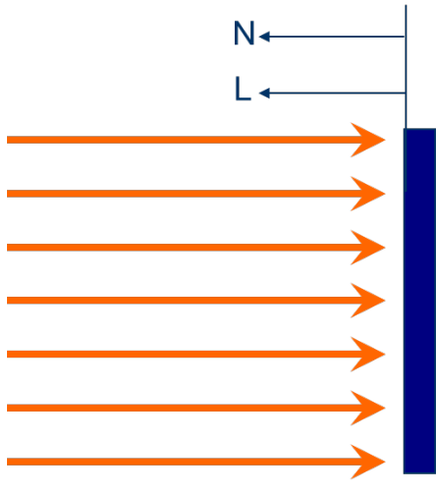
- Ambient light alone is not enough to render a scene or very uninteresting so we need some other lights in a scene as well

Diffuse Reflection (cont…)

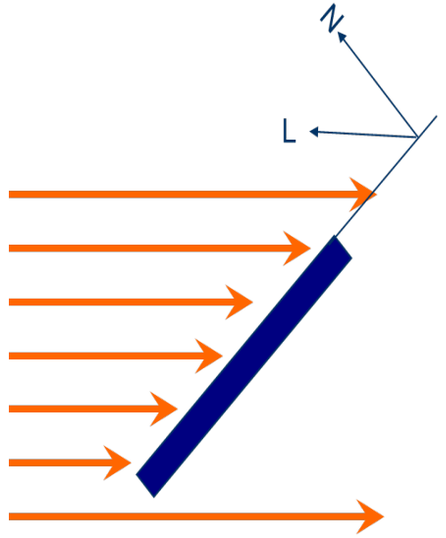
- When a surface is illuminated by a light source, the amount of incident light depends on the orientation of the surface relative to the light source direction



θ = **angle of incidence**



Receive more light

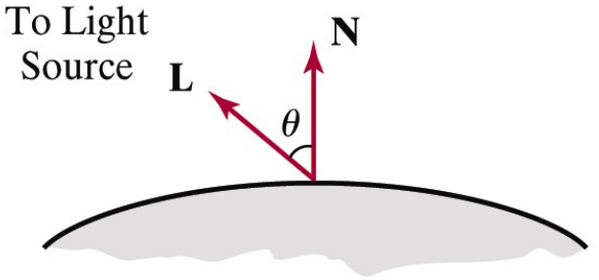


Receive less light

Diffuse Reflection (cont...)

- So the amount of incident light on a surface is given as:

$$I_{l,incident} = I_l \cos\theta$$



- So we can model the diffuse reflections as:

$$I_{l,diff} = k_d I_{l,incident}$$
$$= k_d I_l \cos\theta$$

- if N and L are unit vector,

$$N \cdot L = \cos\theta$$

$$I_{l,diff} = \begin{cases} k_d I_l (N \cdot L) & \text{if } N \cdot L > 0 \\ 0 & \text{if } N \cdot L \le 0 \end{cases}$$

Combining Ambient And Diffuse Reflections

- To combine the diffuse reflections arising from ambient and incident light most graphics packages use two separate diffuse-reflection coefficients:
  - k_a for ambient light
  - k_d for incident light
- The total diffuse reflection equation for a single point source can then be given as:

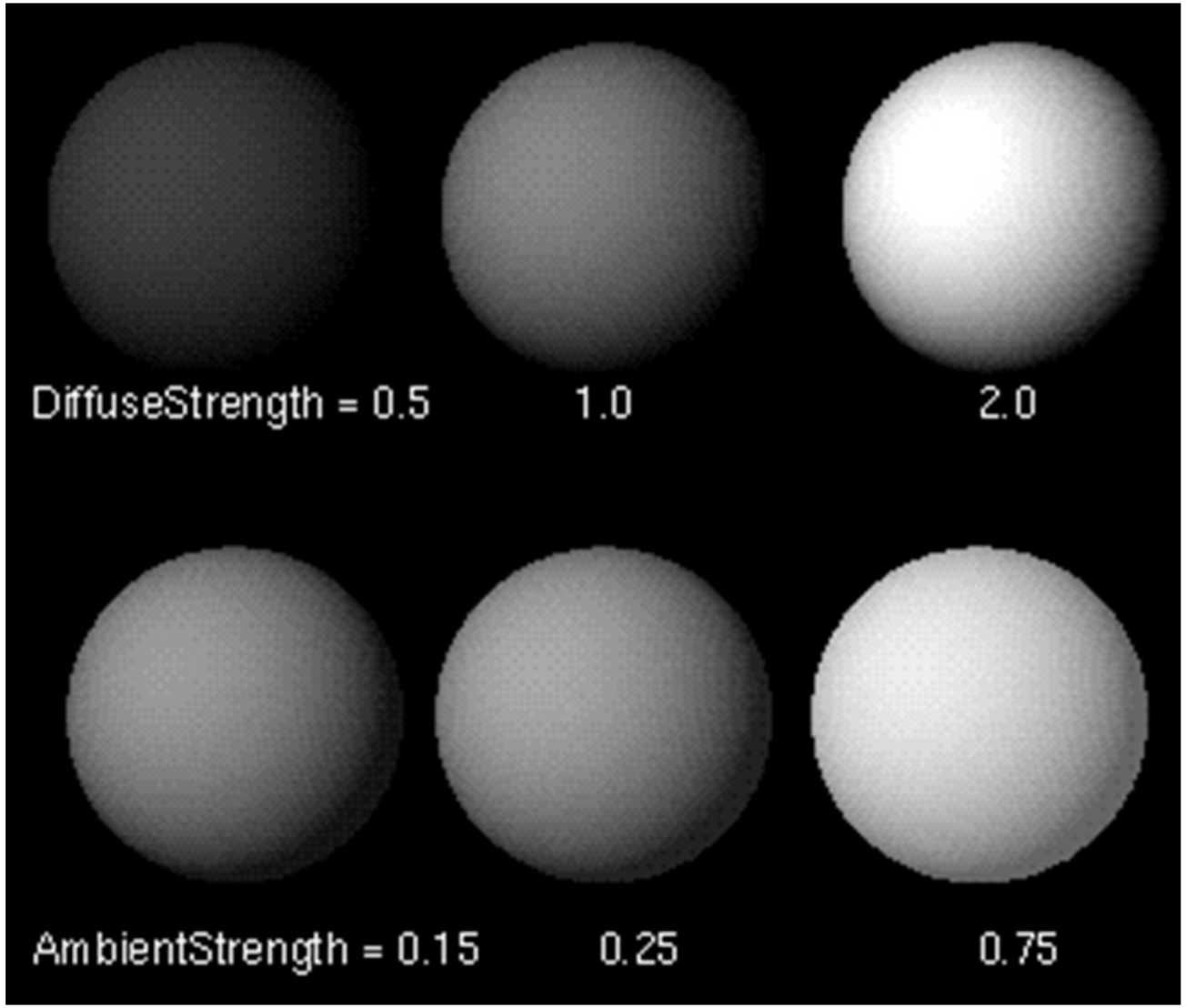
$$I_{diff} = \begin{cases} k_a I_a + k_d I_l (N \cdot L) & \text{if } N \cdot L > 0 \\ k_a I_a & \text{if } N \cdot L \leq 0 \end{cases}$$

Ambient Light Example

Ambient: background light, scattered by environment

Diffuse Lighting Example

Examples

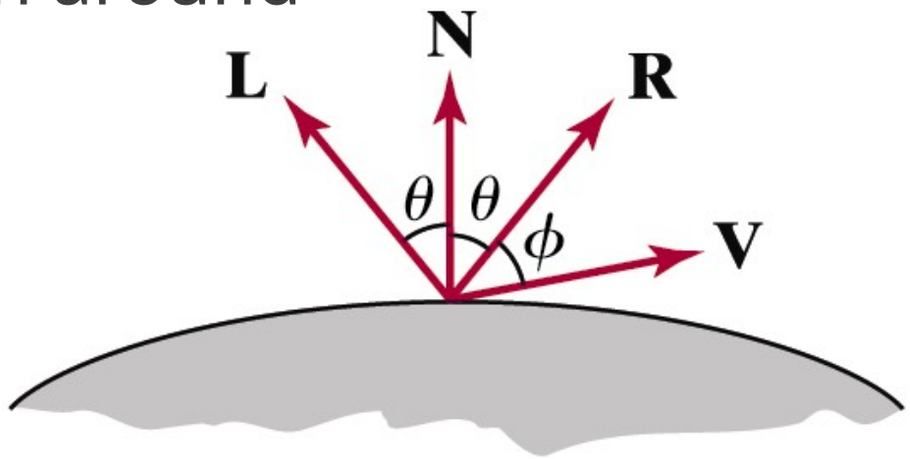


Ambient = 0

Ambient = 1

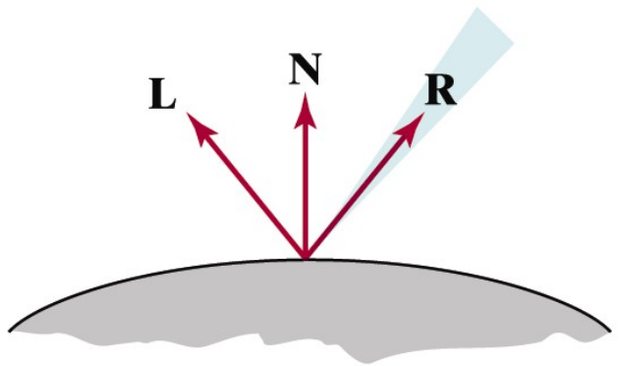
Specular Reflection

- Additionally to diffuse reflection we see a **highlight, or bright spot,** on objects (such as polished metal, apple …) **at certain viewing directions.**
- This phenomenon, called ***specular reflection,***
- It is the result of total, or near total reflection of the incident light in a concentrated region around the specular reflection angle.

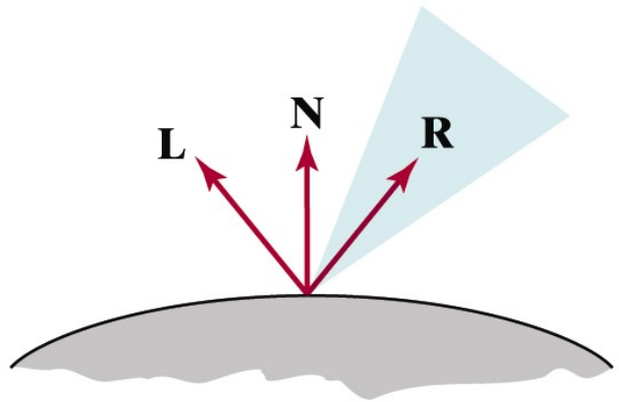


Specular Reflection (cont...)

- A perfect mirror reflects light only in the specular-reflection direction
- Other objects exhibit specular reflections over a finite range of viewing positions around vector R



Shiny Surface
(Large n_s)



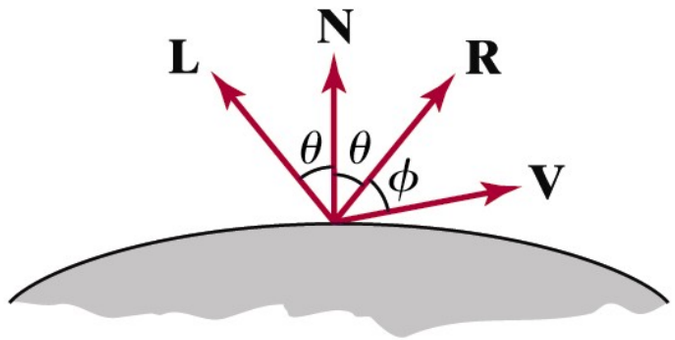
Dull Surface
(Small n_s)

Specular Reflection Model

- The Phong model sets the intensity of specular reflection as proportional to the angle between the viewing vector and the specular reflection vector

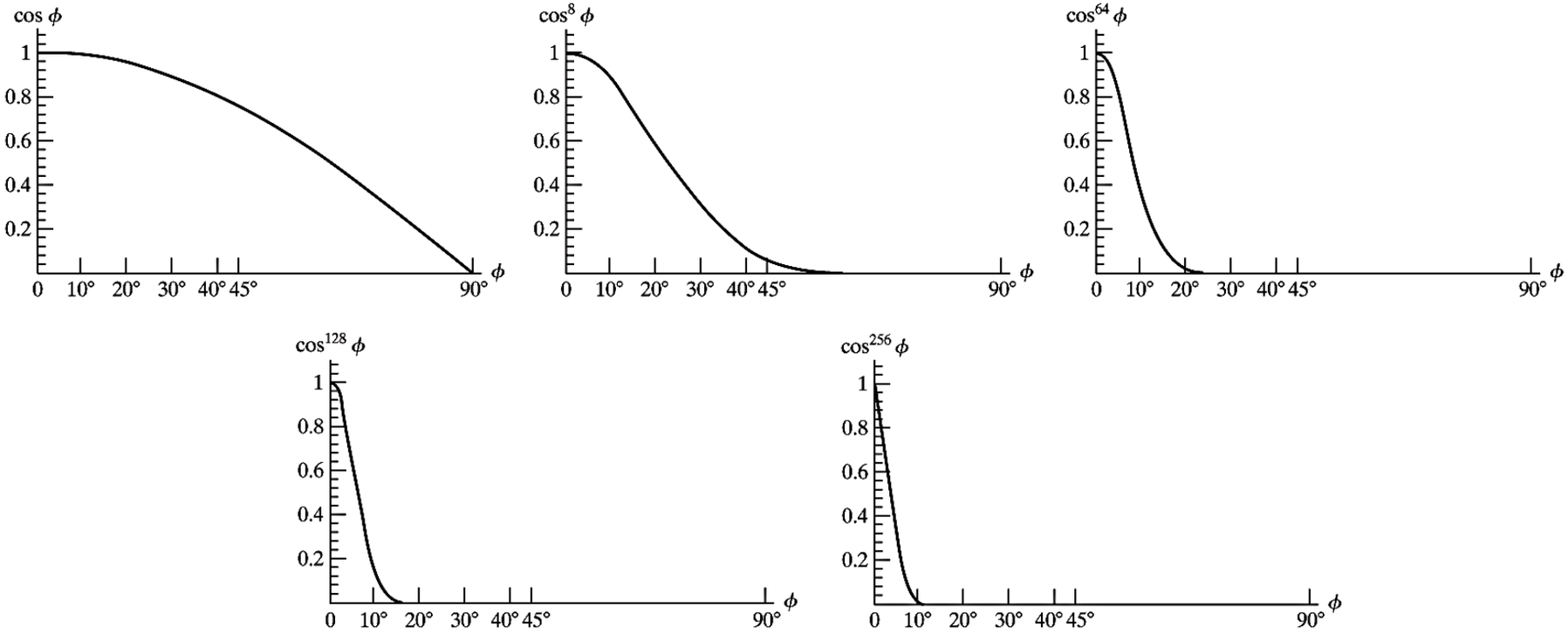
  - So, the specular reflection intensity is proportional to
    - The angle Φ can be varied between 0° and 90° so that cosΦ varies from 1.0 to 0.0

  - The **specular-reflection exponent**, n_s is determined by the type of surface we want to display
    - Shiny surfaces have a very large value (>100)
    - Rough surfaces would have a value near 1



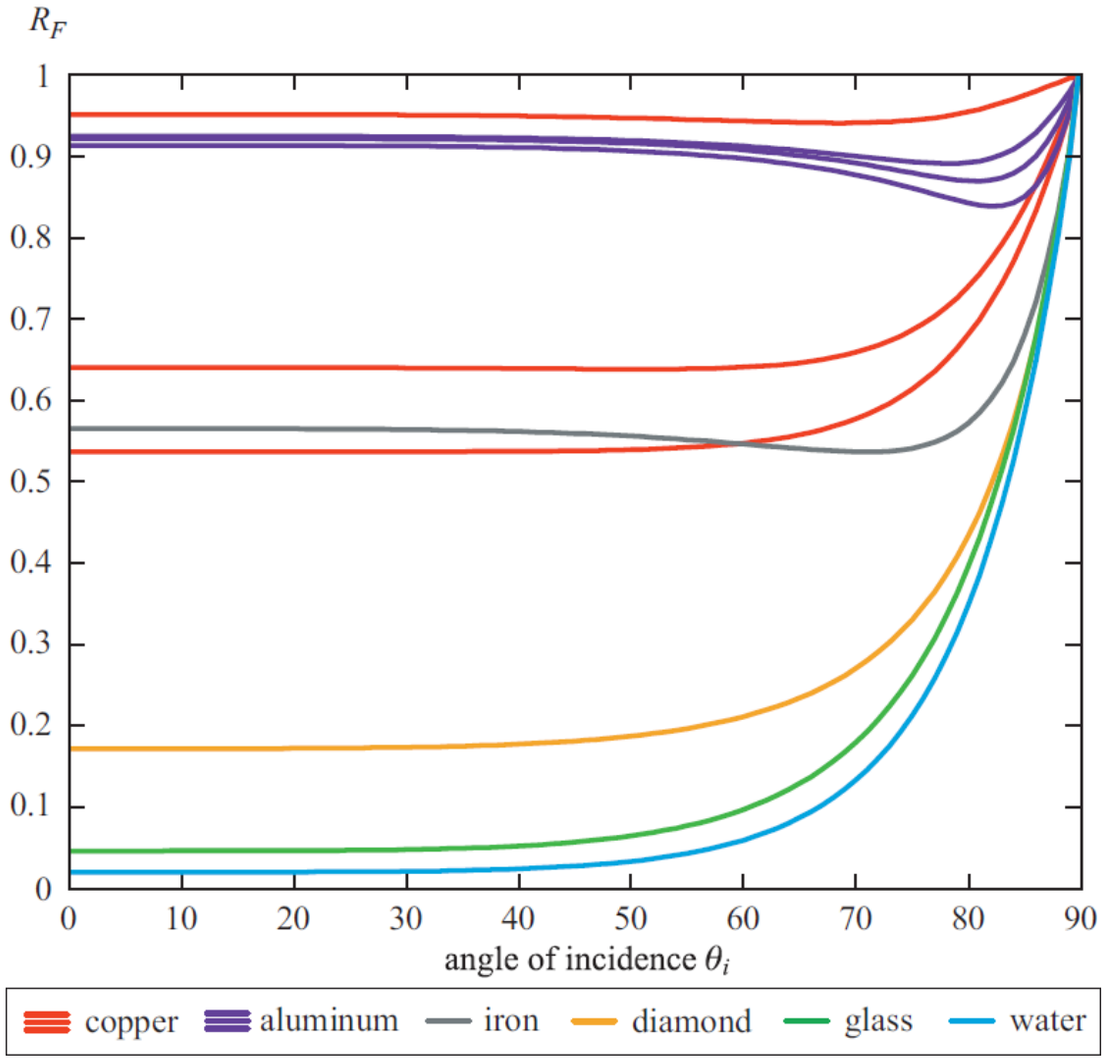
Specular Reflection Model

- The effect of n_s on the angular range in which we can expect to see specular reflections



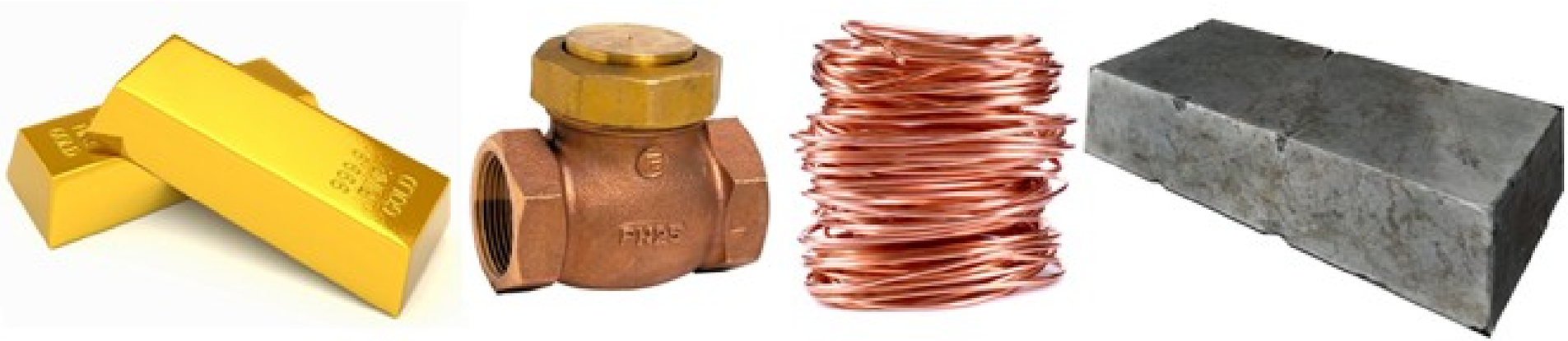
Specular Reflection Model

- For some materials the amount of specular reflection depends heavily on the angle of the incident light
- *Fresnel's Laws of Reflection* describe in great detail how specular reflections behave



Specular Reflection Model

- *Fresnel's Laws of Reflection* describe in great detail how specular reflections behave
  - if the material is a dielectric, in other words, if it is not a conductor/metal. A specular reflection is only a reflection of a light source off of the surface of an object.



  - Some metals have a color (bronze, gold, copper) and are purely reflective. Metals change the color of the specular reflections with their own colors

Specular Reflection Model

- Phong's original formula for the specular term:
  - $W(i)[\cos \phi]^n$
    - ϕ is the angle between the view and specular reflection directions.
    - "$W(i)$ is a function which gives the ratio of the specular reflected light and the incident light as a function of the the incident angle i."
      - Ranges from 10 to 80 percent.
- However, we don't need to worry about this and instead approximate the specular effects with a constant specular reflection coefficient k_s

Specular Reflection Model

- So the specular reflection intensity is given as:

$$I_{l,spec} = k_s I_l \cos^{n_s} \phi$$

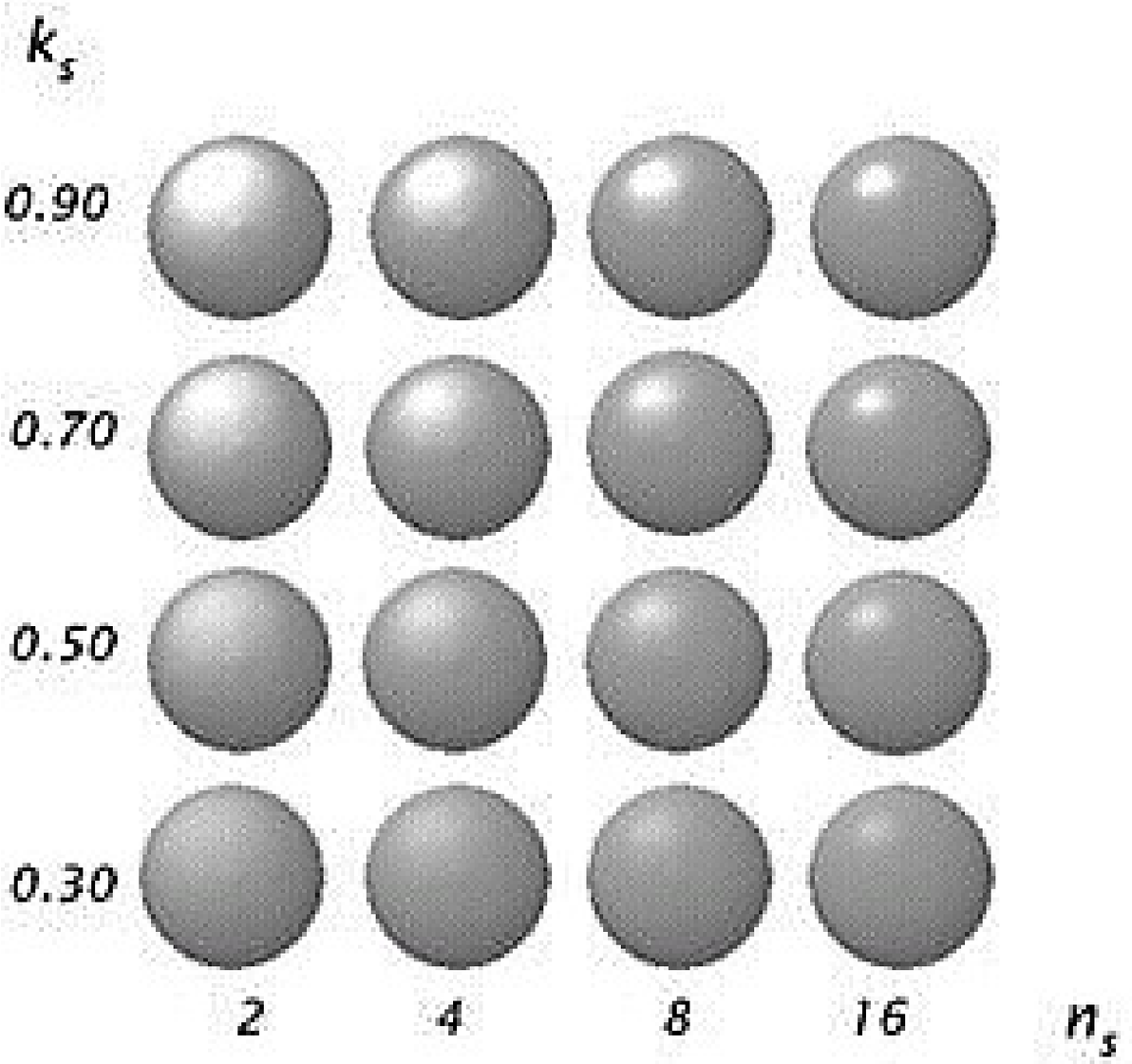
$$V \cdot R = \cos\phi$$

- Remembering that

  We can say:

$$I_{l,spec} = \begin{cases} k_s I_l (V \cdot R)^{n_s} & \text{if } V \cdot R > 0 \text{ and } N \cdot L > 0 \\ 0.0 & \text{if } V \cdot R < 0 \text{ or } N \cdot L \leq 0 \end{cases}$$

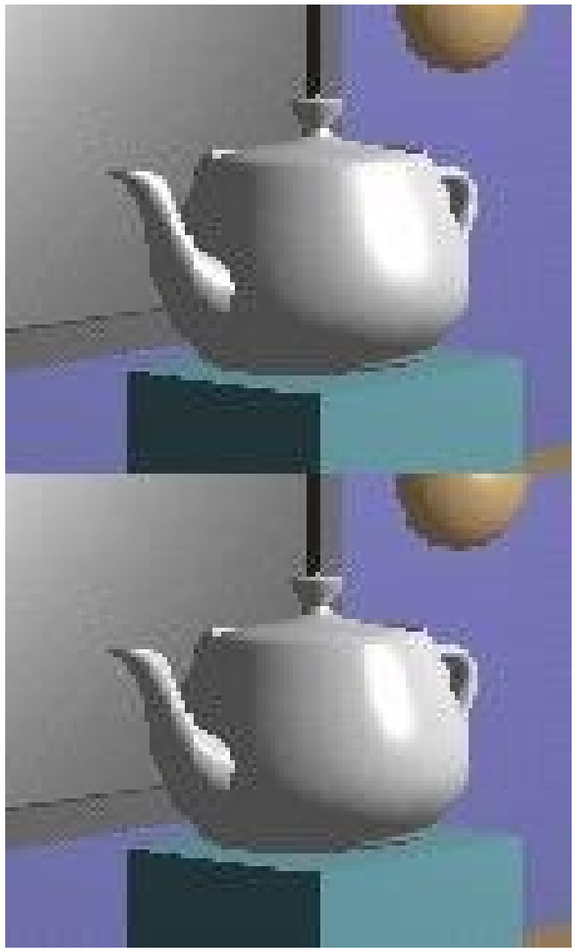
Example



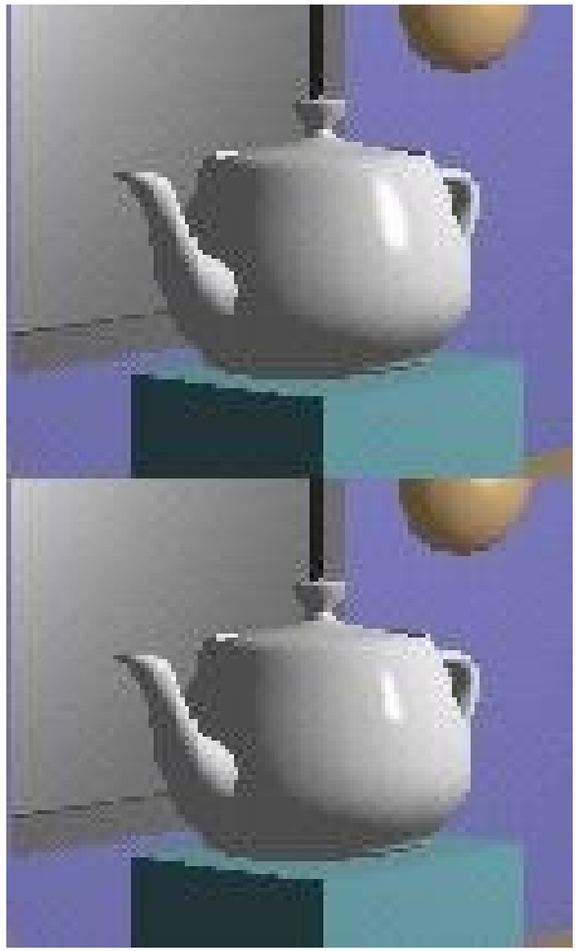
Specular reflection: Effect of 'n_s'

$$\mathbf{I}_s = k_s \mathbf{I} \cos^{ns} \phi$$

n_s = 10



n_s = 90

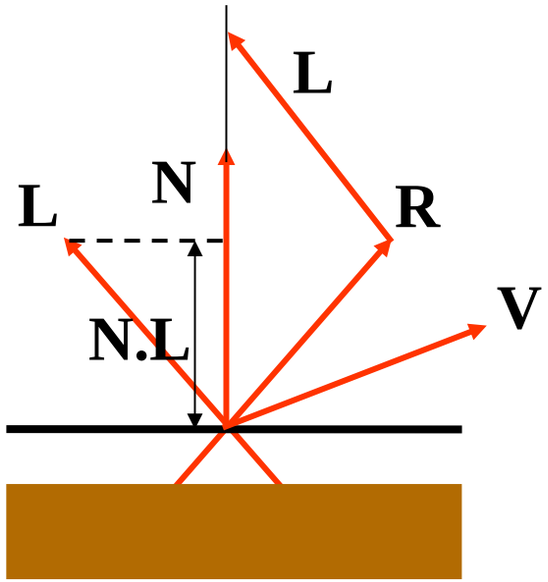


n_s = 30

n_s = 270

Calculating Reflection Vector

Phong model: calculating the vectors



$$\mathbf{R} + \mathbf{L} = (2\mathbf{N} \cdot \mathbf{L})\mathbf{N}$$

hence

$$\mathbf{R} = (2\mathbf{N} \cdot \mathbf{L})\mathbf{N} - \mathbf{L}$$

$$\mathbf{V} = \frac{\mathbf{P}_{view} - \mathbf{P}_{surf}}{|\mathbf{P}_{view} - \mathbf{P}_{surf}|}$$

Half way Vector - Blinn-Phong Model

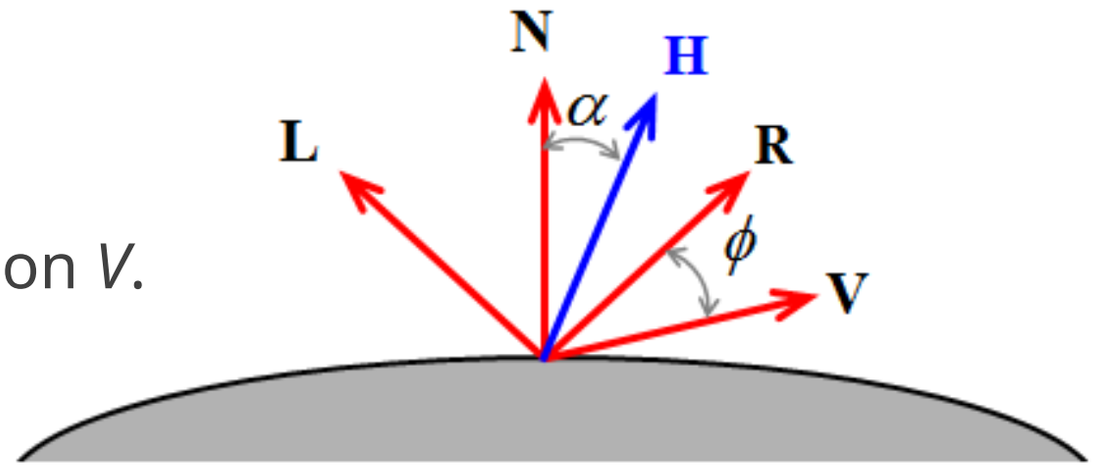
- An alternative way of computing Phong specular lighting is:

$$I_{l,spec} = k_s I_l (N \cdot H)^{n_s}$$

- H (halfway vector): halfway between V and L

$$\mathbf{H} = \frac{\mathbf{L} + \mathbf{V}}{|\mathbf{L} + \mathbf{V}|}$$

- Why?
  - Efficient computations
  - Still view-dependent since *H* depends on *V*.



If light source and viewer far away:

H ± constant.

Combining Diffuse & Specular Reflections

- For a single light source we can combine the effects of diffuse and specular reflections simply as follows:

$$I = I_{diff} + I_{spec}$$

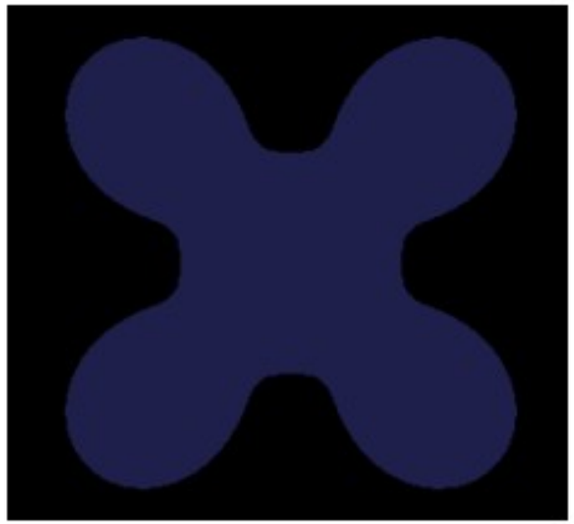
$$= k_a I_a + k_d I_l (N \cdot L) + k_s I_l (V \cdot R)^{n_s}$$

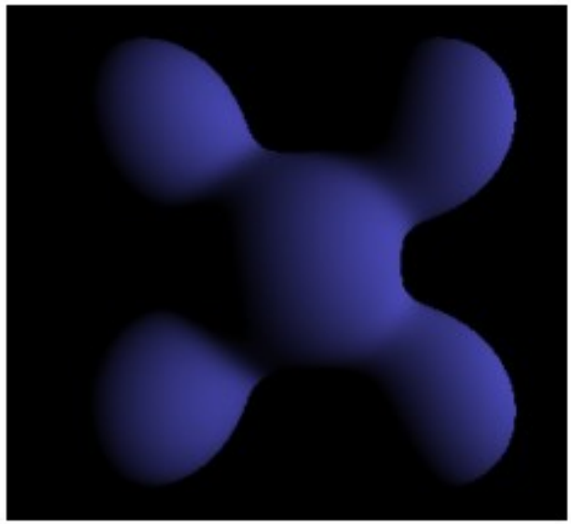
Specular light example

Specular?
Bright spot
on object

Example

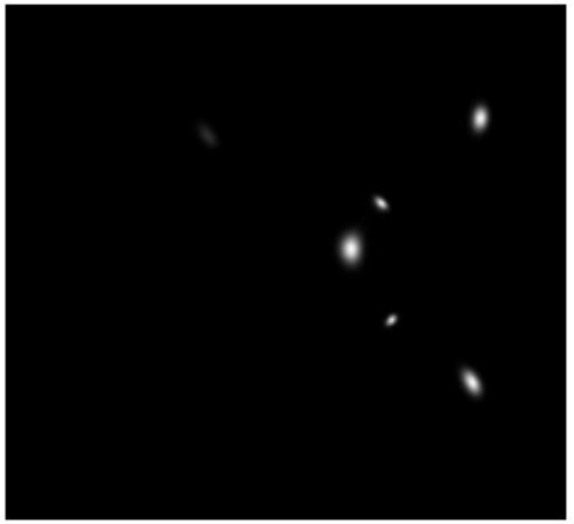
Ambient

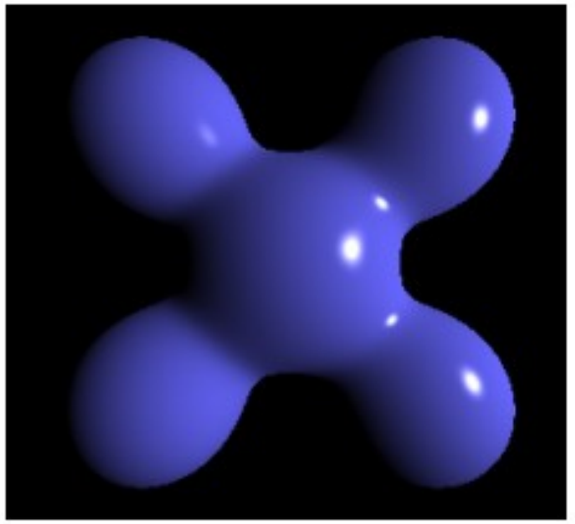




Diffuse

Specular





Final Image

Emission

Some objects are self-luminous
Light intensity due to emission is measured as –

$$I_e = k_i$$

Therefore,

$$k_i = \text{coefficient of the object's intrinsic intensity}$$

$$I = I_{diff} + I_{spec} + I_e$$

$$= k_a I_a + k_d I_l (N \cdot L) + k_s I_l (V \cdot R)^{n_s} + I_e$$

$$I^\lambda = \rho_a^\lambda I_a^{\lambda,\text{in}} + \rho_d^\lambda I_d^{\lambda,\text{in}} (\ell \cdot \mathbf{n}) + \rho_s^\lambda I_s^{\lambda,\text{in}} (\mathbf{r} \cdot \mathbf{v})^f + I_e^\lambda .$$

Multiple Light Sources

- We can place any number of light sources in a scene
- We compute the diffuse and specular reflections as sums of the contributions from the various sources

$$I = I_{ambdiff} + \sum_{l=1}^{n} \left[I_{l,diff} + I_{l,spec} \right]$$

$$= k_a I_a + \sum_{l=1}^{n} I_l \left[k_d (N \cdot L_l) + k_s (V \cdot R_l)^{n_s} \right]$$

$$\mathbf{I} = \rho_a * \mathbf{I}_a^{in} + \rho_d * \sum_{i=1}^{k} \mathbf{I}_d^{in,i} (\ell_i \cdot \mathbf{n}) + \rho_s * \sum_{i=1}^{k} \mathbf{I}_s^{in,i} (\mathbf{r}_i \cdot \mathbf{v})^f + \mathbf{I}_e$$

It is common to specify a global value $\mathbf{I}_{\mathrm{a}}^{\mathrm{in,global}}$, for global ambient light and to have each light source contribute some additional ambient light to the scene. Then

$$\mathbf{I}_{\mathrm{a}}^{\mathrm{in}} = \mathbf{I}_{\mathrm{a}}^{\mathrm{in,global}} + \sum_{i=1}^{k} \mathbf{I}_{\mathrm{a}}^{\mathrm{in},i}.$$

Exercise III.1 *Why is it customary to use the same specular exponent for all wavelengths? What would a specular highlight look like if different wavelengths had different specular exponents?*

Adding Intensity Attenuation

- To incorporate radial and angular intensity attenuation into our model we simply adjust our equation to take these into account
- So, light intensity is now given as:

$$I = I_{ambdiff} + \sum_{l=1}^{n} \left[f_{l,radatten} \, f_{l,angatten} \left(I_{l,diff} + I_{l,spec} \right) \right]$$

- where $f_{radatten}$ and $f_{angatten}$ are as discussed previously

RGB Colour Considerations

- For RGB colour description each intensity specification is a three element vector

$$I_l = (I_{lR}, I_{lG}, I_{lB})$$

- For each light source:

$$k_a = (k_{aR}, k_{aG}, k_{aB})$$

- Similarly all parameters are given as :

$$k_d = (k_{dR}, k_{dG}, k_{dB}) \qquad k_s = (k_{sR}, k_{sG}, k_{sB})$$

- Each component of the surface colour is then calculated with a separate expression

$$I_{lR,diff} = k_{dR} I_{lR} (N \cdot L)$$

$$I_{lB,diff} = k_{dB} I_{lB} (N \cdot L)$$

$$I_{lG,diff} = k_{dG} I_{lG} (N \cdot L)$$

Coefficients for Real Materials

Material	Ambient *Kar, Kag,kab*	Diffuse *Kdr, Kdg,kdb*	Specular *Ksr, Ksg,ksb*	Exponent α
Black plastic	0.0 0.0 0.0	0.01 0.01 0.01	0.5 0.5 0.5	32
Brass	0.329412 0.223529 0.027451	0.780392 0.568627 0.113725	0.992157 0.941176 0.807843	27.8974
Polished Silver	0.23125 0.23125 0.23125	0.2775 0.2775 0.2775	0.773911 0.773911 0.773911	89.6

Figure 8.17, F S Hill, courtesy of McReynolds and Blythe

Exercise

Exercise 11.5. There are two light sources, L^0 and L^1, the respective values of whose lighting properties matrices are

$$\begin{bmatrix} 0.0 & 0.0 & 0.0 \\ 0.7 & 0.1 & 0.1 \\ 0.7 & 0.1 & 0.1 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 0.0 & 0.0 & 0.0 \\ 0.1 & 0.7 & 0.1 \\ 0.1 & 0.7 & 0.1 \end{bmatrix}$$

The material properties matrix at a vertex V is

$$\begin{bmatrix} 0.1 & 0.8 & 0.9 \\ 0.1 & 0.8 & 0.9 \\ 1.0 & 1.0 & 1.0 \end{bmatrix}$$

Furthermore, the shininess exponent of the surface at V is 2.0, the unit outward normal vector at V is

$$[0.0\ 1.0\ 0.0]^T$$

and the emissive light vector at V is

$$[0.01\ 0.0\ 0.0]^T$$

Computer Graphics Through OpenGL
From Theory to Experiments, Third Edition
Sumanta Guha

The position vectors of L^0, L^1, V and the eye are, respectively,

$$[0.0\ 5.0\ 5.0]^T,\ \ [5.0\ 5.0\ 5.0]^T,\ \ [0.0\ 0.0\ 5.0]^T \text{ and } [0.0\ 0.0\ 0.0]^T$$

The global ambient light vector is

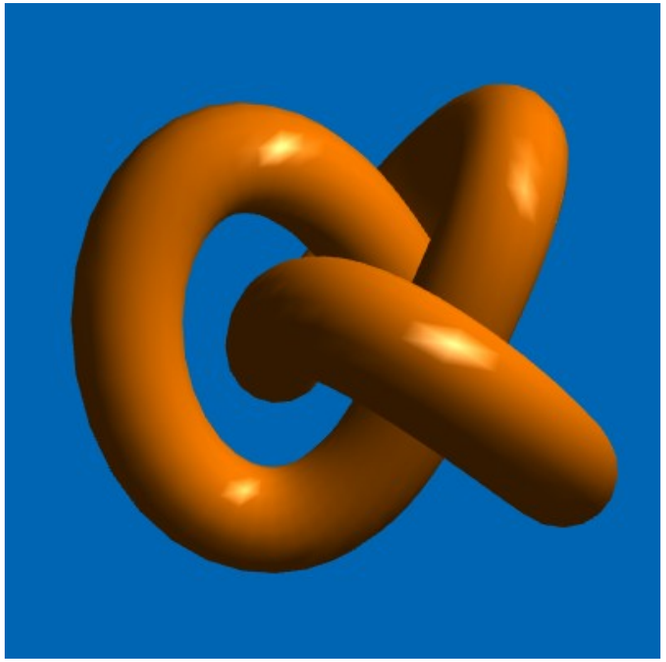
$$[0.1\ 0.1\ 0.1]^T$$

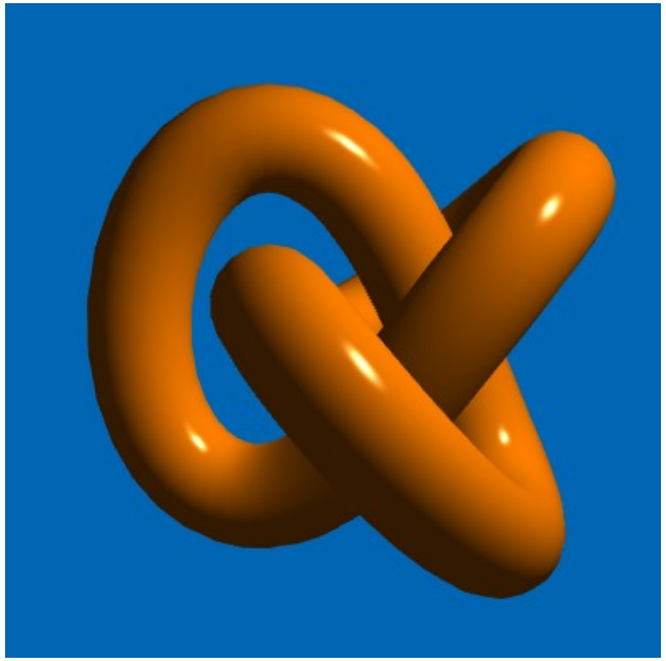
Fill out color values at V in the table below.

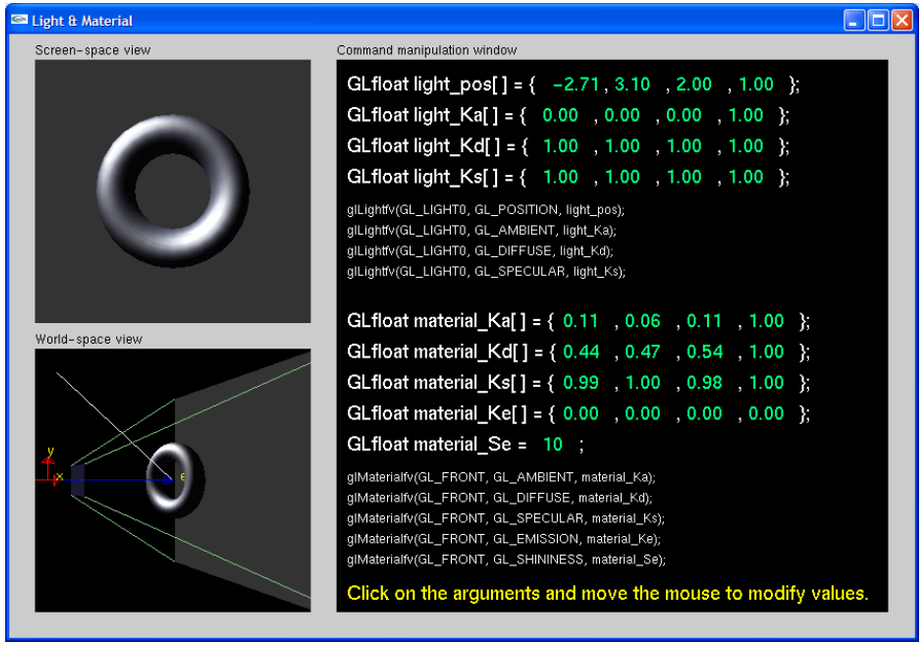
	R	**G**	**B**
Emission			
Global Ambient			
Ambient			
Diffuse			
Specular			
Total (color vector at V)			

Tutorials/Demo

- http://www.cs.toronto.edu/~jacobson/phong-demo/
- https://user.xmission.com/~nate/tutors.html







Nate Robin's OpenGL Tutorials available at: http://users.polytech.unice.fr/~buffa/cours/synthese_image/DOCS/www.xmission.com/Nate/tutors.html

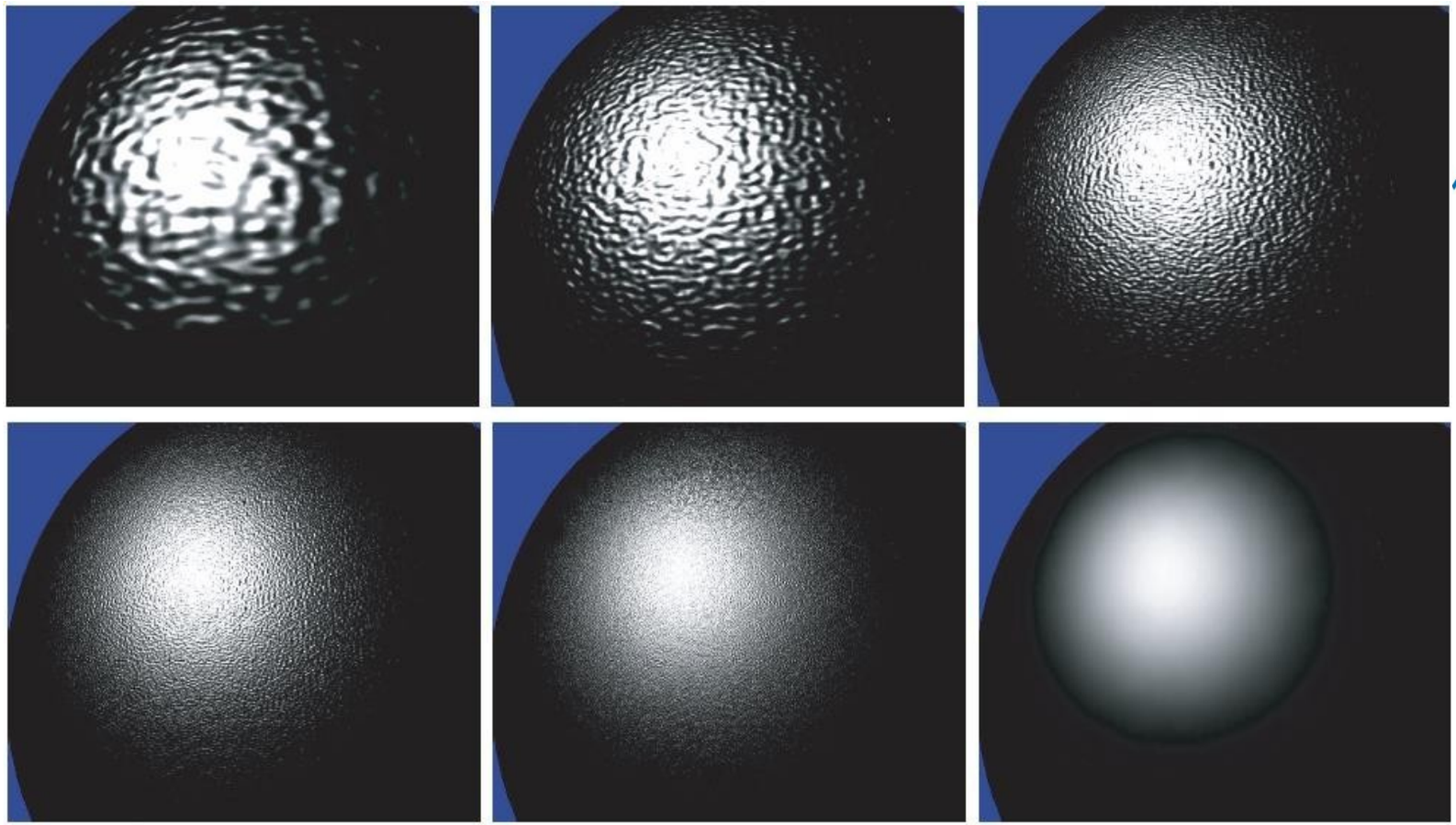
Physically-Based Lighting Models

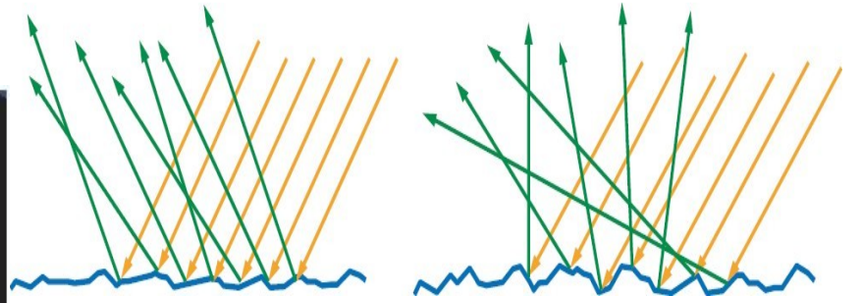
Physically-Based Lighting Models

- Phong model produces pretty pictures
- Cons: empirical (fudged?) ($\cos^{\alpha}\phi$), plastic look
- Shaders can implement better lighting/shading models
- Big trend towards Physically-based lighting models
- Physically-based?
- Based on physics of how light interacts with actual surface
- Apply Optics/Physics theories
- Classic: Cook-Torrance shading model (TOGS 1982)

Microgeometry

- Rougher surfaces bounce light all over the place

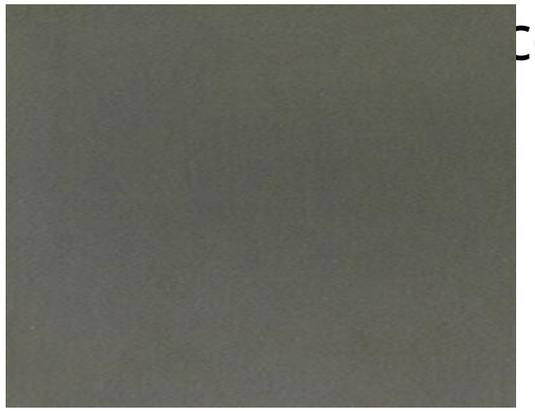




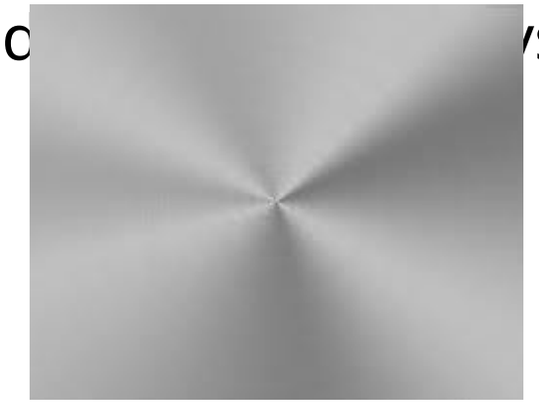
Increasing roughness

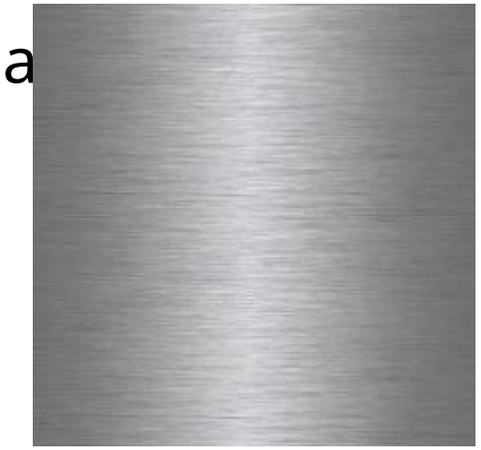
Isotropic Vs Anisotropic Surfaces

- **Isotropic:** light bounced equally in all directions
- **Anisotropic:**
  - Surface has grooves with directions. E.g. Brushed steel



Isotropic





Anisotropic (brushed steel)

Cook-Torrance Shading Model

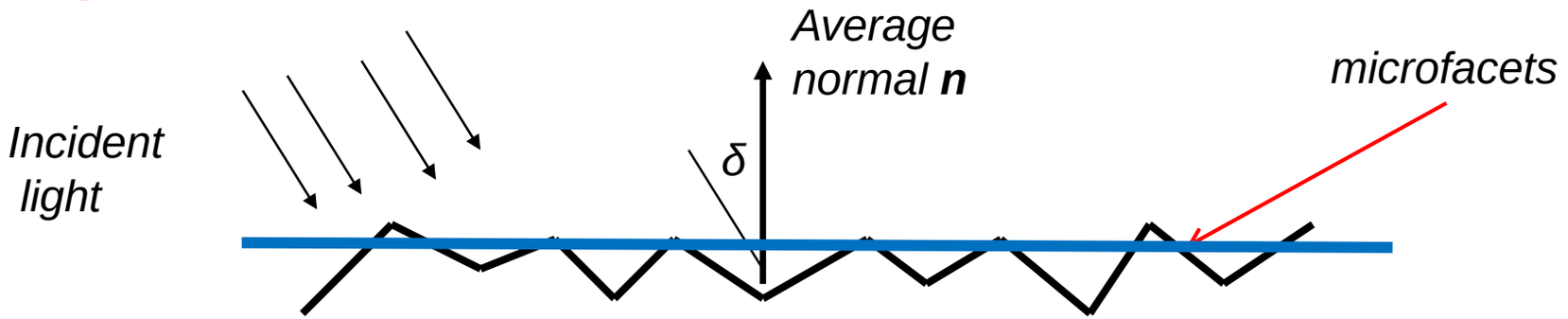
- Same ambient and diffuse terms as Phong
- New, better specular component than ($cos^{\alpha}\phi$),
- Where
  - D - Distribution term
  - G – Geometric term
  - F – Fresnel term

$$\cos^{\alpha}\phi \rightarrow \frac{F(\phi)}{(\mathbf{n}\cdot\eta)}DG$$

$$I_s = \frac{(\mathbf{n}\cdot\ell)}{(\mathbf{n}\cdot\mathbf{v})}\, s\, F\, G\, D \cdot I_s^{\text{in}},$$

Distribution Term, D

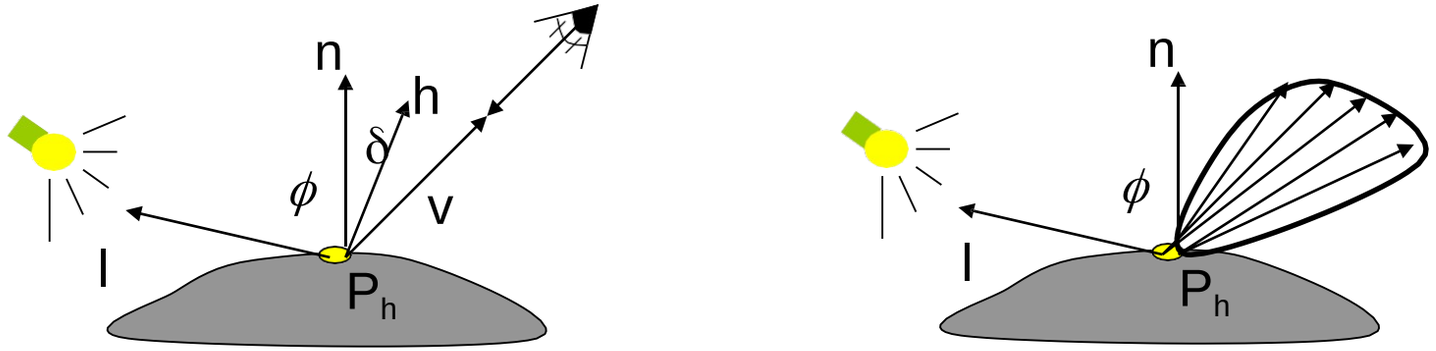
- **Idea:** surfaces consist of small V-shaped **microfacets (grooves)**



- Many grooves at each surface point
- Grooves facing a direction contribute
- D(δ) term: what fraction of grooves facing each angle δ
- E.g. half of grooves at hit point face 30 degrees, etc

Cook-Torrance Shading Model

- Define angle δ as deviation of **h** from surface normal
- Only microfacets with pointing along halfway vector, **h = l + v**, contributes



- Can use old Phong cosine ($cos^n \phi$), as D
- Use Beckmann distribution instead

$$D(\delta) = \frac{1}{4\mathbf{m}^2 \cos^4(\delta_e)} e^{-\left(\frac{\tan(\delta)}{\mathbf{m}}\right)^2}$$

- **m** expresses roughness of surface. How?

Cook-Torrance Shading Model

- **Isotropic Distributions** $D(\omega) \Rightarrow D(\alpha) \quad \alpha = \angle N, H$
  - α : angle to average normal of surface
  - m : average slope of the microfacets

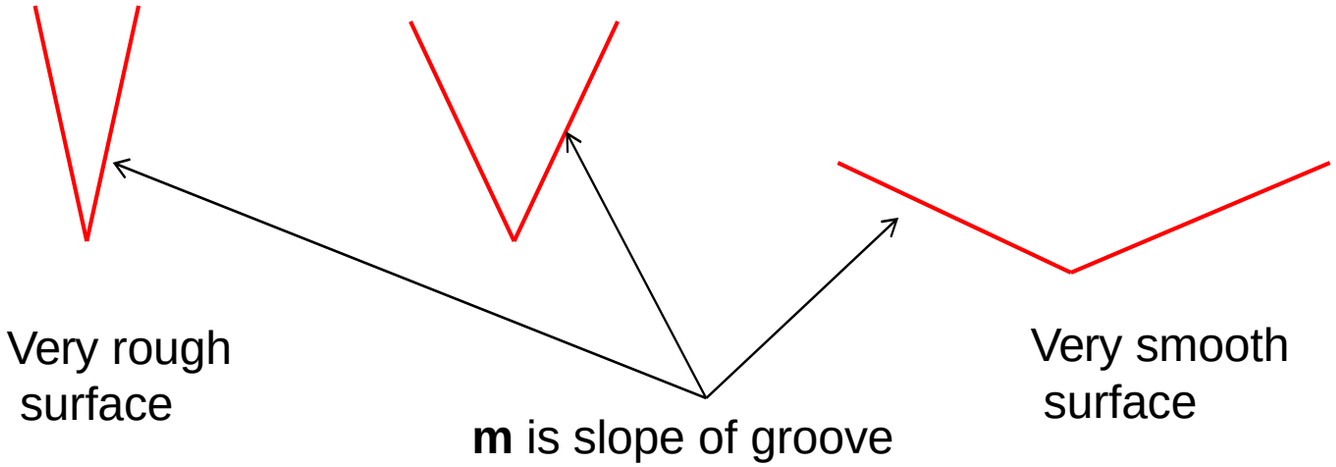
- **Blinn:** $D(\alpha) = cos^{\frac{ln\,2}{ls\,cos\,m_\alpha}}$

- **Torrance-Sparrow** $D(\alpha) = e^{-\left(\frac{\alpha}{m}\right)^2}$
  - Gaussian

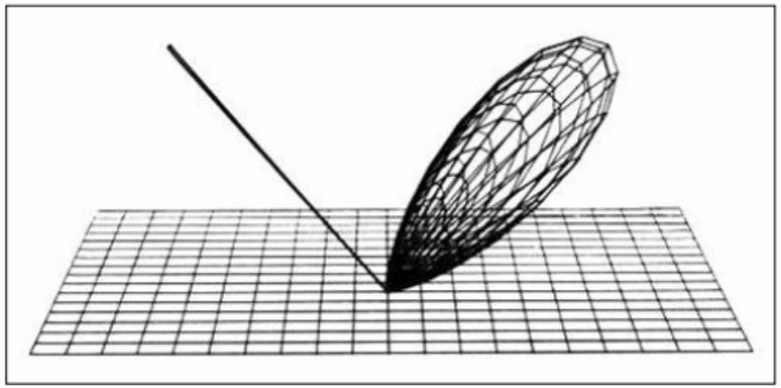
- **Beckmann** $D(\alpha) = \frac{1}{\pi m^2 cos^4 \alpha} e^{-(\frac{tan\alpha}{m})^2}$
  - Used by Cook-Torrance

Cook-Torrance Shading Model

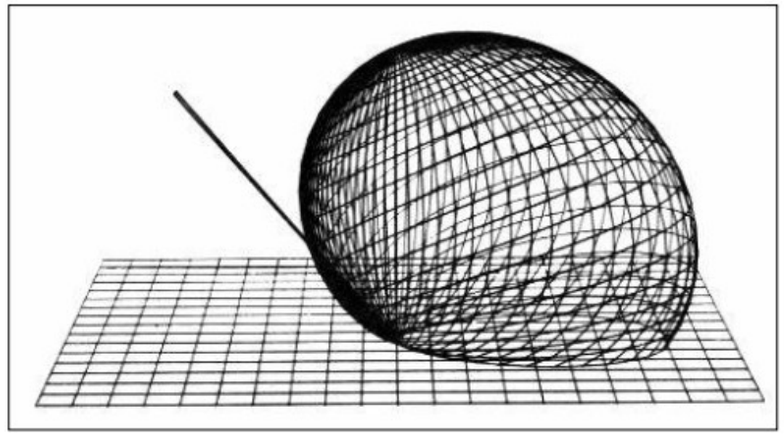
- **m** is Root-mean-square (RMS) of **slope** of V-groove
- m = 0.2 for nearly smooth
- m = 0.6 for very rough



$m=0.2$



$m=0.6$

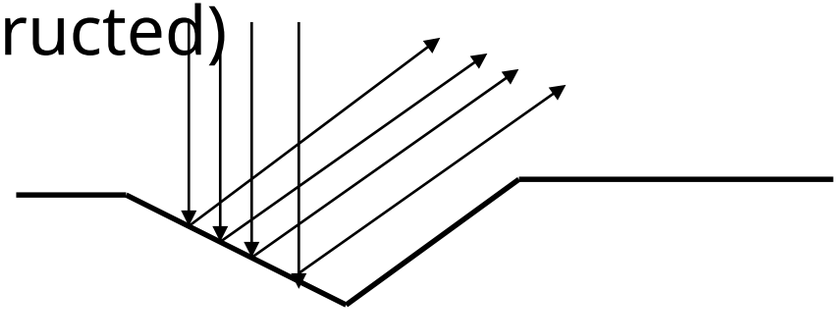


Self-Shadowing (G Term)

- Some grooves on extremely rough surface may block other grooves

Geometric Term, G

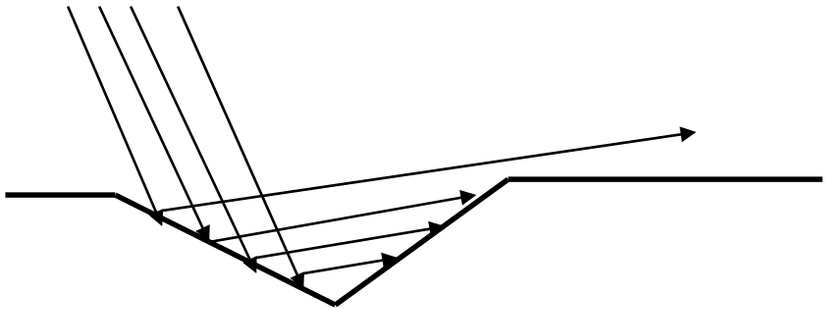
- Surface may be so rough that interior of grooves is blocked from light by edges
- Self blocking known as **shadowing** or **masking**
- Geometric term G accounts for this
- Break G into 3 cases:
- G, case a: No self-shadowing (light in-out unobstructed)



- Mathematically, G = 1

Geometric Term, G

- G_m, case b: No blocking on entry, blocking of exitting light **(masking)**

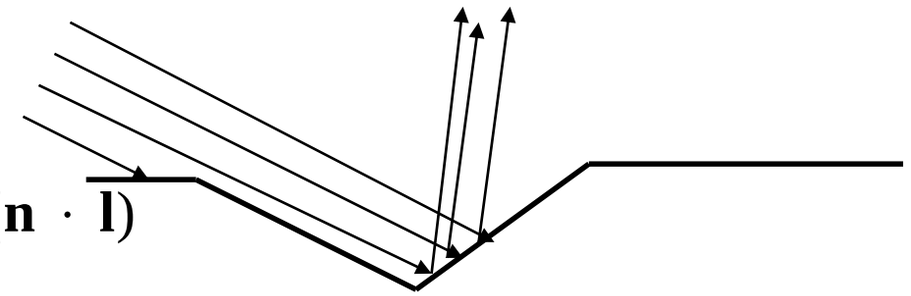


- Mathematically,

$$G_m = \frac{2(\mathbf{n} \cdot \mathbf{h})(\mathbf{n} \cdot \mathbf{v})}{\mathbf{h} \cdot \mathbf{v}}$$

Geometric Term, G

- G_s, case c: blocking of incident light, no blocking of exitting light (**shadowing)**
- Mathematically,



$$G^s = \frac{2(\mathbf{n} \cdot \mathbf{h})(\mathbf{n} \cdot \mathbf{l})}{\mathbf{h} \cdot \mathbf{l}}$$

- G term is minimum of 3 cases, hence

$$G = (1, G_m, G_s)$$

Fresnel Term, F

- F term, $F(\phi, \eta)$ gives fraction of incident light reflected

$$\mathcal{F} = \frac{1}{2}\left(\frac{\sin^2(\varphi-\theta)}{\sin^2(\varphi+\theta)} + \frac{\tan^2(\varphi-\theta)}{\tan^2(\varphi+\theta)}\right)$$

where ϕ is the angle of incidence and θ is the angle of refraction

$$F = \frac{1}{2}\frac{(g-c)^2}{(g+c)^2}\left\{1+\left[\frac{c(g+c)-1}{c(g-c)-1}\right]^2\right\}$$

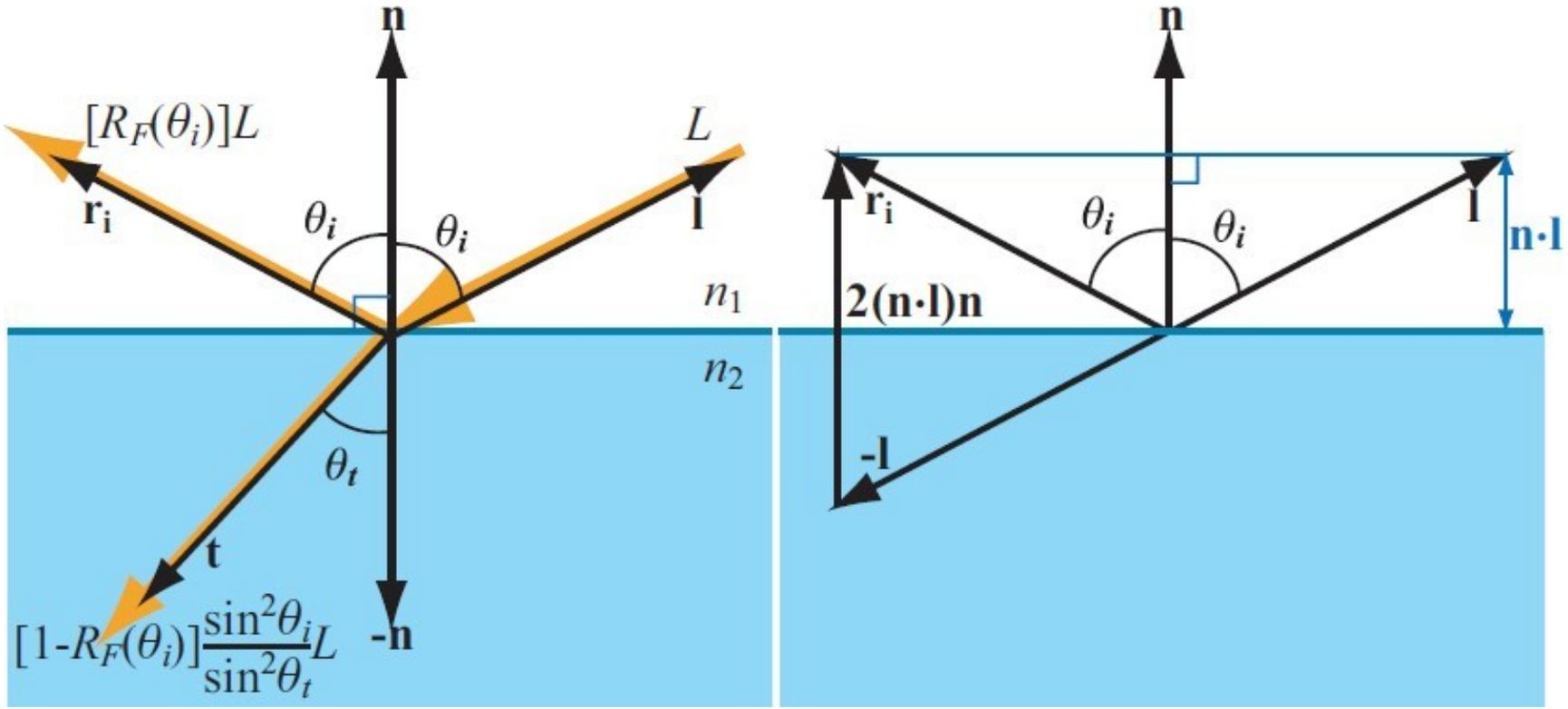
F is function of material and incident angle

- where c = cos(ϕ) = **n.s** and $g^2 = \eta^2 + c^2 + 1$
- ϕ is incident angle, η is refractive index of material

$$\frac{\sin\varphi}{\sin\theta} = \eta.$$

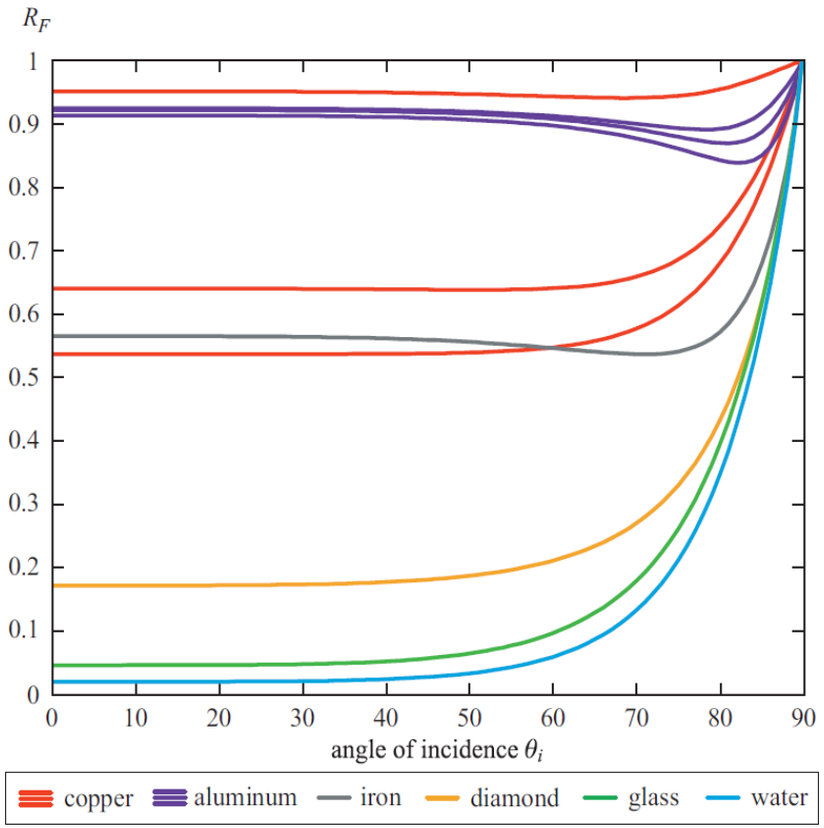
Fresnel Reflectance

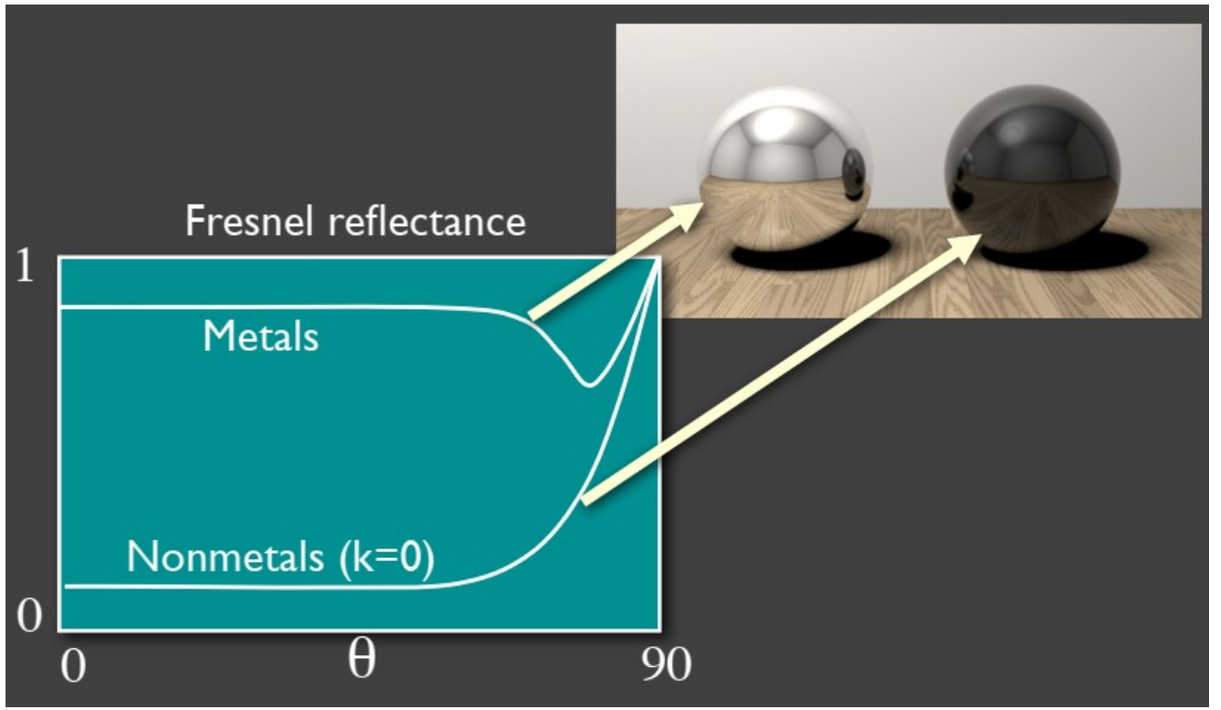
- Equation that determines what fraction of incident light is reflected (and what fraction is transmitted)



Fresnel Reflectance

- Depends on angle of incidence and material





Metal vs. Nonmetal

Fresnel Reflectance

- Usually, physics table for each material's fresnal reflectance at zero degrees of incidence

Material	Fresnel Value (R,G,B)
Water	0.02, 0.02, 0.02
Plastic	0.05, 0.05, 0.05
Glass	0.08, 0.08, 0.08
Diamond	0.17, 0.17, 0.17
Copper	0.95, 0.64, 0.54
Aluminum	0.91, 0.92, 0.92

	Red	Green	Blue
Gold:	0.93	0.88	0.38
Iridium:	0.26	0.28	0.26
Iron:	0.44	0.435	0.43
Nickel:	0.50	0.47	0.36
Copper:	0.93	0.80	0.46
Platinum:	0.63	0.62	0.57
Silver:	0.97	0.97	0.96

- Schlick approximation to get arbitrary F

$$F(\theta) = F(0) + (1 - F(0))(1 - \cos\theta)^5$$

Fresnel Reflectance

- Schlick's approximation of Fresnel

$$R_F(\theta) = R_F(0) + (1 - R_F(0))(1 - cos(\theta))^5$$

- For dielectric

$$R_F(0) = (\frac{\eta - 1}{\eta + 1})^2$$

Exercise III.8★ *Prove that the reflectance $\mathcal{F}$ can also be computed by the formula*

$$\mathcal{F} = \frac{1}{2}\left[\left(\frac{\eta\cos\theta - \cos\varphi}{\eta\cos\theta + \cos\varphi}\right)^2 + \left(\frac{\eta\cos\varphi - \cos\theta}{\eta\cos\varphi + \cos\theta}\right)^2\right]. \qquad \text{III.}$$

[Hint: Use Equation III.20 and use trignometry identities to show

Rob Cook's vases

Sub-Surface Scattering

Crysis skin demo

Marble

Human Skin

More Examples…

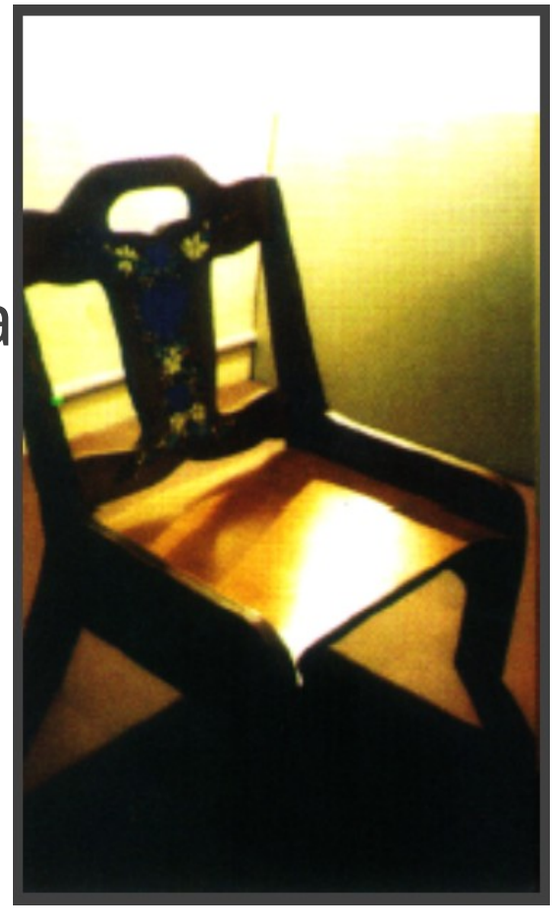
Leaves

Hair

Milk

Ward Model

- Physically valid
  - Energy conserving
  - Satisfies reciprocity
  - Easy to integrate
- Based on empirical data
- Isotropic and anisotropic materia



Ward Model

- Isotropic

$$f_s = \rho_s \frac{1}{4\pi m^2} \frac{1}{\sqrt{N.LN.V}} e^{-\frac{tan^2\theta_h}{m^2}}$$

where,
- m (usually α) is surface roughness

Ward Model

- anisotropic

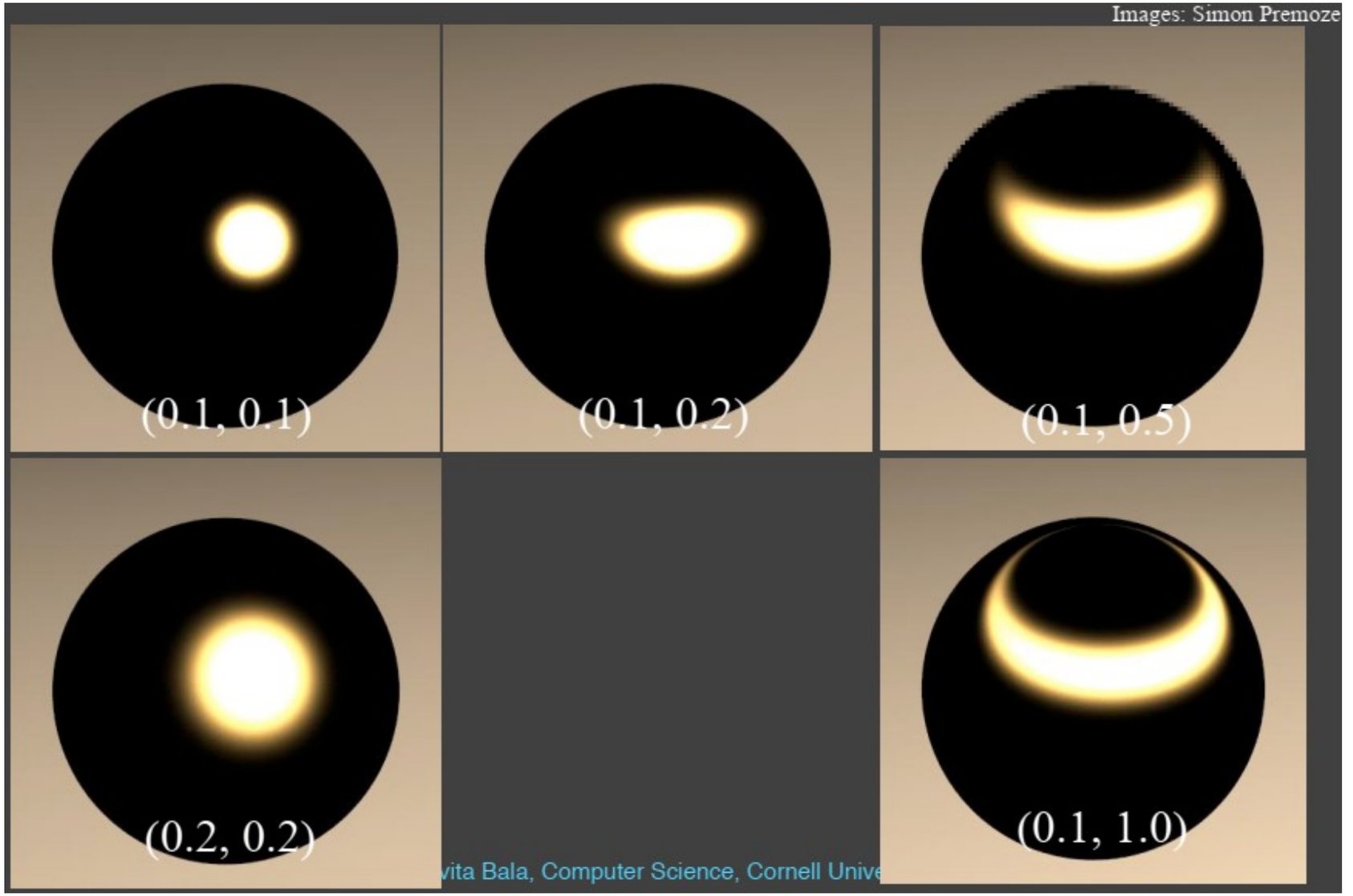
$$f_s = \rho_s \frac{1}{4\pi m_x m_y} \frac{1}{\sqrt{N.LN.V}} e^{-tan^2\theta_h\left(\frac{cos^2\phi_h}{m_x^2}+\frac{sin^2\phi_h}{m_y^2}\right)}$$

$$f_s = \rho_s \frac{1}{4\pi m_x m_y} \frac{1}{\sqrt{N.LN.V}} e^{-2\frac{\left(\frac{H.\hat{x}}{m_x}\right)^2+\left(\frac{H.\hat{y}}{m_y}\right)^2}{1+N.H}}$$

where,

- m_x, m_y are surface roughness in $\hat{x}, \hat{y}$
- $\hat{x}, \hat{y}$ are mutually perpendicular to the normal

Ward Model - Example



Ward Model - Example

Summary

- To create realistic (or even semi-realistic) looking scenes we must model light correctly
- To successfully model lighting effects we need to consider:
  - Ambient light
  - Diffuse reflections
  - Specular reflections

References

- 3-D Computer Graphics A Mathematical Introduction with OpenGL S R Buss
- Interactive Computer Graphics (6th edition), Angel and Shreiner
- Computer Graphics using OpenGL (3rd edition), Hill and Kelley
- procedural-elements-of-computer-graphics C Rogers
- Peter Shirley book

Prof Emmanuel Agu *Computer Science Dept. Worcester Polytechnic Institute (WPI)*

Ray-tracing

Dr. Sk. Md. Masudul Ahsan
Professor
Dept. of CSE, KUET

Ray Tracing

- ~ a global illumination model
- Integrates the task of
  - Shading
  - shadow generation
  - Hidden surface removal
  - Projection
  - Scan conversion
- Ray tracing is a powerful rendering technique that is the foundation of many modern photoreal rendering algorithms
- The original ray tracing technique was proposed in 1980 by Turner Whitted, although there were suggestions about the possibility in scientific papers dating back to 1968

What is ray tracing

- Ray tracing is a technique for rendering 3D graphics with very complex light interactions.
  - You can create pictures full of mirrors, transparent surfaces, and shadows, with stunning results.
- A very simple method to understand and implement.
- It is based on the idea that you can model reflection and refraction by recursively following the path that light takes as it bounces through an environment

Ray-Tracing Examples

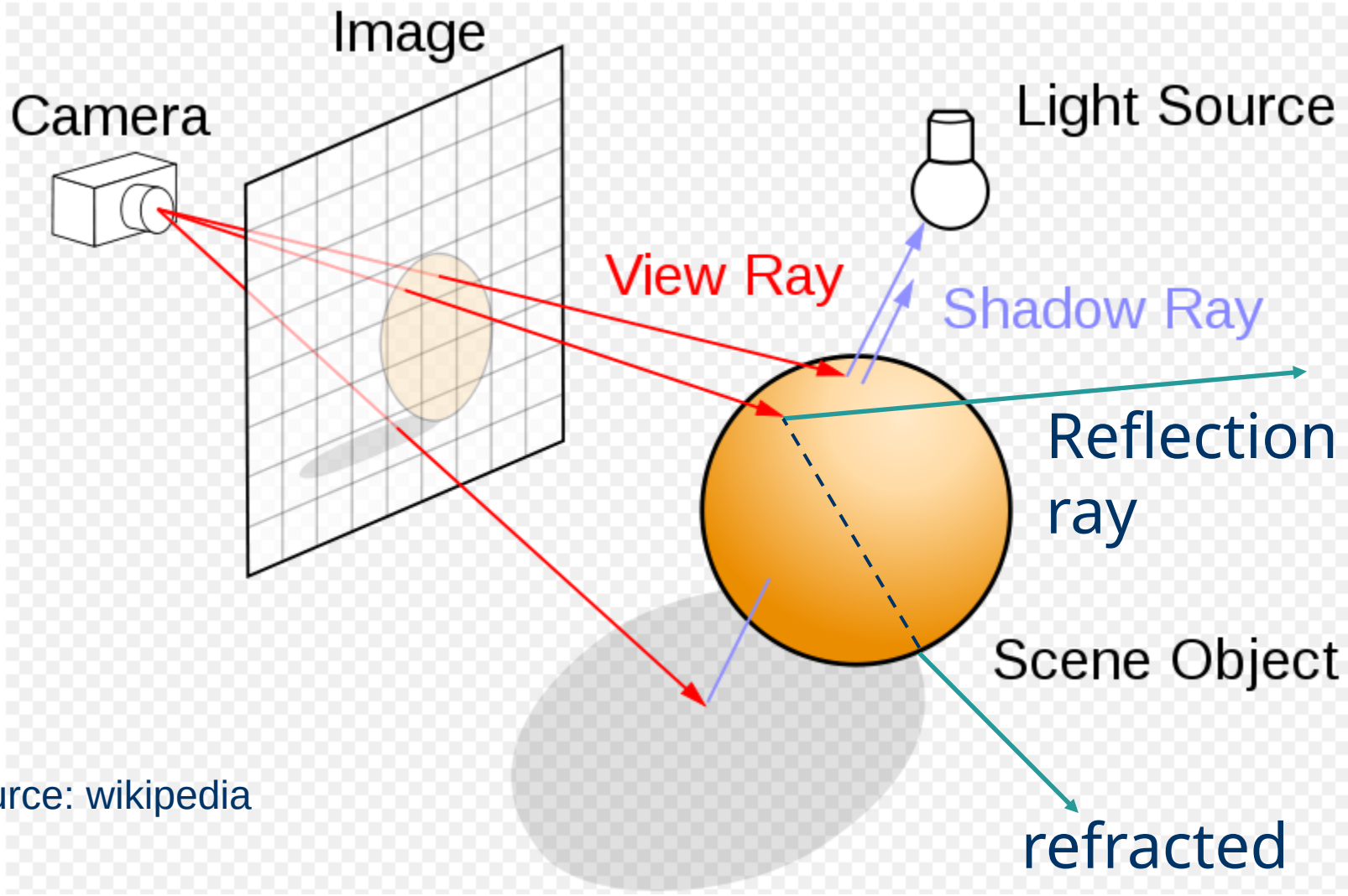
https://deepubalan.com/blog/2010/01/03/pov-ray-the-power-of-ray-tracing/

Ein mit POV-Ray erzeugtes Bild.
© Gilles Tran www.oyonale.com)

Ray-Tracing Examples

Kettle, Mike Miller, POV-Ray

Ray-Tracing Setup



Source: wikipedia

Ray-Tracing Process

- Builds the image pixel by pixel
- Fire a single ray for each pixel position into the scene along the projection path
  - If the ray doesn't hit anything, then we can color the pixel to some specified 'background' color
  - Otherwise,
    - Determine which surfaces the ray intersects and order these by distance from the pixel
      - it is possible that the ray will hit several surfaces, but we only care about the **closest one** to the camera
      - The nearest surface to the pixel is the visible surface for that pixel
    - Shoot rays toward each light. If they hit something, the object is shadowed from that light, otherwise use "standard model" for the light
    - Reflect a ray off the visible surface along the specular reflection angle
    - For transparent surfaces also send a ray through the surface in the refraction direction
    - Repeat the process for these secondary rays

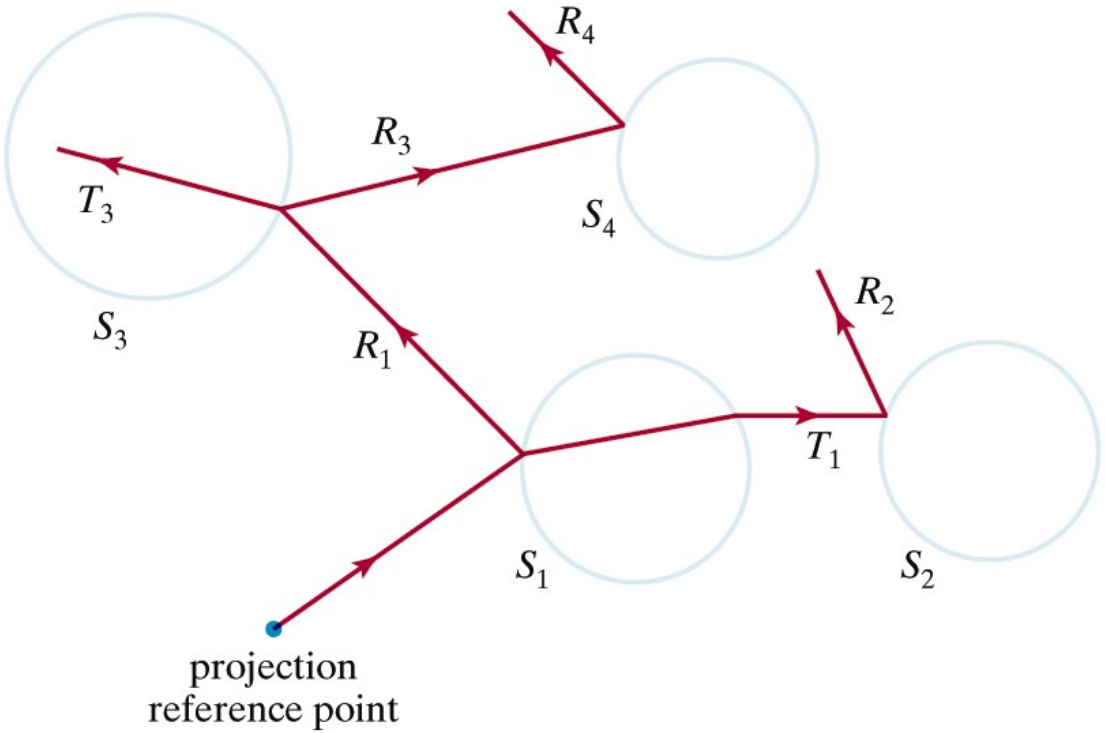
Terminating Ray-Tracing

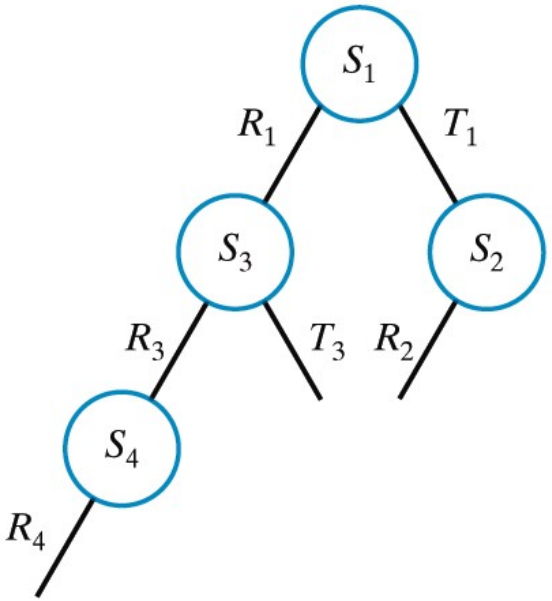
- We terminate a ray-tracing path when any one of the following conditions is satisfied:
  - The ray intersects no surfaces
  - The ray intersects a light source that is not a reflecting surface
  - A maximum allowable number of reflections have taken place

Ray-Tracing Tree

- As the rays ricochet around the scene each intersected surface is added to a binary **ray-tracing tree**
  - The left branches in the tree are used to represent reflection paths
  - The right branches in the tree are used to represent transmission paths
- The tree's nodes store the intensity at that surface
- The tree is used to keep track of all contributions to a given pixel

Ray-Tracing Tree Example (cont…)





Ray-Tracing Tree (cont…)

- After the ray-tracing tree has been completed for a pixel the intensity contributions are accumulated
- We start at the terminal nodes (bottom) of the tree
- The surface intensity at each node is attenuated by the distance from the parent surface and added to the intensity of the parent surface
- The sum of the attenuated intensities at the root node is assigned to the pixel

Recursive Ray Tracer

- Use the camera analogy
- Determine light rays responsible for pixels, in opposite direction (Primary ray)
- If primary ray doesn't intersect any object, corresponding pixel is simply set to Bg color
- If intersects - Secondary rays used for surface shading
  - Local contribution
  - Reflected contribution
  - Transmitted contribution

Recursive Ray Tracer

- Employ the Phong formula to describe the contribution

$$I = I_a k_a + \sum_{i=1}^{n} I_{p_i} (k_d L_i . N + k_s (R_i . V)^k) + k_r I_r + k_t I_t$$

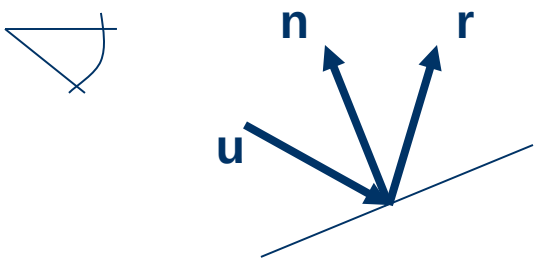
- I_r and I_t are obtained recursively by treating them as primary ray
- I_r is calculated by evaluating the above eq at the closest surface it intersects, as the same as primary ray

Recursive Ray Tracer Algorithm

```
rayTrace(ray, depth, color)
{
    determine closest intersection of ray with an object
    if (no intersection) color = bg
    else
    {
            color = local contribution
            if (depth > 1)
            {
                    calculate reflected ray
                    rayTrace(reflected ray, depth -1, ref_color)
                    calculate transmitted ray
                    rayTrace(transmitted ray, depth -1, trans_color)
                    Color= color + k_r.ref_color + k_t.trans_color
            }
    }
}
```

Recursive Ray Tracer

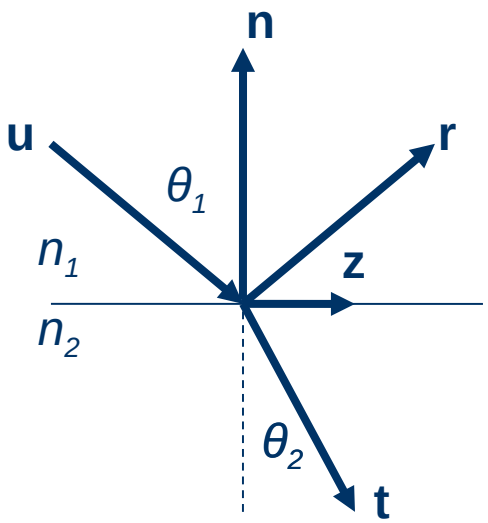
- Local Contribution
  - Direct contribution from light source(s)
  - A shadow ray is sent from the surface point to the light source
  - If the ray is blocked before reaching the light source, the point is in shadow
- Reflected Contribution
  - Refers inter object reflection
  - A reflected ray – reflection of primary ray with surface normal



$$\mathbf{r} = \mathbf{u} - 2(\mathbf{u} \cdot \mathbf{n})\mathbf{n}$$

Recursive Ray Tracer

- Transmitted Contribution
  - Refers the transmission of light energy coming from behind the surface
  - A transmitted ray – refraction of primary ray with surface normal
  - Use Snell's law $\eta_1 \sin\theta_1 = \eta_2 \sin\theta_2$

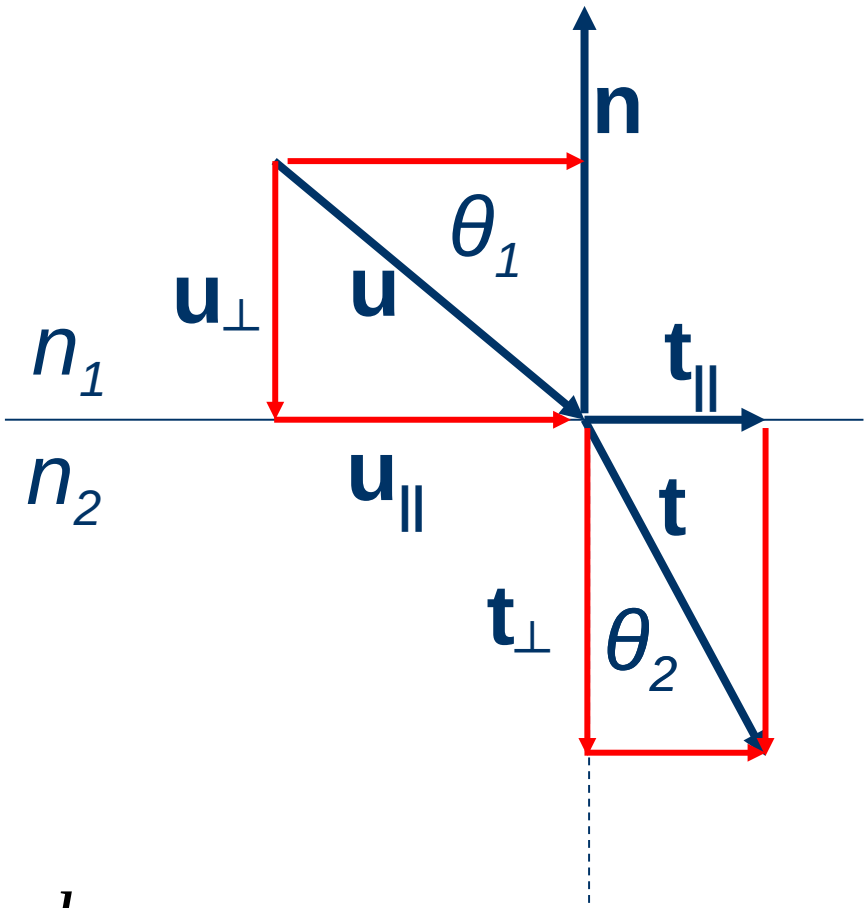


$$\mathbf{r} = \mathbf{u} - 2(\mathbf{u}\cdot\mathbf{n})\mathbf{n}$$

$$\mathbf{z} = \frac{n_1}{n_2}(\mathbf{u} - (\mathbf{u}\cdot\mathbf{n})\mathbf{n})$$

$$\mathbf{t} = \mathbf{z} - \left(\sqrt{1 - |\mathbf{z}|^2}\right)\mathbf{n}$$

Transmitted Ray



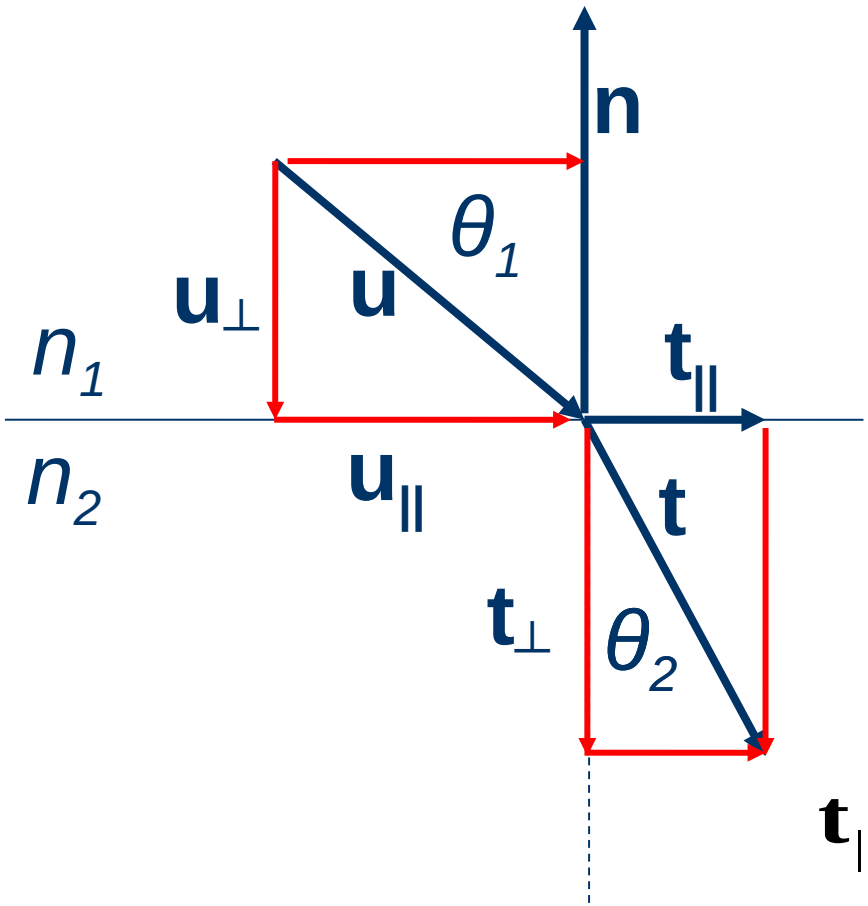
- Let, u, n, t all are unit vector
- $\mathbf{t} = \mathbf{t}_\perp + \mathbf{t}_\parallel$
- $\mathbf{t}_\parallel = \alpha \mathbf{u}_\parallel \quad ; \quad \mathbf{t}_\perp = -\beta \mathbf{n}$

here,

$$\mathbf{u} \bullet \mathbf{n} = |\mathbf{u}||\mathbf{n}| \cos(180^\circ - \theta_1)$$
$$= -\cos\theta_1$$

$$\mathbf{u}_\parallel = \mathbf{u} - \mathbf{u}_\perp$$
$$= \mathbf{u} - (-\gamma \mathbf{n})$$
$$= \mathbf{u} + |\mathbf{u}| \cos\theta_1 \mathbf{n}$$
$$= \mathbf{u} + \cos\theta_1 \mathbf{n}, \text{ since } |\mathbf{u}| = 1$$
$$= \mathbf{u} - (\mathbf{u} \bullet \mathbf{n})\mathbf{n},$$
$$\text{since } \mathbf{u} \bullet \mathbf{n} = -\cos\theta_1$$

Transmitted Ray



$$\eta_1 \sin\theta_1 = \eta_2 \sin\theta_2$$

$$\Rightarrow \eta_1 \frac{|\mathbf{u}_{\parallel}|}{|\mathbf{u}|} = \eta_2 \frac{|\mathbf{t}_{\parallel}|}{|\mathbf{t}|}$$

$$\Rightarrow \eta_1 |\mathbf{u}_{\parallel}| = \eta_2 |\mathbf{t}_{\parallel}|, \text{ since } |\mathbf{u}, \mathbf{t}| = 1$$

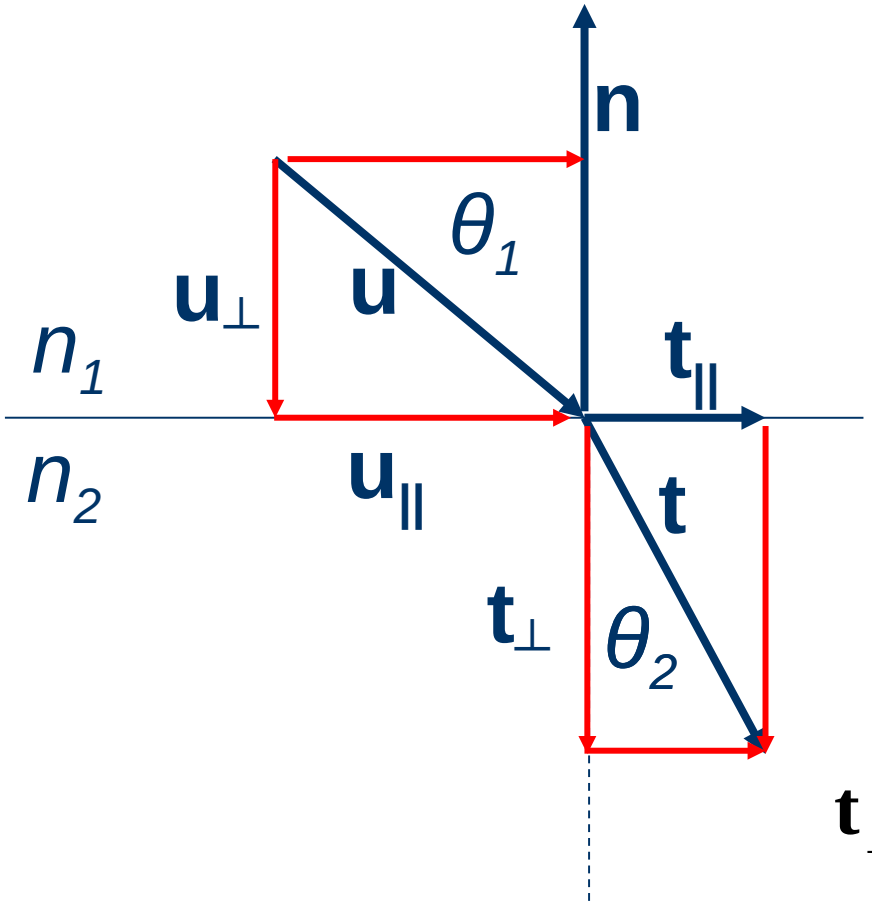
$$\Rightarrow \eta_1 |\mathbf{u}_{\parallel}| = \eta_2 |\alpha \mathbf{u}_{\parallel}|$$

$$\Rightarrow \alpha = \frac{\eta_1}{\eta_2}$$

$$\mathbf{t}_{\parallel} = \frac{\eta_1}{\eta_2} \mathbf{u}_{\parallel}$$

$$= \frac{\eta_1}{\eta_2} (\mathbf{u} - (\mathbf{u} \bullet \mathbf{n})\mathbf{n}) = \mathbf{z}$$

Transmitted Ray



$$|\mathbf{t}|^2 = |\mathbf{t}_\perp|^2 + |\mathbf{t}_\parallel|^2$$

$$\Rightarrow |\mathbf{t}_\perp| = \sqrt{1 - |\mathbf{t}_\parallel|^2}$$

$$\Rightarrow |-\beta\,\mathbf{n}| = \sqrt{1 - |\mathbf{t}_\parallel|^2}$$

$$\Rightarrow \beta\,|\mathbf{n}| = \sqrt{1 - |\mathbf{t}_\parallel|^2}, \text{ since } |\mathbf{V}| = |-\mathbf{V}|$$

$$\Rightarrow \beta = \sqrt{1 - |\mathbf{t}_\parallel|^2}$$

$$\mathbf{t}_\perp = -\beta\,\mathbf{n} = -\left(\sqrt{1 - |\mathbf{t}_\parallel|^2}\right)\mathbf{n}$$

$$\mathbf{t} = \mathbf{t}_\parallel + \mathbf{t}_\perp$$

$$\mathbf{t} = \mathbf{t}_\parallel - \left(\sqrt{1 - |\mathbf{t}_\parallel|^2}\right)\mathbf{n}$$

Transmitted Ray – Hearn n Baker book

Something might be wrong

Checked, seems fine

$$T = \frac{\eta_i}{\eta_r} u - \left(\cos\theta_r - \frac{\eta_i}{\eta_r} \cos\theta_i \right) N$$

$$\cos\theta_r = \sqrt{1 - \left(\frac{\eta_i}{\eta_r} \right)^2 (1 - \cos^2\theta_i)}$$

Transmitted Ray – Peter Shirley book

$$n \sin\theta = n_t \sin\phi. \qquad \cos^2\phi = 1 - \frac{n^2\left(1-\cos^2\theta\right)}{n_t^2}.$$

$$\mathbf{t} = \sin\phi\mathbf{b} - \cos\phi\mathbf{n}.$$

Since we can describe $\mathbf{d}$ in the same basis, and $\mathbf{d}$ is known, we can solve for $\mathbf{b}$:

$$\mathbf{d} = \sin\theta\mathbf{b} - \cos\theta\mathbf{n},$$
$$\mathbf{b} = \frac{\mathbf{d} + \mathbf{n}\cos\theta}{\sin\theta}.$$

This means that we can solve for $\mathbf{t}$ with known variables:

$$\mathbf{t} = \frac{n\left(\mathbf{d} + \mathbf{n}\cos\theta\right)}{n_t} - \mathbf{n}\cos\phi$$
$$= \frac{n\left(\mathbf{d} - \mathbf{n}(\mathbf{d}\cdot\mathbf{n})\right)}{n_t} - \mathbf{n}\sqrt{1 - \frac{n^2\left(1-(\mathbf{d}\cdot\mathbf{n})^2\right)}{n_t^2}}.$$

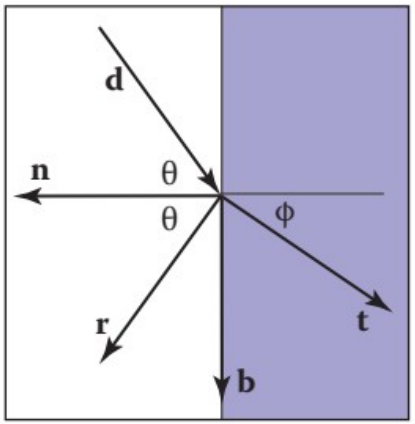
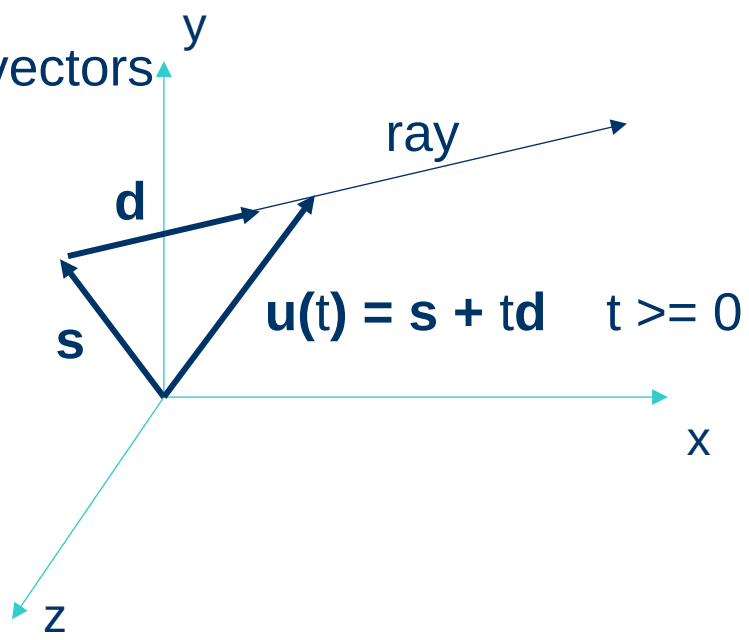


Figure 13.2. The vectors **n** and **b** form a 2D orthonormal basis that is parallel to the transmission vector **t**.

Ray - Object Intersection

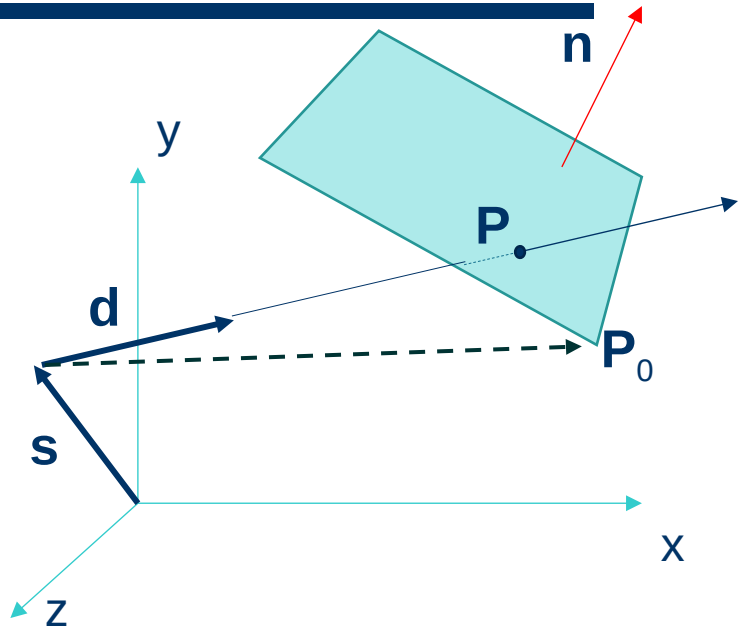
- Representation of ray
  - **s** = $[x_s\ y_s\ z_s]^T$ specify the starting point
  - **d** = $[x_d\ y_d\ z_d]^T$ describe the direction
  - Assume | d | = 1
- Parametric form
  - **u**(t) = **s** + t**d** (0 ≤t)
    - Where **u**(t) denotes a family of vectors



Ray - Object Intersection

- **Ray – Surface intersection**
- Surface
  - $\mathbf{n} = x_n\mathbf{I} + y_n\mathbf{J} + z_n\mathbf{K}$
  - $\mathbf{P_0}\ (x_0, y_0, z_0)$
- Ray, $\mathbf{u} = \mathbf{s} + t\mathbf{d}$ $\quad (0 \le t)$
- A point P(x,y,z) on plane

  $x_n(x - x_0) + y_n(y - y_0) + z_n(z - z_0) = 0$



- For some t the ray intersects the plane at P, so

  $x_n(x_s + tx_d - x_0) + y_n(y_s + ty_d - y_0) + z_n(z_s + tz_d - z_0) = 0$

$$t = -\frac{x_n(x_s - x_0) + y_n(y_s - y_0) + z_n(z_s - z_0)}{x_n x_d + y_n y_d + z_n z_d}$$

$$= \frac{n.\overline{sP_0}}{n.d}$$

n.d = 0, parallel ray

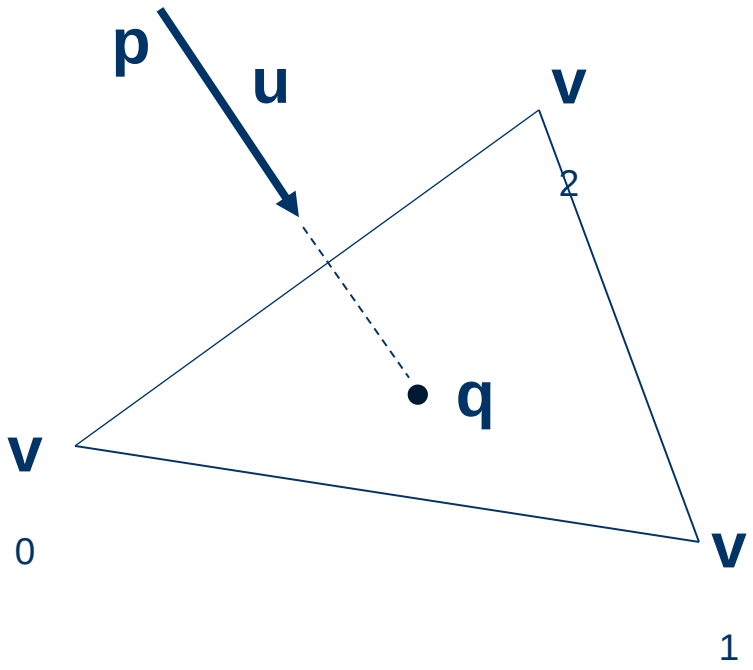
t>0, ray intersects

Ray-Triangle Intersection

- To intersect a ray with a triangle, we must first check if the ray intersects the plane of the triangle
- If we are treating our triangle as one-sided, then we can also verify that the origin of the ray is on the outside of the triangle
- Once we know that the ray hits the plane at point **q**, we must verify that **q** lies inside the 3 edges of the triangle

Ray-Triangle

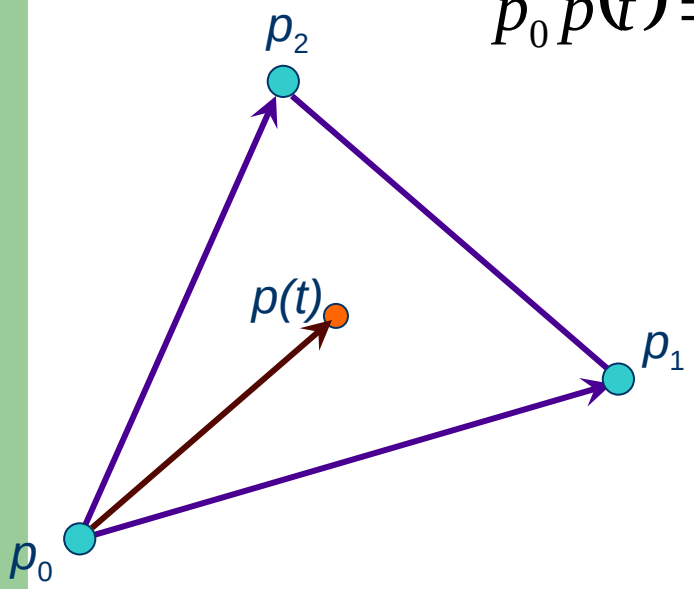
- Does segment **ab** intersect triangle $\mathbf{v}_0\mathbf{v}_1\mathbf{v}_2$?



Badouel's Algorithm

- First, find the intersection p(t) between the ray and the plane containing the triangle.
- Then use the barycentric coordinates of the point to sort the intersection out (2D algorithm).

$$\overrightarrow{p_0 p(t)} = \alpha . \overrightarrow{p_0 p_1} + \beta . \overrightarrow{p_0 p_2} \quad \alpha > 0, \beta > 0, \alpha + \beta < 1$$



$$\begin{aligned} x_t - x_0 &= \alpha(x_1 - x_0) + \beta(x_2 - x_0) \\ y_t - y_0 &= \alpha(y_1 - y_0) + \beta(y_2 - y_0) \\ z_t - z_0 &= \alpha(z_1 - z_0) + \beta(z_2 - z_0) \end{aligned}$$

https://ton.github.io/lightbox/2015/04/27/didier-badouel-ray-triangle-intersection.html
https://graphics.stanford.edu/courses/cs348b-98/gg/intersect.html

Badouel's Algorithm

$$\overrightarrow{p_0 p(t)} = \alpha . \overrightarrow{p_0 p_1} + \beta . \overrightarrow{p_0 p_2} \qquad \alpha > 0, \beta > 0, \alpha + \beta < 1$$

$$\vec{a} = \alpha \vec{b} + \beta \vec{c}$$

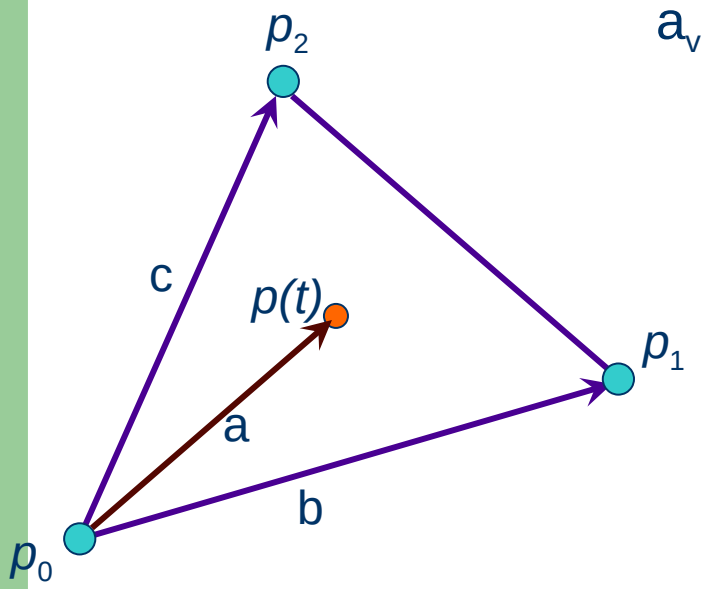
Reduce to 2D: remove smallest dimension say, z

$$x_t - x_0 = \alpha(x_1 - x_0) + \beta(x_2 - x_0)$$

a_u b_u c_u

$$y_t - y_0 = \alpha(y_1 - y_0) + \beta(y_2 - y_0)$$

a_v b_v c_v



$$\begin{pmatrix} b_u & c_u \\ b_v & c_v \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} a_u \\ a_v \end{pmatrix}$$

$$\alpha = \frac{\begin{vmatrix} a_u & c_u \\ a_v & c_v \end{vmatrix}}{\begin{vmatrix} b_u & c_u \\ b_v & c_v \end{vmatrix}} \qquad \beta = \frac{\begin{vmatrix} b_u & a_u \\ b_v & a_v \end{vmatrix}}{\begin{vmatrix} b_u & c_u \\ b_v & c_v \end{vmatrix}}$$

https://ton.github.io/lightbox/2015/04/27/didier-badouel-ray-triangle-intersection.html

https://graphics.stanford.edu/courses/cs348b-98/gg/intersect.html

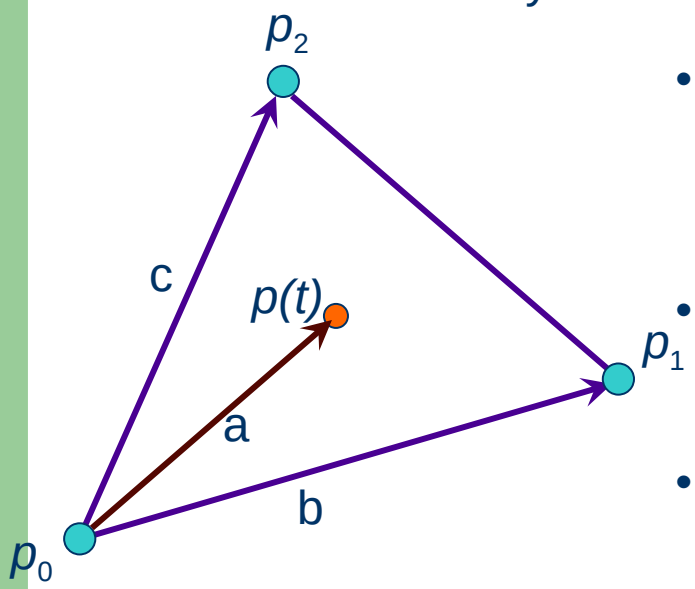
Badouel's Algorithm

$$\overrightarrow{p_0 p(t)} = \alpha . \overrightarrow{p_0 p_1} + \beta . \overrightarrow{p_0 p_2} \qquad \alpha > 0, \beta > 0, \alpha + \beta < 1$$

$$\vec{a} = \alpha \vec{b} + \beta \vec{c}$$

Reduce to 2D: remove smallest dimension say, z

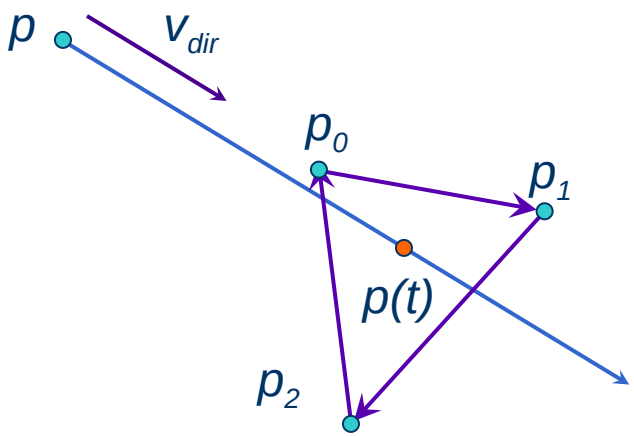
- Badouel now simplifies this system of equations by projecting the triangle on one of the primary planes xy, xz, or yz.
- Badouel uses the fact that after projecting a triangle, the Barycentric coordinate (α,β) of P does not change.



- A requirement for this is that the projected triangle does not degenerate to a line. Thus, we need to make sure that we do not project the triangle on a plane perpendicular to the triangle.
- To avoid this situation, the **dominant** axis of the triangle's normal is determined, and the triangle is **projected** on the plane perpendicular to that axis.
- Note that by taking the dominant axis, the area of the projected triangle is as large as possible, avoiding numerical errors.
- Now, assuming that the X-axis is the dominant axis of the triangle, projecting on the yz plane

Other Ray/Triangle Algorithms

- The edges of the triangles can be considered as oriented.
- The Ray should always be “on the same side” of the edges
- => Three triple products with the same sign.

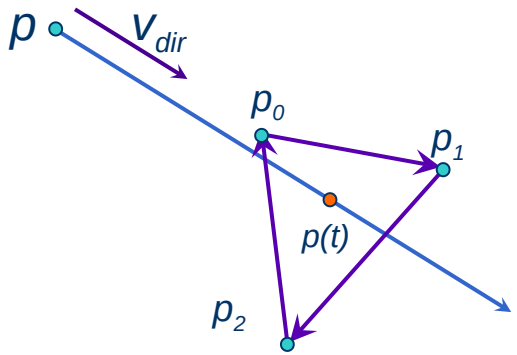


Segura, R.J., Feito, F.R. Segura, R.J., Feito, F.R.: An algorithm for determining intersection segment-polygon in 3D. Computer & Graphics, Vol.22, No.5, pp.587-592, 1998.

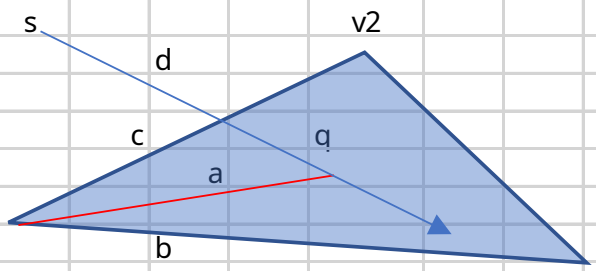
Other Ray/Triangle Algorithms

- There is an intersection Ray/Triangle only if:

$$Sign\left(\left(\overrightarrow{pp_0} \times \overrightarrow{pp_1}\right) \cdot v\right) = Sign\left(\left(\overrightarrow{pp_1} \times \overrightarrow{pp_2}\right) \cdot v\right) = Sign\left(\left(\overrightarrow{pp_2} \times \overrightarrow{pp_0}\right) \cdot v\right)$$



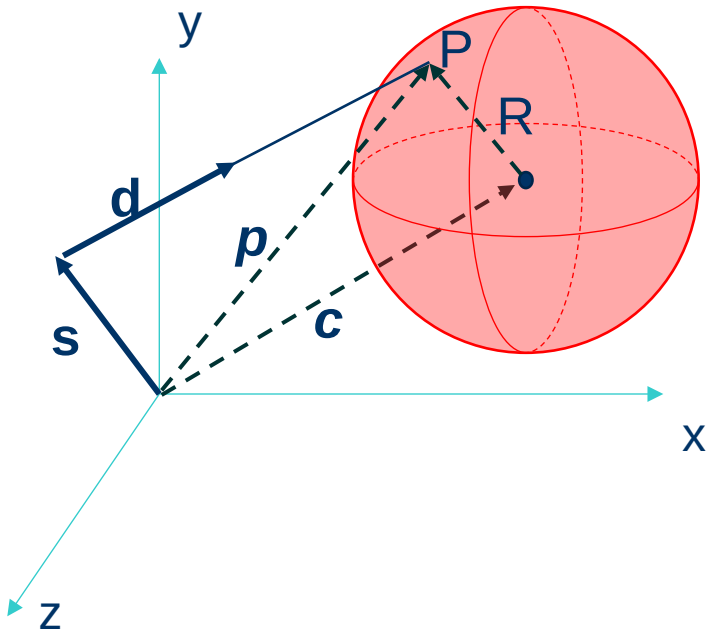
- Fast algorithm.
  - Can take advantage of SIMD instructions on processor.
- But:
  - Only in order to detect an intersection.
  - Extra work must be done in order to get the tri-linear interpolating weights (including a div) and the real intersection location if needed (but not always required).



													x	y	z						
											P0	v0	0	0	0						
												v1	6	-1	-5						
												v2	4	4	1						
												q	1.42	1.21	0.16						
											q-v0	a	1.42	1.21	0.16	1.87	0.76	0.65	0.09	1	
											v1-v0	b	6	-1	-5	7.87	0.76	-0.1	-0.6	1	
											v2-v0	c	4	4	1	5.74	0.7	0.7	0.17	1	
											A		0.84								
	s	1	1	1							B		8.68								
											Det, D		28								
	d	1	0.5	-2		2.2913					beta	A/D	0.03								
		0.44	0.22	-0.9		1					alpha	B/D	0.31								
N	bxc	0.42	-0.6	0.62		0.9434						bu cv	24	l							
		0.45	-0.6	0.66		1						bv cu	-4	m	28	l-m = D					
				dominant is z, project on xy plane																	
	n.d	0.19	-0.1	-0.6		-0.5114						au cv	5.68	n							
												av cu	4.84	o	0.84	n-o = E					
	sP0	-1	-1	-1																	
												bu av	7.26	r							
	N.sP0	-0.4	0.61	-0.7		-0.4921						bv au	-1.4	s	8.68	r - s = F					
												alpha	0.03								
	t	0.96										beta F	0.31								

Ray - Object Intersection

- Ray – Sphere intersection
  - $\mathbf{c} = x_c\mathbf{I} + y_c\mathbf{J} + z_c\mathbf{K}$
  - $\boldsymbol{p} = x\mathbf{I} + y\mathbf{J} + z\mathbf{K}$
- For sphere
  - $|\mathbf{p} - \mathbf{c}| = R$
- If ray u intersects at P



$$|\mathbf{s} + t\mathbf{d} - \mathbf{c}|^2 = R^2$$

$$|\mathbf{s} + t\mathbf{d} - \mathbf{c}|^2 = (\mathbf{s} - \mathbf{c} + t\mathbf{d}).(\mathbf{s} - \mathbf{c} + t\mathbf{d})$$

$$\Rightarrow |\mathbf{s} - \mathbf{c}|^2 + 2t(\mathbf{s} - \mathbf{c}).\mathbf{d} + t^2 |\mathbf{d}|^2 = R^2$$

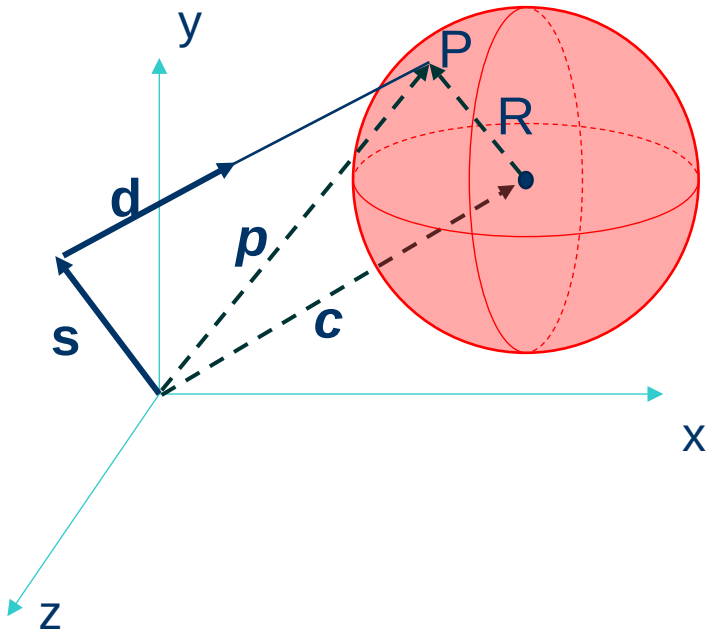
$$At^2 + Bt + C = 0$$

Where, $A \Rightarrow |\mathrm{d}|^2 = 1$; $B = 2(\mathrm{s} - \mathrm{c}).\mathrm{d}$; $C \Rightarrow |\mathrm{s} - \mathrm{c}|^2 - R^2$

Ray - Object Intersection

- **Ray – Sphere intersection**
  - The solution is

$$t = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A} = \frac{-B \pm \sqrt{B^2 - 4C}}{2}$$



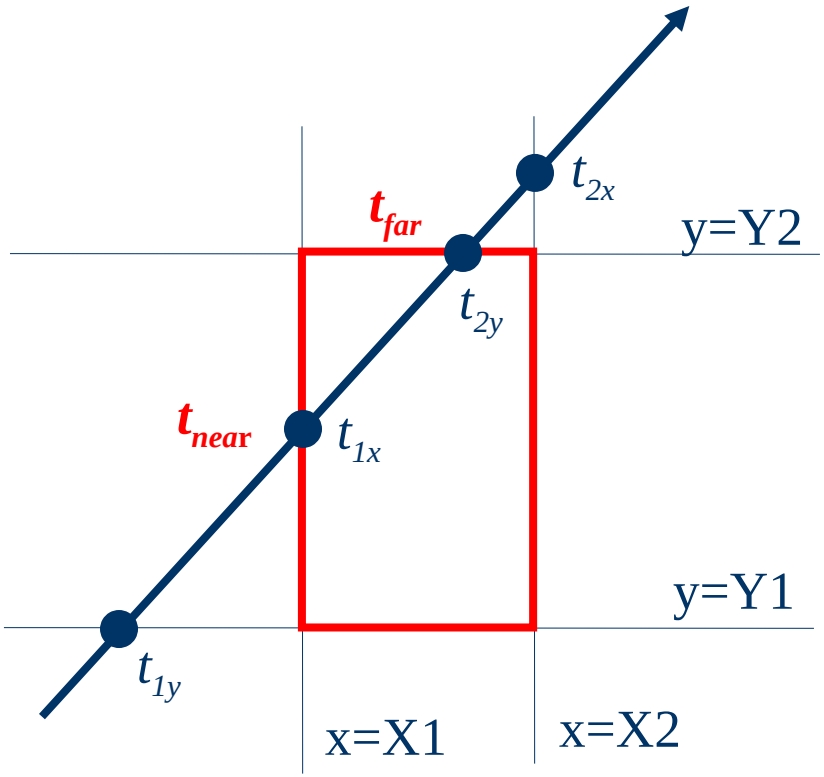
with

$$B^2 - 4C \begin{cases} < 0 & \text{no intersection} \\ = 0 & \text{ray touches sphere} \\ > 0 & \text{two (possible) intersection} \end{cases}$$

Check if t_0, $t_1 > 0$ (ray)

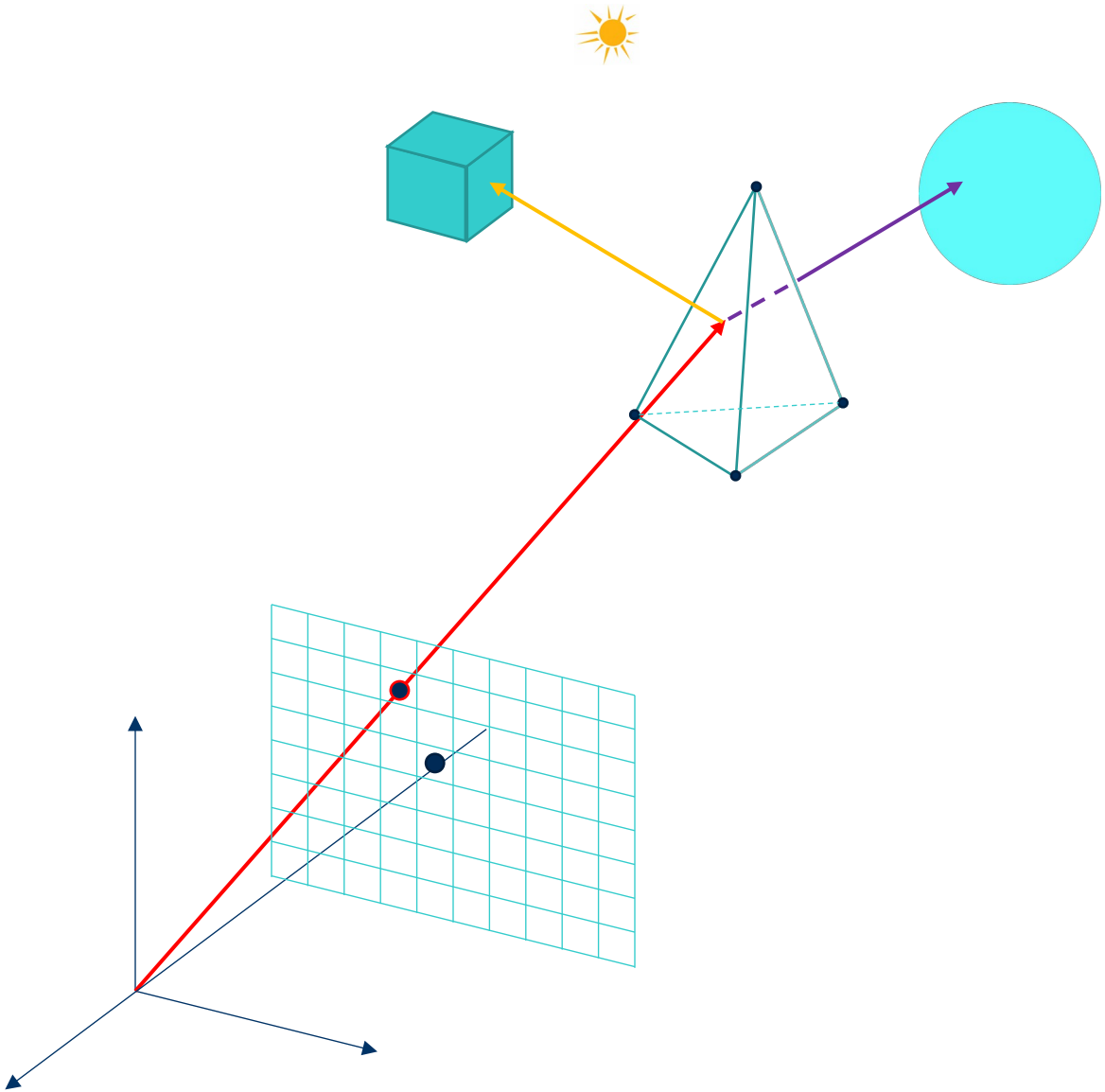
Return $\min(t_0, t_1)$

Intersection with Axis-Aligned Box

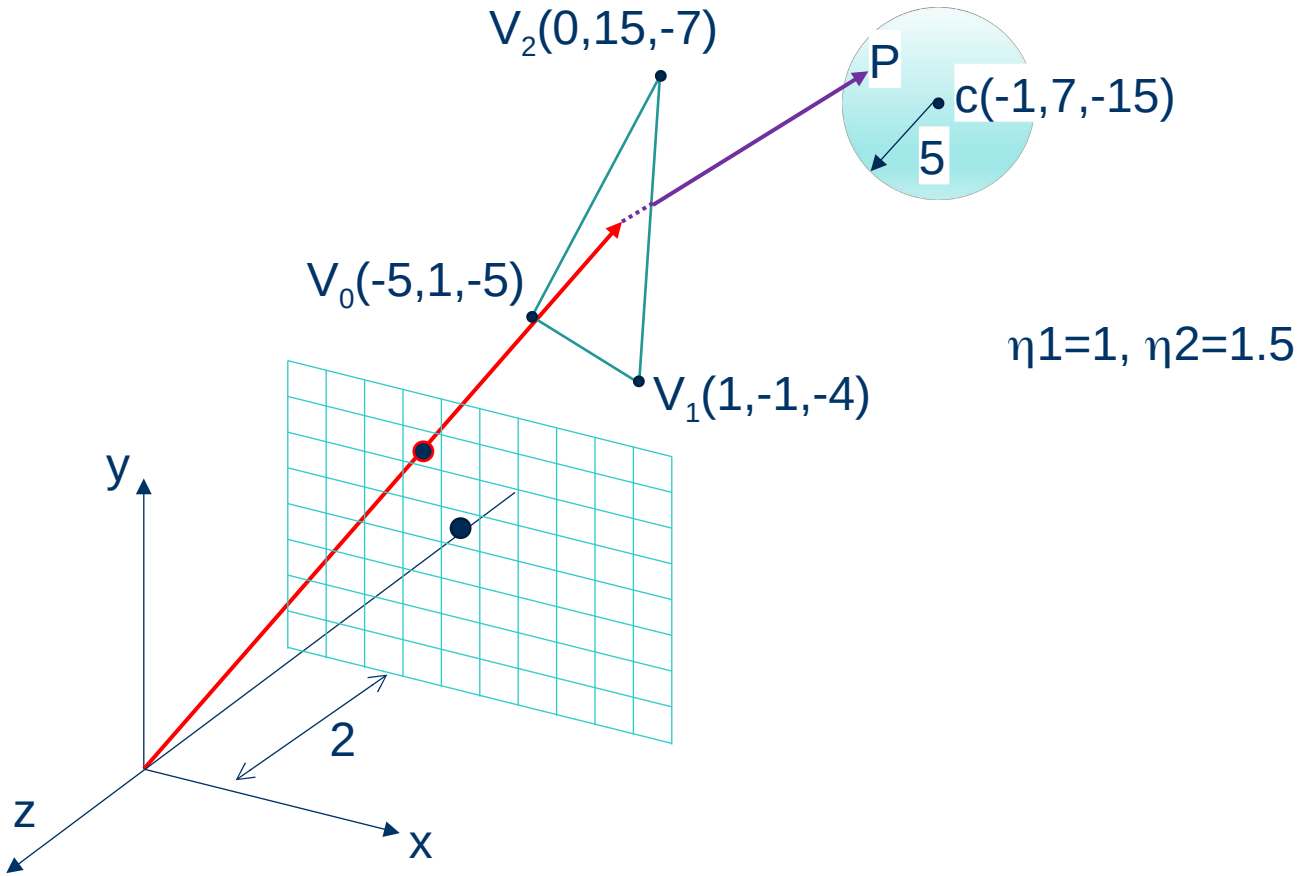


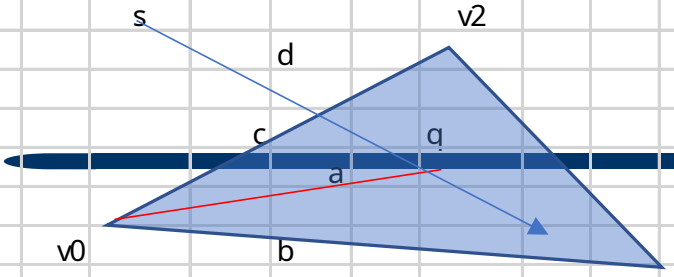
- For all 3 axes, calculate the intersection distances t_1 and t_2
- $t_{near} = \max(t_{1x}, t_{1y}, t_{1z})$
  $t_{far} = \min(t_{2x}, t_{2y}, t_{2z})$
- If $t_{near} > t_{far}$, box is missed
- If $t_{far} < t_{min}$, box is behind
- If box survived tests, report intersection at t_{near}

Find pixel intensity at (-1,2)

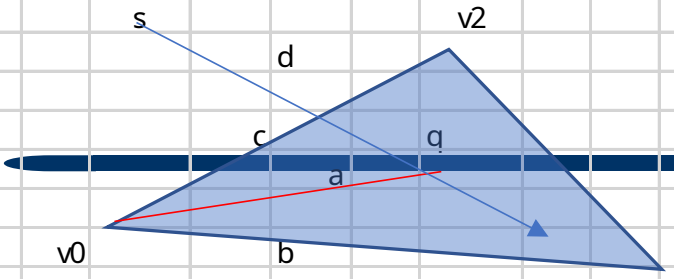


Find P




s
v2
d
c
q
a
v0
b
v1

| | | |


s
v2
d
c
q
a
v0
b
v1

| | | |

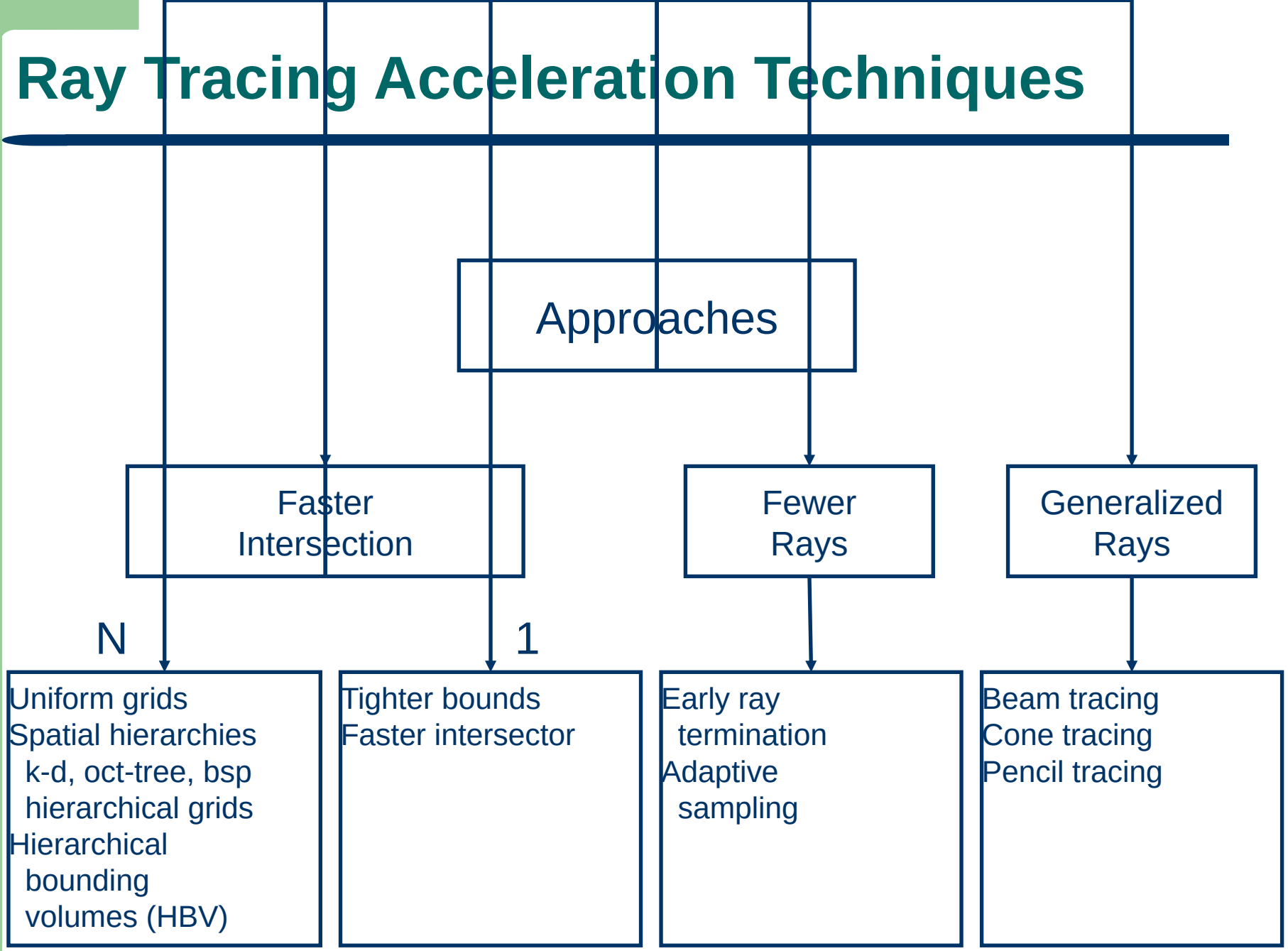
Ray Tracing Algorithm Analysis

- Ray casting
- Lots of primitives
- Recursive
- Distributed Ray Tracing Effects
  - Soft shadows
  - Anti-aliasing
  - Glossy reflection
  - Motion blur
  - Depth of field

cost ≈ height * width *
num primitives *
intersection cost *
size of recursive ray tree *
num shadow rays *
num supersamples *
num glossy rays *
num temporal samples *
num focal samples *
. . .

can we reduce this?

Ray Tracing Acceleration Techniques



Acceleration Structures

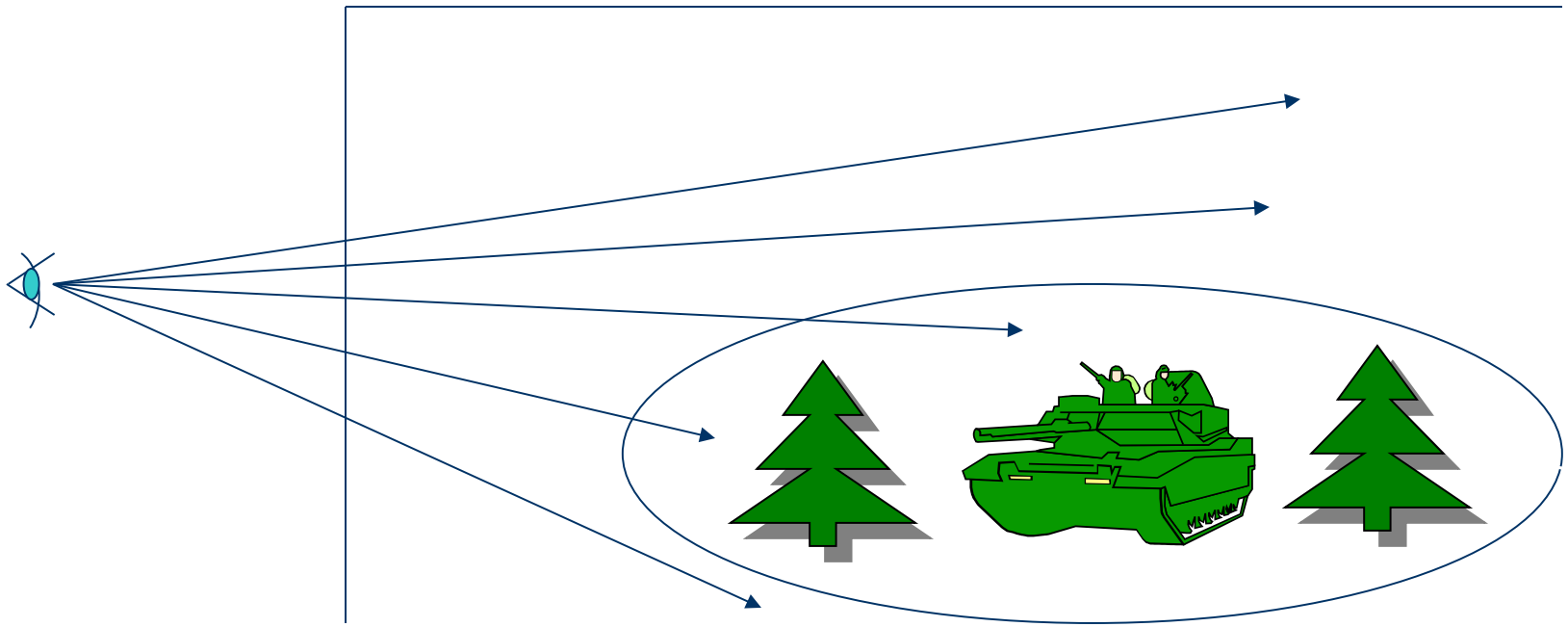
- Complex scenes can contain millions of primitives, and ray tracers need to trace millions of rays
- This means zillions of potential ray-object intersections
- If every ray simply looped through every object and tested if it intersected, we would spend forever just doing loops, not even counting all of the time doing the intersection testing
- Therefore, it is absolutely essential to employ some sort of *acceleration structure* to speed up the ray intersection testing
- An acceleration structure is some sort of data structure that groups objects together into some arrangement that enables the ray intersection to be sped up by limiting which objects are tested
- There are a variety of different acceleration structures in use, but most of the successful ones tend to be based on some variation of hierarchical subdivision of the space around the group of objects

Bounding Volume Hierarchies

- The basic concept of a bounding volume hierarchy is a complex object in a hierarchy of simpler ones
- This works much like the hierarchical
- For example, if one were using spheres as their bounding volume, we could enclose the entire scene in one big sphere
- Within that sphere are several other spheres, each containing more spheres, until we finally get to the bottom level where spheres contain actual geometry like triangles
- To test a ray against the scene, we traverse the hierarchy from the top level
- When a sphere is hit, we test the spheres it contains, and ultimately the triangles/primitives within
- In general, a bounding volume hierarchy can reduce the ray intersection time from O(n) to O(log n), where n is the number of primitives in the scene
- This reduction from linear to logarithmic performance makes a huge difference and makes it possible to construct scenes with millions of primitives

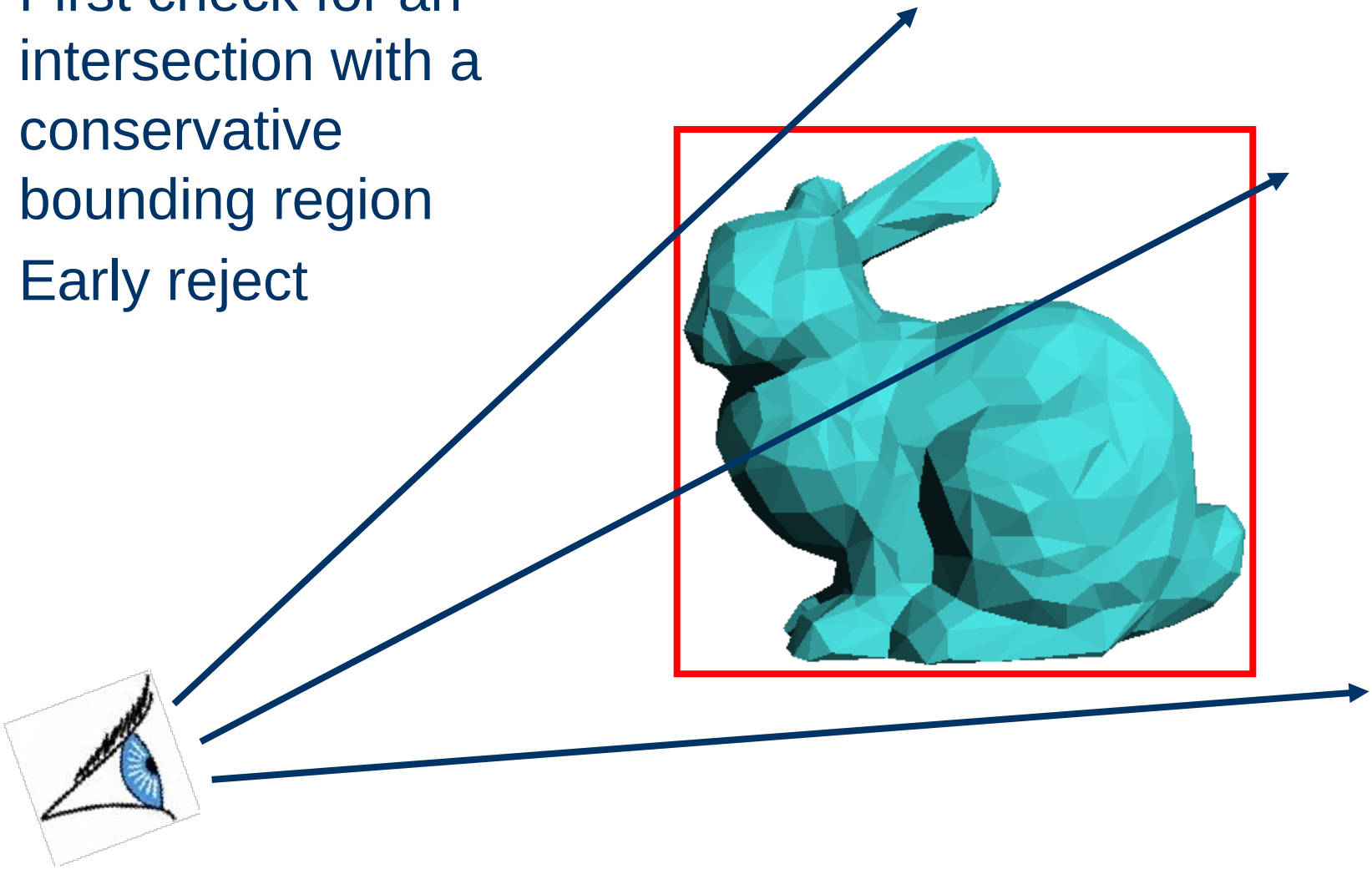
Bounding Volume

Enclosing Groups of objects in sets of bounding volumes and first test for intersection with the bounding volume.



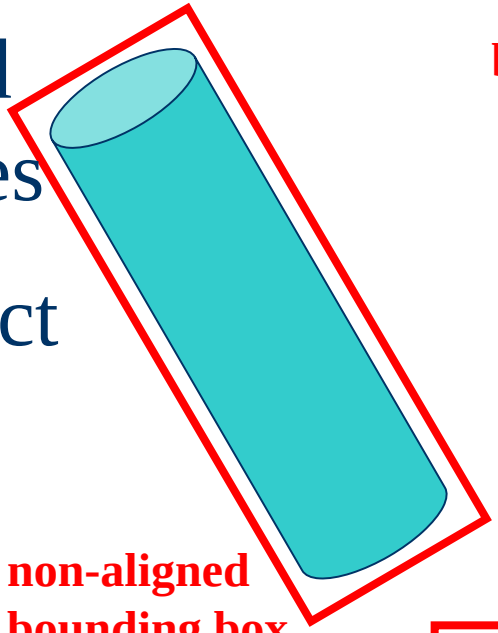
Conservative Bounding Region

- First check for an intersection with a conservative bounding region
- Early reject



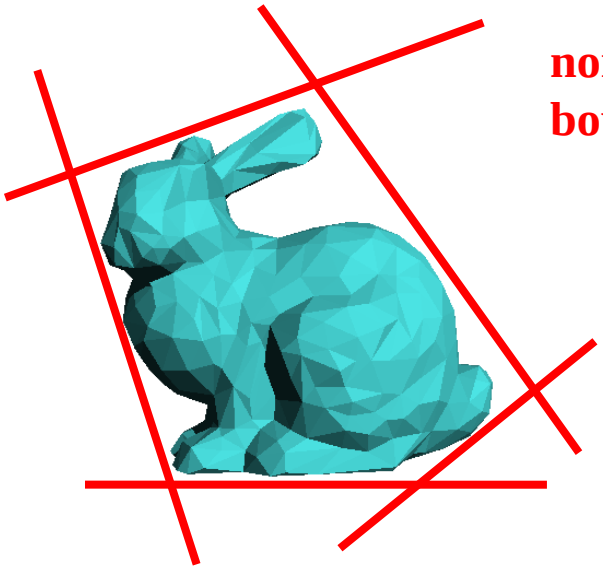
Conservative Bounding Regions

- tight → avoid false positives
- fast to intersect

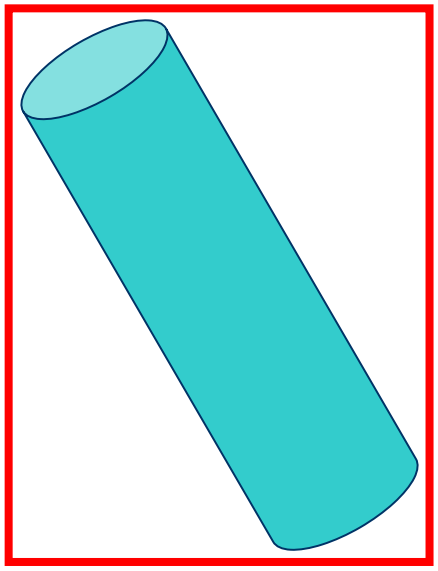


non-aligned bounding box

bounding sphere



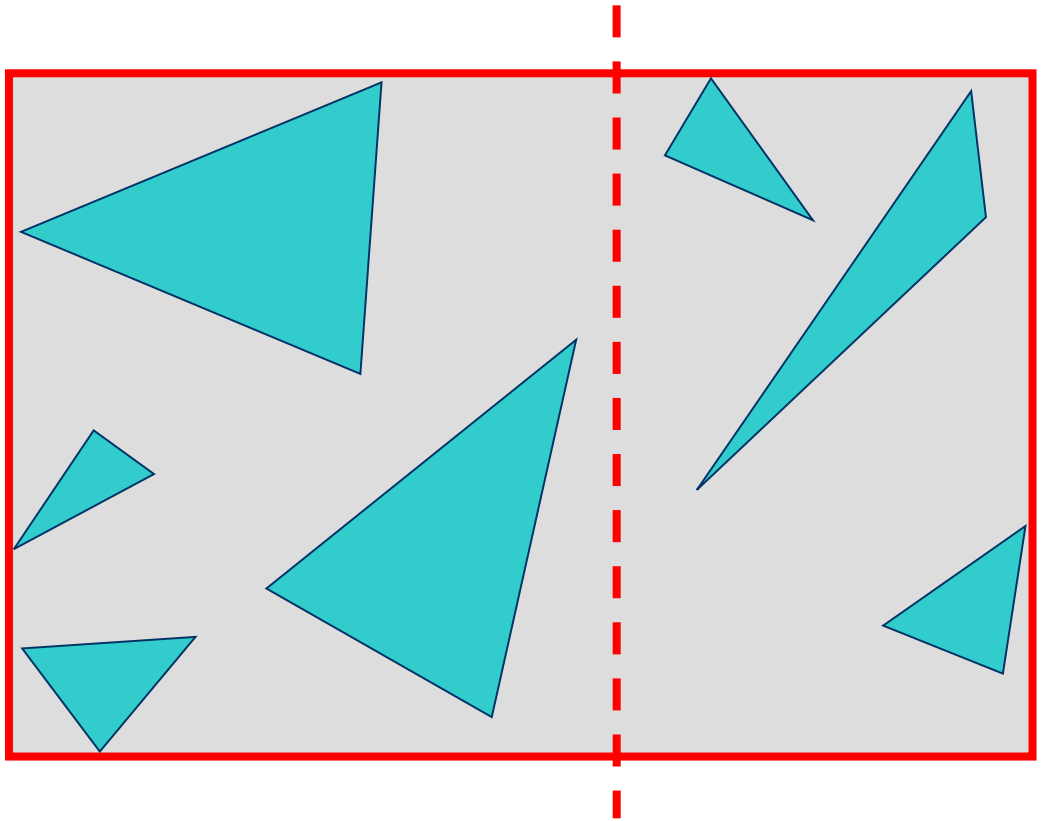
arbitrary convex region (bounding half-spaces)



axis-aligned bounding box

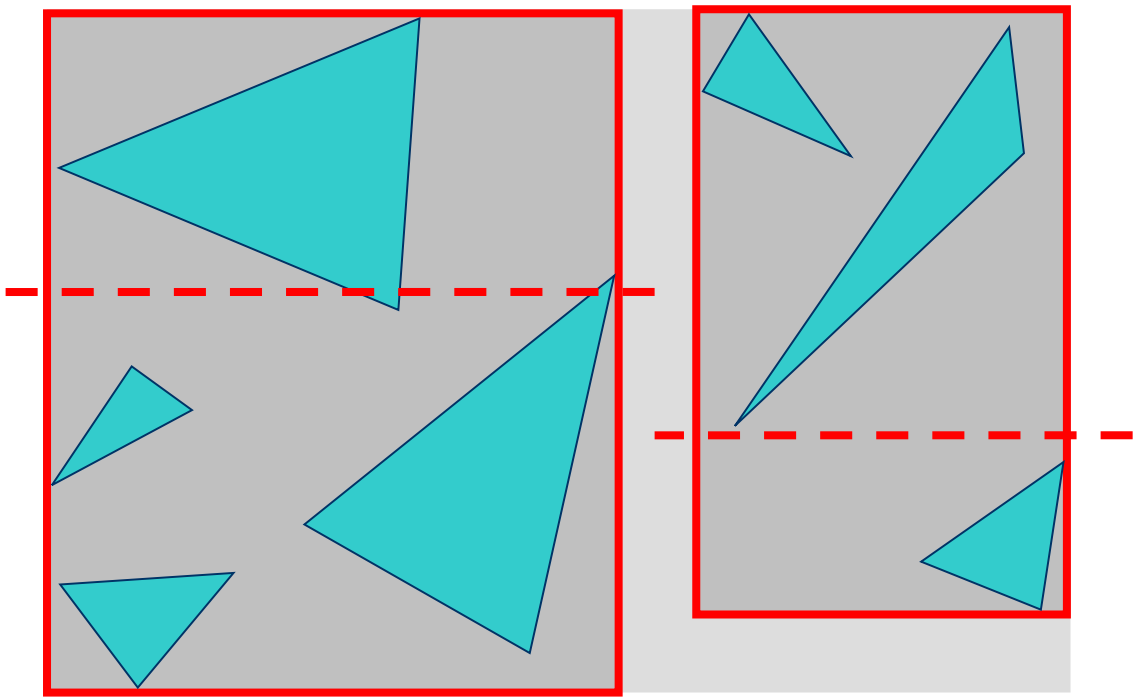
Bounding Volume Hierarchy

- Find bounding box of objects
- Split objects into two groups
- Recurse



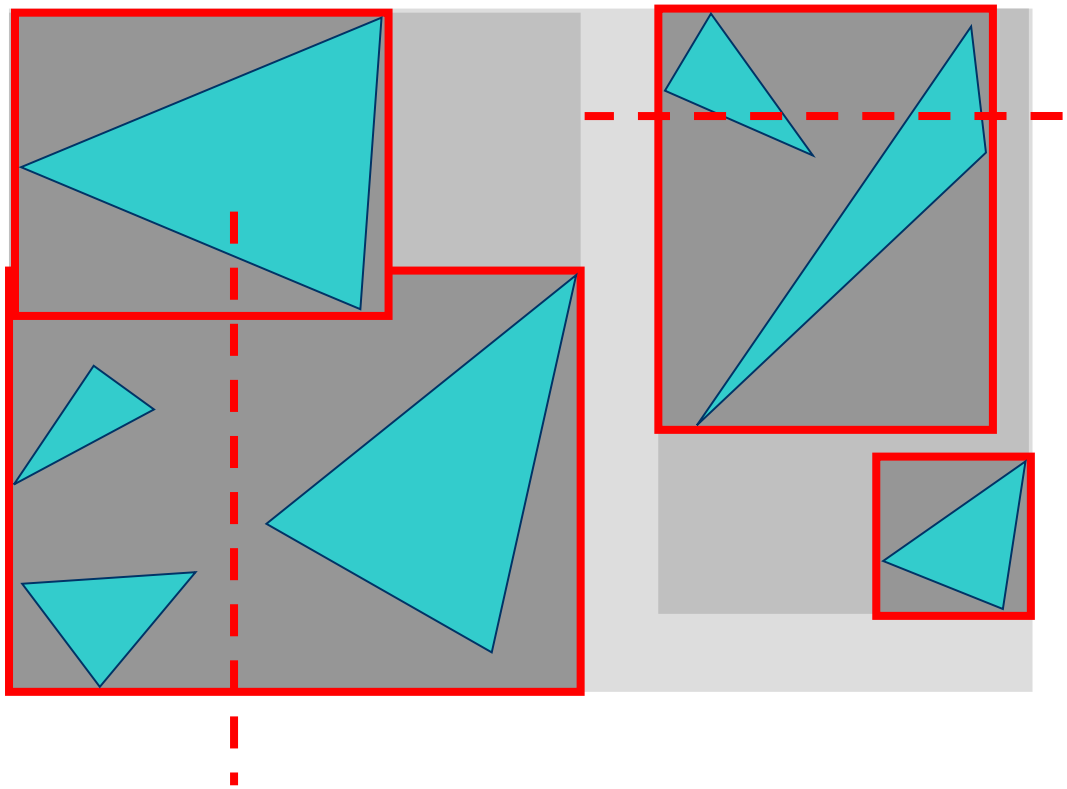
Bounding Volume Hierarchy

- Find bounding box of objects
- Split objects into two groups
- Recurse



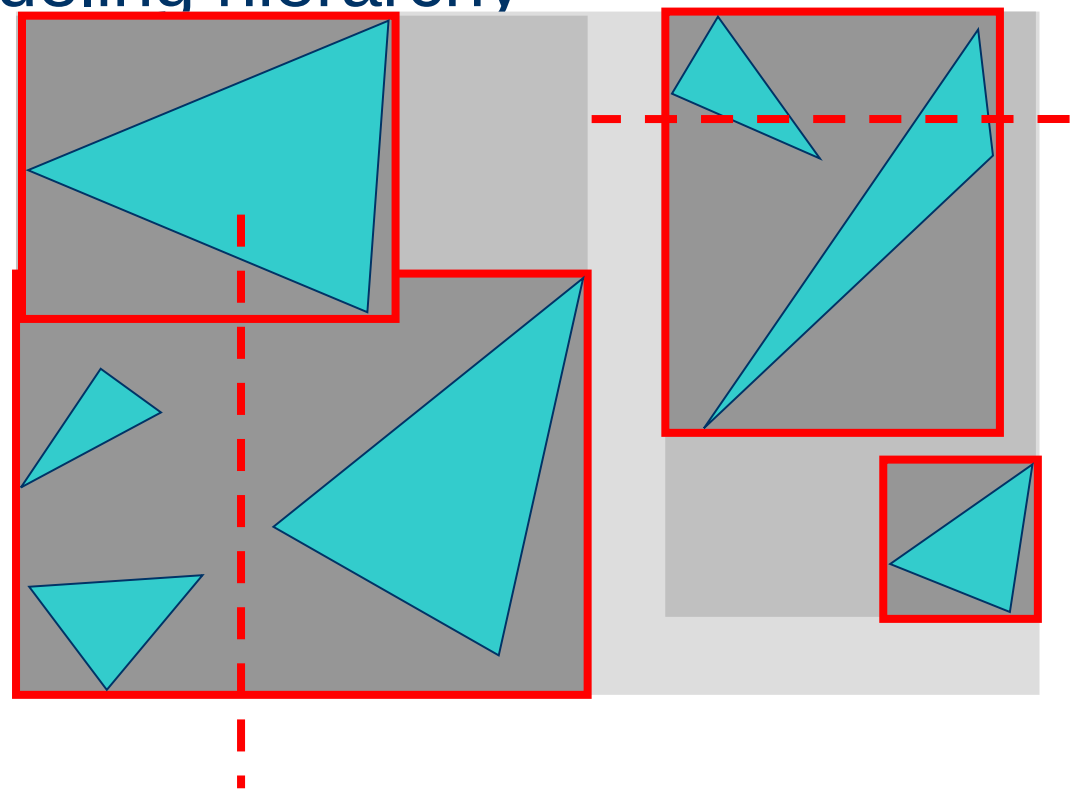
Bounding Volume Hierarchy

- Find bounding box of objects
- Split objects into two groups
- Recurse



Where to split objects?

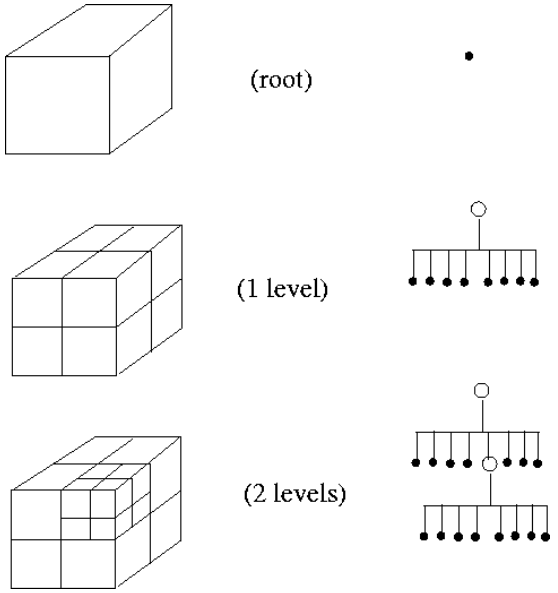
- At midpoint *OR*
- Sort, and put half of the objects on each side *OR*
- Use modeling hierarchy

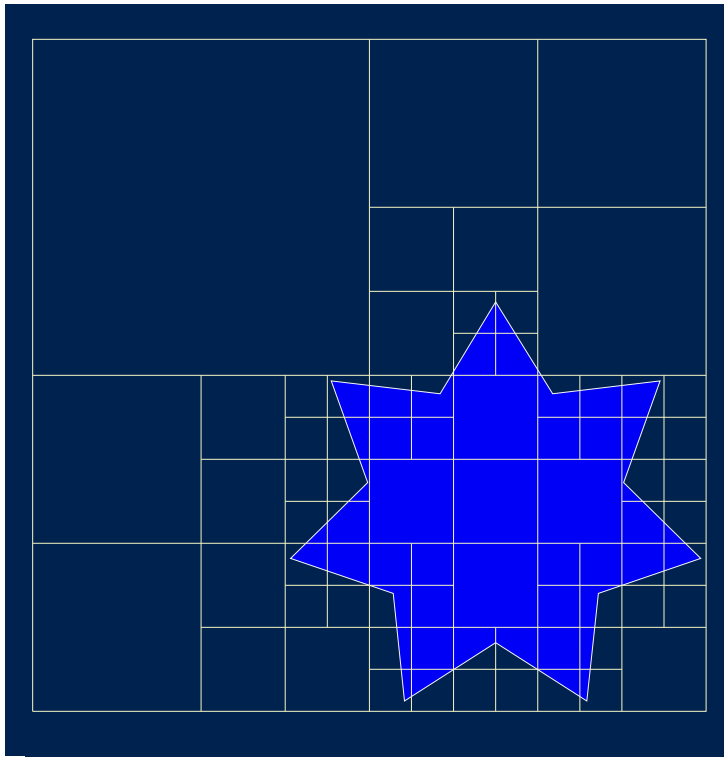


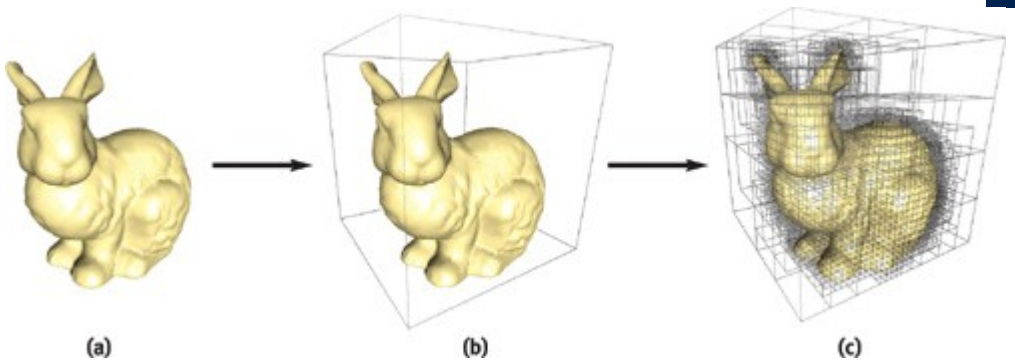
Octrees

- The octree starts by placing a cube around the entire scene
- If the cube contains more than some specified number of primitives (say, 10), then it is split equally into 8 cubes, which are then recursively tested and possibly resplit
- The octree is a more regular structure than the sphere tree and provides a clear rule for subdivision and no overlap between cells
- This makes it a better choice usually, but still not ideal

Octrees



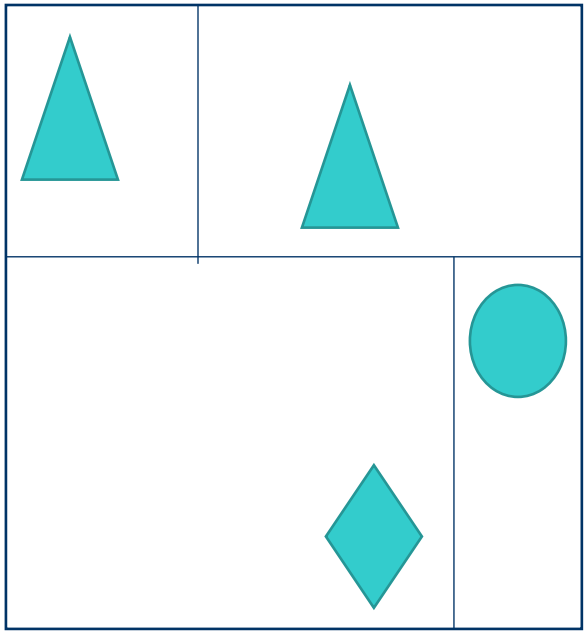


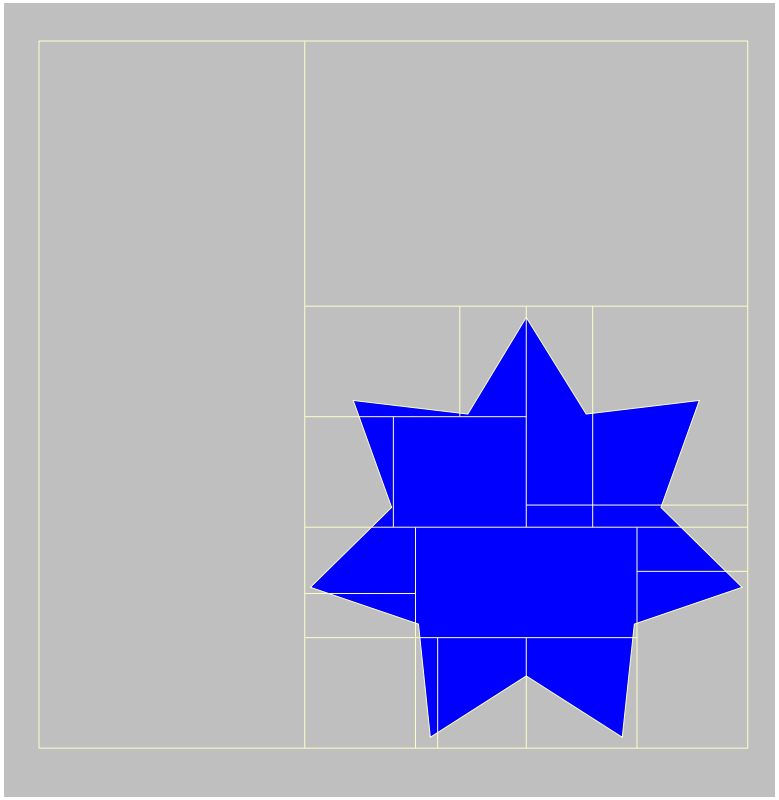


KD Trees

- The KD tree starts by placing a box (not necessarily a cube) around the entire scene
- If the box contains too many primitives, it is split, as with the octree
- However, the KD tree only splits the box into two boxes, that need not be equal
- The split can take place on the x, y, or z place at some arbitrary point within the box
- This makes the KD tree a little bit more adaptable to irregular geometry and able to customize a tighter fit
- In general, KD trees tend to be pretty good for ray tracing
- Their main drawback is that the tree depth can get rather deep, causing the ray intersection to spend a lot of time traversing the tree itself, rather than testing intersections with primitives

KD Trees

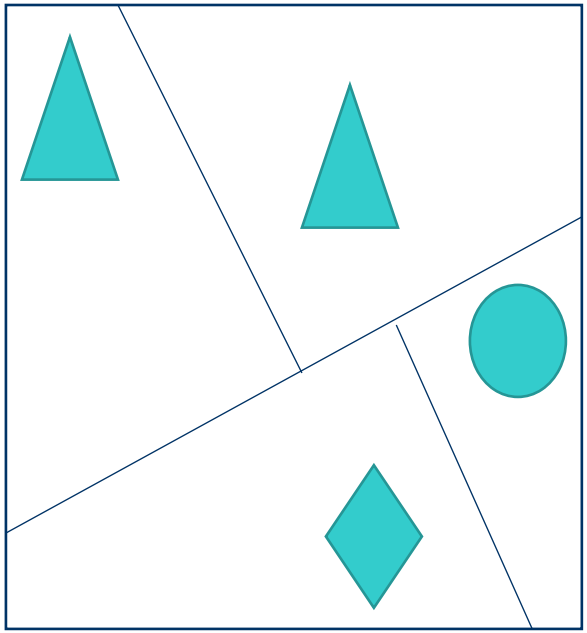


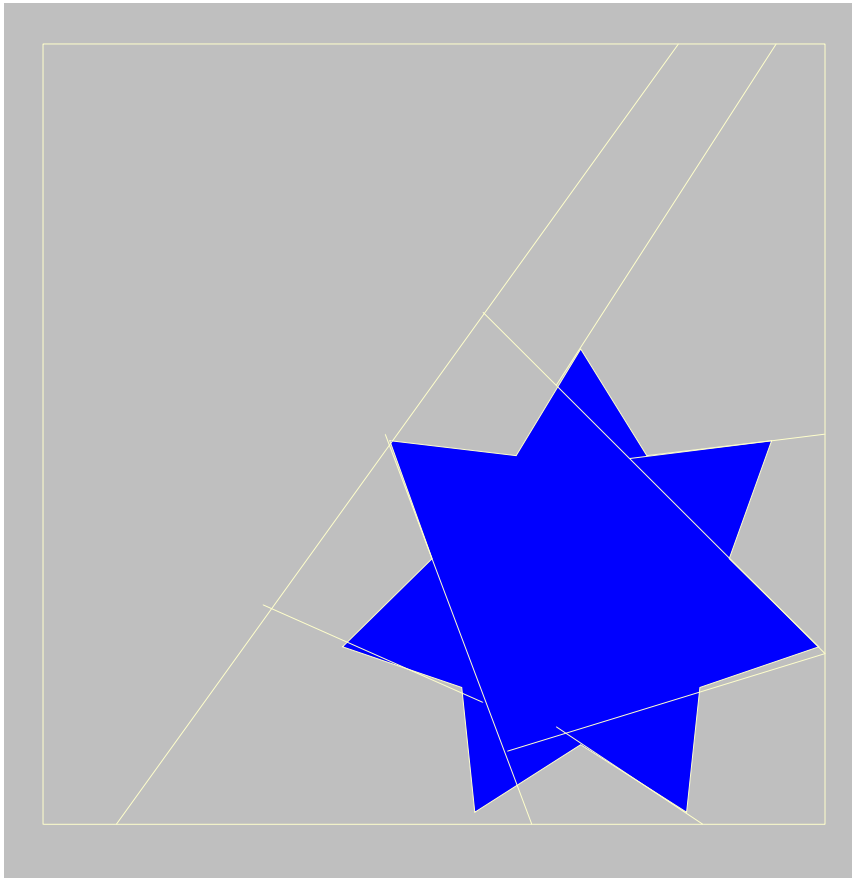


BSP Trees

- The BSP tree (*binary space partitioning*) is much like the KD tree in that it continually splits space into two (not necessarily equal) halves
- Unlike the KD tree which is limited to xyz axis splitting, the BSP tree allows the splitting plane to be placed anywhere in the volume and aligned in any direction
- This makes it a much more difficult problem to choose the location of the splitting plane, and so many heuristics exist
- In practice, BSP trees tend to perform well for ray tracing, much like KD trees

BSP Trees





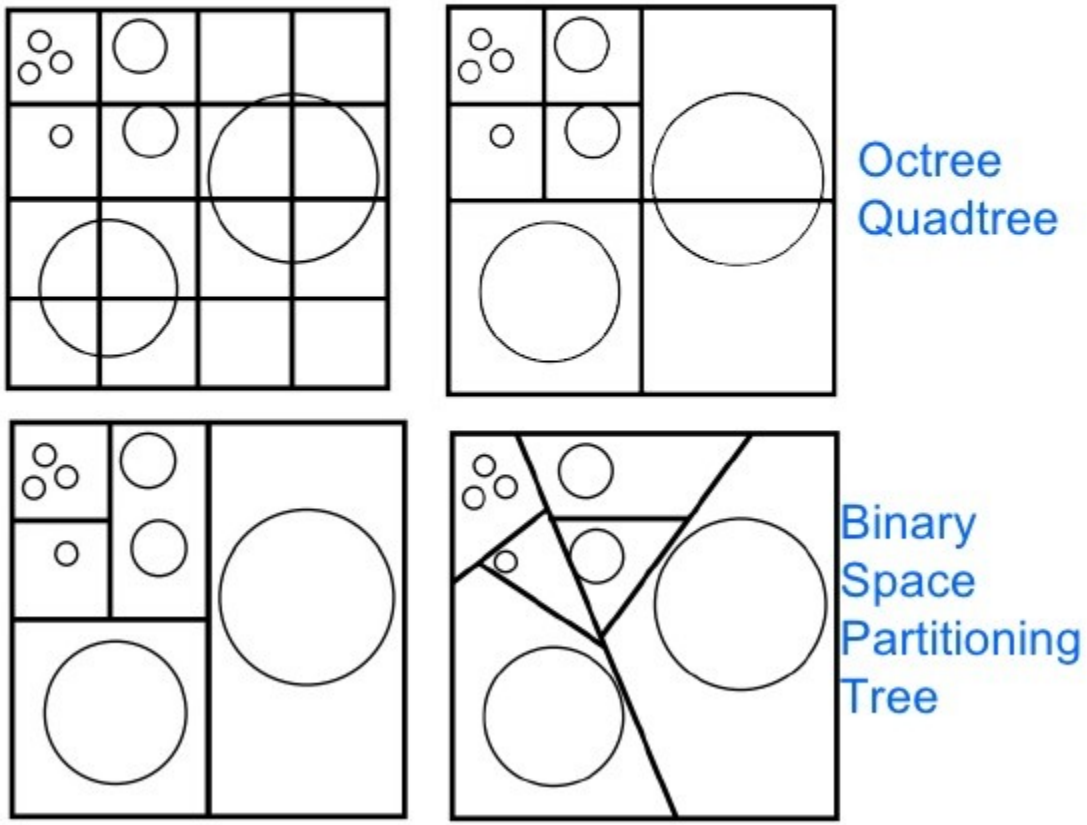
Uniform Grids

- One can also subdivide space into a uniform grid, instead of hierarchically
- This is fast for certain situations, but gets too expensive in terms of memory for large complex scenes
- It also tends to loose its performance advantages in situations where primitives have a large variance in size and location (which is common)
- As a result, they are not really a practical general purpose acceleration structure for ray tracing

Uniform Grids

CS 354 29

Space Subdivision Approaches



Distribution Ray Tracing

Distribution Ray Tracing

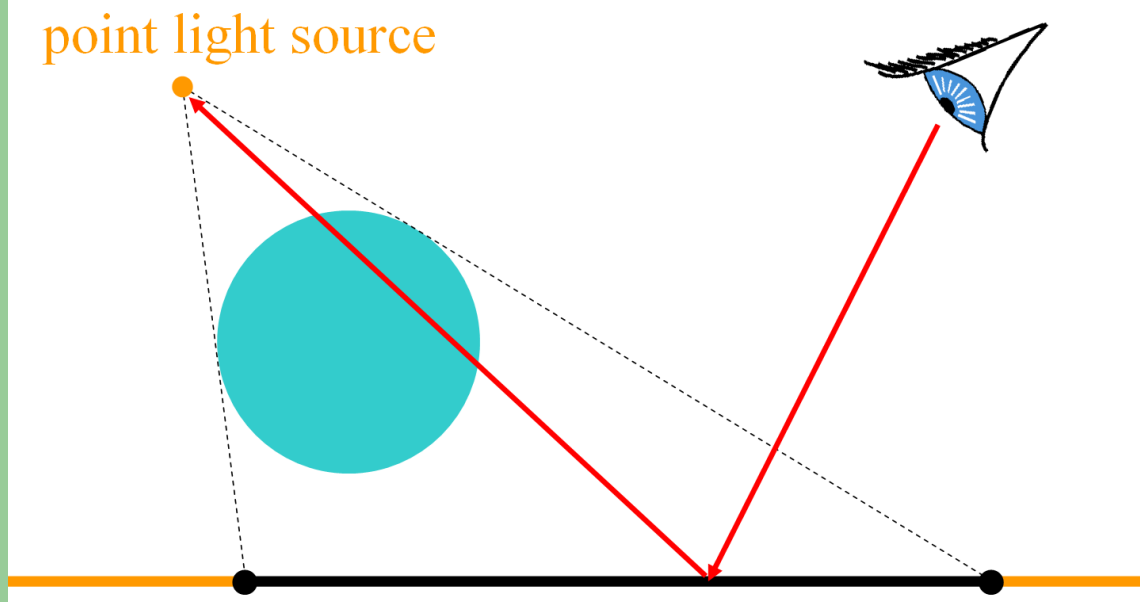
- In 1984, an important modification to the basic ray tracing algorithm was proposed, known as *distributed ray tracing*
- The concept basically involved shooting several distributed rays to achieve what had previously been done with a single ray
- The goal is not to simply make the rendering slower, but to achieve a variety of ‘soft’ lighting effects such as antialiasing, camera focus, soft-edge shadows, blurry reflections, color separation, motion blur, and more
- As the term ‘distributed’ tends to refer to parallel processing in modern days, the distributed ray tracing technique is now called *distribution ray tracing*, and the term ‘distributed’ is reserved for parallel ray tracing, which is also an important subject

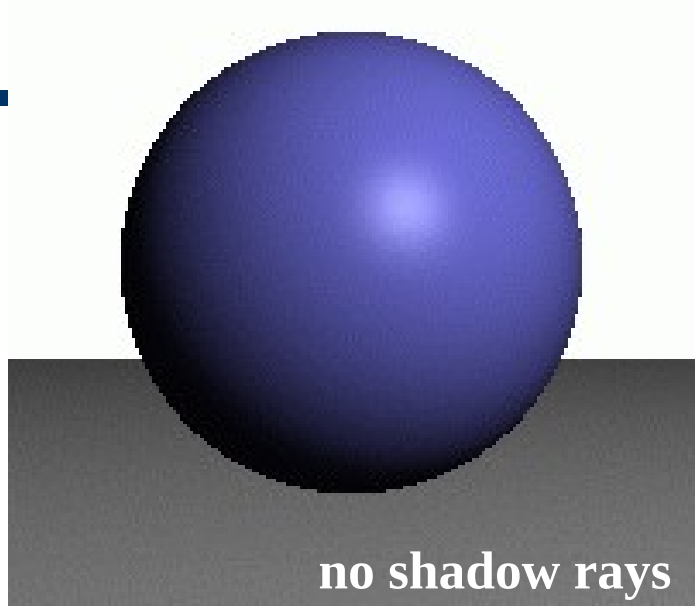
Soft Shadows

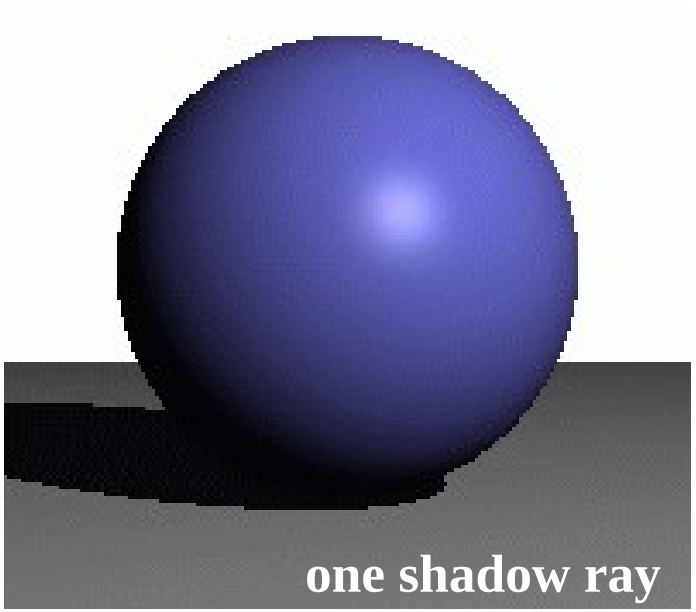
- One nice visual effect we can achieve with distribution ray tracing is *soft shadows*
- Instead of treating a light source as a point and shooting a single ray to test for shadows, we can treat the light source as a sphere and shoot several rays to test for partial blocking of the light source
- If 15% of the shadow rays are blocked, then we get 85% of the incident light from the light source
- In lighting terminology, the completely shadowed region is called the *umbra* and the partially shadowed region is called the *penumbra*

Shadows

- one shadow ray per intersection per point light source



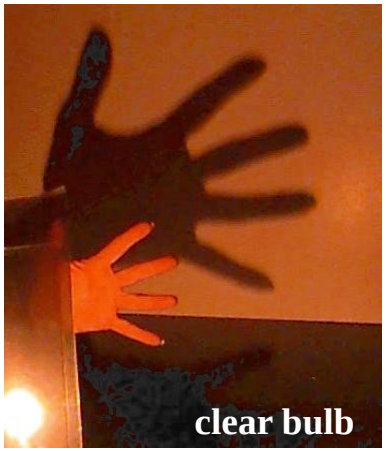


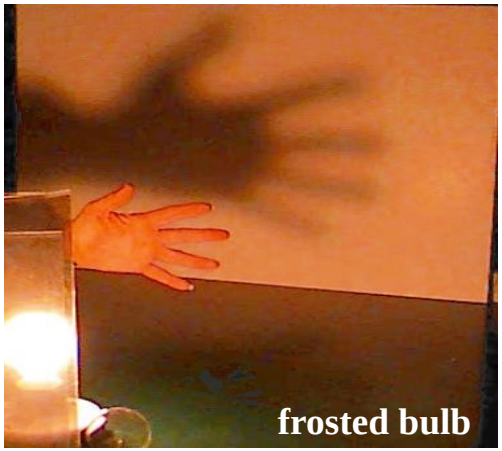


Shadows & Light Sources

http://3media.initialized.org/photos/2000-10-18/index_gall.htm

http://www.davidfay.com/index.php





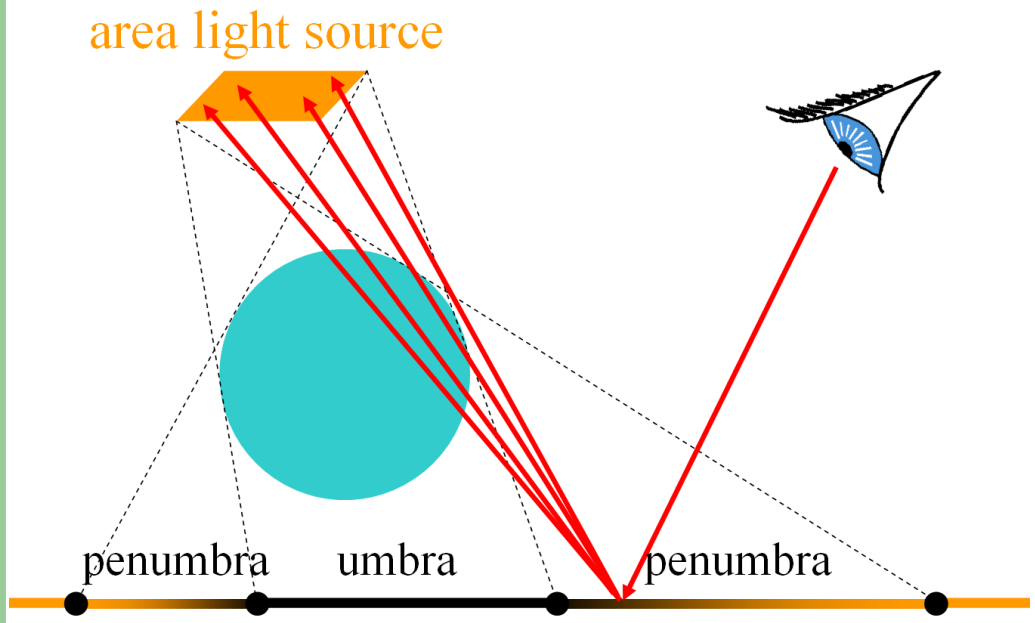
http://www.pa.uky.edu/~sciworks/light/preview/bulb2.htm

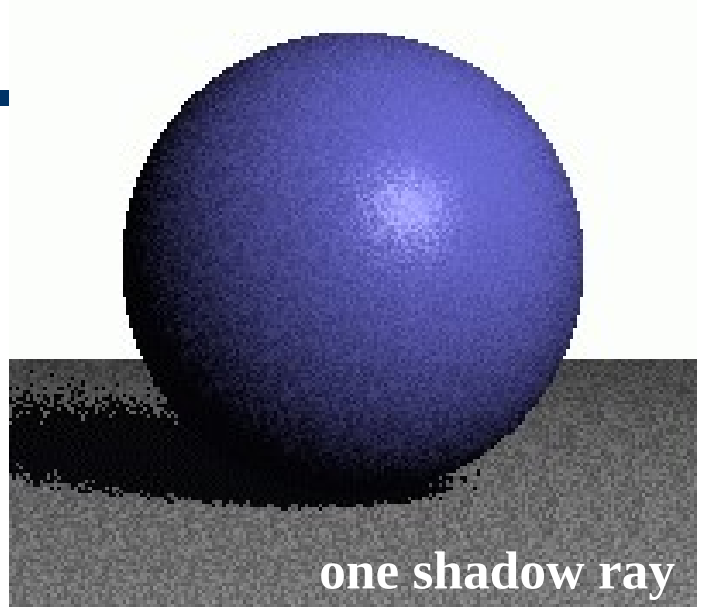
Area Lights

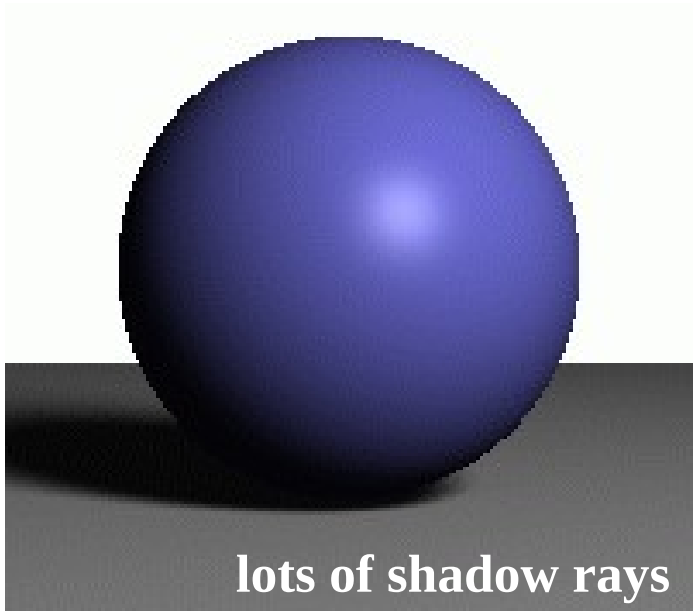
- The soft shadow technique enables us to define lights in a much more complex way than we have previously
- We can now use any geometry to define a light, including triangles, patches, spheres, etc.
- To determine the incident light, we shoot several rays towards the light source, distributed across the surface and weighted according to the surface area of the sample and the direction of the average normal
- Larger light sources create softer, diffuse shadows, while smaller light sources cause sharp, harsh shadows
- Larger light sources also require more rays to adequately sample the shadows, making area lights a lot more expensive than point lights. Inadequate sampling of the light source can cause noise patterns to appear in the penumbra region, known as *shadow aliasing*

Soft Shadows

- multiple shadow rays to sample area light source

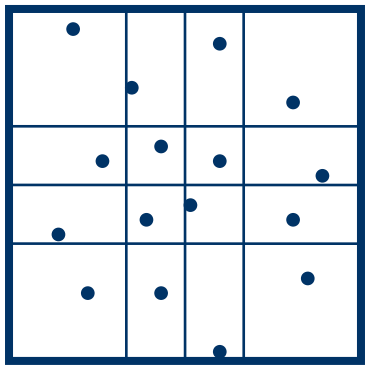






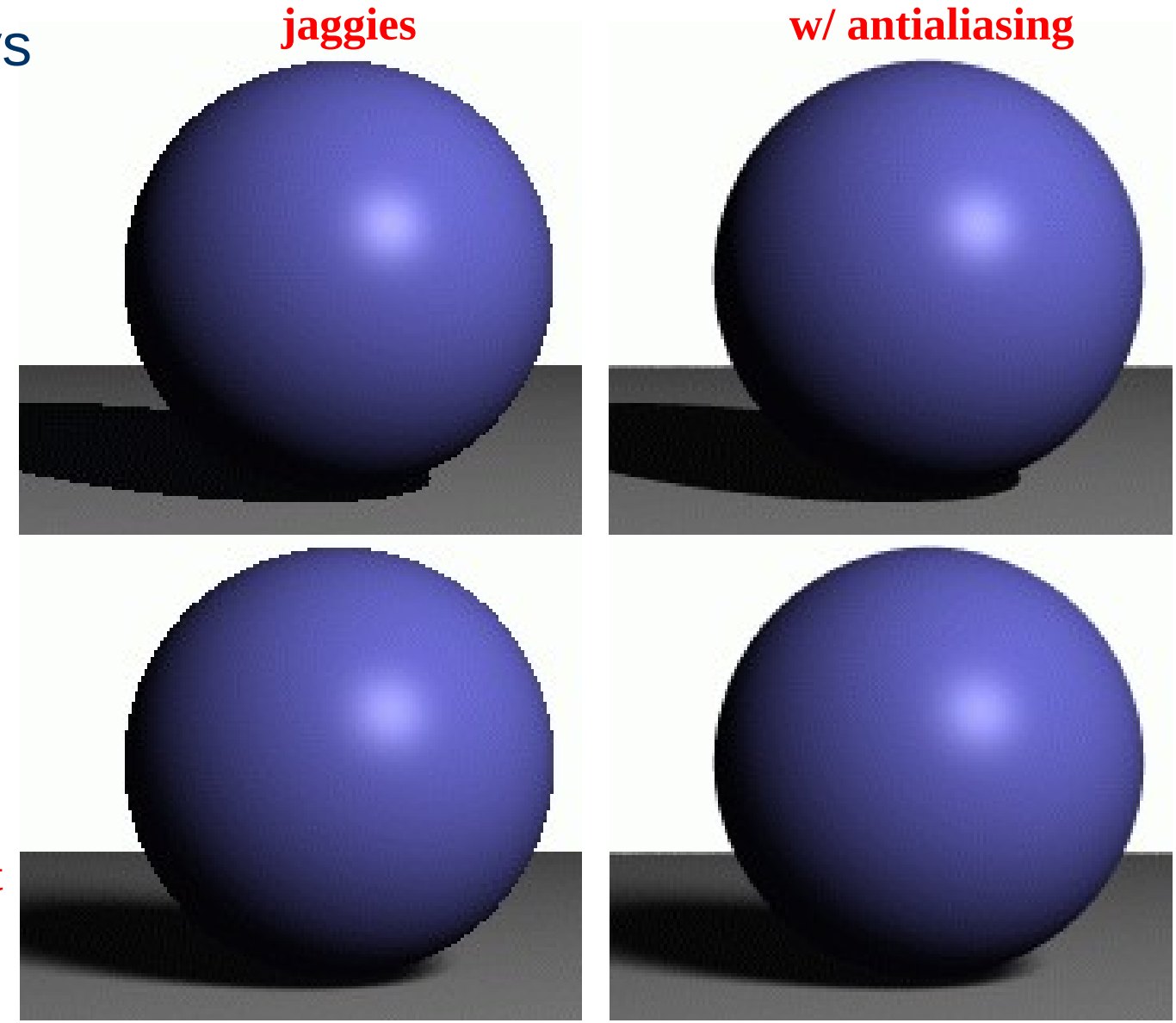
Antialiasing

- For good quality antialiasing, we will can shoot several primary rays for each pixel
- several different antialiasing, area-weighted, jittered Gaussian distribution was chosen as one of the best overall sampling schemes
- As we have complete control over the direction of every ray, we can use any sampling pattern we choose, as well as any number of rays we choose



Antialiasing – Supersampling

- multiple rays per pixel

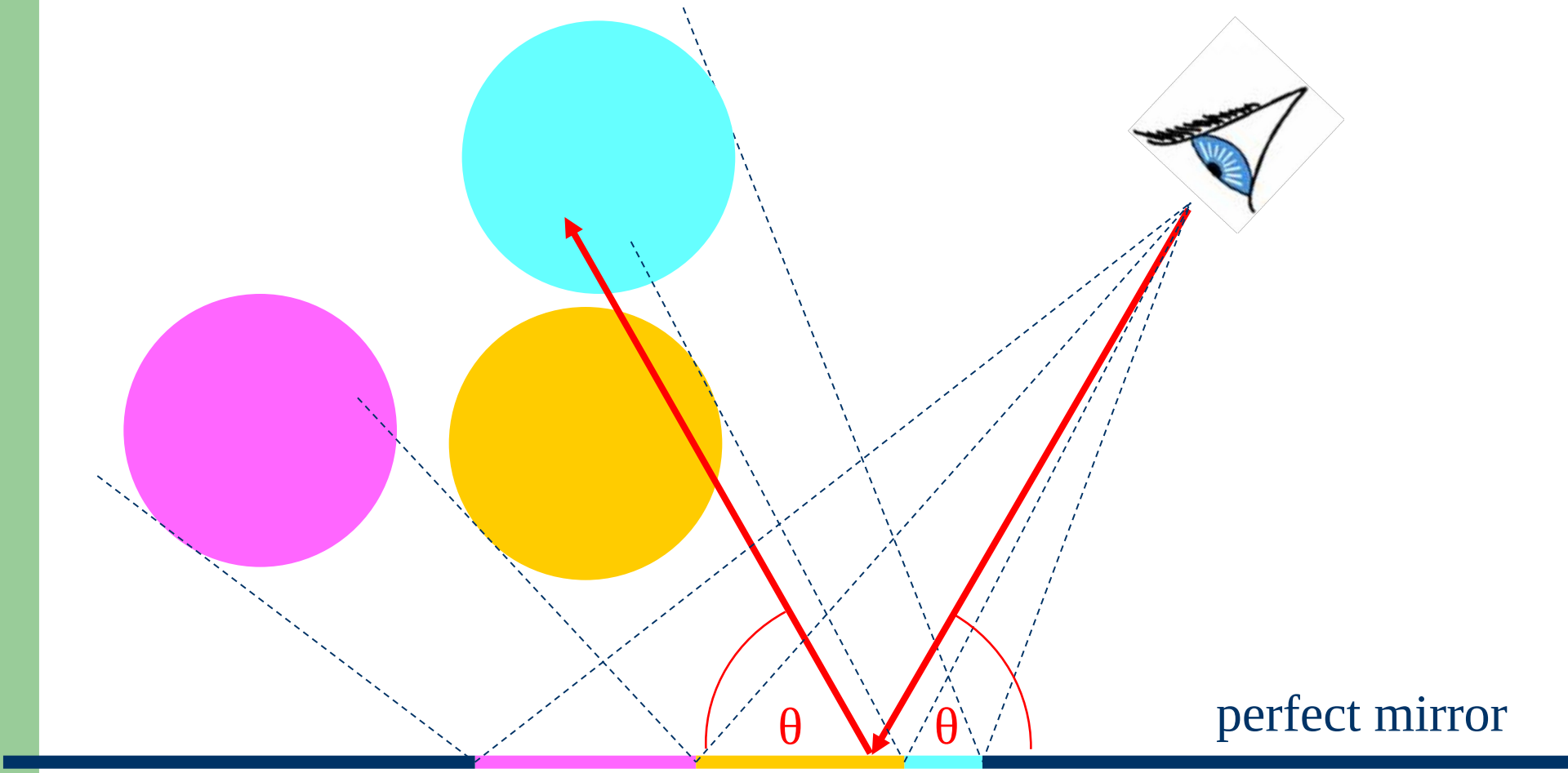


Blurry Reflections

- We can render blurry or *glossy* reflections by creating several reflection rays instead of just one
- The rays can be distributed around the ideal reflection direction
- Blurry surfaces will causes a wider distribution (and require more rays), while more polished surfaces will have a narrow distribution
- The same concept can apply to refraction in order to achieve rendering of unpolished glass

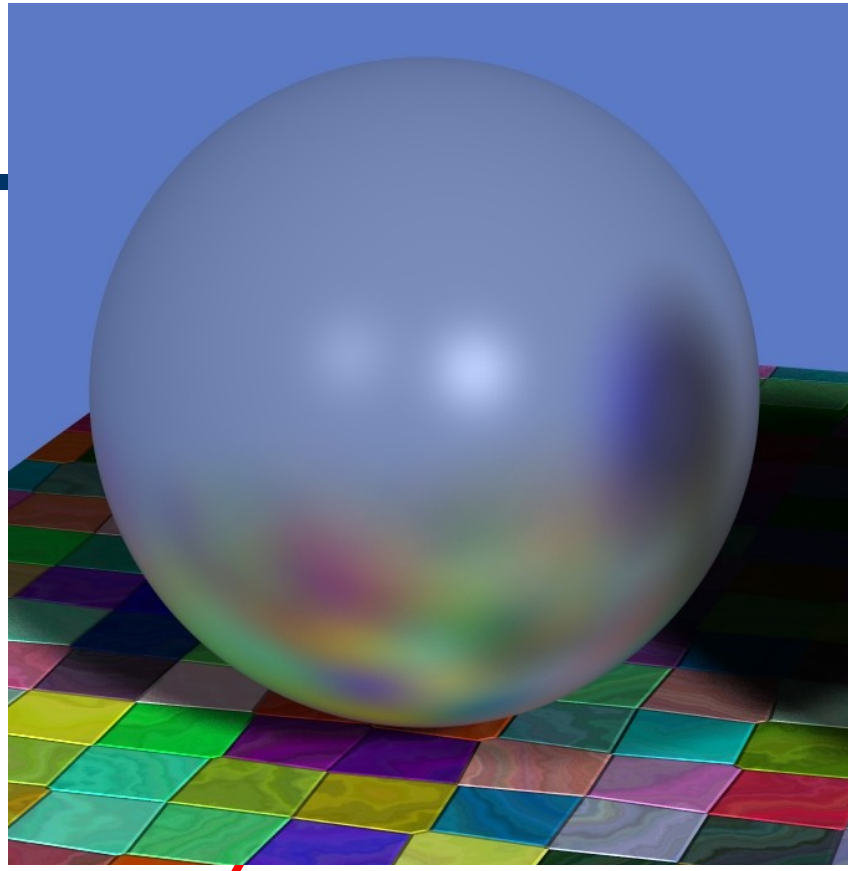
Reflection

- one reflection ray per intersection

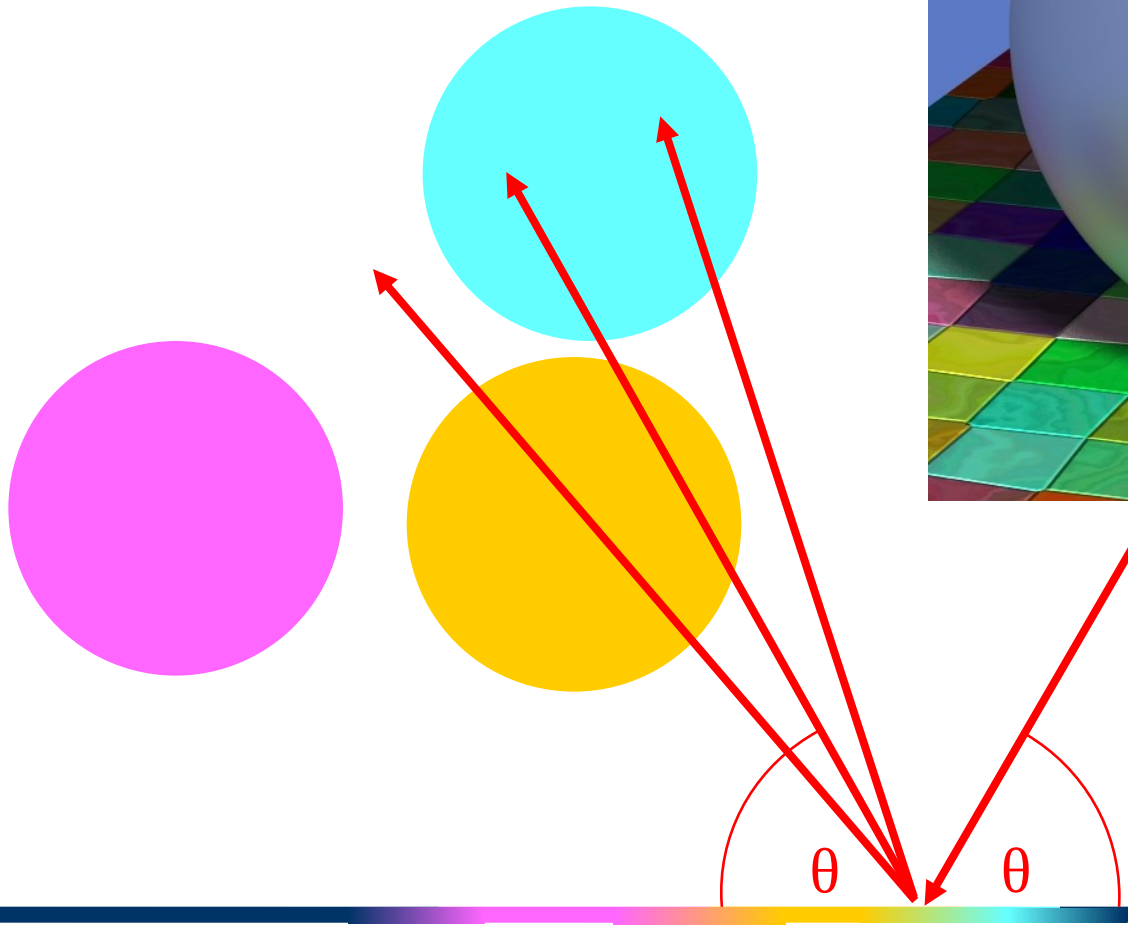


Glossy Reflection

- multiple reflection rays



Justin Legakis



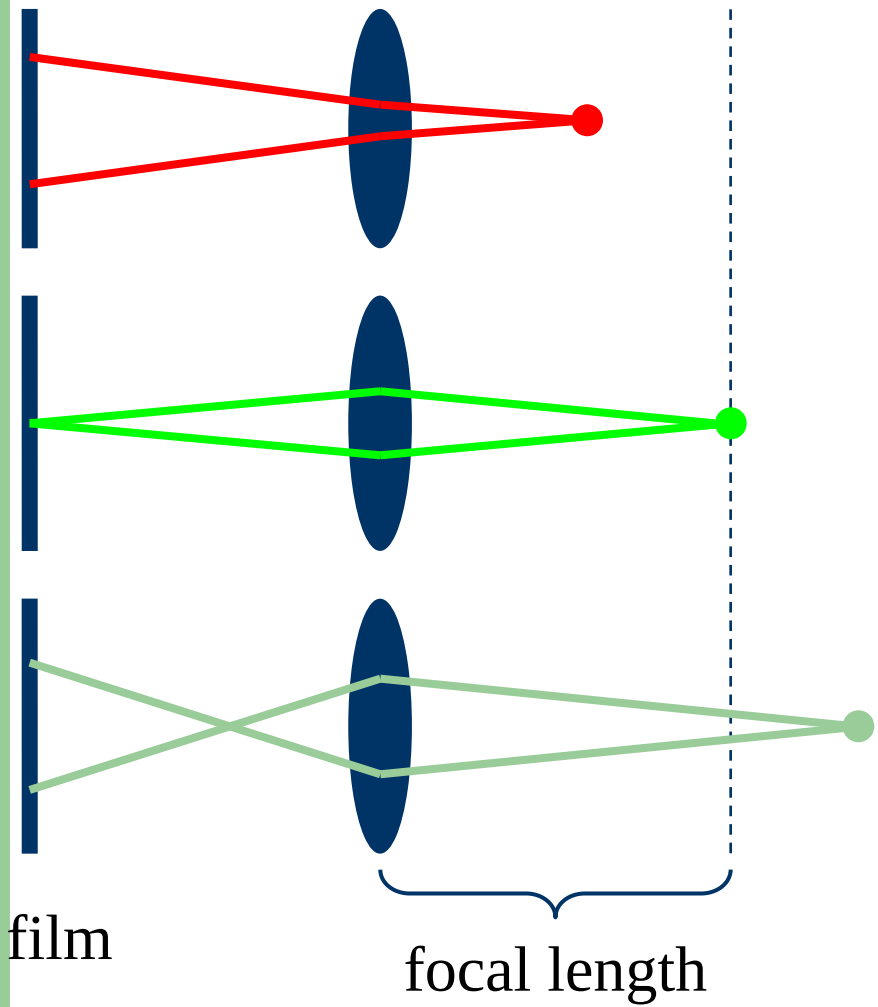
polished surface

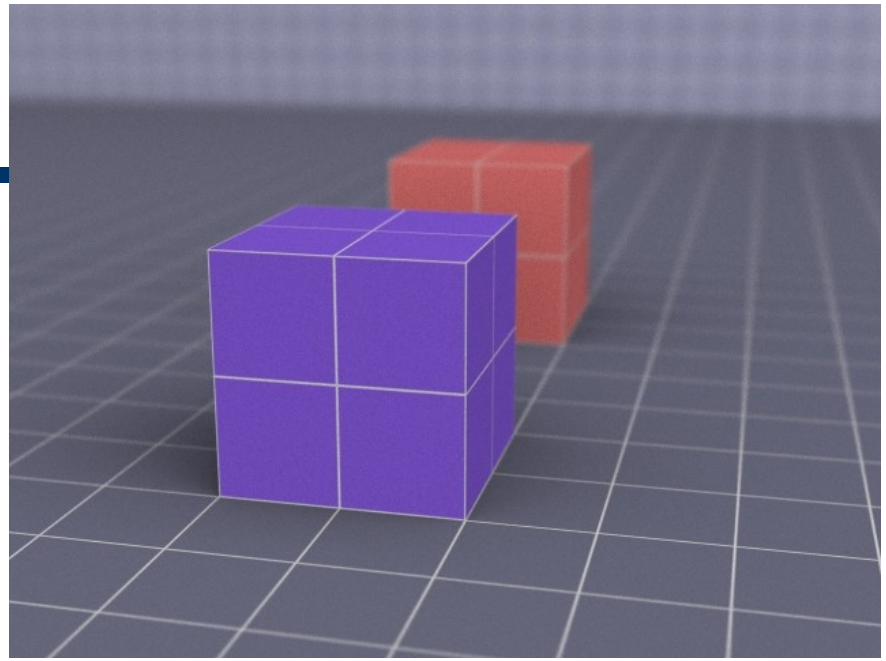
Depth of Field

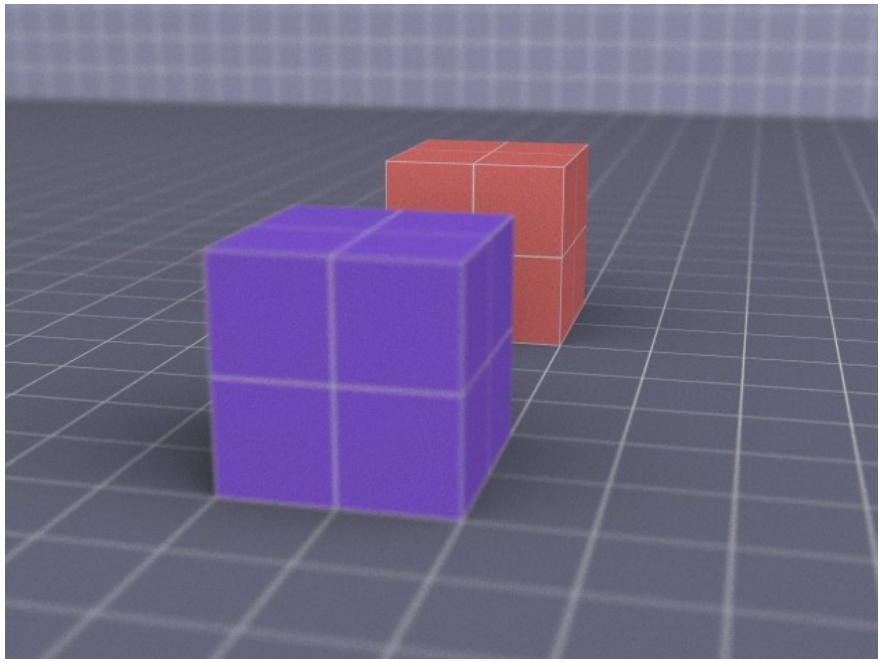
- The blurring caused by a camera lens being out of focus is due to the lens' limited *depth of field*
- In computer graphics, the term 'depth of field' usually refers to the general process of rendering images that include a camera blurring effect
- A lens will typically be set to focus on objects at some distance away, known as the *focal distance*
- Objects closer or farther than the focal distance will be blurry, and the blurriness increases with the distance to the *focal plane*
- Depth of field can be rendered with distribution ray tracing by distributing the primary rays shot from the camera
- Rays area distributed across a virtual *aperture*, which represents the (usually circular) opening of the lens
- The larger the aperture, the more pronounced the blurring effect will be. A *pinhole* camera has an aperture size of 0, and therefore, will not have any blurring due to depth of field

Depth of Field

- multiple rays per pixel







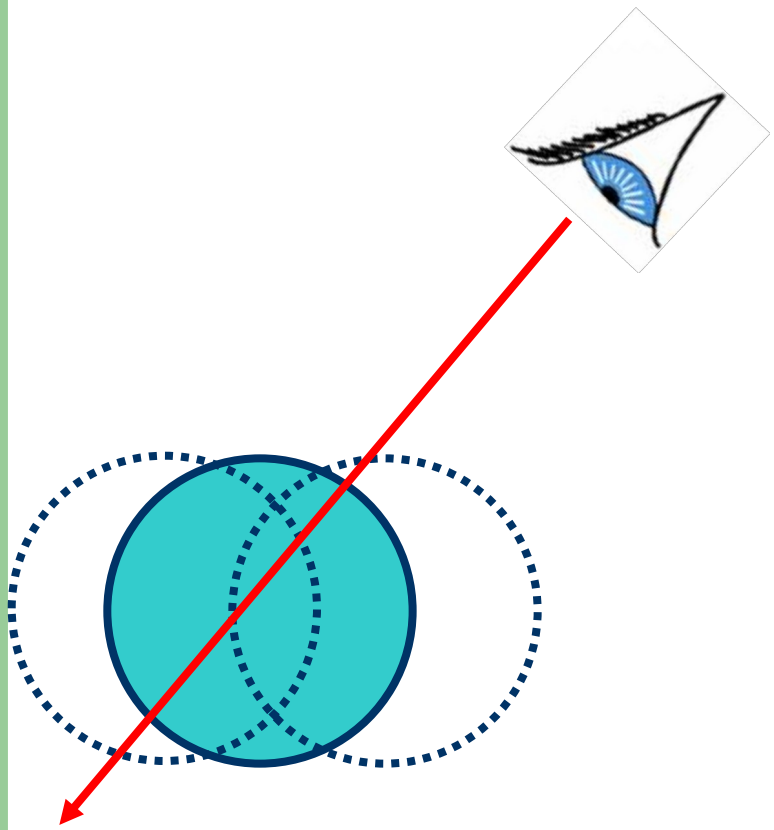
Justin Legakis

Motion Blur

- As if soft shadows, antialiasing, depth of field, and blurry reflections and refractions weren't enough… we can also use distribution ray tracing to include the effect from motion blur
- We do this by distributing rays in time
- Therefore, we need to know the starting and ending transformations for all of the objects in the scene (and the lights & camera)
- In fact, we can even consider objects moving along more complex paths, so we can motion blur bounces as well

Motion Blur

- Sample objects temporally



Rob Cook

Distribution Ray Tracing

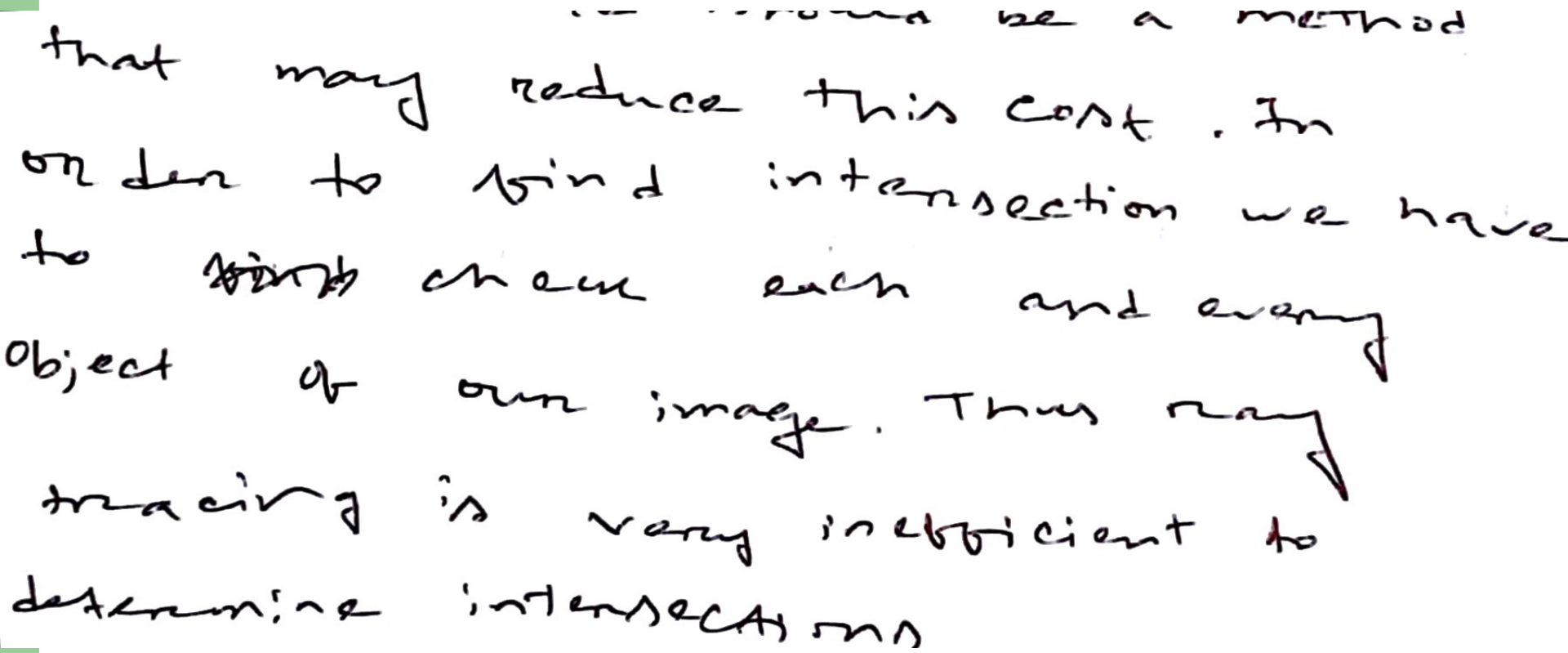
- Ray tracing had a big impact on computer graphics in 1980 with the first images of accurate reflections and refractions from curved surfaces
- Distribution ray tracing had an even bigger impact in 1984, as it re-affirmed the power of the basic ray tracing technique and added a whole bunch of sophisticated effects, all within a consistent framework
- Previously, techniques such as depth of field, motion blur, soft shadows, etc., had only been achieved individually and by using a variety of complex, hacky algorithms

Distribution Ray Tracing

- If ray tracing is slow, then distribution ray tracing must be considerably slower
- Now, instead of one or two splits per level in our recursion, we are have to shoot dozens or even hundreds of rays to achieve some of these effects
- This can cause an exponential expansion in the number of rays
- The good news is that we can combine these features so that we still only need to shoot a small number of primary rays per pixel
- For example, we can shoot 16 rays in a 4x4 antialiasing pattern, where each ray has a random distribution in time and in the camera aperture
- Each of these rays only needs to spawn a few reflection or shadow rays, as the results will be blended with 15 other samples
- Still, we end up with lots and lots of rays and potential for exponential problems in scenes with a lot of soft or blurry features
- This problem is at least partially addressed with *path tracing* which is one of the techniques for *global illumination* that we will see in the next lecture

c) In ray tracing a large computational cost is associated with determining ray-object intersections. Suggest a method that may reduce this cost. (10)

In ray tracing a large computational cost is associated with determining ray-object injections. It requires a lot of computational complexity to do ray tracing. Thus ray tracing is very much computationally inefficient and thus there should be a method that

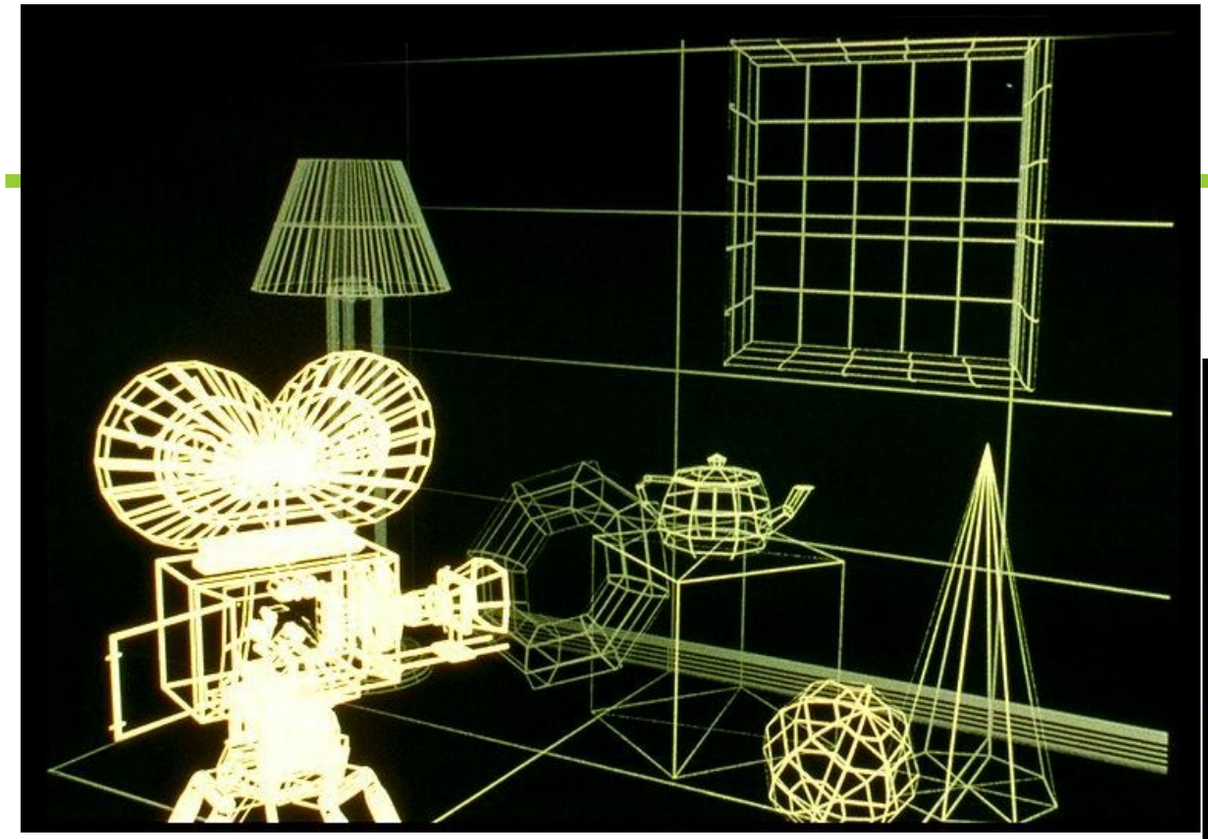


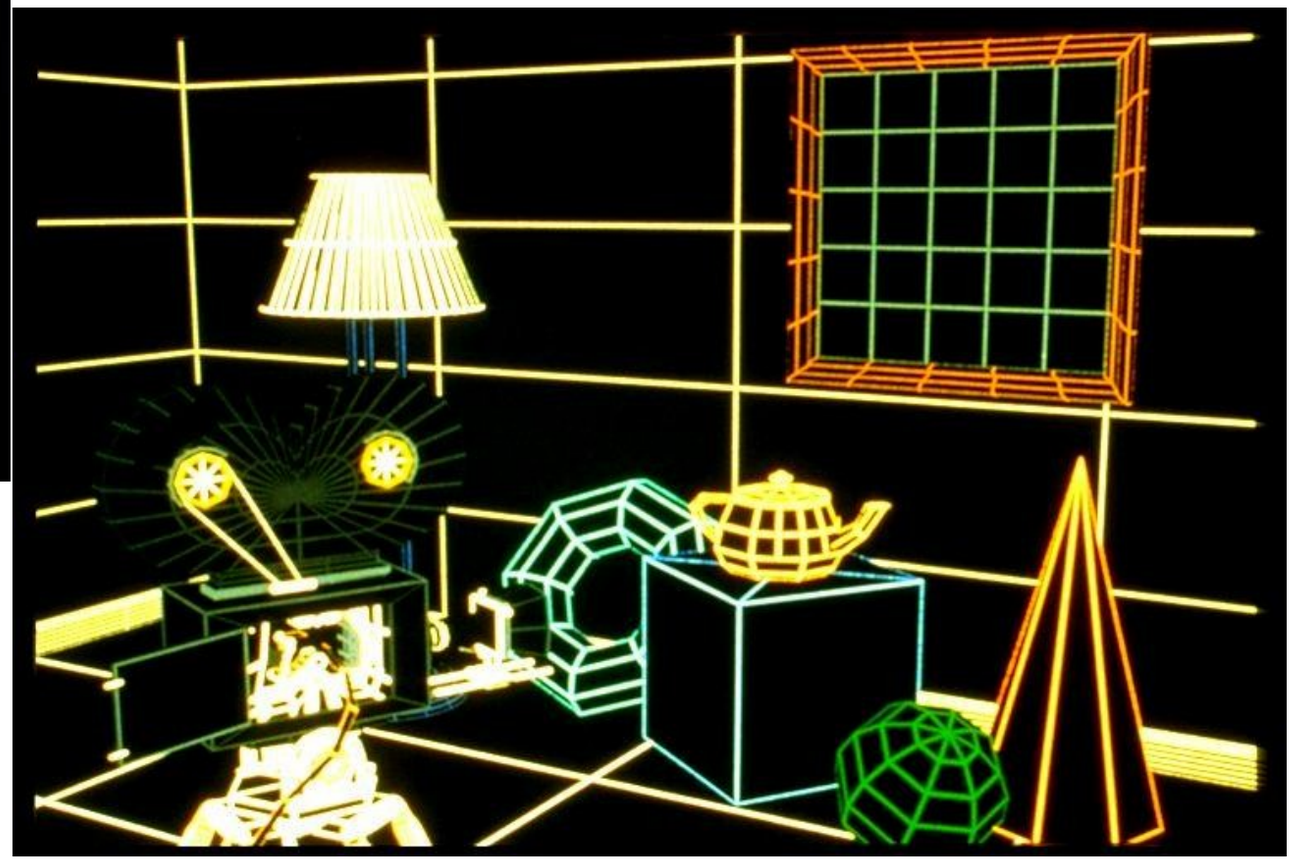
- Don’t answer like this, you will carry only zero marks

- https://www.youtube.com/watch?v=iOlehM5kNSk

Shading

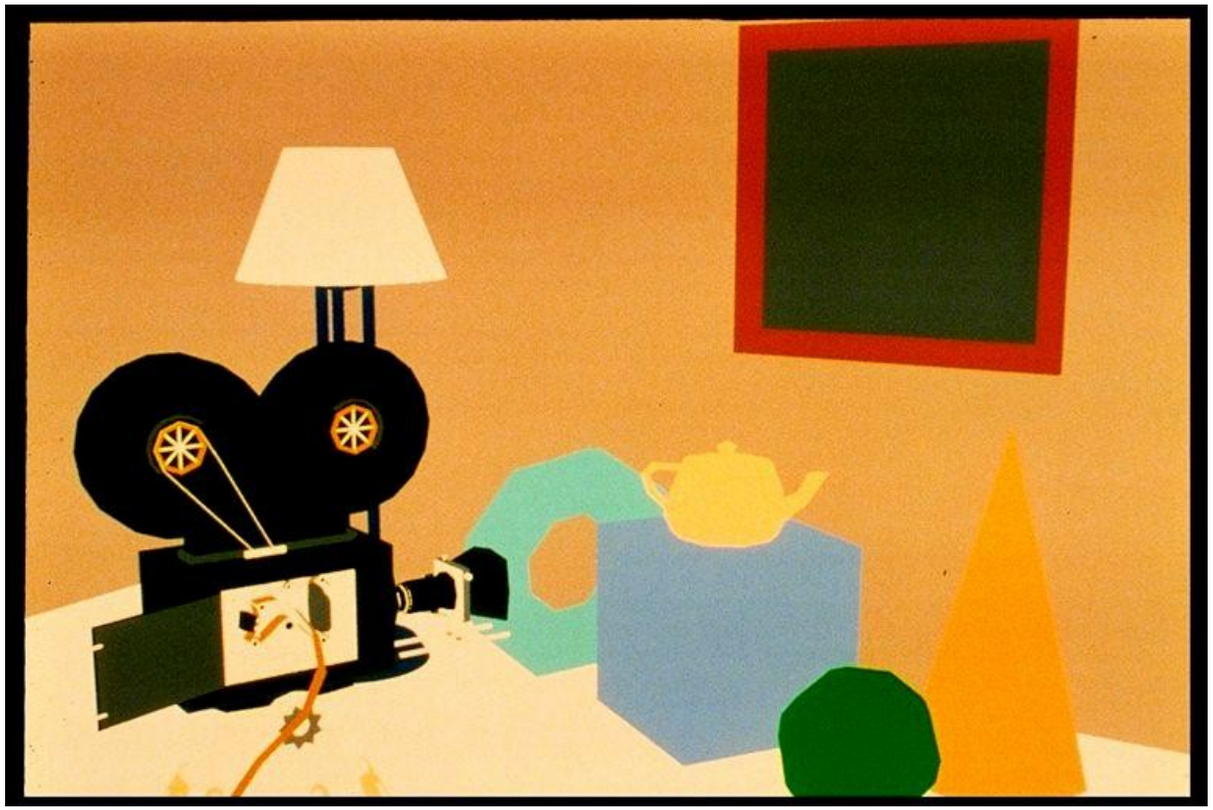
SURFACE RENDERING METHODS



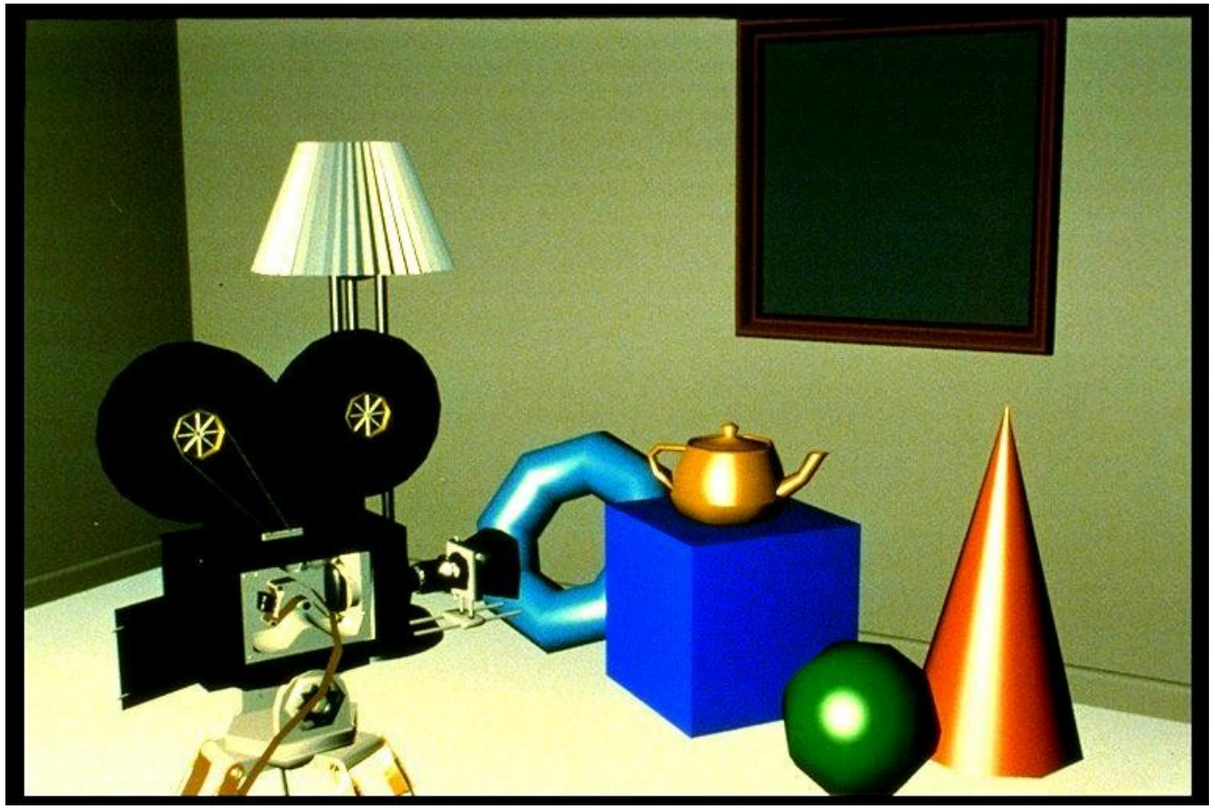


Wireframe model of a scene

No Surface Rendering Vs Surface Rendering



Object
Rendering



With Surface
Rendering

Shading

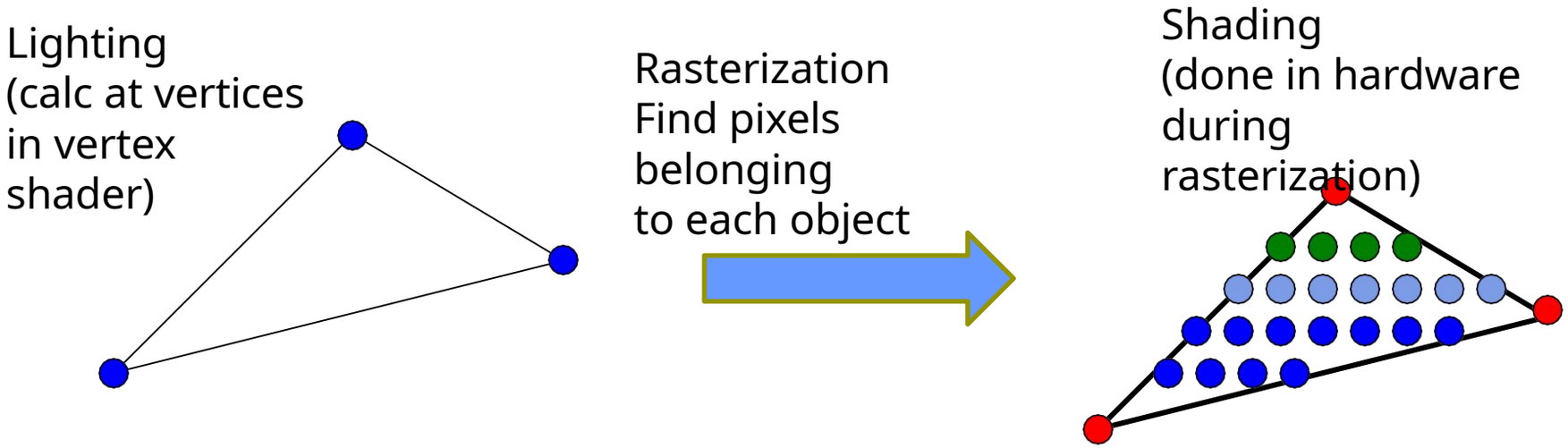
- Surface rendering means ***a procedure for applying a lighting model to obtain*** pixel intensities for all the projected surface positions in a scene.

- A surface-rendering algorithm uses the intensity calculations from an illumination model to determine the light intensity for all projected pixel positions for the various surfaces in a scene.

- Surface rendering can be performed by applying the illumination model to every visible surface point

Shading?

- After triangle is rasterized (converted to pixels)
  - Per-vertex lighting calculation means color at vertices is accurate, known (red dots)
- Shading: Graphics hardware figures out color of interior pixels (blue dots)
- How? Assume linear change => interpolate

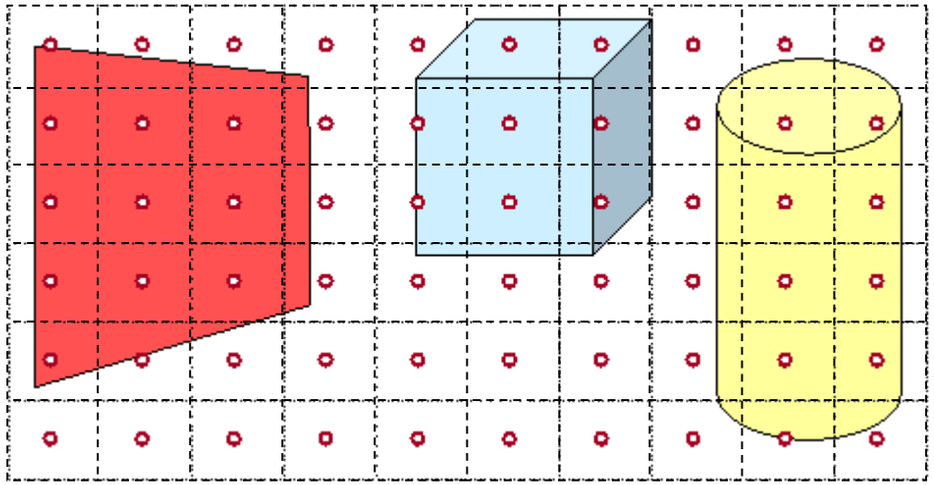


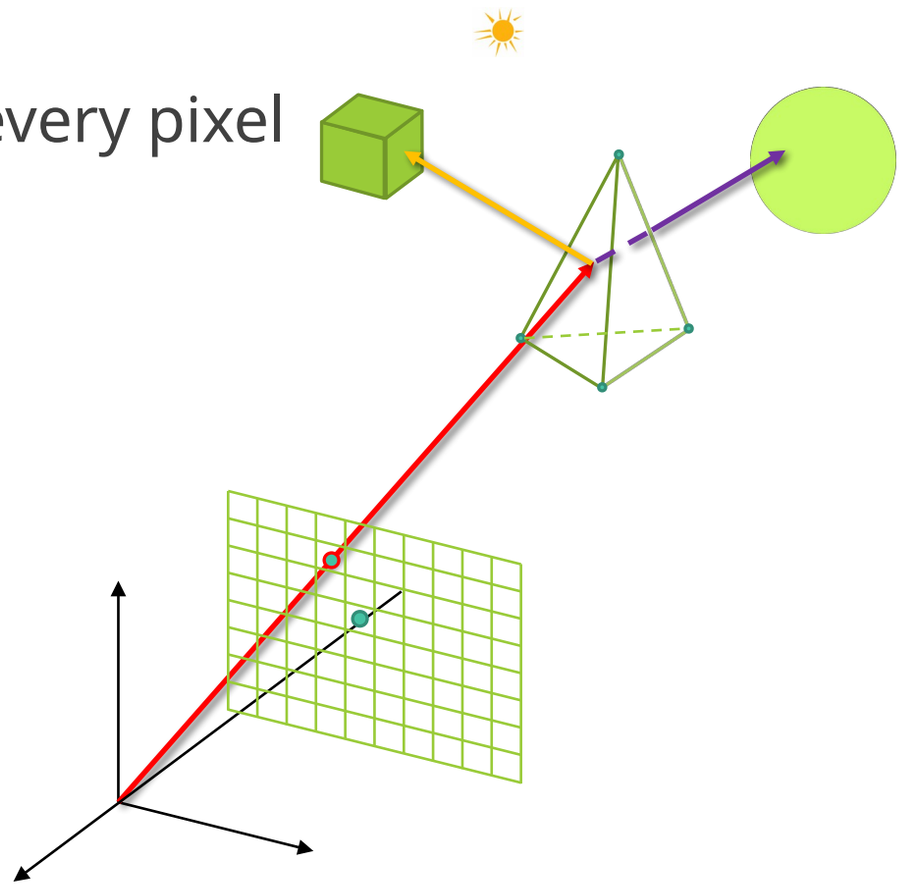
Shading Methods

- Ray Casting
  - Polygon Shading
- Ray Tracing
- Radiosity

Ray Casting

- Simplest Shading Approach
  - Perform independent lighting calculation for every pixel

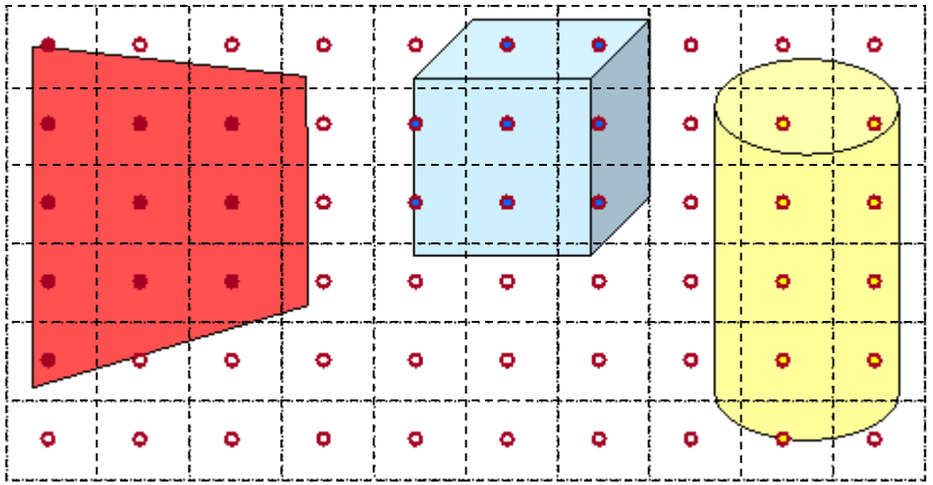




$$I = I_E + K_A I_{AL} + \sum_i (K_D (N \cdot L_i) I_i + K_S (V \cdot R_i)^n I_i)$$

Polygon Shading

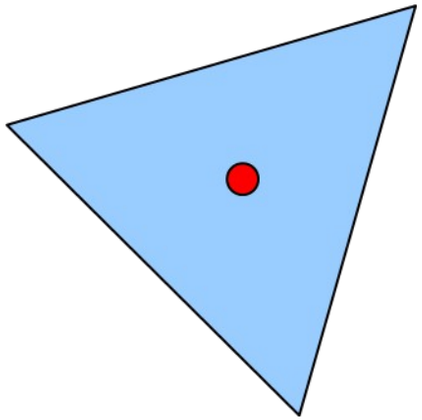
- Can Take Advantage of Spatial Coherence
  - Illumination calculations for pixels covered by same primitive are related to each other

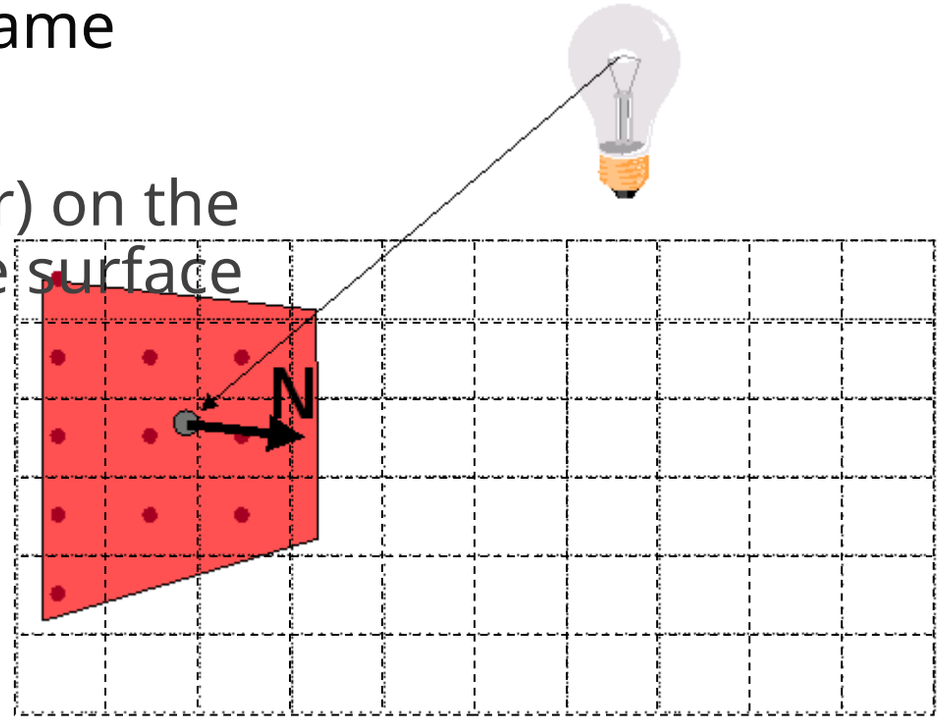


$$I = I_E + K_A I_{AL} + \sum_i (K_D (N \cdot L_i) I_i + K_S (V \cdot R_i)^n I_i)$$

Flat Shading

- Simplest method, same color is assigned to all surface positions
- One Illumination Calculation per Polygon
  - Assign all pixels inside each polygon the same color
- Illumination at a single point (usually center) on the surface is calculated and used for the entire surface

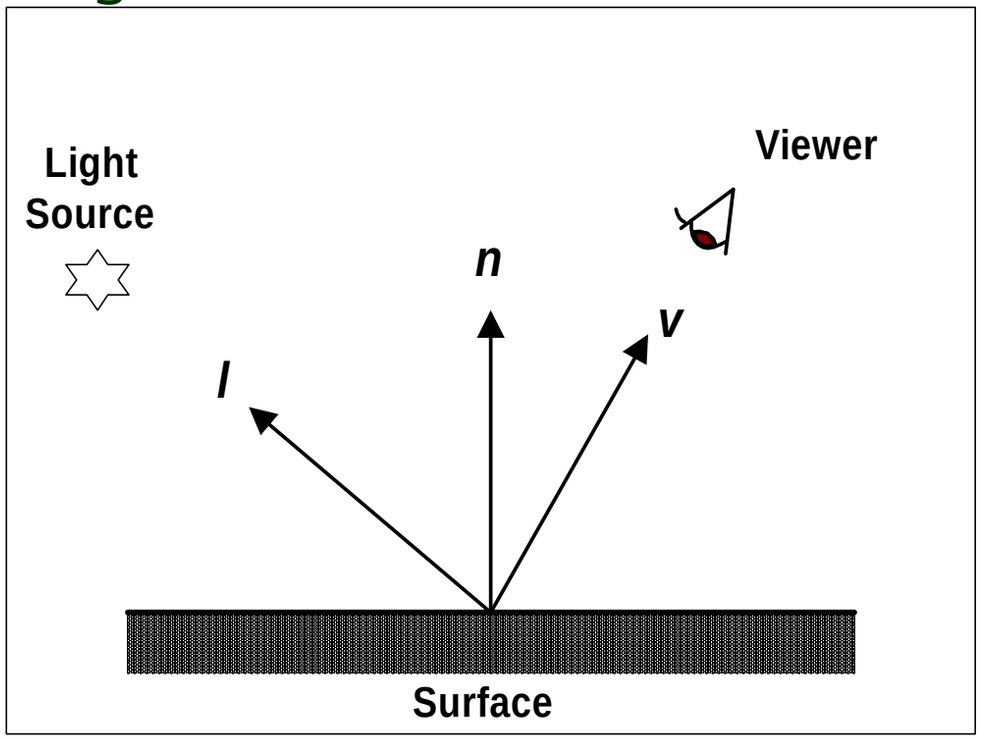




Flat Surface Rendering

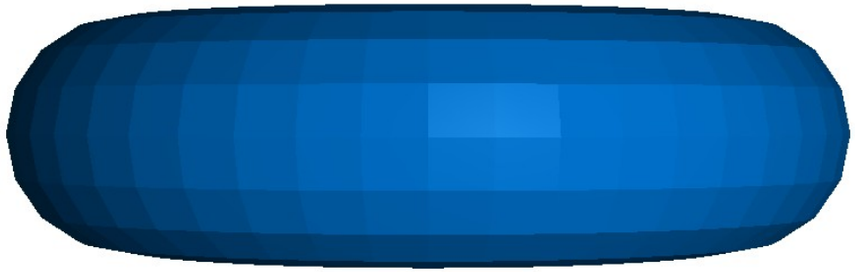
- Assumptions - For each surface
  - Light source at infinity $n \cdot l$ is constant
  - Viewer at infinity $n \cdot v$ is constant
  - The polygon represents the actual surface being modeled

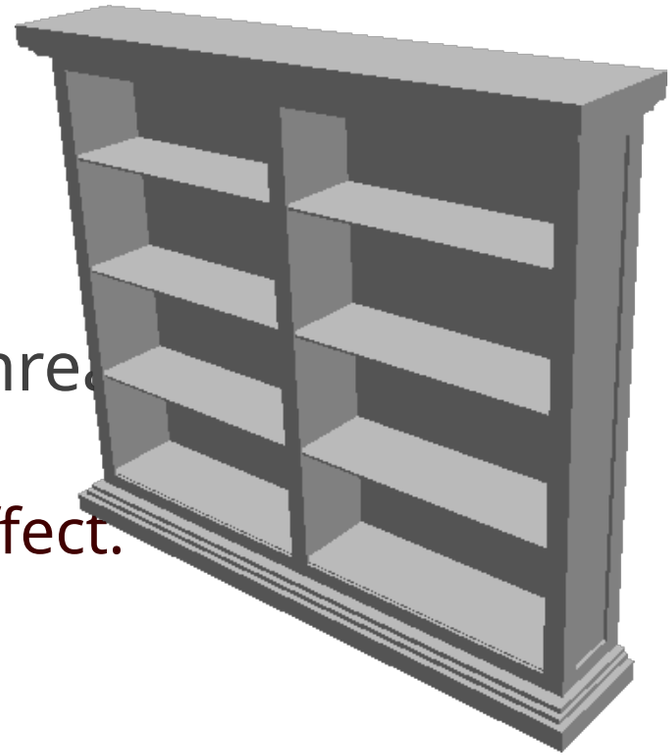
$$I = I_E + K_A I_{AL} + \sum_i (K_D (N \cdot L_i) I_i + K_S (\mathrm{V} \cdot \mathrm{R}_i)^n I_i)$$



Flat Surface Rendering

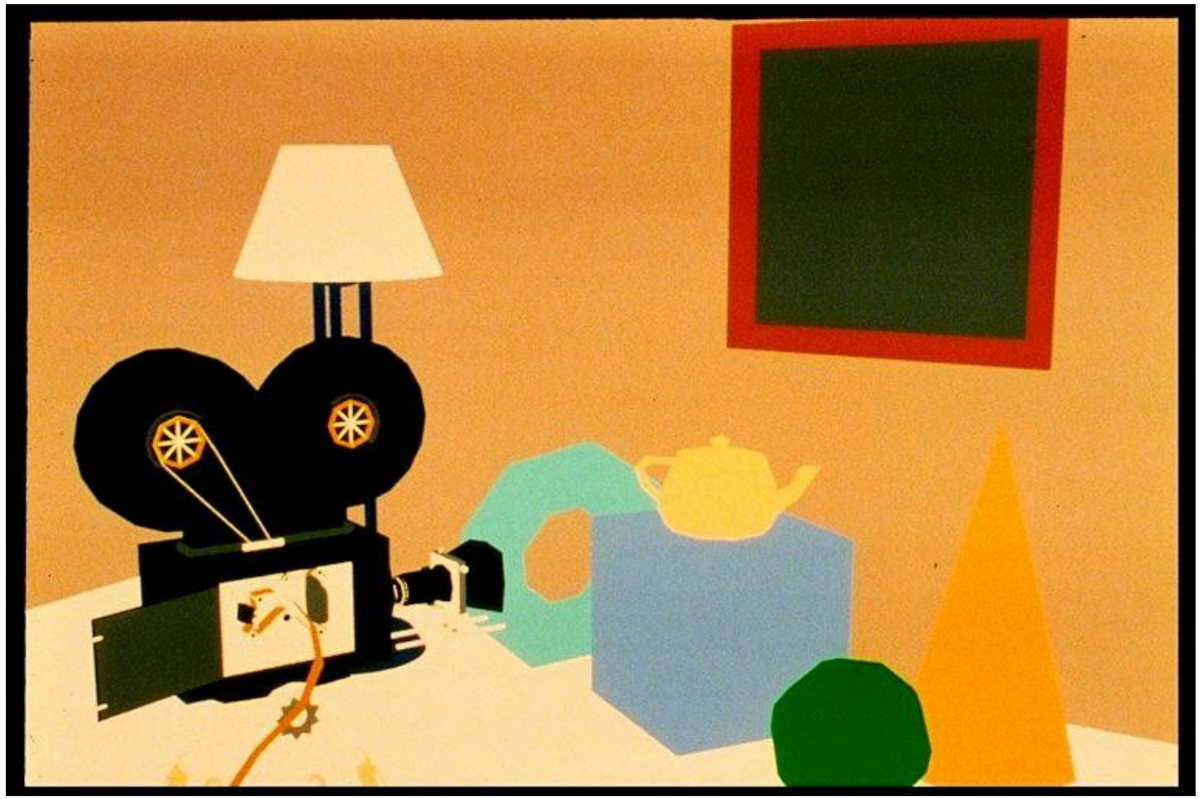
- Ok if:
  - Object consists of planar faces, and
  - Light sources are far away, and
  - Eye point is far away,
- or
  - Polygons are about a pixel in size.



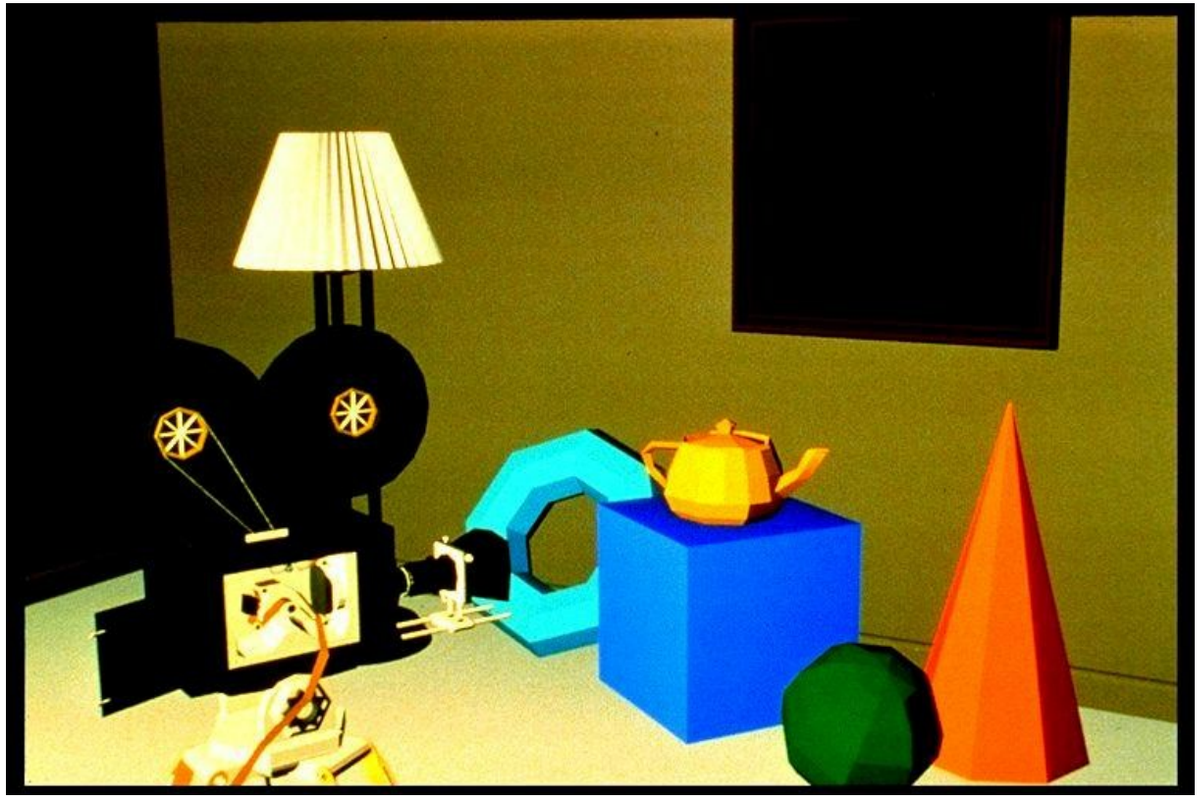


- Surface rendering is extremely fast, but can be unrea
  - Highlights not visible,
  - Facetted appearance, increased by Mach banding effect.

No Surface Rendering Vs Flat Surface Rendering



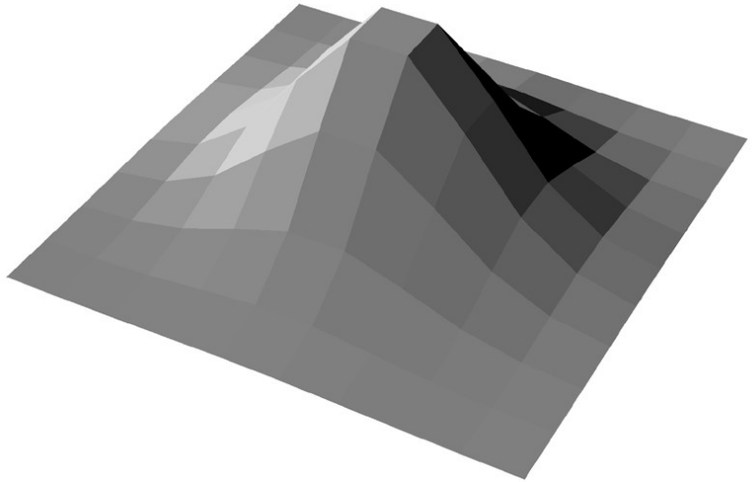
No Surface Rendering

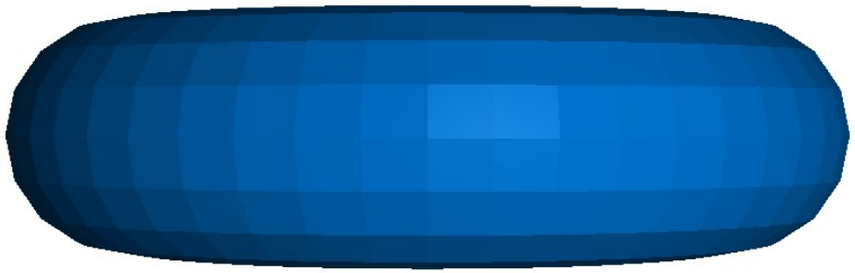


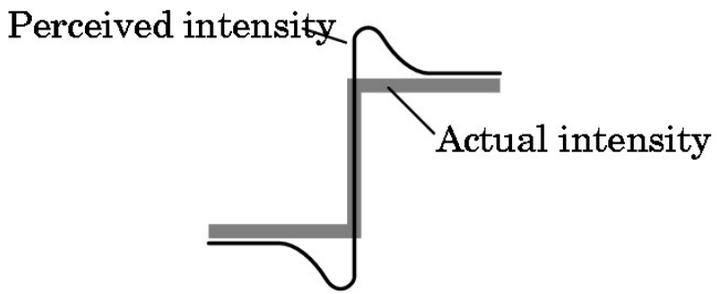
Flat Surface Rendering

Flat shading drawbacks

- The human visual system enhances edges
- We see stripes (known as **Mach Bands**) along edges
- Much like a sharpening convolution!
- How to avoid?







Overcoming Flat Shading Limitations

- Just add lots and lots of polygons – however, this is SLOW!

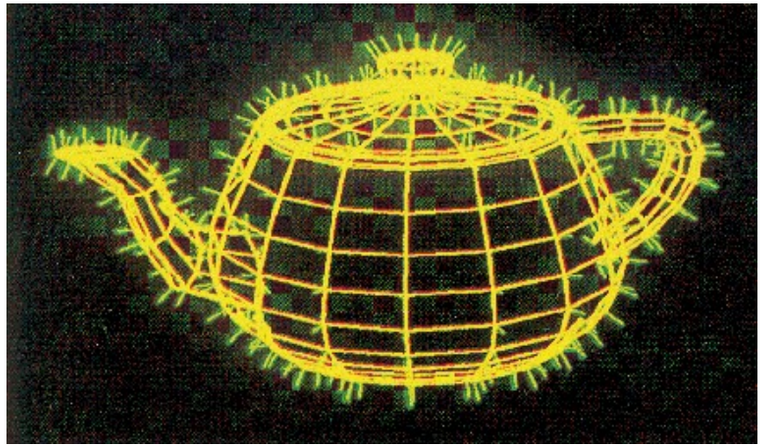
Gouraud Shading

- Developed in the 1970s by Henri Gouraud
- Worked at the University of Utah along with Ivan Sutherland and David Evans
- Often also called **intensity- interpolation surface rendering**

- Intensity levels are calculated at each vertex and interpolated across the surface

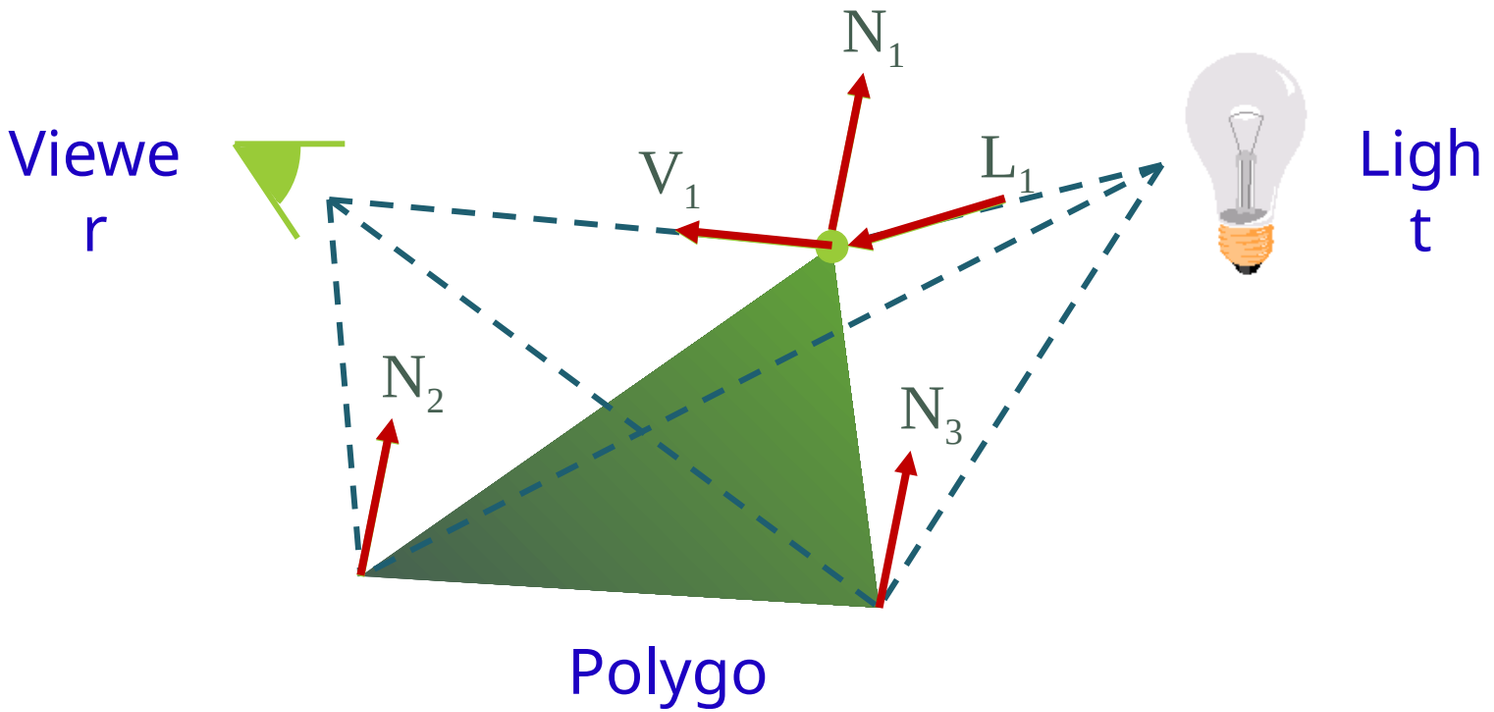
Gouraud Shading

- Smooth Surface are
  - Represented by polygonal mesh with a normal at each vertex



Gouraud Shading

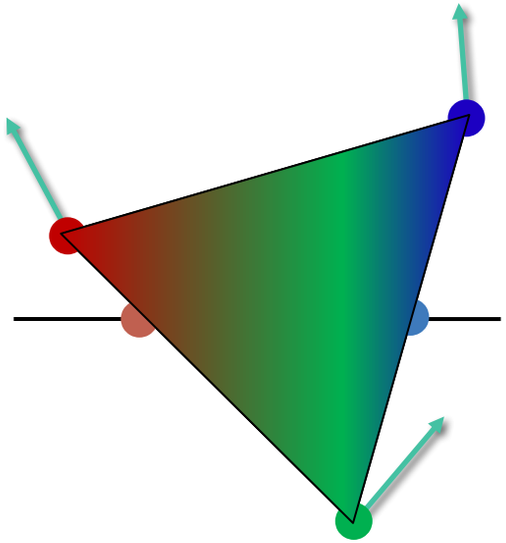
- One Lighting Calculation per Vertex
  - Assign pixels inside polygon by interpolating/ lerping colors computed at vertices



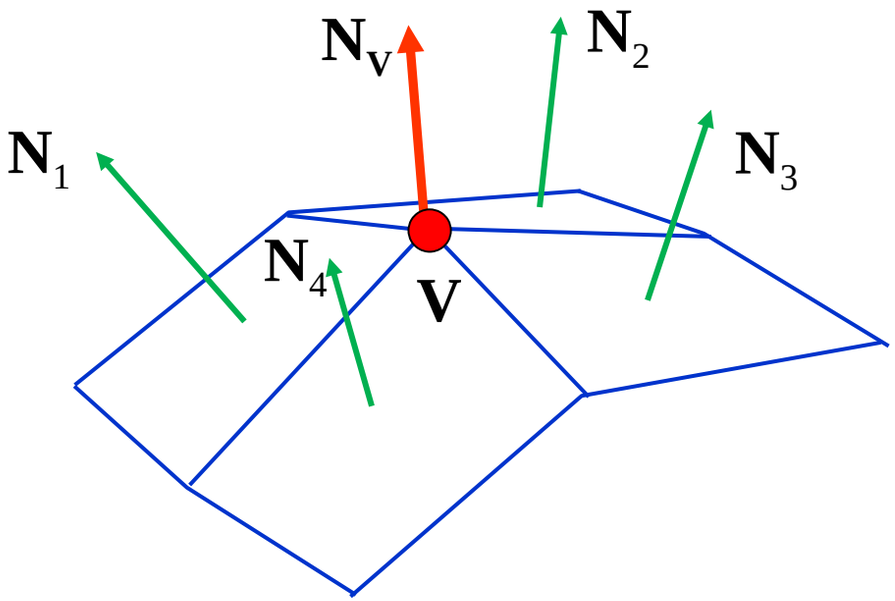
$$I = I_E + K_A I_{AL} + \sum_i (K_D (N \cdot L_i) I_i + K_S (V \cdot R_i)^n I_i)$$

Gouraud Shading

- To render a polygon, Gouraud surface rendering proceeds as follows:
  1. Determine the average unit **normal** vector **at each vertex** of the polygon
  2. **Apply an illumination model** at each polygon vertex to obtain the **light intensity** at that position
  3. **Linearly interpolate** the vertex intensities over the projected area of the polygon



Gouraud Shading



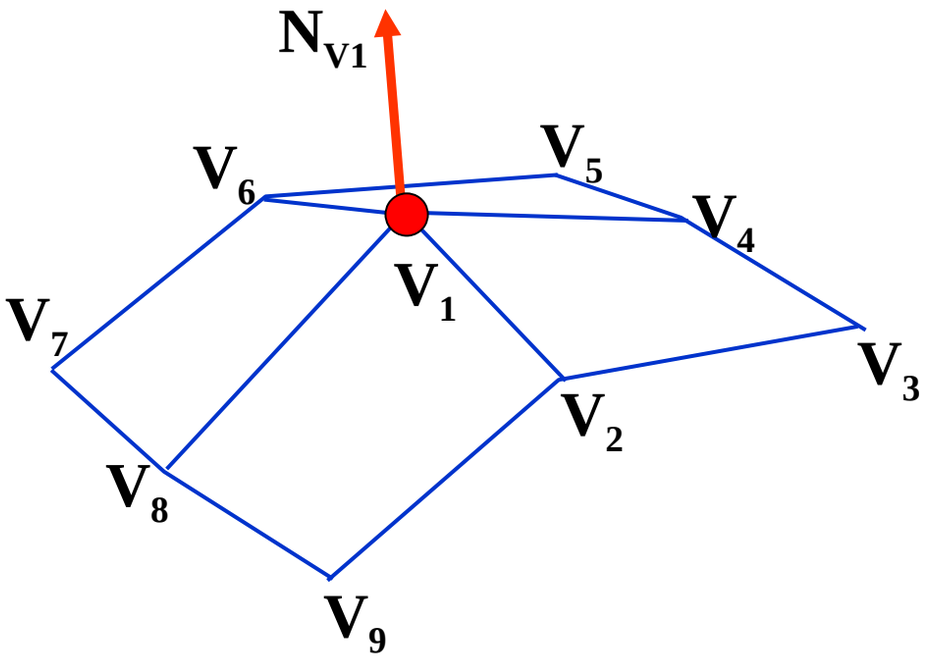
- The average unit normal vector at V is given as:

$$N_v = \frac{N_1 + N_2 + N_3 + N_4}{\left| N_1 + N_2 + N_3 + N_4 \right|}$$

- or more generally:

$$N_v = \frac{\sum_{i=1}^{n} N_i}{\left| \sum_{i=1}^{n} N_i \right|}$$

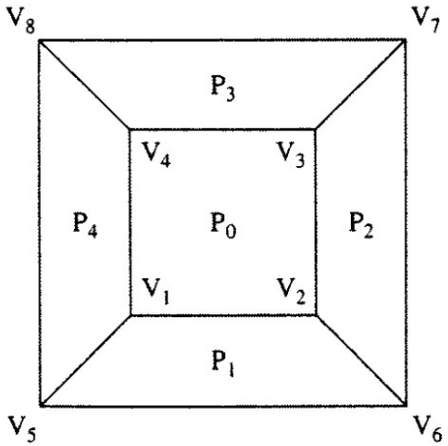
Gouraud Shading

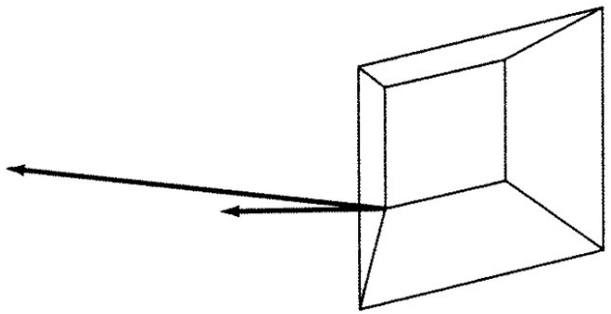


- The average unit normal vector at V is given as:

$$N = V_1V_2 \times V_1V_4 + V_1V_4 \times V_1V_6 + V_1V_6 \times V_1V_8 + V_1V_8 \times V_1V_2$$

$$N_{v1} = \frac{N}{|N|}$$





$$
\begin{aligned}
P_0: \quad & z - 1 = 0 \\
P_1: \quad & -y + z - 2 = 0 \\
P_4: \quad & -x \quad + z - 2 = 0
\end{aligned}
$$

Consider the polygonal surface shown in Fig. 5–9a. The vertex points are $V_1(-1,-1,1)$, $V_2(1,-1,1)$, $V_3(1,1,1)$, $V_4(-1,1,1)$, $V_5(-2,-2,0)$, $V_6(2,-2,0)$, $V_7(2,2,0)$, $V_8(-2,2,0)$. The surface is a truncated pyramid. The plane equations for the faces labeled P_0, P_1, P_4 surrounding V_1 are

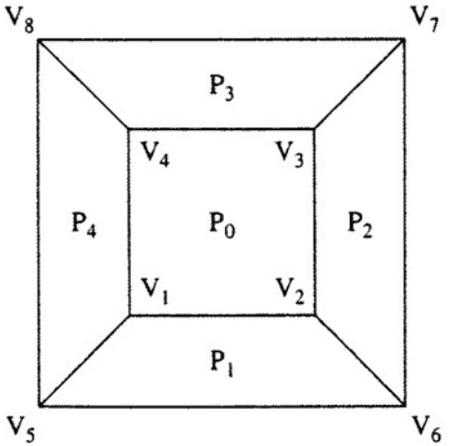
Approximating the normal at V_1

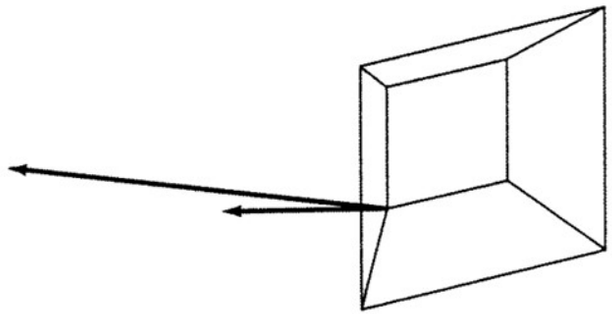
$$
\begin{aligned}
\mathbf{n}_1 &= (a_0 + a_1 + a_4)\mathbf{i} + (b_0 + b_1 + b_4)\mathbf{j} + (c_0 + c_1 + c_4)\mathbf{k} \\
&= (0 + 0 - 1)\mathbf{i} + (0 - 1 + 0)\mathbf{j} + (1 + 1 + 1)\mathbf{k} \\
&= -\mathbf{i} - \mathbf{j} + 3\mathbf{k}
\end{aligned}
$$

$$
|\mathbf{n}_1| = \sqrt{(-1)^2 + (-1)^2 + (3)^2} = \sqrt{11}
$$

$$
\frac{\mathbf{n}_1}{|\mathbf{n}_1|} = -0.3\mathbf{i} - 0.3\mathbf{j} + 0.9\mathbf{k}
$$

$$\mathbf{V_1V_2} \otimes \mathbf{V_1V_4} = 4\mathbf{k}$$
$$\mathbf{V_1V_5} \otimes \mathbf{V_1V_2} = -2\mathbf{j} + 2\mathbf{k}$$
$$\mathbf{V_1V_4} \otimes \mathbf{V_1V_5} = -2\mathbf{i} + 2\mathbf{k}$$





$$\mathbf{n}_1 = -\frac{2}{3}\mathbf{i} - \frac{2}{3}\mathbf{j} + \frac{8}{3}\mathbf{k}$$

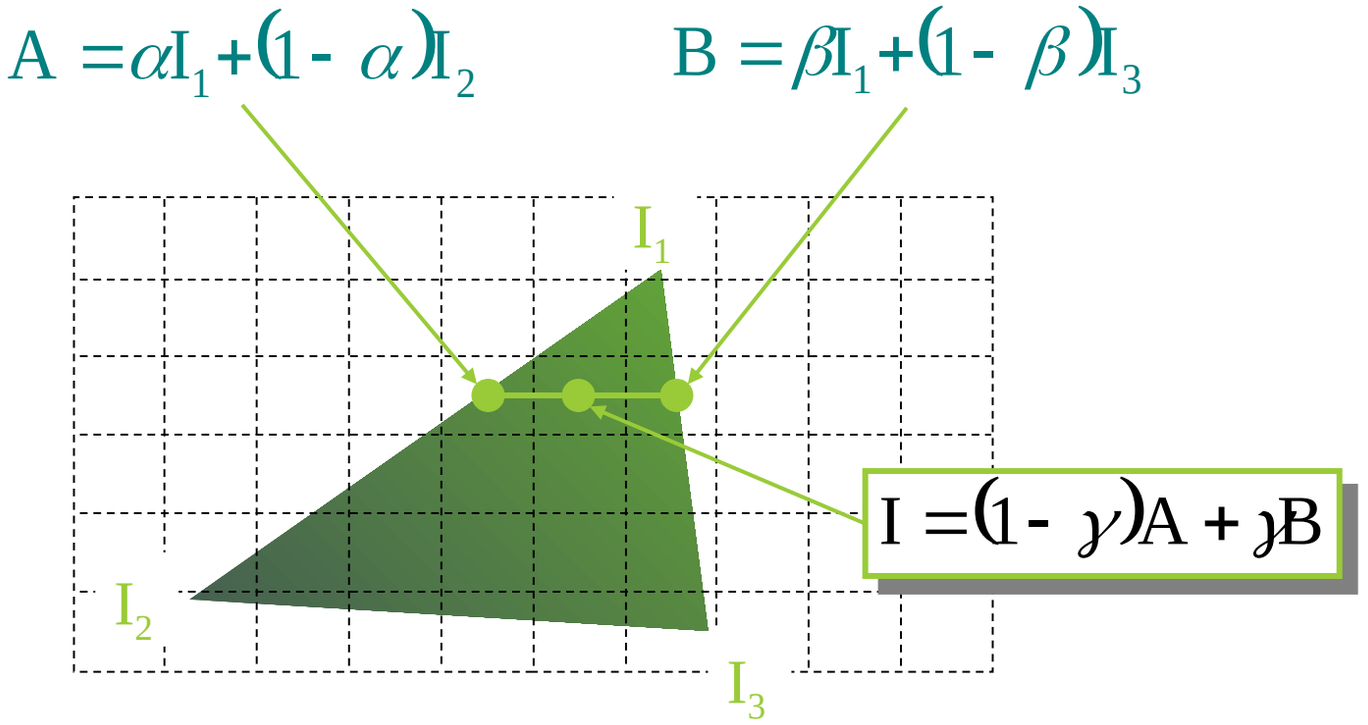
$$|\mathbf{n}_1| = \frac{1}{3}\sqrt{(-2)^2 + (-2)^2 + (8)^2} = \frac{1}{3}\sqrt{72}$$

$$\frac{\mathbf{n}_1}{|\mathbf{n}_1|} = -0.24\mathbf{i} - 0.24\mathbf{j} + 0.94\mathbf{k}$$

Notice that both the direction and the magnitude of the normalized and unnormalized surface normals are different for different approximation techniques. This is shown in Fig. 5–9b. Consequently, an illumination model yields subtly different results depending on the technique used to approximate the surface normal.

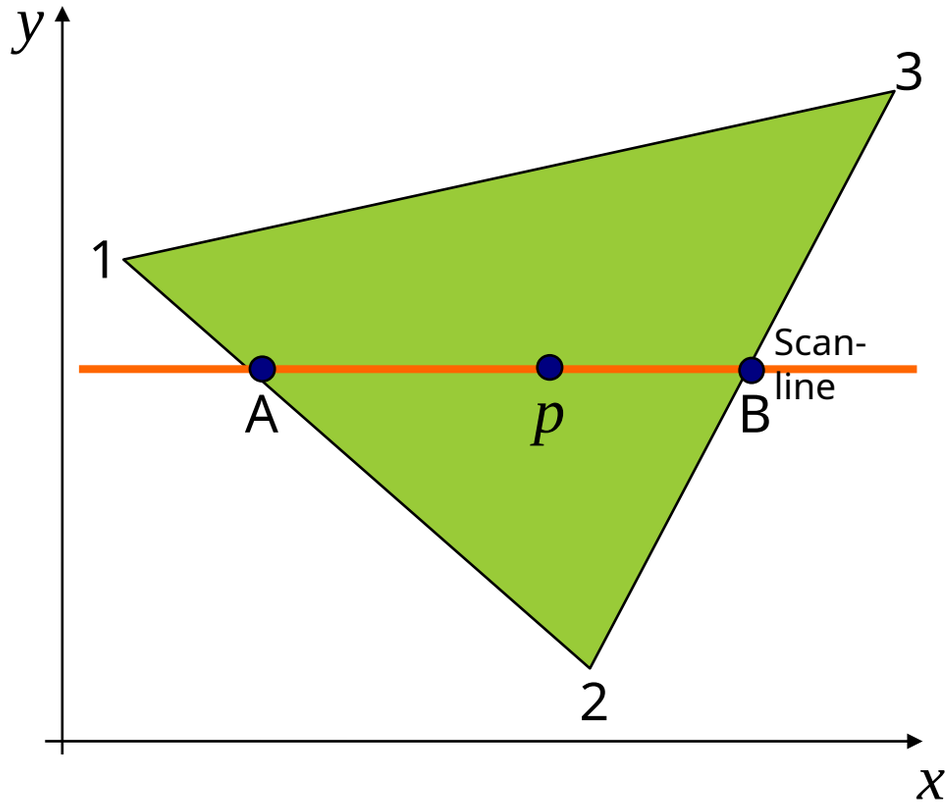
Gouraud Shading

- Bilinearly Interpolate Colors at Vertices Down and Across Scan Lines



Gouraud Shading

- Illumination values are bilinearly interpolated across each scan-line



$$I_A = \frac{y_A - y_2}{y_1 - y_2} I_1 + \frac{y_1 - y_A}{y_1 - y_2} I_2$$

$$I_B = \frac{y_B - y_2}{y_3 - y_2} I_3 + \frac{y_3 - y_B}{y_3 - y_2} I_2$$

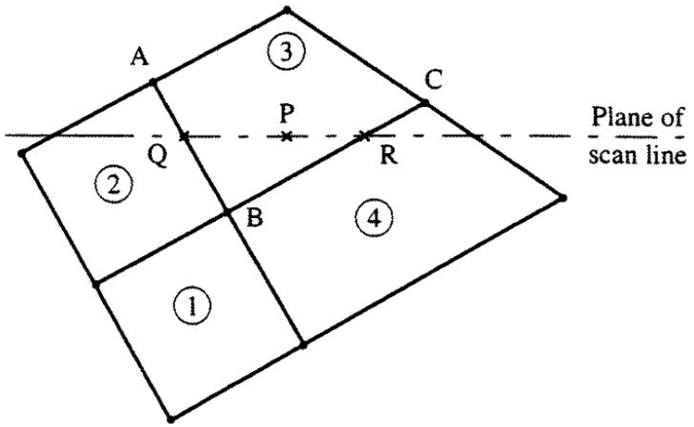
$$I_p = \frac{x_B - x_p}{x_B - x_A} I_A + \frac{x_p - x_A}{x_B - x_A} I_B$$

Gouraud Shading

- Illumination values are bilinearly interpolated across each scan-line

The intensity calculation along the scan line can be performed incrementally. For two pixels at t_1 and t_2 on the scan line

$$I_{P_2} = t_2 I_Q + (1 - t_2) I_R$$



and

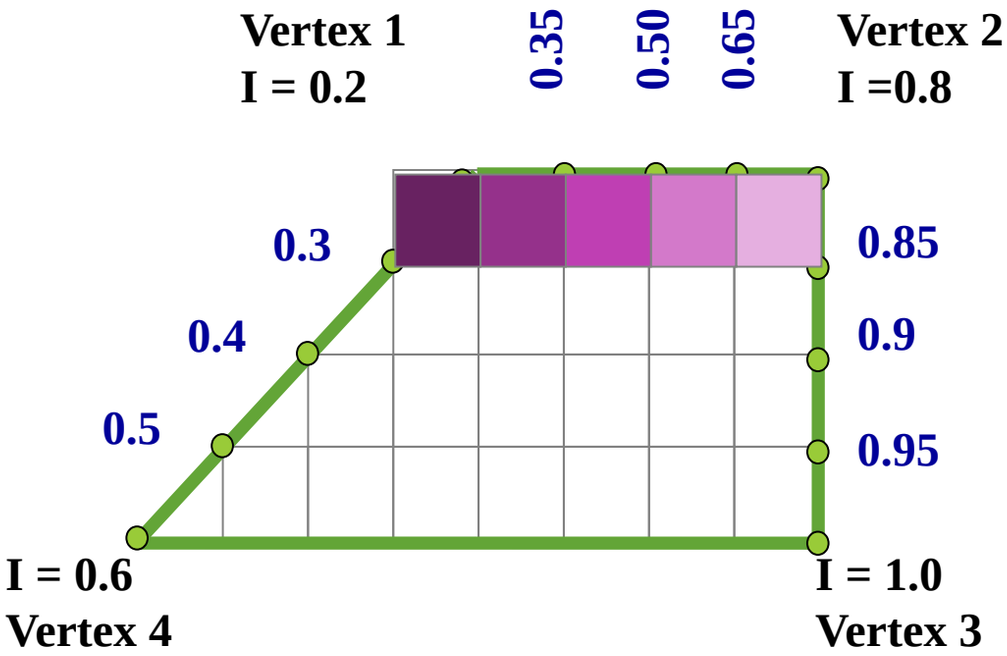
$$I_{P_1} = t_1 I_Q + (1 - t_1) I_R$$

Subtracting yields

$$I_{P_2} = I_{P_1} + (I_Q - I_R)(t_2 - t_1) = I_{P_1} + \Delta I_{QR} \Delta t$$

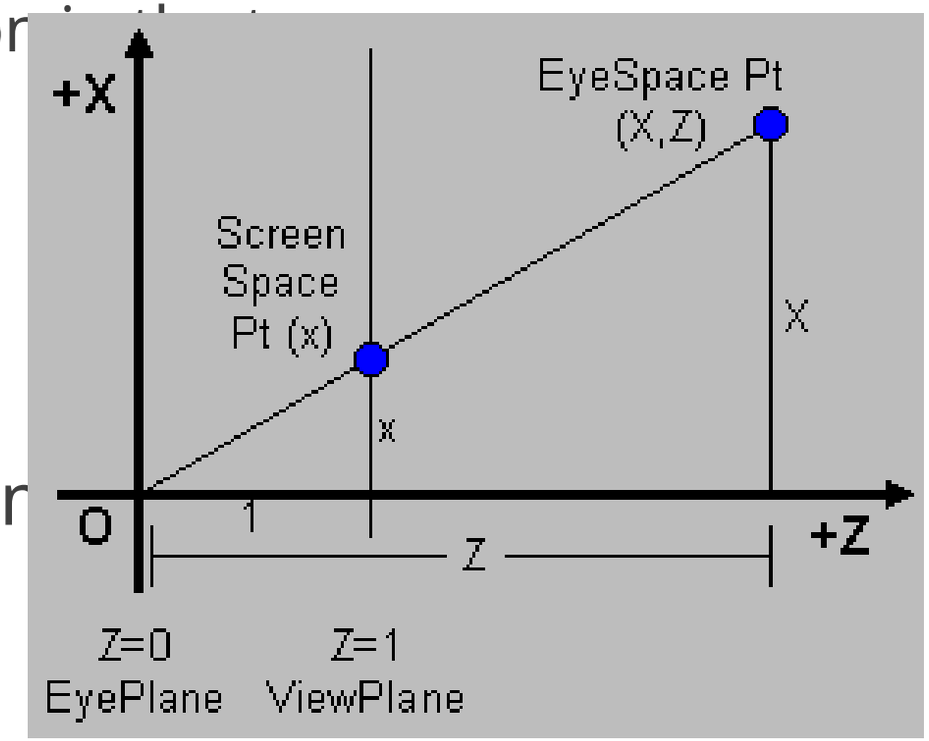
Gouraud Shading

An Example



Linear Interpolation Concerns

- Perspective projection complicates linear interpolation.
  - Relationship between screen space distance and eye space distance is nonlinear.
  - Therefore, relationship between interpolation in the two spaces is also nonlinear.
  - Thus, screen space linear interpolation of colors (and texture coordinates) results in incorrect values.
- Note: potential homework / test problem



Perspectively-correct Interpolation

- Could interpolate in eye space, then project every interpolated point.
  - Way too much work!
- Can we interpolate in screen space and correct for perspective nonlinearity?
  - Yes!

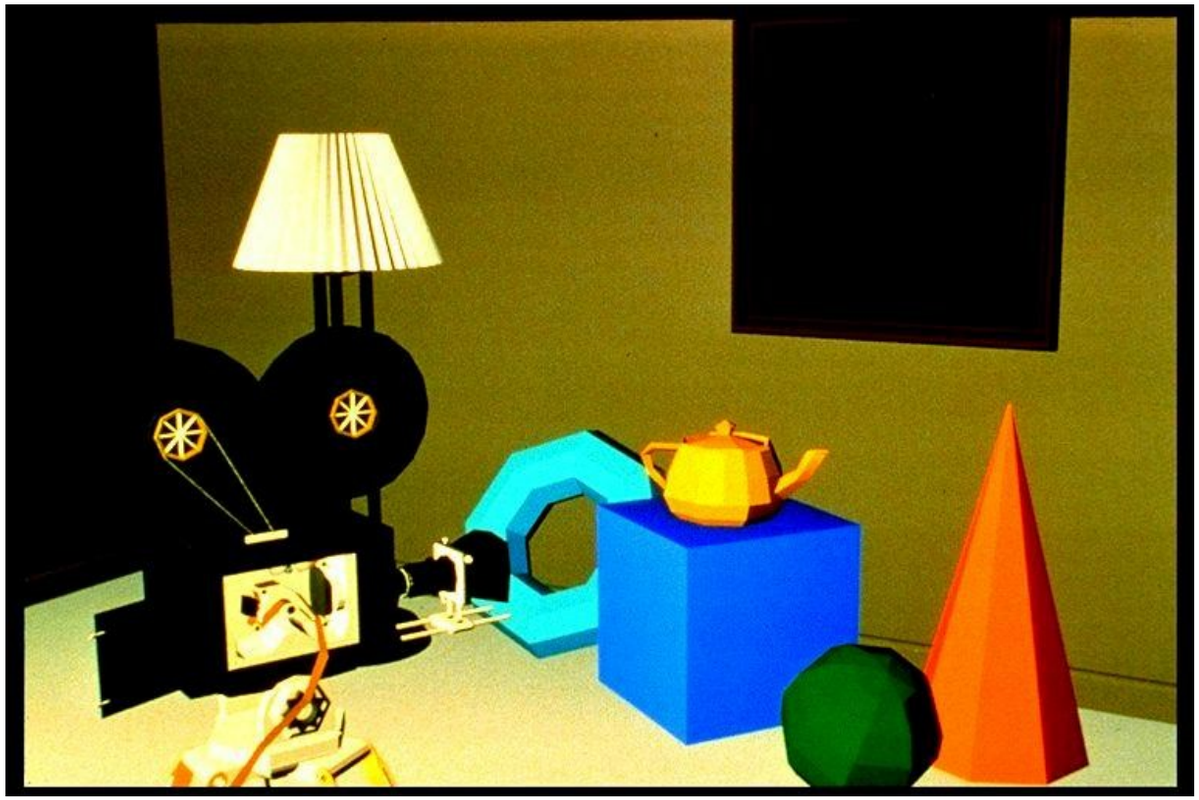
Perspectively-correct Interpolation

- For a detailed derivation, see:
  - http://www.cs.unc.edu/~hoff/techrep/persp/persp.html
- Here, we skip to the punch line:
  - Given two eye space points, E_1 and E_2.
    - Can lerp in eye space: $E(T) = E_1(1-T) + E_2(T)$.
    - T is eye space parameter, t is screen space parameter.
  - To see relationship, express in terms of screen space t:
    - $E(t)= [(E_1/Z_1)*(1-t) + (E_2/Z_2)*t] / [(1/Z_1)*(1-t) + (1/Z_2)*t]$

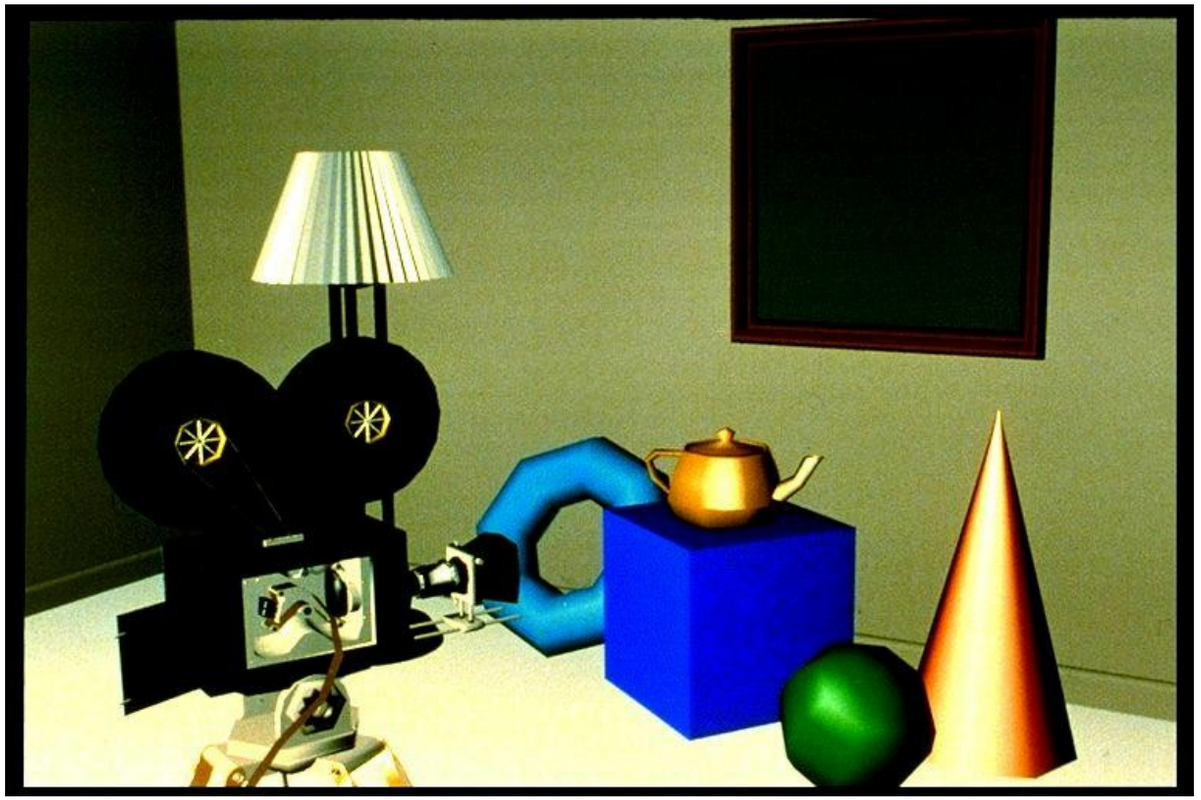
Perspectively-correct Interpolation

- $E(t) = [\ (E_1/Z_1)*(1-t) + (E_2/Z_2)*t\ ] / [\ (1/Z_1)*(1-t) + (1/Z_2)*t\ ]$
  - E_1/Z_1, E_2/Z_2 are projected points.
  - Because Z_1, Z_2 are depths corresponding to E_1, E_2.
- Looking closely, can see that interpolation along an eye space edge = interpolation along projected edge in screen space divided by the interpolation of $1/Z$.

Flat Vs Gouraud Rendering



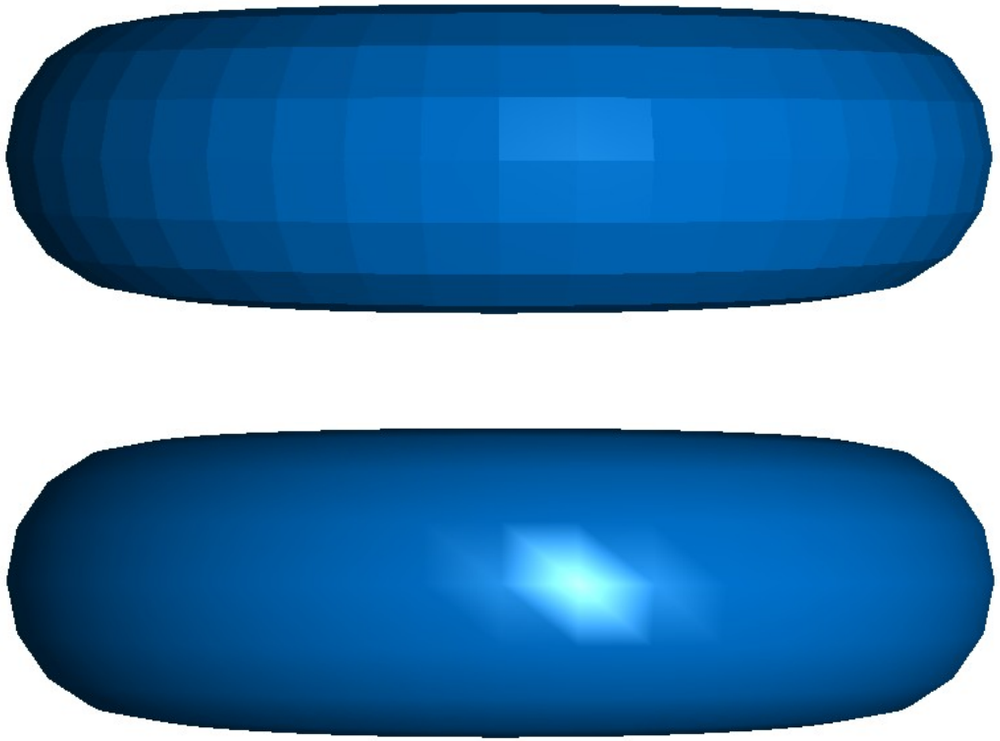
Flat Surface Rendering

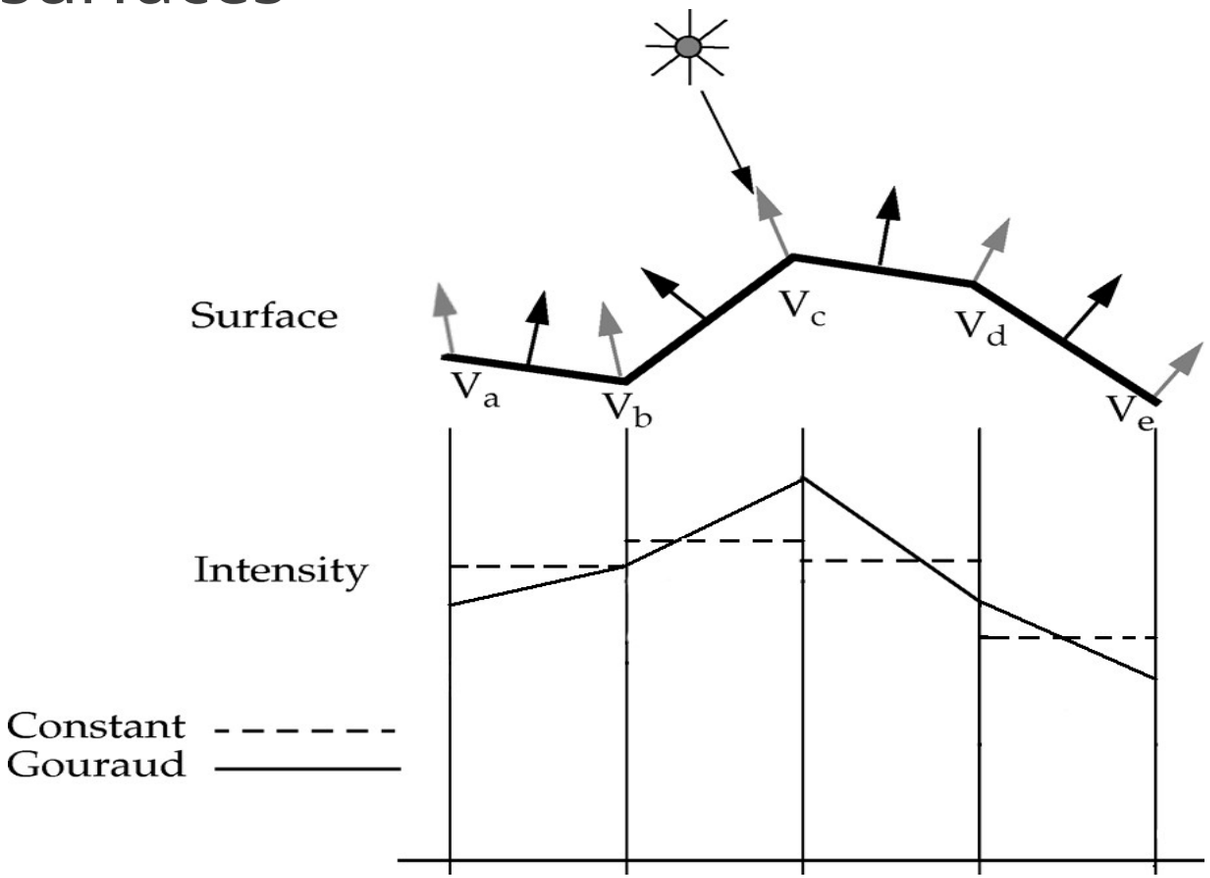


Gouraud Surface Rendering

Gouraud Shading

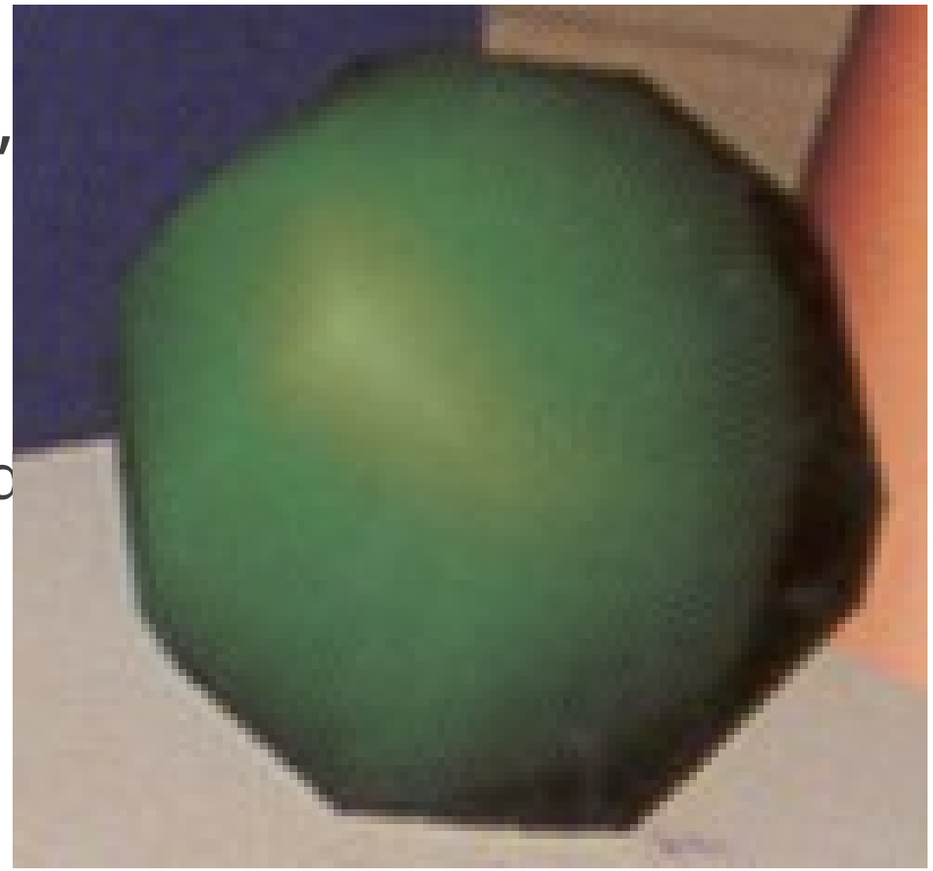
- Much better result for curved surfaces





Gouraud Shading - Drawbacks

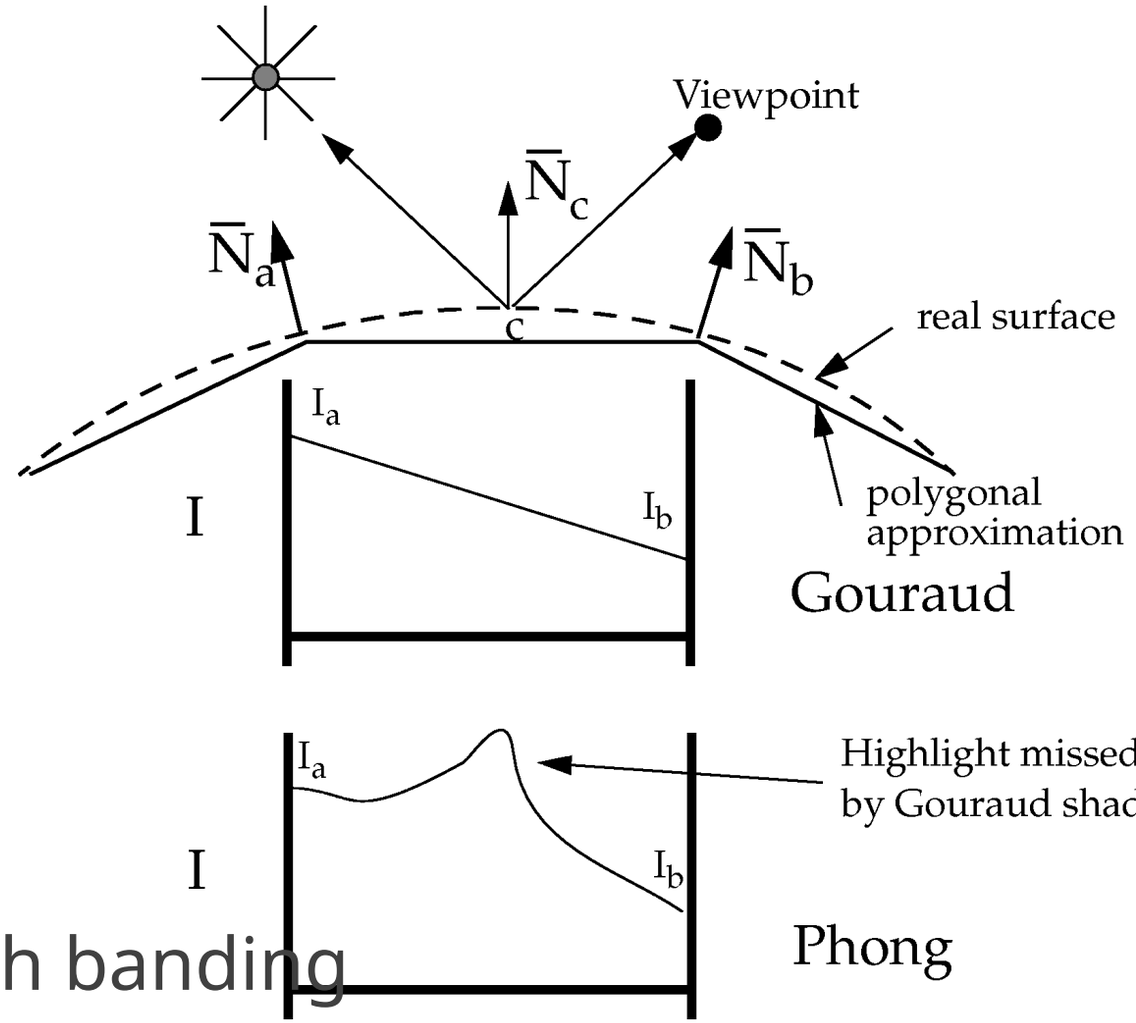
- Polygon edges are still visible
- Brightness is modelled as a linear function, but that's not really accurate
- Real highlights are small and bright, and drop off sharply
  - If polygons are too large, highlights get distorted and dimmed (notice the funny shape)



- How to avoid these artifacts?

Gouraud Shading - Drawbacks

- It has a problem with **specular reflections**
  - Completely miss
  - interpolat



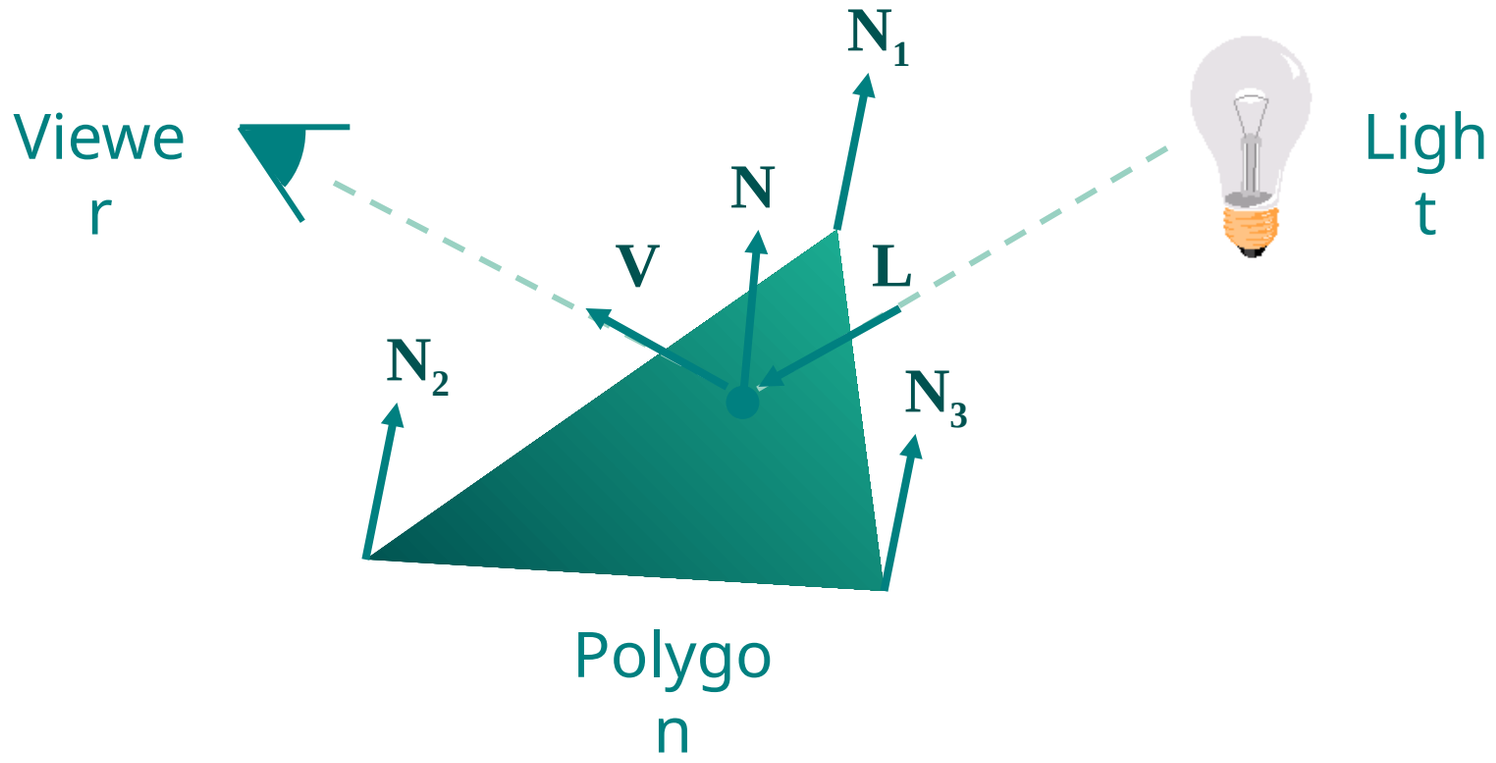
- Linear interpolation still gives Mach banding
  - Silhouettes are still not smooth

Phong Shading

- A more accurate interpolation based approach for rendering a polygon was developed by Phong Bui Tuong
- Basically the Phong surface rendering model (or **normal-vector interpolation rendering**) interpolates normal vectors instead of intensity values

Phong Shading

- One Lighting Calculation per Pixel
  - Approximate surface normals for points inside polygons by bilinear interpolation of normals from vertices

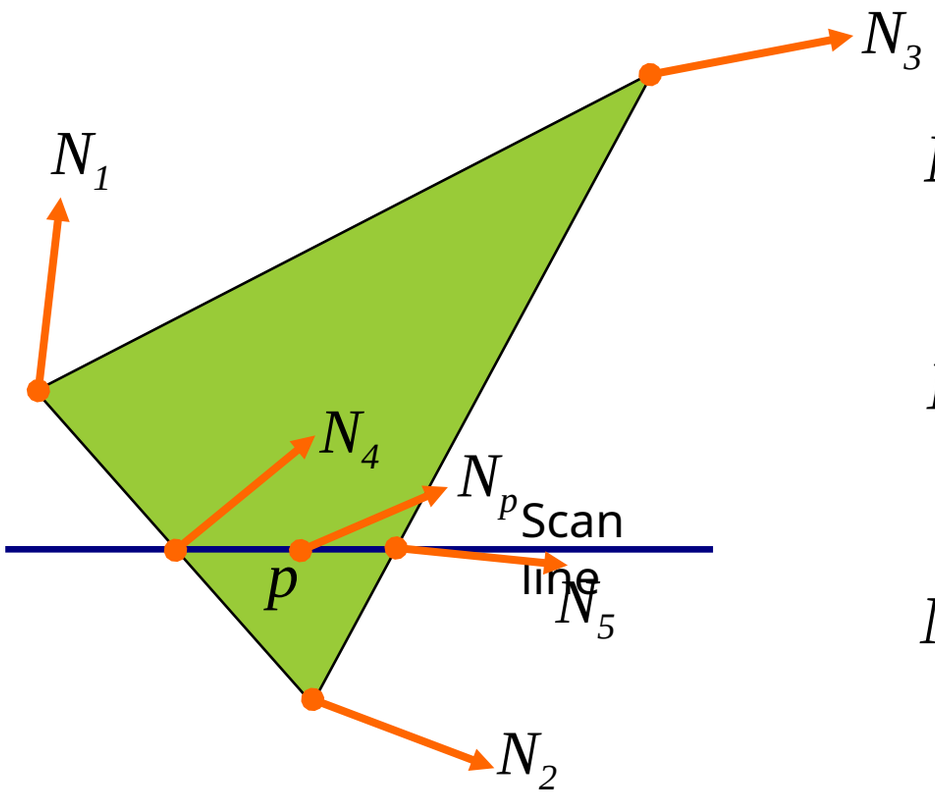


Phong Shading

- To render a polygon, Phong surface rendering proceeds as follows:
  1. Determine the average unit normal vector at each vertex of the polygon
  2. Linearly **interpolate** the vertex **normals** over the projected area of the polygon
     - Normalize it.
     - (Interpolation of unit vectors does not preserve length).
  3. Apply an illumination model at positions along scan lines to calculate pixel intensities using the interpolated normal vectors

Phong Shading

- Bilinearly Interpolate Normals at Vertices Down and Across Scan Lines

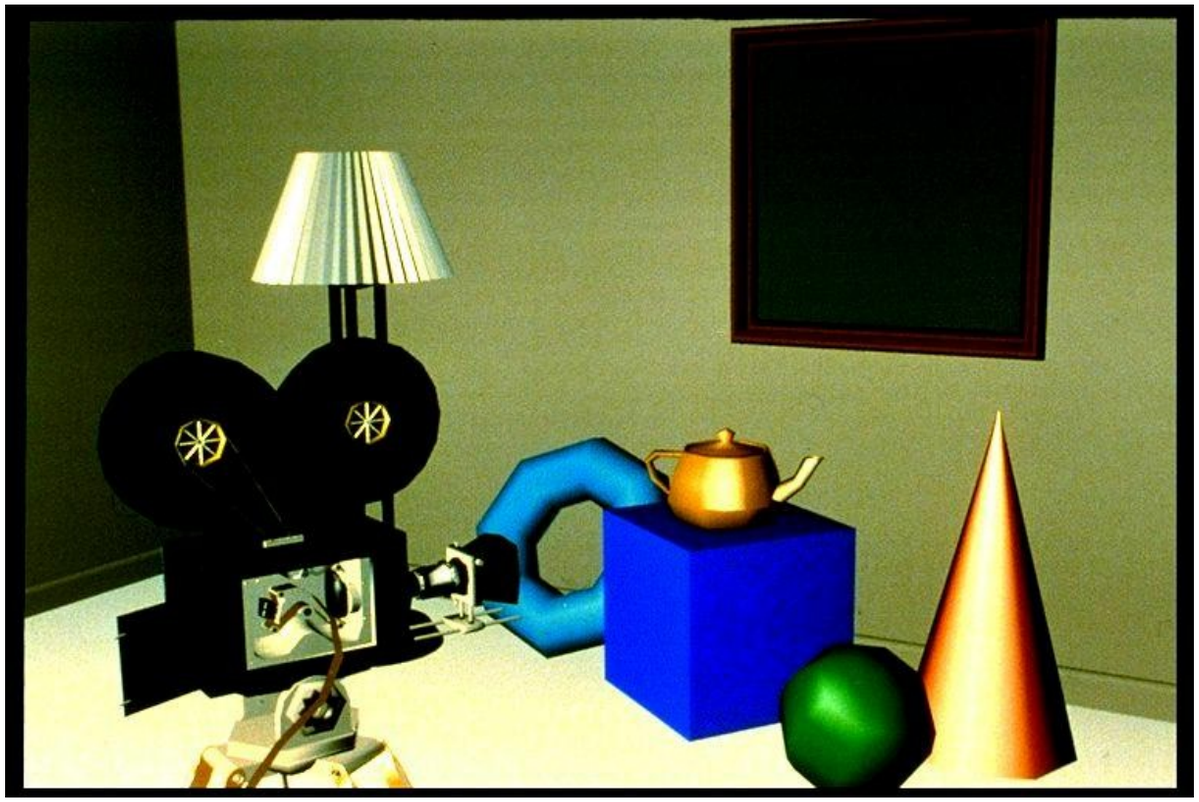


$$N_4 = \frac{y_4 - y_2}{y_1 - y_2} N_1 + \frac{y_1 - y_4}{y_1 - y_2} N_2$$

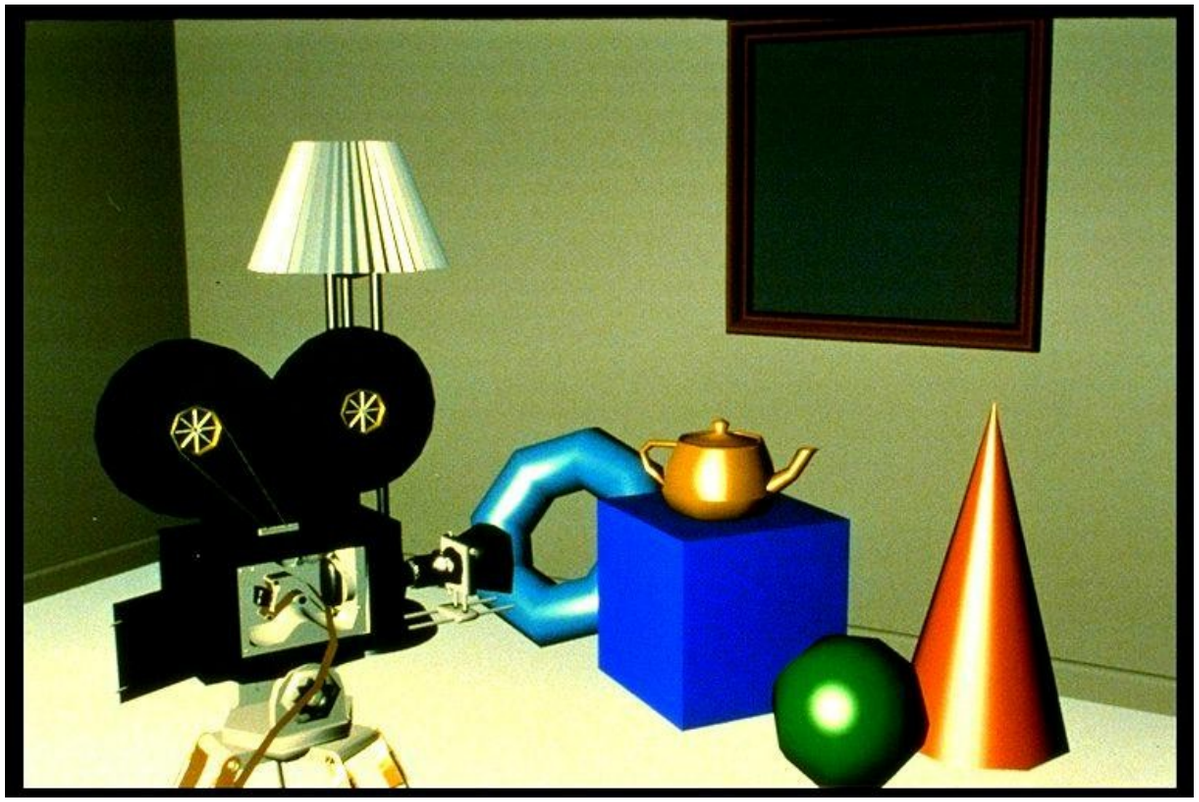
$$N_5 = \frac{y_5 - y_2}{y_3 - y_2} N_3 + \frac{y_3 - y_5}{y_3 - y_2} N_2$$

$$N_p = \frac{y_p - y_5}{y_4 - y_5} N_4 + \frac{y_4 - y_p}{y_4 - y_5} N_5$$

Gouraud Vs Phong Surface Rendering

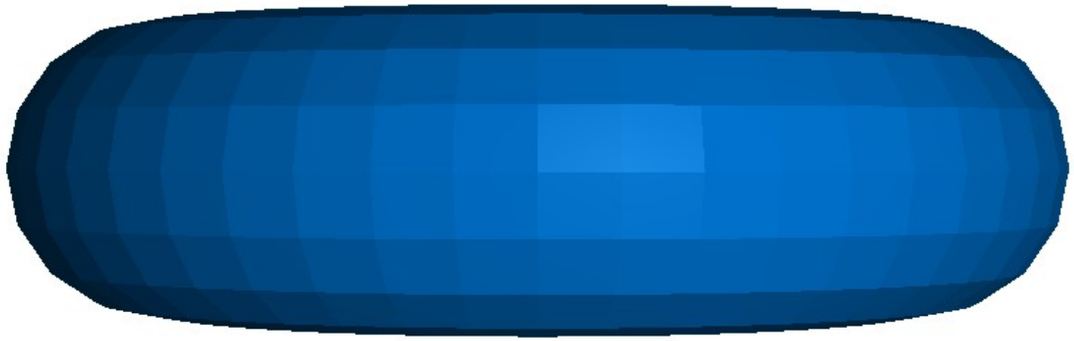


Gouraud Surface Rendering

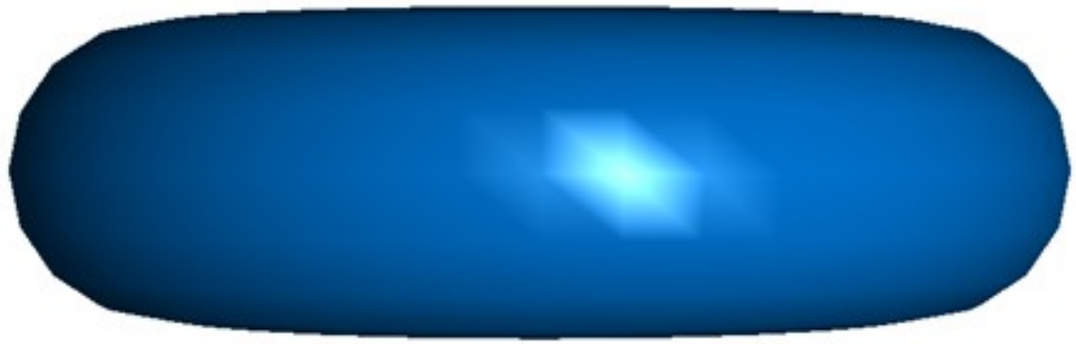


Phong Surface Rendering

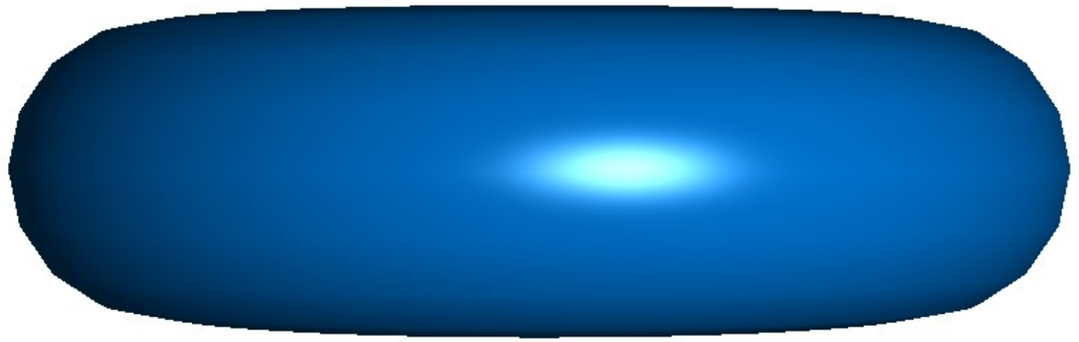
Phong Shading



Flat



Gouraud



Phong

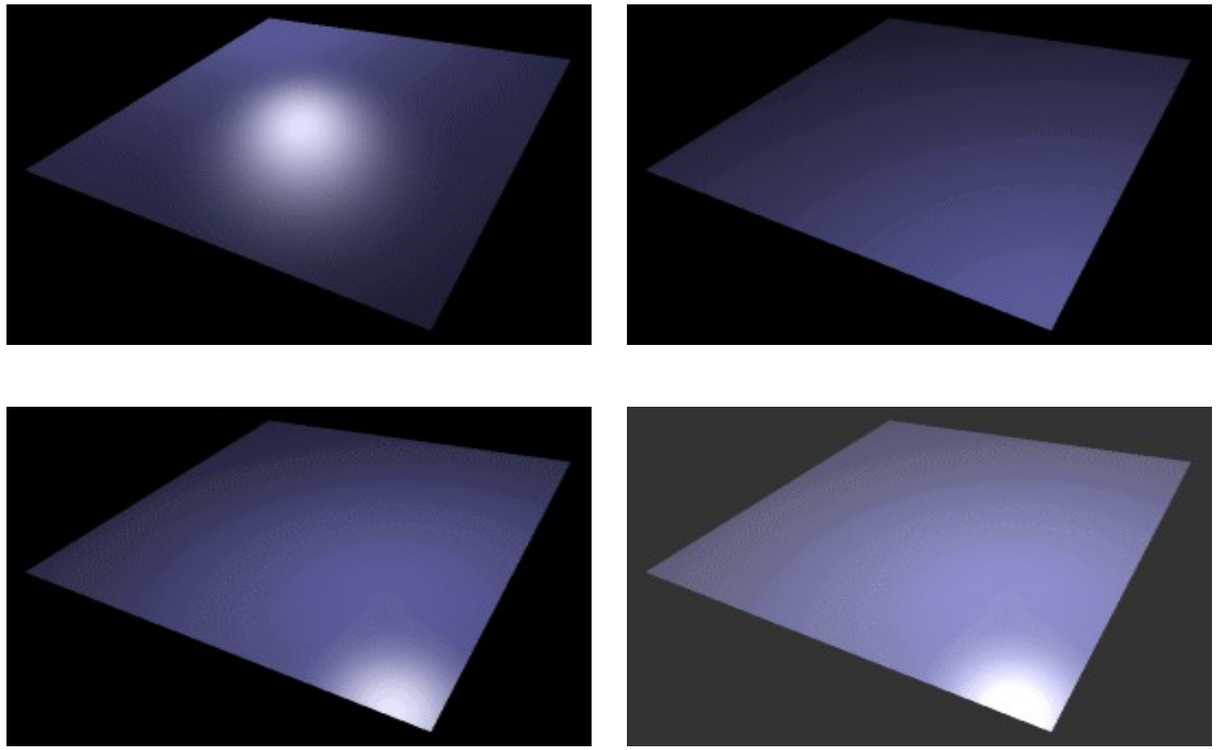
Phong Shading

- Even better result for curved surfaces
- No errors at high lights
- No Mach banding
- Phong shading is much slower than Gouraud shading as the lighting model is revaluated so many times
- There are **fast Phong** surface rendering approaches that can be implemented iteratively
- Typically, implemented as part of a visible surface detection technique
- **Not supported in OpenGL**

Phong vs Gouraud Shading

If a highlight does not fall on a vertex Gouraud shading may miss it completely, but Phong shading does not.

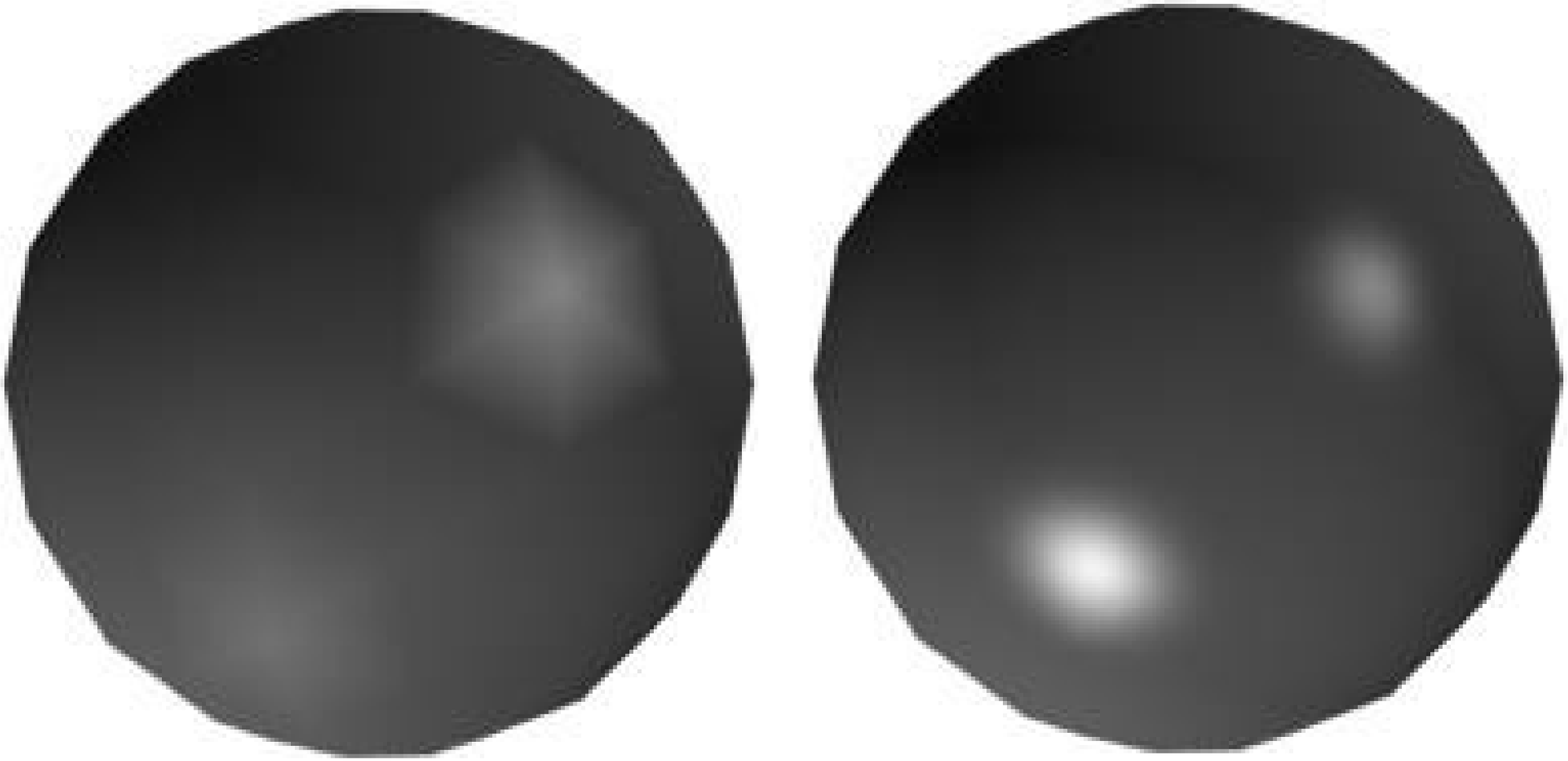
if highlight falls on vertex, Gourad shading will spread the highlight over the polygon



Gouraud Tea Pot Example

Phong Tea Pot Example

Phong vs Gouraud Shading



Gouraud

Phong

Phong vs Gouraud shading – zero marks

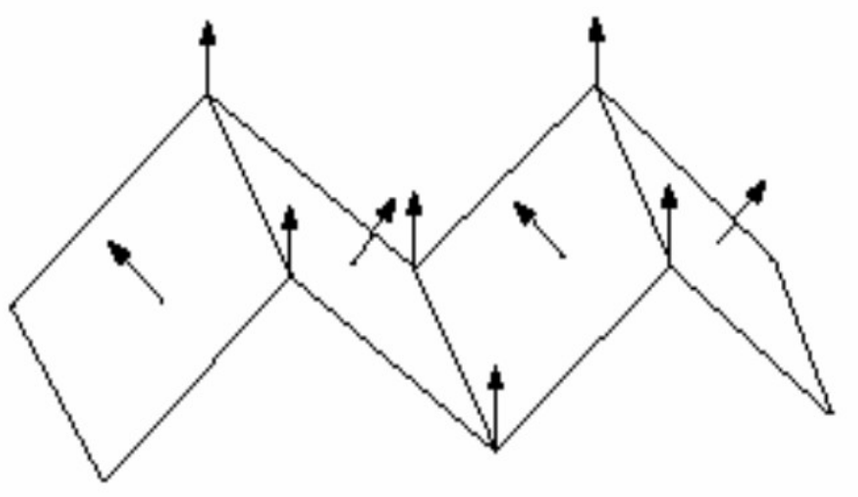
(a) "Phong shading is more realistic than Gouraud shading, but expensive"- Yes, This statement is right. In the Gouraud shading this can be overcome the problem of flat shading. we know that colour is important to understand realistic view and the 3D view. But shading is also important. In the object surface is completely plain, and there are no shading here, we can't understand the 3D object. We can understand when it will be 3D view. In the Gouraud shading, we can't understand the original view of a plane or object

In gouraud shading, it can't make the real view of an object. But in phong shadding, it overcome the problem of gouraud shading. It make the object in realistic to the viewer.

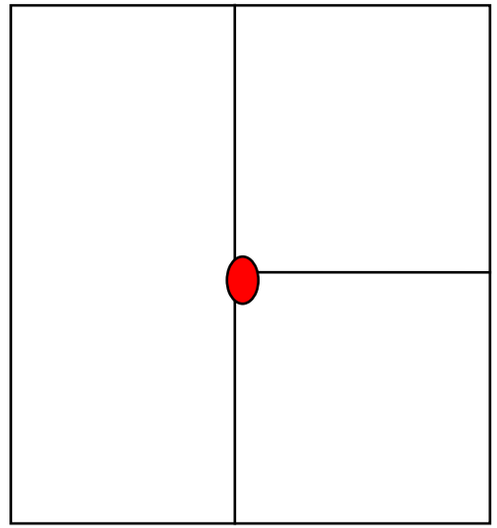
Although phong shading is more realistic, but the cost of the phong shading is much more than gouraud shading. In the gouraud shading, it can't be more realistic but respectly cheaper than phong shading.

Interpolative Shading artifacts

- Vertex normal does not always reflect the curvature of the surface adequately
- Incorrect Vertex normals – no variation in shade
  - Appear less flat than actual



- The shading at the T-junction are different when calculated from different triangles
  - shared by right polygons and not by one on left
  - Shading discontinuity

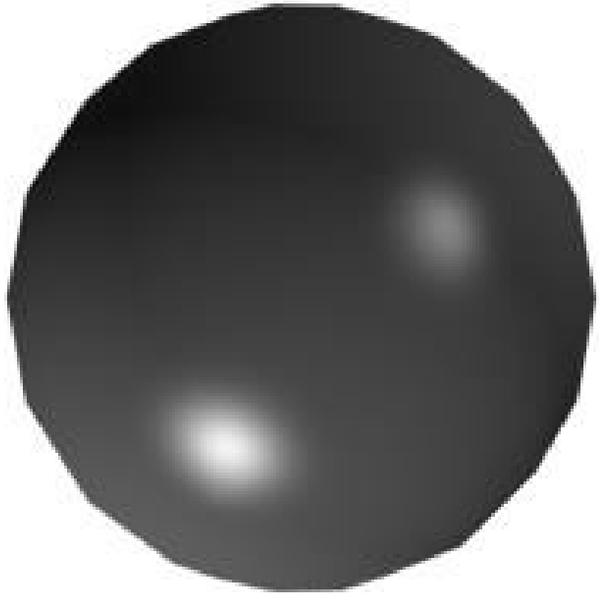


Interpolative shading artifacts – Mach Bands

- Common in flat shading since shading is discontinuous at edges
- Also present in Gouraud shading
  - Gradient of the shading may change suddenly
- Phong shading reduces it significantly
  - But cannot be eliminated
  - At sharp changes in surface gradient

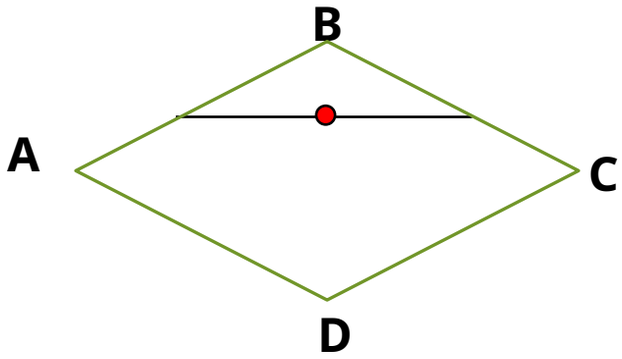
Interpolative Shading artifacts

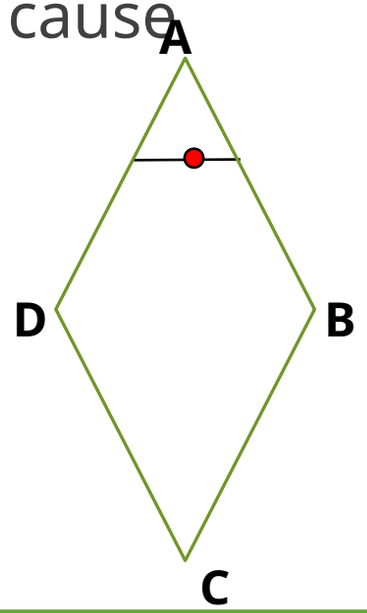
- Polygonal silhouette – edge is always polygonal
- Perspective distortion – interpolation is in screen space and hence foreshortening takes place
- In both cases finer polygons can help !



Phong

- Orientation dependence - small rotations cause problems





Fast Phong Surface Rendering

It significantly reduces the amount of computations at each point by approximating some of the illumination components (G. Bishop and D.M. Weimer, 1986).

Recall that rendering is done for projected polygons.

Consider an object described by triangles.

We interpolate the normal at (x, y) as

$\mathbf{N}(x, y) = \mathbf{A}x + \mathbf{B}y + \mathbf{C}$, where $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{C}$ are determined from the equalities

$\mathbf{N}(x_k, y_k) = \mathbf{A}x_k + \mathbf{B}y_k + \mathbf{C}$, $k = 1, 2, 3.$

Fast Phong Surface Rendering

Considering the first five terms in taylor expension of $I_{\text{diffuse}}(x, y)$ yields

$$I_{\text{diffuse}}(x, y) = T_5 x^2 + T_4 xy + T_3 y^2 + T_2 x + T_1 y + T_0.$$

The various T_i, $1 \leq i \leq 5$, are functions of $a, b, \ldots, i$ calculated before, they are fixed for the entire triangle and can be stored in registers.

Fast Phong Surface Rendering

Since the progression of scan line is $x \rightarrow x+1$ and $y \rightarrow y+1$, $I_{\text{diffuse}}(x, y)$ is fully calculated for the first point and then only two additions per pixel suffice.

The method can be extended to account sepecular reflections terms $(\mathbf{N} \cdot \mathbf{H})^{n_s}$.

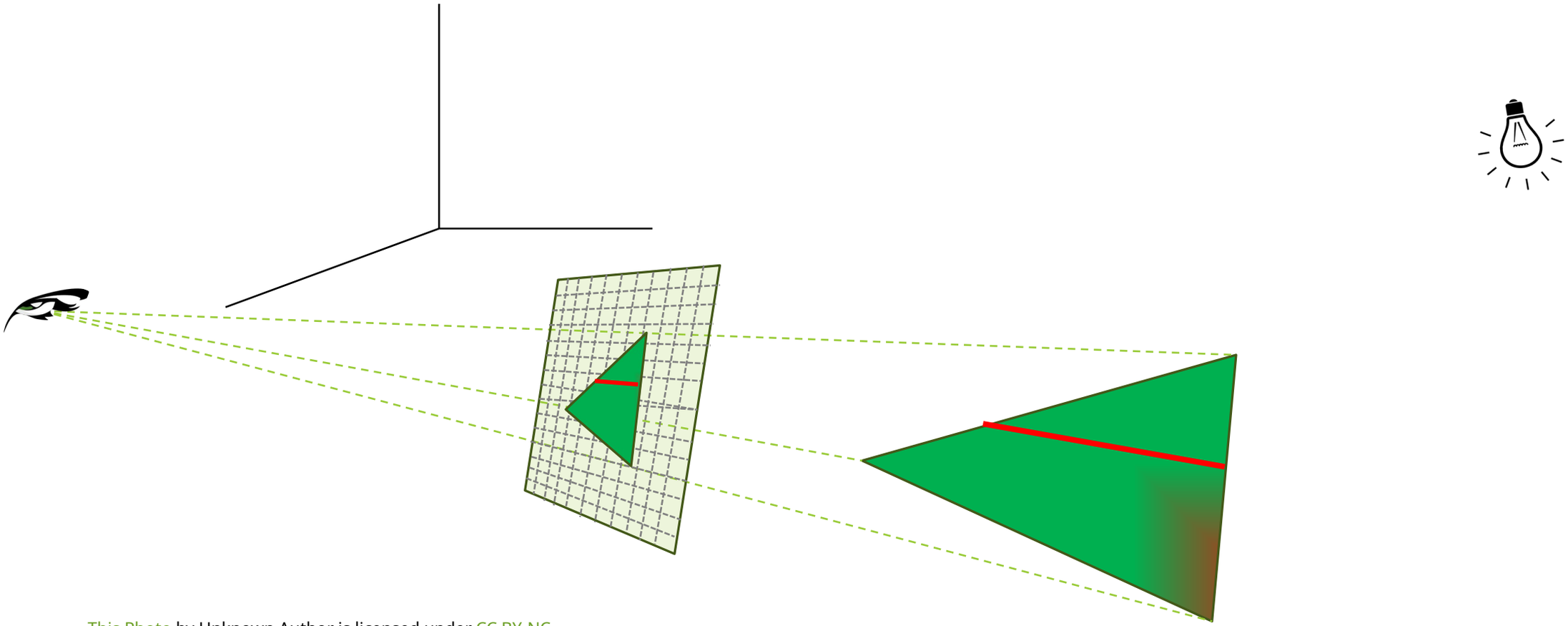
Other Types of Per-pixel Shading

- Ray tracing.
  - Doesn’t use Gouraud or Phong shading.
  - Each pixel uses own ray to determine color.
    - Can apply arbitrary lighting model.
    - Classical (Whitted) ray tracing uses Phong model.
  - Since ray tracing determines colors based on intersections, don’t have to use polygonal geometry.
    - Thus, can potentially use exact normals, rather than interpolation.

Summary

- For realistic rendering of polygons we need interpolation methods to determine lighting positions
- Flat shading is fast, but unrealistic
- Gouraud shading is better, but does not handle specular reflections very well
- Phong shading is even better, but can be slow

Problems - DIY



This Photo by Unknown Author is licensed under CC BY-NC

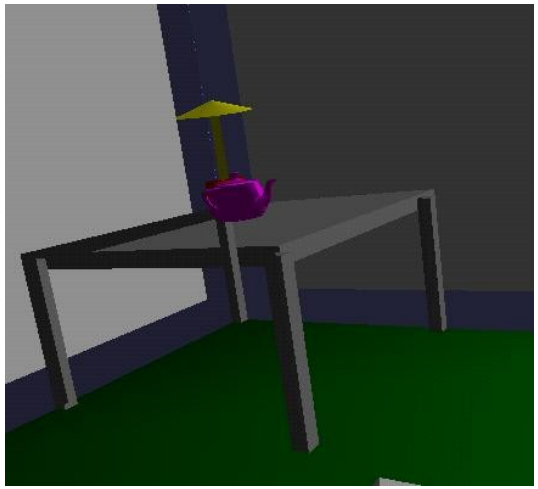
Hidden Surface Removal

Hidden Surface Removal

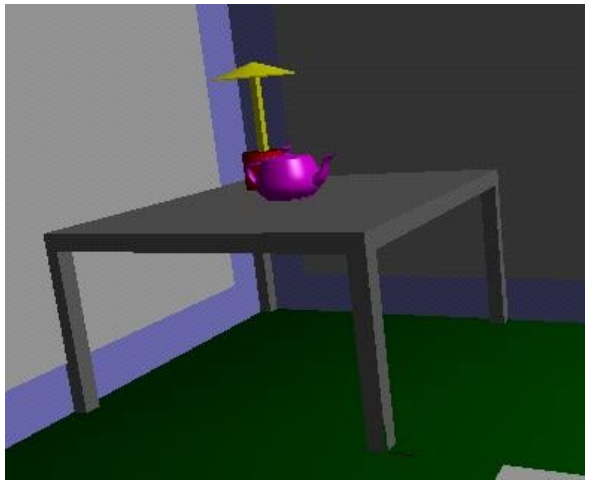
- Drawing polygon faces on screen consumes CPU cycles
- We cannot see every surface in scene
- To save time, draw only surfaces we see
- Surfaces we cannot see and their elimination methods:
  - Occluded surfaces: hidden surface removal (visibility)
  - Back faces: back face culling
  - Faces outside view volume: viewing frustrum culling/clipping
- Definitions:
  - Object space: before vertices are mapped to pixels
  - Image space: after vertices have been rasterized

Visibility (hidden surface removal)

- A correct rendering requires correct visibility calculations
- Correct visibility – when multiple opaque polygons cover the same screen space, only the front most one is visible (remove the hidden surfaces)



wrong visibility



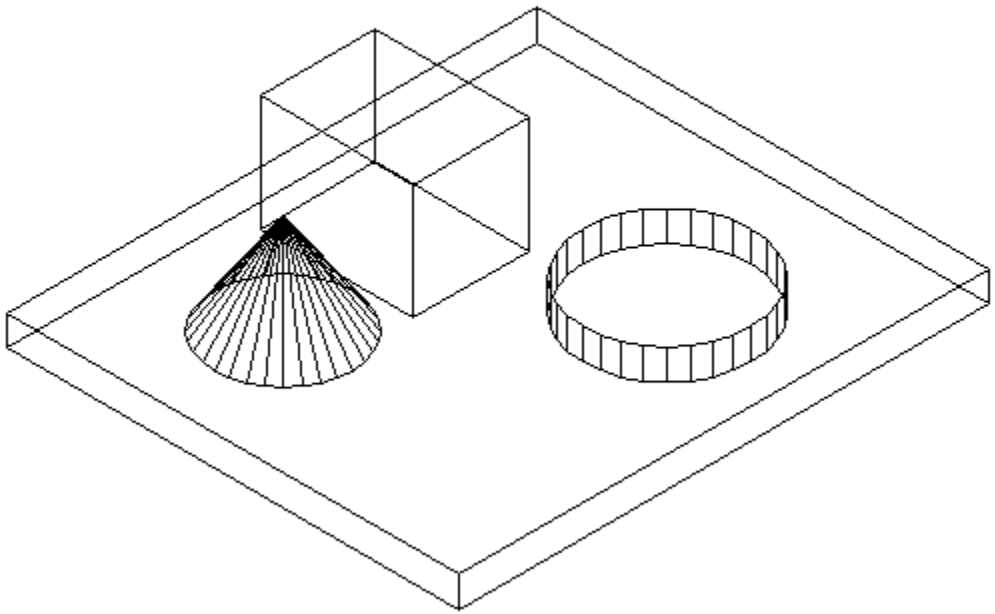
Correct visibility

Visibility (hidden surface removal)

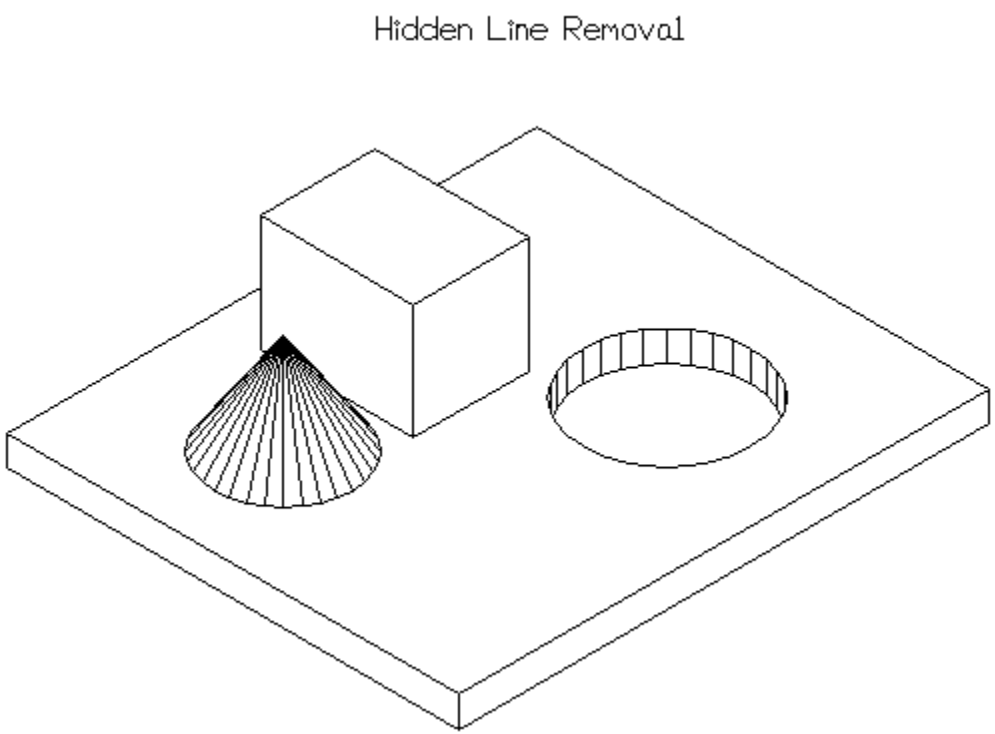
- Goal: determine which objects are visible to the eye
  - Determine what colors to use to paint the pixels
- Active research subject - lots of algorithms have been proposed in the past (and is still a hot topic)

No Lines Removed

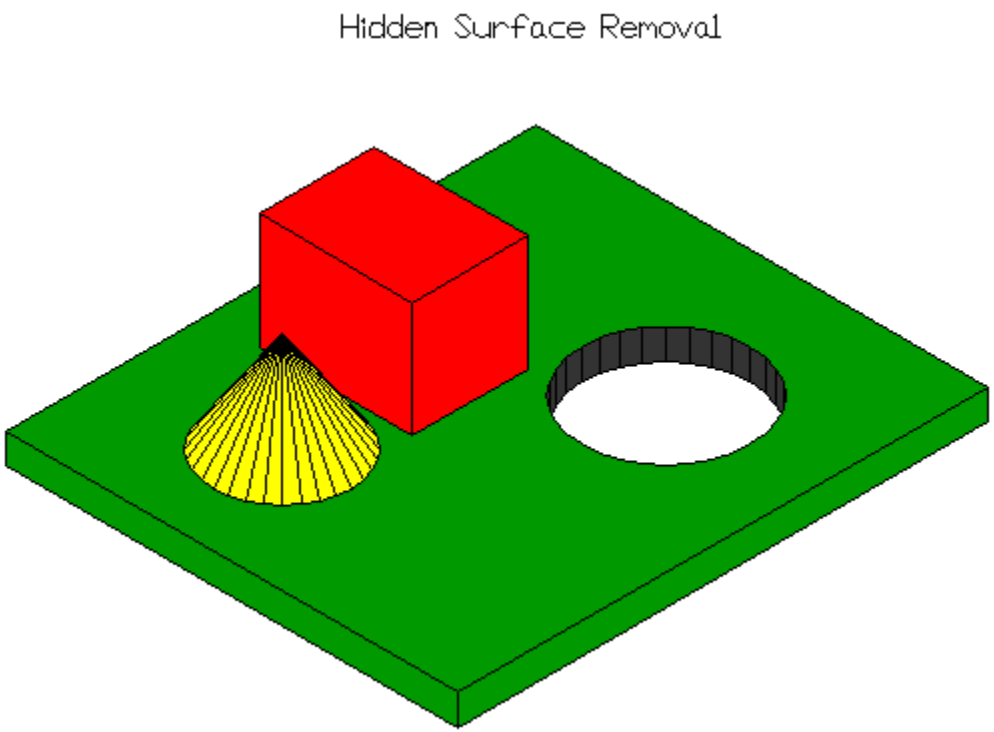
Wireframe



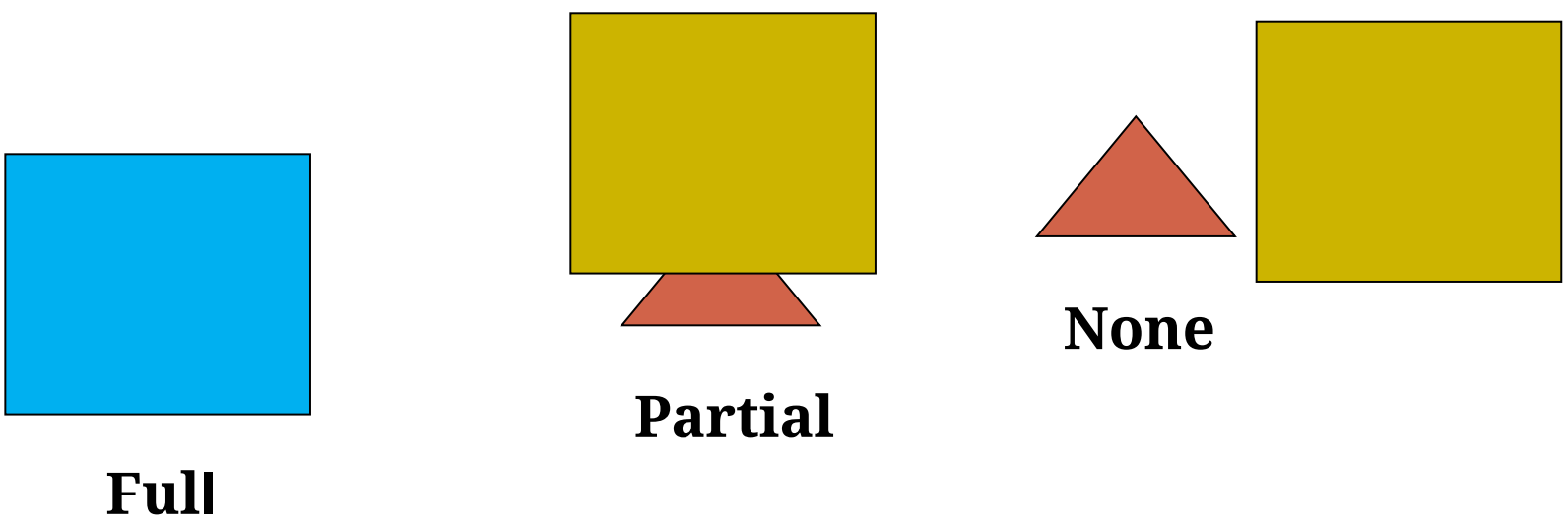
Hidden Lines Removed



Hidden Surfaces Removed



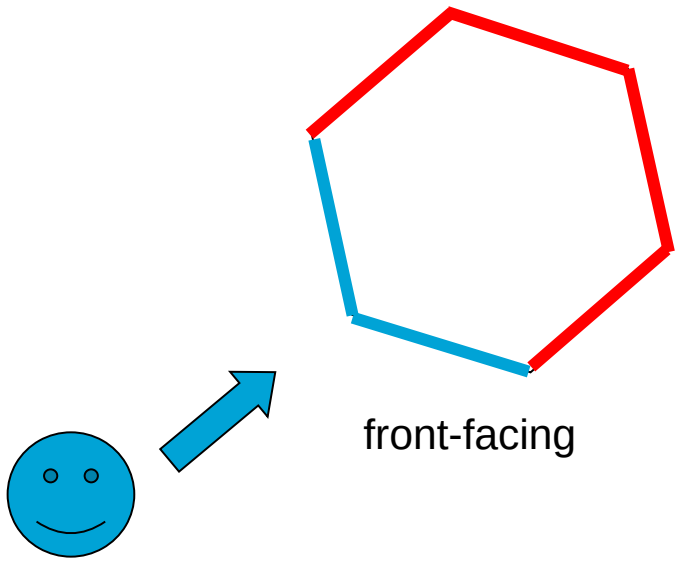
Occlusion: Full, Partial, None



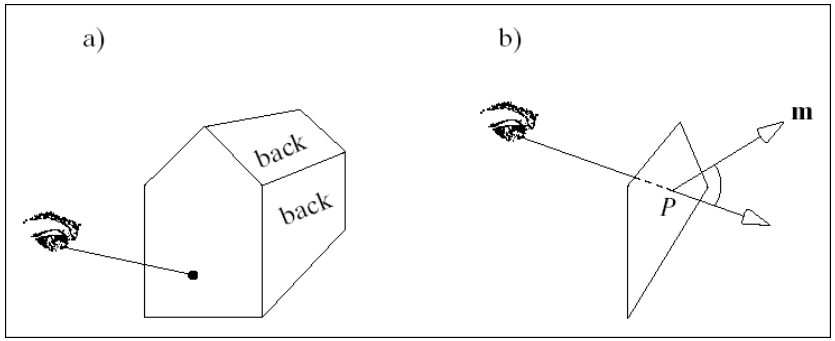
- The rectangle is closer than the triangle
- Should appear in front of the triangle

Backface Culling

- Avoid drawing polygons facing away from the viewer
  - Front-facing polygons occlude these polygons in a closed polyhedron
- Test if a polygon is front- or back-facing?

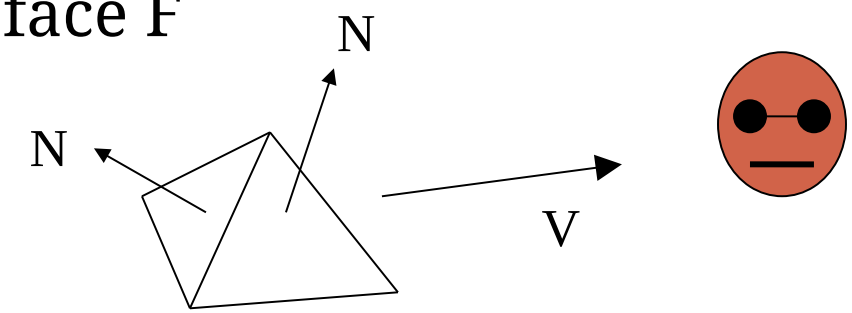


Ideas?



Backface Culling

- If we find backface, do not draw, save rendering resources
- There must be other forward face(s) closer to eye
- F is face of object we want to test if backface
- P is a point on F
- Form view vector, V as (eye – P)
- N is normal to face F

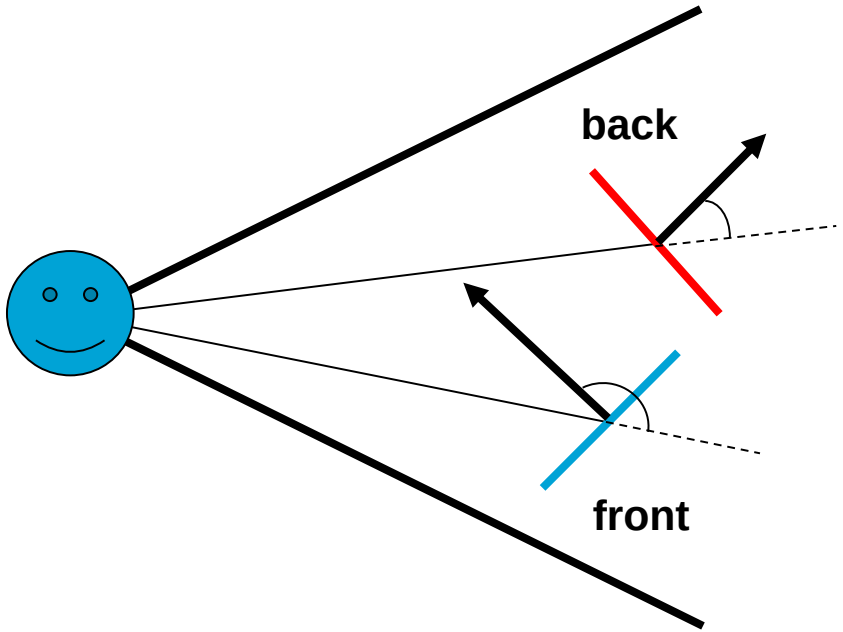


Detecting Back-face Polygons

- The polygon normal of a …
  - front-facing polygon points **towards** the viewer
  - back-facing polygon points **away** from the viewer

If $(\mathbf{n} \cdot v) > 0 \Rightarrow$ "back-face"
If $(\mathbf{n} \cdot v) \leq 0 \Rightarrow$ "front-face"
v = view vector

- Eye-space test … EASY!
  - "back-face" if $\mathbf{n}_z < 0$

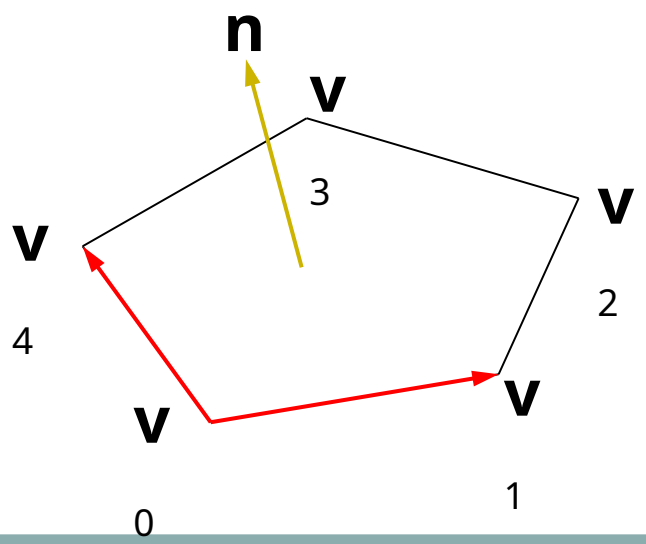


Polygon Normals

- Let polygon vertices $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_{n-1}$ be in counterclockwise order and co-planar
- Calculate normal with cross product:

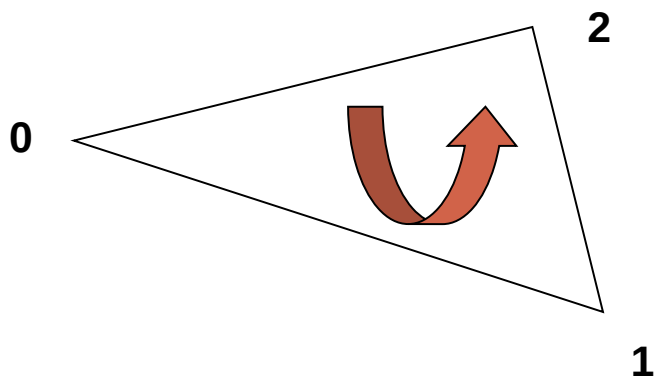
$$\mathbf{n} = (\mathbf{v}_1 - \mathbf{v}_0) \times (\mathbf{v}_{n-1} - \mathbf{v}_0)$$

- Normalize to unit vector with $\mathbf{n} / \| \mathbf{n} \|$

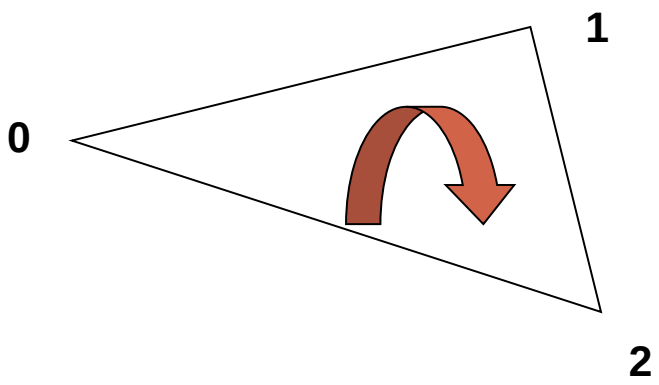


Normal Direction

- Vertices counterclockwise ⇒ Front-facing
- Vertices clockwise ⇒ Back-facing



Front facing



Back facing

Visibility

- How do we ensure that closer polygons overwrite further ones in general?

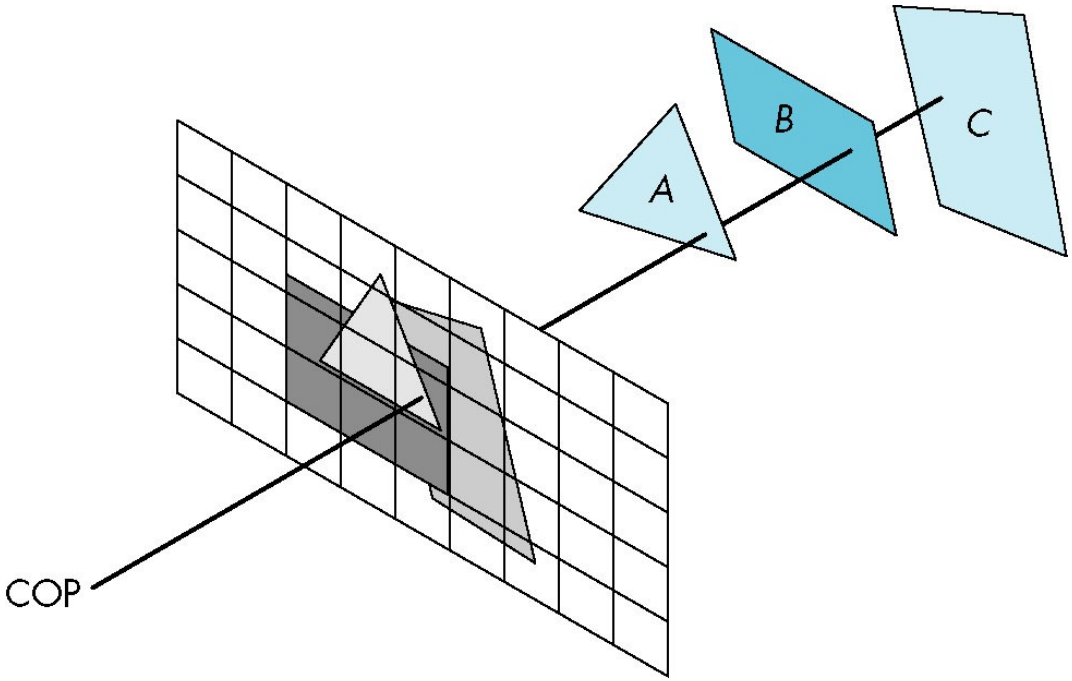


Image Space Approach – Z-buffer

- Method used in most of graphics hardware (and thus OpenGL): Z-buffer algorithm
- Requires lots of memory
- Basic idea:
  - rasterize every input polygon
  - Recall that we have z at polygon vertices
  - Maintains 2 buffers
  - For every pixel in the polygon interior, calculate its corresponding z value (by interpolation) (Z buffer)
  - Choose the color of the polygon whose z value is the closest to the eye to paint the pixel. (Refresh Buffer)

Image Space Approach – Z-buffer

- Recall: after projection transformation
- In viewport transformation
  - x,y used to draw screen image
  - z component is mapped to pseudo-depth with range [0,1]
- However, objects/polygons are made up of vertices
- Hence z is known at vertices
- Point in object seen through pixel may be between vertices
- Need to interpolate to find z

Z (depth) buffer algorithm

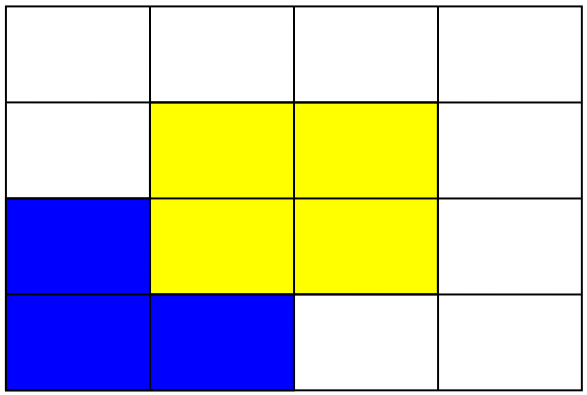
- How to choose the polygon that has the closet Z for a given pixel?
- Assumption for example: eye at z = 0, farther objects have increasingly negative values
  - Initialize (clear) every pixel in the z buffer to a very large negative value
  - Track polygon z's.
  - As we rasterize polygons, check to see if polygon's z through this pixel is less than current minimum z through this pixel
  - Run the following loop:

Z (depth) Buffer Algorithm

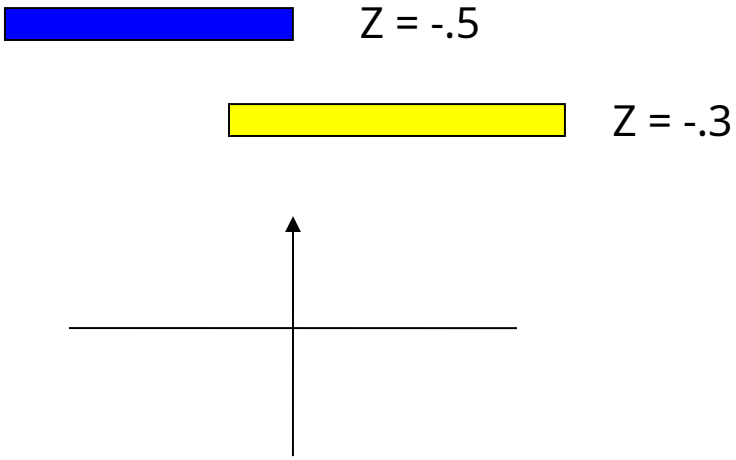

```
For  each polygon  {

  for each pixel (x,y) inside the polygon projection area
{

        if  (z_polygon_pixel(x,y) > depth_buffer(x,y) ) {

            depth_buffer(x,y) = z_polygon_pixel(x,y);

            color_buffer(x,y) = polygon color at (x,y)
        }
    }
}
```

Note: we have depths at vertices. Interpolate for interior depths

Z buffer example



Final image



Top View

Z buffer example

Step 1: Initialize the depth buffer

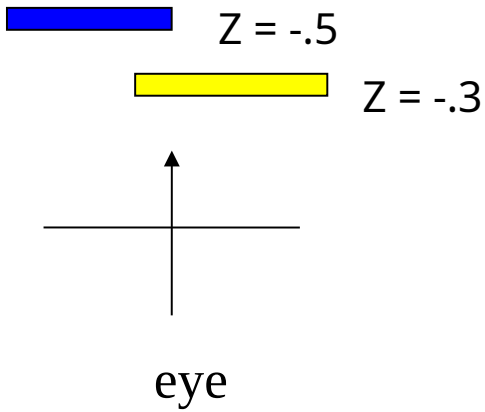
-999	-999	-999	
-999 -999	-999	-999	
-999 -999	-999	-999	
-999 -999	-999	-999	
-999			

Z buffer example

Step 2: Draw the blue polygon (assuming the OpenGL program draws blue polyon first – the order does not affect the final result any way).

-999	-999	-999	
-999 -999	-999	-999	
-999 -.5	-.5	-999	
-999 -.5	-.5	-999	

-999

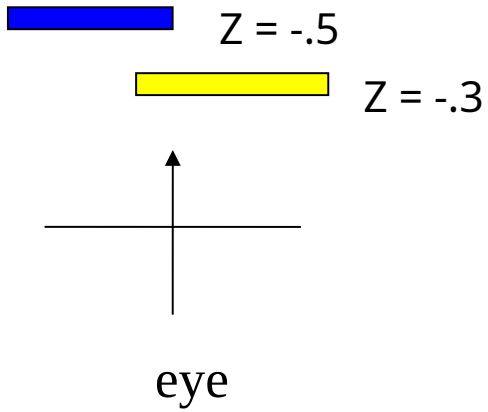


Z buffer example

Step 3: Draw the yellow polygon

-999	-999	-999	
-999	-.3	-.3	
-999	-.3	-.3	
-999	-.5	-999	

-999



Calculating depth values efficiently

- We know the depth values at the vertices. How can we calculate the depth at any other point on the surface of the polygon.
- Using the polygon surface equation:

$$z = \frac{-Ax - By - D}{C}$$

Calculating depth values efficiently

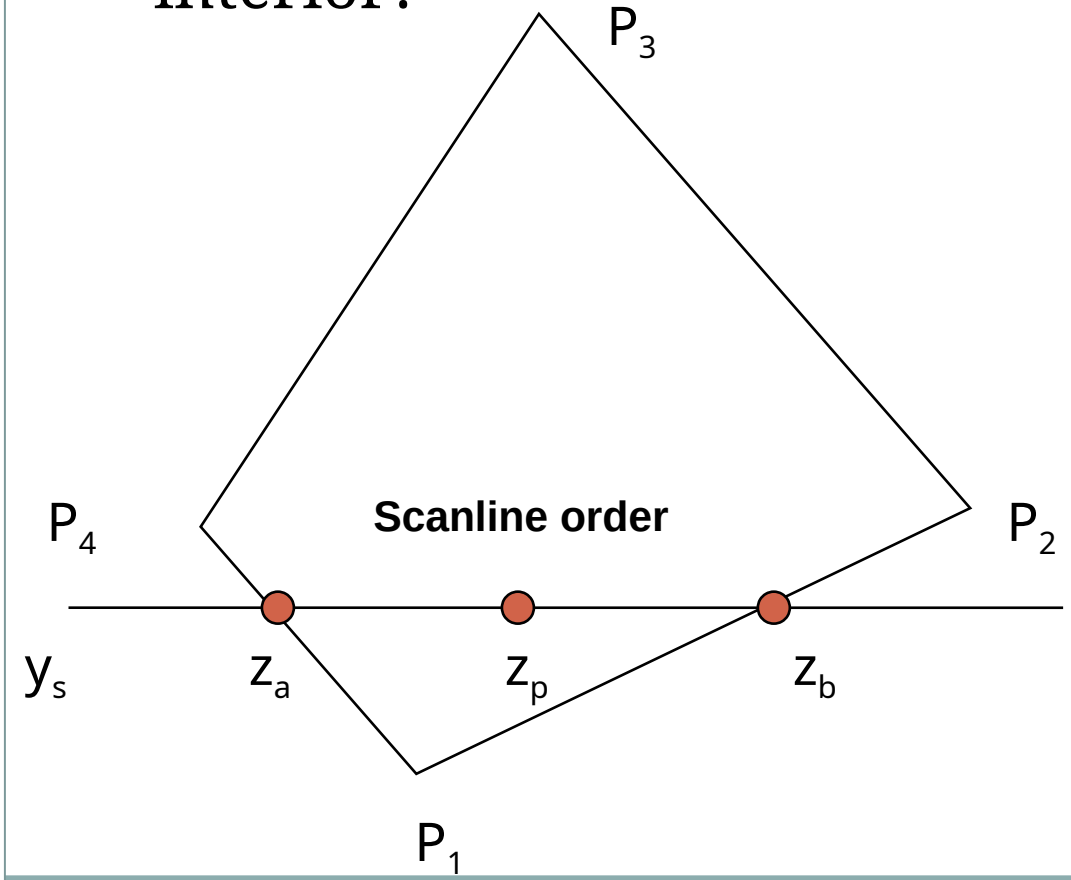
- For any scan line adjacent horizontal x positions or vertical y positions differ by 1 unit.
- The depth value of the next position (x+1,y) on the scan line can be obtained using.

$$z' = \frac{-A(x+1) - By - D}{C}$$

$$= z - \frac{A}{C}$$

Z-Buffer

- How do we calculate the depth values on the polygon interior?



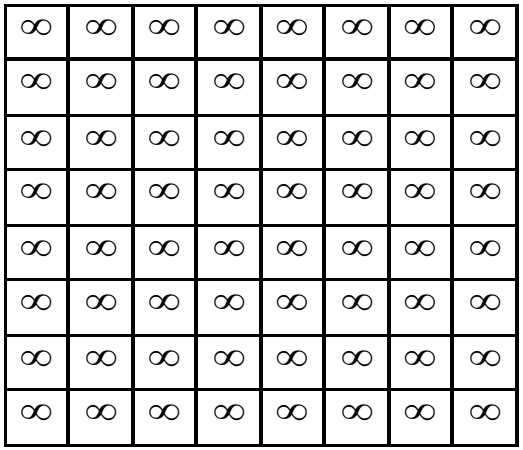
$$z_a = z_1 + (z_4 - z_1)\frac{(y_1 - y_s)}{(y_1 - y_4)}$$

$$z_b = z_1 + (z_2 - z_1)\frac{(y_1 - y_s)}{(y_1 - y_2)}$$

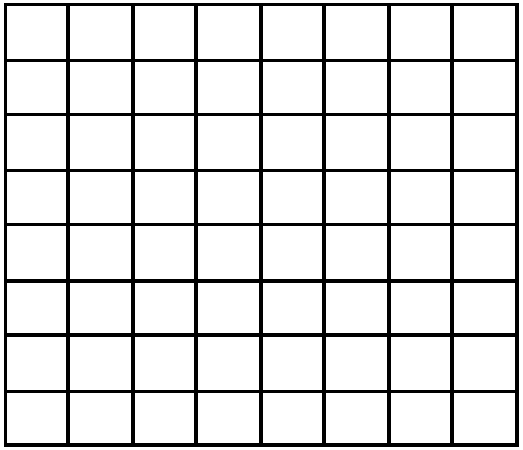
$$z_p = z_a + (z_b - z_a)\frac{(x_a - x_p)}{(x_a - x_b)}$$

Bilinear Interpolation

Z-Buffer Example



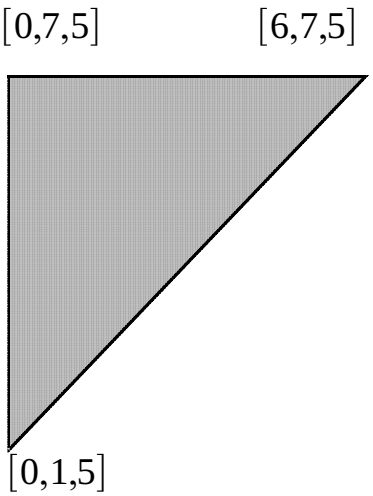
Z-buffer



Screen

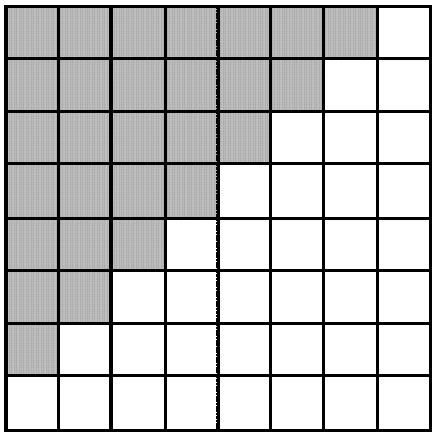
Z-Buffer Example

Parallel with the image plane



5	5	5	5	5	5	5
5	5	5	5	5	5	
5	5	5	5	5		
5	5	5	5			
5	5	5				
5	5					
5						

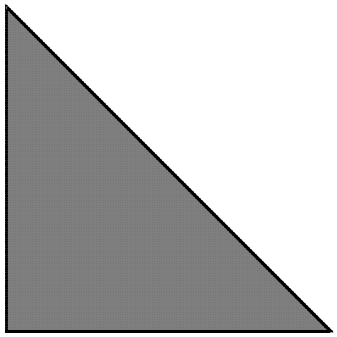
5	5	5	5	5	5	5	∞
5	5	5	5	5	5	∞	∞
5	5	5	5	5	∞	∞	∞
5	5	5	5	∞	∞	∞	∞
5	5	5	∞	∞	∞	∞	∞
5	5	∞	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞



Z-Buffer Example

Not Parallel

[0,6,7]

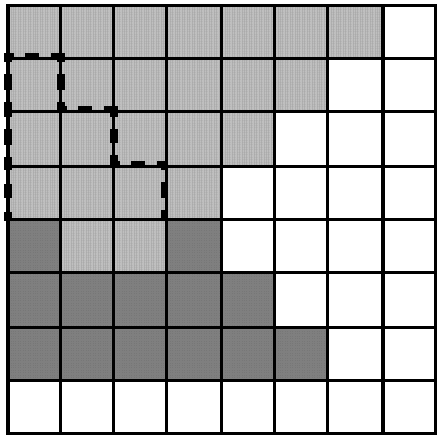


7					
6	7				
5	6	7			
4	5	6	7		
3	4	5	6	7	
2	3	4	5	6	7

[0,1,2]

[5,1,7]

5	5	5	5	5	5	5	∞
5	5	5	5	5	5	∞	∞
5	5	5	5	5	∞	∞	∞
5	5	5	5	∞	∞	∞	∞
4	5	5	7	∞	∞	∞	∞
3	4	5	6	7	∞	∞	∞
2	3	4	5	6	7	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞



Limitation of Z-buffer Algorithm

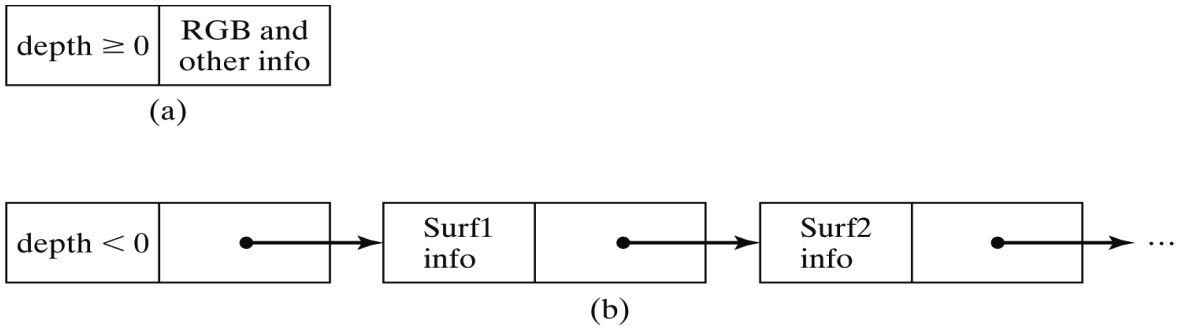
- This algorithm do not work on **transparent surface**, this is only for opaque surface.
- Hence here no method is available to compare the Z values of the surfaces which are transparent.
- For working on transparent surface another method is available which is called A-buffer Algorithm.
- Extra memory is required to store depth value.

A-Buffer Method

- Extends the depth-buffer algorithm so that each position in the buffer can reference a linked list of surfaces.
- More memory is required.
- However, we can correctly compose different surface colors and handle transparent surfaces.

A-Buffer Method

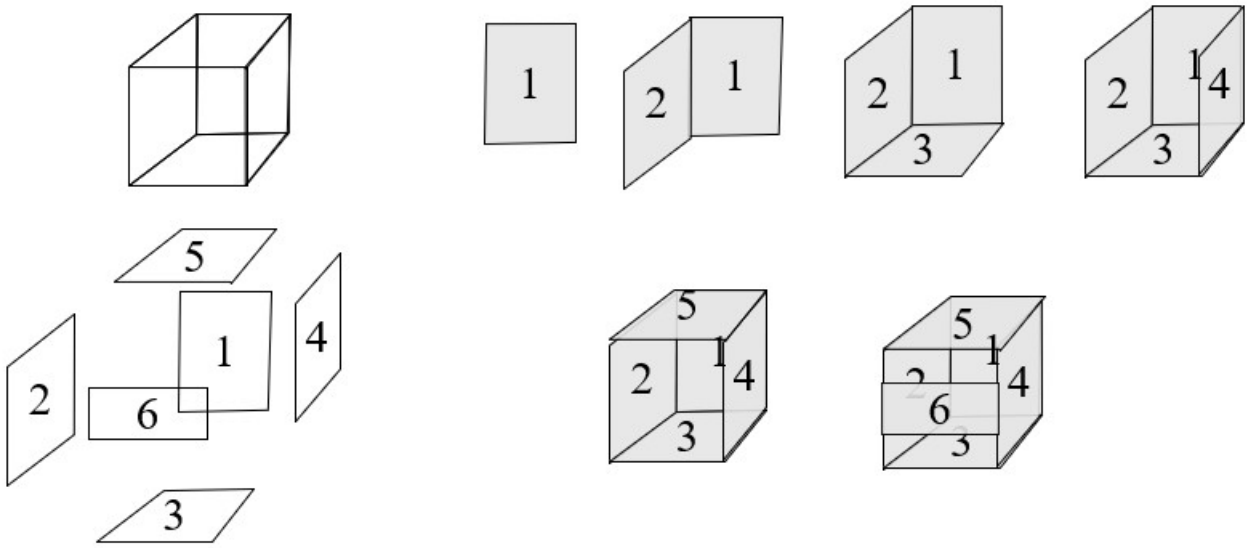
- Each position in the A-buffer has two fields:
  - a depth field
  - surface data field which can be either surface data or a pointer to a linked list of surfaces that contribute to that pixel position
  - Many other information can be stored in the link list such as : % of transparency, % of area coverage, surface rendering factor.



Object Space Methods

Depth Sorting

- Also known as **painters algorithm**. First draw the distant objects than the closer objects. Pixels of each object overwrites the previous objects.



Depth-sort algorithm

- The idea here is to go back to front drawing all the objects into

the frame buffer with nearer objects being drawn over top of objects that are further away.

- Simple algorithm:

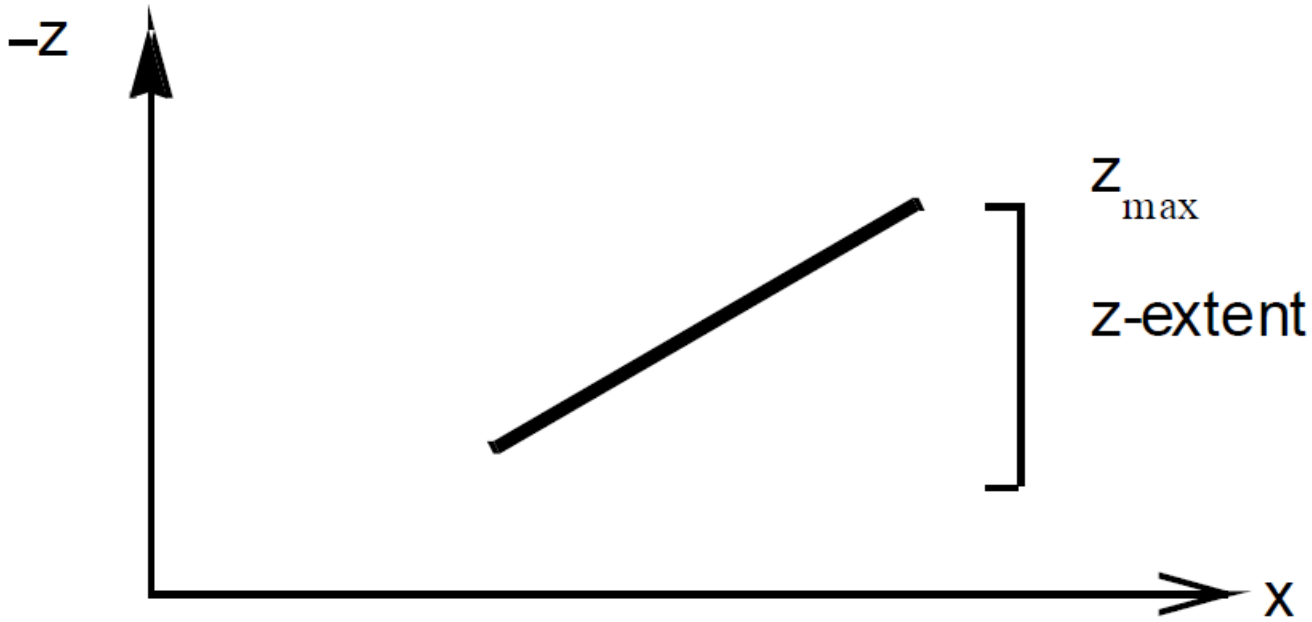
– Sort all polygons based on their farthest z coordinate
– Resolve ambiguities
– Draw the polygons in order from back to front

- This algorithm would be very simple if the z coordinates of the

polygons were guaranteed never to overlap. Unfortunately that
is usually not the case, which means that step 2 can be somewhat complex.

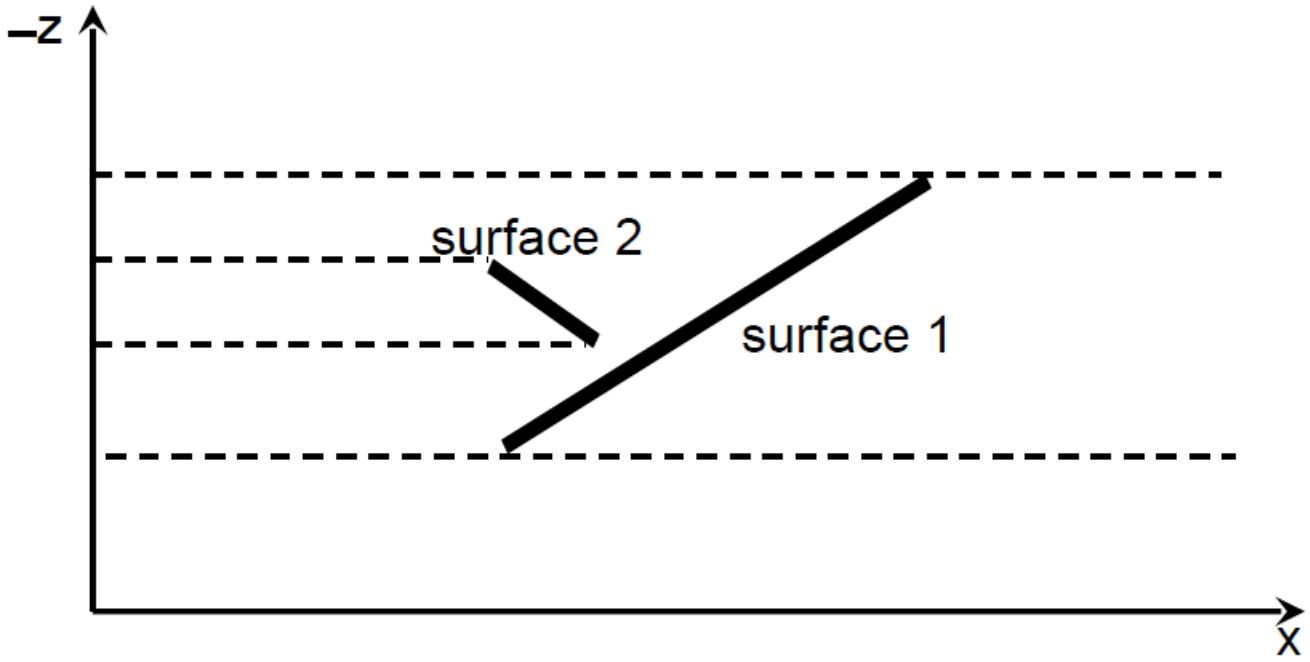
Depth-sort algorithm

- First must determine z-extent for each polygon

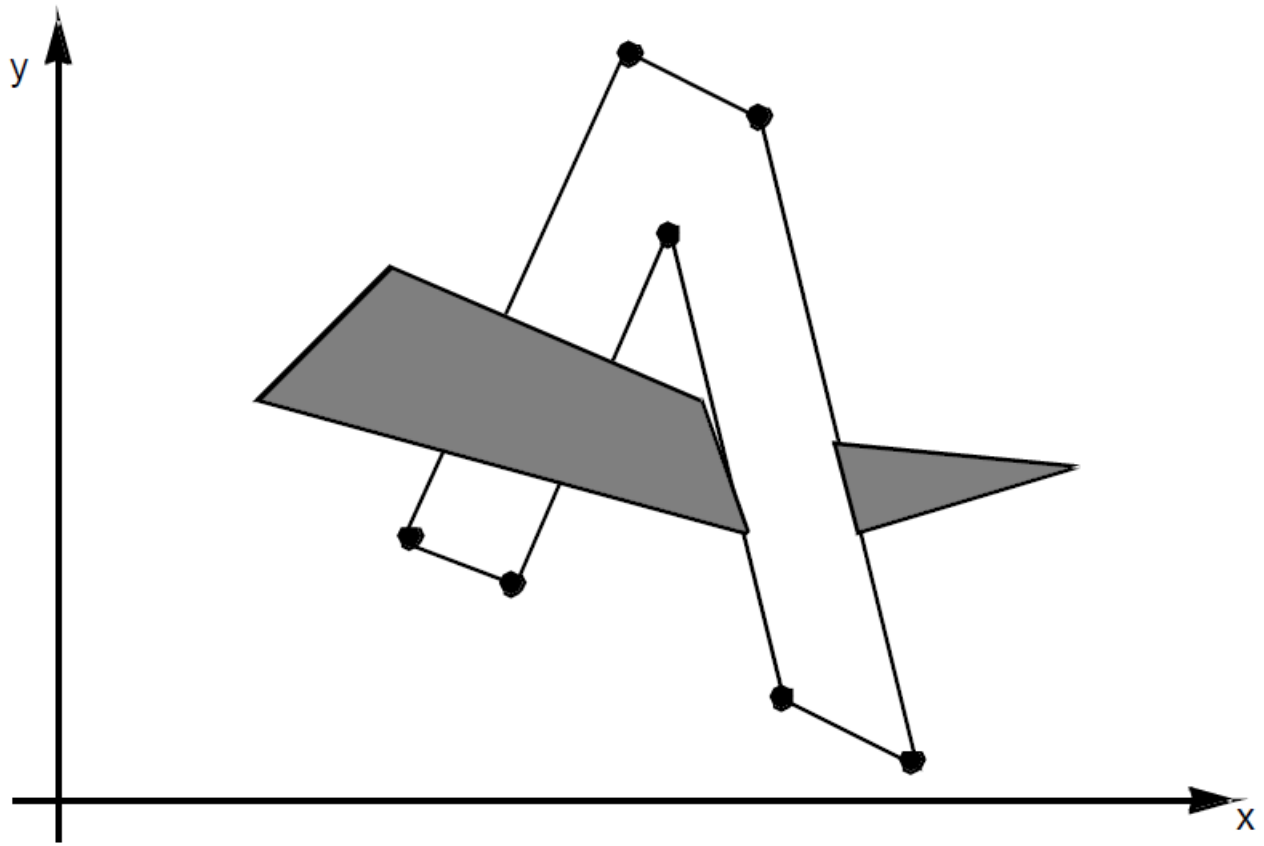


Depth-sort algorithm

- Ambiguities arise when the z-extents of two surfaces overlap.



Depth-sort algorithm



Depth-sort algorithm

- All polygons whose z extents overlap must be tested against each other.

- We start with the furthest polygon and call it P. Polygon P must be compared with every polygon Q whose z extent overlaps P's z extent. 5 comparisons are made. If any comparison is true then P can be written before Q. If at least one comparison is true for each of the Qs then P is drawn and the next polygon from the back is chosen as the new P.

Depth-sort algorithm

1. Do P and Q's x-extents not overlap?
2. Do P and Q's y-extents not overlap?
3. Is P entirely on the opposite side of Q's plane from the viewport?
4. Is Q entirely on the same side of P's plane as the viewport?
5. Do the projections of P and Q onto the (x,y) plane not overlap?

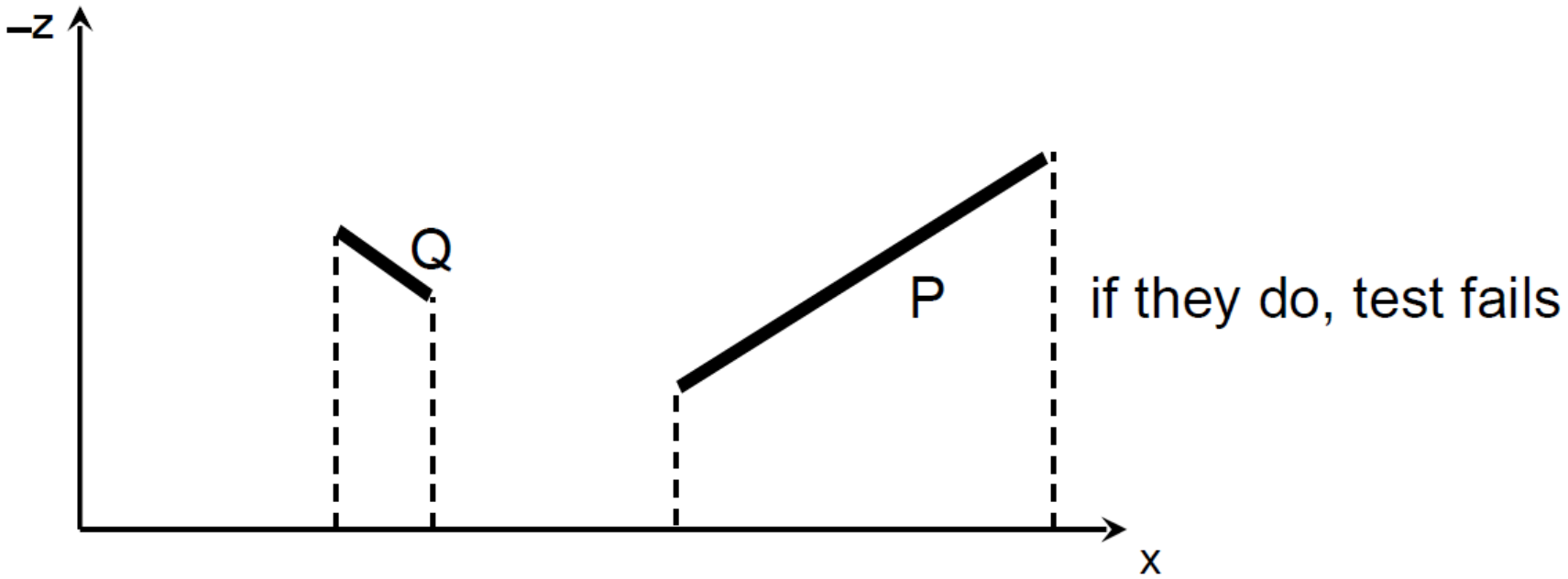
- If all 5 tests fail we quickly check to see if switching P and Q will work. Tests 1, 2, and 5 do not differentiate between P and Q but 3 and 4 do. So we rewrite 3 and 4 as:

3'. Is Q entirely on the opposite side of P's plane from the viewport?

4'. Is P entirely on the same side of Q's plane as the viewport?

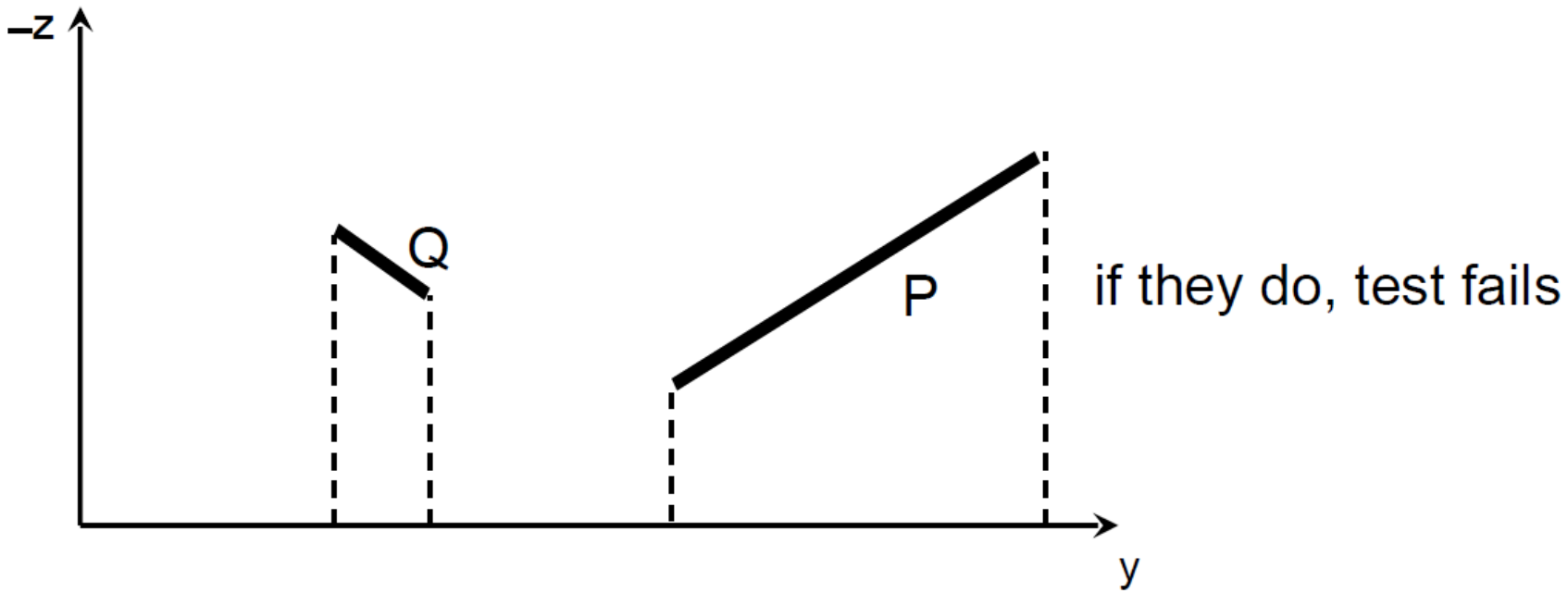
Depth-sort algorithm

- x - extents not overlap?



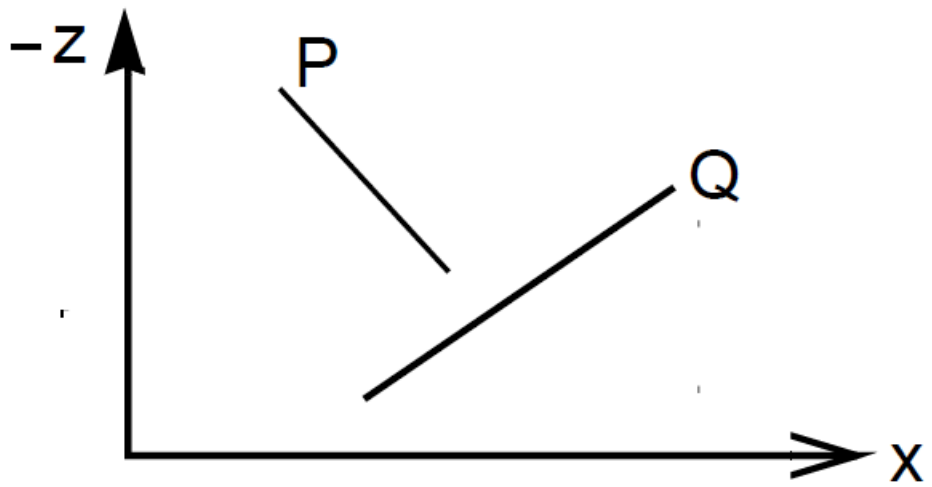
Depth-sort algorithm

- y - extents not overlap?



Depth-sort algorithm

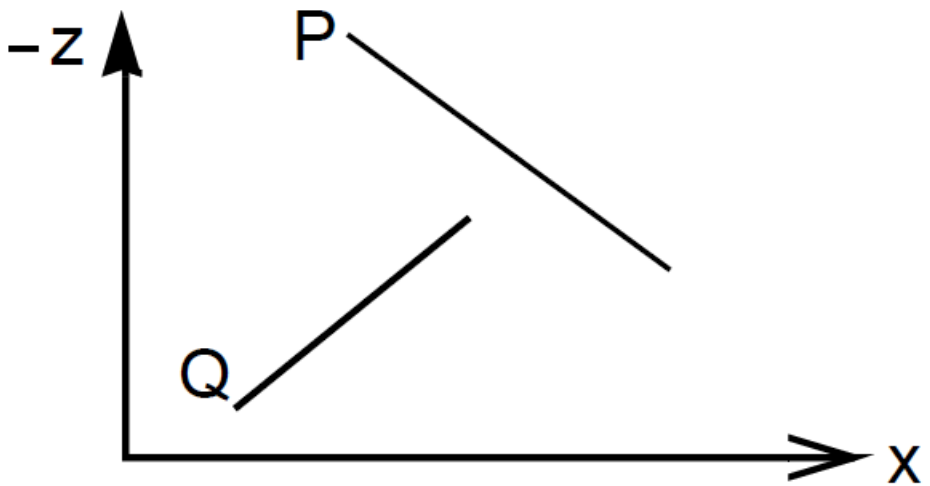
- Is P entirely behind the surface Q relative to the viewing position (i.e., behind Q's plane with respect to the viewport)?



Test is true…

Depth-sort algorithm

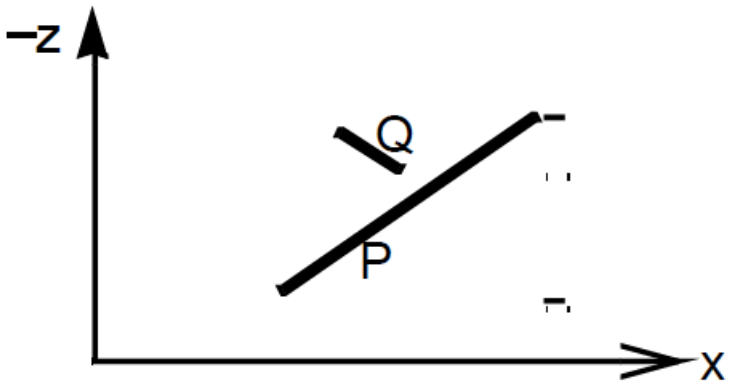
- Is Q entirely in front of P's plane relative to the viewing position (i.e., the viewport)?

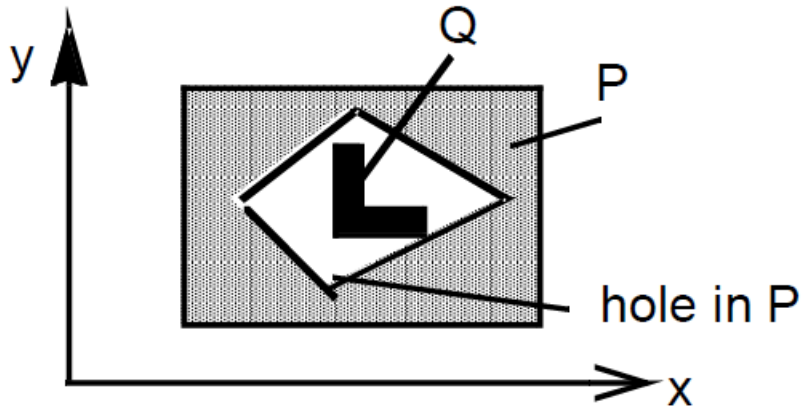


Test is true…

Depth-sort algorithm

- Do the projections of P and Q onto the (x,y) plane not overlap?





Test is true…

Depth-sort algorithm

- If all tests fail…

– … then reverse P and Q in the list of surfaces sorted by maximum depth

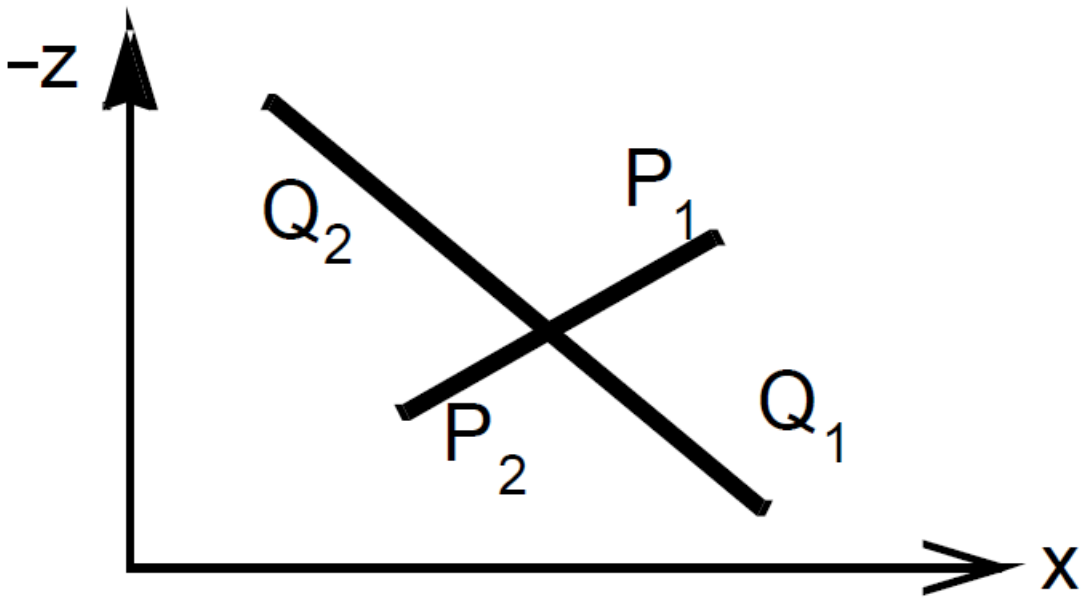
– set a flag to say that the test has been performed once.

– If the tests fail a second time, then it is necessary to split the surfaces and repeat the algorithm on the 4 new split surfaces

Depth-sort algorithm

- Example:

– We end up processing with order Q2,P1,P2,Q1



Reference

- **Computer Graphics**
  - Donald Hearn, M Pauline Baker
  - Pearson Education